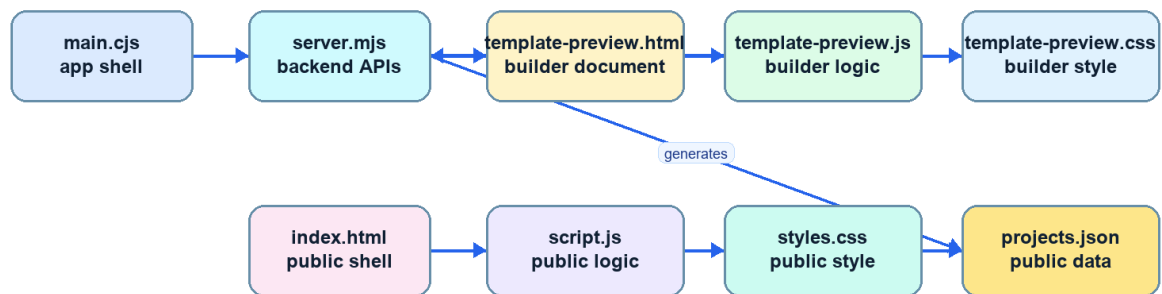


OMB Portfolio Builder Important Code Reference

A generated PDF reference for the code and control files that drive the builder, website, installer, Cloudflare AI worker, workflows, parser, compile workspace, and generated documentation.

How Key Files Communicate



Coverage summary

Item	Explanation
Important files documented	19
Selection rule	Only behavior-driving code/control files are included. Favicons, binary assets, and passive marker files are intentionally skipped here.
Categories	Configuration or structured data: 4, GitHub workflow: 2, JavaScript source: 7, NSIS installer script: 1, Python tooling: 1, Website/builder UI source: 4

File index

Item	Explanation
main.cjs	JavaScript source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
server.mjs	JavaScript source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
template-preview.js	JavaScript source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
template-preview.html	Website/builder UI source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
template-preview.css	Website/builder UI source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
script.js	JavaScript source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
index.html	Website/builder UI source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
styles.css	Website/builder UI source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

Item	Explanation
cloudflare/portfolio-ai-worker.js	JavaScript source. Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment notes.
cloudflare/wrangler.toml	Configuration or structured data. Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment notes.
build/installer.nsh	NSIS installer script. Packaging and installer customization. Used while creating the Windows app installer.
docs/build_complete_guide.py	Python tooling. Documentation and generated reference area. Used to explain the app and source structure.
builder-rich-future-sections.js	JavaScript source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
electronics-search.js	JavaScript source. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
package.json	Configuration or structured data. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
project-templates.json	Configuration or structured data. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
projects.json	Configuration or structured data. Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
.github/workflows/build-windows-builder.yml	GitHub workflow. GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs.
.github/workflows/main-branch-gate.yml	GitHub workflow. GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs.

Important Code Reference: main.cjs

Item	Explanation
File	main.cjs
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	15,134 bytes
Extension	.cjs
MIME type	text/plain
Text lines	453

Why this file exists

- Electron main process: creates the desktop window, starts the local backend, handles menus, update handoff, and application lifecycle.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Imports, requires, and external dependencies

Item	Explanation
Line 3	const fs = require("node:fs");
Line 4	const fsp = require("node:fs/promises");
Line 5	const net = require("node:net");
Line 6	const path = require("node:path");

Important implementation signals

Item	Explanation
Process execution at line 2	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: const { execFile } = require("node:child_process");
Compiler or simulator at line 3	Part of the Compile Code workspace or language/tool detection. Evidence: const fs = require("node:fs");
Compiler or simulator at line 4	Part of the Compile Code workspace or language/tool detection. Evidence: const fsp = require("node:fs/promises");
Compiler or simulator at line 5	Part of the Compile Code workspace or language/tool detection. Evidence: const net = require("node:net");
Compiler or simulator at line 6	Part of the Compile Code workspace or language/tool detection. Evidence: const path = require("node:path");
Compiler or simulator at line 7	Part of the Compile Code workspace or language/tool detection. Evidence: const { pathToFileURL } = require("node:url");
Compiler or simulator at line 8	Part of the Compile Code workspace or language/tool detection. Evidence: const { promisify } = require("node:util");
Process execution at line 13	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: const execFileAsync = promisify(execFile);
Security or auth at line 27	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers"
Cloudflare or AI at line 38	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const directoryRefreshes = ["cloudflare"];
Security or auth at line 48	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers",

Item	Explanation
Git command at line 53	Repository state, publishing, branch checks, or release automation. Evidence: process.env.GIT_EXE,
Git command at line 54	Repository state, publishing, branch checks, or release automation. Evidence: "git",
Git command at line 55	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\cmd\\git.exe",
Git command at line 56	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\bin\\git.exe",
Git command at line 57	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
Git command at line 58	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\bin\\git.exe"
Installer at line 61	Windows setup, update, shortcut, or installation behavior. Evidence: function dispatchBuilderMenuAction(action) {
Installer at line 110	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "New Project", accelerator: "CmdOrCtrl+N", click: () => dispatchBuilderMenuAction({ type: "new-project" }) },
Installer at line 111	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Publishing Target", click: () => dispatchBuilderMenuAction({ type: "publishing-target" }) },
Installer at line 113	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Save Draft", accelerator: "CmdOrCtrl+S", click: () => dispatchBuilderMenuAction({ type: "save-draft" }) },
Installer at line 114	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Apply to Site", accelerator: "CmdOrCtrl+Shift+S", click: () => dispatchBuilderMenuAction({ type: "apply-site" }) },
Rich text at line 127	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: { role: "paste" },
Installer at line 141	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Portfolio Preview", click: () => dispatchBuilderMenuAction({ type: "portfolio-preview" }) },
Installer at line 142	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Check Updates", click: () => dispatchBuilderMenuAction({ type: "check-updates" }) },
Installer at line 154	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Open Preferences", accelerator: "CmdOrCtrl+," , click: () => dispatchBuilderMenuAction({ type: "preferences" }) },
Installer at line 156	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Light Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "light" }) },
Installer at line 157	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Dark Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "dark" }) }
Installer at line 163	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Builder Guide", accelerator: "F1", click: () => dispatchBuilderMenuAction({ type: "builder-guide" }) },
Network request at line 164	Frontend-backend communication or public web/API calls. Evidence: { label: "GitHub Releases", click: () => shell.openExternal("https://github.com/otienomaurice/omb-portfolio-builder/releases/latest") },
File write at line 190	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(path.dirname(target), { recursive: true });
File write at line 191	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.copyFile(source, target);
File write at line 196	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(target, { recursive: true });
File write at line 212	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(path.dirname(target), { recursive: true });
Git command at line 241	Repository state, publishing, branch checks, or release automation. Evidence: throw lastError new Error("Git executable was not found.");
Git command at line 250	Repository state, publishing, branch checks, or release automation. Evidence: return fileExists(path.join(workspaceRoot, ".git"));
File write at line 267	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(workspaceRoot, { recursive: true });
Installer at line 279	Windows setup, update, shortcut, or installation behavior. Evidence: const defaultWorkspaceHome = process.platform === "win32" && process.env.LOCALAPPDATA
Installer at line 280	Windows setup, update, shortcut, or installation behavior. Evidence: ? path.join(process.env.LOCALAPPDATA, defaultWorkspaceHomeName)
File write at line 286	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(workspaceRoot, { recursive: true });

Item	Explanation
File write at line 287	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: <code>await fsp.mkdir(portfolioRoot, { recursive: true });</code>
Network request at line 363	Frontend-backend communication or public web/API calls. Evidence: <code>return `http://127.0.0.1:\${port}`;</code>
Rich text at line 377	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <code>backgroundColor: "#eef8fd",</code>

Important constants and variables

Item	Explanation
fs	Line 3: <code>const fs = require("node:fs");</code>
fsp	Line 4: <code>const fsp = require("node:fs/promises");</code>
net	Line 5: <code>const net = require("node:net");</code>
path	Line 6: <code>const path = require("node:path");</code>
mainWindow	Line 10: <code>let mainWindow = null;</code>
builderOrigin	Line 11: <code>let builderOrigin = "";</code>
updateShutdownStarted	Line 12: <code>let updateShutdownStarted = false;</code>
execFileAsync	Line 13: <code>const execFileAsync = promisify(execFile);</code>
defaultRepositoryUrl	Line 14: <code>const defaultRepositoryUrl = process.env.OMB_BUILDER_REPOSITORY "";</code>
defaultWorkspaceHomeName	Line 15: <code>const defaultWorkspaceHomeName = "OMB Portfolio Builder";</code>
rootFilesToRefresh	Line 17: <code>const rootFilesToRefresh = [</code>
rootFilesToSeed	Line 30: <code>const rootFilesToSeed = [</code>
directoryRefreshes	Line 38: <code>const directoryRefreshes = ["cloudflare"];</code>
directorySeeds	Line 39: <code>const directorySeeds = ["assets", "Backgrounds", "docs", ".well-known"];</code>
portfolioFilesToSeed	Line 40: <code>const portfolioFilesToSeed = [</code>
portfolioDirectoriesToSeed	Line 51: <code>const portfolioDirectoriesToSeed = ["assets", "Backgrounds", "docs", ".well-known"];</code>
gitCandidates	Line 52: <code>const gitCandidates = [</code>
payload	Line 63: <code>const payload = JSON.stringify(action);</code>
shouldFullscreen	Line 98: <code>const shouldFullscreen = typeof nextState === "boolean" ? nextState : !mainWindow.isFullScreen();</code>
entries	Line 197: <code>const entries = await fsp.readdir(source, { withFileTypes: true });</code>
sourcePath	Line 199: <code>const sourcePath = path.join(source, entry.name);</code>
targetPath	Line 200: <code>const targetPath = path.join(target, entry.name);</code>
entries	Line 218: <code>const entries = await fsp.readdir(directory);</code>
lastError	Line 226: <code>let lastError = null;</code>
current	Line 254: <code>let current = path.dirname(process.execPath);</code>
next	Line 257: <code>const next = path.dirname(current);</code>
bundledSiteRoot	Line 278: <code>const bundledSiteRoot = path.join(process.resourcesPath, "site");</code>
defaultWorkspaceHome	Line 279: <code>const defaultWorkspaceHome = process.platform === "win32" && process.env.LOCALAPPDATA</code>
workspaceHome	Line 282: <code>const workspaceHome = process.env.OMB_WORKSPACE_HOME defaultWorkspaceHome;</code>
workspaceRoot	Line 283: <code>const workspaceRoot = path.join(workspaceHome, "builder");</code>
portfolioRoot	Line 284: <code>const portfolioRoot = process.env.OMB_PORTFOLIO_WORKSPACE path.join(workspaceHome, "portfolio");</code>
envWorkspace	Line 315: <code>const envWorkspace = process.env.OMB_BUILDER_WORKSPACE;</code>
workspaceRoot	Line 323: <code>const workspaceRoot = path.resolve(__dirname);</code>
nearbyRepository	Line 329: <code>const nearbyRepository = findRepositoryNearExecutable();</code>

Item	Explanation
tester	Line 341: const tester = net.createServer();
address	Line 344: const address = tester.address();
port	Line 345: const port = typeof address === "object" && address ? address.port : 0;
port	Line 352: const port = await findFreePort();
serverPath	Line 361: const serverPath = path.join(workspaceRoot, "server.mjs");
iconPath	Line 367: const iconPath = path.join(workspaceRoot, "assets", "omb-app-icon-256.png");
refreshFullscreenMenu	Line 388: const refreshFullscreenMenu = () => {
gotLock	Line 426: const gotLock = app.requestSingleInstanceLock();

Function-by-function detail

dispatchBuilderMenuAction (main.cjs, lines 61-68)

dispatchBuilderMenuAction part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 61-68 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isDestroyed, stringify, executeJavaScript, dispatchEvent, and CustomEvent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 62: if (!mainWindow || mainWindow.isDestroyed()) return;.

Important local values include payload at line 63. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

61: function dispatchBuilderMenuAction(action) {
62:   if (!mainWindow || mainWindow.isDestroyed()) return;
63:   const payload = JSON.stringify(action);
64:   mainWindow.webContents.executeJavaScript(
65:     `window.dispatchEvent(new CustomEvent("builder-menu-action", { detail: ${payload} }));`,
66:     true
67:   ).catch(() => {});
68: }

```

quitForBuilderUpdate (main.cjs, lines 70-92)

quitForBuilderUpdate part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 70-92 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isDestroyed, destroy, quit, and exit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 71: if (updateShutdownStarted) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

70: function quitForBuilderUpdate() {
71:   if (updateShutdownStarted) return;
72:   updateShutdownStarted = true;
73:   try {
74:     if (mainWindow && !mainWindow.isDestroyed()) {
75:       mainWindow.destroy();
76:     }
77:   } catch {
78:     // The process exit fallback below still guarantees the updater can continue.
79:   }
80:   try {
81:     app.quit();

```

```

82:   } catch {
83:     // Fall through to app.exit.
84:   }
85:   setTimeout(() => {
86:     try {
87:       app.exit(0);
88:     } catch {
89:       process.exit(0);
90:     }
91:   }, 1500);
92: }

```

setBuilderFullScreen (main.cjs, lines 96-103)

setBuilderFullScreen part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 96-103 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isDestroyed, isFullScreen, setFullScreen, setMenuBarVisibility, setAutoHideMenuBar, setMenu, and createAppMenu. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 97: if (!mainWindow || mainWindow.isDestroyed()) return;.

Important local values include shouldFullscreen at line 98. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

96: function setBuilderFullScreen(nextState = null) {
97:   if (!mainWindow || mainWindow.isDestroyed()) return;
98:   const shouldFullscreen = typeof nextState === "boolean" ? nextState : !mainWindow.isFullScreen();
99:   mainWindow.setFullScreen(shouldFullscreen);
100:  mainWindow.setMenuBarVisibility(true);
101:  mainWindow.setAutoHideMenuBar(false);
102:  mainWindow.setMenu(createAppMenu(builderOrigin));
103: }

```

createAppMenu (main.cjs, lines 105-184)

createAppMenu creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 105-184 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references buildFromTemplate, dispatchBuilderMenuAction, setBuilderFullScreen, openExternal, isDestroyed, isMinimized, restore, focus, fileExists, and accessSync. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 106: return Menu.buildFromTemplate([; line 168: if (!mainWindow || mainWindow.isDestroyed()) return;; line 181: return true;; line 183: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

105: function createAppMenu(origin) {
106:   return Menu.buildFromTemplate([
107:     {
108:       label: "File",
109:       submenu: [
110:         { label: "New Project", accelerator: "CmdOrCtrl+N", click: () => dispatchBuilderMenuAction({
type: "new-project" }) },
111:         { label: "Publishing Target", click: () => dispatchBuilderMenuAction({ type: "publishing-target"
}) },
112:         { type: "separator" },
113:         { label: "Save Draft", accelerator: "CmdOrCtrl+S", click: () => dispatchBuilderMenuAction({ type:
"save-draft" }) },
114:         { label: "Apply to Site", accelerator: "CmdOrCtrl+Shift+S", click: () =>
dispatchBuilderMenuAction({ type: "apply-site" }) },
115:         { type: "separator" },
116:         { role: "quit", label: "Exit" }
117:       ]
118:     },
119:     {
120:       label: "Edit",
121:       submenu: [
122:         { role: "undo" },

```

```

123:         { role: "redo" },
124:         { type: "separator" },
125:         { role: "cut" },
126:         { role: "copy" },
127:         { role: "paste" },
128:         { role: "selectAll" }
129:     ]
130: },
131: {
132:     label: "View",
133:     submenu: [
134:         { role: "reload" },
135:         { role: "forceReload" },
136:         { type: "separator" },
137:         { role: "resetZoom" },
138:         { role: "zoomIn" },
139:         { role: "zoomOut" },
140:         { type: "separator" },
141:         { label: "Portfolio Preview", click: () => dispatchBuilderMenuAction({ type: "portfolio-preview"
142:     }) },
143:         { label: "Check Updates", click: () => dispatchBuilderMenuAction({ type: "check-updates" }) },
144:         { type: "separator" },
145:         {
146:             label: mainWindow?.isFullScreen?.() ? "Exit Full Screen" : "Toggle Full Screen",
147:             accelerator: "F11",
148:             click: () => setBuilderFullScreen()
149:         }
150:     ],
151: },
152: {
153:     label: "Preferences",
154:     submenu: [
155:         { label: "Open Preferences", accelerator: "CmdOrCtrl+," , click: () => dispatchBuilderMenuAction({
156:             type: "preferences" }) },
157:         { type: "separator" },
158:         { label: "Light Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "light"
159:             }) },
160:         { label: "Dark Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "dark"
161:             }) }
162:     ],
163: },
164: {
165:     label: "Help",
166:     submenu: [
167:         { label: "Builder Guide", accelerator: "F1", click: () => dispatchBuilderMenuAction({ type:
168:             "builder-guide" }) },
169:         { label: "GitHub Releases", click: () =>
170:             shell.openExternal("https://github.com/otienomaurice/omb-portfolio-builder/releases/latest") },
171:         {
172:             label: "Focus Builder Window",
173:             click: () => {
174:                 if (!mainWindow || mainWindow.isDestroyed()) return;
175:                 if (mainWindow.isMinimized()) mainWindow.restore();
176:                 mainWindow.focus();
177:             }
178:         }
179:     ],
180: },
181: ];
182: }
183:
184: function fileExists(filePath) {
185:     try {
186:         fs.accessSync(filePath);
187:         return true;
188:     } catch {
189:         return false;
190:     }
191: }

```

fileExists (main.cjs, lines 178-185)

fileExists part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 178-185 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references accessSync. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 181: return true;; line 183: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:


```

178: function fileExists(filePath) {
179:   try {
180:     fs.accessSync(filePath);
181:     return true;
182:   } catch {
183:     return false;
184:   }
185: }

```

copyFileIfAvailable (main.cjs, lines 187-192)

copyFileIfAvailable part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 187-192 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, mkdir, dirname, and copyFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 188: if (!fileExists(source)) return;; line 189: if (!overwrite && fileExists(target)) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

187: async function copyFileIfAvailable(source, target, overwrite = false) {
188:   if (!fileExists(source)) return;
189:   if (!overwrite && fileExists(target)) return;
190:   await fsp.mkdir(path.dirname(target), { recursive: true });
191:   await fsp.copyFile(source, target);
192: }

```

copyDirectoryMissingFiles (main.cjs, lines 194-207)

copyDirectoryMissingFiles part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 194-207 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, mkdir, readdir, join, isDirectory, isFile, and copyFileIfAvailable. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 195: if (!fileExists(source)) return;.

Important local values include entries at line 197; sourcePath at line 199; targetPath at line 200. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

194: async function copyDirectoryMissingFiles(source, target) {
195:   if (!fileExists(source)) return;
196:   await fsp.mkdir(target, { recursive: true });
197:   const entries = await fsp.readdir(source, { withFileTypes: true });
198:   for (const entry of entries) {
199:     const sourcePath = path.join(source, entry.name);
200:     const targetPath = path.join(target, entry.name);
201:     if (entry.isDirectory()) {
202:       await copyDirectoryMissingFiles(sourcePath, targetPath);
203:     } else if (entry.isFile()) {
204:       await copyFileIfAvailable(sourcePath, targetPath, false);
205:     }
206:   }
207: }

```

copyDirectoryRefresh (main.cjs, lines 209-214)

copyDirectoryRefresh part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 209-214 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, rm, mkdir, dirname, and cp. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 210: if (!fileExists(source)) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
209: async function copyDirectoryRefresh(source, target) {
210:   if (!fileExists(source)) return;
211:   await fsp.rm(target, { recursive: true, force: true });
212:   await fsp.mkdir(path.dirname(target), { recursive: true });
213:   await fsp.cp(source, target, { recursive: true, force: true });
214: }
```

directoryIsEmpty (main.cjs, lines 216-223)

directoryIsEmpty part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 216-223 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references readdir. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 219: return entries.length === 0;; line 221: return true;.

Important local values include entries at line 218. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
216: async function directoryIsEmpty(directory) {
217:   try {
218:     const entries = await fsp.readdir(directory);
219:     return entries.length === 0;
220:   } catch {
221:     return true;
222:   }
223: }
```

runGit (main.cjs, lines 225-242)

runGit part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 225-242 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references execFileAsync, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 234: return true;.

Important local values include lastError at line 226. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
225: async function runGit(args, cwd) {
226:   let lastError = null;
227:   for (const candidate of gitCandidates) {
228:     try {
229:       await execFileAsync(candidate, args, {
230:         cwd,
231:         maxBuffer: 10 * 1024 * 1024,
232:         windowsHide: true
233:       });
234:       return true;
235:     } catch (error) {
236:       lastError = error;
237:       if (error.code === "ENOENT") continue;
238:       throw error;
239:     }
240:   }
241:   throw lastError || new Error("Git executable was not found.");
242: }
```

workspaceLooksUsable (main.cjs, lines 244-247)

workspaceLooksUsable part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 244-247 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 245: return fileExists(path.join(workspaceRoot, "server.mjs")).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
244: function workspaceLooksUsable(workspaceRoot) {  
245:   return fileExists(path.join(workspaceRoot, "server.mjs"))  
246:   && fileExists(path.join(workspaceRoot, "index.html"));  
247: }
```

workspacesGitBacked (main.cjs, lines 249-251)

workspacesGitBacked part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 249-251 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 250: return fileExists(path.join(workspaceRoot, ".git"));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
249: function workspaceIsGitBacked(workspaceRoot) {  
250:   return fileExists(path.join(workspaceRoot, ".git"));  
251: }
```

findRepositoryNearExecutable (main.cjs, lines 253-262)

findRepositoryNearExecutable part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 253-262 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references dirname, workspaceLooksUsable, and workspacesGitBacked. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 256: if (workspaceLooksUsable(current) && workspacesGitBacked(current)) return current;; line 261: return "";

Important local values include current at line 254; next at line 257. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
253: function findRepositoryNearExecutable() {  
254:   let current = path.dirname(process.execPath);  
255:   for (let index = 0; index < 8; index += 1) {  
256:     if (workspaceLooksUsable(current) && workspaceIsGitBacked(current)) return current;  
257:     const next = path.dirname(current);  
258:     if (next === current) break;  
259:     current = next;  
260:   }  
261:   return "";  
262: }
```

cloneWorkspaceIfPossible (main.cjs, lines 264-275)

cloneWorkspaceIfPossible part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 264-275 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references workspacesGitBacked, mkdir, directoryIsEmpty, runGit, and dirname. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 265: if (!defaultRepositoryUrl) return false;; line 266: if (workspacesGitBacked(workspaceRoot)) return true;; line 268: if (!(await directoryIsEmpty(workspaceRoot))) return false;; line 271: return workspacesGitBacked(workspaceRoot);. There are 1 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
264: async function cloneWorkspaceIfPossible(workspaceRoot) {
265:   if (!defaultRepositoryUrl) return false;
266:   if (workspacesGitBacked(workspaceRoot)) return true;
267:   await fsp.mkdir(workspaceRoot, { recursive: true });
268:   if (!(await directoryIsEmpty(workspaceRoot))) return false;
269:   try {
270:     await runGit(["clone", "--depth", "1", defaultRepositoryUrl, workspaceRoot],
path.dirname(workspaceRoot));
271:     return workspacesGitBacked(workspaceRoot);
272:   } catch {
273:     return false;
274:   }
275: }
```

preparePackagedWorkspace (main.cjs, lines 277-312)

This function prepares the writable workspace that the packaged desktop application will use at runtime. The installed app cannot safely treat packaged resources as editable source files, so this function copies or refreshes the required builder and portfolio resources into a location the user can actually write to.

Without this function, a fresh install could open with missing website files, stale resources, or files trapped inside the packaged app bundle. It is one of the reasons the builder can behave like a normal desktop app while still using web assets internally.

It calls or references join, getPath, mkdir, cloneWorkspaceIfPossible, copyFileIfAvailable, copyDirectoryRefresh, and copyDirectoryMissingFiles. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 311: return { workspaceRoot, portfolioRoot };

Important local values include bundledSiteRoot at line 278; defaultWorkspaceHome at line 279; workspaceHome at line 282; workspaceRoot at line 283; portfolioRoot at line 284. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
277: async function preparePackagedWorkspace() {
278:   const bundledSiteRoot = path.join(process.resourcesPath, "site");
279:   const defaultWorkspaceHome = process.platform === "win32" && process.env.LOCALAPPDATA
280:     ? path.join(process.env.LOCALAPPDATA, defaultWorkspaceHomeName)
281:     : path.join(app.getPath("userData"), "workspace");
282:   const workspaceHome = process.env.OMB_WORKSPACE_HOME || defaultWorkspaceHome;
283:   const workspaceRoot = path.join(workspaceHome, "builder");
284:   const portfolioRoot = process.env.OMB_PORTFOLIO_WORKSPACE || path.join(workspaceHome, "portfolio");
285:
286:   await fsp.mkdir(workspaceRoot, { recursive: true });
287:   await fsp.mkdir(portfolioRoot, { recursive: true });
288:   await cloneWorkspaceIfPossible(portfolioRoot);
289:
290:   for (const fileName of rootFilesToRefresh) {
291:     await copyFileIfAvailable(path.join(bundledSiteRoot, fileName), path.join(workspaceRoot, fileName),
true);
292:   }
293:   for (const fileName of rootFilesToSeed) {
294:     await copyFileIfAvailable(path.join(bundledSiteRoot, fileName), path.join(workspaceRoot, fileName),
false);
295:   }
296:   for (const directoryName of directoryRefreshes) {
297:     await copyDirectoryRefresh(path.join(bundledSiteRoot, directoryName), path.join(workspaceRoot,
directoryName));
298:   }
299:   for (const directoryName of directorySeeds) {
300:     await copyDirectoryMissingFiles(path.join(bundledSiteRoot, directoryName), path.join(workspaceRoot,
directoryName));
301:   }
```

```

302:
303:   for (const fileName of portfolioFilesToSeed) {
304:     await copyFileIfAvailable(path.join(bundledSiteRoot, fileName), path.join(portfolioRoot, fileName),
false);
305:   }
306:   await copyFileIfAvailable(path.join(bundledSiteRoot, "portfolio-README.md"), path.join(portfolioRoot,
"README.md"), false);
307:   for (const directoryName of portfolioDirectoriesToSeed) {
308:     await copyDirectoryMissingFiles(path.join(bundledSiteRoot, directoryName), path.join(portfolioRoot,
directoryName));
309:   }
310:
311:   return { workspaceRoot, portfolioRoot };
312: }

```

resolveWorkspaceRoots (main.cjs, lines 314-337)

This function decides where the builder workspace and portfolio workspace live for the current run. It combines packaged-app defaults, development-mode paths, and user-owned writable locations so the rest of the app can use stable roots instead of guessing paths repeatedly.

The builder has to work when launched from source, from an installed executable, and from an updated install. Centralizing workspace resolution prevents different parts of the app from accidentally reading or writing different copies of the portfolio.

It calls or references workspaceLooksUsable, join, dirname, resolve, findRepositoryNearExecutable, and preparePackagedWorkspace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 317: return {; line 324: return {; line 331: return {; line 336: return preparePackagedWorkspace();.

Important local values include envWorkspace at line 315; workspaceRoot at line 323; nearbyRepository at line 329. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

314: async function resolveWorkspaceRoots() {
315:   const envWorkspace = process.env.OMB_BUILDER_WORKSPACE;
316:   if (envWorkspace && workspaceLooksUsable(envWorkspace)) {
317:     return {
318:       workspaceRoot: envWorkspace,
319:       portfolioRoot: process.env.OMB_PORTFOLIO_WORKSPACE || path.join(path.dirname(envWorkspace),
"portfolio")
320:     };
321:   }
322:   if (!app.isPackaged) {
323:     const workspaceRoot = path.resolve(__dirname);
324:     return {
325:       workspaceRoot,
326:       portfolioRoot: process.env.OMB_PORTFOLIO_WORKSPACE || workspaceRoot
327:     };
328:   }
329:   const nearbyRepository = findRepositoryNearExecutable();
330:   if (nearbyRepository) {
331:     return {
332:       workspaceRoot: nearbyRepository,
333:       portfolioRoot: process.env.OMB_PORTFOLIO_WORKSPACE || nearbyRepository
334:     };
335:   }
336:   return preparePackagedWorkspace();
337: }

```

findFreePort (main.cjs, lines 339-349)

findFreePort part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 339-349 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createServer, once, listen, address, close, and resolve. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 340: return new Promise((resolve, reject) => {.

Important local values include tester at line 341; address at line 344; port at line 345. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

339: function findFreePort() {
340:   return new Promise((resolve, reject) => {
341:     const tester = net.createServer();
342:     tester.once("error", reject);
343:     tester.listen(0, "127.0.0.1", () => {
344:       const address = tester.address();
345:       const port = typeof address === "object" && address ? address.port : 0;
346:       tester.close(() => resolve(port));
347:     });
348:   });
349: }

```

startBuilderServer (main.cjs, lines 351-364)

This function starts the local backend process that serves the builder UI and handles local APIs. It chooses a port, launches server.mjs with the resolved workspace roots, and gives Electron an origin URL it can load.

The Electron window is only the desktop shell. The heavy work still belongs to the backend: saving files, running Git, compiling code, and publishing. This function is the bridge between the desktop shell and that backend.

It calls or references findFreePort, getVersion, join, pathToFileURL, and now. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 363: return `http://127.0.0.1:\${port}`.

Important local values include port at line 352; serverPath at line 361. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

351: async function startBuilderServer(workspaceRoot, portfolioRoot) {
352:   const port = await findFreePort();
353:   process.env.PORT = String(port);
354:   process.env.HOST = "127.0.0.1";
355:   process.env.OMB_DESKTOP_APP = "1";
356:   process.env.OMB_BUILDER_WORKSPACE = workspaceRoot;
357:   process.env.OMB_PORTFOLIO_WORKSPACE = portfolioRoot;
358:   process.env.OMB_BUILDER_REPOSITORY = defaultRepositoryUrl;
359:   process.env.OMB_APP_VERSION = app.getVersion();
360:
361:   const serverPath = path.join(workspaceRoot, "server.mjs");
362:   await import(`${pathToFileURL(serverPath).href}?desktop=${Date.now()}`);
363:   return `http://127.0.0.1:${port}`;
364: }

```

createWindow (main.cjs, lines 366-423)

This function creates the Electron BrowserWindow and loads the local builder interface. It configures the window behavior, menu integration, navigation safety, fullscreen behavior, and the local URL where the builder runs.

This is the point where the desktop application becomes visible. A mistake here can make the app look like a browser tab, block menus, break fullscreen, or load the wrong workspace.

It calls or references join, BrowserWindow, fileExists, setMenu, createAppMenu, setMenuBarVisibility, setAutoHideMenuBar, isDestroyed, on, preventDefault, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 402: return;; line 411: if (url.startsWith(origin)) return { action: "allow" };;; line 413: return { action: "deny" };;; line 417: if (url.startsWith(origin)) return;;.

Important local values include iconPath at line 367; refreshFullscreenMenu at line 388. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

366: function createWindow(workspaceRoot, origin) {
367:   const iconPath = path.join(workspaceRoot, "assets", "omb-app-icon-256.png");
368:   nativeTheme.themeSource = "light";
369:   mainWindow = new BrowserWindow({
370:     width: 1440,
371:     height: 960,
372:     minWidth: 980,
373:     minHeight: 680,
374:     title: "OMB Portfolio Builder",
375:     autoHideMenuBar: false,
376:     icon: fileExists(iconPath) ? iconPath : undefined,
377:     backgroundColor: "#eef8fd",
378:     webPreferences: {
379:       contextIsolation: true,
380:       nodeIntegration: false,
381:       sandbox: true
382:     }

```

```

383: });
384: mainWindow.setMenu(createAppMenu(origin));
385: mainWindow.setMenuBarVisibility(true);
386: mainWindow.setAutoHideMenuBar(false);
387:
388: const refreshFullscreenMenu = () => {
389:   if (mainWindow && !mainWindow.isDestroyed()) {
390:     mainWindow.setMenuBarVisibility(true);
391:     mainWindow.setAutoHideMenuBar(false);
392:     mainWindow.setMenu(createAppMenu(origin));
393:   }
394: };
395: mainWindow.on("enter-full-screen", refreshFullscreenMenu);
396: mainWindow.on("leave-full-screen", refreshFullscreenMenu);
397:
398: mainWindow.webContents.on("before-input-event", (event, input) => {
399:   if (input.key === "F11") {
400:     event.preventDefault();
401:     setBuilderFullScreen();
402:     return;
403:   }
404:   if (input.key === "Escape" && mainWindow.isFullScreen()) {
405:     event.preventDefault();
406:     setBuilderFullScreen(false);
407:   }
408: });
409:
410: mainWindow.webContents.setWindowOpenHandler(({ url }) => {
411:   if (url.startsWith(origin)) return { action: "allow" };
412:   shell.openExternal(url);
413:   return { action: "deny" };
414: });
415:
416: mainWindow.webContents.on("will-navigate", (event, url) => {
417:   if (url.startsWith(origin)) return;
418:   event.preventDefault();
419:   shell.openExternal(url);
420: });
421:
422: mainWindow.loadURL(`${origin}/template-preview.html`);
423: }

```

refreshFullscreenMenu (main.cjs, lines 388-394)

refreshFullscreenMenu part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 388-394 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isDestroyed, setMenuBarVisibility, setAutoHideMenuBar, setMenu, and createAppMenu. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include refreshFullscreenMenu at line 388. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

388:   const refreshFullscreenMenu = () => {
389:     if (mainWindow && !mainWindow.isDestroyed()) {
390:       mainWindow.setMenuBarVisibility(true);
391:       mainWindow.setAutoHideMenuBar(false);
392:       mainWindow.setMenu(createAppMenu(origin));
393:     }
394:   };

```

boot (main.cjs, lines 425-447)

This is the main startup routine. It prepares the workspace, starts the backend server, creates the desktop window, and connects the pieces in the order required for the builder to become usable.

A desktop app needs a controlled startup sequence. The server must exist before the window loads, and the workspace must exist before the server can serve or save data. boot keeps that sequence explicit.

It calls or references requestSingleInstanceLock, quit, on, isMinimized, restore, focus, whenReady, resolveWorkspaceRoots, startBuilderServer, createWindow, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 429: return;; line 433: if (!mainWindow) return;.

Important local values include gotLock at line 426. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
425: async function boot() {
426:   const gotLock = app.requestSingleInstanceLock();
427:   if (!gotLock) {
428:     app.quit();
429:     return;
430:   }
431:
432:   app.on("second-instance", () => {
433:     if (!mainWindow) return;
434:     if (mainWindow.isMinimized()) mainWindow.restore();
435:     mainWindow.focus();
436:   });
437:
438:   await app.whenReady();
439:   try {
440:     const { workspaceRoot, portfolioRoot } = await resolveWorkspaceRoots();
441:     builderOrigin = await startBuilderServer(workspaceRoot, portfolioRoot);
442:     createWindow(workspaceRoot, builderOrigin);
443:   } catch (error) {
444:     dialog.showErrorBox("OMB Portfolio Builder could not start", error?.message || String(error));
445:     app.quit();
446:   }
447: }
```

Representative opening snippet

```
1: const { app, BrowserWindow, Menu, dialog, nativeTheme, shell } = require("electron");
2: const { execFile } = require("node:child_process");
3: const fs = require("node:fs");
4: const fsp = require("node:fs/promises");
5: const net = require("node:net");
6: const path = require("node:path");
7: const { pathToFileURL } = require("node:url");
8: const { promisify } = require("node:util");
9:
10: let mainWindow = null;
11: let builderOrigin = "";
12: let updateShutdownStarted = false;
13: const execFileAsync = promisify(execFile);
14: const defaultRepositoryUrl = process.env.OMB_BUILDER_REPOSITORY || "";
15: const defaultWorkspaceHomeName = "OMB Portfolio Builder";
16:
17: const rootFilesToRefresh = [
18:   "server.mjs",
19:   "index.html",
20:   "styles.css",
21:   "script.js",
22:   "electronics-search.js",
23:   "builder-rich-future-sections.js",
24:   "template-preview.html",
25:   "template-preview.css",
26:   "template-preview.js",
27:   "_headers"
28: ];
29:
30: const rootFilesToSeed = [
31:   "projects.json",
32:   "project-templates.json",
33:   "README.md",
34:   ".nojekyll",
35:   "robots.txt"
36: ];
37:
38: const directoryRefreshes = ["cloudflare"];
39: const directorySeeds = ["assets", "Backgrounds", "docs", ".well-known"];
40: const portfolioFilesToSeed = [
41:   "projects.json",
42:   "index.html",
43:   "styles.css",
44:   "script.js",
45:   "electronics-search.js",
46:   ".nojekyll",
47:   "robots.txt",
48:   "_headers",
49:   "sitemap.xml"
50: ];
51: const portfolioDirectoriesToSeed = ["assets", "Backgrounds", "docs", ".well-known"];
52: const gitCandidates = [
53:   process.env.GIT_EXE,
54:   "git",
55:   "C:\\Program Files\\Git\\cmd\\git.exe",
56:   "C:\\Program Files\\Git\\bin\\git.exe",
57:   "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
58:   "C:\\Program Files (x86)\\Git\\bin\\git.exe"
```



```
59: ].filter(Boolean);
60:
61: function dispatchBuilderMenuAction(action) {
62:   if (!mainWindow || mainWindow.isDestroyed()) return;
63:   const payload = JSON.stringify(action);
64:   mainWindow.webContents.executeJavaScript(
65:     `window.dispatchEvent(new CustomEvent("builder-menu-action", { detail: ${payload} }));`,
66:     true
67:   ).catch(() => {});
68: }
69:
70: function quitForBuilderUpdate() {
71:   if (updateShutdownStarted) return;
72:   updateShutdownStarted = true;
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: server.mjs

Item	Explanation
File	server.mjs
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	147,116 bytes
Extension	.mjs
MIME type	text/plain
Text lines	3,632

Why this file exists

- Local backend server used by the builder for drafts, parsing, publishing, compiler execution, authentication checks, and file operations.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Imports, requires, and external dependencies

Item	Explanation
Line 1	import { createServer } from "node:http";
Line 2	import { execFile, spawn } from "node:child_process";
Line 3	import { createHash } from "node:crypto";
Line 4	import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from "node:fs/promises";
Line 5	import os from "node:os";
Line 6	import path from "node:path";
Line 7	import { promisify } from "node:util";
Line 8	import { fileURLToPath } from "node:url";

API endpoints mentioned or called

Item	Explanation
/api/catalog	Line 3343. Loads project/catalog data for the builder.
/api/templates	Line 3353. Loads appearance template definitions.
/api/publish-target	Line 3358. Reads or writes the Git publishing target and authorization state.
/api/system-check	Line 3367. Checks local tool/system readiness.
/api/app-update	Line 3372. Checks or starts builder app update behavior.
/api/security-report	Line 3377. Returns security/download/auth reporting for the builder.
/api/code/tools	Line 3386. Checks compiler/runtime availability.
/api/portfolio-ai	Line 3402. Handles AI assistant questions.
/api/code/beautify	Line 3407. Formats source code for cleaner display and editing.

Item	Explanation
/api/code/save	Line 3422. Saves source code into the local compile workspace.
/api/code/compile	Line 3433. Compiles or runs a source file and returns terminal output.
/api/code/install-tools	Line 3444. Attempts to install missing compiler tools.
/api/publish-target	Line 3455. Reads or writes the Git publishing target and authorization state.
/api/install-git	Line 3465. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/app-update/install	Line 3479. Checks or starts builder app update behavior.
/api/github-authenticate	Line 3493. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/sync-from-publish-target	Line 3510. Reads or writes the Git publishing target and authorization state.
/api/save-draft	Line 3525. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/apply-catalog	Line 3525. Loads project/catalog data for the builder.
/api/apply-catalog	Line 3534. Loads project/catalog data for the builder.
/api/upload	Line 3577. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.

Important implementation signals

Item	Explanation
Compiler or simulator at line 1	Part of the Compile Code workspace or language/tool detection. Evidence: import { createServer } from "node:http";
Process execution at line 2	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: import { execFile, spawn } from "node:child_process";
Compiler or simulator at line 3	Part of the Compile Code workspace or language/tool detection. Evidence: import { createHash } from "node:crypto";
File read at line 4	Reads local builder state, project files, website assets, or configuration. Evidence: import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from "node:fs/promises";
Compiler or simulator at line 5	Part of the Compile Code workspace or language/tool detection. Evidence: import os from "node:os";
Compiler or simulator at line 6	Part of the Compile Code workspace or language/tool detection. Evidence: import path from "node:path";
Compiler or simulator at line 7	Part of the Compile Code workspace or language/tool detection. Evidence: import { promisify } from "node:util";
Compiler or simulator at line 8	Part of the Compile Code workspace or language/tool detection. Evidence: import { fileURLToPath } from "node:url";
Process execution at line 15	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: const execFileAsync = promisify(execFile);
File read at line 16	Reads local builder state, project files, website assets, or configuration. Evidence: const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
Browser DOM at line 34	Builder or website interface behavior in the browser/Electron renderer. Evidence: ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
Security or auth at line 40	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
Security or auth at line 51	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers",
Security or auth at line 57	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
Security or auth at line 58	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
Security or auth at line 59	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
Security or auth at line 60	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthExtendedThreshold = 3;
Security or auth at line 63	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthorizationHelp = [
Security or auth at line 65	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Open Publishing target, enter the repository URL, then click Authenticate target.",

Item	Explanation
Git command at line 66	Repository state, publishing, branch checks, or release automation. Evidence: "A GitHub/Git Credential Manager browser sign-in may open. Sign in with an account that has write access to the selected Pages repository.",
Security or auth at line 67	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Authenticate target only verifies write access and remembers it for the matching repository and branch.",
Security or auth at line 68	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Load from target only imports compatible website files from the authenticated repository into the local builder workspace.",
Git command at line 72	Repository state, publishing, branch checks, or release automation. Evidence: process.env.GIT_EXE,
Git command at line 73	Repository state, publishing, branch checks, or release automation. Evidence: "git",
Git command at line 74	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\cmd\\git.exe",
Git command at line 75	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\bin\\git.exe",
Git command at line 76	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
Git command at line 77	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\bin\\git.exe",
Git command at line 78	Repository state, publishing, branch checks, or release automation. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "cmd", "git.exe") : "",
Git command at line 79	Repository state, publishing, branch checks, or release automation. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "bin", "git.exe") : ""
Compiler or simulator at line 87	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["gcc"],
Compiler or simulator at line 97	Part of the Compile Code workspace or language/tool detection. Evidence: verilog: {
Compiler or simulator at line 100	Part of the Compile Code workspace or language/tool detection. Evidence: label: "Verilog",
Compiler or simulator at line 101	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["iverilog", "vvp"],
Compiler or simulator at line 102	Part of the Compile Code workspace or language/tool detection. Evidence: winget: ["Icarus.Verilog"]
Compiler or simulator at line 104	Part of the Compile Code workspace or language/tool detection. Evidence: systemverilog: {
Compiler or simulator at line 107	Part of the Compile Code workspace or language/tool detection. Evidence: label: "SystemVerilog",
Compiler or simulator at line 108	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["iverilog", "vvp"],
Compiler or simulator at line 109	Part of the Compile Code workspace or language/tool detection. Evidence: winget: ["Icarus.Verilog"]
Compiler or simulator at line 118	Part of the Compile Code workspace or language/tool detection. Evidence: java: {
Compiler or simulator at line 119	Part of the Compile Code workspace or language/tool detection. Evidence: defaultFile: "Main.java",
Compiler or simulator at line 120	Part of the Compile Code workspace or language/tool detection. Evidence: extensions: [".java"],
Compiler or simulator at line 121	Part of the Compile Code workspace or language/tool detection. Evidence: label: "Java",
Compiler or simulator at line 122	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["javac", "java"],
Compiler or simulator at line 129	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["node"],
Compiler or simulator at line 132	Part of the Compile Code workspace or language/tool detection. Evidence: python: {
Compiler or simulator at line 135	Part of the Compile Code workspace or language/tool detection. Evidence: label: "Python",

Item	Explanation
Compiler or simulator at line 136	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["python"],
Compiler or simulator at line 149	Part of the Compile Code workspace or language/tool detection. Evidence: gcc: [
Compiler or simulator at line 150	Part of the Compile Code workspace or language/tool detection. Evidence: "gcc",
Compiler or simulator at line 151	Part of the Compile Code workspace or language/tool detection. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages", "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "gcc.exe") : ""
Installer at line 156	Windows setup, update, shortcut, or installation behavior. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages", "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "g++.exe") : ""
Compiler or simulator at line 161	Part of the Compile Code workspace or language/tool detection. Evidence: javac: [
Compiler or simulator at line 162	Part of the Compile Code workspace or language/tool detection. Evidence: "javac",
Compiler or simulator at line 163	Part of the Compile Code workspace or language/tool detection. Evidence: "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\javac.exe"
Compiler or simulator at line 165	Part of the Compile Code workspace or language/tool detection. Evidence: java: [
Compiler or simulator at line 166	Part of the Compile Code workspace or language/tool detection. Evidence: "java",
Compiler or simulator at line 167	Part of the Compile Code workspace or language/tool detection. Evidence: "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\java.exe"
Compiler or simulator at line 169	Part of the Compile Code workspace or language/tool detection. Evidence: node: [
Compiler or simulator at line 170	Part of the Compile Code workspace or language/tool detection. Evidence: /(?:^[\\V])node(?:\\.exe)?\$/i.test(process.execPath "") ? process.execPath : "",
Compiler or simulator at line 171	Part of the Compile Code workspace or language/tool detection. Evidence: "node",
Compiler or simulator at line 172	Part of the Compile Code workspace or language/tool detection. Evidence: "C:\\Program Files\\nodejs\\node.exe",
Compiler or simulator at line 173	Part of the Compile Code workspace or language/tool detection. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "nodejs", "node.exe") : ""
Compiler or simulator at line 175	Part of the Compile Code workspace or language/tool detection. Evidence: python: [process.env.PYTHON, "python", "py"],
Compiler or simulator at line 176	Part of the Compile Code workspace or language/tool detection. Evidence: iverilog: ["iverilog", "C:\\iverilog\\bin\\iverilog.exe"],
Compiler or simulator at line 177	Part of the Compile Code workspace or language/tool detection. Evidence: vvp: ["vvp", "C:\\iverilog\\bin\\vvp.exe"],
Installer at line 180	Windows setup, update, shortcut, or installation behavior. Evidence: "C:\\Program Files\\ADI\\LTspice\\LTspice.exe",
Installer at line 181	Windows setup, update, shortcut, or installation behavior. Evidence: "C:\\Program Files\\LTC\\LTspiceXVII\\XVIIx64.exe",
Installer at line 182	Windows setup, update, shortcut, or installation behavior. Evidence: "C:\\Program Files\\Analog Devices\\LTspice\\LTspice.exe",
Installer at line 183	Windows setup, update, shortcut, or installation behavior. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "ADI", "LTspice", "LTspice.exe") : ""
Local storage at line 186	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: const compileToolCache = new Map();
Local storage at line 187	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: const compileToolVersionCache = new Map();
Cloudflare or AI at line 192	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Answer the visitor's question first in a concise ChatGPT-like format, then add portfolio links or context only when they help.",
Security or auth at line 210	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Do not invent portfolio projects, credentials, employers, files, or test results that are not in the context.",

Item	Explanation
Security or auth at line 218	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function securityHeaders(extra = {}) {
Security or auth at line 222	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",
Security or auth at line 230	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ...securityHeaders(),
Local storage at line 232	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: "Cache-Control": "no-store, no-cache, must-revalidate"
Compiler or simulator at line 263	Part of the Compile Code workspace or language/tool detection. Evidence: systemverilog: "systemverilog",
Compiler or simulator at line 264	Part of the Compile Code workspace or language/tool detection. Evidence: sv: "systemverilog",

Important constants and variables

Item	Explanation
root	Line 10: const root = path.dirname(fileURLToPath(import.meta.url));
portfolioRoot	Line 11: const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE root);
compileRoot	Line 12: const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE path.join(path.dirname(root), "compile-code"));
port	Line 13: const port = Number(process.env.PORT 8080);
host	Line 14: const host = process.env.HOST "0.0.0.0";
execFileAsync	Line 15: const execFileAsync = promisify(execFile);
packageJson	Line 16: const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
types	Line 18: const types = {
draftPath	Line 38: const draftPath = path.join(root, "projects.local.json");
catalogPath	Line 39: const catalogPath = path.join(root, "projects.json");
publishAuthCachePath	Line 40: const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
publishPaths	Line 41: const publishPaths = [
publishAuthCacheTtlMs	Line 57: const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
publishAuthExtendedTtlMs	Line 58: const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
publishAuthHistoryWindowMs	Line 59: const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
publishAuthExtendedThreshold	Line 60: const publishAuthExtendedThreshold = 3;
defaultSiteRepository	Line 61: const defaultSiteRepository = process.env.OMB_BUILDER_REPOSITORY "";
blockedAppUpdateVersions	Line 62: const blockedAppUpdateVersions = new Set(["0.2.16"]);
publishAuthorizationHelp	Line 63: const publishAuthorizationHelp = [
gitCandidates	Line 71: const gitCandidates = [
compileLanguageProfiles	Line 82: const compileLanguageProfiles = {
compileToolCandidates	Line 148: const compileToolCandidates = {
compileToolCache	Line 186: const compileToolCache = new Map();
compileToolVersionCache	Line 187: const compileToolVersionCache = new Map();
portfolioAiInstructions	Line 189: const portfolioAiInstructions = [
clean	Line 257: const clean = String(value "").trim().toLowerCase().replace(/[_\s-]+/g, "");
aliases	Line 258: const aliases = {
text	Line 283: const text = String(source "");

Item	Explanation
text	Line 289: const text = String(source "");
ext	Line 295: const ext = path.extname(String(fileName "").toLowerCase());
byName	Line 305: const byName = languageFromFileName(fileName, code);
source	Line 307: const source = String(code "");
profile	Line 322: const profile = compileLanguageProfiles[normalizeCodeLanguage(language)] compileLanguageProfiles.javascript;
fallback	Line 323: const fallback = profile.defaultFile;
parsed	Line 324: const parsed = path.parse(String(value fallback));
safeName	Line 325: const safeName = safeSegment(parsed.name, path.parse(fallback).name);
ext	Line 326: let ext = String(parsed.ext path.extname(fallback)).toLowerCase();
depth	Line 332: let depth = 0;
line	Line 337: const line = rawLine.trim();
formatted	Line 340: const formatted = `\${" ".repeat(depth)}\${line}`;
opens	Line 341: const opens = (line.match(/[{}()]/g) []).length;
closes	Line 342: const closes = (line.match(/[\[\]\{\}]/g) []).length;
normalized	Line 351: const normalized = String(code "").replace(/r\n?/g, "\n").replace(/t/g, " ");
lang	Line 352: const lang = normalizeCodeLanguage(language);
entries	Line 376: let entries = [];
fullPath	Line 383: const fullPath = path.join(folder, entry.name);
found	Line 388: const found = await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1);
candidates	Line 396: const candidates = (compileToolCandidates[toolName] [toolName]).filter(Boolean);
command	Line 406: const command = process.platform === "win32" ? "where" : "command";
args	Line 407: const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
result	Line 408: const result = await execFileAsync(command, args, { timeout: 5000, windowsHide: true });
found	Line 409: const found = String(result.stdout "").split(/\r?\n/).map((line) => line.trim()).find(Boolean);
found	Line 420: const found = await findExecutableUnder(
found	Line 430: const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`\${toolName}.exe`], 5);
elapsedSeconds	Line 444: const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
elapsed	Line 445: const elapsed = elapsedSeconds ? `in \${elapsedSeconds}s` : "";
status	Line 446: const status = result.timedOut
output	Line 449: const output = [result.stdout, result.stderr].map((part) => String(part "").trimEnd()).filter(Boolean).join("\n");
clean	Line 454: const clean = String(value "");
output	Line 464: let output = String(text "");
from	Line 466: const from = replacement?.from "";
to	Line 467: const to = replacement?.to "";
isCpp	Line 481: const isCpp = language === "cpp";
ext	Line 482: const ext = path.extname(String(fileName "").toLowerCase());
header	Line 483: const header = [".h", ".hpp", ".hh", ".hxx"].includes(ext);
parsed	Line 504: const parsed = path.parse(String(fileName "program"));
result	Line 511: const result = await runProcess(toolPath, ["--version"], { timeoutMs: 7000 });
version	Line 512: const version = [result.stdout, result.stderr]

Item	Explanation
elapsedSeconds	Line 524: const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
status	Line 525: const status = result.timedOut
output	Line 528: const output = [result.stdout, result.stderr].map((part) => String(part "").trimEnd()).filter(Boolean).join("\n");
raw	Line 533: const raw = [result.stdout, result.stderr].map((part) => String(part "").trimEnd()).filter(Boolean).join("\n");
lines	Line 534: const lines = raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean);
finishLines	Line 535: const finishLines = [];
signalLines	Line 536: const signalLines = [];
elapsedSeconds	Line 544: const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
status	Line 545: const status = result.timedOut
body	Line 548: const body = signalLines.length
suffix	Line 555: const suffix = signalLines.length && finishLines.length ? ` \n\${finishLines.join("\n")}` : "";
timeoutMs	Line 560: const timeoutMs = options.timeoutMs 20000;

JSON/YAML-style keys detected

Item	Explanation
.css	Line 19: ".css": "text/css",
.csv	Line 20: ".csv": "text/csv",
.html	Line 21: ".html": "text/html",
.ico	Line 22: ".ico": "image/x-icon",
.js	Line 23: ".js": "application/javascript",
.json	Line 24: ".json": "application/json",
.md	Line 25: ".md": "text/markdown",
.pdf	Line 26: ".pdf": "application/pdf",
.png	Line 27: ".png": "image/png",
.jpg	Line 28: ".jpg": "image/jpeg",
.jpeg	Line 29: ".jpeg": "image/jpeg",
.svg	Line 30: ".svg": "image/svg+xml",
.txt	Line 31: ".txt": "text/plain",
.webp	Line 32: ".webp": "image/webp",
.xml	Line 33: ".xml": "application/xml",
.xlsx	Line 34: ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
.zip	Line 35: ".zip": "application/zip"
g++	Line 153: "g++": [
clang++	Line 159: "clang++": ["clang++"],
X-Content-Type-Options	Line 220: "X-Content-Type-Options": "nosniff",
Referrer-Policy	Line 221: "Referrer-Policy": "strict-origin-when-cross-origin",
Permissions-Policy	Line 222: "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",
Cross-Origin-Opener-Policy	Line 223: "Cross-Origin-Opener-Policy": "same-origin",
Content-Type	Line 231: "Content-Type": "application/json",
Cache-Control	Line 232: "Cache-Control": "no-store, no-cache, must-revalidate"
c++	Line 262: "c++": "cpp",

Item	Explanation
Accept	Line 1386: "Accept": accept,
User-Agent	Line 1387: "User-Agent": "OMB-Portfolio-Builder-AI"
Authorization	Line 3314: "Authorization": `Bearer \${apiKey}`,
Content-Type	Line 3315: "Content-Type": "application/json"
Content-Type	Line 3621: "Content-Type": types[path.extname(filePath)] "application/octet-stream",
Cache-Control	Line 3622: "Cache-Control": "no-store, no-cache, must-revalidate"

Function-by-function detail

securityHeaders (server.mjs, lines 218-226)

securityHeaders part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 218-226 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 219: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

218: function securityHeaders(extra = {}) {
219:   return {
220:     "X-Content-Type-Options": "nosniff",
221:     "Referrer-Policy": "strict-origin-when-cross-origin",
222:     "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",
223:     "Cross-Origin-Opener-Policy": "same-origin",
224:     ...extra
225:   };
226: }

```

sendJson (server.mjs, lines 228-235)

sendJson part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 228-235 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references writeHead, securityHeaders, end, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

228: function sendJson(response, status, data) {
229:   response.writeHead(status, {
230:     ...securityHeaders(),
231:     "Content-Type": "application/json",
232:     "Cache-Control": "no-store, no-cache, must-revalidate"
233:   });
234:   response.end(JSON.stringify(data, null, 2));
235: }

```

clampText (server.mjs, lines 237-239)

clampText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 237-239 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 238: return String(value || "").slice(0, maxLength);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
237: function clampText(value, maxLength = 12000) {
238:   return String(value || "").slice(0, maxLength);
239: }
```

stripHtmlToText (server.mjs, lines 241-254)

stripHtmlToText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 241-254 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 242: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
241: function stripHtmlToText(value = "") {
242:   return String(value || "")
243:     .replace(/<script[\s\S]*?<\script>/gi, " ")
244:     .replace(/<style[\s\S]*?<\style>/gi, " ")
245:     .replace(/<[>]+>/g, " ")
246:     .replace(/&nbsp;/gi, " ")
247:     .replace(/&amp;/gi, "&")
248:     .replace(/&lt;/gi, "<")
249:     .replace(/&gt;/gi, ">")
250:     .replace(/&quot;/gi, "'")
251:     .replace(/&#39;/g, "'")
252:     .replace(/\\s+/g, " ")
253:     .trim();
254: }
```

normalizeCodeLanguage (server.mjs, lines 256-280)

normalizeCodeLanguage validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 256-280 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 279: return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");.

Important local values include clean at line 257; aliases at line 258. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
256: function normalizeCodeLanguage(value = "") {
257:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
258:   const aliases = {
259:     c: "c",
260:     cpp: "cpp",
261:     cplusplus: "cpp",
262:     "c++": "cpp",
263:     systemverilog: "systemverilog",
264:     sv: "systemverilog",
265:     verilog: "verilog",
266:     v: "verilog",
```

```

267:     ltspice: "ltspice",
268:     spice: "ltspice",
269:     cir: "ltspice",
270:     java: "java",
271:     javascript: "javascript",
272:     js: "javascript",
273:     node: "javascript",
274:     python: "python",
275:     py: "python",
276:     html: "html",
277:     htm: "html"
278: };
279: return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");
280: }

```

sourceLooksCpp (server.mjs, lines 282-286)

sourceLooksCpp part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 282-286 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 284: return
 /#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text) ||.

Important local values include text at line 283. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

282: function sourceLooksCpp(source = "") {
283:   const text = String(source || "");
284:   return
  /#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text)
  ||
  285:   /\b(std::|cout|cin|cerr|namespace|template\s*<|class\s+\w+|new\s+\w+|delete\s+|constexpr|nullptr|using
\s+namespace)\b/.test(text);
286: }

```

sourceLooksC (server.mjs, lines 288-292)

sourceLooksC part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 288-292 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 290: return
 /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||.

Important local values include text at line 289. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

288: function sourceLooksC(source = "") {
289:   const text = String(source || "");
290:   return /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||
291:   /\b(printf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text);
292: }

```

languageFromFileName (server.mjs, lines 294-302)

languageFromFileName part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 294-302 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `extname`, `toLowerCase`, `sourceLooksCpp`, `sourceLooksC`, `entries`, `find`, and `includes`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 297: `if (sourceLooksCpp(code)) return "cpp";`; line 298: `if (sourceLooksC(code)) return "c";`; line 299: `return "c";`; line 301: `return Object.entries(compileLanguageProfiles).find(([, profile]) => profile.extensions.includes(ext))?.[0] || ""`;

Important local values include `ext` at line 295. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
294: function languageFromFileName(fileName = "", code = "") {
295:   const ext = path.extname(String(fileName || "").toLowerCase());
296:   if (ext === ".h") {
297:     if (sourceLooksCpp(code)) return "cpp";
298:     if (sourceLooksC(code)) return "c";
299:     return "c";
300:   }
301:   return Object.entries(compileLanguageProfiles).find(([, profile]) =>
profile.extensions.includes(ext))?.[0] || "";
302: }
```

detectCodeLanguageFromSource (server.mjs, lines 304-319)

`detectCodeLanguageFromSource` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 304-319 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `languageFromFileName`, `b`, `sourceLooksCpp`, and `sourceLooksC`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 306: `if (byName) return byName;`; line 308: `if (/<V?[a-z][\s\S]*?>/i.test(source) || /<!doctype\s+html/i.test(source)) return "html";`; line 309: `if (/b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+lw+)\b/i.test(source)) return "systemverilog";`; line 310: `if (/b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/i.test(source)) return "verilog";`. There are 6 additional return statements in the function body.

Important local values include `byName` at line 305; `source` at line 307. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
304: function detectCodeLanguageFromSource(code = "", fileName = "") {
305:   const byName = languageFromFileName(fileName, code);
306:   if (byName) return byName;
307:   const source = String(code || "");
308:   if (/<V?[a-z][\s\S]*?>/i.test(source) || /<!doctype\s+html/i.test(source)) return "html";
309:   if (/b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|
class\s+lw+)\b/i.test(source)) return "systemverilog";
310:   if (/b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/i.test(source)) return
"verilog";
311:   if (/^s*(?:tran|ac|dc|op|model|subckt|ends|param)\b/im.test(source) ||
/bv\w+\s+\w+\s+\w+\s+(?:DC|SIN|PULSE)?/i.test(source)) return "ltspice";
312:   if (/b(def|elif|from\s+\w+\s+import|self|None|True|False)\b/i.test(source)) return "python";
313:   if (/b(public\s+class|static\s+void\s+main|System\.out)\b/i.test(source)) return "java";
314:   if (sourceLooksCpp(source) || sourceLooksC(source) ||
/b(#include|main\s*(|puts\s*(|fprintf|sizeof\s*(|)\b/i.test(source)) {
315:     return sourceLooksCpp(source) ? "cpp" : "c";
316:   }
317:   if (/b(const|let|var|function|=>|console\.log|document\.|window\.)\b/i.test(source)) return
"javascript";
318:   return "javascript";
319: }
```

safeCodeFileName (server.mjs, lines 321-329)

`safeCodeFileName` validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 321-329 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizeCodeLanguage`, `parse`, `safeSegment`, `extname`, `toLowerCase`, and `includes`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 328: `return `${safeName}${ext}``;

Important local values include profile at line 322; fallback at line 323; parsed at line 324; safeName at line 325; ext at line 326. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
321: function safeCodeFileName(value = "", language = "javascript") {
322:   const profile = compileLanguageProfiles[normalizeCodeLanguage(language)] ||
compileLanguageProfiles.javascript;
323:   const fallback = profile.defaultFile;
324:   const parsed = path.parse(String(value || fallback));
325:   const safeName = safeSegment(parsed.name, path.parse(fallback).name);
326:   let ext = String(parsed.ext || path.extname(fallback)).toLowerCase();
327:   if (!profile.extensions.includes(ext)) ext = path.extname(fallback);
328:   return `${safeName}${ext}`;
329: }
```

indentBraceCode (server.mjs, lines 331-345)

indentBraceCode part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 331-345 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, split, map, trim, max, repeat, and match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 333: return String(code || ""); line 338: if (!line) return ""; line 344: return formatted;.

Important local values include depth at line 332; line at line 337; formatted at line 340; opens at line 341; closes at line 342. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
331: function indentBraceCode(code = "") {
332:   let depth = 0;
333:   return String(code || "")
334:     .replace(/\r\n?/g, "\n")
335:     .split("\n")
336:     .map((rawLine) => {
337:       const line = rawLine.trim();
338:       if (!line) return "";
339:       if (/^[{}]\]/.test(line)) depth = Math.max(0, depth - 1);
340:       const formatted = `${" ".repeat(depth)}${line}`;
341:       const opens = (line.match(/[{}(]/g) || []).length;
342:       const closes = (line.match(/[})\])/g) || [].length;
343:       depth = Math.max(0, depth + opens - closes);
344:       return formatted;
345:     })
}
```

beautifyCode (server.mjs, lines 350-372)

beautifyCode part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 350-372 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, normalizeCodeLanguage, includes, indentBraceCode, split, map, trim, join, and trimEnd. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 354: return indentBraceCode(normalized); line 359: return normalized; line 366: return normalized.

Important local values include normalized at line 351; lang at line 352. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
350: function beautifyCode(code = "", language = "javascript") {
351:   const normalized = String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ");
352:   const lang = normalizeCodeLanguage(language);
353:   if (["c", "cpp", "java", "javascript", "verilog", "systemverilog"].includes(lang)) {
354:     return indentBraceCode(normalized)
355:       .replace(/[ \t]+$/gm, "")
356:       .replace(/\n{4,}/g, "\n\n\n");
357:   }
358:   if (lang === "html") {
```

```

359:     return normalized
360:       .replace(/>\s+</g, ">\n<")
361:       .split("\n")
362:       .map((line) => line.trim())
363:       .join("\n")
364:       .trim();
365:   }
366:   return normalized
367:     .split("\n")
368:     .map((line) => line.trimEnd())
369:     .join("\n")
370:     .replace(/\n{4,}/g, "\n\n\n")
371:     .trimEnd();
372: }

```

findExecutableUnder (server.mjs, lines 374-392)

findExecutableUnder part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 374-392 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pathExists, readdir, join, isFile, some, toLowerCase, and isDirectory. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 375: if (!folder || maxDepth < 0 || !(await pathExists(folder))) return "";; line 380: return "";; line 384: if (entry.isFile() && names.some((name) => entry.name.toLowerCase() === name.toLowerCase())) return fullPath;; line 389: if (found) return found;. There are 1 additional return statements in the function body.

Important local values include entries at line 376; fullPath at line 383; found at line 388. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

374: async function findExecutableUnder(folder, names = [], maxDepth = 5) {
375:   if (!folder || maxDepth < 0 || !(await pathExists(folder))) return "";
376:   let entries = [];
377:   try {
378:     entries = await readdir(folder, { withFileTypes: true });
379:   } catch {
380:     return "";
381:   }
382:   for (const entry of entries) {
383:     const fullPath = path.join(folder, entry.name);
384:     if (entry.isFile() && names.some((name) => entry.name.toLowerCase() === name.toLowerCase())) return
fullPath;
385:   }
386:   for (const entry of entries) {
387:     if (!entry.isDirectory()) continue;
388:     const found = await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1);
389:     if (found) return found;
390:   }
391:   return "";
392: }

```

findTool (server.mjs, lines 394-437)

This function locates external tools such as compilers, runtimes, Git, or simulators. It checks known paths and command availability, then caches the result so repeated compile/run actions do not keep searching the machine.

Compile Code depends on tools that may or may not be installed. The builder needs fast, clear tool detection so it can explain missing tools and avoid lag every time the user presses Compile or Run.

It calls or references has, get, filter, isAbsolute, pathExists, set, execFileAsync, split, map, trim, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 395: if (compileToolCache.has(toolName)) return compileToolCache.get(toolName);; line 401: return candidate;; line 412: return found;; line 426: return found || "";. There are 2 additional return statements in the function body.

Important local values include candidates at line 396; command at line 406; args at line 407; result at line 408; found at line 409; found at line 420; found at line 430. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

394: async function findTool(toolName) {
395:   if (compileToolCache.has(toolName)) return compileToolCache.get(toolName);
396:   const candidates = (compileToolCandidates[toolName] || [toolName]).filter(Boolean);
397:   for (const candidate of candidates) {

```

```

398:     if (path.isAbsolute(candidate)) {
399:       if (await pathExists(candidate)) {
400:         compileToolCache.set(toolName, candidate);
401:         return candidate;
402:       }
403:       continue;
404:     }
405:     try {
406:       const command = process.platform === "win32" ? "where" : "command";
407:       const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
408:       const result = await execFileAsync(command, args, { timeout: 5000, windowsHide: true });
409:       const found = String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean);
410:       if (found) {
411:         compileToolCache.set(toolName, found);
412:         return found;
413:       }
414:     } catch {
415:       // Try the next candidate.
416:     }
417:   }
418: }
419: if ([ "gcc", "g++" ].includes(toolName) && process.env.LOCALAPPDATA) {
420:   const found = await findExecutableUnder(
421:     path.join(process.env.LOCALAPPDATA, "Microsoft", "winGet", "Packages"),
422:     [ toolName === "gcc" ? "gcc.exe" : "g++.exe" ],
423:     7
424:   );
425:   compileToolCache.set(toolName, found || "");
426:   return found || "";
427: }
428:
429: if ([ "javac", "java" ].includes(toolName)) {
430:   const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [ `${toolName}.exe` ],
5);
431:   compileToolCache.set(toolName, found || "");
432:   return found || "";
433: }
434:
435: compileToolCache.set(toolName, "");
436: return "";
437: }

```

terminalLine (server.mjs, lines 439-441)

terminalLine part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 439-441 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 440: return [`\${label}`, String(text || "").trim()).filter(Boolean).join("\n");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

439: function terminalLine(label, text = "") {
440:   return [`${label}`, String(text || "").trim()).filter(Boolean).join("\n");
441: }

```

processTerminalText (server.mjs, lines 443-451)

processTerminalText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 443-451 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, toFixed, map, trimEnd, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 450: return [status, output || (result.ok ? "Completed without diagnostic output." : "No compiler output was returned.")].join("\n");.

Important local values include elapsedSeconds at line 444; elapsed at line 445; status at line 446; output at line 449. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
443: function processTerminalText(result = {}) {
444:   const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
445:   const elapsed = elapsedSeconds ? ` in ${elapsedSeconds}s` : "";
446:   const status = result.timedOut
447:     ? `Timed out after ${elapsedSeconds} ? ${elapsedSeconds}s` : "the configured timeout"`
448:     : `Exit code ${result.code ?? "unknown"}${elapsed}`;
449:   const output = [result.stdout, result.stderr].map((part) => String(part ||
""))
.trimEnd()).filter(Boolean).join("\n");
450:   return [status, output || (result.ok ? "Completed without diagnostic output." : "No compiler output was
returned.")]
.join("\n");
451: }
```

pathVariantsForReplacement (server.mjs, lines 453-461)

pathVariantsForReplacement part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 453-461 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references from, Set, normalize, replace, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 455: return Array.from(new Set([.

Important local values include clean at line 454. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
453: function pathVariantsForReplacement(value = "") {
454:   const clean = String(value || "");
455:   return Array.from(new Set([
456:     clean,
457:     path.normalize(clean),
458:     clean.replace(/\\\/g, "/"),
459:     path.normalize(clean).replace(/\\\/g, "/")
460:   ]
.filter(Boolean)));
461: }
```

replacePathReferences (server.mjs, lines 463-474)

replacePathReferences part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 463-474 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pathVariantsForReplacement, split, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 473: return output;

Important local values include output at line 464; from at line 466; to at line 467. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
463: function replacePathReferences(text = "", replacements = []) {
464:   let output = String(text || "");
465:   for (const replacement of replacements) {
466:     const from = replacement?.from || "";
467:     const to = replacement?.to || "";
468:     if (!from || !to) continue;
469:     for (const variant of pathVariantsForReplacement(from)) {
470:       output = output.split(variant).join(to);
471:     }
472:   }
473:   return output;
474: }
```

processTerminalTextWithPaths (server.mjs, lines 476-478)

processTerminalTextWithPaths part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 476-478 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `replacePathReferences`, and `processTerminalText`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 477: `return replacePathReferences(processTerminalText(result), replacements);`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
476: function processTerminalTextWithPaths(result = {}, replacements = []) {
477:   return replacePathReferences(processTerminalText(result), replacements);
478: }
```

cFamilyCompileProfile (server.mjs, lines 480-501)

`cFamilyCompileProfile` part of the compile code language/tool/build/run flow. In this file, it appears around lines 480-501 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `extname`, `toLowerCase`, and `includes`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 484: `return {`.

Important local values include `isCpp` at line 481; `ext` at line 482; `header` at line 483. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
480: function cFamilyCompileProfile(language = "c", fileName = "main.c") {
481:   const isCpp = language === "cpp";
482:   const ext = path.extname(String(fileName || "")).toLowerCase();
483:   const header = [".h", ".hpp", ".hh", ".hxx"].includes(ext);
484:   return {
485:     compiler: isCpp ? "g++" : "gcc",
486:     standard: isCpp ? "-std=c++20" : "-std=c17",
487:     languageFlag: isCpp ? "c++" : "c",
488:     label: isCpp ? "C++" : "C",
489:     header,
490:     flags: [
491:       isCpp ? "-std=c++20" : "-std=c17",
492:       "-Wall",
493:       "-Wextra",
494:       "-Wpedantic",
495:       "-Wshadow",
496:       "-O0",
497:       "-g3",
498:       "-fdiagnostics-color=never"
499:     ]
500:   };
501: }
```

cFamilyBinaryName (server.mjs, lines 503-506)

`cFamilyBinaryName` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 503-506 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `parse`, and `safeSegment`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 505: `return `${safeSegment(parsed.name, "program")}.exe`;`.

Important local values include `parsed` at line 504. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
503: function cFamilyBinaryName(fileName = "program") {
504:   const parsed = path.parse(String(fileName || "program"));
505:   return `${safeSegment(parsed.name, "program")}.exe`;
```

506: }

toolVersionLine (server.mjs, lines 508-521)

toolVersionLine part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 508-521 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references has, get, runProcess, map, trim, filter, join, split, find, basename, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 509: if (!toolPath) return ""; line 510: if (compileToolVersionCache.has(toolPath)) return compileToolVersionCache.get(toolPath); line 520: return version;

Important local values include result at line 511; version at line 512. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
508: async function toolVersionLine(toolPath = "") {
509:   if (!toolPath) return "";
510:   if (compileToolVersionCache.has(toolPath)) return compileToolVersionCache.get(toolPath);
511:   const result = await runProcess(toolPath, ["--version"], { timeoutMs: 7000 });
512:   const version = [result.stdout, result.stderr]
513:     .map((part) => String(part || "").trim())
514:     .filter(Boolean)
515:     .join("\n")
516:     .split(/\r?\n/)
517:     .map((line) => line.trim())
518:     .find(Boolean) || path.basename(toolPath);
519:   compileToolVersionCache.set(toolPath, version);
520:   return version;
521: }
```

cFamilyRunOutput (server.mjs, lines 523-530)

cFamilyRunOutput part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 523-530 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, toFixed, map, trimEnd, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 529: return [status, output || (result.ok ? "Program completed without output." : "No program output was returned.")]

Important local values include elapsedSeconds at line 524; status at line 525; output at line 528. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
523: function cFamilyRunOutput(result = {}) {
524:   const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
525:   const status = result.timedOut
526:     ? `Program timed out after ${elapsedSeconds} s : "the configured timeout"`
527:     : `Program exit code ${result.code ?? "unknown"}${elapsedSeconds} s : ""`;
528:   const output = [result.stdout, result.stderr].map((part) => String(part || ""))
529:     .trimEnd().filter(Boolean).join("\n");
529:   return [status, output || (result.ok ? "Program completed without output." : "No program output was returned.")]
530: }
```

cleanHdlSimulationOutput (server.mjs, lines 532-557)

cleanHdlSimulationOutput part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 532-557 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, trimEnd, filter, join, split, push, replace, isFinite, and toFixed. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 556: return `\${status}\n\${body}\${suffix}`;

Important local values include raw at line 533; lines at line 534; finishLines at line 535; signalLines at line 536; elapsedSeconds at line 544; status at line 545; body at line 548; suffix at line 555. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
532: function cleanHdlSimulationOutput(result = {}) {
533:   const raw = [result.stdout, result.stderr].map((part) => String(part ||
"").trimEnd()).filter(Boolean).join("\n");
534:   const lines = raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean);
535:   const finishLines = [];
536:   const signalLines = [];
537:   for (const line of lines) {
538:     if (/^$finish called at/i.test(line)) {
539:       finishLines.push(line.replace(/^.*?:\d+:\s*/, ""));
540:     } else {
541:       signalLines.push(line);
542:     }
543:   }
544:   const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
545:   const status = result.timedOut
546:     ? `Simulation timed out after ${elapsedSeconds} s` : "the configured timeout"
547:     : `Simulation exit code ${result.code ?? "unknown"}${elapsedSeconds} s` :
"";
548:   const body = signalLines.length
549:     ? signalLines.join("\n")
550:     : finishLines.length
551:     ? finishLines.join("\n")
552:     : result.ok
553:     ? "Simulation completed without printed output."
554:     : "No simulator output was returned.";
555:   const suffix = signalLines.length && finishLines.length ? `\n${finishLines.join("\n")}` : "";
556:   return `${status}\n${body}${suffix}`;
557: }
```

runProcess (server.mjs, lines 559-599)

This is the backend's controlled process runner. It launches a command with arguments, captures stdout and stderr, enforces a timeout, records elapsed time, and returns a structured result to the caller.

Git commands, compilers, simulators, and installers all run outside Node. This wrapper makes those risky external calls predictable and lets the UI show terminal output instead of freezing or silently failing.

It calls or references now, spawn, execFile, kill, on, toString, call, write, end, resolve, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 562: return new Promise((resolve) => {.

Important local values include timeoutMs at line 560; cwd at line 561; startedAt at line 563; child at line 564; stdout at line 570; stderr at line 571; timedOut at line 572; timer at line 573. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
559: async function runProcess(command, args = [], options = {}) {
560:   const timeoutMs = options.timeoutMs || 20000;
561:   const cwd = options.cwd || compileRoot;
562:   return new Promise((resolve) => {
563:     const startedAt = Date.now();
564:     const child = spawn(command, args, {
565:       cwd,
566:       env: { ...process.env, ...(options.env || {}) },
567:       shell: false,
568:       windowsHide: true
569:     });
570:     let stdout = "";
571:     let stderr = "";
572:     let timedOut = false;
573:     const timer = setTimeout(() => {
574:       timedOut = true;
575:       if (process.platform === "win32" && child.pid) {
576:         execFile("taskkill", ["/pid", String(child.pid), "/T", "/F"], { windowsHide: true }, () => {});
577:       }
578:       child.kill("SIGKILL");
579:     }, timeoutMs);
580:     child.stdout?.on("data", (chunk) => {
581:       stdout += chunk.toString();
582:     });
583:     child.stderr?.on("data", (chunk) => {
584:       stderr += chunk.toString();
585:     });
```

```

586:     if (Object.prototype.hasOwnProperty.call(options, "input")) {
587:       child.stdin?.write(String(options.input || ""));
588:     }
589:     child.stdin?.end();
590:     child.on("error", (error) => {
591:       clearTimeout(timer);
592:       resolve({ ok: false, code: -1, stdout, stderr: `${stderr}\n${error.message}`.trim(), timedOut,
elapsedMs: Date.now() - startedAt });
593:     });
594:     child.on("close", (code) => {
595:       clearTimeout(timer);
596:       resolve({ ok: !timedOut && code === 0, code, stdout, stderr, timedOut, elapsedMs: Date.now() -
startedAt });
597:     });
598:   });
599: }

```

compileToolStatus (server.mjs, lines 601-620)

compileToolStatus part of the compile code language/tool/build/run flow. In this file, it appears around lines 601-620 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, values, flatMap, findTool, entries, every, fromEntries, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 619: return { compileRoot, languages, tools };

Important local values include tools at line 602; uniqueTools at line 603; languages at line 607; required at line 609. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

601: async function compileToolStatus() {
602:   const tools = {};
603:   const uniqueTools = [...new Set(Object.values(compileLanguageProfiles).flatMap((profile) =>
profile.primaryTools))];
604:   for (const tool of uniqueTools) {
605:     tools[tool] = await findTool(tool);
606:   }
607:   const languages = {};
608:   for (const [id, profile] of Object.entries(compileLanguageProfiles)) {
609:     const required = profile.primaryTools;
610:     languages[id] = {
611:       label: profile.label,
612:       defaultFile: profile.defaultFile,
613:       extensions: profile.extensions,
614:       ready: required.every((tool) => tools[tool]),
615:       tools: Object.fromEntries(required.map((tool) => [tool, tools[tool] || "missing"])),
616:       installHints: profile.winget || []
617:     };
618:   }
619:   return { compileRoot, languages, tools };
620: }

```

isHdlLanguage (server.mjs, lines 622-624)

isHdlLanguage part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 622-624 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 623: return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

622: function isHdlLanguage(language = "") {
623:   return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));
624: }

```

inferCompileFileRole (server.mjs, lines 626-633)

inferCompileFileRole part of the compile code language/tool/build/run flow. In this file, it appears around lines 626-633 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isHdlLanguage, toLowerCase, b, and tb. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 627: if (!isHdlLanguage(language)) return "source";; line 630: return "testbench";; line 632: return "design";.

Important local values include haystack at line 628. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
626: function inferCompileFileRole(fileName = "", code = "", language = "") {
627:   if (!isHdlLanguage(language)) return "source";
628:   const haystack = `${fileName}\n${code}`.toLowerCase();
629:   if (/\\b(tb|testbench)\\b|(^|[-])tb([-]|\\.)|test[-]?bench|\\$dumpfile|\\$dumpvars|module\\s+tb[_a-z0-
9]*\\/i.test(haystack)) {
630:     return "testbench";
631:   }
632:   return "design";
633: }
```

normalizeCompileFileRole (server.mjs, lines 635-639)

normalizeCompileFileRole part of the compile code language/tool/build/run flow. In this file, it appears around lines 635-639 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isHdlLanguage, trim, toLowerCase, replace, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 636: if (!isHdlLanguage(language)) return "source";; line 638: return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";.

Important local values include clean at line 637. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
635: function normalizeCompileFileRole(value = "", language = "") {
636:   if (!isHdlLanguage(language)) return "source";
637:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
638:   return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";
639: }
```

saveCompileSource (server.mjs, lines 641-662)

saveCompileSource persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 641-662 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodeLanguage, detectCodeLanguageFromSource, safeSegment, safeCodeFileName, parse, resolveInsideCompileRoot, mkdir, writeFile, clampText, normalizeCompileFileRole, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 661: return metadata;.

Important local values include lang at line 642; projectFolder at line 643; codeFileName at line 644; fileFolder at line 645; folder at line 646; sourcePath at line 647; metadata at line 651. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
641: async function saveCompileSource({ projectId, fileId, title, fileName, language, role = "", code, stdin =
"" }) {
642:   const lang = normalizeCodeLanguage(language || detectCodeLanguageFromSource(code, fileName));
643:   const projectFolder = safeSegment(projectId, "project");
644:   const codeFileName = safeCodeFileName(fileName, lang);
645:   const fileFolder = safeSegment(fileId || path.parse(codeFileName).name, path.parse(codeFileName).name);
646:   const folder = resolveInsideCompileRoot(projectFolder, fileFolder);
647:   const sourcePath = resolveInsideCompileRoot(projectFolder, fileFolder, codeFileName);
```

```

648:   await mkdir(folder, { recursive: true });
649:   await writeFile(sourcePath, String(code || ""), "utf8");
650:   if (stdin) await writeFile(resolveInsideCompileRoot(projectFolder, fileFolder, "stdin.txt"),
String(stdin), "utf8");
651:   const metadata = {
652:     id: fileFolder,
653:     title: clampText(title || codeFileName, 180),
654:     fileName: codeFileName,
655:     language: lang,
656:     role: normalizeCompileFileRole(role || inferCompileFileRole(codeFileName, code, lang), lang),
657:     sourcePath,
658:     savedAt: new Date().toISOString()
659:   };
660:   await writeFile(resolveInsideCompileRoot(projectFolder, fileFolder, "compile-meta.json"),
`${JSON.stringify(metadata, null, 2)}\n`, "utf8");
661:   return metadata;
662: }

```

javaMainClassName (server.mjs, lines 664-670)

javaMainClassName part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 664-670 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references match, and parse. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 667: if (publicMatch) return publicMatch[1]; line 669: return classMatch?.[1] || path.parse(fileName).name || "Main";.

Important local values include source at line 665; publicMatch at line 666; classMatch at line 668. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

664: function javaMainClassName(code = "", fileName = "Main.java") {
665:   const source = String(code || "");
666:   const publicMatch = source.match(/bpublic\s+class\s+([A-Za-z_$][\w$]*)/);
667:   if (publicMatch) return publicMatch[1];
668:   const classMatch = source.match(/bclass\s+([A-Za-z_$][\w$]*)/);
669:   return classMatch?.[1] || path.parse(fileName).name || "Main";
670: }

```

compileCacheKey (server.mjs, lines 672-677)

compileCacheKey part of the compile code language/tool/build/run flow. In this file, it appears around lines 672-677 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createHash, update, stringify, digest, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 673: return createHash("sha256").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

672: function compileCacheKey(parts = {}) {
673:   return createHash("sha256")
674:     .update(JSON.stringify(parts))
675:     .digest("hex")
676:     .slice(0, 24);
677: }

```

compileCacheDirectory (server.mjs, lines 679-681)

compileCacheDirectory part of the compile code language/tool/build/run flow. In this file, it appears around lines 679-681 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `resolveInsideCompileRoot`, and `safeSegment`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 680: `return resolveInsideCompileRoot(safeSegment(projectId, "project"), ".build-cache", safeSegment(language, "language"), key);`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
679: function compileCacheDirectory(projectId, language, key) {
680:   return resolveInsideCompileRoot(safeSegment(projectId, "project"), ".build-cache",
safeSegment(language, "language"), key);
681: }
```

cachedBuildLine (server.mjs, lines 683-685)

`cachedBuildLine` reads, writes, or validates cached state. In this file, it appears around lines 683-685 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 684: `return `${type} cache hit: using existing artifact\n${outputPath}`;`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
683: function cachedBuildLine(type, outputPath) {
684:   return `${type} cache hit: using existing artifact\n${outputPath}`;
685: }
```

hdlModuleNames (server.mjs, lines 687-689)

`hdlModuleNames` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 687-689 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `matchAll`, and `map`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 688: `return [...String(code || "").matchAll(/bmodule\s+([A-Za-z_$][\w$]*)/g)].map((match) => match[1]);`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
687: function hdlModuleNames(code = "") {
688:   return [...String(code || "").matchAll(/bmodule\s+([A-Za-z_$][\w$]*)/g)].map((match) => match[1]);
689: }
```

hdlHasWaveDump (server.mjs, lines 691-694)

`hdlHasWaveDump` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 691-694 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 693: `return /^$dumpfile\s*(/\.test(source) && /^$dumpvars\s*(/\.test(source);`.

Important local values include source at line 692. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
691: function hdlHasWaveDump(code = "") {
692:   const source = String(code || "");
693:   return /\$dumpfile\s*(\/.test(source) && /\$dumpvars\s*(\/.test(source);
694: }
```

hdlFilesFromPayload (server.mjs, lines 696-727)

hdlFilesFromPayload part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 696-727 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, Map, normalizeCodeLanguage, detectCodeLanguageFromSource, isHdlLanguage, trim, safeCodeFileName, safeSegment, parse, set, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 702: if (!isHdlLanguage(language) || !code.trim()) return;; line 723: return [...byId.values()].sort((a, b) => {; line 724: if (a.role === b.role) return a.fileName.localeCompare(b.fileName);; line 725: return a.role === "testbench" ? 1 : -1;;

Important local values include incoming at line 697; byId at line 698; addFile at line 699; code at line 700; language at line 701; fileName at line 703; id at line 704. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
696: function hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog") {
697:   const incoming = Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : [];
698:   const byId = new Map();
699:   const addFile = (item = {}, fallbackId = "") => {
700:     const code = String(item.code || "");
701:     const language = normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code,
item.fileName));
702:     if (!isHdlLanguage(language) || !code.trim()) return;
703:     const fileName = safeCodeFileName(item.fileName || activeFileName ||
compileLanguageProfiles[language].defaultFile, language);
704:     const id = safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name);
705:     byId.set(id, {
706:       id,
707:       title: clampText(item.title || fileName, 180),
708:       fileName,
709:       language,
710:       role: normalizeCompileFileRole(item.role || inferCompileFileRole(fileName, code, language),
language),
711:       code
712:     });
713:   };
714:   incoming.forEach((item, index) => addFile(item, `hdl-${index + 1}`));
715:   addFile({
716:     id: payload.fileId,
717:     title: payload.title,
718:     fileName: activeFileName,
719:     language: activeLanguage,
720:     role: payload.role,
721:     code: payload.code
722:   }, "active");
723:   return [...byId.values()].sort((a, b) => {
724:     if (a.role === b.role) return a.fileName.localeCompare(b.fileName);
725:     return a.role === "testbench" ? 1 : -1;
726:   });
727: }
```

compileActionFromPayload (server.mjs, lines 729-733)

compileActionFromPayload part of the compile code language/tool/build/run flow. In this file, it appears around lines 729-733 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, replace, includes, and isHdlLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 731: if (!["compile", "build", "run", "simulate"].includes(clean)) return clean;; line 732: return isHdlLanguage(language) ? "simulate" : "run";.

Important local values include `clean` at line 730. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
729: function compileActionFromPayload(value = "", language = "") {
730:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "-");
731:   if (["compile", "build", "run", "simulate"].includes(clean)) return clean;
732:   return isHdlLanguage(language) ? "simulate" : "run";
733: }
```

compileActionLabel (server.mjs, lines 735-740)

`compileActionLabel` part of the compile code language/tool/build/run flow. In this file, it appears around lines 735-740 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `isHdlLanguage`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 736: `if (action === "compile") return "Compile";`; line 737: `if (action === "build") return "Build project";`; line 738: `if (action === "simulate") return "Simulate";`; line 739: `return isHdlLanguage(language) ? "Simulate" : "Run";`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
735: function compileActionLabel(action = "run", language = "") {
736:   if (action === "compile") return "Compile";
737:   if (action === "build") return "Build project";
738:   if (action === "simulate") return "Simulate";
739:   return isHdlLanguage(language) ? "Simulate" : "Run";
740: }
```

compileWorkspaceFilesFromPayload (server.mjs, lines 742-770)

`compileWorkspaceFilesFromPayload` part of the compile code language/tool/build/run flow. In this file, it appears around lines 742-770 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `isArray`, `Map`, `trim`, `normalizeCodeLanguage`, `detectCodeLanguageFromSource`, `safeCodeFileName`, `safeSegment`, `parse`, `set`, `clampText`, and 7 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 747: `if (!code.trim()) return;`; line 769: `return [...byId.values()].sort((a, b) => a.fileName.localeCompare(b.fileName));`.

Important local values include `incoming` at line 743; `byId` at line 744; `addFile` at line 745; `code` at line 746; `language` at line 748; `fileName` at line 749; `id` at line 750. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
742: function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage =
"javascript") {
743:   const incoming = Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : [];
744:   const byId = new Map();
745:   const addFile = (item = {}, fallbackId = "") => {
746:     const code = String(item.code || "");
747:     if (!code.trim()) return;
748:     const language = normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code,
item.fileName));
749:     const fileName = safeCodeFileName(item.fileName || activeFileName ||
compileLanguageProfiles[language]?.defaultFile || "source.txt", language);
750:     const id = safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name);
751:     byId.set(id, {
752:       id,
753:       title: clampText(item.title || fileName, 180),
754:       fileName,
755:       language,
756:       role: normalizeCompileFileRole(item.role || inferCompileFileRole(fileName, code, language),
language),
757:       code
758:     });
759:   };
760:   incoming.forEach((item, index) => addFile(item, `workspace-${index + 1}`));
761:   addFile({
```

```

762:     id: payload.fileId,
763:     title: payload.title,
764:     fileName: activeFileName,
765:     language: activeLanguage,
766:     role: payload.role,
767:     code: payload.code
768:   }, "active");
769:   return [...byId.values()].sort((a, b) => a.fileName.localeCompare(b.fileName));
770: }

```

writeCompileWorkspaceSources (server.mjs, lines 772-788)

writeCompileWorkspaceSources persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 772-788 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mkdir, Map, includes, normalizeCodeLanguage, extname, toLowerCase, parse, get, set, join, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 787: return written;.

Important local values include seen at line 774; written at line 775; parsed at line 779; count at line 780; uniqueName at line 782; sourcePath at line 783. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

772: async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {}) {
773:   await mkdir(targetDir, { recursive: true });
774:   const seen = new Map();
775:   const written = [];
776:   for (const file of files) {
777:     if (options.languages?.length && !options.languages.includes(normalizeCodeLanguage(file.language)))
continue;
778:     if (options.extensions?.length &&
!options.extensions.includes(path.extname(file.fileName).toLowerCase())) continue;
779:     const parsed = path.parse(file.fileName);
780:     const count = seen.get(file.fileName) || 0;
781:     seen.set(file.fileName, count + 1);
782:     const uniqueName = count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName;
783:     const sourcePath = path.join(targetDir, uniqueName);
784:     await writeFile(sourcePath, file.code, "utf8");
785:     written.push({ ...file, uniqueName, sourcePath });
786:   }
787:   return written;
788: }

```

sourceDisplayName (server.mjs, lines 790-792)

sourceDisplayName part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 790-792 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references basename. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 791: return file.uniqueName || file.fileName || path.basename(file.sourcePath || "source");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

790: function sourceDisplayName(file = {}) {
791:   return file.uniqueName || file.fileName || path.basename(file.sourcePath || "source");
792: }

```

cFamilyWorkspaceSources (server.mjs, lines 794-803)

cFamilyWorkspaceSources part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 794-803 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, filter, has, extname, toLowerCase, some, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 802: return selected;.

Important local values include exts at line 795; selected at line 798. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
794: function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "") {
795:   const exts = language === "cpp"
796:     ? new Set([".cpp", ".cc", ".cxx", ".c++", ".hpp", ".hh", ".hxx", ".h"])
797:     : new Set([".c", ".h"]);
798:   const selected = files.filter((file) => exts.has(path.extname(file.fileName).toLowerCase()));
799:   if (!selected.some((file) => file.fileName === activeFileName)) {
800:     selected.push(...files.filter((file) => file.fileName === activeFileName));
801:   }
802:   return selected;
803: }
```

writeHdlSimulationSources (server.mjs, lines 805-821)

writeHdlSimulationSources persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 805-821 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join, rm, mkdir, Map, parse, get, set, writeFile, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 820: return written;.

Important local values include sourceDir at line 806; seen at line 809; written at line 810; parsed at line 812; count at line 813; uniqueName at line 815; sourcePath at line 816. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
805: async function writeHdlSimulationSources(files = [], cacheDir = "") {
806:   const sourceDir = path.join(cacheDir, "sources");
807:   await rm(sourceDir, { recursive: true, force: true });
808:   await mkdir(sourceDir, { recursive: true });
809:   const seen = new Map();
810:   const written = [];
811:   for (const file of files) {
812:     const parsed = path.parse(file.fileName);
813:     const count = seen.get(file.fileName) || 0;
814:     seen.set(file.fileName, count + 1);
815:     const uniqueName = count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName;
816:     const sourcePath = path.join(sourceDir, uniqueName);
817:     await writeFile(sourcePath, file.code, "utf8");
818:     written.push({ ...file, uniqueName, sourcePath });
819:   }
820:   return written;
821: }
```

findFilesByExtension (server.mjs, lines 823-836)

findFilesByExtension part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 823-836 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pathExists, readdir, join, isDirectory, push, isFile, toLowerCase, and endsWith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 824: if (!folder || maxDepth < 0 || !(await pathExists(folder))) return []; line 835: return matches;.

Important local values include entries at line 825; matches at line 826; fullPath at line 828. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

823: async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3) {
824:   if (!folder || maxDepth < 0 || !(await pathExists(folder))) return [];
825:   const entries = await readdir(folder, { withFileTypes: true }).catch(() => []);
826:   const matches = [];
827:   for (const entry of entries) {
828:     const fullPath = path.join(folder, entry.name);
829:     if (entry.isDirectory()) {
830:       matches.push(...await findFilesByExtension(fullPath, extension, maxDepth - 1));
831:     } else if (entry.isFile() && entry.name.toLowerCase().endsWith(extension.toLowerCase())) {
832:       matches.push(fullPath);
833:     }
834:   }
835:   return matches;
836: }

```

normalizeVcdValue (server.mjs, lines 838-844)

normalizeVcdValue validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 838-844 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, slice, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 840: if (!clean) return "x"; line 841: if (/^[01xz]\$/i.test(clean)) return clean.toLowerCase(); line 842: if (/^[br][01xz_]+\$/i.test(clean)) return clean.slice(1).replace(/_/g, "").toLowerCase(); line 843: return clean.toLowerCase();.

Important local values include clean at line 839. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

838: function normalizeVcdValue(raw = "") {
839:   const clean = String(raw || "").trim();
840:   if (!clean) return "x";
841:   if (/^[01xz]$/i.test(clean)) return clean.toLowerCase();
842:   if (/^[br][01xz_]+$/i.test(clean)) return clean.slice(1).replace(/_/g, "").toLowerCase();
843:   return clean.toLowerCase();
844: }

```

parseVcdScopeText (server.mjs, lines 846-933)

This function parses VCD waveform text emitted by HDL simulations. It reads signal definitions, time markers, value changes, scopes, and symbols, then converts the raw waveform file into structured signal data.

The scope UI cannot draw from raw VCD text directly. This parser is what lets a SystemVerilog or Verilog simulation become a visible timing waveform in the builder.

It calls or references split, Map, trim, startsWith, replace, push, includes, match, pop, has, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 927: return {.

Important local values include lines at line 847; scopes at line 848; signalsByCode at line 849; timeScaleParts at line 850; inTimescale at line 851; time at line 852; maxTime at line 853; definitionDone at line 854; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

846: function parseVcdScopeText(text = "", source = "waveform.vcd") {
847:   const lines = String(text || "").split(/\r?\n/);
848:   const scopes = [];
849:   const signalsByCode = new Map();
850:   const timeScaleParts = [];
851:   let inTimescale = false;
852:   let time = 0;
853:   let maxTime = 0;
854:   let definitionDone = false;
855:   for (const rawLine of lines) {
856:     const line = rawLine.trim();
857:     if (!line) continue;
858:     if (line.startsWith("$timescale")) {
859:       inTimescale = true;
860:       const inline = line.replace("$timescale", "").replace("$end", "").trim();
861:       if (inline) timeScaleParts.push(inline);
862:       continue;
863:     }
864:     if (inTimescale) {
865:       if (line === "$end") {

```

```

866:         inTimescale = false;
867:     } else {
868:         timeScaleParts.push(line.replace("$end", "").trim());
869:         if (line.includes("$end")) inTimescale = false;
870:     }
871:     continue;
872: }
873: const scopeMatch = line.match(/^\$scope\$s+\$s+(.?)\$s+\$end$/);
874: if (scopeMatch) {
875:     scopes.push(scopeMatch[1]);
876:     continue;
877: }
878: if (/^\$supscope\$b/.test(line)) {
879:     scopes.pop();
880:     continue;
881: }
882: const varMatch = line.match(/^\$var\$s+\$s+(\d+)\$s+(\$s+)\$s+(.?)\$s+\$end$/);
883: if (varMatch) {
884:     const [, width, code, reference] = varMatch;
885:     if (!signalsByCode.has(code) && signalsByCode.size < 64) {
886:         const cleanReference = reference.replace(/\$s+\[^\$]\$s+/, "");
887:         signalsByCode.set(code, {
888:             code,
889:             name: [...scopes, cleanReference].filter(Boolean).join("."),
890:             reference: cleanReference,
891:             width: Number(width) || 1,
892:             changes: []
893:         });
894:     }
895:     continue;
896: }
897: if (line.startsWith("$enddefinitions")) {
898:     definitionDone = true;
899:     continue;
900: }
901: if (!definitionDone) continue;
902: if (line.startsWith("#")) {
903:     time = Number(line.slice(1)) || 0;
904:     maxTime = Math.max(maxTime, time);
905:     continue;
906: }
907: let code = "";
908: let value = "";
909: const vectorMatch = line.match(/^\([br\][01xz_]+\)\$s+(\$s+)\$/i);
910: if (vectorMatch) {
911:     value = normalizeVcdValue(vectorMatch[1]);
912:     code = vectorMatch[2];
913: } else if (/^\[01xz]/i.test(line)) {
914:     value = normalizeVcdValue(line[0]);
915:     code = line.slice(1).trim();
916: }
917: const signal = signalsByCode.get(code);
918: if (!signal || !value) continue;
919: const previous = signal.changes[signal.changes.length - 1];
920: if (!previous || previous.value !== value || previous.time !== time) {
921:     if (signal.changes.length < 600) signal.changes.push({ time, value });
922: }
923: }
924: const signals = [...signalsByCode.values()]
925:     .filter((signal) => signal.changes.length)
926:     .slice(0, 32);
927: return {
928:     source: path.basename(source),
929:     timeScale: timeScaleParts.join(" ").replace(/\$s+/g, " ").trim() || "ticks",
930:     maxTime,
931:     signals
932: };
933: }

```

readHdlWaveform (server.mjs, lines 935-942)

This function searches the compile cache for generated VCD waveform files, selects an appropriate waveform, and runs it through the VCD parser.

A simulator may generate files in a build directory rather than returning all useful information through stdout. This function lets the builder recover waveform evidence after simulation finishes.

It calls or references findFilesByExtension, sort, readFile, slice, and parseVcdScopeText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 937: if (!vcdFiles.length) return null;; line 941: return parseVcdScopeText(limited, selected);.

Important local values include vcdFiles at line 936; selected at line 938; text at line 939; limited at line 940. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

935: async function readHdlWaveform(cacheDir = "") {
936:   const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);
937:   if (!vcdFiles.length) return null;
938:   const selected = vcdFiles.sort((a, b) => a.length - b.length)[0];
939:   const text = await readFile(selected, "utf8");
940:   const limited = text.length > 6_000_000 ? text.slice(0, 6_000_000) : text;
941:   return parseVcdScopeText(limited, selected);
942: }

```

clearHdlWaveforms (server.mjs, lines 944-947)

clearHdlWaveforms part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 944-947 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references findFilesByExtension, all, map, and rm. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include vcdFiles at line 945. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

944: async function clearHdlWaveforms(cacheDir = "") {
945:   const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);
946:   await Promise.all(vcdFiles.map((filePath) => rm(filePath, { force: true })).catch(() => {}));
947: }

```

compileAndRunCode (server.mjs, lines 949-1319)

This function is the core Compile Code engine. It saves or gathers workspace files, detects the requested language/action, checks required tools, compiles when needed, runs when requested, handles HDL simulation/testbench requirements, reads generated waveform files, and returns terminal output plus metadata.

It turns the builder's code workspace into a small IDE. Keeping this logic in one backend function lets the frontend stay focused on editing while the backend owns language-specific build behavior.

It calls or references trim, toLowerCase, normalizeCodeLanguage, detectCodeLanguageFromSource, safeCodeFileName, saveCompileSource, safeSegment, resolveInsideCompileRoot, compileActionFromPayload, compileWorkspaceFilesFromPayload, and 20 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 966: if (runDir && runSourcePath) return { runDir, runSourcePath, writtenSources: runWorkspaceSources }; line 990: return { runDir, runSourcePath, writtenSources: runWorkspaceSources }; line 1003: return { found, missingTools }; line 1010: return { }. There are 8 additional return statements in the function body.

Important local values include requestedLanguage at line 950; language at line 951; profile at line 954; fileName at line 955; saved at line 956; projectFolder at line 957; sourcePath at line 958; sourceCode at line 959; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

949: async function compileAndRunCode(payload = {}) {
950:   const requestedLanguage = String(payload.language || "").trim().toLowerCase();
951:   const language = requestedLanguage && requestedLanguage !== "auto"
952:     ? normalizeCodeLanguage(requestedLanguage)
953:     : detectCodeLanguageFromSource(payload.code, payload.fileName);
954:   const profile = compileLanguageProfiles[language] || compileLanguageProfiles.javascript;
955:   const fileName = safeCodeFileName(payload.fileName, language);
956:   const saved = await saveCompileSource({ ...payload, language, fileName });
957:   const projectFolder = safeSegment(payload.projectId, "project");
958:   const sourcePath = resolveInsideCompileRoot(projectFolder, safeSegment(saved.id), saved.fileName);
959:   const sourceCode = String(payload.code || "");
960:   const action = compileActionFromPayload(payload.action, language);
961:   const allWorkspaceFiles = compileWorkspaceFilesFromPayload(payload, saved.fileName, language);
962:   let runDir = "";
963:   let runSourcePath = "";
964:   let runWorkspaceSources = [];
965:   const ensureRunSource = async (options = {}) => {
966:     if (runDir && runSourcePath) return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
967:     const runId = `${Date.now()}-${Math.random().toString(16).slice(2)}`;
968:     runDir = resolveInsideCompileRoot(projectFolder, ".runs", runId);
969:     await mkdir(runDir, { recursive: true });
970:     const filesToWrite = Array.isArray(options.files) && options.files.length ? options.files :
allWorkspaceFiles;

```

```

971:     runWorkspaceSources = await writeCompileWorkspaceSources(filesToWrite, runDir, {
972:       languages: options.languages,
973:       extensions: options.extensions
974:     });
975:     const active = runWorkspaceSources.find((file) => file.id === saved.id || file.fileName ===
saved.fileName);
976:     runSourcePath = active?.sourcePath || path.join(runDir, saved.fileName);
977:     if (!active) {
978:       await cp(sourcePath, runSourcePath, { force: true });
979:       runWorkspaceSources.push({
980:         id: saved.id,
981:         title: saved.title,
982:         fileName: saved.fileName,
983:         language,
984:         role: saved.role,
985:         code: sourceCode,
986:         uniqueName: saved.fileName,
987:         sourcePath: runSourcePath
988:       });
989:     }
990:     return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
991:   };
992:   const stdin = String(payload.stdin || "");
993:   const forceRebuild = payload.forceRebuild === true;
994:   const terminal = [];
995:   terminal.push(`${compileActionLabel(action, language)} request for ${saved.fileName}`);
996:   const append = (label, result) => {
997:     terminal.push(terminalLine(label, processTerminalText(result)));
998:   };
999:   const missing = async (tools) => {
1000:     const found = {};
1001:     for (const tool of tools) found[tool] = await findTool(tool);
1002:     const missingTools = tools.filter((tool) => !found[tool]);
1003:     return { found, missingTools };
1004:   };
1005:
1006:   if (language === "html") {
1007:     const openTags = (String(payload.code || "").match(/<[a-z][\w:-]*\b[^>*/>/gi) || []).length;
1008:     const closeTags = (String(payload.code || "").match(/<\/[a-z][\w:-]*>/gi) || []).length;
1009:     const ok = openTags >= closeTags;
1010:     return {
1011:       ok,
1012:       language,
1013:       saved,
1014:       terminal: [
1015:         "HTML validation pass",
1016:         ok ? "No obvious tag-balance issue was detected." : "The file may have more closing tags than
opening tags."
1017:       ],
1018:       `Saved source: ${saved.sourcePath}`
1019:     }.join("\n");
1020:   }
1021:
1022:   const stdinOptions = stdin ? { input: stdin } : {};
1023:   let ok = false;
1024:
1025:   if (language === "javascript") {
1026:     const node = await findTool("node");
1027:     if (!node) return { ok: false, language, saved, terminal: "Node.js was not found. Install Node.js to
run JavaScript." };
1028:     const jsFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) ===
"javascript");
1029:     const runSource = await ensureRunSource({ files: jsFiles.length ? jsFiles : allWorkspaceFiles,
languages: ["javascript"] });
1030:     const targets = action === "build"
1031:       ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "javascript")
1032:       : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
1033:     ok = true;
1034:     for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName:
saved.fileName }]) {
1035:       const check = await runProcess(node, ["--check", target.sourcePath], { cwd: runSource.runDir,
timeoutMs: 15000 });
1036:       terminal.push(terminalLine(`${path.basename(node)} --check ${sourceDisplayName(target)}`,
processTerminalText(check)));
1037:       ok = ok && check.ok;
1038:     }
1039:     if (action === "compile" || action === "build") {
1040:       terminal.push(ok ? "JavaScript syntax check completed." : "JavaScript syntax check found errors.");
1041:     } else if (ok) {
1042:       const run = await runProcess(node, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs:
20000, ...stdinOptions });
1043:       append(`${path.basename(node)} ${saved.fileName}`, run);
1044:       ok = run.ok;
1045:     }
1046:   } else if (language === "python") {
1047:     const python = await findTool("python");
1048:     if (!python) return { ok: false, language, saved, terminal: "Python was not found. Install Python to
run Python code." };
1049:     const pyFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) ===
"python");
1050:     const runSource = await ensureRunSource({ files: pyFiles.length ? pyFiles : allWorkspaceFiles,
languages: ["python"] });

```

```

1051:     const targets = action === "build"
1052:       ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "python")
1053:       : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
1054:     ok = true;
1055:     for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName:
saved.fileName }]) {
1056:       const check = await runProcess(python, ["-m", "py_compile", target.sourcePath], { cwd:
runSource.runDir, timeoutMs: 15000 });
1057:       terminal.push(terminalLine(`${path.basename(python)} -m py_compile ${sourceDisplayName(target)}`,
processTerminalText(check)));
1058:       ok = ok && check.ok;
1059:     }
1060:     if (action === "compile" || action === "build") {
1061:       terminal.push(ok ? "Python bytecode compile check completed." : "Python compile check found
errors.");
1062:     } else if (ok) {
1063:       const run = await runProcess(python, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs:
20000, ...stdinOptions });
1064:       append(`${path.basename(python)} ${saved.fileName}`, run);
1065:       ok = run.ok;
1066:     }
1067:   } else if (language === "c" || language === "cpp") {
1068:     const cProfile = cFamilyCompileProfile(language, saved.fileName);
    ... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

append (server.mjs, lines 996-998)

append part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 996-998 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, terminalLine, and processTerminalText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include append at line 996. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

996:     const append = (label, result) => {
997:       terminal.push(terminalLine(label, processTerminalText(result)));
998:     };

```

missing (server.mjs, lines 999-1004)

missing part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 999-1004 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references async, findTool, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1003: return { found, missingTools };

Important local values include missing at line 999; found at line 1000; missingTools at line 1002. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

999:     const missing = async (tools) => {
1000:       const found = {};
1001:       for (const tool of tools) found[tool] = await findTool(tool);
1002:       const missingTools = tools.filter((tool) => !found[tool]);
1003:       return { found, missingTools };
1004:     };

```

installCompilerTools (server.mjs, lines 1321-1352)

installCompilerTools part of the compile code language/tool/build/run flow. In this file, it appears around lines 1321-1352 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizeCodeLanguage`, `findTool`, `runProcess`, `push`, `terminalLine`, `join`, `processTerminalText`, `clear`, and `compileToolStatus`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1325: `return {}`; line 1351: `return { ok, language: lang, terminal: terminal.join("\n\n"), tools: await compileToolStatus() };`

Important local values include `lang` at line 1322; `profile` at line 1323; `winget` at line 1331; `terminal` at line 1332; `ok` at line 1333; `args` at line 1335; `result` at line 1346. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1321: async function installCompilerTools(language = "") {
1322:   const lang = normalizeCodeLanguage(language);
1323:   const profile = compileLanguageProfiles[lang] || compileLanguageProfiles.javascript;
1324:   if (!profile.winget?.length) {
1325:     return {
1326:       ok: false,
1327:       language: lang,
1328:       terminal: `${profile.label} does not have an automatic Winget compiler install configured. Install
the required tools manually, then restart the builder.`
1329:     };
1330:   }
1331:   const winget = await findTool("winget") || "winget";
1332:   const terminal = [];
1333:   let ok = true;
1334:   for (const packageId of profile.winget) {
1335:     const args = [
1336:       "install",
1337:       "--source",
1338:       "winget",
1339:       "--accept-source-agreements",
1340:       "--accept-package-agreements",
1341:       "--silent",
1342:       "--id",
1343:       packageId,
1344:       "--exact"
1345:     ];
1346:     const result = await runProcess(winget, args, { cwd: root, timeoutMs: 600000 });
1347:     terminal.push(terminalLine(`winget ${args.join(" ")}`, processTerminalText(result)));
1348:     ok = ok && result.ok;
1349:   }
1350:   compileToolCache.clear();
1351:   return { ok, language: lang, terminal: terminal.join("\n\n"), tools: await compileToolStatus() };
1352: }
```

sourceLooksTextual (server.mjs, lines 1354-1360)

`sourceLooksTextual` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1354-1360 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `toLowerCase`, and `includes`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1357: `if ([ "text", "webpage", "github", "linkedin", "public_profile", "resume_text", "code" ].includes(kind)) return true;`; line 1358: `return /\.(txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|tsx|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb|ino|xdc|sdc|tcl)$/i.test(url)`.

Important local values include `url` at line 1355; `kind` at line 1356. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1354: function sourceLooksTextual(source = {}) {
1355:   const url = String(source.url || "");
1356:   const kind = String(source.kind || "").toLowerCase();
1357:   if ([ "text", "webpage", "github", "linkedin", "public_profile", "resume_text", "code" ].includes(kind))
return true;
1358:   return /\.(txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|tsx|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb|ino|xdc|sdc|tcl)$/i.test(url)
1359:   || /github\.com|raw\.githubusercontent\.com/i.test(url);
1360: }
```

sourceUrlAllowed (server.mjs, lines 1362-1379)

`sourceUrlAllowed` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1362-1379 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level

documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URL, toLowerCase, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1366: return [; line 1377: return false;.

Important local values include parsed at line 1364; hostName at line 1365. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1362: function sourceUrlAllowed(url) {
1363:   try {
1364:     const parsed = new URL(url);
1365:     const hostName = parsed.hostname.toLowerCase();
1366:     return [
1367:       "localhost",
1368:       "127.0.0.1",
1369:       "github.com",
1370:       "api.github.com",
1371:       "raw.githubusercontent.com",
1372:       "gist.githubusercontent.com",
1373:       "linkedin.com",
1374:       "www.linkedin.com"
1375:     ].includes(hostName);
1376:   } catch {
1377:     return false;
1378:   }
1379: }
```

gitHubHeaders (server.mjs, lines 1384-1389)

gitHubHeaders part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1384-1389 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1385: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1384: function gitHubHeaders(accept = "application/vnd.github+json") {
1385:   return {
1386:     "Accept": accept,
1387:     "User-Agent": "OMB-Portfolio-Builder-AI"
1388:   };
1389: }
```

parseGitHubSourceUrl (server.mjs, lines 1391-1418)

parseGitHubSourceUrl part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1391-1418 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URL, toLowerCase, split, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 1397: return {}; line 1405: if (host !== "github.com" || !parts.length) return null;; line 1406: if (parts.length === 1) return { type: "profile", owner: parts[0] };; line 1409: return { ...base, type: "file", branch: parts[3], filePath: parts.slice(4).join("/") };. There are 3 additional return statements in the function body.

Important local values include parsed at line 1393; host at line 1394; parts at line 1395; base at line 1407. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1391: function parseGitHubSourceUrl(url) {
1392:   try {
```

```

1393:     const parsed = new URL(url);
1394:     const host = parsed.hostname.toLowerCase();
1395:     const parts = parsed.pathname.split("/").filter(Boolean);
1396:     if (host === "raw.githubusercontent.com" && parts.length >= 4) {
1397:       return {
1398:         type: "file",
1399:         owner: parts[0],
1400:         repo: parts[1],
1401:         branch: parts[2],
1402:         filePath: parts.slice(3).join("/")
1403:       };
1404:     }
1405:     if (host !== "github.com" || !parts.length) return null;
1406:     if (parts.length === 1) return { type: "profile", owner: parts[0] };
1407:     const base = { owner: parts[0], repo: parts[1] };
1408:     if (parts[2] === "blob" && parts.length >= 5) {
1409:       return { ...base, type: "file", branch: parts[3], filePath: parts.slice(4).join("/") };
1410:     }
1411:     if (parts[2] === "tree" && parts.length >= 4) {
1412:       return { ...base, type: "repo", branch: parts[3] };
1413:     }
1414:     return { ...base, type: "repo" };
1415:   } catch {
1416:     return null;
1417:   }
1418: }

```

fetchGitHubJson (server.mjs, lines 1420-1424)

fetchGitHubJson loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1420-1424 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, gitHubHeaders, and json. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1422: if (!response.ok) return null;; line 1423: return response.json().catch(() => null);.

Important local values include response at line 1421. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1420: async function fetchGitHubJson(url) {
1421:   const response = await fetch(url, { headers: gitHubHeaders() });
1422:   if (!response.ok) return null;
1423:   return response.json().catch(() => null);
1424: }

```

fetchLimitedText (server.mjs, lines 1426-1434)

fetchLimitedText loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1426-1434 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, gitHubHeaders, clampText, text, get, and stripHtmlToText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1430: if (!response.ok) return "";; line 1433: return /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;.

Important local values include response at line 1427; rawText at line 1431; contentType at line 1432. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1426: async function fetchLimitedText(url, maxLength = 12000) {
1427:   const response = await fetch(url, {
1428:     headers: gitHubHeaders("text/plain,text/markdown,text/html,application/json;q=0.8,*/*;q=0.1")
1429:   });
1430:   if (!response.ok) return "";
1431:   const rawText = clampText(await response.text(), maxLength);
1432:   const contentType = response.headers.get("Content-Type") || "";
1433:   return /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;
1434: }

```

githubQuestionTokens (server.mjs, lines 1436-1442)

githubQuestionTokens part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1436-1442 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, toLowerCase, replace, split, filter, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1437: return [...new Set(String(question || ""))].

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1436: function githubQuestionTokens(question = "") {
1437:   return [...new Set(String(question || ""))
1438:     .toLowerCase()
1439:     .replace(/^[a-z0-9_#\+\.\s-]+/g, " ")
1440:     .split(/\s+/)
1441:     .filter((token) => token.length >= 3 && ![ "the", "and", "with", "from", "show", "code", "file",
"github"].includes(token))];
1442: }
```

scoreGitHubFile (server.mjs, lines 1444-1455)

scoreGitHubFile part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1444-1455 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, githubQuestionTokens, b, forEach, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1454: return score;.

Important local values include cleanPath at line 1445; tokens at line 1446; score at line 1447. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1444: function scoreGitHubFile(filePath = "", question = "") {
1445:   const cleanPath = filePath.toLowerCase();
1446:   const tokens = githubQuestionTokens(question);
1447:   let score = githubTextFilePattern.test(cleanPath) ? 10 : 0;
1448:   if (/readme\.md$/i.test(cleanPath)) score += 55;
1449:   if (/\.(\.c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|ino|tcl|xdc|sdc)$/i.test(cleanPath)) score += 28;
1450:   if (/b(test|tb|bench|sim|simulation|src|firmware|hardware|rtl|driver|main)\b/i.test(cleanPath)) score
+= 12;
1451:   tokens.forEach((token) => {
1452:     if (cleanPath.includes(token)) score += 18;
1453:   });
1454:   return score;
1455: }
```

wantsGitHubCode (server.mjs, lines 1457-1459)

wantsGitHubCode part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1457-1459 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1458: return /\b(code|source|snippet|implementation|firmware|driver|module|verilog|vhd|python|javascript|typescript|c\+\+|cpp|c\s+code|pull|show|display)\b/i.test(question);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1457: function wantsGitHubCode(question = "") {
1458:   return /\b(code|source|snippet|implementation|firmware|driver|module|verilog|vhd|python|javascript|typ
escript|c\+\+|cpp|c\s+code|pull|show|display)\b/i.test(question);
1459: }
```

scoreGitHubRepo (server.mjs, lines 1461-1481)

scoreGitHubRepo part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1461-1481 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references githubQuestionTokens, isArray, join, toLowerCase, forEach, includes, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1480: return score;.

Important local values include tokens at line 1462; haystack at line 1463; score at line 1470. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1461: function scoreGitHubRepo(repo = {}, question = "") {
1462:   const tokens = githubQuestionTokens(question);
1463:   const haystack = [
1464:     repo.full_name,
1465:     repo.name,
1466:     repo.description,
1467:     repo.language,
1468:     ...(Array.isArray(repo.topics) ? repo.topics : [])
1469:   ].join(" ").toLowerCase();
1470:   let score = repo.fork ? -10 : 12;
1471:   if (!repo.archived) score += 6;
1472:   if (repo.language) score += 4;
1473:   if (repo.name && !/\.github\.io$/i.test(repo.name)) score += 3;
1474:   tokens.forEach((token) => {
1475:     if (haystack.includes(token)) score += 22;
1476:   });
1477:   if (/\\b(vco|oscillator|pwm|analog|mixed|pcb|firmware|stm32|fpga|asic|verilog|embedded|signal|design)\\b/
i.test(haystack)) {
1478:     score += 12;
1479:   }
1480:   return score;
1481: }
```

fetchGitHubProfileSource (server.mjs, lines 1483-1543)

fetchGitHubProfileSource loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1483-1543 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references all, fetchGitHubJson, isArray, wantsGitHubCode, sort, scoreGitHubRepo, slice, fetchGitHubRepositorySource, trim, push, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1536: return {.

Important local values include owner at line 1484; repoList at line 1489; includeCode at line 1490; selectedRepos at line 1491; repoBlocks at line 1496; repoSource at line 1500; lines at line 1517. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1483: async function fetchGitHubProfileSource(source = {}, parsed = {}) {
1484:   const owner = parsed.owner;
1485:   const [profile, repos] = await Promise.all([
1486:     fetchGitHubJson(`https://api.github.com/users/${encodeURIComponent(owner)}`),
1487:     fetchGitHubJson(`https://api.github.com/users/${encodeURIComponent(owner)}/repos?per_page=30&sort=updated`)
1488:   ]);
1489:   const repoList = Array.isArray(repos) ? repos : [];
1490:   const includeCode = wantsGitHubCode(source.question || "");
1491:   const selectedRepos = includeCode
1492:     ? [...repoList]
1493:     .sort((a, b) => scoreGitHubRepo(b, source.question || "") - scoreGitHubRepo(a, source.question || ""))
1494:     .slice(0, 3)
1495:     : [];
1496:   const repoBlocks = [];
1497:
1498:   for (const repo of selectedRepos) {
1499:     if (!repo?.name || !repo?.html_url) continue;
1500:     const repoSource = await fetchGitHubRepositorySource({
1501:       ...source,
1502:       context: `Public GitHub repository selected from ${owner}'s profile`,
```

```

1503:     label: repo.full_name,
1504:     maxCodeFiles: 3,
1505:     maxFileTextLength: 3600,
1506:     maxRepositoryTextLength: 9000,
1507:     url: repo.html_url
1508:   }, {
1509:     owner,
1510:     repo: repo.name,
1511:     type: "repo",
1512:     branch: repo.default_branch
1513:   });
1514:   if (repoSource?.text?.trim()) repoBlocks.push(repoSource.text.trim());
1515: }
1516:
1517: const lines = [
1518:   `GitHub public profile: ${owner}`,
1519:   profile?.name ? `Name: ${profile.name}` : "",
1520:   profile?.bio ? `Bio: ${profile.bio}` : "",
1521:   profile?.html_url ? `Profile URL: ${profile.html_url}` : source.url,
1522:   "",
1523:   "Public repositories visible from the profile:",
1524:   ...repoList.slice(0, 20).map((repo) => [
1525:     `- ${repo.full_name}`,
1526:     repo.description ? `: ${repo.description}` : "",
1527:     repo.language ? ` | Language: ${repo.language}` : "",
1528:     repo.html_url ? ` | URL: ${repo.html_url}` : "",
1529:     repo.updated_at ? ` | Updated: ${repo.updated_at}` : ""
1530:   ]).join("\n"),
1531:   includeCode && selectedRepos.length ? "" : "",
1532:   includeCode && selectedRepos.length ? "Selected public repositories inspected for source code:" : "",
1533:   ...selectedRepos.map((repo) => `- ${repo.full_name}${repo.language ? ` | Language: ${repo.language}` : ""}${repo.description ? ` | ${repo.description}` : ""}`),
1534:   ...repoBlocks.flatMap((block) => ["", "---", block])
1535: ].filter(Boolean);
1536: return {
1537:   context: source.context || "Public GitHub profile",
1538:   title: source.title || source.label || `GitHub profile: ${owner}`,
1539:   type: includeCode ? "public GitHub profile and selected repository code" : "public GitHub profile",
1540:   url: source.url,
1541:   text: clampText(lines.join("\n"), includeCode ? 30000 : 9000)
1542: };
1543: }

```

fetchGitHubRepositorySource (server.mjs, lines 1545-1612)

fetchGitHubRepositorySource loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1545-1612 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetchGitHubJson, wantsGitHubCode, max, min, isFinite, filter, fetchLimitedText, trim, push, clampText, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1569: return {}; line 1605: return {}.

Important local values include metadata at line 1547; branch at line 1548; question at line 1549; requestedMaxCodeFiles at line 1550; maxCodeFiles at line 1551; requestedFileTextLength at line 1552; maxFileTextLength at line 1553; requestedRepoTextLength at line 1554; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1545: async function fetchGitHubRepositorySource(source = {}, parsed = {}) {
1546:   const { owner, repo } = parsed;
1547:   const metadata = await
fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}`);
1548:   const branch = parsed.branch || metadata?.default_branch || "main";
1549:   const question = source.question || "";
1550:   const requestedMaxCodeFiles = Number(source.maxCodeFiles || (wantsGitHubCode(question) ? 6 : 3));
1551:   const maxCodeFiles = Math.max(1, Math.min(8, Number.isFinite(requestedMaxCodeFiles) ?
requestedMaxCodeFiles : 3));
1552:   const requestedFileTextLength = Number(source.maxFileTextLength || (wantsGitHubCode(question) ? 7000 :
3000));
1553:   const maxFileTextLength = Math.max(1000, Math.min(10000, Number.isFinite(requestedFileTextLength) ?
requestedFileTextLength : 3000));
1554:   const requestedRepoTextLength = Number(source.maxRepositoryTextLength || (wantsGitHubCode(question) ?
22000 : 14000));
1555:   const maxRepositoryTextLength = Math.max(5000, Math.min(26000, Number.isFinite(requestedRepoTextLength) ?
requestedRepoTextLength : 14000));
1556:   const lines = [
1557:     `GitHub repository: ${owner}/${repo}`,
1558:     metadata?.description ? `Description: ${metadata.description}` : "",
1559:     metadata?.language ? `Primary language: ${metadata.language}` : "",
1560:     metadata?.html_url ? `Repository URL: ${metadata.html_url}` : source.url,
1561:     metadata?.default_branch ? `Default branch: ${metadata.default_branch}` : "",
1562:     ""

```

```

1563:   ].filter(Boolean);
1564:
1565:   if (parsed.type === "file" && parsed.filePath) {
1566:     const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${parsed.filePath}`;
1567:     const text = await fetchLimitedText(rawUrl, 14000);
1568:     if (text.trim()) lines.push(`Source file: ${parsed.filePath}`, text.trim());
1569:     return {
1570:       context: source.context || "Public GitHub source file",
1571:       title: source.title || source.label || `${owner}/${repo}/${parsed.filePath}`,
1572:       type: "public GitHub source file",
1573:       url: source.url,
1574:       text: clampText(lines.join("\n"), 14000)
1575:     };
1576:   }
1577:
1578:   const tree = await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/git/trees/${encodeURIComponent(branch)}?recursive=1`);
1579:   const files = Array.isArray(tree?.tree)
1580:     ? tree.tree
1581:     : [];
1582:   const scoreGitHubFile = (b, a) => scoreGitHubFile(b.path, question) - scoreGitHubFile(a.path, question);
1583:   files.sort((a, b) => scoreGitHubFile(b.path, question) - scoreGitHubFile(a.path, question));
1584:
1585:   const readmePath = files.find((file) => /^(^|\/)readme\.md$/i.test(file.path)).path || "README.md";
1586:   const readmeText = await
1587:   fetchLimitedText(`https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${readmePath}`, 9000).catch(() =>
1588:   "");
1589:   if (readmeText.trim()) lines.push(`README excerpt from ${readmePath}:`, readmeText.trim(), "");
1590:   if (files.length) {
1591:     lines.push("Repository text/code file list:", ...files.slice(0, 30).map((file) => `- ${file.path}`),
1592:     "");
1593:   }
1594:
1595:   const includeCode = wantsGitHubCode(question);
1596:   const selectedFiles = files
1597:     .filter((file) => !/readme\.md$/i.test(file.path))
1598:     .slice(0, maxCodeFiles);
1599:
1600:   for (const file of selectedFiles) {
1601:     const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${file.path}`;
1602:     const fileText = await fetchLimitedText(rawUrl, maxFileTextLength).catch(() => "");
1603:     if (!fileText.trim()) continue;
1604:     lines.push(`Source file: ${file.path}`, fileText.trim(), "");
1605:   }
1606:
1607:   return {
1608:     context: source.context || "Public GitHub repository",
1609:     title: source.title || source.label || `${owner}/${repo}`,
1610:     type: includeCode ? "public GitHub repository and code" : "public GitHub repository",
1611:     url: source.url,
1612:     text: clampText(lines.join("\n"), maxRepositoryTextLength)
1613:   };
1614: }

```

fetchGitHubSourceText (server.mjs, lines 1614-1619)

fetchGitHubSourceText loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1614-1619 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parseGitHubSourceUrl, fetchGitHubProfileSource, and fetchGitHubRepositorySource. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1616: if (!parsed) return null;; line 1617: if (parsed.type === "profile") return fetchGitHubProfileSource(source, parsed);; line 1618: return fetchGitHubRepositorySource(source, parsed);.

Important local values include parsed at line 1615. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1614: async function fetchGitHubSourceText(source = {}) {
1615:   const parsed = parseGitHubSourceUrl(source.url || "");
1616:   if (!parsed) return null;
1617:   if (parsed.type === "profile") return fetchGitHubProfileSource(source, parsed);
1618:   return fetchGitHubRepositorySource(source, parsed);
1619: }

```

readLocalSourceText (server.mjs, lines 1621-1639)

readLocalSourceText loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1621-1639 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URL, Set, has, toLowerCase, replace, includes, extname, readFile, and resolveInsideRoot. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 1626: return ""; line 1629: if (!localHosts.has(parsed.hostname.toLowerCase())) return ""; line 1631: if (!pathname || pathname.includes("..")) return ""; line 1633: if (![".txt", ".md", ".json", ".csv", ".log", ".xml", ".js", ".mjs", ".css", ".c", ".h", ".cpp", ".hpp", ".py", ".v", ".sv", ".spice", ".cir", ".net", ".asc", ".sch", ".kicad_sch", ".kicad_pcb"].includes(ext)) return ""; There are 2 additional return statements in the function body.

Important local values include localHosts at line 1628; pathname at line 1630; ext at line 1632. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1621: async function readLocalSourceText(url) {
1622:   let parsed;
1623:   try {
1624:     parsed = new URL(url);
1625:   } catch {
1626:     return "";
1627:   }
1628:   const localHosts = new Set(["localhost", "127.0.0.1"]);
1629:   if (!localHosts.has(parsed.hostname.toLowerCase())) return "";
1630:   const pathname = decodeURIComponent(parsed.pathname.replace(/^\/+/, ""));
1631:   if (!pathname || pathname.includes("..")) return "";
1632:   const ext = path.extname(pathname).toLowerCase();
1633:   if (![".txt", ".md", ".json", ".csv", ".log", ".xml", ".js", ".mjs", ".css", ".c", ".h", ".cpp",
".hpp", ".py", ".v", ".sv", ".spice", ".cir", ".net", ".asc", ".sch", ".kicad_sch", ".kicad_pcb"].includes(ext))
return "";
1634:   try {
1635:     return await readFile(resolveInsideRoot(pathname), "utf8");
1636:   } catch {
1637:     return "";
1638:   }
1639: }
```

fetchSourceText (server.mjs, lines 1641-1679)

fetchSourceText loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1641-1679 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sourceLooksTextual, sourceUrlAllowed, fetchGitHubSourceText, trim, readLocalSourceText, clampText, replace, fetch, get, text, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 8 return statement(s). The most important visible returns are: line 1643: if (!url || !sourceLooksTextual(source) || !sourceUrlAllowed(url)) return null;; line 1646: if (githubText?.text?.trim()) return githubText;; line 1650: return {}; line 1663: if (!response.ok) return null;. There are 4 additional return statements in the function body.

Important local values include url at line 1642; githubText at line 1645; localText at line 1648; response at line 1660; contentType at line 1664; rawText at line 1666; text at line 1667. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1641: async function fetchSourceText(source = {}) {
1642:   const url = String(source.url || "");
1643:   if (!url || !sourceLooksTextual(source) || !sourceUrlAllowed(url)) return null;
1644:   if (/github\.com|raw\.githubusercontent\.com/i.test(url)) {
1645:     const githubText = await fetchGitHubSourceText(source);
1646:     if (githubText?.text?.trim()) return githubText;
1647:   }
1648:   const localText = await readLocalSourceText(url);
1649:   if (localText.trim()) {
1650:     return {
1651:       context: source.context || "",
1652:       title: source.title || source.label || url,
1653:       type: source.type || source.kind || "text source",
1654:       url,
1655:       text: clampText(localText.replace(/\s+\n/g, "\n").trim(), 9000)
1656:     };
1657:   }
1658:
1659:   try {
1660:     const response = await fetch(url, {
```



```

1661:     headers: { "Accept": "text/plain,text/markdown,text/html,application/json;q=0.8,*/*;q=0.1" }
1662:   });
1663:   if (!response.ok) return null;
1664:   const contentType = response.headers.get("Content-Type") || "";
1665:   if (!/text|json|xml|html|markdown/i.test(contentType)) return null;
1666:   const rawText = clampText(await response.text(), 12000);
1667:   const text = /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;
1668:   if (!text.trim()) return null;
1669:   return {
1670:     context: source.context || "",
1671:     title: source.title || source.label || url,
1672:     type: source.type || source.kind || "public source",
1673:     url,
1674:     text: clampText(text.trim(), 9000)
1675:   };
1676: } catch {
1677:   return null;
1678: }
1679: }

```

enrichPortfolioContext (server.mjs, lines 1681-1692)

This function expands the AI context with portfolio content and public source text when allowed. It can read local public files or fetch public GitHub/link data so the assistant has richer material for portfolio-aware answers.

The AI should not give canned answers. It needs enough context to distinguish a general electronics question from a project-specific question and answer with relevant evidence.

It calls or references `isArray`, `slice`, `map`, `all`, and `filter`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1687: `return {`.

Important local values include `question` at line 1682; `sources` at line 1683; `sourceExcerpts` at line 1686. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1681: async function enrichPortfolioContext(context = {}) {
1682:   const question = String(context.question || "");
1683:   const sources = Array.isArray(context.sourceFetches)
1684:     ? context.sourceFetches.slice(0, 10).map((source) => ({ ...source, question: source.question ||
question }))
1685:     : [];
1686:   const sourceExcerpts = (await Promise.all(sources.map(fetchSourceText))).filter(Boolean);
1687:   return {
1688:     ...context,
1689:     sourceExcerpts,
1690:     sourceFetchPolicy: "Fetched excerpts are limited to safe text-like same-site, GitHub/raw GitHub,
LinkedIn, and public portfolio URLs. GitHub repository links can be expanded into repository metadata, README
text, and selected public source files when a question asks for code. PDFs/images are represented by captions,
metadata, filenames, and companion text files when available."
1691:   };
1692: }

```

cleanConversationHistory (server.mjs, lines 1694-1703)

`cleanConversationHistory` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1694-1703 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `isArray`, `slice`, `map`, `clampText`, `trim`, and `filter`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1695: `if (!Array.isArray(value)) return [];`; line 1696: `return value`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1694: function cleanConversationHistory(value) {
1695:   if (!Array.isArray(value)) return [];
1696:   return value
1697:     .slice(-8)
1698:     .map((item) => ({
1699:       role: item?.role === "assistant" ? "assistant" : "user",
1700:       content: clampText(item?.content, 1400).trim()
1701:     }))
1702:     .filter((item) => item.content);
1703: }

```

extractOpenAiText (server.mjs, lines 1705-1717)

extractOpenAiText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1705-1717 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, isArray, forEach, push, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1706: if (typeof data?.output_text === "string" && data.output_text.trim()) return data.output_text.trim(); line 1716: return chunks.join("\n\n").trim();.

Important local values include output at line 1707; chunks at line 1708. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1705: function extractOpenAiText(data) {
1706:   if (typeof data?.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
1707:   const output = Array.isArray(data?.output) ? data.output : [];
1708:   const chunks = [];
1709:
1710:   output.forEach((item) => {
1711:     (item.content || []).forEach((content) => {
1712:       if (typeof content.text === "string" && content.text.trim()) chunks.push(content.text.trim());
1713:     });
1714:   });
1715:
1716:   return chunks.join("\n\n").trim();
1717: }
```

extractOllamaText (server.mjs, lines 1719-1723)

extractOllamaText part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1719-1723 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1720: if (typeof data.message?.content === "string") return data.message.content.trim(); line 1721: if (typeof data.response === "string") return data.response.trim(); line 1722: return "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1719: function extractOllamaText(data = {}) {
1720:   if (typeof data.message?.content === "string") return data.message.content.trim();
1721:   if (typeof data.response === "string") return data.response.trim();
1722:   return "";
1723: }
```

callOllamaPortfolioAi (server.mjs, lines 1725-1773)

callOllamaPortfolioAi part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1725-1773 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references AbortController, abort, fetch, replace, stringify, clampText, join, json, and extractOllamaText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 1762: return { ok: false, error: data?.error || `Ollama returned HTTP \${response.status}.` }; line 1765: return answer; line 1767: : { ok: false, error: "Ollama did not return an answer." }; line 1769: return { ok: false, error: error?.name === "AbortError" ? "Ollama timed out." : "Ollama is not reachable on this machine." };.

Important local values include host at line 1726; model at line 1727; controller at line 1728; timeout at line 1729; response at line 1731; data at line 1760; answer at line 1764. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1725: async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch }) {
1726:   const host = process.env.OLLAMA_HOST || "http://127.0.0.1:11434";
1727:   const model = process.env.OLLAMA_MODEL || "llama3.2";
1728:   const controller = new AbortController();
1729:   const timeout = setTimeout(() => controller.abort(), Number(process.env.OLLAMA_TIMEOUT_MS || 45000));
1730:   try {
1731:     const response = await fetch(`${host.replace(/\/+$/, "")}/api/chat`, {
1732:       method: "POST",
1733:       headers: { "Content-Type": "application/json" },
1734:       signal: controller.signal,
1735:       body: JSON.stringify({
1736:         model,
1737:         stream: false,
1738:         messages: [
1739:           { role: "system", content: portfolioAiInstructions },
1740:           {
1741:             role: "user",
1742:             content: [
1743:               `Visitor question: ${question}`,
1744:               `Question intent: ${intent}`,
1745:               `Web/public lookup requested by browser: ${allowWebSearch ? "yes" : "no"}`,
1746:               "",
1747:               "Recent conversation JSON:",
1748:               clampText(JSON.stringify(conversation, null, 2), 8000),
1749:               "",
1750:               "Portfolio context JSON:",
1751:               clampText(JSON.stringify(context, null, 2), 18000)
1752:             ].join("\n")
1753:           }
1754:         ],
1755:         options: {
1756:           temperature: Number(process.env.OLLAMA_TEMPERATURE || 0.35)
1757:         }
1758:       })
1759:     });
1760:     const data = await response.json().catch(() => ({}));
1761:     if (!response.ok) {
1762:       return { ok: false, error: data?.error || `Ollama returned HTTP ${response.status}` };
1763:     }
1764:     const answer = extractOllamaText(data);
1765:     return answer
1766:       ? { ok: true, answer, model }
1767:       : { ok: false, error: "Ollama did not return an answer." };
1768:   } catch (error) {
1769:     return { ok: false, error: error?.name === "AbortError" ? "Ollama timed out." : "Ollama is not reachable on this machine." };
1770:   } finally {
1771:     clearTimeout(timeout);
1772:   }
1773: }
```

ruleBasedConversationAnswer (server.mjs, lines 1775-1786)

ruleBasedConversationAnswer part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1775-1786 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1778: return "You're welcome. I can keep helping with this portfolio, project evidence, resume links, or related electronics and embedded-systems questions."; line 1782: return "I am this portfolio's assistant. I can help visitors explore the saved engineering work, explain project details, summarize portfolio evidence, and answer related electronics, embedded, analog, digital, FPGA, ASI; line 1785: return "Hi. I can help you explore this portfolio, explain projects, summarize project evidence, open relevant sections, or answer related electronics and embedded-systems questions. You can ask about a specific project,.

Important local values include clean at line 1776. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1775: function ruleBasedConversationAnswer(question = "") {
1776:   const clean = String(question || "").toLowerCase();
1777:   if (/^b(thanks|thank you|appreciate it)\b/.test(clean)) {
1778:     return "You're welcome. I can keep helping with this portfolio, project evidence, resume links, or related electronics and embedded-systems questions.";
1779:   }
1780:
1781:   if (/^b(who are you|what are you)\b/.test(clean)) {
1782:     return "I am this portfolio's assistant. I can help visitors explore the saved engineering work,
```

```

explain project details, summarize portfolio evidence, and answer related electronics, embedded, analog,
digital, FPGA, ASIC, PCB, and firmware questions.";
1783: }
1784:
1785: return "Hi. I can help you explore this portfolio, explain projects, summarize project evidence, open
relevant sections, or answer related electronics and embedded-systems questions. You can ask about a specific
project, a tool like KiCad or STM32CubeIDE, or a general topic such as embedded systems, op amps, FPGA design,
ASICs, or PCB testing.";
1786: }

```

isLocalRequest (server.mjs, lines 1788-1791)

isLocalRequest part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1788-1791 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1790: return address === "127.0.0.1" || address === "::1" || address === "::ffff:127.0.0.1";.

Important local values include address at line 1789. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1788: function isLocalRequest(request) {
1789:   const address = request.socket.remoteAddress || "";
1790:   return address === "127.0.0.1" || address === "::1" || address === "::ffff:127.0.0.1";
1791: }

```

readJsonFile (server.mjs, lines 1793-1795)

readJsonFile loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1793-1795 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, and readFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1794: return JSON.parse(await readFile(filePath, "utf8"));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1793: async function readJsonFile(filePath) {
1794:   return JSON.parse(await readFile(filePath, "utf8"));
1795: }

```

readRequestJson (server.mjs, lines 1797-1810)

readRequestJson loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1797-1810 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Error, push, parse, concat, toString, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1809: return JSON.parse((Buffer.concat(chunks).toString("utf8") || "{}").replace(/^\uFEFF/, ""));.

Important local values include chunks at line 1798; size at line 1799. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1797: async function readRequestJson(request) {
1798:   const chunks = [];
1799:   let size = 0;

```

```

1800:
1801:   for await (const chunk of request) {
1802:     size += chunk.length;
1803:     if (size > 60 * 1024 * 1024) {
1804:       throw new Error("Request body is too large.");
1805:     }
1806:     chunks.push(chunk);
1807:   }
1808:
1809:   return JSON.parse((Buffer.concat(chunks).toString("utf8") || "{}").replace(/^\uFEFF/, ""));
1810: }

```

safeSegment (server.mjs, lines 1812-1819)

safeSegment validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1812-1819 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, trim, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1818: return segment || fallback;.

Important local values include segment at line 1813. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1812: function safeSegment(value, fallback = "item") {
1813:   const segment = String(value || fallback)
1814:     .toLowerCase()
1815:     .trim()
1816:     .replace(/^[a-z0-9._-]+/g, "-")
1817:     .replace(/^-+|-+$/g, "");
1818:   return segment || fallback;
1819: }

```

safeFileName (server.mjs, lines 1821-1826)

safeFileName validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1821-1826 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, safeSegment, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1825: return ext ? `\${name}.\${ext}` : name;.

Important local values include parsed at line 1822; name at line 1823; ext at line 1824. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1821: function safeFileName(value) {
1822:   const parsed = path.parse(String(value || "file"));
1823:   const name = safeSegment(parsed.name, "file");
1824:   const ext = safeSegment(parsed.ext.replace(".", ""), "");
1825:   return ext ? `${name}.${ext}` : name;
1826: }

```

resolveInsideRoot (server.mjs, lines 1828-1834)

resolveInsideRoot part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1828-1834 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, join, startsWith, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1833: return target;.

Important local values include target at line 1829. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1828: function resolveInsideRoot(...segments) {
1829:   const target = path.normalize(path.join(root, ...segments));
1830:   if (!target.startsWith(root)) {
1831:     throw new Error("Resolved path is outside the workspace.");
1832:   }
1833:   return target;
1834: }
```

resolveInsidePortfolioRoot (server.mjs, lines 1836-1843)

resolveInsidePortfolioRoot part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1836-1843 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resolve, relative, startsWith, isAbsolute, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1842: return target;.

Important local values include target at line 1837; relative at line 1838. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1836: function resolveInsidePortfolioRoot(...segments) {
1837:   const target = path.resolve(portfolioRoot, ...segments);
1838:   const relative = path.relative(portfolioRoot, target);
1839:   if (relative.startsWith(".") || path.isAbsolute(relative)) {
1840:     throw new Error("Resolved path is outside the portfolio workspace.");
1841:   }
1842:   return target;
1843: }
```

resolveInsideCompileRoot (server.mjs, lines 1845-1852)

resolveInsideCompileRoot part of the compile code language/tool/build/run flow. In this file, it appears around lines 1845-1852 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resolve, relative, startsWith, isAbsolute, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1851: return target;.

Important local values include target at line 1846; relative at line 1847. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1845: function resolveInsideCompileRoot(...segments) {
1846:   const target = path.resolve(compileRoot, ...segments);
1847:   const relative = path.relative(compileRoot, target);
1848:   if (relative.startsWith(".") || path.isAbsolute(relative)) {
1849:     throw new Error("Resolved path is outside the compile workspace.");
1850:   }
1851:   return target;
1852: }
```

samePath (server.mjs, lines 1854-1856)

samePath part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1854-1856 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `resolve`, and `toLowerCase`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1855: `return path.resolve(left).toLowerCase() === path.resolve(right).toLowerCase();`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1854: function samePath(left, right) {
1855:   return path.resolve(left).toLowerCase() === path.resolve(right).toLowerCase();
1856: }
```

pathExists (server.mjs, lines 1858-1865)

`pathExists` part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1858-1865 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `fsAccess`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1861: `return true;`; line 1863: `return false;`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1858: async function pathExists(filePath) {
1859:   try {
1860:     await fsAccess(filePath);
1861:     return true;
1862:   } catch {
1863:     return false;
1864:   }
1865: }
```

publishAccessError (server.mjs, lines 1867-1877)

`publishAccessError` part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1867-1877 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `Error`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1876: `return error;`.

Important local values include `error` at line 1868. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1867: function publishAccessError(message, details = "", extra = {}) {
1868:   const error = new Error(message);
1869:   error.code = "PUBLISH_AUTHORIZATION_REQUIRED";
1870:   error.details = details || publishAuthorizationHelp;
1871:   error.publishAccess = {
1872:     authorizationRequired: true,
1873:     help: publishAuthorizationHelp,
1874:     ...extra
1875:   };
1876:   return error;
1877: }
```

gitFailureText (server.mjs, lines 1879-1885)

`gitFailureText` part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1879-1885 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, join, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1880: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1879: function gitFailureText(error) {
1880:   return [
1881:     error?.stderr,
1882:     error?.stdout,
1883:     error?.message
1884:   ].filter(Boolean).join("\n").trim();
1885: }
```

remoteUrlForDisplay (server.mjs, lines 1887-1891)

remoteUrlForDisplay part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1887-1891 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1888: return String(remoteUrl || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1887: function remoteUrlForDisplay(remoteUrl = "") {
1888:   return String(remoteUrl || "")
1889:     .replace(/^https:\\\\\/[^\:\/]+):[^\:\/]+@\/i, "https://$1:***@")
1890:     .replace(/^https:\\\\\/[^\:\/]+@\/i, "https://");
1891: }
```

parseGitHubRemote (server.mjs, lines 1893-1907)

parseGitHubRemote part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1893-1907 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, match, replace, URL, and split. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 1896: if (sshMatch) return { owner: sshMatch[1], repo: sshMatch[2].replace(/\.git\$/i, "") }; line 1900: if (!/github\.com\$/i.test(parsed.hostname)) return null; line 1902: if (parts.length < 2) return null; line 1903: return { owner: parts[0], repo: parts[1].replace(/\.git\$/i, "") };. There are 1 additional return statements in the function body.

Important local values include trimmed at line 1894; sshMatch at line 1895; parsed at line 1899; parts at line 1901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1893: function parseGitHubRemote(remoteUrl = "") {
1894:   const trimmed = String(remoteUrl || "").trim();
1895:   const sshMatch = trimmed.match(/^git@github\.com:([^\:\/]+)\.(.+)?(?:\.git)?$/i);
1896:   if (sshMatch) return { owner: sshMatch[1], repo: sshMatch[2].replace(/\.git$/i, "") };
1897:
1898:   try {
1899:     const parsed = new URL(trimmed.replace(/^(git|https)\/, ""));
1900:     if (!/github\.com$/i.test(parsed.hostname)) return null;
1901:     const parts = parsed.pathname.replace(/^(\/|\/)/, "").split("/");
1902:     if (parts.length < 2) return null;
1903:     return { owner: parts[0], repo: parts[1].replace(/\.git$/i, "") };
1904:   } catch {
1905:     return null;
1906:   }
1907: }
```

validatePublishRemoteUrl (server.mjs, lines 1909-1927)

validatePublishRemoteUrl part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1909-1927 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, URL, toLowerCase, Error, replace, split, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1911: if (!remoteUrl) return ""; line 1912: if (/^git@github\.com:[A-Za-z0-9_.-]+\V[A-Za-z0-9_.-]+(?:\.git)?\$/i.test(remoteUrl)) return remoteUrl; line 1922: return `https://github.com/\${parts[0]}/\${parts[1].replace(/\./i, "")}.git`;

Important local values include remoteUrl at line 1910; parsed at line 1914; parts at line 1918. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1909: function validatePublishRemoteUrl(value = "") {
1910:   const remoteUrl = String(value || "").trim();
1911:   if (!remoteUrl) return "";
1912:   if (/^git@github\.com:[A-Za-z0-9_.-]+\V[A-Za-z0-9_.-]+(?:\.git)?$/i.test(remoteUrl)) return remoteUrl;
1913:   try {
1914:     const parsed = new URL(remoteUrl);
1915:     if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {
1916:       throw new Error("Only GitHub HTTPS repository URLs are supported by the guided setup.");
1917:     }
1918:     const parts = parsed.pathname.replace(/^\/+/, "").split("/");
1919:     if (parts.length < 2 || !parts[0] || !parts[1]) {
1920:       throw new Error("GitHub repository URL must include owner and repository name.");
1921:     }
1922:     return `https://github.com/${parts[0]}/${parts[1].replace(/\./i, "")}.git`;
1923:   } catch (error) {
1924:     if (error.message?.includes("Only GitHub") || error.message?.includes("owner")) throw error;
1925:     throw new Error("Enter a GitHub repository URL such as
https://github.com/USERNAME/USERNAME.github.io.git.");
1926:   }
1927: }
```

validateCustomDomain (server.mjs, lines 1929-1936)

validateCustomDomain validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1929-1936 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1931: if (!domain) return ""; line 1935: return domain;

Important local values include domain at line 1930. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1929: function validateCustomDomain(value = "") {
1930:   const domain = String(value || "").trim().toLowerCase();
1931:   if (!domain) return "";
1932:   if (!/^(?:[a-z0-9]{0,61}[a-z0-9])?\.\.[a-z]{2,63}$/i.test(domain)) {
1933:     throw new Error("Custom domain must look like example.com or portfolio.example.com.");
1934:   }
1935:   return domain;
1936: }
```

compareVersions (server.mjs, lines 1938-1947)

compareVersions part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1938-1947 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1944: if (delta !== 0) return delta;; line 1946: return 0;.

Important local values include leftParts at line 1939; rightParts at line 1940; length at line 1941; delta at line 1943. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1938: function compareVersions(left = "", right = "") {
1939:   const leftParts = String(left || "0").split(/[.-]/).map((part) => Number.parseInt(part, 10) || 0);
1940:   const rightParts = String(right || "0").split(/[.-]/).map((part) => Number.parseInt(part, 10) || 0);
1941:   const length = Math.max(leftParts.length, rightParts.length);
1942:   for (let index = 0; index < length; index += 1) {
1943:     const delta = (leftParts[index] || 0) - (rightParts[index] || 0);
1944:     if (delta !== 0) return delta;
1945:   }
1946:   return 0;
1947: }
```

bumpPublishedAssetVersions (server.mjs, lines 1949-1965)

bumpPublishedAssetVersions part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1949-1965 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join, readFile, now, toString, replace, and writeFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1955: return false;; line 1962: if (next === html) return false;; line 1964: return true;.

Important local values include html at line 1950; indexPath at line 1951; version at line 1957; next at line 1958. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1949: async function bumpPublishedAssetVersions(baseRoot = portfolioRoot) {
1950:   let html = "";
1951:   const indexPath = path.join(baseRoot, "index.html");
1952:   try {
1953:     html = await readFile(indexPath, "utf8");
1954:   } catch {
1955:     return false;
1956:   }
1957:   const version = Date.now().toString();
1958:   const next = html
1959:     .replace(/(href="styles\.css)(?:\?v=[^"]*)?"/g, ` $1?v=${version}`)
1960:     .replace(/(src="script\.js)(?:\?v=[^"]*)?"/g, ` $1?v=${version}`)
1961:     .replace(/(src="electronics-search\.js)(?:\?v=[^"]*)?"/g, ` $1?v=${version}`);
1962:   if (next === html) return false;
1963:   await writeFile(indexPath, next, "utf8");
1964:   return true;
1965: }
```

workspaceHasCompatibleSiteFiles (server.mjs, lines 1967-1976)

workspaceHasCompatibleSiteFiles part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 1967-1976 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references readFile, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1972: return false;; line 1975: return true;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1967: async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot) {
1968:   for (const fileName of ["index.html", "styles.css", "script.js"]) {
1969:     try {
1970:       await readFile(path.join(baseRoot, fileName));
1971:     } catch {
1972:       return false;
1973:     }
1974:   }
1975:   return true;
1976: }
```

```

1974:   }
1975:   return true;
1976: }

```

getPublishTargetInfo (server.mjs, lines 1978-2044)

getPublishTargetInfo part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1978-2044 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references samePath, workspaceHasCompatibleSiteFiles, runPublishGit, trim, remoteUrlForDisplay, parseGitHubRemote, readFile, resolveInsidePortfolioRoot, readPublishAuthCache, publishAuthCachelsFresh, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1995: return info;; line 2043: return info;.

Important local values include info at line 1979; parsedRemote at line 2010; accessShape at line 2019; cached at line 2024. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1978: async function getPublishTargetInfo() {
1979:   const info = {
1980:     workspace: root,
1981:     portfolioWorkspace: portfolioRoot,
1982:     separatedWorkspace: !samePath(root, portfolioRoot),
1983:     defaultRepository: defaultSiteRepository,
1984:     gitBacked: false,
1985:     compatible: await workspaceHasCompatibleSiteFiles(),
1986:     remote: "",
1987:     branch: "",
1988:     repository: "",
1989:     customDomain: ""
1990:   };
1991:
1992:   try {
1993:     info.gitBacked = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim() ===
"true";
1994:   } catch {
1995:     return info;
1996:   }
1997:
1998:   try {
1999:     info.remote = remoteUrlForDisplay((await runPublishGit(["remote", "get-url",
"origin"])).stdout.trim());
2000:   } catch {
2001:     info.remote = "";
2002:   }
2003:
2004:   try {
2005:     info.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2006:   } catch {
2007:     info.branch = "";
2008:   }
2009:
2010:   const parsedRemote = parseGitHubRemote(info.remote);
2011:   info.repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : info.remote;
2012:
2013:   try {
2014:     info.customDomain = (await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8")).trim();
2015:   } catch {
2016:     info.customDomain = "";
2017:   }
2018:
2019:   const accessShape = {
2020:     branch: info.branch,
2021:     remote: info.remote,
2022:     repository: info.repository
2023:   };
2024:   const cached = await readPublishAuthCache();
2025:   if (publishAuthCacheIsFresh(cached, accessShape)) {
2026:     info.authorizationChecked = true;
2027:     info.authorizationCached = true;
2028:     info.checkedAt = cached.checkedAt;
2029:     info.expiresAt = publishAuthCacheExpiresAt(cached);
2030:     info.extendedTrust = Boolean(cached.extendedTrust);
2031:     info.successCountLastWeek = cached.successCountLastWeek || 0;
2032:     info.trustDays = cached.trustDays || (cached.extendedTrust ? 30 : 1);
2033:   } else {
2034:     info.authorizationChecked = false;
2035:     info.authorizationCached = false;
2036:     info.checkedAt = "";
2037:     info.expiresAt = "";
2038:     info.extendedTrust = false;
2039:     info.successCountLastWeek = cached?.successCountLastWeek || 0;

```

```

2040:     info.trustDays = 0;
2041:   }
2042:
2043:   return info;
2044: }

```

runGit (server.mjs, lines 2046-2064)

runGit part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2046-2064 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references execFileAsync, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2056: return { ...result, git: candidate };

Important local values include lastError at line 2047; result at line 2050. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2046: async function runGit(args, options = {}) {
2047:   let lastError = null;
2048:   for (const candidate of gitCandidates) {
2049:     try {
2050:       const result = await execFileAsync(candidate, args, {
2051:         cwd: options.cwd || root,
2052:         maxBuffer: options.maxBuffer || 10 * 1024 * 1024,
2053:         timeout: options.timeout || 0,
2054:         windowsHide: options.windowsHide !== false
2055:       });
2056:       return { ...result, git: candidate };
2057:     } catch (error) {
2058:       lastError = error;
2059:       if (error.code === "ENOENT") continue;
2060:       throw error;
2061:     }
2062:   }
2063:   throw lastError || new Error("Git executable was not found.");
2064: }

```

runPublishGit (server.mjs, lines 2066-2068)

runPublishGit part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2066-2068 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2067: return runGit(args, { ...options, cwd: options.cwd || portfolioRoot });

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2066: async function runPublishGit(args, options = {}) {
2067:   return runGit(args, { ...options, cwd: options.cwd || portfolioRoot });
2068: }

```

runOptionalCommand (server.mjs, lines 2070-2087)

runOptionalCommand part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2070-2087 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references execFileAsync. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2078: return { ok: true, stdout: result.stdout || "", stderr: result.stderr || "" };; line 2080: return {.

Important local values include result at line 2072. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2070: async function runOptionalCommand(command, args = [], options = {}) {
2071:   try {
2072:     const result = await execFileAsync(command, args, {
2073:       cwd: options.cwd || root,
2074:       maxBuffer: options.maxBuffer || 2 * 1024 * 1024,
2075:       timeout: options.timeout || 15000,
2076:       windowsHide: options.windowsHide !== false
2077:     });
2078:     return { ok: true, stdout: result.stdout || "", stderr: result.stderr || "" };
2079:   } catch (error) {
2080:     return {
2081:       ok: false,
2082:       stdout: error.stdout || "",
2083:       stderr: error.stderr || "",
2084:       error: error.message || String(error)
2085:     };
2086:   }
2087: }
```

getGitStatus (server.mjs, lines 2089-2136)

getGitStatus part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2089-2136 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runGit, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2113: return {.

Important local values include git at line 2090; gitResult at line 2092; credentialManager at line 2097; gcm at line 2099; gcmResult at line 2101. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2089: async function getGitStatus() {
2090:   let git = { ok: false, stdout: "", stderr: "", error: "Git for Windows was not found." };
2091:   try {
2092:     const gitResult = await runGit(["--version"]);
2093:     git = { ok: true, stdout: gitResult.stdout || "", stderr: gitResult.stderr || "" };
2094:   } catch (error) {
2095:     git = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message ||
"Git for Windows was not found." };
2096:   }
2097:   let credentialManager = { ok: false, version: "", error: "Git Credential Manager was not found." };
2098:   if (git.ok) {
2099:     let gcm = { ok: false, stdout: "", stderr: "", error: "Git Credential Manager was not found." };
2100:     try {
2101:       const gcmResult = await runGit(["credential-manager", "--version"]);
2102:       gcm = { ok: true, stdout: gcmResult.stdout || "", stderr: gcmResult.stderr || "" };
2103:     } catch (error) {
2104:       gcm = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message ||
"Git Credential Manager was not found." };
2105:     }
2106:     credentialManager = {
2107:       ok: gcm.ok,
2108:       version: (gcm.stdout || gcm.stderr || "").trim(),
2109:       error: gcm.ok ? "" : (gcm.stderr || gcm.error || "Git Credential Manager was not found.")
2110:     };
2111:   }
2112:
2113:   return {
2114:     git: {
2115:       ok: git.ok,
2116:       version: (git.stdout || git.stderr || "").trim(),
2117:       error: git.ok ? "" : (git.stderr || git.error || "Git for Windows was not found.")
2118:     },
2119:     credentialManager,
2120:     nodeRuntime: {
2121:       ok: true,
2122:       version: process.versions.node,
2123:       bundled: true,
2124:       note: "Bundled inside the desktop app. No manual Node.js install is required."
2125:     },
2126:     npnm: {
2127:       ok: true,
```

```

2128:         required: false,
2129:         note: "pnpm is only needed by developers building the installer from source."
2130:     },
2131:     app: {
2132:         version: process.env.OMB_APP_VERSION || packageJson.version || "0.0.0",
2133:         desktopApp: process.env.OMB_DESKTOP_APP === "1"
2134:     }
2135: };
2136: }

```

publishPathIsStageable (server.mjs, lines 2138-2152)

publishPathIsStageable part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2138-2152 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fsAccess, resolveInsidePortfolioRoot, and runPublishGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2141: return true;; line 2148: return true;; line 2150: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2138: async function publishPathIsStageable(relativePath) {
2139:   try {
2140:     await fsAccess(resolveInsidePortfolioRoot(relativePath));
2141:     return true;
2142:   } catch {
2143:     // Missing files can still be stageable if Git already tracks them and they were deleted.
2144:   }
2145:
2146:   try {
2147:     await runPublishGit(["ls-files", "--error-unmatch", "--", relativePath]);
2148:     return true;
2149:   } catch {
2150:     return false;
2151:   }
2152: }

```

stageablePublishPaths (server.mjs, lines 2154-2160)

stageablePublishPaths part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2154-2160 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references publishPathIsStageable, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2159: return stageable;.

Important local values include stageable at line 2155. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2154: async function stageablePublishPaths(pathsToCheck) {
2155:   const stageable = [];
2156:   for (const relativePath of pathsToCheck) {
2157:     if (await publishPathIsStageable(relativePath)) stageable.push(relativePath);
2158:   }
2159:   return stageable;
2160: }

```

runGitWithInput (server.mjs, lines 2162-2204)

runGitWithInput part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2162-2204 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references spawn, on, toString, once, resolve, Error, reject, and end. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2166: `return await new Promise((resolve, reject) => {`; line 2185: `return;`.

Important local values include `lastError` at line 2163; `child` at line 2167; `stdout` at line 2172; `stderr` at line 2173; `error` at line 2187. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2162: async function runGitWithInput(args, input, options = {}) {
2163:   let lastError = null;
2164:   for (const candidate of gitCandidates) {
2165:     try {
2166:       return await new Promise((resolve, reject) => {
2167:         const child = spawn(candidate, args, {
2168:           cwd: options.cwd || portfolioRoot,
2169:           windowsHide: true,
2170:           stdio: ["pipe", "pipe", "pipe"]
2171:         });
2172:         let stdout = "";
2173:         let stderr = "";
2174:
2175:         child.stdout.on("data", (chunk) => {
2176:           stdout += chunk.toString();
2177:         });
2178:         child.stderr.on("data", (chunk) => {
2179:           stderr += chunk.toString();
2180:         });
2181:         child.once("error", reject);
2182:         child.once("close", (code) => {
2183:           if (code === 0) {
2184:             resolve({ stdout, stderr, git: candidate });
2185:             return;
2186:           }
2187:           const error = new Error(stderr || stdout || `Git exited with code ${code}.`);
2188:           error.stdout = stdout;
2189:           error.stderr = stderr;
2190:           error.code = code;
2191:           error.git = candidate;
2192:           reject(error);
2193:         });
2194:
2195:         child.stdin.end(input);
2196:       });
2197:     } catch (error) {
2198:       lastError = error;
2199:       if (error.code === "ENOENT") continue;
2200:       throw error;
2201:     }
2202:   }
2203:   throw lastError || new Error("Git executable was not found.");
2204: }
```

storeGitCredentials (server.mjs, lines 2206-2237)

`storeGitCredentials` persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2206-2237 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `server.mjs` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `trim`, `Error`, `URL`, `toLowerCase`, `replace`, `runPublishGit`, `join`, and `runGitWithInput`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2209: `if (!cleanUsername && !cleanPassword) return { stored: false };`; line 2236: `return { stored: true, username: cleanUsername };`.

Important local values include `cleanUsername` at line 2207; `cleanPassword` at line 2208; `parsed` at line 2214; `pathName` at line 2225; `credentialInput` at line 2227. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2206: async function storeGitCredentials(remoteUrl, username, password) {
2207:   const cleanUsername = String(username || "").trim();
2208:   const cleanPassword = String(password || "");
2209:   if (!cleanUsername && !cleanPassword) return { stored: false };
2210:   if (!cleanUsername || !cleanPassword) {
2211:     throw new Error("Both username and password/token are required when credentials are provided.");
2212:   }
2213:
2214:   let parsed = null;
2215:   try {
2216:     parsed = new URL(remoteUrl);
2217:   } catch {
2218:     throw new Error("Username and password/token credentials require a GitHub HTTPS repository URL.");
2219:   }
```

```

2220:
2221:   if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {
2222:     throw new Error("Username and password/token credentials require a GitHub HTTPS repository URL.");
2223:   }
2224:
2225:   const pathName = parsed.pathname.replace(/^\/+/, "");
2226:   await runPublishGit(["config", "--local", "credential.useHttpPath", "true"]);
2227:   const credentialInput = [
2228:     "protocol=https",
2229:     "host=github.com",
2230:     `path=${pathName}`,
2231:     `username=${cleanUsername}`,
2232:     `password=${cleanPassword}`,
2233:     ""
2234:   ].join("\n");
2235:   await runGitWithInput(["credential", "approve"], credentialInput, { cwd: portfolioRoot });
2236:   return { stored: true, username: cleanUsername };
2237: }

```

validateCredentialPair (server.mjs, lines 2239-2246)

validateCredentialPair part of authentication, authorization, or credential checking. In this file, it appears around lines 2239-2246 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2245: return { cleanUsername, cleanPassword };

Important local values include cleanUsername at line 2240; cleanPassword at line 2241. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2239: function validateCredentialPair(username, password) {
2240:   const cleanUsername = String(username || "").trim();
2241:   const cleanPassword = String(password || "").trim();
2242:   if ((cleanUsername && !cleanPassword) || (!cleanUsername && cleanPassword)) {
2243:     throw new Error("Both username and password/token are required when credentials are provided.");
2244:   }
2245:   return { cleanUsername, cleanPassword };
2246: }

```

normalizeGitBranchName (server.mjs, lines 2248-2327)

normalizeGitBranchName part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2248-2327 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, includes, endsWith, startsWith, detectRemoteDefaultBranch, runPublishGit, match, originRemoteExists, setOriginRemote, checkoutPublishBranch, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 2261: return "";; line 2263: return branch;; line 2267: if (!remoteUrl) return "";; line 2271: return normalizeGitBranchName(match?.[1] || "");. There are 6 additional return statements in the function body.

Important local values include branch at line 2249; result at line 2269; match at line 2270; cleanBranch at line 2295; current at line 2296; checkBranch at line 2307. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2248: function normalizeGitBranchName(value = "") {
2249:   const branch = String(value || "").trim();
2250:   if (
2251:     !branch ||
2252:     branch.length > 180 ||
2253:     branch.includes(".") ||
2254:     branch.endsWith(".") ||
2255:     branch.includes("@") ||
2256:     /[\\:\s~^*\[\]\x00-\x1f]/.test(branch) ||
2257:     branch.startsWith("/") ||
2258:     branch.endsWith("/") ||
2259:     branch.includes("//")
2260:   ) {
2261:     return "";
2262:   }

```



```

2263:   return branch;
2264: }
2265:
2266: async function detectRemoteDefaultBranch(remoteUrl = "") {
2267:   if (!remoteUrl) return "";
2268:   try {
2269:     const result = await runPublishGit(["ls-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 });
2270:     const match = String(result.stdout || "").match(/^ref:\s+refs\/heads\/(.+?)\s+HEAD$/m);
2271:     return normalizeGitBranchName(match?.[1] || "");
2272:   } catch {
2273:     return "";
2274:   }
2275: }
2276:
2277: async function originRemoteExists() {
2278:   try {
2279:     await runPublishGit(["remote", "get-url", "origin"]);
2280:     return true;
2281:   } catch {
2282:     return false;
2283:   }
2284: }
2285:
2286: async function setOriginRemote(remoteUrl) {
2287:   if (await originRemoteExists()) {
2288:     await runPublishGit(["remote", "set-url", "origin", remoteUrl]);
2289:   } else {
2290:     await runPublishGit(["remote", "add", "origin", remoteUrl]);
2291:   }
2292: }
2293:
2294: async function checkoutPublishBranch(branch) {
2295:   const cleanBranch = normalizeGitBranchName(branch) || "main";
2296:   const current = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2297:   if (current === cleanBranch) return cleanBranch;
2298:   try {
2299:     await runPublishGit(["checkout", cleanBranch]);
2300:   } catch {
2301:     await runPublishGit(["checkout", "-B", cleanBranch]);
2302:   }
2303:   return cleanBranch;
2304: }
2305:
2306: async function verifyPublishWriteAccess(access = {}) {
2307:   const checkBranch = `omb-auth-check-${safeSegment(os.hostname(), "device")}-${Date.now()}`;
2308:   try {
2309:     await runPublishGit(["push", "--dry-run", "origin", `HEAD:refs/heads/${checkBranch}`], {
2310:       timeout: 60 * 1000
2311:     });
2312:     return {
2313:       method: "temporary-dry-run-ref",
2314:       ref: checkBranch
2315:     };
2316:   } catch (error) {
2317:     throw publishAccessError(
2318:       "GitHub authorization is required before this website can be changed.",
2319:       gitFailureText(error),
2320:       access
2321:     );
2322:   }
2323: }
2324:
2325: async function ensurePublishHeadForWriteCheck() {
2326:   try {
2327:     await runPublishGit(["rev-parse", "--verify", "HEAD"]);

```

detectRemoteDefaultBranch (server.mjs, lines 2266-2275)

detectRemoteDefaultBranch part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2266-2275 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runPublishGit, match, and normalizeGitBranchName. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2267: if (!remoteUrl) return "";; line 2271: return normalizeGitBranchName(match?.[1] || "");; line 2273: return "";

Important local values include result at line 2269; match at line 2270. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2266: async function detectRemoteDefaultBranch(remoteUrl = "") {

```

```

2267:   if (!remoteUrl) return "";
2268:   try {
2269:     const result = await runPublishGit(["ls-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 });
2270:     const match = String(result.stdout || "").match(/^ref:\s+refs\/heads\/(.+)\s+HEAD$/m);
2271:     return normalizeGitBranchName(match?.[1] || "");
2272:   } catch {
2273:     return "";
2274:   }
2275: }

```

originRemoteExists (server.mjs, lines 2277-2284)

originRemoteExists part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2277-2284 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runPublishGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2280: return true;; line 2282: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2277: async function originRemoteExists() {
2278:   try {
2279:     await runPublishGit(["remote", "get-url", "origin"]);
2280:     return true;
2281:   } catch {
2282:     return false;
2283:   }
2284: }

```

setOriginRemote (server.mjs, lines 2286-2292)

setOriginRemote part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2286-2292 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references originRemoteExists, and runPublishGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2286: async function setOriginRemote(remoteUrl) {
2287:   if (await originRemoteExists()) {
2288:     await runPublishGit(["remote", "set-url", "origin", remoteUrl]);
2289:   } else {
2290:     await runPublishGit(["remote", "add", "origin", remoteUrl]);
2291:   }
2292: }

```

checkoutPublishBranch (server.mjs, lines 2294-2304)

checkoutPublishBranch part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2294-2304 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeGitBranchName, runPublishGit, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2297: if (current === cleanBranch) return cleanBranch;; line 2303: return cleanBranch;.

Important local values include cleanBranch at line 2295; current at line 2296. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2294: async function checkoutPublishBranch(branch) {
2295:   const cleanBranch = normalizeGitBranchName(branch) || "main";
2296:   const current = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2297:   if (current === cleanBranch) return cleanBranch;
2298:   try {
2299:     await runPublishGit(["checkout", cleanBranch]);
2300:   } catch {
2301:     await runPublishGit(["checkout", "-B", cleanBranch]);
2302:   }
2303:   return cleanBranch;
2304: }

```

verifyPublishWriteAccess (server.mjs, lines 2306-2323)

verifyPublishWriteAccess part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2306-2323 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references safeSegment, hostname, now, runPublishGit, publishAccessError, and gitFailureText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2312: return {.

Important local values include checkBranch at line 2307. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2306: async function verifyPublishWriteAccess(access = {}) {
2307:   const checkBranch = `omb-auth-check-${safeSegment(os.hostname(), "device")}-${Date.now()}`;
2308:   try {
2309:     await runPublishGit(["push", "--dry-run", "origin", `HEAD:refs/heads/${checkBranch}`], {
2310:       timeout: 60 * 1000
2311:     });
2312:     return {
2313:       method: "temporary-dry-run-ref",
2314:       ref: checkBranch
2315:     };
2316:   } catch (error) {
2317:     throw publishAccessError(
2318:       "GitHub authorization is required before this website can be changed.",
2319:       gitFailureText(error),
2320:       access
2321:     );
2322:   }
2323: }

```

ensurePublishHeadForWriteCheck (server.mjs, lines 2325-2342)

ensurePublishHeadForWriteCheck part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2325-2342 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runPublishGit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2328: return { created: false };; line 2340: return { created: true };.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2325: async function ensurePublishHeadForWriteCheck() {
2326:   try {
2327:     await runPublishGit(["rev-parse", "--verify", "HEAD"]);
2328:     return { created: false };
2329:   } catch {
2330:     await runPublishGit([
2331:       "-c",
2332:       "user.name=OMB Portfolio Builder",
2333:       "-c",
2334:       "user.email=omb-builder@localhost",
2335:       "commit",
2336:       "--allow-empty",
2337:       "-m",
2338:       "Initialize publish authorization check"
2339:     ]);
2340:     return { created: true };

```

```
2341:   }  
2342: }
```

synchronizePublishBranchFromRemote (server.mjs, lines 2344-2373)

synchronizePublishBranchFromRemote part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2344-2373 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeGitBranchName, runPublishGit, trim, checkoutPublishBranch, publishAccessError, and gitFailureText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2351: return {; line 2361: return {.

Important local values include cleanBranch at line 2345; remoteBranch at line 2347. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2344: async function synchronizePublishBranchFromRemote(branch, access = {}) {  
2345:   const cleanBranch = normalizeGitBranchName(branch) || "main";  
2346:   try {  
2347:     const remoteBranch = await runPublishGit(["ls-remote", "--heads", "origin", cleanBranch], {  
2348:       timeout: 45 * 1000  
2349:     });  
2350:     if (!String(remoteBranch.stdout || "").trim()) {  
2351:       return {  
2352:         branch: cleanBranch,  
2353:         remoteBranchExists: false,  
2354:         synchronized: false  
2355:       };  
2356:     }  
2357:   }  
2358:   await runPublishGit(["fetch", "origin", cleanBranch], { timeout: 2 * 60 * 1000 });  
2359:   await checkoutPublishBranch(cleanBranch);  
2360:   await runPublishGit(["reset", "--hard", `origin/${cleanBranch}`], { timeout: 60 * 1000 });  
2361:   return {  
2362:     branch: cleanBranch,  
2363:     remoteBranchExists: true,  
2364:     synchronized: true  
2365:   };  
2366: } catch (error) {  
2367:   throw publishAccessError(  
2368:     "The selected website target could not be synchronized from GitHub.",  
2369:     gitFailureText(error),  
2370:     { ...access, branch: cleanBranch }  
2371:   );  
2372: }  
2373: }
```

capturePublishTargetState (server.mjs, lines 2375-2403)

capturePublishTargetState part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2375-2403 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runPublishGit, trim, readFile, and resolveInsidePortfolioRoot. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2402: return snapshot;.

Important local values include snapshot at line 2376. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2375: async function capturePublishTargetState() {  
2376:   const snapshot = {  
2377:     branch: "",  
2378:     customDomainExists: false,  
2379:     customDomainText: "",  
2380:     remote: "",  
2381:     remoteExists: false  
2382:   };  
2383:   try {  
2384:     snapshot.remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
```

```

2385:     snapshot.remoteExists = true;
2386:   } catch {
2387:     snapshot.remote = "";
2388:     snapshot.remoteExists = false;
2389:   }
2390:   try {
2391:     snapshot.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2392:   } catch {
2393:     snapshot.branch = "";
2394:   }
2395:   try {
2396:     snapshot.customDomainText = await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8");
2397:     snapshot.customDomainExists = true;
2398:   } catch {
2399:     snapshot.customDomainText = "";
2400:     snapshot.customDomainExists = false;
2401:   }
2402:   return snapshot;
2403: }

```

restorePublishTargetState (server.mjs, lines 2405-2429)

restorePublishTargetState part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2405-2429 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setOriginRemote, originRemoteExists, runPublishGit, checkoutPublishBranch, writeFile, resolveInsidePortfolioRoot, and rm. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2405: async function restorePublishTargetState(snapshot = {}) {
2406:   try {
2407:     if (snapshot.remoteExists && snapshot.remote) {
2408:       await setOriginRemote(snapshot.remote);
2409:     } else if (await originRemoteExists()) {
2410:       await runPublishGit(["remote", "remove", "origin"]);
2411:     }
2412:   } catch {
2413:     // Best-effort rollback; preserve the original error for the caller.
2414:   }
2415:   try {
2416:     if (snapshot.branch) await checkoutPublishBranch(snapshot.branch);
2417:   } catch {
2418:     // Best-effort rollback; preserve the original error for the caller.
2419:   }
2420:   try {
2421:     if (snapshot.customDomainExists) {
2422:       await writeFile(resolveInsidePortfolioRoot("CNAME"), snapshot.customDomainText, "utf8");
2423:     } else {
2424:       await rm(resolveInsidePortfolioRoot("CNAME"), { force: true });
2425:     }
2426:   } catch {
2427:     // Best-effort rollback; preserve the original error for the caller.
2428:   }
2429: }

```

writeTargetCustomDomain (server.mjs, lines 2431-2439)

writeTargetCustomDomain persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2431-2439 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resolveInsidePortfolioRoot, samePath, push, resolveInsideRoot, writeFile, and rm. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include targetPaths at line 2432. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2431: async function writeTargetCustomDomain(domain, customDomainProvided) {

```

```

2432:   const targetPaths = [resolveInsidePortfolioRoot("CNAME")];
2433:   if (!samePath(root, portfolioRoot)) targetPaths.push(resolveInsideRoot("CNAME"));
2434:   if (customDomainProvided && domain) {
2435:     for (const targetPath of targetPaths) await writeFile(targetPath, `${domain}\n`, "utf8");
2436:   } else if (customDomainProvided && !domain) {
2437:     for (const targetPath of targetPaths) await rm(targetPath, { force: true });
2438:   }
2439: }

```

ensureGitRepository (server.mjs, lines 2441-2454)

ensureGitRepository part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2441-2454 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mkdir, runPublishGit, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2445: if (insideWorkTree === "true") return;.

Important local values include insideWorkTree at line 2444; branch at line 2450. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2441: async function ensureGitRepository() {
2442:   await mkdir(portfolioRoot, { recursive: true });
2443:   try {
2444:     const insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
2445:     if (insideWorkTree === "true") return;
2446:   } catch {
2447:     await runPublishGit(["init"]);
2448:   }
2449:
2450:   const branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2451:   if (!branch) {
2452:     await runPublishGit(["checkout", "-B", "main"]);
2453:   }
2454: }

```

configurePublishTarget (server.mjs, lines 2456-2486)

This function accepts a publishing target, validates it, configures the local publish mirror repository, verifies write access, and saves the target only after authentication succeeds.

Publishing is dangerous if the target is wrong. This function protects the portfolio by making target setup explicit and verified before Apply to site can push changes.

It calls or references call, validatePublishRemoteUrl, validateCustomDomain, validateCredentialPair, ensureGitRepository, setOriginRemote, detectRemoteDefaultBranch, checkoutPublishBranch, writeTargetCustomDomain, getPublishTargetInfo, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2481: return {.

Important local values include customDomainProvided at line 2463; remoteUrl at line 2464; domain at line 2465; defaultBranch at line 2472; currentRemote at line 2478; credentials at line 2479; target at line 2480. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2456: async function configurePublishTarget(options = {}) {
2457:   const {
2458:     repositoryUrl = "",
2459:     customDomain = "",
2460:     authUsername = "",
2461:     authPassword = ""
2462:   } = options;
2463:   const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
2464:   const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
2465:   const domain = validateCustomDomain(customDomain);
2466:   validateCredentialPair(authUsername, authPassword);
2467:
2468:   await ensureGitRepository();
2469:
2470:   if (remoteUrl) {
2471:     await setOriginRemote(remoteUrl);
2472:     const defaultBranch = await detectRemoteDefaultBranch(remoteUrl);
2473:     if (defaultBranch) await checkoutPublishBranch(defaultBranch);
2474:   }

```

```

2475:
2476:   await writeTargetCustomDomain(domain, customDomainProvided);
2477:
2478:   const currentRemote = remoteUrl || (await getPublishTargetInfo()).remote;
2479:   const credentials = await storeGitCredentials(currentRemote, authUsername, authPassword);
2480:   const target = await getPublishTargetInfo();
2481:   return {
2482:     ...target,
2483:     credentialsStored: credentials.stored,
2484:     credentialUsername: credentials.username || ""
2485:   };
2486: }

```

readPublishAuthCache (server.mjs, lines 2488-2494)

readPublishAuthCache loads data from disk, network, browser storage, or a service. In this file, it appears around lines 2488-2494 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, and readFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2490: return JSON.parse(await readFile(publishAuthCachePath, "utf8")); line 2492: return null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2488: async function readPublishAuthCache() {
2489:   try {
2490:     return JSON.parse(await readFile(publishAuthCachePath, "utf8"));
2491:   } catch {
2492:     return null;
2493:   }
2494: }

```

writePublishAuthCache (server.mjs, lines 2496-2535)

writePublishAuthCache persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2496-2535 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references readPublishAuthCache, getTime, isArray, map, parsePublishAuthTimestamp, filter, isFinite, toISOString, publishAuthCacheScope, writeFile, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2534: return cache;.

Important local values include previousCache at line 2497; checkedAt at line 2498; checkedAtMs at line 2499; sameTarget at line 2500; previousHistory at line 2504; successTimestamps at line 2507; successHistory at line 2514; successCountLastWeek at line 2515; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2496: async function writePublishAuthCache(access = {}) {
2497:   const previousCache = await readPublishAuthCache();
2498:   const checkedAt = new Date();
2499:   const checkedAtMs = checkedAt.getTime();
2500:   const sameTarget = previousCache
2501:     && previousCache.remote === access.remote
2502:     && previousCache.branch === access.branch
2503:     && previousCache.repository === access.repository;
2504:   const previousHistory = sameTarget && Array.isArray(previousCache.successHistory)
2505:     ? previousCache.successHistory
2506:     : [];
2507:   const successTimestamps = [
2508:     ...previousHistory
2509:       .map((entry) => parsePublishAuthTimestamp(entry))
2510:       .filter((timestamp) => Number.isFinite(timestamp))
2511:       .filter((timestamp) => checkedAtMs - timestamp < publishAuthExtendedTtlMs),
2512:     checkedAtMs
2513:   ];
2514:   const successHistory = successTimestamps.map((timestamp) => new Date(timestamp).toISOString());
2515:   const successCountLastWeek = successTimestamps
2516:     .filter((timestamp) => checkedAtMs - timestamp < publishAuthHistoryWindowMs)
2517:     .length;
2518:   const extendedTrust = successCountLastWeek > publishAuthExtendedThreshold;

```

```

2519: const ttlMs = extendedTrust ? publishAuthExtendedTtlMs : publishAuthCacheTtlMs;
2520: const cache = {
2521:   branch: access.branch || "",
2522:   checkedAt: checkedAt.toISOString(),
2523:   expiresAt: new Date(checkedAtMs + ttlMs).toISOString(),
2524:   extendedTrust,
2525:   remote: access.remote || "",
2526:   repository: access.repository || "",
2527:   scope: publishAuthCacheScope(),
2528:   successCountLastWeek,
2529:   successHistory,
2530:   trustDays: extendedTrust ? 30 : 1
2531: };
2532: await writeFile(publishAuthCachePath, `${JSON.stringify(cache, null, 2)}\n`, "utf8");
2533: await chmod(publishAuthCachePath, 0o600).catch(() => {});
2534: return cache;
2535: }

```

publishAuthCacheScope (server.mjs, lines 2537-2552)

publishAuthCacheScope part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2537-2552 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references userInfo, createHash, update, hostname, join, and digest. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2544: return createHash("sha256").

Important local values include username at line 2538. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2537: function publishAuthCacheScope() {
2538:   let username = process.env.USERNAME || process.env.USER || "";
2539:   try {
2540:     username = os.userInfo().username || username;
2541:   } catch {
2542:     // Environment username is a sufficient fallback for cache scoping.
2543:   }
2544:   return createHash("sha256")
2545:     .update([
2546:       os.hostname(),
2547:       username,
2548:       root,
2549:       portfolioRoot
2550:     ].join("\n"))
2551:     .digest("hex");
2552: }

```

parsePublishAuthTimestamp (server.mjs, lines 2554-2559)

parsePublishAuthTimestamp part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2554-2559 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, isFinite, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2556: if (Number.isFinite(direct)) return direct;; line 2558: return Date.parse(normalized);.

Important local values include direct at line 2555; normalized at line 2557. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2554: function parsePublishAuthTimestamp(value = "") {
2555:   const direct = Date.parse(String(value || ""));
2556:   if (Number.isFinite(direct)) return direct;
2557:   const normalized = String(value || "").replace(/(\.\d{3})\d+(Z|[-+]\d{2}:?\d{2})$/i, "$1$2");
2558:   return Date.parse(normalized);
2559: }

```

publishAuthCacheExpiresAt (server.mjs, lines 2561-2567)

publishAuthCacheExpiresAt part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2561-2567 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parsePublishAuthTimestamp, isFinite, and toISOString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2563: if (Number.isFinite(explicitExpiresAt)) return new Date(explicitExpiresAt).toISOString();; line 2565: if (!Number.isFinite(checkedException)) return "";; line 2566: return new Date(checkedException + publishAuthCacheTtlMs).toISOString();.

Important local values include explicitExpiresAt at line 2562; checkedAt at line 2564. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2561: function publishAuthCacheExpiresAt(cache) {
2562:   const explicitExpiresAt = parsePublishAuthTimestamp(cache?.expiresAt || "");
2563:   if (Number.isFinite(explicitExpiresAt)) return new Date(explicitExpiresAt).toISOString();
2564:   const checkedAt = parsePublishAuthTimestamp(cache?.checkedAt || "");
2565:   if (!Number.isFinite(checkedAt)) return "";
2566:   return new Date(checkedAt + publishAuthCacheTtlMs).toISOString();
2567: }
```

publishAuthCachelsFresh (server.mjs, lines 2569-2576)

This function decides whether a saved publishing authorization cache still applies to the current target. It checks scope, repository, branch, timestamp, and trust rules before reusing the cached authorization.

The user should not have to authenticate on every click, but the app also cannot trust stale credentials forever. This function is the balance between convenience and publishing safety.

It calls or references publishAuthCacheScope, parse, publishAuthCacheExpiresAt, isFinite, and now. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 2570: if (!cache || !access) return false;; line 2571: if (cache.remote !== access.remote || cache.branch !== access.branch || cache.repository !== access.repository) return false;; line 2572: if (cache.scope !== publishAuthCacheScope()) return false;; line 2574: if (!Number.isFinite(expiresAt)) return false;. There are 1 additional return statements in the function body.

Important local values include expiresAt at line 2573. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2569: function publishAuthCacheIsFresh(cache, access) {
2570:   if (!cache || !access) return false;
2571:   if (cache.remote !== access.remote || cache.branch !== access.branch || cache.repository !==
access.repository) return false;
2572:   if (cache.scope !== publishAuthCacheScope()) return false;
2573:   const expiresAt = Date.parse(publishAuthCacheExpiresAt(cache));
2574:   if (!Number.isFinite(expiresAt)) return false;
2575:   return Date.now() < expiresAt;
2576: }
```

assertPublishAccess (server.mjs, lines 2578-2669)

This function is the gatekeeper before website-changing operations. It checks whether target authentication exists and is still fresh enough, and it returns a clear access error when publishing is not allowed.

Apply to site should only run when the current user has proven write access. This function prevents accidental pushes from an unauthenticated or mismatched target state.

It calls or references runPublishGit, trim, publishAccessError, gitFailureText, remoteUrlForDisplay, workspaceHasCompatibleSiteFiles, parseGitHubRemote, readPublishAuthCache, publishAuthCacheIsFresh, publishAuthCacheExpiresAt, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2639: return {}; line 2656: return {}.

Important local values include insideWorkTree at line 2579; remote at line 2598; branch at line 2599; parsedRemote at line 2627; repository at line 2628; access at line 2630; cached at line 2637; remoteSync at line 2651; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2578: async function assertPublishAccess(options = {}) {
2579:   let insideWorkTree = "";
2580:   try {
2581:     insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
2582:   } catch (error) {
2583:     throw publishAccessError(
2584:       "This workspace is not connected to a Git repository.",
2585:       gitFailureText(error),
2586:       { repository: defaultSiteRepository }
2587:     );
2588:   }
2589:
2590:   if (insideWorkTree !== "true") {
2591:     throw publishAccessError(
2592:       "This workspace is local-only and cannot publish yet.",
2593:       "Clone or associate a GitHub Pages/static-site repository before applying to site.",
2594:       { repository: defaultSiteRepository }
2595:     );
2596:   }
2597:
2598:   let remote = "";
2599:   let branch = "";
2600:   try {
2601:     remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
2602:     branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2603:   } catch (error) {
2604:     throw publishAccessError(
2605:       "A publish remote or branch is missing.",
2606:       gitFailureText(error),
2607:       { repository: defaultSiteRepository }
2608:     );
2609:   }
2610:
2611:   if (!remote || !branch) {
2612:     throw publishAccessError(
2613:       "A publish remote or branch is missing.",
2614:       "Set the origin remote and use a named branch before applying to site.",
2615:       { remote: remoteUrlForDisplay(remote), branch }
2616:     );
2617:   }
2618:
2619:   if (options.requireCompatible !== false && !await workspaceHasCompatibleSiteFiles()) {
2620:     throw publishAccessError(
2621:       "This repository does not look like a compatible static portfolio website.",
2622:       "The workspace must include index.html, styles.css, and script.js before it can be used as a
publish target.",
2623:       { remote: remoteUrlForDisplay(remote), branch }
2624:     );
2625:   }
2626:
2627:   const parsedRemote = parseGitHubRemote(remote);
2628:   const repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` :
remoteUrlForDisplay(remote);
2629:
2630:   const access = {
2631:     branch,
2632:     remote: remoteUrlForDisplay(remote),
2633:     repository,
2634:     authorizationChecked: false
2635:   };
2636:
2637:   const cached = await readPublishAuthCache();
2638:   if (!options.force && publishAuthCacheIsFresh(cached, access)) {
2639:     return {
2640:       ...access,
2641:       authorizationCached: true,
2642:       authorizationChecked: true,
2643:       checkedAt: cached.checkedAt,
2644:       expiresAt: publishAuthCacheExpiresAt(cached),
2645:       extendedTrust: Boolean(cached.extendedTrust),
2646:       successCountLastWeek: cached.successCountLastWeek || 0,
2647:       trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1)
2648:     };
2649:   }
2650:
2651:   const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
2652:   const localHead = await ensurePublishHeadForWriteCheck();
2653:   const writeCheck = await verifyPublishWriteAccess(access);
2654:
2655:   const authCache = await writePublishAuthCache(access);
2656:   return {
2657:     ...access,
2658:     remoteSync,
2659:     localHead,
2660:     writeCheck,
2661:     authorizationChecked: true,
2662:     authorizationCached: false,
2663:     checkedAt: authCache.checkedAt,
2664:     expiresAt: authCache.expiresAt,
2665:     extendedTrust: Boolean(authCache.extendedTrust),
```

```

2666:     successCountLastWeek: authCache.successCountLastWeek || 0,
2667:     trustDays: authCache.trustDays || 1
2668:   };
2669: }

```

syncFromPublishTarget (server.mjs, lines 2671-2745)

This function loads compatible website files from the configured GitHub target into the local builder workspace. It pulls or fetches the target state, reads supported portfolio files, and updates local fields when the target is valid.

This is what lets the same owner move to another machine, authenticate, and pull the current website into the builder instead of starting blank.

It calls or references `assertPublishAccess`, `runPublishGit`, `trim`, `mkdtemp`, `join`, `tmpdir`, `runGit`, `fsAccess`, `push`, `includes`, and 11 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2735: `return {`.

Important local values include `publishAccess` at line 2672; `branch` at line 2673; `remote` at line 2674; `importRoot` at line 2675; `cloneRoot` at line 2676; `importPaths` at line 2677; `availablePaths` at line 2694; `backupRoot` at line 2712; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2671: async function syncFromPublishTarget() {
2672:   const publishAccess = await assertPublishAccess({ requireCompatible: false });
2673:   const branch = publishAccess.branch || "main";
2674:   const remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
2675:   const importRoot = await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-"));
2676:   const cloneRoot = path.join(importRoot, "target");
2677:   const importPaths = [
2678:     "projects.json",
2679:     "assets",
2680:     "docs",
2681:     "Backgrounds",
2682:     "CNAME",
2683:     ".nojekyll",
2684:     "robots.txt",
2685:     "sitemap.xml"
2686:   ];
2687:
2688:   try {
2689:     await runGit(["clone", "--depth", "1", "--branch", branch, remote, cloneRoot], {
2690:       cwd: importRoot,
2691:       timeout: 3 * 60 * 1000
2692:     });
2693:
2694:     const availablePaths = [];
2695:     for (const importPath of importPaths) {
2696:       try {
2697:         await fsAccess(path.join(cloneRoot, importPath));
2698:         availablePaths.push(importPath);
2699:       } catch {
2700:         // Optional path is absent in the selected site repository.
2701:       }
2702:     }
2703:
2704:     if (!availablePaths.includes("projects.json")) {
2705:       throw publishAccessError(
2706:         "The selected repository does not contain a projects.json portfolio catalog.",
2707:         "The target repository must contain a compatible OMB Portfolio Builder catalog before it can be
imported.",
2708:         publishAccess
2709:       );
2710:     }
2711:
2712:     const backupRoot = resolveInsideRoot(
2713:       ".omb-backups",
2714:       `load-target-${new Date().toISOString().replace(/[:.]/g, "-")}`
2715:     );
2716:     await mkdir(backupRoot, { recursive: true });
2717:
2718:     for (const importPath of availablePaths) {
2719:       const sourcePath = path.join(cloneRoot, importPath);
2720:       const targetPath = resolveInsideRoot(importPath);
2721:       try {
2722:         await fsAccess(targetPath);
2723:         await cp(targetPath, path.join(backupRoot, importPath), { recursive: true, force: true });
2724:         await rm(targetPath, { recursive: true, force: true });
2725:       } catch {
2726:         // Nothing local to back up for this path.
2727:       }
2728:       await cp(sourcePath, targetPath, { recursive: true, force: true });
2729:     }
2730:
2731:     const importedCatalog = await readFile(path.join(cloneRoot, "projects.json"), "utf8");

```

```

2732:     await writeFile(draftPath, importedCatalog, "utf8");
2733:     const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: false });
2734:
2735:     return {
2736:       ...publishAccess,
2737:       imported: availablePaths,
2738:       portfolioSync,
2739:       backup: path.relative(root, backupRoot),
2740:       draftUpdated: true
2741:     };
2742:   } finally {
2743:     await rm(importRoot, { recursive: true, force: true });
2744:   }
2745: }

```

authenticateGitHubForTarget (server.mjs, lines 2747-2843)

authenticateGitHubForTarget part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2747-2843 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call, validatePublishRemoteUrl, validateCustomDomain, validateCredentialPair, publishAccessError, getPublishTargetInfo, ensureGitRepository, capturePublishTargetState, setOriginRemote, getGitStatus, and 10 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2794: return {}; line 2831: return {}.

Important local values include customDomainProvided at line 2754; remoteUrl at line 2755; domain at line 2756; credentials at line 2757; snapshot at line 2767; status at line 2771; preliminaryTarget at line 2772; targetBranch at line 2781; plus 7 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2747: async function authenticateGitHubForTarget(options = {}) {
2748:   const {
2749:     repositoryUrl = "",
2750:     customDomain = "",
2751:     authUsername = "",
2752:     authPassword = ""
2753:   } = options;
2754:   const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
2755:   const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
2756:   const domain = validateCustomDomain(customDomain);
2757:   const credentials = validateCredentialPair(authUsername, authPassword);
2758:   if (!remoteUrl) {
2759:     throw publishAccessError(
2760:       "A GitHub repository URL is required before authentication.",
2761:       "Enter the GitHub Pages or compatible static-site repository URL, then click Authenticate target.",
2762:       await getPublishTargetInfo()
2763:     );
2764:   }
2765:
2766:   await ensureGitRepository();
2767:   const snapshot = await capturePublishTargetState();
2768:
2769:   try {
2770:     await setOriginRemote(remoteUrl);
2771:     const status = await getGitStatus();
2772:     const preliminaryTarget = await getPublishTargetInfo();
2773:     if (!status.git.ok) {
2774:       throw publishAccessError(
2775:         "Git for Windows is required for publishing.",
2776:         "Install Git for Windows, then reopen the builder and authenticate again.",
2777:         { ...preliminaryTarget, system: status }
2778:       );
2779:     }
2780:
2781:     const targetBranch = await detectRemoteDefaultBranch(remoteUrl) || "main";
2782:     await checkoutPublishBranch(targetBranch);
2783:
2784:     const targetBeforeAuth = await getPublishTargetInfo();
2785:     const cached = await readPublishAuthCache();
2786:     const cachedAccess = {
2787:       branch: targetBeforeAuth.branch,
2788:       remote: targetBeforeAuth.remote,
2789:       repository: targetBeforeAuth.repository
2790:     };
2791:     if (!credentials.cleanUsername && publishAuthCacheIsFresh(cached, cachedAccess)) {
2792:       await writeTargetCustomDomain(domain, customDomainProvided);
2793:       const target = await getPublishTargetInfo();
2794:       return {
2795:         ...cachedAccess,
2796:         branch: target.branch || cachedAccess.branch,

```

```

2797:         authorizationCached: true,
2798:         authorizationChecked: true,
2799:         checkedAt: cached.checkedAt,
2800:         expiresAt: publishAuthCacheExpiresAt(cached),
2801:         extendedTrust: Boolean(cached.extendedTrust),
2802:         successCountLastWeek: cached.successCountLastWeek || 0,
2803:         trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1),
2804:         credentialsStored: false,
2805:         credentialUsername: "",
2806:         system: status,
2807:         target
2808:     };
2809: }
2810:
2811: if (!status.credentialManager.ok && !credentials.cleanUsername) {
2812:     throw publishAccessError(
2813:         "Git Credential Manager is required for guided GitHub sign-in.",
2814:         "Install or repair Git for Windows with Git Credential Manager enabled, or provide a GitHub
username and personal access token.",
2815:         { ...targetBeforeAuth, system: status }
2816:     );
2817: }
2818:
2819: const storedCredentials = await storeGitCredentials(remoteUrl, authUsername, authPassword);
2820: if (!storedCredentials.stored) {
2821:     await runGit(["credential-manager", "github", "login"], {
2822:         cwd: portfolioRoot,
2823:         timeout: 5 * 60 * 1000,
2824:         windowsHide: false
2825:     });
2826: }
2827:
2828: const access = await assertPublishAccess({ force: true, requireCompatible: false });
2829: await writeTargetCustomDomain(domain, customDomainProvided);
2830: const target = await getPublishTargetInfo();
2831: return {
2832:     ...access,
2833:     branch: target.branch || access.branch,
2834:     credentialsStored: storedCredentials.stored,
2835:     credentialUsername: storedCredentials.username || "",
2836:     system: await getGitStatus(),
2837:     target
2838: };
2839: } catch (error) {
2840:     await restorePublishTargetState(snapshot);
2841:     throw error;
2842: }
2843: }

```

installGitForWindows (server.mjs, lines 2845-2868)

installGitForWindows part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2845-2868 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references runOptionalCommand, spawn, and unref. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2849: return {}; line 2863: return {}.

Important local values include winget at line 2846; downloadUrl at line 2847; child at line 2856. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2845: async function installGitForWindows() {
2846:     const winget = await runOptionalCommand("winget", ["--version"]);
2847:     const downloadUrl = "https://git-scm.com/download/win";
2848:     if (!winget.ok) {
2849:         return {
2850:             launched: false,
2851:             downloadUrl,
2852:             message: "winget was not found. Open the Git for Windows download page and install Git with Git
Credential Manager enabled."
2853:         };
2854:     }
2855:
2856:     const child = spawn("winget", ["install", "--id", "Git.Git", "-e", "--source", "winget"], {
2857:         detached: true,
2858:         shell: true,
2859:         stdio: "ignore",
2860:         windowsHide: false
2861:     });
2862:     child.unref();
2863:     return {

```

```

2864:     launched: true,
2865:     downloadUrl,
2866:     message: "The Git for Windows installer was launched through winget. Follow the installer prompts,
then reopen the builder."
2867:   };
2868: }

```

getUpdateInfo (server.mjs, lines 2870-2906)

getUpdateInfo part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2870-2906 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, Error, json, replace, find, has, and compareVersions. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2884: return {; line 2899: return {.

Important local values include currentVersion at line 2871; owner at line 2872; repo at line 2873; response at line 2875; release at line 2879; latestVersion at line 2880; installer at line 2881; portable at line 2882; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2870: async function getUpdateInfo() {
2871:   const currentVersion = process.env.OMB_APP_VERSION || packageJson.version || "0.0.0";
2872:   const owner = "otienomaurice";
2873:   const repo = "omb-portfolio-builder";
2874:   try {
2875:     const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
2876:       headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
2877:     });
2878:     if (!response.ok) throw new Error(`GitHub returned ${response.status}`);
2879:     const release = await response.json();
2880:     const latestVersion = String(release.tag_name || "").replace(/^builder-v/i, "");
2881:     const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
2882:     const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
2883:     const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
2884:     return {
2885:       ok: true,
2886:       currentVersion,
2887:       latestVersion,
2888:       updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
2889:       updateBlocked,
2890:       blockedReason: updateBlocked
2891:         ? `Builder ${latestVersion} is skipped because its installer can hang while running the old
uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
2892:         : "",
2893:       releaseUrl: release.html_url || "",
2894:       installerUrl: installer?.browser_download_url || "",
2895:       installerName: installer?.name || "",
2896:       portableUrl: portable?.browser_download_url || ""
2897:     };
2898:   } catch (error) {
2899:     return {
2900:       ok: false,
2901:       currentVersion,
2902:       latestVersion: currentVersion,
2903:       updateAvailable: false,
2904:       error: error.message || "Could not check for updates."
2905:     };
2906:   }

```

installer (server.mjs, lines 2881-2897)

installer part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2881-2897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, has, and compareVersions. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2884: return {.

Important local values include installer at line 2881; portable at line 2882; updateBlocked at line 2883. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2881:     const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
2882:     const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
2883:     const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
2884:     return {
2885:       ok: true,
2886:       currentVersion,
2887:       latestVersion,
2888:       updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
2889:       updateBlocked,
2890:       blockedReason: updateBlocked
2891:       ? `Builder ${latestVersion} is skipped because its installer can hang while running the old
uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
2892:       : "",
2893:       releaseUrl: release.html_url || "",
2894:       installerUrl: installer?.browser_download_url || "",
2895:       installerName: installer?.name || "",
2896:       portableUrl: portable?.browser_download_url || ""
2897:     };
```

portable (server.mjs, lines 2882-2897)

portable part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2882-2897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, has, and compareVersions. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2884: return {.

Important local values include portable at line 2882; updateBlocked at line 2883. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2882:     const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
2883:     const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
2884:     return {
2885:       ok: true,
2886:       currentVersion,
2887:       latestVersion,
2888:       updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
2889:       updateBlocked,
2890:       blockedReason: updateBlocked
2891:       ? `Builder ${latestVersion} is skipped because its installer can hang while running the old
uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
2892:       : "",
2893:       releaseUrl: release.html_url || "",
2894:       installerUrl: installer?.browser_download_url || "",
2895:       installerName: installer?.name || "",
2896:       portableUrl: portable?.browser_download_url || ""
2897:     };
```

getBuilderReleaseDownloadReport (server.mjs, lines 2909-2914)

getBuilderReleaseDownloadReport part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2909-2914 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include owner at line 2910; repo at line 2911; response at line 2912. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2909: async function getBuilderReleaseDownloadReport() {
2910:   const owner = "otienomaurice";
2911:   const repo = "omb-portfolio-builder";
2912:   const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
2913:     headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
2914:   });
```

```
2914: });
```

assets (server.mjs, lines 2917-2923)

assets part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2917-2923 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include assets at line 2917. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2917: const assets = (release.assets || []).map((asset) => ({
2918:   name: asset.name || "",
2919:   downloadCount: Number(asset.download_count || 0),
2920:   sizeBytes: Number(asset.size || 0),
2921:   updatedAt: asset.updated_at || "",
2922:   url: asset.browser_download_url || ""
2923: }));
```

getSecurityReport (server.mjs, lines 2933-2977)

getSecurityReport part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2933-2977 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references allSettled, getBuilderReleaseDownloadReport, readPublishAuthCache, getPublishTargetInfo, toISOString, publishAuthCachelsFresh, and publishAuthCacheExpiresAt. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2941: return {.

Important local values include authCache at line 2939; target at line 2940. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2933: async function getSecurityReport() {
2934:   const [downloadResult, authCacheResult, targetResult] = await Promise.allSettled([
2935:     getBuilderReleaseDownloadReport(),
2936:     readPublishAuthCache(),
2937:     getPublishTargetInfo()
2938:   ]);
2939:   const authCache = authCacheResult.status === "fulfilled" ? authCacheResult.value : null;
2940:   const target = targetResult.status === "fulfilled" ? targetResult.value : null;
2941:   return {
2942:     ok: true,
2943:     generatedAt: new Date().toISOString(),
2944:     appDownloads: downloadResult.status === "fulfilled"
2945:       ? { ok: true, ...downloadResult.value }
2946:       : { ok: false, error: downloadResult.reason?.message || "GitHub release download counts were not
available." },
2947:     publishingAuth: {
2948:       targetRepository: target?.repository || "",
2949:       branch: target?.branch || "",
2950:       authenticated: Boolean(authCache && publishAuthCacheIsFresh(authCache, {
2951:         remote: target?.remote || "",
2952:         repository: target?.repository || "",
2953:         branch: target?.branch || ""
2954:       })),
2955:       expiresAt: publishAuthCacheExpiresAt(authCache) || "",
2956:       trustDays: Number(authCache?.trustDays || 0),
2957:       successCountLastWeek: Number(authCache?.successCountLastWeek || 0)
2958:     },
2959:     websiteAccess: {
2960:       available: false,
2961:       summary: "Visitor counts and visitor IP addresses are not available from a static GitHub Pages site
or GitHub release download counts.",
2962:       recommendedSetup: [
2963:         "Proxy the custom domain through Cloudflare instead of DNS-only.",
2964:         "Enable Cloudflare Web Analytics for aggregate page-view counts.",
```



```

2965:         "Use Cloudflare Logpush, Analytics Engine, or a Worker-backed endpoint if you need IP-level
security logs.",
2966:         "Publish a privacy notice before collecting raw IP addresses or persistent identifiers."
2967:     ],
2968: },
2969: securityControls: [
2970:     "Local builder write APIs are restricted to local requests.",
2971:     "Publishing target details are only returned to the local machine.",
2972:     "GitHub publishing authorization is cached for one day by default, or longer after repeated
successful authorizations.",
2973:     "The desktop app runs with Electron nodeIntegration disabled, contextIsolation enabled, and sandbox
enabled.",
2974:     "Deployable website security headers and security.txt metadata are included in the publishable site
files."
2975: ],
2976: };
2977: }

```

safeUpdateFileSegment (server.mjs, lines 2979-2984)

safeUpdateFileSegment validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2979-2984 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2980: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2979: function safeUpdateFileSegment(value = "") {
2980:     return String(value || "")
2981:         .replace(/^[a-z0-9._-]+/gi, "-")
2982:         .replace(/^-+|-+$/g, "")
2983:         .slice(0, 80) || "latest";
2984: }

```

powershellSingleQuoted (server.mjs, lines 2986-2988)

powershellSingleQuoted part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 2986-2988 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2987: return `\${String(value).replaceAll("'", "'')}";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2986: function powershellSingleQuoted(value = "") {
2987:     return `${String(value).replaceAll("'", "'')}";
2988: }

```

downloadAndLaunchAppUpdate (server.mjs, lines 2990-3165)

This function downloads a selected app installer release, writes an update handoff script, closes the running app, starts the installer, and attempts to relaunch the updated executable afterward.

Windows cannot cleanly overwrite a running app. This function coordinates update work so the user sees a simple Update button instead of manually closing, installing, and reopening everything.

It calls or references getUpdateInfo, Error, join, tmpdir, mkdir, safeUpdateFileSegment, fetch, from, arrayBuffer, writeFile, and 20 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 3056: " if (\$candidate -and (Test-Path -LiteralPath \$candidate)) { return \$candidate }";, line 3060: " return \$null";, line 3090: " if (-not \$process) { return 0 }";, line 3094: " return 124";. There are 2 additional

return statements in the function body.

Important local values include update at line 2991; updateRoot at line 3006; installerName at line 3008; installerPath at line 3011; response at line 3013; installerBuffer at line 3017; updateLogPath at line 3023; currentExecutable at line 3024; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2990: async function downloadAndLaunchAppUpdate() {
2991:   const update = await getUpdateInfo();
2992:   if (!update.ok) throw new Error(update.error || "Could not check for updates.");
2993:   if (update.updateBlocked) {
2994:     throw new Error(update.blockedReason || `Builder ${update.latestVersion} is temporarily blocked from
automatic updates.`);
2995:   }
2996:   if (!update.updateAvailable) {
2997:     throw new Error(`OMB Portfolio Builder ${update.currentVersion || ""} is already up to date.`);
2998:   }
2999:   if (!update.installerUrl) {
3000:     throw new Error("The latest GitHub Release does not include a Windows setup installer.");
3001:   }
3002:   if (process.platform !== "win32") {
3003:     throw new Error("Automatic installer updates are currently available only on Windows.");
3004:   }
3005:
3006:   const updateRoot = path.join(os.tmpdir(), "omb-portfolio-builder-updates");
3007:   await mkdir(updateRoot, { recursive: true });
3008:   const installerName = update.installerName && /\.exe$/i.test(update.installerName)
3009:     ? update.installerName
3010:     : `OMB-Portfolio-Builder-Setup-${safeUpdateFileSegment(update.latestVersion)}-x64.exe`;
3011:   const installerPath = path.join(updateRoot, installerName);
3012:
3013:   const response = await fetch(update.installerUrl, {
3014:     headers: { "Accept": "application/octet-stream", "User-Agent": "OMB-Portfolio-Builder" }
3015:   });
3016:   if (!response.ok) throw new Error(`GitHub returned ${response.status} while downloading the
installer.`);
3017:   const installerBuffer = Buffer.from(await response.arrayBuffer());
3018:   if (installerBuffer.length < 1024 * 1024) {
3019:     throw new Error("The downloaded installer was unexpectedly small, so the update was stopped.");
3020:   }
3021:   await writeFile(installerPath, installerBuffer);
3022:
3023:   const updateLogPath = path.join(updateRoot,
`update-${safeUpdateFileSegment(update.latestVersion)}.log`);
3024:   const currentExecutable = /OMB Portfolio Builder\.exe$/i.test(process.execPath || "") ?
process.execPath : "";
3025:   const currentInstallDirectory = currentExecutable ? path.dirname(currentExecutable) : "";
3026:   const localAppDataExecutable = process.env.LOCALAPPDATA
3027:     ? path.join(process.env.LOCALAPPDATA, "Programs", "OMB Portfolio Builder", "OMB Portfolio
Builder.exe")
3028:     : "";
3029:   const legacyManagedApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio
Builder", "OMB Portfolio Builder.exe");
3030:   const legacyFlatApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio
Builder.exe");
3031:   const relaunchCandidates = Array.from(new Set([
3032:     localAppDataExecutable,
3033:     currentExecutable,
3034:     currentInstallDirectory ? path.join(currentInstallDirectory, "OMB Portfolio Builder.exe") : "",
3035:     legacyFlatApplicationExecutable,
3036:     legacyManagedApplicationExecutable,
3037:     process.env.ProgramFiles ? path.join(process.env.ProgramFiles, "OMB Portfolio Builder", "OMB
Portfolio Builder.exe") : "",
3038:     process.env["ProgramFiles(x86)"] ? path.join(process.env["ProgramFiles(x86)"], "OMB Portfolio
Builder", "OMB Portfolio Builder.exe") : ""
3039:   ].filter(Boolean)));
3040:   const relaunchCandidatesPs = `@(${relaunchCandidates.map(powershellSingleQuoted).join(", ")});`;
3041:   const launcherPath = path.join(updateRoot, `run-
update-${safeUpdateFileSegment(update.latestVersion)}.ps1`);
3042:   const launcherCommandPath = path.join(updateRoot, `run-
update-${safeUpdateFileSegment(update.latestVersion)}.cmd`);
3043:   const launcherScript = [
3044:     "ErrorActionPreference = 'Continue'",
3045:     `$parentPid = ${process.pid}`,
3046:     `$installer = ${powershellSingleQuoted(installerPath)}`,
3047:     `$log = ${powershellSingleQuoted(updateLogPath)}`,
3048:     `$candidates = ${relaunchCandidatesPs}`,
3049:     "function Write-UpdateLog([string]$Message) {",
3050:     "  try { Add-Content -LiteralPath $log -Value ((Get-Date).ToString('s') + ' ' + $Message) } catch",
3051:     "}",
3052:     "function Wait-ForExecutable([string[]]$Paths, [int]$Seconds) {",
3053:     "  $deadline = (Get-Date).AddSeconds($Seconds)",
3054:     "  do {",
3055:     "    foreach ($candidate in $Paths) {",
3056:     "      if ($candidate -and (Test-Path -LiteralPath $candidate)) { return $candidate }",
3057:     "    }",
3058:     "    Start-Sleep -Seconds 1",
3059:     "  } while ((Get-Date) -lt $deadline)",
```

```

3060:     " return $null",
3061:     "}",
3062:     "function Stop-BuilderProcesses([int]$ProcessId, [string[]]$Paths) {",
3063:     " try {",
3064:     "     $normalizedPaths = @($Paths | Where-Object { $_ } | ForEach-Object {",
3065:     "         try { [System.IO.Path]::GetFullPath($_).ToLowerInvariant() } catch { $_.ToLowerInvariant() }",
3066:     "     }",
3067:     "     $processes = Get-CimInstance Win32_Process | Where-Object {",
3068:     "         $_.ProcessId -eq $ProcessId -or",
3069:     "         $_.Name -eq 'OMB Portfolio Builder.exe' -or",
3070:     "         ($_.ExecutablePath -and ($normalizedPaths -contains",
3071:     "             ([System.IO.Path]::GetFullPath($_.ExecutablePath).ToLowerInvariant()))",
3072:     "         } | Sort-Object ProcessId -Unique",
3073:     "         foreach ($process in $processes) {",
3074:     "             try {",
3075:     "                 Write-UpdateLog ('Stopping previous builder process PID ' + $process.ProcessId + ' at ' +",
3076:     "                     $process.ExecutablePath)",
3077:     "                 Stop-Process -Id $process.ProcessId -Force -ErrorAction SilentlyContinue",
3078:     "             } catch { Write-UpdateLog ('Could not stop PID ' + $process.ProcessId + ': ' +",
3079:     "                 $_.Exception.Message) }",
3080:     "             }",
3081:     "             Start-Sleep -Seconds 2",
3082:     "         } catch { Write-UpdateLog ('Could not stop previous builder processes: ' + $_.Exception.Message)",
3083:     "     }",
3084:     "     function Run-Installer([switch]$Elevated) {",
3085:     "         $process = $null",
3086:     "         if ($Elevated) {",
3087:     "             Write-UpdateLog 'Starting installer with Windows elevation prompt.'",
3088:     "             $process = Start-Process -FilePath $installer -ArgumentList @('/S') -Verb RunAs -PassThru",
3089:     "         } else {",
3090:     "             Write-UpdateLog 'Starting installer silently.'",
3091:     "             $process = Start-Process -FilePath $installer -ArgumentList @('/S') -WindowStyle Hidden",
3092:     "             -PassThru",
3093:     "         }",
3094:     "         if (-not $process) { return 0 }",
3095:     "         if (-not $process.WaitForExit(900000)) {",
3096:     "             Write-UpdateLog ('Installer timed out after 15 minutes. PID: ' + $process.Id)",
3097:     "             try { Stop-Process -Id $process.Id -Force -ErrorAction SilentlyContinue } catch { Write-",
3098:     "                 UpdateLog ('Could not stop timed-out installer: ' + $_.Exception.Message) }",
3099:     "             return 124",
3100:     "         }",
3101:     "         return $process.ExitCode",
3102:     "     }",
3103:     "     Write-UpdateLog 'Update launcher started.'",
3104:     "     Start-Sleep -Seconds 3",
3105:     "     try { Wait-Process -Id $parentPid -Timeout 8; Write-UpdateLog 'Previous app process exited.' } catch",
3106:     "     { Write-UpdateLog ('Previous app wait finished: ' + $_.Exception.Message) }",
3107:     "     Stop-BuilderProcesses $parentPid $candidates",
3108:     "     Start-Sleep -Seconds 1",
3109:     "     $installerProcess = $null",
3110:     "     $installerExit = 1",
3111:     "     try {",
3112:     "         Write-UpdateLog ('Starting installer: ' + $installer)",
3113:     "         $installerExit = Run-Installer",
3114:     "         Write-UpdateLog ('Installer exit code: ' + $installerExit)",
3115:     "     } catch {",
3116:     "         ... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

syncPortfolioPublishFiles (server.mjs, lines 3167-3200)

syncPortfolioPublishFiles part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 3167-3200 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references samePath, mkdir, resolveInsideRoot, resolveInsidePortfolioRoot, pathExists, rm, dirname, cp, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3169: return { copied: [], skipped: [], workspace: portfolioRoot, separatedWorkspace: false };; line 3194: return {.

Important local values include copied at line 3173; skipped at line 3174; sourcePath at line 3179; targetPath at line 3180. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3167: async function syncPortfolioPublishFiles(options = {}) {
3168:   if (samePath(root, portfolioRoot)) {
3169:     return { copied: [], skipped: [], workspace: portfolioRoot, separatedWorkspace: false };
3170:   }
3171:
3172:   const { removeMissing = false } = options;
3173:   const copied = [];
3174:   const skipped = [];

```

```

3175:
3176:   await mkdir(portfolioRoot, { recursive: true });
3177:
3178:   for (const relativePath of publishPaths) {
3179:     const sourcePath = resolveInsideRoot(relativePath);
3180:     const targetPath = resolveInsidePortfolioRoot(relativePath);
3181:     if (await pathExists(sourcePath)) {
3182:       await rm(targetPath, { recursive: true, force: true });
3183:       await mkdir(path.dirname(targetPath), { recursive: true });
3184:       await cp(sourcePath, targetPath, { recursive: true, force: true });
3185:       copied.push(relativePath);
3186:     } else if (removeMissing) {
3187:       await rm(targetPath, { recursive: true, force: true });
3188:       skipped.push(relativePath);
3189:     } else {
3190:       skipped.push(relativePath);
3191:     }
3192:   }
3193:
3194:   return {
3195:     copied,
3196:     skipped,
3197:     workspace: portfolioRoot,
3198:     separatedWorkspace: true
3199:   };
3200: }

```

publishSiteChanges (server.mjs, lines 3202-3238)

This function applies the generated portfolio output to the publish mirror, stages allowed files, commits the changes, and pushes them to the configured website repository.

This is the final local-to-public handoff. It keeps publishing constrained to generated website files so the builder app does not blindly push unrelated local files.

It calls or references `assertPublishAccess`, `runPublishGit`, `trim`, `synchronizePublishBranchFromRemote`, `syncPortfolioPublishFiles`, `bumpPublishedAssetVersions`, `stageablePublishPaths`, `Error`, `toISOString`, and `slice`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3227: `return {`.

Important local values include `access` at line 3203; `branch` at line 3204; `remoteSync` at line 3205; `portfolioSync` at line 3206; `stageablePaths` at line 3209; `status` at line 3215; `hasChanges` at line 3216; `commit` at line 3218; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3202: async function publishSiteChanges(publishAccess = null) {
3203:   const access = publishAccess || await assertPublishAccess();
3204:   const branch = access.branch || (await runPublishGit(["branch", "--show-current"])).stdout.trim();
3205:   const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
3206:   const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: true });
3207:   await bumpPublishedAssetVersions(portfolioRoot);
3208:
3209:   const stageablePaths = await stageablePublishPaths(publishPaths);
3210:   if (!stageablePaths.length) {
3211:     throw new Error("No compatible site files were available to publish.");
3212:   }
3213:
3214:   await runPublishGit(["add", "-A", "--", ...stageablePaths]);
3215:   const status = await runPublishGit(["status", "--porcelain", "--", ...stageablePaths]);
3216:   const hasChanges = status.stdout.trim().length > 0;
3217:
3218:   let commit = null;
3219:   if (hasChanges) {
3220:     const message = `Update portfolio site ${new Date().toISOString().slice(0, 10)}`;
3221:     commit = await runPublishGit(["commit", "-m", message]);
3222:   }
3223:
3224:   const pushArgs = branch ? ["push", "origin", branch] : ["push"];
3225:   const push = await runPublishGit(pushArgs);
3226:
3227:   return {
3228:     ...access,
3229:     portfolioWorkspace: portfolioRoot,
3230:     remoteSync,
3231:     portfolioSync,
3232:     branch: branch || "current branch",
3233:     committed: hasChanges,
3234:     commitOutput: commit?.stdout || commit?.stderr || "",
3235:     pushed: true,
3236:     pushOutput: push.stdout || push.stderr || ""
3237:   };
3238: }

```

handlePortfolioAi (server.mjs, lines 3240-3322)

This backend handler receives AI chat requests, classifies the question, builds the model payload, chooses an available AI route, and falls back to local rule-based answers when remote AI is unavailable.

It provides the conversational intelligence layer while keeping secrets and model calls away from static frontend code.

It calls or references `readRequestJson`, `clampText`, `trim`, `cleanConversationHistory`, `Set`, `has`, `sendJson`, `enrichPortfolioContext`, `callOllamaPortfolioAi`, `toLowerCase`, and 6 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3250: `return;;`; line 3265: `return;;`; line 3268: `return;;`; line 3321: `return { data, openAiResponse, selectedModel };;`.

Important local values include `body` at line 3241; `question` at line 3242; `conversation` at line 3243; `validIntents` at line 3244; `intent` at line 3245; `allowWebSearch` at line 3246; `context` at line 3253; `apiKey` at line 3255; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3240: async function handlePortfolioAi(request, response) {
3241:   const body = await readRequestJson(request);
3242:   const question = clampText(body.question, 1200).trim();
3243:   const conversation = cleanConversationHistory(body.conversation);
3244:   const validIntents = new Set(["general_conversation", "general_engineering", "general_knowledge",
"portfolio_specific"]);
3245:   const intent = validIntents.has(body.intent) ? body.intent : "portfolio_specific";
3246:   const allowWebSearch = body.allowWebSearch === true;
3247:
3248:   if (!question) {
3249:     sendJson(response, 400, { error: "Question is required." });
3250:     return;
3251:   }
3252:
3253:   const context = await enrichPortfolioContext({ ...(body.context || {}), question });
3254:
3255:   const apiKey = process.env.OPENAI_API_KEY || "";
3256:   if (!apiKey) {
3257:     const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch
});
3258:     if (ollama.ok) {
3259:       sendJson(response, 200, {
3260:         answer: ollama.answer,
3261:         model: ollama.model,
3262:         provider: "ollama",
3263:         usedWebSearch: false
3264:       });
3265:       return;
3266:     }
3267:     sendJson(response, 503, { error: ollama.error || "No local Ollama model or OPENAI_API_KEY is
available for the local backend." });
3268:     return;
3269:   }
3270:
3271:   const model = process.env.OPENAI_MODEL || "gpt-5.4";
3272:   const fallbackModel = process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1";
3273:   const webSearchMode = String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase();
3274:   const enableWebSearch = webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch);
3275:   const buildPayload = (selectedModel) => ({
3276:     model: selectedModel,
3277:     input: [
3278:       {
3279:         role: "developer",
3280:         content: [{ type: "input_text", text: portfolioAiInstructions }]
3281:       },
3282:       {
3283:         role: "user",
3284:         content: [{
3285:           type: "input_text",
3286:           text: [
3287:             `Visitor question: ${question}`,
3288:             `Question intent: ${intent}`,
3289:             `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
3290:             "",
3291:             "Recent conversation JSON:",
3292:             clampText(JSON.stringify(conversation, null, 2), 8000),
3293:             "",
3294:             "Portfolio context JSON:",
3295:             clampText(JSON.stringify(context, null, 2), 18000)
3296:           ].join("\n")
3297:         ]
3298:       }
3299:     ],
3300:     max_output_tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
3301:     reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
3302:     text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
3303:   });
```

```

3304:
3305:   const callModel = async (selectedModel) => {
3306:     const payload = buildPayload(selectedModel);
3307:     if (enableWebSearch) {
3308:       payload.tools = [{ type: "web_search", search_context_size:
process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE || "low" }];
3309:     }
3310:
3311:     const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
3312:       method: "POST",
3313:       headers: {
3314:         "Authorization": `Bearer ${apiKey}`,
3315:         "Content-Type": "application/json"
3316:       },
3317:       body: JSON.stringify(payload)
3318:     });
3319:
3320:     const data = await openAiResponse.json().catch(() => ({}));
3321:     return { data, openAiResponse, selectedModel };
3322:   };

```

buildPayload (server.mjs, lines 3275-3303)

buildPayload creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 3275-3303 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clampText, stringify, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include buildPayload at line 3275. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3275:   const buildPayload = (selectedModel) => ({
3276:     model: selectedModel,
3277:     input: [
3278:       {
3279:         role: "developer",
3280:         content: [{ type: "input_text", text: portfolioAiInstructions }]
3281:       },
3282:       {
3283:         role: "user",
3284:         content: [{
3285:           type: "input_text",
3286:           text: [
3287:             `Visitor question: ${question}`,
3288:             `Question intent: ${intent}`,
3289:             `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
3290:             "",
3291:             "Recent conversation JSON:",
3292:             clampText(JSON.stringify(conversation, null, 2), 8000),
3293:             "",
3294:             "Portfolio context JSON:",
3295:             clampText(JSON.stringify(context, null, 2), 18000)
3296:           ].join("\n")
3297:         }
3298:       ]
3299:     },
3300:     max_output_tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
3301:     reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
3302:     text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
3303:   });

```

callModel (server.mjs, lines 3305-3318)

callModel part of the local backend api, filesystem, compiler, git, ai, update, or publish workflow. In this file, it appears around lines 3305-3318 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so server.mjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references async, buildPayload, fetch, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include `callModel` at line 3305; `payload` at line 3306; `openAiResponse` at line 3311. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3305: const callModel = async (selectedModel) => {
3306:   const payload = buildPayload(selectedModel);
3307:   if (enableWebSearch) {
3308:     payload.tools = [{ type: "web_search", search_context_size:
process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE || "low" }];
3309:   }
3310:
3311:   const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
3312:     method: "POST",
3313:     headers: {
3314:       "Authorization": `Bearer ${apiKey}`,
3315:       "Content-Type": "application/json"
3316:     },
3317:     body: JSON.stringify(payload)
3318:   });
```

handleApi (server.mjs, lines 3342-3602)

This is the local backend router. It reads the request path, selects the matching API behavior, calls the responsible function, and sends the response back to the builder frontend.

Almost every builder action crosses this function. It is the traffic director between button clicks and backend work.

It calls or references `readJsonFile`, `sendJson`, `join`, `isLocalRequest`, `getPublishTargetInfo`, `getGitStatus`, `getUpdateInfo`, `getSecurityReport`, `compileToolStatus`, `handlePortfolioAi`, and 20 more. Endpoint references found inside it are `/api/app-update`, `/api/app-update/install`, `/api/apply-catalog`, `/api/catalog`, `/api/code/beautify`, `/api/code/compile`, `/api/code/install-tools`, `/api/code/save`, `/api/code/tools`, `/api/github-authenticate`, and 9 more. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 3350: `return true`; line 3355: `return true`; line 3361: `return true`; line 3364: `return true`; There are 8 additional return statements in the function body.

Important local values include `catalog` at line 3345; `body` at line 3409; `language` at line 3410; `body` at line 3424; `saved` at line 3425; `body` at line 3435; `result` at line 3436; `body` at line 3446; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3342: async function handleApi(request, response, url) {
3343:   if (request.method === "GET" && url.pathname === "/api/catalog") {
3344:     try {
3345:       const catalog = await readJsonFile(draftPath);
3346:       sendJson(response, 200, { source: "draft", catalog });
3347:     } catch {
3348:       sendJson(response, 200, { source: "published", catalog: await readJsonFile(catalogPath) });
3349:     }
3350:     return true;
3351:   }
3352:
3353:   if (request.method === "GET" && url.pathname === "/api/templates") {
3354:     sendJson(response, 200, await readJsonFile(path.join(root, "project-templates.json")));
3355:     return true;
3356:   }
3357:
3358:   if (request.method === "GET" && url.pathname === "/api/publish-target") {
3359:     if (!isLocalRequest(request)) {
3360:       sendJson(response, 403, { error: "Publishing target details are only available from this computer." });
3361:       return true;
3362:     }
3363:     sendJson(response, 200, { ok: true, target: await getPublishTargetInfo() });
3364:     return true;
3365:   }
3366:
3367:   if (request.method === "GET" && url.pathname === "/api/system-check") {
3368:     sendJson(response, 200, { ok: true, system: await getGitStatus(), target: await
getPublishTargetInfo() });
3369:     return true;
3370:   }
3371:
3372:   if (request.method === "GET" && url.pathname === "/api/app-update") {
3373:     sendJson(response, 200, await getUpdateInfo());
3374:     return true;
3375:   }
3376:
3377:   if (request.method === "GET" && url.pathname === "/api/security-report") {
3378:     if (!isLocalRequest(request)) {
3379:       sendJson(response, 403, { error: "Security reports are only available from this computer." });
3380:       return true;
3381:     }
3382:     sendJson(response, 200, await getSecurityReport());
```

```

3383:     return true;
3384: }
3385:
3386: if (request.method === "GET" && url.pathname === "/api/code/tools") {
3387:   if (!isLocalRequest(request)) {
3388:     sendJson(response, 403, { error: "Compiler details are only available from this computer." });
3389:     return true;
3390:   }
3391:   sendJson(response, 200, { ok: true, tools: await compileToolStatus() });
3392:   return true;
3393: }
3394:
3395: if (request.method !== "POST") return false;
3396:
3397: if (!isLocalRequest(request)) {
3398:   sendJson(response, 403, { error: "Write actions are only allowed from this computer." });
3399:   return true;
3400: }
3401:
3402: if (url.pathname === "/api/portfolio-ai") {
3403:   await handlePortfolioAi(request, response);
3404:   return true;
3405: }
3406:
3407: if (url.pathname === "/api/code/beautify") {
3408:   try {
3409:     const body = await readRequestJson(request);
3410:     const language = normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code,
body.fileName));
3411:     sendJson(response, 200, {
3412:       ok: true,
3413:       language,
3414:       code: beautifyCode(body.code || "", language)
3415:     });
3416:   } catch (error) {
3417:     sendJson(response, 400, { ok: false, error: error.message || "Code could not be beautified." });
3418:   }
3419:   return true;
3420: }
3421:
3422: if (url.pathname === "/api/code/save") {
3423:   try {
3424:     const body = await readRequestJson(request);
3425:     const saved = await saveCompileSource(body);
3426:     sendJson(response, 200, { ok: true, saved });
3427:   } catch (error) {
3428:     sendJson(response, 400, { ok: false, error: error.message || "Code could not be saved." });
3429:   }
3430:   return true;
3431: }
3432:
3433: if (url.pathname === "/api/code/compile") {
3434:   try {
3435:     const body = await readRequestJson(request);
3436:     const result = await compileAndRunCode(body);
3437:     sendJson(response, 200, { ok: result.ok, result });
3438:   } catch (error) {
3439:     sendJson(response, 400, { ok: false, error: error.message || "Code could not be compiled.", result:
null });
3440:   }
3441:   return true;
3442: }
3443:
3444: if (url.pathname === "/api/code/install-tools") {
3445:   try {
3446:     const body = await readRequestJson(request);
3447:     const result = await installCompilerTools(body.language || "");
3448:     sendJson(response, 200, result);
3449:   } catch (error) {
3450:     sendJson(response, 400, { ok: false, error: error.message || "Compiler tools could not be
installed." });
3451:   }
3452:   return true;
3453: }
3454:
3455: if (url.pathname === "/api/publish-target") {
3456:   await readRequestJson(request).catch(() => ({}));
3457:   sendJson(response, 400, {
3458:     ok: false,
3459:     error: "Publishing target setup now requires GitHub authentication.",
3460:     details: "Use Authenticate target. The builder keeps the previous target until GitHub write access
is verified."
3461:   });
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

Representative opening snippet

```

1: import { createServer } from "node:http";
2: import { execFile, spawn } from "node:child_process";
3: import { createHash } from "node:crypto";

```



```

    4: import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from
"node:fs/promises";
    5: import os from "node:os";
    6: import path from "node:path";
    7: import { promisify } from "node:util";
    8: import { fileURLToPath } from "node:url";
    9:
   10: const root = path.dirname(fileURLToPath(import.meta.url));
   11: const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE || root);
   12: const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE || path.join(path.dirname(root),
"compile-code"));
   13: const port = Number(process.env.PORT || 8080);
   14: const host = process.env.HOST || "0.0.0.0";
   15: const execFileAsync = promisify(execFile);
   16: const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() =>
"{}"));
   17:
   18: const types = {
   19:   ".css": "text/css",
   20:   ".csv": "text/csv",
   21:   ".html": "text/html",
   22:   ".ico": "image/x-icon",
   23:   ".js": "application/javascript",
   24:   ".json": "application/json",
   25:   ".md": "text/markdown",
   26:   ".pdf": "application/pdf",
   27:   ".png": "image/png",
   28:   ".jpg": "image/jpeg",
   29:   ".jpeg": "image/jpeg",
   30:   ".svg": "image/svg+xml",
   31:   ".txt": "text/plain",
   32:   ".webp": "image/webp",
   33:   ".xml": "application/xml",
   34:   ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
   35:   ".zip": "application/zip"
   36: };
   37:
   38: const draftPath = path.join(root, "projects.local.json");
   39: const catalogPath = path.join(root, "projects.json");
   40: const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
   41: const publishPaths = [
   42:   "projects.json",
   43:   "docs",
   44:   "assets",
   45:   "index.html",
   46:   "styles.css",
   47:   "script.js",
   48:   "electronics-search.js",
   49:   "Backgrounds",
   50:   ".nojekyll",
   51:   "_headers",
   52:   ".well-known",
   53:   "robots.txt",
   54:   "sitemap.xml",
   55:   "CNAME"
   56: ];
   57: const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
   58: const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
   59: const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
   60: const publishAuthExtendedThreshold = 3;
   61: const defaultSiteRepository = process.env.OMB_BUILDER_REPOSITORY || "";
   62: const blockedAppUpdateVersions = new Set(["0.2.16"]);
   63: const publishAuthorizationHelp = [
   64:   "Publishing was blocked before live website files were applied.",
   65:   "Open Publishing target, enter the repository URL, then click Authenticate target.",
   66:   "A GitHub/Git Credential Manager browser sign-in may open. Sign in with an account that has write
access to the selected Pages repository.",
   67:   "Authenticate target only verifies write access and remembers it for the matching repository and
branch.",
   68:   "Load from target only imports compatible website files from the authenticated repository into the
local builder workspace.",
   69:   "Until a compatible writable website repository is associated, the builder remains local-only."
   70: ].join(" ");
   71: const gitCandidates = [
   72:   process.env.GIT_EXE,

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: template-preview.js

Item	Explanation
File	template-preview.js
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	454,437 bytes
Extension	.js
MIME type	application/javascript
Text lines	10,826

Why this file exists

- Builder frontend brain: state handling, rich editors, project builder, previews, parser triggers, publishing UI, compile workspace, and guide windows.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

API endpoints mentioned or called

Item	Explanation
/api/save-draft	Line 1637. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/upload	Line 5430. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/upload	Line 8413. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/upload	Line 8473. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/apply-catalog	Line 8613. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8637. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8647. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8662. Loads project/catalog data for the builder.
/api/apply-catalog	Line 8674. Loads project/catalog data for the builder.
/api/save-draft	Line 8685. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/save-draft	Line 10770. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/apply-catalog	Line 10772. Loads project/catalog data for the builder.
/api/save-draft	Line 10796. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.

Important implementation signals

Item	Explanation
Browser DOM at line 1	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideOpen = document.querySelector("#builder-guide-open");
Browser DOM at line 2	Builder or website interface behavior in the browser/Electron renderer. Evidence: const builderGuideDialog = document.querySelector("#builder-guide-dialog");

Item	Explanation
Browser DOM at line 3	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const builderGuideClose = document.querySelector("#builder-guide-close");</code>
Browser DOM at line 4	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const builderGuideSearch = document.querySelector("#builder-guide-search");</code>
Browser DOM at line 5	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const builderGuideResults = document.querySelector("#builder-guide-results");</code>
Browser DOM at line 6	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const builderGuideSections = document.querySelector("#builder-guide-sections");</code>
Browser DOM at line 7	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const builderGuideTopicDialog = document.querySelector("#builder-guide-topic-dialog");</code>
Browser DOM at line 8	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const builderGuideTopicTitle = document.querySelector("#builder-guide-topic-title");</code>
Browser DOM at line 9	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const builderGuideTopicBody = document.querySelector("#builder-guide-topic-body");</code>
Browser DOM at line 10	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const builderGuideTopicClose = document.querySelector("#builder-guide-topic-close");</code>
Browser DOM at line 11	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const templateSelect = document.querySelector("#template-select");</code>
Browser DOM at line 12	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const newCategorySelect = document.querySelector("#new-category-select");</code>
Browser DOM at line 13	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const placementSelect = document.querySelector("#placement-select");</code>
Browser DOM at line 14	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const newProjectTitle = document.querySelector("#new-project-title");</code>
Browser DOM at line 15	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const addProjectButton = document.querySelector("#add-project");</code>
Browser DOM at line 16	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const addCategoryButton = document.querySelector("#add-category");</code>
Browser DOM at line 17	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const addSiteSectionButton = document.querySelector("#add-site-section");</code>
Browser DOM at line 18	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const funFactsInput = document.querySelector("#fun-facts-input");</code>
Browser DOM at line 19	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const funFactsCount = document.querySelector("#fun-facts-count");</code>
Browser DOM at line 20	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const saveFunFactsButton = document.querySelector("#save-fun-facts");</code>
Browser DOM at line 21	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const clearFunFactsButton = document.querySelector("#clear-fun-facts");</code>
Browser DOM at line 22	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const siteHeroEyebrowInput = document.querySelector("#site-hero-eyebrow");</code>
Browser DOM at line 23	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const siteHeroTitleInput = document.querySelector("#site-hero-title");</code>
Browser DOM at line 24	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const siteHeroCopyInput = document.querySelector("#site-hero-copy");</code>
Browser DOM at line 25	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const saveSiteContentButton = document.querySelector("#save-site-content");</code>
Browser DOM at line 26	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileDisplayNameInput = document.querySelector("#profile-display-name");</code>
Browser DOM at line 27	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profilePortfolioLabelInput = document.querySelector("#profile-portfolio-label");</code>
Browser DOM at line 28	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileContactIntroInput = document.querySelector("#profile-contact-intro");</code>
Browser DOM at line 29	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileEmailInput = document.querySelector("#profile-email");</code>

Item	Explanation
Browser DOM at line 30	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profilePhoneInput = document.querySelector("#profile-phone");</code>
Browser DOM at line 31	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileGithubInput = document.querySelector("#profile-github");</code>
Browser DOM at line 32	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileLinkedInInput = document.querySelector("#profile-linkedin");</code>
Browser DOM at line 33	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileWebsiteInput = document.querySelector("#profile-website");</code>
Browser DOM at line 34	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const saveProfileContentButton = document.querySelector("#save-profile-content");</code>
Browser DOM at line 35	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileImageUploadButton = document.querySelector("#profile-image-upload");</code>
Browser DOM at line 36	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileHeroUploadButton = document.querySelector("#profile-hero-upload");</code>
Browser DOM at line 37	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileResumeUploadButton = document.querySelector("#profile-resume-upload");</code>
Browser DOM at line 38	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileBrandUploadButton = document.querySelector("#profile-brand-upload");</code>
Browser DOM at line 39	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profileAssetStatus = document.querySelector("#profile-asset-status");</code>
Browser DOM at line 40	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const categoryDropdown = document.querySelector("#category-dropdown");</code>
Browser DOM at line 41	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const siteSectionList = document.querySelector("#site-section-list");</code>
Browser DOM at line 42	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectCreateDialog = document.querySelector("#project-create-dialog");</code>
Browser DOM at line 43	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectCreateForm = document.querySelector("#project-create-form");</code>
Browser DOM at line 44	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectCreateCategory = document.querySelector("#project-create-category");</code>
Browser DOM at line 45	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectCreateCancel = document.querySelector("#project-create-cancel");</code>
Browser DOM at line 46	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const categoryDialog = document.querySelector("#category-dialog");</code>
Browser DOM at line 47	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const categoryForm = document.querySelector("#category-form");</code>
Browser DOM at line 48	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const categoryTitle = document.querySelector("#category-title");</code>
Browser DOM at line 49	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const categoryDescription = document.querySelector("#category-description");</code>
Browser DOM at line 50	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const categoryAccent = document.querySelector("#category-accent");</code>
Browser DOM at line 51	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const categoryCancel = document.querySelector("#category-cancel");</code>
Browser DOM at line 52	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectTree = document.querySelector("#project-tree");</code>
Browser DOM at line 53	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectSearchInput = document.querySelector("#project-search");</code>
Browser DOM at line 54	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectSearchClear = document.querySelector("#project-search-clear");</code>
Browser DOM at line 55	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectSearchStatus = document.querySelector("#project-search-status");</code>
Browser DOM at line 56	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectDialog = document.querySelector("#project-dialog");</code>

Item	Explanation
Browser DOM at line 57	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectWindowTitle = document.querySelector("#project-window-title");</code>
Browser DOM at line 58	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectWindowClose = document.querySelector("#project-window-close");</code>
Browser DOM at line 59	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectWindowDelete = document.querySelector("#project-window-delete");</code>
Browser DOM at line 60	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const viewProjectPreviewButton = document.querySelector("#view-project-preview");</code>
Browser DOM at line 61	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const saveProjectButton = document.querySelector("#save-project");</code>
Browser DOM at line 62	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const saveProjectCloseButton = document.querySelector("#save-project-close");</code>
Browser DOM at line 63	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectTitleMenu = document.querySelector("#project-title-menu");</code>
Browser DOM at line 64	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const titleRenameActions = document.querySelector("#title-rename-actions");</code>
Browser DOM at line 65	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const titleRenameSave = document.querySelector("#title-rename-save");</code>
Browser DOM at line 66	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const titleRenameCancel = document.querySelector("#title-rename-cancel");</code>
Browser DOM at line 67	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectMetaDialog = document.querySelector("#project-meta-dialog");</code>
Browser DOM at line 68	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectMetaTitle = document.querySelector("#project-meta-title");</code>
Browser DOM at line 69	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectMetaGrid = document.querySelector("#project-meta-grid");</code>
Browser DOM at line 70	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectMetaClose = document.querySelector("#project-meta-close");</code>
Browser DOM at line 71	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectFields = document.querySelector("#project-fields");</code>
Browser DOM at line 72	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const sectionTabs = document.querySelector("#section-tabs");</code>
Browser DOM at line 73	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const sectionContent = document.querySelector("#section-content");</code>
Browser DOM at line 74	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectWindowContent = document.querySelector(".project-window-content");</code>
Browser DOM at line 75	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewDialog = document.querySelector("#project-preview-dialog");</code>
Browser DOM at line 76	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewTitle = document.querySelector("#project-preview-title");</code>
Browser DOM at line 77	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewContent = document.querySelector("#project-preview-content");</code>
Browser DOM at line 78	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewClose = document.querySelector("#project-preview-close");</code>
Browser DOM at line 79	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewBack = document.querySelector("#project-preview-back");</code>
Browser DOM at line 80	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const projectPreviewForward = document.querySelector("#project-preview-forward");</code>

Important constants and variables

Item	Explanation
builderGuideOpen	Line 1: <code>const builderGuideOpen = document.querySelector("#builder-guide-open");</code>

Item	Explanation
builderGuideDialog	Line 2: const builderGuideDialog = document.querySelector("#builder-guide-dialog");
builderGuideClose	Line 3: const builderGuideClose = document.querySelector("#builder-guide-close");
builderGuideSearch	Line 4: const builderGuideSearch = document.querySelector("#builder-guide-search");
builderGuideResults	Line 5: const builderGuideResults = document.querySelector("#builder-guide-results");
builderGuideSections	Line 6: const builderGuideSections = document.querySelector("#builder-guide-sections");
builderGuideTopicDialog	Line 7: const builderGuideTopicDialog = document.querySelector("#builder-guide-topic-dialog");
builderGuideTopicTitle	Line 8: const builderGuideTopicTitle = document.querySelector("#builder-guide-topic-title");
builderGuideTopicBody	Line 9: const builderGuideTopicBody = document.querySelector("#builder-guide-topic-body");
builderGuideTopicClose	Line 10: const builderGuideTopicClose = document.querySelector("#builder-guide-topic-close");
templateSelect	Line 11: const templateSelect = document.querySelector("#template-select");
newCategorySelect	Line 12: const newCategorySelect = document.querySelector("#new-category-select");
placementSelect	Line 13: const placementSelect = document.querySelector("#placement-select");
newProjectTitle	Line 14: const newProjectTitle = document.querySelector("#new-project-title");
addProjectButton	Line 15: const addProjectButton = document.querySelector("#add-project");
addCategoryButton	Line 16: const addCategoryButton = document.querySelector("#add-category");
addSiteSectionButton	Line 17: const addSiteSectionButton = document.querySelector("#add-site-section");
funFactsInput	Line 18: const funFactsInput = document.querySelector("#fun-facts-input");
funFactsCount	Line 19: const funFactsCount = document.querySelector("#fun-facts-count");
saveFunFactsButton	Line 20: const saveFunFactsButton = document.querySelector("#save-fun-facts");
clearFunFactsButton	Line 21: const clearFunFactsButton = document.querySelector("#clear-fun-facts");
siteHeroEyebrowInput	Line 22: const siteHeroEyebrowInput = document.querySelector("#site-hero-eyebrow");
siteHeroTitleInput	Line 23: const siteHeroTitleInput = document.querySelector("#site-hero-title");
siteHeroCopyInput	Line 24: const siteHeroCopyInput = document.querySelector("#site-hero-copy");
saveSiteContentButton	Line 25: const saveSiteContentButton = document.querySelector("#save-site-content");
profileDisplayNameInput	Line 26: const profileDisplayNameInput = document.querySelector("#profile-display-name");
profilePortfolioLabelInput	Line 27: const profilePortfolioLabelInput = document.querySelector("#profile-portfolio-label");
profileContactIntroInput	Line 28: const profileContactIntroInput = document.querySelector("#profile-contact-intro");
profileEmailInput	Line 29: const profileEmailInput = document.querySelector("#profile-email");
profilePhoneInput	Line 30: const profilePhoneInput = document.querySelector("#profile-phone");
profileGithubInput	Line 31: const profileGithubInput = document.querySelector("#profile-github");
profileLinkedInInput	Line 32: const profileLinkedInInput = document.querySelector("#profile-linkedin");
profileWebsiteInput	Line 33: const profileWebsiteInput = document.querySelector("#profile-website");
saveProfileContentButton	Line 34: const saveProfileContentButton = document.querySelector("#save-profile-content");
profileImageUploadButton	Line 35: const profileImageUploadButton = document.querySelector("#profile-image-upload");
profileHeroUploadButton	Line 36: const profileHeroUploadButton = document.querySelector("#profile-hero-upload");
profileResumeUploadButton	Line 37: const profileResumeUploadButton = document.querySelector("#profile-resume-upload");
profileBrandUploadButton	Line 38: const profileBrandUploadButton = document.querySelector("#profile-brand-upload");
profileAssetStatus	Line 39: const profileAssetStatus = document.querySelector("#profile-asset-status");
categoryDropdown	Line 40: const categoryDropdown = document.querySelector("#category-dropdown");
siteSectionList	Line 41: const siteSectionList = document.querySelector("#site-section-list");
projectCreateDialog	Line 42: const projectCreateDialog = document.querySelector("#project-create-dialog");

Item	Explanation
projectcreateForm	Line 43: const projectcreateForm = document.querySelector("#project-create-form");
projectCreateCategory	Line 44: const projectCreateCategory = document.querySelector("#project-create-category");
projectCreateCancel	Line 45: const projectCreateCancel = document.querySelector("#project-create-cancel");
categoryDialog	Line 46: const categoryDialog = document.querySelector("#category-dialog");
categoryForm	Line 47: const categoryForm = document.querySelector("#category-form");
categoryTitle	Line 48: const categoryTitle = document.querySelector("#category-title");
categoryDescription	Line 49: const categoryDescription = document.querySelector("#category-description");
categoryAccent	Line 50: const categoryAccent = document.querySelector("#category-accent");
categoryCancel	Line 51: const categoryCancel = document.querySelector("#category-cancel");
projectTree	Line 52: const projectTree = document.querySelector("#project-tree");
projectSearchInput	Line 53: const projectSearchInput = document.querySelector("#project-search");
projectSearchClear	Line 54: const projectSearchClear = document.querySelector("#project-search-clear");
projectSearchStatus	Line 55: const projectSearchStatus = document.querySelector("#project-search-status");
projectDialog	Line 56: const projectDialog = document.querySelector("#project-dialog");
projectWindowTitle	Line 57: const projectWindowTitle = document.querySelector("#project-window-title");
projectWindowClose	Line 58: const projectWindowClose = document.querySelector("#project-window-close");
projectWindowDelete	Line 59: const projectWindowDelete = document.querySelector("#project-window-delete");
viewProjectPreviewButton	Line 60: const viewProjectPreviewButton = document.querySelector("#view-project-preview");
saveProjectButton	Line 61: const saveProjectButton = document.querySelector("#save-project");
saveProjectCloseButton	Line 62: const saveProjectCloseButton = document.querySelector("#save-project-close");
projectTitleMenu	Line 63: const projectTitleMenu = document.querySelector("#project-title-menu");
titleRenameActions	Line 64: const titleRenameActions = document.querySelector("#title-rename-actions");
titleRenameSave	Line 65: const titleRenameSave = document.querySelector("#title-rename-save");
titleRenameCancel	Line 66: const titleRenameCancel = document.querySelector("#title-rename-cancel");
projectMetaDialog	Line 67: const projectMetaDialog = document.querySelector("#project-meta-dialog");
projectMetaTitle	Line 68: const projectMetaTitle = document.querySelector("#project-meta-title");
projectMetaGrid	Line 69: const projectMetaGrid = document.querySelector("#project-meta-grid");
projectMetaClose	Line 70: const projectMetaClose = document.querySelector("#project-meta-close");
projectFields	Line 71: const projectFields = document.querySelector("#project-fields");
sectionTabs	Line 72: const sectionTabs = document.querySelector("#section-tabs");
sectionContent	Line 73: const sectionContent = document.querySelector("#section-content");
projectWindowContent	Line 74: const projectWindowContent = document.querySelector(".project-window-content");
projectPreviewDialog	Line 75: const projectPreviewDialog = document.querySelector("#project-preview-dialog");
projectPreviewTitle	Line 76: const projectPreviewTitle = document.querySelector("#project-preview-title");
projectPreviewContent	Line 77: const projectPreviewContent = document.querySelector("#project-preview-content");
projectPreviewClose	Line 78: const projectPreviewClose = document.querySelector("#project-preview-close");
projectPreviewBack	Line 79: const projectPreviewBack = document.querySelector("#project-preview-back");
projectPreviewForward	Line 80: const projectPreviewForward = document.querySelector("#project-preview-forward");

JSON/YAML-style keys detected

Item	Explanation
skin-light-blue	Line 433: "skin-light-blue": "appearance-light-blue-red-click",
skin-clean-white	Line 434: "skin-clean-white": "appearance-white-blue-click",
skin-deep-navy	Line 435: "skin-deep-navy": "appearance-deep-navy-cyan-click",
skin-red-warm	Line 436: "skin-red-warm": "appearance-warm-red-pale-click",
skin-emerald-instrument	Line 437: "skin-emerald-instrument": "appearance-emerald-mint-click",
skin-amber-power	Line 438: "skin-amber-power": "appearance-amber-cream-click",
skin-violet-mixed	Line 439: "skin-violet-mixed": "appearance-violet-lilac-click",
skin-graphite-asic	Line 440: "skin-graphite-asic": "appearance-graphite-white-click",
analog-opamp-topology	Line 441: "analog-opamp-topology": "appearance-light-blue-red-click",
analog-power-charger	Line 442: "analog-power-charger": "appearance-amber-cream-click",
analog-filter-front-end	Line 443: "analog-filter-front-end": "appearance-light-blue-red-click",
analog-sensor-interface	Line 444: "analog-sensor-interface": "appearance-emerald-mint-click",
analog-mixed-signal-timing	Line 445: "analog-mixed-signal-timing": "appearance-violet-lilac-click",
digital-fpga-pipeline	Line 446: "digital-fpga-pipeline": "appearance-deep-navy-cyan-click",
digital-asic-block	Line 447: "digital-asic-block": "appearance-graphite-white-click",
digital-verification-suite	Line 448: "digital-verification-suite": "appearance-graphite-white-click",
digital-interface-controller	Line 449: "digital-interface-controller": "appearance-deep-navy-cyan-click",
digital-signal-processing	Line 450: "digital-signal-processing": "appearance-violet-lilac-click",
embedded-stm32-sensor	Line 451: "embedded-stm32-sensor": "appearance-emerald-mint-click",
embedded-rtos-control	Line 452: "embedded-rtos-control": "appearance-graphite-white-click",
embedded-power-monitor	Line 453: "embedded-power-monitor": "appearance-amber-cream-click",
embedded-wireless-node	Line 454: "embedded-wireless-node": "appearance-deep-navy-cyan-click",
embedded-motor-control	Line 455: "embedded-motor-control": "appearance-violet-lilac-click"
engineering-file	Line 2512: "engineering-file": "Engineering file link",
google-drive	Line 2515: "google-drive": "Google Drive link",
Brief	Line 2656: "Brief": "Overview",
Project Brief	Line 2657: "Project Brief": "Overview",
Design Brief	Line 2658: "Design Brief": "Design Overview",
Simulation Brief	Line 2659: "Simulation Brief": "Simulation Overview"
--responsive-section-card-min	Line 2901: "--responsive-section-card-min": profile.sectionCardMin,
--responsive-content-max	Line 2902: "--responsive-content-max": profile.contentMax,
--responsive-media-max	Line 2903: "--responsive-media-max": profile.mediaMax,
--responsive-touch-target	Line 2904: "--responsive-touch-target": profile.touchTarget
--accent	Line 2919: "--accent": accent,
--template-bg	Line 2920: "--template-bg": visual.background,
--template-panel	Line 2921: "--template-panel": visual.panel,
--template-accent	Line 2922: "--template-accent": visual.accent,
--template-hover	Line 2923: "--template-hover": visual.hover,
--template-click	Line 2924: "--template-click": visual.click,
--template-click-text	Line 2925: "--template-click-text": visual.clickText,
--template-line	Line 2926: "--template-line": visual.line,

Item	Explanation
--template-text	Line 2927: "--template-text": visual.text,
--accent	Line 2952: "--accent": accent,
--template-bg	Line 2953: "--template-bg": visual.background,
--template-panel	Line 2954: "--template-panel": visual.panel,
--template-accent	Line 2955: "--template-accent": visual.accent,
--template-hover	Line 2956: "--template-hover": visual.hover,
--template-click	Line 2957: "--template-click": visual.click,
--template-click-text	Line 2958: "--template-click-text": visual.clickText,
--template-line	Line 2959: "--template-line": visual.line,
--template-text	Line 2960: "--template-text": visual.text,
compile-code	Line 7449: "compile-code": () => renderCompileCodeSection(project),

Event handlers and user interactions

Item	Explanation
Line 1048	document.addEventListener("pointerdown", beginDialogResize);
Line 1049	document.addEventListener("pointerdown", beginDialogDrag);
Line 1050	document.addEventListener("pointermove", moveDialogResize);
Line 1051	document.addEventListener("pointermove", moveDialogDrag);
Line 1052	document.addEventListener("pointerup", endDialogResize);
Line 1053	document.addEventListener("pointerup", endDialogDrag);
Line 1054	document.addEventListener("pointercancel", endDialogResize);
Line 1055	document.addEventListener("pointercancel", endDialogDrag);
Line 1056	document.addEventListener("click", (event) => {
Line 1061	document.addEventListener("click", (event) => {
Line 1067	dialog.addEventListener("close", () => {
Line 1076	window.addEventListener("resize", () => {
Line 1319	document.addEventListener("visibilitychange", () => {
Line 6166	siteLinkDialog.addEventListener("close", onClose);
Line 8325	dialog.addEventListener("close", onClose);
Line 8747	templateSelect.addEventListener("change", () => {
Line 8752	templatePreviewClose.addEventListener("click", () => {
Line 8880	builderGuideOpen.addEventListener("click", () => {
Line 8887	builderGuideClose.addEventListener("click", () => {
Line 8891	builderGuideTopicClose?.addEventListener("click", () => {
Line 8895	builderGuideTopicDialog?.addEventListener("close", () => {
Line 8899	builderGuideSearch?.addEventListener("input", filterBuilderGuide);
Line 8900	builderGuideDialog?.addEventListener("click", (event) => {
Line 8915	builderGuideDialog?.addEventListener("keydown", (event) => {
Line 8923	projectWindowTitle.addEventListener("dblclick", (event) => {
Line 8929	projectWindowTitle.addEventListener("keydown", (event) => {
Line 8946	projectWindowTitle.addEventListener("contextmenu", (event) => {

Item	Explanation
Line 8954	projectTitleMenu.addEventListener("click", async (event) => {
Line 8991	titleRenameSave.addEventListener("click", () => endTitleRename(true));
Line 8992	titleRenameCancel.addEventListener("click", () => endTitleRename(false));
Line 8993	projectMetaClose.addEventListener("click", () => closeDialogElement(projectMetaDialog));
Line 8995	viewProjectPreviewButton.addEventListener("click", openProjectPortfolioPreview);
Line 8996	projectPreviewClose.addEventListener("click", closeProjectPreviewWindow);
Line 8997	projectPreviewDialog.addEventListener("close", () => {
Line 9001	projectPreviewBack.addEventListener("click", projectPreviewBackStep);
Line 9002	projectPreviewForward.addEventListener("click", projectPreviewForwardStep);
Line 9004	projectPreviewContent.addEventListener("click", (event) => {
Line 9012	saveProjectButton.addEventListener("click", saveSelectedProjectToPortfolio);
Line 9013	saveProjectCloseButton.addEventListener("click", saveSelectedProjectAndClose);
Line 9014	openPortfolioPreviewButton.addEventListener("click", openPortfolioPreview);
Line 9015	saveAllSectionsButton.addEventListener("click", saveAllSections);
Line 9016	portfolioPreviewClose.addEventListener("click", () => {
Line 9019	portfolioPreviewDialog.addEventListener("close", () => {
Line 9022	publishTargetOpen?.addEventListener("click", openPublishTargetDialog);
Line 9023	publishTargetClose?.addEventListener("click", () => {
Line 9026	publishTargetCancel?.addEventListener("click", () => {
Line 9029	publishTargetForm?.addEventListener("submit", savePublishTarget);
Line 9030	publishTargetSync?.addEventListener("click", syncFromPublishTarget);
Line 9031	publishTargetCheck?.addEventListener("click", loadSystemReadiness);
Line 9032	publishTargetInstallGit?.addEventListener("click", installGitForPublishing);
Line 9033	publishTargetRepository?.addEventListener("input", () => {
Line 9036	publishTargetDomain?.addEventListener("input", () => {
Line 9040	publishTargetUsername?.addEventListener("input", markPublishTargetNeedsAuthentication);
Line 9041	publishTargetPassword?.addEventListener("input", markPublishTargetNeedsAuthentication);
Line 9042	publishResultClose.addEventListener("click", () => {
Line 9046	appUpdateClose?.addEventListener("click", () => {
Line 9050	appUpdateLater?.addEventListener("click", () => {
Line 9058	appUpdateSkip?.addEventListener("click", () => {
Line 9065	appUpdateApply?.addEventListener("click", () => {
Line 9069	appUpdateCheckButton?.addEventListener("click", () => {
Line 9073	securityReportOpen?.addEventListener("click", openSecurityReport);
Line 9074	securityReportRefresh?.addEventListener("click", openSecurityReport);
Line 9075	securityReportClose?.addEventListener("click", () => {
Line 9078	securityReportOk?.addEventListener("click", () => {
Line 9082	addProjectButton.addEventListener("click", () => {
Line 9086	projectSearchInput?.addEventListener("input", () => {
Line 9091	projectSearchClear?.addEventListener("click", () => {
Line 9100	quickOpenSelected?.addEventListener("click", () => {

Item	Explanation
Line 9106	quickPreviewSelected?.addEventListener("click", () => {
Line 9112	addCategoryButton?.addEventListener("click", openCategoryDialog);
Line 9113	categoryCancel?.addEventListener("click", () => closeDialogElement(categoryDialog, "cancel"));
Line 9114	categoryForm?.addEventListener("submit", (event) => {
Line 9120	addSiteSectionButton.addEventListener("click", addSiteSection);
Line 9122	funFactsInput?.addEventListener("input", () => {
Line 9128	saveFunFactsButton?.addEventListener("click", () => {
Line 9135	clearFunFactsButton?.addEventListener("click", () => {
Line 9146	input.addEventListener("input", () => {
Line 9154	saveSiteContentButton?.addEventListener("click", () => {
Line 9171	input.addEventListener("input", () => {
Line 9179	saveProfileContentButton?.addEventListener("click", () => {

Function-by-function detail

isHdlLanguage (template-preview.js, lines 240-242)

isHdlLanguage part of the private builder frontend and editing experience. In this file, it appears around lines 240-242 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 241: return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
240: function isHdlLanguage(language = "") {
241:   return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));
242: }
```

inferCompileFileRole (template-preview.js, lines 244-251)

inferCompileFileRole part of the compile code language/tool/build/run flow. In this file, it appears around lines 244-251 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isHdlLanguage, toLowerCase, b, and tb. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 245: if (!isHdlLanguage(language)) return "source";; line 248: return "testbench";; line 250: return "design";.

Important local values include haystack at line 246. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
244: function inferCompileFileRole(fileName = "", code = "", language = "") {
245:   if (!isHdlLanguage(language)) return "source";
246:   const haystack = `${fileName}\n${code}`.toLowerCase();
247:   if (/^(\b(tb|testbench)\b|^([_-])tb([_-]|\.)|test[_-]?bench|\$dumpfile|\$dumpvars|module\$s+tb[_a-z0-9]*\/i.test(haystack)) {
248:     return "testbench";
249:   }
250:   return "design";
251: }
```

normalizeCompileFileRole (template-preview.js, lines 253-257)

normalizeCompileFileRole part of the compile code language/tool/build/run flow. In this file, it appears around lines 253-257 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isHdlLanguage, trim, toLowerCase, replace, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 254: if (!isHdlLanguage(language)) return "source";; line 256: return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";.

Important local values include clean at line 255. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
253: function normalizeCompileFileRole(value = "", language = "") {
254:   if (!isHdlLanguage(language)) return "source";
255:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
256:   return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";
257: }
```

compileFileRoleLabel (template-preview.js, lines 259-263)

compileFileRoleLabel part of the compile code language/tool/build/run flow. In this file, it appears around lines 259-263 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 260: if (role === "testbench") return "Testbench";; line 261: if (role === "design") return "Design";; line 262: return "Program source";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
259: function compileFileRoleLabel(role = "source") {
260:   if (role === "testbench") return "Testbench";
261:   if (role === "design") return "Design";
262:   return "Program source";
263: }
```

setStatus (template-preview.js, lines 509-511)

setStatus part of the private builder frontend and editing experience. In this file, it appears around lines 509-511 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
509: function setStatus(message) {
510:   builderStatus.textContent = message;
511: }
```

setSaveState (template-preview.js, lines 513-517)

setSaveState part of the private builder frontend and editing experience. In this file, it appears around lines 513-517 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 514: if (!builderSaveState) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
513: function setSaveState(state, message) {
514:   if (!builderSaveState) return;
515:   builderSaveState.dataset.state = state || "loaded";
516:   builderSaveState.textContent = message || "Loaded";
517: }
```

projectSearchText (template-preview.js, lines 519-521)

projectSearchText part of the private builder frontend and editing experience. In this file, it appears around lines 519-521 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, replace, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 520: return String(value || "").toLowerCase().replace(/\s+/g, " ").trim();.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
519: function projectSearchText(value = "") {
520:   return String(value || "").toLowerCase().replace(/\s+/g, " ").trim();
521: }
```

regexEscape (template-preview.js, lines 523-525)

regexEscape part of the private builder frontend and editing experience. In this file, it appears around lines 523-525 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 524: return String(value).replace(/[\.*+?^\$()\[\]\\\]/g, "\\\$&");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
523: function regexEscape(value = "") {
524:   return String(value).replace(/[\.*+?^$()\[\]\\\]/g, "\\$&");
525: }
```

highlightSearchText (template-preview.js, lines 527-533)

highlightSearchText part of the private builder frontend and editing experience. In this file, it appears around lines 527-533 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText, escapeHtml, regexEscape, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 530: if (!cleanQuery) return escapeHtml(text);; line 532: return escapeHtml(text).replace(matcher, "<mark>\$1</mark>");.

Important local values include text at line 528; cleanQuery at line 529; matcher at line 531. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
527: function highlightSearchText(value = "", query = "") {
```

```

528:   const text = String(value || "");
529:   const cleanQuery = projectSearchText(query);
530:   if (!cleanQuery) return escapeHtml(text);
531:   const matcher = new RegExp(`${regexEscape(cleanQuery)}\}`, "ig");
532:   return escapeHtml(text).replace(matcher, "<mark>$1</mark>");
533: }

```

projectSearchHaystack (template-preview.js, lines 535-555)

projectSearchHaystack part of the private builder frontend and editing experience. In this file, it appears around lines 535-555 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText, stringify, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 536: return projectSearchText([

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

535: function projectSearchHaystack(project = {}, category = {}) {
536:   return projectSearchText([
537:     category.label,
538:     category.description,
539:     project.title,
540:     project.summary,
541:     project.status,
542:     project.category,
543:     project.templateLabel,
544:     ...(project.focus || []),
545:     ...(project.highlights || []),
546:     JSON.stringify(project.sections || []),
547:     JSON.stringify(project.design || {}),
548:     JSON.stringify(project.documents || []),
549:     JSON.stringify(project.tests || []),
550:     JSON.stringify(project.tools || []),
551:     JSON.stringify(project.links || []),
552:     JSON.stringify(project.media || []),
553:     JSON.stringify(project.compileCode || {}),
554:   ].filter(Boolean).join(" "));
555: }

```

projectMatchesSearch (template-preview.js, lines 557-561)

projectMatchesSearch part of the private builder frontend and editing experience. In this file, it appears around lines 557-561 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText, projectSearchHaystack, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 559: if (!cleanQuery) return true;; line 560: return projectSearchHaystack(project, category).includes(cleanQuery);.

Important local values include cleanQuery at line 558. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

557: function projectMatchesSearch(project, category, query) {
558:   const cleanQuery = projectSearchText(query);
559:   if (!cleanQuery) return true;
560:   return projectSearchHaystack(project, category).includes(cleanQuery);
561: }

```

updateProjectSearchStatus (template-preview.js, lines 563-575)

updateProjectSearchStatus updates ui, state, cache, or stored data. In this file, it appears around lines 563-575 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 569: if (!projectSearchStatus) return;; line 572: return;.

Important local values include cleanQuery at line 564. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
563: function updateProjectSearchStatus(matchCount = catalog.projects.length) {
564:   const cleanQuery = projectSearchText(projectSearchQuery);
565:   if (projectSearchInput && document.activeElement !== projectSearchInput) {
566:     projectSearchInput.value = projectSearchQuery;
567:   }
568:   if (projectSearchClear) projectSearchClear.hidden = !cleanQuery;
569:   if (!projectSearchStatus) return;
570:   if (!cleanQuery) {
571:     projectSearchStatus.textContent = "Showing all projects.";
572:     return;
573:   }
574:   projectSearchStatus.textContent = `${matchCount} matching project${matchCount === 1 ? "" : "s"} for
"${projectSearchQuery}"`;
575: }
```

updateBuilderWorkflow (template-preview.js, lines 577-597)

updateBuilderWorkflow updates ui, state, cache, or stored data. In this file, it appears around lines 577-597 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, Set, map, filter, has, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include project at line 578; savedIds at line 579; savedCount at line 580; visibleSections at line 581. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
577: function updateBuilderWorkflow() {
578:   const project = selectedProject();
579:   const savedIds = new Set((savedPortfolioCatalog.projects || []).map((item) => item.id));
580:   const savedCount = (catalog.projects || []).filter((projectItem) =>
savedIds.has(projectItem.id)).length;
581:   const visibleSections = (catalog.siteSections || []).filter(siteSectionRenderable).length;
582:   if (workflowSelectedProject) {
583:     workflowSelectedProject.textContent = project ? project.title || "Untitled project" : "No project
selected";
584:   }
585:   if (workflowTotalProjects) {
586:     workflowTotalProjects.textContent = `${catalog.projects.length} project${catalog.projects.length ===
1 ? "" : "s"}`;
587:   }
588:   if (workflowSavedProjects) {
589:     workflowSavedProjects.textContent = `${savedCount} parsed`;
590:   }
591:   if (workflowVisibleSections) {
592:     workflowVisibleSections.textContent = `${visibleSections} live section${visibleSections === 1 ? "" :
"s"}`;
593:   }
594:   [quickOpenSelected, quickPreviewSelected].forEach((button) => {
595:     if (button) button.disabled = !project;
596:   });
597: }
```

savedCount (template-preview.js, lines 580-584)

savedCount persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 580-584 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, and has. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include savedCount at line 580; visibleSections at line 581. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
580:   const savedCount = (catalog.projects || []).filter((projectItem) =>
savedIds.has(projectItem.id)).length;
581:   const visibleSections = (catalog.siteSections || []).filter(siteSectionRenderable).length;
582:   if (workflowSelectedProject) {
583:     workflowSelectedProject.textContent = project ? project.title || "Untitled project" : "No project
selected";
584:   }
```

dialogDragHandles (template-preview.js, lines 599-604)

dialogDragHandles part of the private builder frontend and editing experience. In this file, it appears around lines 599-604 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 600: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
599: function dialogDragHandles(dialog) {
600:   return [
601:     ...dialog.querySelectorAll(".project-dialog-heading, .project-preview-heading, .template-dialog-
heading"),
602:     ...dialog.querySelectorAll(".asset-dialog form > div:first-child")
603:   ];
604: }
```

dialogWindowActionTarget (template-preview.js, lines 606-608)

dialogWindowActionTarget part of the private builder frontend and editing experience. In this file, it appears around lines 606-608 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 607: return handle.querySelector(".project-window-actions, .section-view-actions") || handle;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
606: function dialogWindowActionTarget(handle) {
607:   return handle.querySelector(".project-window-actions, .section-view-actions") || handle;
608: }
```

updateDialogWindowButtons (template-preview.js, lines 610-635)

updateDialogWindowButtons updates ui, state, cache, or stored data. In this file, it appears around lines 610-635 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, querySelectorAll, forEach, setAttribute, dialogCanStepBack, and dialogCanStepForward. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include isMinimized at line 611; isHidden at line 612; isMaximized at line 613. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:


```

610: function updateDialogWindowButtons(dialog) {
611:   const isMinimized = dialog.classList.contains("is-minimized-dialog");
612:   const isHidden = dialog.classList.contains("is-hidden-dialog");
613:   const isMaximized = dialog.classList.contains("is-maximized-dialog");
614:   dialog.querySelectorAll("[data-dialog-minimize='true']").forEach((button) => {
615:     button.textContent = isMinimized ? "+" : "\u2212";
616:     button.title = isMinimized ? "Restore window" : "Minimize window";
617:     button.setAttribute("aria-label", isMinimized ? "Restore window" : "Minimize window");
618:   });
619:   dialog.querySelectorAll("[data-dialog-hide='true']").forEach((button) => {
620:     button.textContent = isHidden ? "\u25b4" : "\u25be";
621:     button.title = isHidden ? "Show window" : "Hide window";
622:     button.setAttribute("aria-label", isHidden ? "Show window" : "Hide window");
623:   });
624:   dialog.querySelectorAll("[data-dialog-maximize='true']").forEach((button) => {
625:     button.textContent = isMaximized ? "\u2750" : "\u25a1";
626:     button.title = isMaximized ? "Restore size" : "Maximize window";
627:     button.setAttribute("aria-label", isMaximized ? "Restore size" : "Maximize window");
628:   });
629:   dialog.querySelectorAll("[data-dialog-back='true']").forEach((button) => {
630:     button.disabled = !dialogCanStepBack(dialog);
631:   });
632:   dialog.querySelectorAll("[data-dialog-forward='true']").forEach((button) => {
633:     button.disabled = !dialogCanStepForward(dialog);
634:   });
635: }

```

dialogTitleForDock (template-preview.js, lines 637-639)

dialogTitleForDock part of the private builder frontend and editing experience. In this file, it appears around lines 637-639 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 638: return dialog.querySelector("h2")?.textContent?.trim() || "Window";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

637: function dialogTitleForDock(dialog) {
638:   return dialog.querySelector("h2")?.textContent?.trim() || "Window";
639: }

```

makeDialogControl (template-preview.js, lines 641-650)

makeDialogControl part of the private builder frontend and editing experience. In this file, it appears around lines 641-650 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, and setAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 649: return button;.

Important local values include button at line 642. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

641: function makeDialogControl(action, label, title) {
642:   const button = document.createElement("button");
643:   button.type = "button";
644:   button.className = `dialog-window-control dialog-window-${action}-control`;
645:   button.dataset.dialogAction = action;
646:   button.textContent = label;
647:   button.title = title;
648:   button.setAttribute("aria-label", title);
649:   return button;
650: }

```

ensureDialogLeftControls (template-preview.js, lines 652-674)

ensureDialogLeftControls part of the private builder frontend and editing experience. In this file, it appears around lines 652-674 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, createElement, makeDialogControl, append, prepend, forEach, toUpperCase, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 653: if (handle.querySelector(".dialog-window-left-controls")) return;.

Important local values include titlebar at line 654; controls at line 655; refresh at line 659. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
652: function ensureDialogLeftControls(dialog, handle) {
653:   if (handle.querySelector(".dialog-window-left-controls")) return;
654:   const titlebar = handle.querySelector(".section-view-titlebar");
655:   const controls = document.createElement("div");
656:   controls.className = "dialog-window-left-controls";
657:
658:   if (titlebar?.querySelector(".section-view-back, .section-view-forward")) {
659:     const refresh = makeDialogControl("refresh", "\u21bb", "Refresh window");
660:     refresh.dataset.dialogRefresh = "true";
661:     controls.append(refresh);
662:     titlebar.prepend(controls);
663:   } else {
664:     [
665:       makeDialogControl("back", "\u2190", "Back"),
666:       makeDialogControl("forward", "\u2192", "Forward"),
667:       makeDialogControl("refresh", "\u21bb", "Refresh window")
668:     ].forEach((button) => {
669:       button.dataset[`dialog${button.dataset.dialogAction[0].toUpperCase()}${button.dataset.dialogAction.slice(1)}`] =
        "true";
670:       controls.append(button);
671:     });
672:     handle.prepend(controls);
673:   }
674: }
```

ensureDialogWindowControls (template-preview.js, lines 676-720)

ensureDialogWindowControls part of the private builder frontend and editing experience. In this file, it appears around lines 676-720 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, remove, ensureDialogLeftControls, dialogWindowActionTarget, querySelector, createElement, append, makeDialogControl, find, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include target at line 679; cluster at line 680; hide at line 688; minimize at line 693; maximize at line 698; closeButton at line 703; cloneClose at line 714. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
676: function ensureDialogWindowControls(dialog, handle) {
677:   dialog.querySelectorAll(".dialog-minimize-button").forEach((button) => button.remove());
678:   ensureDialogLeftControls(dialog, handle);
679:   const target = dialogWindowActionTarget(handle);
680:   let cluster = target.querySelector(".dialog-window-control-cluster");
681:   if (!cluster) {
682:     cluster = document.createElement("div");
683:     cluster.className = "dialog-window-control-cluster";
684:     target.append(cluster);
685:   }
686:
687:   if (!cluster.querySelector("[data-dialog-hide='true']")) {
688:     const hide = makeDialogControl("hide", "\u25be", "Hide window");
689:     hide.dataset.dialogHide = "true";
690:     cluster.append(hide);
691:   }
692:   if (!cluster.querySelector("[data-dialog-minimize='true']")) {
693:     const minimize = makeDialogControl("minimize", "\u2212", "Minimize window");
694:     minimize.dataset.dialogMinimize = "true";
695:     cluster.append(minimize);
```

```

696:   }
697:   if (!cluster.querySelector("[data-dialog-maximize='true']")) {
698:     const maximize = makeDialogControl("maximize", "\u25a1", "Maximize window");
699:     maximize.dataset.dialogMaximize = "true";
700:     cluster.append(maximize);
701:   }
702:
703:   let closeButton = cluster.querySelector(".project-window-close, .close-control, [data-dialog-
close='true']");
704:   if (!closeButton) {
705:     closeButton = [...dialog.querySelectorAll(".project-window-close, .close-control")]
706:       .find((button) => button.closest(".project-dialog-heading, .project-preview-heading, .template-
dialog-heading, .asset-dialog form > div:first-child") === handle);
707:   }
708:   if (closeButton) {
709:     closeButton.classList.add("dialog-window-control", "dialog-close-button");
710:     closeButton.dataset.dialogAction = "close";
711:     closeButton.dataset.dialogClose = "true";
712:     cluster.append(closeButton);
713:   } else {
714:     const cloneClose = makeDialogControl("close", "\u00d7", "Close window");
715:     cloneClose.classList.add("project-window-close", "dialog-close-button");
716:     cloneClose.dataset.dialogClose = "true";
717:     cluster.append(cloneClose);
718:   }
719:   updateDialogWindowButtons(dialog);
720: }

```

ensureDialogResizeHandles (template-preview.js, lines 722-731)

ensureDialogResizeHandles part of the private builder frontend and editing experience. In this file, it appears around lines 722-731 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, forEach, createElement, setAttribute, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 723: if (dialog.querySelector("[data-dialog-resize-handle]")) return;

Important local values include handle at line 725. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

722: function ensureDialogResizeHandles(dialog) {
723:   if (dialog.querySelector("[data-dialog-resize-handle]")) return;
724:   ["n", "e", "s", "w", "ne", "nw", "se", "sw"].forEach((direction) => {
725:     const handle = document.createElement("div");
726:     handle.className = `dialog-resize-handle resize-${direction}`;
727:     handle.dataset.dialogResizeHandle = direction;
728:     handle.setAttribute("aria-hidden", "true");
729:     dialog.append(handle);
730:   });
731: }

```

markDialogDragHandles (template-preview.js, lines 733-740)

markDialogDragHandles part of the private builder frontend and editing experience. In this file, it appears around lines 733-740 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references dialogDragHandles, forEach, ensureDialogWindowControls, and ensureDialogResizeHandles. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

733: function markDialogDragHandles(dialog) {
734:   dialogDragHandles(dialog).forEach((handle) => {
735:     handle.dataset.dialogDragHandle = "true";
736:     handle.title = handle.title || "Drag to move this window";
737:     ensureDialogWindowControls(dialog, handle);
738:   });
739:   ensureDialogResizeHandles(dialog);
740: }

```

draggableDialogFromEvent (template-preview.js, lines 742-751)

draggableDialogFromEvent part of the private builder frontend and editing experience. In this file, it appears around lines 742-751 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and contains. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 743: if (event.target.closest("[data-dialog-resize-handle]")) return null;; line 745: if (!handle) return null;; line 747: if (!dialog?.open) return null;; line 748: if (dialog.classList.contains("is-hidden-dialog")) return null;. There are 2 additional return statements in the function body.

Important local values include handle at line 744; dialog at line 746. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
742: function draggableDialogFromEvent(event) {
743:   if (event.target.closest("[data-dialog-resize-handle]")) return null;
744:   const handle = event.target.closest("[data-dialog-drag-handle='true']");
745:   if (!handle) return null;
746:   const dialog = handle.closest("dialog");
747:   if (!dialog?.open) return null;
748:   if (dialog.classList.contains("is-hidden-dialog")) return null;
749:   if (event.target.closest("button, a, input, textarea, select, label, summary,
[contenteditable='true']")) return null;
750:   return dialog;
751: }
```

clampDialogPosition (template-preview.js, lines 753-762)

clampDialogPosition part of the private builder frontend and editing experience. In this file, it appears around lines 753-762 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, max, and min. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 758: return {.

Important local values include rect at line 754; margin at line 755; maxLeft at line 756; maxTop at line 757. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
753: function clampDialogPosition(left, top, dialog) {
754:   const rect = dialog.getBoundingClientRect();
755:   const margin = 12;
756:   const maxLeft = Math.max(margin, window.innerWidth - rect.width - margin);
757:   const maxTop = Math.max(margin, window.innerHeight - rect.height - margin);
758:   return {
759:     left: Math.min(Math.max(left, margin), maxLeft),
760:     top: Math.min(Math.max(top, margin), maxTop)
761:   };
762: }
```

anchorDialogForDrag (template-preview.js, lines 764-773)

anchorDialogForDrag part of the private builder frontend and editing experience. In this file, it appears around lines 764-773 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, clampDialogPosition, and add. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include rect at line 765; position at line 766. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

764: function anchorDialogForDrag(dialog) {
765:   const rect = dialog.getBoundingClientRect();
766:   const position = clampDialogPosition(rect.left, rect.top, dialog);
767:   dialog.classList.add("is-draggable-dialog");
768:   dialog.style.left = `${position.left}px`;
769:   dialog.style.top = `${position.top}px`;
770:   dialog.style.right = "auto";
771:   dialog.style.bottom = "auto";
772:   dialog.style.margin = "0";
773: }

```

saveDialogRestoreState (template-preview.js, lines 775-788)

saveDialogRestoreState persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 775-788 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 776: if (dialog.dataset.restoreWindowState) return;.

Important local values include rect at line 777. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

775: function saveDialogRestoreState(dialog) {
776:   if (dialog.dataset.restoreWindowState) return;
777:   const rect = dialog.getBoundingClientRect();
778:   dialog.dataset.restoreWindowState = JSON.stringify({
779:     bottom: dialog.style.bottom || "",
780:     height: dialog.style.height || `${rect.height}px`,
781:     left: dialog.style.left || `${rect.left}px`,
782:     margin: dialog.style.margin || "",
783:     maxHeight: dialog.style.maxHeight || "",
784:     right: dialog.style.right || "",
785:     top: dialog.style.top || `${rect.top}px`,
786:     width: dialog.style.width || `${rect.width}px`
787:   });
788: }

```

restoreDialogWindowState (template-preview.js, lines 790-805)

restoreDialogWindowState part of the private builder frontend and editing experience. In this file, it appears around lines 790-805 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, removeAttribute, entries, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 800: return;.

Important local values include state at line 791. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

790: function restoreDialogWindowState(dialog) {
791:   let state = null;
792:   try {
793:     state = JSON.parse(dialog.dataset.restoreWindowState || "null");
794:   } catch {
795:     state = null;
796:   }
797:   delete dialog.dataset.restoreWindowState;
798:   if (!state) {
799:     dialog.removeAttribute("style");
800:     return;
801:   }
802:   Object.entries(state).forEach(([key, value]) => {
803:     dialog.style[key] = value;
804:   });
805: }

```

anchorDialogForResize (template-preview.js, lines 807-814)

anchorDialogForResize part of the private builder frontend and editing experience. In this file, it appears around lines 807-814 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references anchorDialogForDrag, getBoundingClientRect, and add. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include rect at line 809. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
807: function anchorDialogForResize(dialog) {
808:   anchorDialogForDrag(dialog);
809:   const rect = dialog.getBoundingClientRect();
810:   dialog.classList.add("is-resized-dialog");
811:   dialog.style.width = `${rect.width}px`;
812:   dialog.style.height = `${rect.height}px`;
813:   dialog.style.maxHeight = "none";
814: }
```

resizeDialogBounds (template-preview.js, lines 816-843)

resizeDialogBounds part of the private builder frontend and editing experience. In this file, it appears around lines 816-843 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references min, max, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 842: return { height, left, top, width };

Important local values include margin at line 817; direction at line 818; minWidth at line 819; minHeight at line 820; dx at line 822; dy at line 823; right at line 832; bottom at line 837. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
816: function resizeDialogBounds(state, event) {
817:   const margin = 12;
818:   const direction = state.direction;
819:   const minWidth = Math.min(320, Math.max(180, window.innerWidth - margin * 2));
820:   const minHeight = Math.min(170, Math.max(120, window.innerHeight - margin * 2));
821:   let { left, top, width, height } = state.rect;
822:   const dx = event.clientX - state.startX;
823:   const dy = event.clientY - state.startY;
824:
825:   if (direction.includes("e")) {
826:     width = Math.min(Math.max(state.rect.width + dx, minWidth), window.innerWidth - left - margin);
827:   }
828:   if (direction.includes("s")) {
829:     height = Math.min(Math.max(state.rect.height + dy, minHeight), window.innerHeight - top - margin);
830:   }
831:   if (direction.includes("w")) {
832:     const right = state.rect.left + state.rect.width;
833:     left = Math.min(Math.max(state.rect.left + dx, margin), right - minWidth);
834:     width = right - left;
835:   }
836:   if (direction.includes("n")) {
837:     const bottom = state.rect.top + state.rect.height;
838:     top = Math.min(Math.max(state.rect.top + dy, margin), bottom - minHeight);
839:     height = bottom - top;
840:   }
841:
842:   return { height, left, top, width };
843: }
```

beginDialogResize (template-preview.js, lines 845-863)

beginDialogResize part of the private builder frontend and editing experience. In this file, it appears around lines 845-863 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, contains, anchorDialogForResize, getBoundingClientRect, add, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 846: if (event.button !== 0) return;; line 849: if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog") || dialog.classList.contains("is-hidden-dialog")) return;.

Important local values include handle at line 847; dialog at line 848. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
845: function beginDialogResize(event) {
846:   if (event.button !== 0) return;
847:   const handle = event.target.closest("[data-dialog-resize-handle]");
848:   const dialog = handle?.closest("dialog");
849:   if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog") ||
dialog.classList.contains("is-hidden-dialog")) return;
850:   anchorDialogForResize(dialog);
851:   activeDialogResize = {
852:     dialog,
853:     direction: handle.dataset.dialogResizeHandle,
854:     pointerId: event.pointerId,
855:     pointerTarget: event.target,
856:     rect: dialog.getBoundingClientRect(),
857:     startX: event.clientX,
858:     startY: event.clientY
859:   };
860:   event.target.setPointerCapture?.(event.pointerId);
861:   dialog.classList.add("is-resizing-dialog");
862:   event.preventDefault();
863: }
```

moveDialogResize (template-preview.js, lines 865-872)

moveDialogResize part of the private builder frontend and editing experience. In this file, it appears around lines 865-872 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resizeDialogBounds. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 866: if (!activeDialogResize) return;.

Important local values include next at line 867. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
865: function moveDialogResize(event) {
866:   if (!activeDialogResize) return;
867:   const next = resizeDialogBounds(activeDialogResize, event);
868:   activeDialogResize.dialog.style.left = `${next.left}px`;
869:   activeDialogResize.dialog.style.top = `${next.top}px`;
870:   activeDialogResize.dialog.style.width = `${next.width}px`;
871:   activeDialogResize.dialog.style.height = `${next.height}px`;
872: }
```

endDialogResize (template-preview.js, lines 874-879)

endDialogResize part of the private builder frontend and editing experience. In this file, it appears around lines 874-879 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 875: if (!activeDialogResize) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
874: function endDialogResize() {
875:   if (!activeDialogResize) return;
876:   activeDialogResize.pointerTarget?.releasePointerCapture?.(activeDialogResize.pointerId);
877:   activeDialogResize.dialog.classList.remove("is-resizing-dialog");
878:   activeDialogResize = null;
```

879: }

beginDialogDrag (template-preview.js, lines 881-897)

beginDialogDrag part of the private builder frontend and editing experience. In this file, it appears around lines 881-897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references draggableDialogFromEvent, anchorDialogForDrag, getBoundingClientRect, add, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 882: if (event.button !== 0) return;; line 884: if (!dialog) return;.

Important local values include dialog at line 883; rect at line 886. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
881: function beginDialogDrag(event) {
882:   if (event.button !== 0) return;
883:   const dialog = draggableDialogFromEvent(event);
884:   if (!dialog) return;
885:   anchorDialogForDrag(dialog);
886:   const rect = dialog.getBoundingClientRect();
887:   activeDialogDrag = {
888:     dialog,
889:     offsetX: event.clientX - rect.left,
890:     offsetY: event.clientY - rect.top,
891:     pointerId: event.pointerId,
892:     pointerTarget: event.target
893:   };
894:   event.target.setPointerCapture?.(event.pointerId);
895:   dialog.classList.add("is-dragging-dialog");
896:   event.preventDefault();
897: }
```

moveDialogDrag (template-preview.js, lines 899-908)

moveDialogDrag part of the private builder frontend and editing experience. In this file, it appears around lines 899-908 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clampDialogPosition. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 900: if (!activeDialogDrag) return;.

Important local values include next at line 901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
899: function moveDialogDrag(event) {
900:   if (!activeDialogDrag) return;
901:   const next = clampDialogPosition(
902:     event.clientX - activeDialogDrag.offsetX,
903:     event.clientY - activeDialogDrag.offsetY,
904:     activeDialogDrag.dialog
905:   );
906:   activeDialogDrag.dialog.style.left = `${next.left}px`;
907:   activeDialogDrag.dialog.style.top = `${next.top}px`;
908: }
```

endDialogDrag (template-preview.js, lines 910-915)

endDialogDrag part of the private builder frontend and editing experience. In this file, it appears around lines 910-915 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 911: if (!activeDialogDrag) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
910: function endDialogDrag() {
911:   if (!activeDialogDrag) return;
912:   activeDialogDrag.pointerTarget?.releasePointerCapture?.(activeDialogDrag.pointerId);
913:   activeDialogDrag.dialog.classList.remove("is-dragging-dialog");
914:   activeDialogDrag = null;
915: }
```

toggleDialogMinimized (template-preview.js, lines 917-927)

toggleDialogMinimized part of the private builder frontend and editing experience. In this file, it appears around lines 917-927 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, toggleDialogHidden, anchorDialogForDrag, remove, toggle, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 918: if (!dialog) return;; line 921: return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
917: function toggleDialogMinimized(dialog) {
918:   if (!dialog) return;
919:   if (dialog.classList.contains("is-hidden-dialog")) {
920:     toggleDialogHidden(dialog);
921:     return;
922:   }
923:   anchorDialogForDrag(dialog);
924:   dialog.classList.remove("is-maximized-dialog");
925:   dialog.classList.toggle("is-minimized-dialog");
926:   updateDialogWindowButtons(dialog);
927: }
```

toggleDialogMaximized (template-preview.js, lines 929-949)

toggleDialogMaximized part of the private builder frontend and editing experience. In this file, it appears around lines 929-949 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, remove, restoreDialogWindowState, saveDialogRestoreState, add, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 930: if (!dialog) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
929: function toggleDialogMaximized(dialog) {
930:   if (!dialog) return;
931:   if (dialog.classList.contains("is-hidden-dialog")) dialog.classList.remove("is-hidden-dialog");
932:   if (dialog.classList.contains("is-maximized-dialog")) {
933:     dialog.classList.remove("is-maximized-dialog");
934:     restoreDialogWindowState(dialog);
935:   } else {
936:     saveDialogRestoreState(dialog);
937:     dialog.classList.remove("is-minimized-dialog");
938:     dialog.classList.add("is-draggable-dialog", "is-maximized-dialog");
939:     dialog.style.left = "0";
940:     dialog.style.top = "0";
941:     dialog.style.right = "auto";
942:     dialog.style.bottom = "auto";
943:     dialog.style.margin = "0";
944:     dialog.style.width = "100vw";
945:     dialog.style.height = "100dvh";
946:     dialog.style.maxHeight = "none";
947:   }
948:   updateDialogWindowButtons(dialog);
949: }
```

toggleDialogHidden (template-preview.js, lines 951-970)

toggleDialogHidden part of the private builder frontend and editing experience. In this file, it appears around lines 951-970 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, remove, restoreDialogWindowState, saveDialogRestoreState, add, min, calc, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 952: if (!dialog) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
951: function toggleDialogHidden(dialog) {
952:   if (!dialog) return;
953:   if (dialog.classList.contains("is-hidden-dialog")) {
954:     dialog.classList.remove("is-hidden-dialog");
955:     restoreDialogWindowState(dialog);
956:   } else {
957:     saveDialogRestoreState(dialog);
958:     dialog.classList.remove("is-minimized-dialog", "is-maximized-dialog");
959:     dialog.classList.add("is-draggable-dialog", "is-hidden-dialog");
960:     dialog.style.left = "auto";
961:     dialog.style.top = "auto";
962:     dialog.style.right = "12px";
963:     dialog.style.bottom = "10px";
964:     dialog.style.margin = "0";
965:     dialog.style.width = "min(300px, calc(100vw - 24px))";
966:     dialog.style.height = "32px";
967:     dialog.style.maxHeight = "32px";
968:   }
969:   updateDialogWindowButtons(dialog);
970: }
```

dialogCanStepBack (template-preview.js, lines 972-976)

dialogCanStepBack part of the private builder frontend and editing experience. In this file, it appears around lines 972-976 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 973: if (dialog === projectDialog) return projectWindowBackStack.length > 0;; line 974: if (dialog === projectPreviewDialog) return activeProjectPreviewSectionIndex >= 0;; line 975: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
972: function dialogCanStepBack(dialog) {
973:   if (dialog === projectDialog) return projectWindowBackStack.length > 0;
974:   if (dialog === projectPreviewDialog) return activeProjectPreviewSectionIndex >= 0;
975:   return false;
976: }
```

dialogCanStepForward (template-preview.js, lines 978-982)

dialogCanStepForward part of the private builder frontend and editing experience. In this file, it appears around lines 978-982 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 979: if (dialog === projectDialog) return projectWindowForwardStack.length > 0;; line 980: if (dialog === projectPreviewDialog) return activeProjectPreviewForwardStack.length > 0;; line 981: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
978: function dialogCanStepForward(dialog) {
979:   if (dialog === projectDialog) return projectWindowForwardStack.length > 0;
980:   if (dialog === projectPreviewDialog) return activeProjectPreviewForwardStack.length > 0;
981:   return false;
982: }
```

renderProjectWindowSectionFromHistory (template-preview.js, lines 984-990)

renderProjectWindowSectionFromHistory renders ui, markup, or display output. In this file, it appears around lines 984-990 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderAll, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
984: function renderProjectWindowSectionFromHistory(sectionId) {
985:   suppressProjectWindowHistory = true;
986:   activeSectionId = sectionId || "brief";
987:   renderAll();
988:   suppressProjectWindowHistory = false;
989:   updateDialogWindowButtons(projectDialog);
990: }
```

stepProjectWindowBack (template-preview.js, lines 992-996)

stepProjectWindowBack part of the private builder frontend and editing experience. In this file, it appears around lines 992-996 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, renderProjectWindowSectionFromHistory, and pop. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 993: if (!projectWindowBackStack.length) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
992: function stepProjectWindowBack() {
993:   if (!projectWindowBackStack.length) return;
994:   projectWindowForwardStack.push(activeSectionId);
995:   renderProjectWindowSectionFromHistory(projectWindowBackStack.pop());
996: }
```

stepProjectWindowForward (template-preview.js, lines 998-1002)

stepProjectWindowForward part of the private builder frontend and editing experience. In this file, it appears around lines 998-1002 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, renderProjectWindowSectionFromHistory, and pop. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 999: if (!projectWindowForwardStack.length) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
998: function stepProjectWindowForward() {
999:   if (!projectWindowForwardStack.length) return;
1000:   projectWindowBackStack.push(activeSectionId);
1001:   renderProjectWindowSectionFromHistory(projectWindowForwardStack.pop());
1002: }
```

refreshDialogWindow (template-preview.js, lines 1004-1015)

refreshDialogWindow part of the private builder frontend and editing experience. In this file, it appears around lines 1004-1015 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveVisibleRichEditors, refreshOpenPreviews, renderPreview, renderAll, setStatus, and updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1004: function refreshDialogWindow(dialog) {
1005:   saveVisibleRichEditors();
1006:   if (dialog === projectPreviewDialog) {
1007:     refreshOpenPreviews();
1008:   } else if (dialog === portfolioPreviewDialog) {
1009:     renderPreview();
1010:   } else if (dialog === projectDialog) {
1011:     renderAll();
1012:   }
1013:   setStatus("Window refreshed.");
1014:   updateDialogWindowButtons(dialog);
1015: }
```

closeDialogFromControl (template-preview.js, lines 1017-1025)

closeDialogFromControl controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1017-1025 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, find, contains, click, and closeDialogElement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1022: return;.

Important local values include originalClose at line 1018. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1017: function closeDialogFromControl(dialog, button) {
1018:   const originalClose = [...dialog.querySelectorAll(".project-window-close, .close-control")]
1019:   .find((item) => item !== button && !item.dataset.dialogClose);
1020:   if (originalClose && !button.classList.contains("close-control") && !button.id) {
1021:     originalClose.click();
1022:     return;
1023:   }
1024:   closeDialogElement(dialog, "close");
1025: }
```

handleDialogWindowAction (template-preview.js, lines 1027-1044)

handleDialogWindowAction handles an event, request, command, or user action. In this file, it appears around lines 1027-1044 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, toggleDialogHidden, toggleDialogMinimized, toggleDialogMaximized, refreshDialogWindow, stepProjectWindowBack, projectPreviewBackStep, stepProjectWindowForward, projectPreviewForwardStep, and closeDialogFromControl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1029: if (!dialog?.open) return;.

Important local values include dialog at line 1028; action at line 1030. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1027: function handleDialogWindowAction(button) {
1028:   const dialog = button.closest("dialog");
1029:   if (!dialog?.open) return;
1030:   const action = button.dataset.dialogAction;
1031:   if (action === "hide") toggleDialogHidden(dialog);
1032:   if (action === "minimize") toggleDialogMinimized(dialog);
1033:   if (action === "maximize") toggleDialogMaximized(dialog);
1034:   if (action === "refresh") refreshDialogWindow(dialog);
1035:   if (action === "back") {
1036:     if (dialog === projectDialog) stepProjectWindowBack();
1037:     else if (dialog === projectPreviewDialog) projectPreviewBackStep();
1038:   }
1039:   if (action === "forward") {
1040:     if (dialog === projectDialog) stepProjectWindowForward();
1041:     else if (dialog === projectPreviewDialog) projectPreviewForwardStep();
1042:   }
1043:   if (action === "close") closeDialogFromControl(dialog, button);
1044: }

```

enableDraggableDialogs (template-preview.js, lines 1046-1084)

enableDraggableDialogs part of the private builder frontend and editing experience. In this file, it appears around lines 1046-1084 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, addEventListener, closest, handleDialogWindowAction, toggleDialogHidden, remove, updateDialogWindowButtons, getBoundingClientRect, and clampDialogPosition. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1058: if (!button) return;; line 1063: if (!hiddenDialog || event.target.closest("button")) return;.

Important local values include button at line 1057; hiddenDialog at line 1062; rect at line 1078; next at line 1079. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1046: function enableDraggableDialogs() {
1047:   document.querySelectorAll("dialog").forEach(markDialogDragHandles);
1048:   document.addEventListener("pointerdown", beginDialogResize);
1049:   document.addEventListener("pointerdown", beginDialogDrag);
1050:   document.addEventListener("pointermove", moveDialogResize);
1051:   document.addEventListener("pointermove", moveDialogDrag);
1052:   document.addEventListener("pointerup", endDialogResize);
1053:   document.addEventListener("pointerup", endDialogDrag);
1054:   document.addEventListener("pointercancel", endDialogResize);
1055:   document.addEventListener("pointercancel", endDialogDrag);
1056:   document.addEventListener("click", (event) => {
1057:     const button = event.target.closest("[data-dialog-action]");
1058:     if (!button) return;
1059:     handleDialogWindowAction(button);
1060:   });
1061:   document.addEventListener("click", (event) => {
1062:     const hiddenDialog = event.target.closest("dialog.is-hidden-dialog");
1063:     if (!hiddenDialog || event.target.closest("button")) return;
1064:     toggleDialogHidden(hiddenDialog);
1065:   });
1066:   document.querySelectorAll("dialog").forEach((dialog) => {
1067:     dialog.addEventListener("close", () => {
1068:       dialog.classList.remove("is-minimized-dialog");
1069:       dialog.classList.remove("is-hidden-dialog");
1070:       dialog.classList.remove("is-maximized-dialog");
1071:       dialog.classList.remove("is-resizing-dialog");
1072:       delete dialog.dataset.restoreWindowState;
1073:       updateDialogWindowButtons(dialog);
1074:     });
1075:   });
1076:   window.addEventListener("resize", () => {
1077:     document.querySelectorAll("dialog.is-draggable-dialog[open]").forEach((dialog) => {
1078:       const rect = dialog.getBoundingClientRect();
1079:       const next = clampDialogPosition(rect.left, rect.top, dialog);
1080:       dialog.style.left = `${next.left}px`;
1081:       dialog.style.top = `${next.top}px`;
1082:     });
1083:   });
1084: }

```

markDraftNeedsSave (template-preview.js, lines 1086-1089)

markDraftNeedsSave part of the private builder frontend and editing experience. In this file, it appears around lines 1086-1089 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `setSaveState`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1086: function markDraftNeedsSave() {  
1087:   draftSavedSinceChanges = false;  
1088:   setSaveState("dirty", "Save draft needed");  
1089: }
```

showBuilderError (template-preview.js, lines 1091-1097)

`showBuilderError` controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1091-1097 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `showModal`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1091: function showBuilderError(title, message, details = "") {  
1092:   publishResultEyebrow.textContent = "Builder check";  
1093:   publishResultTitle.textContent = title;  
1094:   publishResultMessage.textContent = message;  
1095:   publishResultOutput.textContent = details;  
1096:   publishResultDialog.showModal();  
1097: }
```

appUpdateWasSnoozed (template-preview.js, lines 1105-1110)

`appUpdateWasSnoozed` part of the private builder frontend and editing experience. In this file, it appears around lines 1105-1110 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `getItem`, and `now`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are `omb-snoozed-update-at`, and `omb-snoozed-update-version`.

It has 2 return statement(s). The most important visible returns are: line 1106: `if (!version) return false;`; line 1109: `return snoozedVersion === version && Date.now() - snoozedAt < APP_UPDATE_SNOOZE_MS;`.

Important local values include `snoozedVersion` at line 1107; `snoozedAt` at line 1108. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1105: function appUpdateWasSnoozed(version = "") {  
1106:   if (!version) return false;  
1107:   const snoozedVersion = localStorage.getItem("omb-snoozed-update-version") || "";  
1108:   const snoozedAt = Number(localStorage.getItem("omb-snoozed-update-at") || 0);  
1109:   return snoozedVersion === version && Date.now() - snoozedAt < APP_UPDATE_SNOOZE_MS;  
1110: }
```

shouldShowUpdateDialog (template-preview.js, lines 1112-1117)

`shouldShowUpdateDialog` part of the private builder frontend and editing experience. In this file, it appears around lines 1112-1117 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `getItem`, and `appUpdateWasSnoozed`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are `omb-dismissed-update-version`, and `omb-skipped-update-version`.

It has 3 return statement(s). The most important visible returns are: line 1113: if (!update.updateAvailable || !update.latestVersion) return false;; line 1115: if (skippedVersion === update.latestVersion) return false;; line 1116: return !appUpdateWasSnoozed(update.latestVersion);.

Important local values include skippedVersion at line 1114. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1112: function shouldShowUpdateDialog(update = {}) {
1113:   if (!update.updateAvailable || !update.latestVersion) return false;
1114:   const skippedVersion = localStorage.getItem("omb-skipped-update-version") || localStorage.getItem("omb-
dismissed-update-version") || "";
1115:   if (skippedVersion === update.latestVersion) return false;
1116:   return !appUpdateWasSnoozed(update.latestVersion);
1117: }
```

showUpdateDialog (template-preview.js, lines 1119-1158)

showUpdateDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1119-1158 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references shouldShowUpdateDialog, escapeHtml, filter, join, showModal, and show. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1121: if (!appUpdateDialog || (!force && !shouldShowUpdateDialog(update))) return;; line 1152: if (appUpdateDialog.open) return;.

Important local values include force at line 1120; hasInstaller at line 1123; updateBlocked at line 1124; hasUpdate at line 1125. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1119: function showUpdateDialog(update = {}, options = {}) {
1120:   const force = Boolean(options.force);
1121:   if (!appUpdateDialog || (!force && !shouldShowUpdateDialog(update))) return;
1122:   pendingAppUpdate = update;
1123:   const hasInstaller = Boolean(update.installerUrl);
1124:   const updateBlocked = Boolean(update.updateBlocked);
1125:   const hasUpdate = Boolean(update.updateAvailable && update.latestVersion);
1126:   appUpdateTitle.textContent = updateBlocked ? "Update paused" : hasUpdate ? "Update available" :
"Builder is up to date";
1127:   appUpdateMessage.textContent = updateBlocked
1128:     ? `OMB Portfolio Builder ${update.latestVersion || "this release"} is available, but this version is
paused for automatic updates.`
1129:     : hasUpdate
1130:     ? `OMB Portfolio Builder ${update.latestVersion} is available. You are running
${update.currentVersion || "an older version"}`
1131:     : `You are running OMB Portfolio Builder ${update.currentVersion || "the current installed
version"}.`;
1132:   appUpdateDetails.textContent = updateBlocked
1133:     ? (update.blockedReason || "This release is temporarily blocked from automatic updates. Check again
after the next release is available.")
1134:     : hasUpdate
1135:     ? "Choose Update to download the latest installer, close this app, and install the new version
automatically. Remind me later postpones the prompt; Skip this version ignores this release."
1136:     : "No newer released builder was found on GitHub Releases.";
1137:   appUpdateReleaseMeta.innerHTML = (hasUpdate || updateBlocked)
1138:     ? [
1139:       `<span>Current version: ${escapeHtml(update.currentVersion || "unknown")}</span>`,
1140:       `<span>Available version: ${escapeHtml(update.latestVersion || "unknown")}</span>`,
1141:       `update.releaseUrl ? `<span>Release source: GitHub Releases</span>` : ""
1142:     ].filter(Boolean).join("")
1143:     : [
1144:       `<span>Current version: ${escapeHtml(update.currentVersion || "unknown")}</span>`,
1145:       `update.checkedAt ? `<span>Checked: ${escapeHtml(update.checkedAt)}</span>` : ""
1146:     ].filter(Boolean).join("");
1147:   appUpdateLater.textContent = hasUpdate ? "Remind me later" : "Close";
1148:   appUpdateSkip.hidden = !hasUpdate;
1149:   appUpdateApply.hidden = !hasUpdate;
1150:   appUpdateApply.disabled = hasUpdate && !hasInstaller;
1151:   appUpdateApply.textContent = hasInstaller ? "Update" : "Open release page";
1152:   if (appUpdateDialog.open) return;
1153:   try {
1154:     appUpdateDialog.showModal();
1155:   } catch {
1156:     appUpdateDialog.show();
1157:   }
1158: }
```

startAppUpdateInstall (template-preview.js, lines 1160-1193)

startAppUpdateInstall part of the private builder frontend and editing experience. In this file, it appears around lines 1160-1193 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, stringify, json, Error, and showBuilderError. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include update at line 1161; response at line 1168; result at line 1174; fallback at line 1184. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1160: async function startAppUpdateInstall() {
1161:   const update = pendingAppUpdate || {};
1162:   appUpdateApply.disabled = true;
1163:   appUpdateLater.disabled = true;
1164:   appUpdateSkip.disabled = true;
1165:   appUpdateApply.textContent = "Updating...";
1166:   appUpdateDetails.textContent = "Downloading the installer. The builder will close automatically, then
the installer will update the app.";
1167:   try {
1168:     const response = await fetch(`/api/app-update/install?t=${Date.now()}`, {
1169:       method: "POST",
1170:       cache: "no-store",
1171:       headers: { "Content-Type": "application/json" },
1172:       body: JSON.stringify({ latestVersion: update.latestVersion || "" })
1173:     });
1174:     const result = await response.json();
1175:     if (!response.ok || !result.ok) throw new Error(result.error || "The update could not be started.");
1176:     appUpdateMessage.textContent = "Update started";
1177:     appUpdateDetails.textContent = "The installer is ready. This window will close so the new version can
be installed.";
1178:   } catch (error) {
1179:     appUpdateApply.disabled = false;
1180:     appUpdateLater.disabled = false;
1181:     appUpdateSkip.disabled = false;
1182:     appUpdateApply.textContent = "Update";
1183:     appUpdateDetails.textContent = error.message || "The update could not be started.";
1184:     const fallback = update.releaseUrl || update.installerUrl || update.portableUrl || "";
1185:     if (fallback) {
1186:       showBuilderError(
1187:         "Automatic update could not start",
1188:         "The app could not run the installer automatically. The release page can still be opened
manually.",
1189:         fallback
1190:       );
1191:     }
1192:   }
1193: }
```

checkForAppUpdates (template-preview.js, lines 1195-1235)

checkForAppUpdates part of the private builder frontend and editing experience. In this file, it appears around lines 1195-1235 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references now, fetch, json, showUpdateDialog, and toLocaleString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1198: if (!force && Date.now() - lastAppUpdateCheckAt < 5 * 60 * 1000) return;.

Important local values include force at line 1196; manual at line 1197; response at line 1207; update at line 1208. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1195: async function checkForAppUpdates(options = {}) {
1196:   const force = Boolean(options.force);
1197:   const manual = Boolean(options.manual);
1198:   if (!force && Date.now() - lastAppUpdateCheckAt < 5 * 60 * 1000) return;
1199:   lastAppUpdateCheckAt = Date.now();
1200:   if (manual) {
1201:     if (appUpdateCheckButton) {
1202:       appUpdateCheckButton.disabled = true;
1203:       appUpdateCheckButton.textContent = "Checking...";
1204:     }
1205:   }
```



```

1206:   try {
1207:     const response = await fetch(`/api/app-update?t=${Date.now()}`, { cache: "no-store" });
1208:     const update = await response.json();
1209:     if (response.ok && update.ok !== false) {
1210:       showUpdateDialog({ ...update, checkedAt: new Date().toLocaleString() }, { force: manual });
1211:     } else if (manual) {
1212:       showUpdateDialog({
1213:         currentVersion: update.currentVersion || "unknown",
1214:         updateAvailable: false,
1215:         checkedAt: new Date().toLocaleString()
1216:       }, { force: true });
1217:     }
1218:   } catch {
1219:     if (manual) {
1220:       showUpdateDialog({
1221:         currentVersion: "unknown",
1222:         updateAvailable: false,
1223:         checkedAt: new Date().toLocaleString()
1224:       }, { force: true });
1225:     }
1226:     // Update checks should never interrupt builder work.
1227:   } finally {
1228:     if (manual) {
1229:       if (appUpdateCheckButton) {
1230:         appUpdateCheckButton.disabled = false;
1231:         appUpdateCheckButton.textContent = "Check updates";
1232:       }
1233:     }
1234:   }
1235: }

```

formatReportDate (template-preview.js, lines 1237-1240)

formatReportDate part of the private builder frontend and editing experience. In this file, it appears around lines 1237-1240 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isNaN, getTime, and toLocaleString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1239: return Number.isNaN(date.getTime()) ? "" : date.toLocaleString();.

Important local values include date at line 1238. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1237: function formatReportDate(value = "") {
1238:   const date = new Date(value);
1239:   return Number.isNaN(date.getTime()) ? "" : date.toLocaleString();
1240: }

```

renderSecurityReport (template-preview.js, lines 1242-1294)

renderSecurityReport renders ui, markup, or display output. In this file, it appears around lines 1242-1294 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, escapeHtml, toLocaleString, join, and formatReportDate. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1243: if (!securityReportBody) return;.

Important local values include downloads at line 1244; assets at line 1245; auth at line 1246; access at line 1247; controls at line 1248; assetRows at line 1249. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1242: function renderSecurityReport(report = {}) {
1243:   if (!securityReportBody) return;
1244:   const downloads = report.appDownloads || {};
1245:   const assets = Array.isArray(downloads.assets) ? downloads.assets : [];
1246:   const auth = report.publishingAuth || {};
1247:   const access = report.websiteAccess || {};
1248:   const controls = Array.isArray(report.securityControls) ? report.securityControls : [];
1249:   const assetRows = assets.length
1250:     ? assets.map((asset) => `
1251:       <li>

```

```

1252:         <span>${escapeHtml(asset.name || "Release asset")}</span>
1253:         <span>${Number(asset.downloadCount || 0).toLocaleString()} downloads</span>
1254:     </li>
1255:     `).join("")
1256:     : `<li><span>No release assets found</span><span>0 downloads</span></li>`;
1257:     securityReportBody.innerHTML = `
1258:         <div class="security-report-grid">
1259:             <section class="security-report-card">
1260:                 <h3>Builder downloads</h3>
1261:                 <strong>${downloads.ok ? Number(downloads.totalDownloads || 0).toLocaleString() :
"Unavailable"}</strong>
1262:                 <p>${downloads.ok
1263:                     ? `Latest release: ${escapeHtml(downloads.releaseTag || downloads.releaseName || "latest")}`
1264:                     : escapeHtml(downloads.error || "GitHub release counts are not available.")}</p>
1265:                 <ul class="security-report-assets">${assetRows}</ul>
1266:             </section>
1267:             <section class="security-report-card">
1268:                 <h3>Publishing authorization</h3>
1269:                 <strong>${auth.authenticated ? "Active" : "Not active"}</strong>
1270:                 <p>${auth.authenticated
1271:                     ? `This builder has a remembered GitHub publishing session until
${escapeHtml(formatReportDate(auth.expiresAt) || auth.expiresAt || "the cache expires")}`
1272:                     : "No fresh GitHub publishing session is cached for the current target."}</p>
1273:                 <ul>
1274:                     <li>Target: ${escapeHtml(auth.targetRepository || "No publishing target saved")}</li>
1275:                     <li>Branch: ${escapeHtml(auth.branch || "Not set")}</li>
1276:                     <li>Successful authorizations this week: ${Number(auth.successCountLastWeek ||
0).toLocaleString()}</li>
1277:                 </ul>
1278:             </section>
1279:             <section class="security-report-card">
1280:                 <h3>Website access reporting</h3>
1281:                 <strong>${access.available ? "Configured" : "Needs Cloudflare"}</strong>
1282:                 <p>${escapeHtml(access.summary || "Visitor and IP reporting needs a server-side analytics
source.")}</p>
1283:                 <ul>${(access.recommendedSetup || []).map((item) =>
`<li>${escapeHtml(item)}</li>`).join("")}</ul>
1284:             </section>
1285:             <section class="security-report-card">
1286:                 <h3>Security controls</h3>
1287:                 <strong>${controls.length.toLocaleString()}</strong>
1288:                 <p>Controls currently enabled or prepared for publishing.</p>
1289:                 <ul>${controls.map((item) => `<li>${escapeHtml(item)}</li>`).join("")}</ul>
1290:             </section>
1291:         </div>
1292:         <p>Generated ${escapeHtml(formatReportDate(report.generatedAt) || "just now")}</p>
1293:     `;
1294: }

```

openSecurityReport (template-preview.js, lines 1296-1314)

openSecurityReport controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1296-1314 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references showModal, updateDialogWindowButtons, fetch, now, json, Error, renderSecurityReport, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1297: if (!securityReportDialog || !securityReportBody) return;.

Important local values include response at line 1302; report at line 1303. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1296: async function openSecurityReport() {
1297:   if (!securityReportDialog || !securityReportBody) return;
1298:   securityReportBody.innerHTML = "<p>Loading security report...</p>";
1299:   if (!securityReportDialog.open) securityReportDialog.showModal();
1300:   updateDialogWindowButtons(securityReportDialog);
1301:   try {
1302:     const response = await fetch(`/api/security-report?t=${Date.now()}`, { cache: "no-store" });
1303:     const report = await response.json();
1304:     if (!response.ok || report.ok === false) throw new Error(report.error || "Security report could not
be loaded.");
1305:     renderSecurityReport(report);
1306:   } catch (error) {
1307:     securityReportBody.innerHTML = `
1308:       <div class="security-report-card">
1309:         <h3>Security report unavailable</h3>
1310:         <p>${escapeHtml(error.message || "The report could not be loaded.")}</p>
1311:       </div>
1312:     `;
1313:   }

```

```
1314: }
```

scheduleAppUpdateChecks (template-preview.js, lines 1316-1325)

scheduleAppUpdateChecks part of the private builder frontend and editing experience. In this file, it appears around lines 1316-1325 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references checkForAppUpdates, addEventListener, and now. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1320: if (document.visibilityState !== "visible") return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1316: function scheduleAppUpdateChecks() {
1317:   checkForAppUpdates({ force: true });
1318:   window.setInterval(() => checkForAppUpdates({ force: true }), APP_UPDATE_CHECK_INTERVAL_MS);
1319:   document.addEventListener("visibilitychange", () => {
1320:     if (document.visibilityState !== "visible") return;
1321:     if (Date.now() - lastAppUpdateCheckAt >= APP_UPDATE_FOCUS_RECHECK_MS) {
1322:       checkForAppUpdates({ force: true });
1323:     }
1324:   });
1325: }
```

renderPublishTargetInfo (template-preview.js, lines 1327-1353)

renderPublishTargetInfo renders ui, markup, or display output. In this file, it appears around lines 1327-1353 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLocaleString, map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1328: if (!publishTargetCurrent) return;.

Important local values include checkedText at line 1330; expiresText at line 1331; trustText at line 1332; authorizationStatus at line 1333; rows at line 1336. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1327: function renderPublishTargetInfo(target = {}) {
1328:   if (!publishTargetCurrent) return;
1329:   currentPublishTarget = target;
1330:   const checkedText = target.checkedAt ? ` at ${new Date(target.checkedAt).toLocaleString()}` : "";
1331:   const expiresText = target.expiresAt ? `; remembered until ${new
Date(target.expiresAt).toLocaleString()}` : "";
1332:   const trustText = target.extendedTrust ? " (30-day trusted target)" : target.authorizationCached ? "
(daily cache)" : "";
1333:   const authorizationStatus = target.authorizationChecked
1334:     ? `Verified${checkedText}${trustText}${expiresText}`
1335:     : "Not verified for this repository and branch";
1336:   const rows = [
1337:     ["Builder workspace", target.workspace || "Not available"],
1338:     ["Portfolio workspace", target.portfolioWorkspace || target.workspace || "Not available"],
1339:     ["Repository", target.repository || target.remote || "No repository connected"],
1340:     ["Branch", target.branch || "No branch selected"],
1341:     ["GitHub authorization", authorizationStatus],
1342:     ["Compatible site", target.compatible ? "Yes" : "No"],
1343:     ["Git-backed", target.gitBacked ? "Yes" : "No"]
1344:   ];
1345:   publishTargetCurrent.innerHTML = rows
1346:     .map(([label, value]) =>
`<div><strong>${escapeHtml(label)}</strong><span>${escapeHtml(value)}</span></div>`)
1347:     .join("");
1348:   if (publishTargetRepository) {
1349:     publishTargetRepository.value = target.remote || "";
1350:     publishTargetRepository.dataset.loadedValue = target.remote || "";
1351:   }
1352:   if (publishTargetSync) publishTargetSync.disabled = !target.authorizationChecked;
1353: }
```

renderSystemReadiness (template-preview.js, lines 1355-1394)

renderSystemReadiness renders ui, markup, or display output. In this file, it appears around lines 1355-1394 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLocaleString, map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1356: if (!publishTargetReadinessList) return;.

Important local values include rows at line 1357. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1355: function renderSystemReadiness(system = {}, target = {}) {
1356:   if (!publishTargetReadinessList) return;
1357:   const rows = [
1358:     {
1359:       ok: true,
1360:       text: `Electron/Node runtime: bundled with the app${system.nodeRuntime?.version ? `
1361: ($${system.nodeRuntime.version})` : ""}.`
1362:     },
1363:     {
1364:       ok: system.pnpm?.required === false,
1365:       text: "pnpm: not required for installed users."
1366:     },
1367:     {
1368:       ok: Boolean(system.git?.ok),
1369:       text: system.git?.ok
1370:         ? `Git for Windows: ${system.git.version || "installed"}.`
1371:         : "Git for Windows: missing. Install it before loading or publishing a target."
1372:     },
1373:     {
1374:       ok: Boolean(system.credentialManager?.ok),
1375:       text: system.credentialManager?.ok
1376:         ? `Git Credential Manager: ${system.credentialManager.version || "installed"}.`
1377:         : "Git Credential Manager: missing. Install Git for Windows with Git Credential Manager enabled."
1378:     },
1379:     {
1380:       ok: Boolean(target?.remote),
1381:       text: target?.remote
1382:         ? `Publishing target: ${target.repository || target.remote}.`
1383:         : "Publishing target: enter a repository URL and click Authenticate target."
1384:     },
1385:     {
1386:       ok: Boolean(target?.authorizationChecked),
1387:       text: target?.authorizationChecked
1388:         ? `GitHub authorization: verified for ${target.branch || "the selected branch"}${target.expiresAt
1389: ? ` until ${new Date(target.expiresAt).toLocaleString()}` : ""}.`
1390:         : "GitHub authorization: required before Load from target or Apply to site."
1391:     }
1392:   ];
1393:   publishTargetReadinessList.innerHTML = rows
1394:     .map((row) => `<li class="${row.ok ? "ready" : "blocked"}">${escapeHtml(row.ok ? `OK - ${row.text}` :
1395: `Needs action - ${row.text}`)}</li>`)
1396:     .join("");
1397: }
```

markPublishTargetNeedsAuthentication (template-preview.js, lines 1396-1405)

markPublishTargetNeedsAuthentication part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1396-1405 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1396: function markPublishTargetNeedsAuthentication() {
1397:   if (publishTargetSync) publishTargetSync.disabled = true;
1398:   if (currentPublishTarget) {
1399:     currentPublishTarget = {
1400:       ...currentPublishTarget,
1401:       authorizationChecked: false,
```

```

1402:     authorizationCached: false
1403:   };
1404: }
1405: }

```

loadSystemReadiness (template-preview.js, lines 1407-1418)

loadSystemReadiness loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1407-1418 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, renderSystemReadiness, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1408: if (!publishTargetReadinessList) return;.

Important local values include response at line 1411; result at line 1412. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1407: async function loadSystemReadiness() {
1408:   if (!publishTargetReadinessList) return;
1409:   publishTargetReadinessList.innerHTML = "<li>Checking local publishing tools...</li>";
1410:   try {
1411:     const response = await fetch(`/api/system-check?t=${Date.now()}`, { cache: "no-store" });
1412:     const result = await response.json();
1413:     if (!response.ok || !result.ok) throw new Error(result.error || "System check failed.");
1414:     renderSystemReadiness(result.system || {}, result.target || currentPublishTarget || {});
1415:   } catch (error) {
1416:     publishTargetReadinessList.innerHTML = `<li class="blocked">${escapeHtml(error.message || "System
check failed.")}</li>`;
1417:   }
1418: }

```

setPublishTargetProgress (template-preview.js, lines 1420-1426)

setPublishTargetProgress part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1420-1426 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, toLocaleTimeString, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1421: if (!publishTargetCurrent) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1420: function setPublishTargetProgress(title, steps = []) {
1421:   if (!publishTargetCurrent) return;
1422:   publishTargetCurrent.innerHTML = `
1423:     <div><strong>${escapeHtml(title)}</strong><span>${escapeHtml(new
Date().toLocaleTimeString())}</span></div>
1424:     ${steps.map((step) =>
`<div><strong>${escapeHtml(step.label)}</strong><span>${escapeHtml(step.value)}</span></div>`).join("")}
1425:   `;
1426: }

```

loadPublishTargetInfo (template-preview.js, lines 1428-1439)

loadPublishTargetInfo loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1428-1439 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, and renderPublishTargetInfo. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1429: if (!publishTargetDialog) return;.

Important local values include response at line 1432; result at line 1433. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1428: async function loadPublishTargetInfo() {
1429:   if (!publishTargetDialog) return;
1430:   publishTargetCurrent.textContent = "Current target is loading.";
1431:   try {
1432:     const response = await fetch(`/api/publish-target?t=${Date.now()}`, { cache: "no-store" });
1433:     const result = await response.json();
1434:     if (!response.ok || !result.ok) throw new Error(result.error || "Publishing target could not be
loaded.");
1435:     renderPublishTargetInfo(result.target || {});
1436:   } catch (error) {
1437:     publishTargetCurrent.textContent = error.message || "Publishing target could not be loaded.";
1438:   }
1439: }
```

openPublishTargetDialog (template-preview.js, lines 1441-1445)

openPublishTargetDialog part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1441-1445 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references loadPublishTargetInfo, loadSystemReadiness, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1441: async function openPublishTargetDialog() {
1442:   await loadPublishTargetInfo();
1443:   await loadSystemReadiness();
1444:   publishTargetDialog.showModal();
1445: }
```

savePublishTarget (template-preview.js, lines 1447-1450)

savePublishTarget persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 1447-1450 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references preventDefault, and authenticatePublishTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1447: async function savePublishTarget(event) {
1448:   event.preventDefault();
1449:   await authenticatePublishTarget({ fromSave: true });
1450: }
```

currentPublishTargetPayload (template-preview.js, lines 1452-1456)

currentPublishTargetPayload part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1452-1456 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1453: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1452: function currentPublishTargetPayload() {
```

```

1453:   return {
1454:     repositoryUrl: publishTargetRepository?.value || ""
1455:   };
1456: }

```

installGitForPublishing (template-preview.js, lines 1458-1481)

installGitForPublishing part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1458-1481 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, open, loadSystemReadiness, and showBuilderError. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include response at line 1461; result at line 1467. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1458: async function installGitForPublishing() {
1459:   publishTargetCurrent.textContent = "Starting Git for Windows setup..";
1460:   try {
1461:     const response = await fetch(`/api/install-git?t=${Date.now()}`, {
1462:       method: "POST",
1463:       cache: "no-store",
1464:       headers: { "Content-Type": "application/json" },
1465:       body: "{}"
1466:     });
1467:     const result = await response.json();
1468:     if (!response.ok || !result.ok) throw new Error(result.error || "Git installer could not be
started.");
1469:     if (!result.install?.launched && result.install?.downloadUrl) {
1470:       window.open(result.install.downloadUrl, "_blank", "noreferrer");
1471:     }
1472:     await loadSystemReadiness();
1473:     showBuilderError(
1474:       result.install?.launched ? "Git setup started" : "Open Git download",
1475:       result.install?.message || "Install Git for Windows with Git Credential Manager enabled, then
reopen the builder.",
1476:       result.install?.downloadUrl || ""
1477:     );
1478:   } catch (error) {
1479:     publishTargetCurrent.textContent = error.message || "Git installer could not be started.";
1480:   }
1481: }

```

authenticatePublishTarget (template-preview.js, lines 1483-1536)

authenticatePublishTarget part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1483-1536 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setPublishTargetProgress, fetch, now, stringify, currentPublishTargetPayload, json, Error, filter, join, renderPublishTargetInfo, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include fromSave at line 1484; response at line 1496; result at line 1502; message at line 1504; usedCachedAuth at line 1510. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1483: async function authenticatePublishTarget(options = {}) {
1484:   const fromSave = Boolean(options.fromSave);
1485:   if (publishTargetSync) publishTargetSync.disabled = true;
1486:   setPublishTargetProgress(
1487:     fromSave ? "Authenticating publishing target" : "Checking GitHub authorization",
1488:     [
1489:       { label: "Step 1", value: "Validating the repository URL and local Git tools." },
1490:       { label: "Step 2", value: "Using the saved daily authorization if it is still valid." },
1491:       { label: "Step 3", value: "Finding the target repository default branch." },
1492:       { label: "Step 4", value: "Opening GitHub sign-in only when a fresh authorization is required." }
1493:     ]
1494:   );
1495:   try {
1496:     const response = await fetch(`/api/github-authenticate?t=${Date.now()}`, {

```

```

1497:     method: "POST",
1498:     cache: "no-store",
1499:     headers: { "Content-Type": "application/json" },
1500:     body: JSON.stringify(currentPublishTargetPayload())
1501:   });
1502:   const result = await response.json();
1503:   if (!response.ok || !result.ok) {
1504:     const message = result.error || "GitHub authentication did not complete.";
1505:     throw new Error([message, result.details].filter(Boolean).join("\n\n"));
1506:   }
1507:   renderPublishTargetInfo(result.target || {});
1508:   renderSystemReadiness(result.auth?.system || {}, result.target || {});
1509:   if (publishTargetPassword) publishTargetPassword.value = "";
1510:   const usedCachedAuth = Boolean(result.auth?.authorizationCached ||
result.target?.authorizationCached);
1511:   setPublishTargetProgress(
1512:     usedCachedAuth ? "Target authenticated from saved authorization" : "Target authenticated",
1513:     [
1514:       { label: "Repository", value: result.target?.repository || result.auth?.repository || "Selected
target" },
1515:       { label: "Branch", value: result.target?.branch || result.auth?.branch || "Detected target
branch" },
1516:       { label: "Authorization", value: usedCachedAuth ? "A valid saved authorization was reused; no new
GitHub sign-in was needed." : "GitHub write access was verified now." },
1517:       { label: "Next step", value: "Click Load from target when you want to import the latest
compatible website files into this builder." }
1518:     ]
1519:   );
1520:   if (publishTargetSync) publishTargetSync.disabled = false;
1521: } catch (error) {
1522:   if (publishTargetSync) publishTargetSync.disabled = true;
1523:   setPublishTargetProgress(
1524:     "Target was not saved",
1525:     [
1526:       { label: "Reason", value: error.message || "GitHub authentication did not complete." },
1527:       { label: "Status", value: "The previous publishing target was kept. Fix the repository or sign-
in, then try Authenticate target again." }
1528:     ]
1529:   );
1530:   showBuilderError(
1531:     "Target authentication failed",
1532:     "The builder did not save this publishing target because GitHub write access was not verified.",
1533:     error.message || "GitHub authentication did not complete."
1534:   );
1535: }
1536: }

```

syncFromPublishTarget (template-preview.js, lines 1538-1584)

syncFromPublishTarget part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1538-1584 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, filter, join, renderPublishTargetInfo, closeDialogElement, loadData, and showBuilderError. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1539: if (!publishTargetDialog) return;.

Important local values include afterAuth at line 1540; response at line 1545; result at line 1551. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1538: async function syncFromPublishTarget(options = {}) {
1539:   if (!publishTargetDialog) return;
1540:   const afterAuth = Boolean(options.afterAuth);
1541:   publishTargetCurrent.textContent = afterAuth
1542:     ? "GitHub authentication was accepted. Importing the target portfolio now..."
1543:     : "Checking GitHub authorization and loading compatible portfolio files from the target
repository...";
1544:   try {
1545:     const response = await fetch(`/api/sync-from-publish-target?t=${Date.now()}`, {
1546:       method: "POST",
1547:       cache: "no-store",
1548:       headers: { "Content-Type": "application/json" },
1549:       body: "{}"
1550:     });
1551:     const result = await response.json();
1552:     if (!response.ok || !result.ok) {
1553:       throw new Error([
1554:         result.error || "Publishing target could not be imported.",
1555:         result.details || "Click Authenticate target, complete the browser sign-in if prompted, then try
Load from target again."
1556:       ].filter(Boolean).join("\n\n"));

```



```

1557:     }
1558:     renderPublishTargetInfo(result.target || {});
1559:     closeDialogElement(publishTargetDialog);
1560:     await loadData();
1561:     draftSavedSinceChanges = true;
1562:     showBuilderError(
1563:       afterAuth ? "GitHub connected and target loaded" : "Target content loaded",
1564:       afterAuth
1565:         ? "Authentication succeeded, and compatible portfolio files were loaded into this local builder
workspace. The local draft now matches the imported target catalog."
1566:         : "Compatible portfolio files were loaded from the authenticated publishing target into this
local builder workspace. The local draft now matches the imported target catalog.",
1567:       [
1568:         `Imported: ${result.sync?.imported || []}.join(", ")`,
1569:         result.sync?.backup ? `Backup: ${result.sync.backup}` : ""
1570:       ].filter(Boolean).join("\n")
1571:     );
1572:   } catch (error) {
1573:     publishTargetCurrent.textContent = afterAuth
1574:       ? `Authentication completed, but target loading failed. ${error.message || "Publishing target could
not be imported."}`
1575:       : error.message || "Publishing target could not be imported.";
1576:     if (afterAuth) {
1577:       showBuilderError(
1578:         "Target saved, loading failed",
1579:         "GitHub authorization succeeded, but the builder could not import compatible portfolio files from
that target.",
1580:         error.message || "Publishing target could not be imported."
1581:       );
1582:     }
1583:   }
1584: }

```

showPublishResult (template-preview.js, lines 1586-1619)

showPublishResult part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 1586-1619 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, join, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include publish at line 1587; pushed at line 1588; authorizationRequired at line 1589; output at line 1602. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1586: function showPublishResult(result) {
1587:   const publish = result.publish || {};
1588:   const pushed = Boolean(publish.pushed);
1589:   const authorizationRequired = Boolean(publish.authorizationRequired);
1590:   publishResultEyebrow.textContent = "GitHub publish";
1591:   publishResultTitle.textContent = pushed
1592:     ? "Successfully pushed to GitHub"
1593:     : authorizationRequired
1594:       ? "Publishing access required"
1595:       : "Site applied, push failed";
1596:   publishResultMessage.textContent = pushed
1597:     ? `Your portfolio changes were committed and pushed to ${publish.branch || "GitHub"}`
1598:     : authorizationRequired
1599:       ? "No live website files were applied. Open Publishing target, click Authenticate with GitHub,
complete the browser sign-in, then try Apply to site again."
1600:       : "The site was applied locally, but Git push did not complete.";
1601:
1602:   const output = [
1603:     pushed ? "PUSH STATUS: SUCCESS" : "PUSH STATUS: FAILED",
1604:     `FILE: ${result.file || "projects.json"}`,
1605:     authorizationRequired ? "AUTHORIZATION: REQUIRED BEFORE WEBSITE CHANGES" : "",
1606:     publish.repository ? `REPOSITORY: ${publish.repository}` : "",
1607:     publish.remote ? `REMOTE: ${publish.remote}` : "",
1608:     publish.branch ? `BRANCH: ${publish.branch}` : "",
1609:     publish.committed === false ? "COMMIT: No file changes to commit." : "",
1610:     publish.commitOutput ? `COMMIT OUTPUT:\n${publish.commitOutput}` : "",
1611:     publish.pushOutput ? `PUSH OUTPUT:\n${publish.pushOutput}` : "",
1612:     publish.error ? `ERROR:\n${publish.error}` : "",
1613:     publish.details ? `DETAILS:\n${publish.details}` : "",
1614:     publish.help ? `NEXT STEP:\n${publish.help}` : ""
1615:   ].filter(Boolean).join("\n\n");
1616:
1617:   publishResultOutput.textContent = output || "No Git output was returned.";
1618:   publishResultDialog.showModal();
1619: }

```

scheduleAutosave (template-preview.js, lines 1621-1627)

scheduleAutosave part of the private builder frontend and editing experience. In this file, it appears around lines 1621-1627 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references markDraftNeedsSave, and autosaveDraft. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1621: function scheduleAutosave(delay = 900) {
1622:   markDraftNeedsSave();
1623:   clearTimeout(autosaveTimer);
1624:   autosaveTimer = setTimeout(() => {
1625:     autosaveDraft();
1626:   }, delay);
1627: }
```

autosaveDraft (template-preview.js, lines 1629-1659)

autosaveDraft part of the private builder frontend and editing experience. In this file, it appears around lines 1629-1659 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setSaveState, fetch, stringify, json, setStatus, and scheduleAutosave. Endpoint references found inside it are /api/save-draft. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1632: return;.

Important local values include response at line 1637; result at line 1642. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1629: async function autosaveDraft() {
1630:   if (autosaveInFlight) {
1631:     autosaveQueued = true;
1632:     return;
1633:   }
1634:   autosaveInFlight = true;
1635:   setSaveState("saving", "Autosaving...");
1636:   try {
1637:     const response = await fetch("/api/save-draft", {
1638:       method: "POST",
1639:       headers: { "Content-Type": "application/json" },
1640:       body: JSON.stringify({ catalog })
1641:     });
1642:     const result = await response.json();
1643:     if (!response.ok) {
1644:       setStatus(result.error || "Autosave failed.");
1645:       setSaveState("error", "Autosave failed");
1646:     } else if (!draftSavedSinceChanges) {
1647:       setSaveState("dirty", "Autosaved locally");
1648:     }
1649:   } catch {
1650:     setStatus("Autosave failed. Make sure the local server is running.");
1651:     setSaveState("error", "Autosave failed");
1652:   } finally {
1653:     autosaveInFlight = false;
1654:     if (autosaveQueued) {
1655:       autosaveQueued = false;
1656:       scheduleAutosave(300);
1657:     }
1658:   }
1659: }
```

schedulePreviewRender (template-preview.js, lines 1661-1667)

schedulePreviewRender part of the private builder frontend and editing experience. In this file, it appears around lines 1661-1667 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1662: if (!portfolioPreviewDialog?.open) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1661: function schedulePreviewRender() {
1662:   if (!portfolioPreviewDialog?.open) return;
1663:   clearTimeout(previewRenderTimer);
1664:   previewRenderTimer = setTimeout(() => {
1665:     renderPreview();
1666:   }, 220);
1667: }
```

scheduleChromeRender (template-preview.js, lines 1669-1676)

scheduleChromeRender part of the private builder frontend and editing experience. In this file, it appears around lines 1669-1676 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderTree, renderSectionTabs, selectedProject, and renderPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1669: function scheduleChromeRender() {
1670:   clearTimeout(chromeRenderTimer);
1671:   chromeRenderTimer = setTimeout(() => {
1672:     renderTree();
1673:     renderSectionTabs(selectedProject());
1674:     if (portfolioPreviewDialog?.open) renderPreview();
1675:   }, 260);
1676: }
```

updateAssetDialogVisibility (template-preview.js, lines 1678-1691)

updateAssetDialogVisibility updates ui, state, cache, or stored data. In this file, it appears around lines 1678-1691 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, trim, labelForUrlAssetKind, and inferUrlAssetKind. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include isLocal at line 1679; isCaptionFile at line 1680; url at line 1685. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1678: function updateAssetDialogVisibility() {
1679:   const isLocal = assetSource.value === "local";
1680:   const isCaptionFile = captionSource.value === "file";
1681:   document.querySelector(".asset-local-field").hidden = !isLocal;
1682:   document.querySelector(".asset-url-field").hidden = isLocal;
1683:   document.querySelector(".caption-text-field").hidden = isCaptionFile;
1684:   document.querySelector(".caption-file-field").hidden = !isCaptionFile;
1685:   const url = assetUrl?.value?.trim() || "";
1686:   if (assetDetectedNote) {
1687:     assetDetectedNote.textContent = isLocal || !url
1688:       ? ""
1689:       : `Detected: ${labelForUrlAssetKind(inferUrlAssetKind(url, assetSource.value))}`;
1690:   }
1691: }
```

slugify (template-preview.js, lines 1693-1699)

slugify part of the private builder frontend and editing experience. In this file, it appears around lines 1693-1699 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, trim, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1694: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1693: function slugify(value) {
1694:   return String(value || "")
1695:     .toLowerCase()
1696:     .trim()
1697:     .replace(/[^\a-z0-9]+/g, "-")
1698:     .replace(/^-+|-+$/g, "");
1699: }
```

wordCount (template-preview.js, lines 1701-1703)

wordCount part of the private builder frontend and editing experience. In this file, it appears around lines 1701-1703 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, split, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1702: return String(value || "").trim().split(/\s+/).filter(Boolean).length;

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1701: function wordCount(value) {
1702:   return String(value || "").trim().split(/\s+/).filter(Boolean).length;
1703: }
```

limitWords (template-preview.js, lines 1705-1708)

limitWords part of the private builder frontend and editing experience. In this file, it appears around lines 1705-1708 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, split, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1707: return words.length > maxWords ? words.slice(0, maxWords).join(" ") : value;

Important local values include words at line 1706. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1705: function limitWords(value, maxWords = 1000) {
1706:   const words = String(value || "").trim().split(/\s+/).filter(Boolean);
1707:   return words.length > maxWords ? words.slice(0, maxWords).join(" ") : value;
1708: }
```

normalizeFunFacts (template-preview.js, lines 1710-1716)

normalizeFunFacts validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1710-1716 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `isArray`, `split`, `map`, `replace`, `trim`, `filter`, and `slice`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1712: `return items`.

Important local values include `items` at line 1711. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1710: function normalizeFunFacts(value) {
1711:   const items = Array.isArray(value) ? value : String(value || "").split(/\r?\n/);
1712:   return items
1713:     .map((item) => String(item || "").replace(/\s+/g, " ").trim())
1714:     .filter(Boolean)
1715:     .slice(0, 3);
1716: }
```

syncFunFactsFromInput (template-preview.js, lines 1718-1726)

`syncFunFactsFromInput` keeps two places aligned, such as local data and target files. In this file, it appears around lines 1718-1726 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `extractRichSummary`, `normalizeFunFacts`, and `plainTextFromRich`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1719: `if (!funFactsInput) return [];` line 1725: `return facts;`.

Important local values include `rich` at line 1720; `facts` at line 1721. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1718: function syncFunFactsFromInput() {
1719:   if (!funFactsInput) return [];
1720:   const rich = extractRichSummary(funFactsInput);
1721:   const facts = normalizeFunFacts(plainTextFromRich(rich));
1722:   catalog.funFacts = facts;
1723:   catalog.funFactsRich = rich;
1724:   if (funFactsCount) funFactsCount.textContent = `${facts.length}/3`;
1725:   return facts;
1726: }
```

renderFunFactsEditor (template-preview.js, lines 1728-1734)

`renderFunFactsEditor` renders ui, markup, or display output. In this file, it appears around lines 1728-1734 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizeFunFacts`, `populateStandaloneRichEditor`, and `join`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1729: `if (!funFactsInput) return;` line 1732: `if (document.activeElement === funFactsInput) return;`.

Important local values include `facts` at line 1730. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1728: function renderFunFactsEditor() {
1729:   if (!funFactsInput) return;
1730:   const facts = normalizeFunFacts(catalog.funFacts || []);
1731:   if (funFactsCount) funFactsCount.textContent = `${facts.length}/3`;
1732:   if (document.activeElement === funFactsInput) return;
1733:   populateStandaloneRichEditor(funFactsInput, catalog.funFactsRich, facts.join("\n"));
1734: }
```

populateTextOnlyRichField (template-preview.js, lines 1736-1740)

`populateTextOnlyRichField` part of the private builder frontend and editing experience. In this file, it appears around lines 1736-1740 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references populateStandaloneRichEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1737: if (!field) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1736: function populateTextOnlyRichField(field, rich, fallbackText = "") {
1737:   if (!field) return;
1738:   populateStandaloneRichEditor(field, rich, fallbackText);
1739:   field.dataset.richTextOnly = "true";
1740: }
```

richFieldValue (template-preview.js, lines 1742-1754)

richFieldValue part of the private builder frontend and editing experience. In this file, it appears around lines 1742-1754 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, textBlocksFromPlainText, extractRichSummary, and plainTextFromRich. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1744: return {}; line 1750: return {}.

Important local values include rich at line 1749. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1742: function richFieldValue(field, fallbackText = "") {
1743:   if (!field) {
1744:     return {
1745:       plain: String(fallbackText || "").trim(),
1746:       rich: { blocks: textBlocksFromPlainText(fallbackText) }
1747:     };
1748:   }
1749:   const rich = extractRichSummary(field);
1750:   return {
1751:     plain: plainTextFromRich(rich, "\n").trim(),
1752:     rich
1753:   };
1754: }
```

mergeRichFieldMaps (template-preview.js, lines 1756-1761)

mergeRichFieldMaps part of the private builder frontend and editing experience. In this file, it appears around lines 1756-1761 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1757: return {}.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1756: function mergeRichFieldMaps(current = {}, fallback = {}) {
1757:   return {
1758:     ...clone(fallback || {}),
1759:     ...clone(current || {})
1760:   };
1761: }
```

parsedSiteContentRich (template-preview.js, lines 1763-1765)

parsedSiteContentRich part of the private builder frontend and editing experience. In this file, it appears around lines 1763-1765 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mergeRichFieldMaps. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1764: return mergeRichFieldMaps(catalog.siteContentRich, savedPortfolioCatalog.siteContentRich);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1763: function parsedSiteContentRich() {  
1764:   return mergeRichFieldMaps(catalog.siteContentRich, savedPortfolioCatalog.siteContentRich);  
1765: }
```

parsedProfileRich (template-preview.js, lines 1767-1769)

parsedProfileRich part of the private builder frontend and editing experience. In this file, it appears around lines 1767-1769 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mergeRichFieldMaps. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1768: return mergeRichFieldMaps(catalog.profileRich, savedPortfolioCatalog.profileRich);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1767: function parsedProfileRich() {  
1768:   return mergeRichFieldMaps(catalog.profileRich, savedPortfolioCatalog.profileRich);  
1769: }
```

parsedFunFactsRich (template-preview.js, lines 1771-1775)

parsedFunFactsRich part of the private builder frontend and editing experience. In this file, it appears around lines 1771-1775 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call, and clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1772: return Object.prototype.hasOwnProperty.call(catalog, "funFactsRich").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1771: function parsedFunFactsRich() {  
1772:   return Object.prototype.hasOwnProperty.call(catalog, "funFactsRich")  
1773:     ? clone(catalog.funFactsRich || null)  
1774:     : clone(savedPortfolioCatalog.funFactsRich || null);  
1775: }
```

syncPortfolioTextInputsForParsing (template-preview.js, lines 1777-1781)

syncPortfolioTextInputsForParsing keeps two places aligned, such as local data and target files. In this file, it appears around lines 1777-1781 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references syncFunFactsFromInput, syncSiteContentFromInputs, and syncProfileFromInputs. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1777: function syncPortfolioTextInputsForParsing() {
1778:   if (funFactsInput) syncFunFactsFromInput();
1779:   if (siteHeroTitleInput || siteHeroCopyInput || siteHeroEyebrowInput) syncSiteContentFromInputs();
1780:   if (profileDisplayNameInput || profileEmailInput || profilePhoneInput) syncProfileFromInputs();
1781: }
```

parsedPortfolioGlobals (template-preview.js, lines 1783-1796)

parsedPortfolioGlobals part of the private builder frontend and editing experience. In this file, it appears around lines 1783-1796 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references syncPortfolioTextInputsForParsing, clone, normalizeFieldStyles, normalizeFunFacts, parsedFunFactsRich, normalizeProfile, parsedProfileRich, normalizeSiteContent, parsedSiteContentRich, filter, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1785: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1783: function parsedPortfolioGlobals() {
1784:   syncPortfolioTextInputsForParsing();
1785:   return {
1786:     categories: clone(catalog.categories || savedPortfolioCatalog.categories || []),
1787:     fieldStyles: clone(normalizeFieldStyles(catalog.fieldStyles || savedPortfolioCatalog.fieldStyles ||
1788:   {})),
1789:     funFacts: normalizeFunFacts(catalog.funFacts || savedPortfolioCatalog.funFacts || []),
1790:     funFactsRich: parsedFunFactsRich(),
1791:     profile: clone(normalizeProfile(catalog.profile || savedPortfolioCatalog.profile || {})),
1792:     profileRich: parsedProfileRich(),
1793:     siteContent: clone(normalizeSiteContent(catalog.siteContent || savedPortfolioCatalog.siteContent ||
1794:   {})),
1795:     siteContentRich: parsedSiteContentRich(),
1796:     siteSections: (catalog.siteSections || savedPortfolioCatalog.siteSections ||
1797:   []).filter(siteSectionRenderable).map(clone)
1798:   };
1799: }
```

syncParsedPortfolioGlobalsIntoSaved (template-preview.js, lines 1798-1800)

syncParsedPortfolioGlobalsIntoSaved keeps two places aligned, such as local data and target files. In this file, it appears around lines 1798-1800 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assign, and parsedPortfolioGlobals. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1798: function syncParsedPortfolioGlobalsIntoSaved() {
1799:   Object.assign(savedPortfolioCatalog, parsedPortfolioGlobals());
1800: }
```

siteContentRichValue (template-preview.js, lines 1802-1804)

siteContentRichValue part of the private builder frontend and editing experience. In this file, it appears around lines 1802-1804 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1803: return catalog.siteContentRich?.[key] || savedPortfolioCatalog.siteContentRich?.[key] || null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1802: function siteContentRichValue(key) {  
1803:   return catalog.siteContentRich?.[key] || savedPortfolioCatalog.siteContentRich?.[key] || null;  
1804: }
```

profileRichValue (template-preview.js, lines 1806-1808)

profileRichValue part of the private builder frontend and editing experience. In this file, it appears around lines 1806-1808 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1807: return catalog.profileRich?.[key] || savedPortfolioCatalog.profileRich?.[key] || null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1806: function profileRichValue(key) {  
1807:   return catalog.profileRich?.[key] || savedPortfolioCatalog.profileRich?.[key] || null;  
1808: }
```

syncSiteRichField (template-preview.js, lines 1810-1823)

syncSiteRichField keeps two places aligned, such as local data and target files. In this file, it appears around lines 1810-1823 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richFieldValue, normalizeSiteContent, markDraftNeedsSave, scheduleAutosave, and schedulePreviewRender. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1812: if (!key) return;.

Important local values include key at line 1811. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1810: function syncSiteRichField(editor) {  
1811:   const key = editor?.dataset.siteField;  
1812:   if (!key) return;  
1813:   catalog.siteContentRich = catalog.siteContentRich || {};  
1814:   const { plain, rich } = richFieldValue(editor, catalog.siteContent?.[key] || "");  
1815:   catalog.siteContent = normalizeSiteContent({  
1816:     ...(catalog.siteContent || {}),  
1817:     [key]: plain  
1818:   });  
1819:   catalog.siteContentRich[key] = rich;  
1820:   markDraftNeedsSave();  
1821:   scheduleAutosave();  
1822:   schedulePreviewRender();  
1823: }
```

syncProfileRichField (template-preview.js, lines 1825-1838)

syncProfileRichField keeps two places aligned, such as local data and target files. In this file, it appears around lines 1825-1838 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richFieldValue, normalizeProfile, markDraftNeedsSave, scheduleAutosave, and schedulePreviewRender. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1827: if (!key) return;.

Important local values include key at line 1826. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1825: function syncProfileRichField(editor) {
1826:   const key = editor?.dataset.profileField;
1827:   if (!key) return;
1828:   catalog.profileRich = catalog.profileRich || {};
1829:   const { plain, rich } = richFieldValue(editor, catalog.profile?.[key] || "");
1830:   catalog.profile = normalizeProfile({
1831:     ...(catalog.profile || {}),
1832:     [key]: plain
1833:   });
1834:   catalog.profileRich[key] = rich;
1835:   markDraftNeedsSave();
1836:   scheduleAutosave();
1837:   schedulePreviewRender();
1838: }
```

renderSiteContentEditor (template-preview.js, lines 1840-1851)

renderSiteContentEditor renders ui, markup, or display output. In this file, it appears around lines 1840-1851 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeSiteContent, populateTextOnlyRichField, and siteContentRichValue. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include content at line 1841. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1840: function renderSiteContentEditor() {
1841:   const content = normalizeSiteContent(catalog.siteContent || {});
1842:   if (siteHeroEyebrowInput && document.activeElement !== siteHeroEyebrowInput) {
1843:     populateTextOnlyRichField(siteHeroEyebrowInput, siteContentRichValue("heroEyebrow"),
content.heroEyebrow);
1844:   }
1845:   if (siteHeroTitleInput && document.activeElement !== siteHeroTitleInput) {
1846:     populateTextOnlyRichField(siteHeroTitleInput, siteContentRichValue("heroTitle"), content.heroTitle);
1847:   }
1848:   if (siteHeroCopyInput && document.activeElement !== siteHeroCopyInput) {
1849:     populateTextOnlyRichField(siteHeroCopyInput, siteContentRichValue("heroCopy"), content.heroCopy);
1850:   }
1851: }
```

renderProfileEditor (template-preview.js, lines 1853-1881)

renderProfileEditor renders ui, markup, or display output. In this file, it appears around lines 1853-1881 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeProfile, forEach, populateTextOnlyRichField, profileRichValue, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include profile at line 1854; fields at line 1855; assetLabels at line 1871. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1853: function renderProfileEditor() {
1854:   const profile = normalizeProfile(catalog.profile || {});
1855:   const fields = [
1856:     [profileDisplayNameInput, profile.displayName],
1857:     [profilePortfolioLabelInput, profile.portfolioLabel],
1858:     [profileContactIntroInput, profile.contactIntro],
1859:     [profileEmailInput, profile.email],
1860:     [profilePhoneInput, profile.phone],
1861:     [profileGithubInput, profile.githubUrl],
1862:     [profileLinkedinInput, profile.linkedinUrl],
```

```

1863:     [profileWebsiteInput, profile.websiteUrl]
1864:   ];
1865:   fields.forEach(([field, value]) => {
1866:     if (field && document.activeElement !== field) {
1867:       populateTextOnlyRichField(field, profileRichValue(field.dataset.profileField), value);
1868:     }
1869:   });
1870:   if (profileAssetStatus) {
1871:     const assetLabels = [
1872:       profile.profileImage ? "profile photo" : "",
1873:       profile.heroImage ? "main background" : "",
1874:       profile.resumeUrl ? "resume" : "",
1875:       profile.brandImage ? "brand icon" : ""
1876:     ].filter(Boolean);
1877:     profileAssetStatus.textContent = assetLabels.length
1878:       ? `Added: ${assetLabels.join(", ")}`
1879:       : "No profile photo, resume, brand icon, or main background added yet.";
1880:   }
1881: }

```

syncSiteContentFromInputs (template-preview.js, lines 1883-1897)

syncSiteContentFromInputs keeps two places aligned, such as local data and target files. In this file, it appears around lines 1883-1897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richFieldValue, and normalizeSiteContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1896: return catalog.siteContent;

Important local values include eyebrow at line 1885; title at line 1886; copy at line 1887. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1883: function syncSiteContentFromInputs() {
1884:   catalog.siteContentRich = catalog.siteContentRich || {};
1885:   const eyebrow = richFieldValue(siteHeroEyebrowInput, catalog.siteContent?.heroEyebrow || "");
1886:   const title = richFieldValue(siteHeroTitleInput, catalog.siteContent?.heroTitle || "");
1887:   const copy = richFieldValue(siteHeroCopyInput, catalog.siteContent?.heroCopy || "");
1888:   catalog.siteContent = normalizeSiteContent({
1889:     heroCopy: copy.plain,
1890:     heroEyebrow: eyebrow.plain,
1891:     heroTitle: title.plain
1892:   });
1893:   catalog.siteContentRich.heroCopy = copy.rich;
1894:   catalog.siteContentRich.heroEyebrow = eyebrow.rich;
1895:   catalog.siteContentRich.heroTitle = title.rich;
1896:   return catalog.siteContent;
1897: }

```

syncProfileFromInputs (template-preview.js, lines 1899-1926)

syncProfileFromInputs keeps two places aligned, such as local data and target files. In this file, it appears around lines 1899-1926 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richFieldValue, normalizeProfile, entries, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1925: return catalog.profile;

Important local values include fields at line 1901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1899: function syncProfileFromInputs() {
1900:   catalog.profileRich = catalog.profileRich || {};
1901:   const fields = {
1902:     contactIntro: richFieldValue(profileContactIntroInput, catalog.profile?.contactIntro || ""),
1903:     displayName: richFieldValue(profileDisplayNameInput, catalog.profile?.displayName || ""),
1904:     email: richFieldValue(profileEmailInput, catalog.profile?.email || ""),
1905:     githubUrl: richFieldValue(profileGithubInput, catalog.profile?.githubUrl || ""),
1906:     linkedinUrl: richFieldValue(profileLinkedinInput, catalog.profile?.linkedinUrl || ""),
1907:     phone: richFieldValue(profilePhoneInput, catalog.profile?.phone || ""),
1908:     portfolioLabel: richFieldValue(profilePortfolioLabelInput, catalog.profile?.portfolioLabel || ""),
1909:     websiteUrl: richFieldValue(profileWebsiteInput, catalog.profile?.websiteUrl || "")

```

```

1910:   };
1911:   catalog.profile = normalizeProfile({
1912:     ...(catalog.profile || {}),
1913:     contactIntro: fields.contactIntro.plain,
1914:     displayName: fields.displayName.plain,
1915:     email: fields.email.plain,
1916:     githubUrl: fields.githubUrl.plain,
1917:     linkedinUrl: fields.linkedinUrl.plain,
1918:     phone: fields.phone.plain,
1919:     portfolioLabel: fields.portfolioLabel.plain,
1920:     websiteUrl: fields.websiteUrl.plain
1921:   });
1922:   Object.entries(fields).forEach(([key, value]) => {
1923:     catalog.profileRich[key] = value.rich;
1924:   });
1925:   return catalog.profile;
1926: }

```

clone (template-preview.js, lines 1928-1930)

clone part of the private builder frontend and editing experience. In this file, it appears around lines 1928-1930 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1929: return JSON.parse(JSON.stringify(value));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1928: function clone(value) {
1929:   return JSON.parse(JSON.stringify(value));
1930: }

```

closeDialogElement (template-preview.js, lines 1932-1948)

closeDialogElement controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 1932-1948 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references hasAttribute, close, removeAttribute, dispatchEvent, and Event. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1933: if (!dialog) return;; line 1942: if (!dialog.open && !dialog.hasAttribute("open")) return;.

Important local values include closeValue at line 1934; wasOpen at line 1935. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1932: function closeDialogElement(dialog, returnValue = "close") {
1933:   if (!dialog) return;
1934:   const closeValue = String(returnValue || "close");
1935:   const wasOpen = Boolean(dialog.open || dialog.hasAttribute("open"));
1936:   if (typeof dialog.close === "function") {
1937:     try {
1938:       if (dialog.open) dialog.close(closeValue);
1939:     } catch {
1940:       // Fall through to manual cleanup if the browser rejects the native close call.
1941:     }
1942:     if (!dialog.open && !dialog.hasAttribute("open")) return;
1943:   }
1944:   dialog.returnValue = closeValue;
1945:   dialog.open = false;
1946:   dialog.removeAttribute("open");
1947:   if (wasOpen) dialog.dispatchEvent(new Event("close"));
1948: }

```

getByPath (template-preview.js, lines 1950-1952)

getByPath part of the private builder frontend and editing experience. In this file, it appears around lines 1950-1952 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, filter, and reduce. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1951: return String(pathValue || "").split(".").filter(Boolean).reduce((current, key) => current?.[key], target);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1950: function getByPath(target, pathValue) {  
1951:   return String(pathValue || "").split(".").filter(Boolean).reduce((current, key) => current?.[key],  
target);  
1952: }
```

setByPath (template-preview.js, lines 1954-1963)

setByPath part of the private builder frontend and editing experience. In this file, it appears around lines 1954-1963 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, filter, pop, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include keys at line 1955; last at line 1956; current at line 1957. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1954: function setByPath(target, pathValue, value) {  
1955:   const keys = String(pathValue || "").split(".").filter(Boolean);  
1956:   const last = keys.pop();  
1957:   let current = target;  
1958:   keys.forEach((key) => {  
1959:     current[key] = current[key] || {};  
1960:     current = current[key];  
1961:   });  
1962:   if (last) current[last] = value;  
1963: }
```

defaultDesignModel (template-preview.js, lines 1965-1971)

defaultDesignModel part of the private builder frontend and editing experience. In this file, it appears around lines 1965-1971 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1966: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1965: function defaultDesignModel() {  
1966:   return {  
1967:     brief: { summary: "", summaryRich: null, files: [] },  
1968:     documentation: { summary: "", summaryRich: null, files: [], references: [], mathAnalysis: [] },  
1969:     simulation: { summary: "", summaryRich: null, files: [], results: [] }  
1970:   };  
1971: }
```

isAnalogProject (template-preview.js, lines 1973-1975)

isAnalogProject part of the private builder frontend and editing experience. In this file, it appears around lines 1973-1975 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1974: return project?.category === "analog-mixed-signal";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1973: function isAnalogProject(project) {
1974:   return project?.category === "analog-mixed-signal";
1975: }
```

ensureDesignModel (template-preview.js, lines 1977-1992)

ensureDesignModel part of the private builder frontend and editing experience. In this file, it appears around lines 1977-1992 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references defaultDesignModel. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1978: if (!project) return null;; line 1991: return project.design;;

Important local values include defaults at line 1979. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1977: function ensureDesignModel(project) {
1978:   if (!project) return null;
1979:   const defaults = defaultDesignModel();
1980:   project.design = project.design || {};
1981:   project.design.brief = { ...defaults.brief, ...(project.design.brief || {}) };
1982:   project.design.documentation = { ...defaults.documentation, ...(project.design.documentation || {}) };
1983:   project.design.documentation.summaryRich = project.design.documentation.summaryRich || null;
1984:   project.design.simulation = { ...defaults.simulation, ...(project.design.simulation || {}) };
1985:   project.design.brief.files = project.design.brief.files || [];
1986:   project.design.documentation.files = project.design.documentation.files || [];
1987:   project.design.documentation.references = project.design.documentation.references || [];
1988:   project.design.documentation.mathAnalysis = project.design.documentation.mathAnalysis || [];
1989:   project.design.simulation.files = project.design.simulation.files || [];
1990:   project.design.simulation.results = project.design.simulation.results || [];
1991:   return project.design;
1992: }
```

ensureCompileCode (template-preview.js, lines 1994-2034)

ensureCompileCode part of the compile code language/tool/build/run flow. In this file, it appears around lines 1994-2034 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, normalizeCodeLanguage, detectCodeLanguage, safeClientCodeFileName, defaultCodeFileName, normalizeCompileFileRole, inferCompileFileRole, slugify, now, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1995: if (!project) return null;; line 2002: return {; line 2033: return project.compileCode;;

Important local values include language at line 1999; fileName at line 2000; role at line 2001; appendDestinations at line 2021. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1994: function ensureCompileCode(project) {
1995:   if (!project) return null;
1996:   project.compileCode = project.compileCode || {};
1997:   project.compileCode.files = Array.isArray(project.compileCode.files) ? project.compileCode.files : [];
1998:   project.compileCode.files = project.compileCode.files.map((file, index) => {
1999:     const language = normalizeCodeLanguage(file.language || detectCodeLanguage(file.code || "",
file.fileName || ""));
2000:     const fileName = safeClientCodeFileName(file.fileName || defaultCodeFileName(language), language);
2001:     const role = normalizeCompileFileRole(file.role || inferCompileFileRole(fileName, file.code || "",
```

```

language), language);
2002:   return {
2003:     id: file.id || slugify(`${fileName}-${index}-${Date.now()}`),
2004:     title: file.title || fileName,
2005:     fileName,
2006:     language,
2007:     role,
2008:     code: String(file.code || ""),
2009:     stdin: String(file.stdin || ""),
2010:     savedPath: file.savedPath || file.workspacePath || "",
2011:     savedAt: file.savedAt || "",
2012:     waveform: file.waveform || file.lastResult?.waveform || null,
2013:     lastResult: file.lastResult || null,
2014:     dirty: Boolean(file.dirty)
2015:   };
2016: });
2017: if (!project.compileCode.activeFileId || !project.compileCode.files.some((file) => file.id ===
project.compileCode.activeFileId)) {
2018:   project.compileCode.activeFileId = project.compileCode.files[0]?.id || "";
2019: }
2020: project.compileCode.terminal = String(project.compileCode.terminal || "");
2021: const appendDestinations = compileAppendDestinations(project);
2022: if (!project.compileCode.appendTargetId || !appendDestinations.some((destination) => destination.id ===
project.compileCode.appendTargetId)) {
2023:   project.compileCode.appendTargetId = appendDestinations[0]?.id || "";
2024: }
2025: project.compileCode.messages = Array.isArray(project.compileCode.messages)
2026:   ? project.compileCode.messages.map((message) => ({
2027:     id: message.id || slugify(`message-${Date.now()}-${Math.random().toString(16).slice(2)}`),
2028:     at: message.at || new Date().toISOString(),
2029:     level: ["info", "success", "warning", "error"].includes(message.level) ? message.level : "info",
2030:     text: String(message.text || "")
2031:   })).filter((message) => message.text)
2032:   : [];
2033: return project.compileCode;
2034: }

```

escapeHtml (template-preview.js, lines 2036-2043)

escapeHtml part of the private builder frontend and editing experience. In this file, it appears around lines 2036-2043 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2037: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2036: function escapeHtml(value) {
2037:   return String(value || "")
2038:     .replaceAll("&", "&amp;")
2039:     .replaceAll("<", "&lt;")
2040:     .replaceAll(">", "&gt;")
2041:     .replaceAll("'", "&quot;")
2042:     .replaceAll('"', "&#039;");
2043: }

```

escapeCodeHtml (template-preview.js, lines 2045-2051)

escapeCodeHtml part of the private builder frontend and editing experience. In this file, it appears around lines 2045-2051 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2046: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2045: function escapeCodeHtml(value) {
2046:   return String(value || "")
2047:     .replaceAll("&", "&amp;")
2048:     .replaceAll("<", "&lt;")

```

```

2049:     .replaceAll(">", "&gt;");
2050:     .replaceAll("'", "&quot;");
2051: }

```

normalizeCodeLanguage (template-preview.js, lines 2053-2059)

normalizeCodeLanguage validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2053-2059 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, replace, find, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2058: return match?.id || "javascript";.

Important local values include clean at line 2054; match at line 2055. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2053: function normalizeCodeLanguage(value = "") {
2054:   const clean = String(value || "").trim().toLowerCase().replace(/[_-]+/g, " ");
2055:   const match = supportedCodeLanguages.find((language) =>
2056:     language.id === clean || language.aliases.includes(clean)
2057:   );
2058:   return match?.id || "javascript";
2059: }

```

codeLanguageProfile (template-preview.js, lines 2061-2063)

codeLanguageProfile part of the private builder frontend and editing experience. In this file, it appears around lines 2061-2063 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2062: return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value)) || supportedCodeLanguages.find((language) => language.id === "javascript");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2061: function codeLanguageProfile(value = "") {
2062:   return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value)) ||
supportedCodeLanguages.find((language) => language.id === "javascript");
2063: }

```

codeLanguageLabel (template-preview.js, lines 2065-2067)

codeLanguageLabel part of the private builder frontend and editing experience. In this file, it appears around lines 2065-2067 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2066: return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label || "Code";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2065: function codeLanguageLabel(value = "") {
2066:   return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label
|| "Code";
2067: }

```


sourceLooksCpp (template-preview.js, lines 2069-2073)

sourceLooksCpp part of the private builder frontend and editing experience. In this file, it appears around lines 2069-2073 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2071: return
`/`#include`s\*<(?::iostream|string|vector|array|map|unordered\_map|memory|algorithm|optional|variant)>/.test(text) `|`.

Important local values include text at line 2070. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2069: function sourceLooksCpp(source = "") {
2070:   const text = String(source || "");
2071:   return
`/`#include`s*<(?::iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text)
`|`
2072:   /\b(std::|cout|cin|cerr|namespace|template`s*<|class`s+\w+|new`s+\w|delete`s+|constexpr|nullptr|using
\s+namespace)\b/.test(text);
2073: }
```

sourceLooksC (template-preview.js, lines 2075-2079)

sourceLooksC part of the private builder frontend and editing experience. In this file, it appears around lines 2075-2079 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2077: return
`/`#include`s\*<(?::stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) `|`.

Important local values include text at line 2076. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2075: function sourceLooksC(source = "") {
2076:   const text = String(source || "");
2077:   return `/`#include`s*<(?::stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) `|`
2078:   /\b(printf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text);
2079: }
```

languageFromFileName (template-preview.js, lines 2081-2090)

languageFromFileName part of the private builder frontend and editing experience. In this file, it appears around lines 2081-2090 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, includes, split, pop, sourceLooksCpp, sourceLooksC, and find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2085: if (sourceLooksCpp(source)) return "cpp";; line 2086: if (sourceLooksC(source)) return "c";; line 2087: return "c";; line 2089: return supportedCodeLanguages.find((language) => language.extensions?.includes(extension))?.id || "";

Important local values include clean at line 2082; extension at line 2083. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2081: function languageFromFileName(fileName = "", source = "") {
2082:   const clean = String(fileName || "").toLowerCase();
2083:   const extension = clean.includes(".") ? `.`.${clean.split(".").pop()}` : "";
2084:   if (extension === ".h") {
2085:     if (sourceLooksCpp(source)) return "cpp";
```

```

2086:     if (sourceLooksC(source)) return "c";
2087:     return "c";
2088:   }
2089:   return supportedCodeLanguages.find((language) => language.extensions?.includes(extension))?.id || "";
2090: }

```

defaultCodeFileName (template-preview.js, lines 2092-2094)

defaultCodeFileName part of the private builder frontend and editing experience. In this file, it appears around lines 2092-2094 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references codeLanguageProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2093: return codeLanguageProfile(language)?.defaultFile || "main.js";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2092: function defaultCodeFileName(language = "javascript") {
2093:   return codeLanguageProfile(language)?.defaultFile || "main.js";
2094: }

```

safeClientCodeFileName (template-preview.js, lines 2096-2104)

safeClientCodeFileName validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2096-2104 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references codeLanguageProfile, trim, replace, match, toLowerCase, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2103: return `\${namePart}\${profile.extensions?.includes(ext) ? ext : fallback.match(/\.[^.]\*/)?.[1] || ".js"}`;.

Important local values include profile at line 2097; fallback at line 2098; clean at line 2099; namePart at line 2100; extMatch at line 2101; ext at line 2102. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2096: function safeClientCodeFileName(fileName = "", language = "javascript") {
2097:   const profile = codeLanguageProfile(language);
2098:   const fallback = profile.defaultFile || "main.js";
2099:   const clean = String(fileName || fallback).trim().replace(/[\\/:*?<>|]+/g, "-");
2100:   const namePart = clean.replace(/\.[^.]*/g, "").replace(/^[a-zA-Z0-9_-]+/g, "-").replace(/^-+|-+$/g, "") || fallback.replace(/\.[^.]*/g, "");
2101:   const extMatch = clean.match(/(\.[a-zA-Z0-9_-]+)$/);
2102:   const ext = (extMatch?.[1] || fallback.match(/(\.[^.]*)$/)?.[1] || ".js").toLowerCase();
2103:   return `${namePart}${profile.extensions?.includes(ext) ? ext : fallback.match(/(\.[^.]*)$/)?.[1] || ".js"}`;
2104: }

```

normalizeCodePasteMode (template-preview.js, lines 2106-2108)

normalizeCodePasteMode validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2106-2108 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2107: return value === "source" ? "source" : "basic";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2106: function normalizeCodePasteMode(value = "") {
2107:   return value === "source" ? "source" : "basic";
2108: }

```

normalizeCodeText (template-preview.js, lines 2110-2119)

normalizeCodeText validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2110-2119 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, normalizeCodePasteMode, split, map, trimEnd, join, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2112: if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, " ").replace(/\s+\$/g, ""); line 2113: return normalized.

Important local values include normalized at line 2111. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2110: function normalizeCodeText(value = "", pasteMode = "source") {
2111:   const normalized = String(value || "").replace(/\r\n?/g, "\n").replace(/\u00a0/g, " ");
2112:   if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, " ").replace(/\s+$/g, "");
2113:   return normalized
2114:     .split("\n")
2115:     .map((line) => line.trimEnd())
2116:     .join("\n")
2117:     .replace(/\n{4,}/g, "\n\n\n")
2118:     .trim();
2119: }

```

detectCodeLanguage (template-preview.js, lines 2121-2136)

detectCodeLanguage part of the private builder frontend and editing experience. In this file, it appears around lines 2121-2136 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references languageFromFileName, b, sourceLooksCpp, and sourceLooksC. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 2123: if (byFile) return byFile;; line 2125: if (/<V?[a-z][\s\S]*?>/i.test(code) || /<!doctype\s+html/i.test(code)) return "html"; line 2126: if (/^(\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+lw+)\b)/.test(code)) return "systemverilog"; line 2127: if (/^(\b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b)/.test(code)) return "verilog";. There are 6 additional return statements in the function body.

Important local values include byFile at line 2122; code at line 2124. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2121: function detectCodeLanguage(value = "", fileName = "") {
2122:   const byFile = languageFromFileName(fileName, value);
2123:   if (byFile) return byFile;
2124:   const code = String(value || "");
2125:   if (/<\/?[a-z][\s\S]*?>/i.test(code) || /<!doctype\s+html/i.test(code)) return "html";
2126:   if (/^(\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+lw+)\b)/.test(code)) return "systemverilog";
2127:   if (/^(\b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b)/.test(code)) return "verilog";
2128:   if (/^(\s*\.?(tran|ac|dc|op|model|subckt|ends|param)\b/im.test(code) || /\bVWw\s+Ww\s+Ww\s+(?:DC|SIN|PULSE)?/i.test(code)) return "ltspice";
2129:   if (/^(\b(def|elif|import\s+Ww+|from\s+Ww\s+import|self|None|True|False)\b)/.test(code)) return "python";
2130:   if (/^(\b(public\s+class|private\s+protected\s+static\s+void\s+main|System\.out)\b)/.test(code)) return "java";
2131:   if (sourceLooksCpp(code) || sourceLooksC(code) || /\b(#include|main\s*(|puts\s*(|fprintf|sizeof\s*(|)\b)/.test(code)) {
2132:     return sourceLooksCpp(code) ? "cpp" : "c";
2133:   }
2134:   if (/^(\b(const|let|var|function|=>|console\.log|document\.|window\.)\b)/.test(code)) return "javascript";
2135:   return "javascript";
2136: }

```

tokenizedCodeHtml (template-preview.js, lines 2138-2192)

tokenizedCodeHtml part of the private builder frontend and editing experience. In this file, it appears around lines 2138-2192 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `escapeCodeHtml`, `fromCharCode`, `floor`, `replace`, `tokenName`, `push`, `protect`, `includes`, `normalizeCodeLanguage`, `b`, and 2 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 2149: return "\uE000\${output}\uE001"; line 2155: return token;; line 2176: c: " _Alignas _Alignof _Atomic _Bool _Complex _Generic _Imaginary _Noreturn _Static_assert _Thread_local _auto _break _case _ch ar _const _continue _default _do _double _else _enum _extern _float _for _goto _if _inline _int _long _register _re; line 2177: cpp: "alignas _alignof _and _and_eq _a sm _auto _bitand _bitor _bool _break _case _catch _char _char8 _t _char16 _t _char32 _t _class _compl _concept _const _constexpr _constexpr _const _c_ ast _continue _co_await _co_return _co_yield _decltype. There are 5 additional return statements in the function body.

Important local values include `html` at line 2139; `tokens` at line 2140; `tokenName` at line 2141; `value` at line 2142; `output` at line 2143; `protect` at line 2151; `token` at line 2153; `commentPattern` at line 2163; plus 2 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2138: function tokenizedCodeHtml(code = "", language = "javascript") {
2139:   let html = escapeCodeHtml(code);
2140:   const tokens = [];
2141:   const tokenName = (index) => {
2142:     let value = Number(index) + 1;
2143:     let output = "";
2144:     while (value > 0) {
2145:       value -= 1;
2146:       output = String.fromCharCode(65 + (value % 26)) + output;
2147:       value = Math.floor(value / 26);
2148:     }
2149:     return `\\uE000${output}\\uE001`;
2150:   };
2151:   const protect = (pattern, className) => {
2152:     html = html.replace(pattern, (match) => {
2153:       const token = tokenName(tokens.length);
2154:       tokens.push({ token, html: `
```

```

|void|volatile|while",
2182:   javascript: "async|await|break|case|catch|class|const|continue|debugger|default|delete|do|else|export
|extends|false|finally|for|from|function|if|import|in|instanceof|let|new|null|of|return|static|super|switch|this
|throw|true|try|typeof|undefined|var|void|while|yield",
2183:   python: "and|as|assert|async|await|break|class|continue|def|del|elif|else|except|False|finally|for|fr
om|global|if|import|in|is|lambda|None|nonlocal|not|or|pass|raise|return|True|try|while|with|yield",
2184:   html: "html|head|body|main|section|article|div|span|script|style|link|meta|title|header|footer|nav|bu
tton|input|form|label|img|a|p|pre|code"
2185:   };
2186:   const keywords = keywordMap[normalizeCodeLanguage(language)] || keywordMap.javascript;
2187:   html = html.replace(new RegExp(`\\b(${keywords})\\b`, "g"), '<span class="code-token-
keyword">$1</span>');
2188:   tokens.forEach((token) => {
2189:     html = html.replaceAll(token.token, token.html);
2190:   });
2191:   return html;
2192: }

```

tokenName (template-preview.js, lines 2141-2150)

tokenName part of the private builder frontend and editing experience. In this file, it appears around lines 2141-2150 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fromCharCode, and floor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2149: return ``\uE000\${output}\uE001``.

Important local values include tokenName at line 2141; value at line 2142; output at line 2143. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2141:   const tokenName = (index) => {
2142:     let value = Number(index) + 1;
2143:     let output = "";
2144:     while (value > 0) {
2145:       value -= 1;
2146:       output = String.fromCharCode(65 + (value % 26)) + output;
2147:       value = Math.floor(value / 26);
2148:     }
2149:     return ``\uE000${output}\uE001`;
2150:   };

```

protect (template-preview.js, lines 2151-2157)

protect part of the private builder frontend and editing experience. In this file, it appears around lines 2151-2157 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, tokenName, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2155: return token;

Important local values include protect at line 2151; token at line 2153. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2151:   const protect = (pattern, className) => {
2152:     html = html.replace(pattern, (match) => {
2153:       const token = tokenName(tokens.length);
2154:       tokens.push({ token, html: `<span class="${className}">${match}</span>` });
2155:       return token;
2156:     });
2157:   };

```

renderRichCodeBlock (template-preview.js, lines 2194-2204)

renderRichCodeBlock renders ui, markup, or display output. In this file, it appears around lines 2194-2204 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizeCodeLanguage`, `detectCodeLanguage`, `normalizeCodeText`, `codeLanguageLabel`, `escapeHtml`, and `tokenizedCodeHtml`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2198: `return ``.

Important local values include `language` at line 2195; `code` at line 2196; `label` at line 2197. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2194: function renderRichCodeBlock(block = {}) {
2195:   const language = normalizeCodeLanguage(block.language || detectCodeLanguage(block.code || ""));
2196:   const code = normalizeCodeText(block.code || "", block.pasteMode || "source");
2197:   const label = codeLanguageLabel(language);
2198:   return `
2199:     <figure class="rich-code-block rich-code-language-${language}">
2200:       <figcaption><span>&lt;/span> ${escapeHtml(label)}</figcaption>
2201:       <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
2202:     </figure>
2203:   `;
2204: }
```

renderPlainMultiline (template-preview.js, lines 2206-2208)

`renderPlainMultiline` renders ui, markup, or display output. In this file, it appears around lines 2206-2208 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `escapeHtml`, and `replace`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2207: `return escapeHtml(value).replace(/\r\n?|\n/g, "<br>");`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2206: function renderPlainMultiline(value = "") {
2207:   return escapeHtml(value).replace(/\r\n?|\n/g, "<br>");
2208: }
```

normalizeSiteContent (template-preview.js, lines 2210-2216)

`normalizeSiteContent` validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2210-2216 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `trim`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2211: `return {`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2210: function normalizeSiteContent(content = {}) {
2211:   return {
2212:     heroCopy: String(content.heroCopy || "").trim() || defaultSiteContent.heroCopy,
2213:     heroEyebrow: String(content.heroEyebrow || "").trim() || defaultSiteContent.heroEyebrow,
2214:     heroTitle: String(content.heroTitle || "").trim() || defaultSiteContent.heroTitle
2215:   };
2216: }
```

normalizeProfile (template-preview.js, lines 2218-2234)

`normalizeProfile` validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2218-2234 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2219: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2218: function normalizeProfile(profile = {}) {
2219:   return {
2220:     brandImage: String(profile.brandImage || "").trim(),
2221:     brandText: String(profile.brandText || "").trim() || String(profile.displayName || "").trim() ||
defaultProfile.brandText,
2222:     contactIntro: String(profile.contactIntro || "").trim(),
2223:     displayName: String(profile.displayName || "").trim(),
2224:     email: String(profile.email || "").trim(),
2225:     githubUrl: String(profile.githubUrl || "").trim(),
2226:     heroImage: String(profile.heroImage || "").trim(),
2227:     linkedinUrl: String(profile.linkedinUrl || "").trim(),
2228:     phone: String(profile.phone || "").trim(),
2229:     portfolioLabel: String(profile.portfolioLabel || "").trim() || defaultProfile.portfolioLabel,
2230:     profileImage: String(profile.profileImage || "").trim(),
2231:     resumeUrl: String(profile.resumeUrl || "").trim(),
2232:     websiteUrl: String(profile.websiteUrl || "").trim()
2233:   };
2234: }
```

normalizePlainFieldStyle (template-preview.js, lines 2236-2250)

normalizePlainFieldStyle validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2236-2250 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, normalizeTextColor, and rgbColorToHex. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2249: return normalized;.

Important local values include fontFamily at line 2237; fontPx at line 2238; color at line 2239; normalized at line 2240. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2236: function normalizePlainFieldStyle(style = {}) {
2237:   const fontFamily = cleanFontFamily(style.fontFamily || "");
2238:   const fontPx = normalizeFontPx(style.fontPx || style.fontSize || "");
2239:   const color = normalizeTextColor(style.color || "");
2240:   const normalized = {};
2241:
2242:   if (fontFamily) normalized.fontFamily = fontFamily;
2243:   if (fontPx) normalized.fontPx = fontPx;
2244:   if (color) normalized.color = rgbColorToHex(color);
2245:   if (style.bold === true || style.bold === "true") normalized.bold = true;
2246:   if (style.italic === true || style.italic === "true") normalized.italic = true;
2247:   if (style.underline === true || style.underline === "true") normalized.underline = true;
2248:
2249:   return normalized;
2250: }
```

normalizeFieldStyles (template-preview.js, lines 2252-2259)

normalizeFieldStyles validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2252-2259 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fromEntries, entries, filter, has, map, normalizePlainFieldStyle, and keys. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2253: return Object.fromEntries(.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2252: function normalizeFieldStyles(styles = {}) {
2253:   return Object.fromEntries(
2254:     Object.entries(styles || {})
2255:       .filter(([fieldId]) => persistentPlainStyleFieldIds.has(fieldId))
2256:       .map(([fieldId, style]) => [fieldId, normalizePlainFieldStyle(style)])
2257:       .filter(([ , style]) => Object.keys(style).length)
2258:   );
2259: }

```

loadBuilderPreferences (template-preview.js, lines 2261-2273)

loadBuilderPreferences loads data from disk, network, browser storage, or a service. In this file, it appears around lines 2261-2273 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse, getItem, includes, and applyBuilderPreferences. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include stored at line 2263. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2261: function loadBuilderPreferences() {
2262:   try {
2263:     const stored = JSON.parse(localStorage.getItem(preferenceStorageKey) || "{}");
2264:     builderPreferences = {
2265:       ...defaultBuilderPreferences,
2266:       ...stored,
2267:       theme: ["light", "dark"].includes(stored.theme) ? stored.theme : defaultBuilderPreferences.theme
2268:     };
2269:   } catch {
2270:     builderPreferences = { ...defaultBuilderPreferences };
2271:   }
2272:   applyBuilderPreferences();
2273: }

```

applyBuilderPreferences (template-preview.js, lines 2275-2280)

applyBuilderPreferences part of the private builder frontend and editing experience. In this file, it appears around lines 2275-2280 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include theme at line 2276. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2275: function applyBuilderPreferences() {
2276:   const theme = ["light", "dark"].includes(builderPreferences.theme) ? builderPreferences.theme :
"light";
2277:   document.documentElement.dataset.builderTheme = theme;
2278:   document.body.dataset.builderTheme = theme;
2279:   if (preferenceTheme) preferenceTheme.value = theme;
2280: }

```

setBuilderTheme (template-preview.js, lines 2282-2287)

setBuilderTheme part of the private builder frontend and editing experience. In this file, it appears around lines 2282-2287 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes, setItem, stringify, applyBuilderPreferences, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2282: function setBuilderTheme(theme = "light") {
2283:   builderPreferences.theme = ["light", "dark"].includes(theme) ? theme : "light";
2284:   localStorage.setItem(preferenceStorageKey, JSON.stringify(builderPreferences));
2285:   applyBuilderPreferences();
2286:   setStatus(`Builder ${builderPreferences.theme} mode enabled.`);
2287: }
```

openPreferencesDialog (template-preview.js, lines 2289-2292)

openPreferencesDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 2289-2292 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references applyBuilderPreferences, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2289: function openPreferencesDialog() {
2290:   applyBuilderPreferences();
2291:   if (!preferencesDialog?.open) preferencesDialog.showModal();
2292: }
```

saveBuilderPreferencesFromDialog (template-preview.js, lines 2294-2297)

saveBuilderPreferencesFromDialog persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2294-2297 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setBuilderTheme, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2294: function saveBuilderPreferencesFromDialog() {
2295:   setBuilderTheme(preferenceTheme?.value || "light");
2296:   setStatus("Preferences saved.");
2297: }
```

plainFieldStyleToCss (template-preview.js, lines 2299-2309)

plainFieldStyleToCss part of the private builder frontend and editing experience. In this file, it appears around lines 2299-2309 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizePlainFieldStyle, push, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2308: return rules.join("; ");.

Important local values include normalized at line 2300; rules at line 2301. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2299: function plainFieldStyleToCss(style = {}) {
2300:   const normalized = normalizePlainFieldStyle(style);
2301:   const rules = [];
2302:   if (normalized.fontFamily) rules.push(`font-family: ${normalized.fontFamily}`);
2303:   if (normalized.fontSize) rules.push(`font-size: ${normalized.fontSize}px`);
```

```

2304:   if (normalized.color) rules.push(`color: ${normalized.color}`);
2305:   if (normalized.bold) rules.push("font-weight: 700");
2306:   if (normalized.italic) rules.push("font-style: italic");
2307:   if (normalized.underline) rules.push("text-decoration: underline");
2308:   return rules.join("; ");
2309: }

```

plainFieldStyleAttribute (template-preview.js, lines 2311-2314)

plainFieldStyleAttribute part of the private builder frontend and editing experience. In this file, it appears around lines 2311-2314 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references plainFieldStyleToCss, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2313: return css ? ` style="\${escapeHtml(css)}" ` : "";

Important local values include css at line 2312. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2311: function plainFieldStyleAttribute(styles = {}, fieldId = "") {
2312:   const css = plainFieldStyleToCss(styles[fieldId]);
2313:   return css ? ` style="${escapeHtml(css)}" ` : "";
2314: }

```

plainStyleFromControl (template-preview.js, lines 2316-2326)

plainStyleFromControl part of the private builder frontend and editing experience. In this file, it appears around lines 2316-2326 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizePlainFieldStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2317: if (!control) return {}; line 2318: return normalizePlainFieldStyle({

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2316: function plainStyleFromControl(control) {
2317:   if (!control) return {};
2318:   return normalizePlainFieldStyle({
2319:     bold: control.dataset.builderBold,
2320:     color: control.dataset.builderColor,
2321:     fontFamily: control.dataset.builderFontFamily,
2322:     fontPx: control.dataset.builderFontPx,
2323:     italic: control.dataset.builderItalic,
2324:     underline: control.dataset.builderUnderline
2325:   });
2326: }

```

applyPlainFieldStyleToControl (template-preview.js, lines 2328-2343)

applyPlainFieldStyleToControl part of the private builder frontend and editing experience. In this file, it appears around lines 2328-2343 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizePlainFieldStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2329: if (!control) return;

Important local values include normalized at line 2330. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2328: function applyPlainFieldStyleToControl(control, style = {}) {

```

```

2329:   if (!control) return;
2330:   const normalized = normalizePlainFieldStyle(style);
2331:   control.dataset.builderFontFamily = normalized.fontFamily || "";
2332:   control.dataset.builderFontPx = normalized.fontPx || "";
2333:   control.dataset.builderColor = normalized.color || "";
2334:   control.dataset.builderBold = normalized.bold ? "true" : "false";
2335:   control.dataset.builderItalic = normalized.italic ? "true" : "false";
2336:   control.dataset.builderUnderline = normalized.underline ? "true" : "false";
2337:   control.style.fontFamily = normalized.fontFamily || "";
2338:   control.style.fontSize = normalized.fontPx ? `${normalized.fontPx}px` : "";
2339:   control.style.color = normalized.color || "";
2340:   control.style.fontWeight = normalized.bold ? "700" : "";
2341:   control.style.fontStyle = normalized.italic ? "italic" : "";
2342:   control.style.textDecoration = normalized.underline ? "underline" : "";
2343: }

```

applyStoredPlainFieldStyle (template-preview.js, lines 2345-2349)

applyStoredPlainFieldStyle part of the private builder frontend and editing experience. In this file, it appears around lines 2345-2349 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references has, normalizeFieldStyles, and applyPlainFieldStyleToControl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2346: if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;.

Important local values include styles at line 2347. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2345: function applyStoredPlainFieldStyle(control) {
2346:   if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;
2347:   const styles = normalizeFieldStyles(catalog.fieldStyles || {});
2348:   applyPlainFieldStyleToControl(control, styles[control.id] || {});
2349: }

```

storePlainControlStyle (template-preview.js, lines 2351-2365)

storePlainControlStyle persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2351-2365 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references has, normalizeFieldStyles, plainStyleFromControl, keys, clone, markDraftNeedsSave, scheduleAutosave, and schedulePreviewRender. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2352: if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;.

Important local values include nextStyles at line 2353; style at line 2354. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2351: function storePlainControlStyle(control) {
2352:   if (!control?.id || !persistentPlainStyleFieldIds.has(control.id)) return;
2353:   const nextStyles = normalizeFieldStyles(catalog.fieldStyles || {});
2354:   const style = plainStyleFromControl(control);
2355:   if (Object.keys(style).length) {
2356:     nextStyles[control.id] = style;
2357:   } else {
2358:     delete nextStyles[control.id];
2359:   }
2360:   catalog.fieldStyles = nextStyles;
2361:   savedPortfolioCatalog.fieldStyles = clone(nextStyles);
2362:   markDraftNeedsSave();
2363:   scheduleAutosave();
2364:   schedulePreviewRender();
2365: }

```

mailComposeLink (template-preview.js, lines 2367-2370)

mailComposeLink part of the private builder frontend and editing experience. In this file, it appears around lines 2367-2370 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2369: return clean ?

`https://mail.google.com/mail/?view=cm&fs=1&to=\${encodeURIComponent(clean)}` : "";

Important local values include clean at line 2368. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2367: function mailComposeLink(email = "") {
2368:   const clean = String(email || "").trim();
2369:   return clean ? `https://mail.google.com/mail/?view=cm&fs=1&to=${encodeURIComponent(clean)}` : "";
2370: }
```

phoneLink (template-preview.js, lines 2372-2375)

phoneLink part of the private builder frontend and editing experience. In this file, it appears around lines 2372-2375 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2374: return clean ? `tel:\${clean.replace(/[^\d+]/g, "")}` : "";

Important local values include clean at line 2373. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2372: function phoneLink(phone = "") {
2373:   const clean = String(phone || "").trim();
2374:   return clean ? `tel:${clean.replace(/[^\d+]/g, "")}` : "";
2375: }
```

textBlocksFromPlainText (template-preview.js, lines 2377-2385)

textBlocksFromPlainText part of the private builder frontend and editing experience. In this file, it appears around lines 2377-2385 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2378: return [{.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2377: function textBlocksFromPlainText(text) {
2378:   return [{
2379:     align: "left",
2380:     fontFamily: "Arial",
2381:     fontSize: "normal",
2382:     text: String(text || ""),
2383:     type: "paragraph"
2384:   }];
2385: }
```

richBlocksForProject (template-preview.js, lines 2387-2389)

richBlocksForProject part of the private builder frontend and editing experience. In this file, it appears around lines 2387-2389 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clone, and textBlocksFromPlainText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2388: return project?.summaryRich?.blocks?.length ? clone(project.summaryRich.blocks) : textBlocksFromPlainText(project?.summary || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2387: function richBlocksForProject(project) {
2388:   return project?.summaryRich?.blocks?.length ? clone(project.summaryRich.blocks) :
textBlocksFromPlainText(project?.summary || "");
2389: }
```

plainTextFromRich (template-preview.js, lines 2391-2402)

plainTextFromRich part of the private builder frontend and editing experience. In this file, it appears around lines 2391-2402 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 2392: return (rich?.blocks || []); line 2394: if (block.type === "paragraph") return block.text || ""; line 2395: if (block.type === "formula") return block.formula || ""; line 2396: if (block.type === "code") return block.code || ""; line 2397: if (block.type === "image") return [block.title, block.caption].filter(Boolean).join(" ");. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2391: function plainTextFromRich(rich, separator = "\n\n") {
2392:   return (rich?.blocks || [])
2393:     .map((block) => {
2394:       if (block.type === "paragraph") return block.text || "";
2395:       if (block.type === "formula") return block.formula || "";
2396:       if (block.type === "code") return block.code || "";
2397:       if (block.type === "image") return [block.title, block.caption].filter(Boolean).join(" ");
2398:       return "";
2399:     })
2400:     .filter(Boolean)
2401:     .join(separator);
2402: }
```

unwrapFormula (template-preview.js, lines 2404-2417)

unwrapFormula part of the private builder frontend and editing experience. In this file, it appears around lines 2404-2417 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2414: if (match) return match[group].trim(); line 2416: return text;.

Important local values include text at line 2405; wrappers at line 2406; match at line 2413. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2404: function unwrapFormula(value) {
2405:   const text = String(value || "").trim();
2406:   const wrappers = [
2407:     [/^$\$(\[s\S]+\)\$\$/ , 1],
2408:     [/^\\\[([s\S]+)\\\[ \]/ , 1],
2409:     [/^\\\[([s\S]+)\\\[ \]/ , 1],
2410:     [/^$\$([s\S]+)\$\$/ , 1]
2411:   ];
2412:   for (const [pattern, group] of wrappers) {
2413:     const match = text.match(pattern);
2414:     if (match) return match[group].trim();
2415:   }
```

```
2416:   return text;
2417: }
```

isFormulaOnly (template-preview.js, lines 2419-2426)

isFormulaOnly part of the private builder frontend and editing experience. In this file, it appears around lines 2419-2426 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and looksLikeWebOrContactText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2421: if (!text) return false;; line 2422: if (looksLikeWebOrContactText(text)) return false;; line 2423: if (/^\$[\s\S]+\\$\$/.test(text) || /^\\[\\s\S]+\\\$/.test(text) || /^\\([\\s\S]+\\)\$/.test(text)) return true;; line 2425: return text.length <= 180 && mathSignals.test(text) && !/[.!?]s*\$/ .test(text);.

Important local values include text at line 2420; mathSignals at line 2424. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2419: function isFormulaOnly(value) {
2420:   const text = String(value || "").trim();
2421:   if (!text) return false;
2422:   if (looksLikeWebOrContactText(text)) return false;
2423:   if (/^$[\s\S]+\$$/.test(text) || /^\\[\\s\S]+\\$/.test(text) || /^\\([\\s\S]+\\)$/.test(text))
return true;
2424:   const mathSignals = /\frac|\sqrt|\sum|\int|\Delta|\omega|\pi|[\u0333\u0335\u0337]/;
2425:   return text.length <= 180 && mathSignals.test(text) && !/[.!?]s*$/ .test(text);
2426: }
```

renderInlineMath (template-preview.js, lines 2428-2437)

renderInlineMath renders ui, markup, or display output. In this file, it appears around lines 2428-2437 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, replace, b, match, slice, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2431: return withMath.replace(/b(https?:V\\[\\s<]+|www\\.\\[\\s<]+|(?github|linkedin|gitlab|bitbucket|drive|docs)\\.comV[\\s<]+)/gi, (match) => {; line 2435: return `<a href="\${href}" target="\_blank" rel="noreferrer">\${clean}</a>\${trailing}`;.

Important local values include escaped at line 2429; withMath at line 2430; trailing at line 2432; clean at line 2433; href at line 2434. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2428: function renderInlineMath(text) {
2429:   const escaped = escapeHtml(text);
2430:   const withMath = escaped.replace(/$([^\s]+)$/g, '<span class="rich-inline-formula">${1}</span>');
2431:   return withMath.replace(/b(https?:V\\[\\s<]+|www\\.\\[\\s<]+|(?github|linkedin|gitlab|bitbucket|drive|docs)\\.comV[\\s<]+)/gi, (match) => {
2432:     const trailing = match.match(/[,.;!?:]+$/)?.[0] || "";
2433:     const clean = trailing ? match.slice(0, -trailing.length) : match;
2434:     const href = normalizeLinkTarget(clean, { assumeWeb: true });
2435:     return `<a href="${href}" target="_blank" rel="noreferrer">${clean}</a>${trailing}`;
2436:   });
2437: }
```

renderMultilineInlineText (template-preview.js, lines 2439-2441)

renderMultilineInlineText renders ui, markup, or display output. In this file, it appears around lines 2439-2441 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderInlineMath, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2440: return renderInlineMath(text).replace(/\\r\\n?|\\n/g, "
");.

Relevant source excerpt:

looksLikeBareWebAddress (template-preview.js, lines 2443-2450)

Important local values include `clean` at line 2444; `host` at line 2447. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

looksLikeWebOrContactText (*template-preview.js*, lines 2452-2460)

Important local values include `clean` at line 2453. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

normalizeLinkTarget (template-preview.js, lines 2462-2469)

normalizeLinkTarget validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2462-2469 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and looksLikeBareWebAddress. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 2464: if (!clean) return ""; line 2465: if (/^\\/.test(clean)) return `https:\${clean}`; line 2466: if (/^www\\./i.test(clean)) return `https://\${clean}`; line 2467: if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://\${clean}`. There are 1 additional return statements in the function body.

Important local values include clean at line 2463. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2462: function normalizeLinkTarget(target = "", options = {}) {
2463:   const clean = String(target || "").trim();
2464:   if (!clean) return "";
2465:   if (/^\\/.test(clean)) return `https:${clean}`;
2466:   if (/^www\\./i.test(clean)) return `https://${clean}`;
2467:   if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://${clean}`;
2468:   return clean;
2469: }
```

isWebsiteLinkItem (template-preview.js, lines 2471-2479)

isWebsiteLinkItem part of the private builder frontend and editing experience. In this file, it appears around lines 2471-2479 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, filter, join, toLowerCase, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2473: if (/^(https?:)?\\/.test(clean) || /^www\\./i.test(clean)) return true; line 2478: return /\b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText);.

Important local values include clean at line 2472; typeText at line 2474. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2471: function isWebsiteLinkItem(item = {}, target = "") {
2472:   const clean = String(target || "").trim();
2473:   if (/^(https?:)?\\/.test(clean) || /^www\\./i.test(clean)) return true;
2474:   const typeText = [item.kind, item.type, item.status, item.meta, item.label, item.title]
2475:     .filter(Boolean)
2476:     .join(" ")
2477:     .toLowerCase();
2478:   return /\b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText);
2479: }
```

extensionFromUrl (template-preview.js, lines 2481-2485)

extensionFromUrl part of the private builder frontend and editing experience. In this file, it appears around lines 2481-2485 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, match, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2484: return match ? match[1].toLowerCase() : "";

Important local values include clean at line 2482; match at line 2483. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2481: function extensionFromUrl(value = "") {
2482:   const clean = String(value || "").split(/[?#]/)[0];
2483:   const match = clean.match(/([a-z0-9_]+)$/i);
2484:   return match ? match[1].toLowerCase() : "";
2485: }
```

urlLooksLikeDirectImage (template-preview.js, lines 2487-2491)

urlLooksLikeDirectImage part of the private builder frontend and editing experience. In this file, it appears around lines 2487-2491 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2489: return /^(data:imageV|blob:)/i.test(clean) ||.

Important local values include clean at line 2488. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2487: function urlLooksLikeDirectImage(value = "") {
2488:   const clean = normalizeLinkTarget(value, { assumeWeb: true });
2489:   return /^(data:imageV|blob:)/i.test(clean) ||
2490:     /\. (png|jpe?g|webp|gif|svg|bmp|avif) ([?#].*)?$/i.test(clean);
2491: }
```

inferUrlAssetKind (template-preview.js, lines 2493-2506)

inferUrlAssetKind part of the private builder frontend and editing experience. In this file, it appears around lines 2493-2506 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, extensionFromUrl, urlLooksLikeDirectImage, includes, and looksLikeBareWebAddress. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 2496: if (source === "drive" || (/^https?:\/\//)?(drive|docs)\.google\.com\/i.test(clean)) return "google-drive"; line 2497: if (/github\.com|raw\.githubusercontent\.com/i.test(clean)) return "github"; line 2498: if (/linkedin\.com/i.test(clean)) return "linkedin"; line 2499: if (urlLooksLikeDirectImage(clean)) return "image";. There are 6 additional return statements in the function body.

Important local values include clean at line 2494; extension at line 2495. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2493: function inferUrlAssetKind(value = "", source = "web") {
2494:   const clean = normalizeLinkTarget(value, { assumeWeb: true });
2495:   const extension = extensionFromUrl(clean);
2496:   if (source === "drive" || (/^https?:\/\//)?(drive|docs)\.google\.com\/i.test(clean)) return "google-drive";
2497:   if (/github\.com|raw\.githubusercontent\.com/i.test(clean)) return "github";
2498:   if (/linkedin\.com/i.test(clean)) return "linkedin";
2499:   if (urlLooksLikeDirectImage(clean)) return "image";
2500:   if ([".pdf"].includes(extension)) return "pdf";
2501:   if ([".doc", ".docx", ".ppt", ".pptx", ".xls", ".xlsx", ".csv", ".txt", ".md"].includes(extension))
return "document";
2502:   if ([".zip", ".7z", ".rar"].includes(extension)) return "archive";
2503:   if ([".c", ".h", ".cpp", ".hpp", ".py", ".js", ".mjs", ".ts", ".v", ".sv", ".vhd", ".vhdl", ".spice", ".cir", ".net", ".asc", ".sch", ".kicad_sch", ".kicad_pcb"].includes(extension)) return "engineering-file";
2504:   if (/^https?:\/\//i.test(clean) || /^www\./i.test(value) || looksLikeBareWebAddress(value)) return "webpage";
2505:   return extension ? "file" : "link";
2506: }
```

labelForUrlAssetKind (template-preview.js, lines 2508-2522)

labelForUrlAssetKind part of the private builder frontend and editing experience. In this file, it appears around lines 2508-2522 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2509: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2508: function labelForUrlAssetKind(kind = "link") {
2509:   return {
2510:     archive: "Archive link",
2511:     document: "Document link",
2512:     "engineering-file": "Engineering file link",
2513:     file: "File link",
2514:     github: "GitHub link",
2515:     "google-drive": "Google Drive link",
2516:     image: "Image URL",
2517:     linkedin: "LinkedIn link",
2518:     link: "Link",
2519:     pdf: "PDF link",
2520:     webpage: "Website link"
2521:   }[kind] || "Link";
2522: }
```

displayNameFromUrl (template-preview.js, lines 2524-2533)

displayNameFromUrl part of the private builder frontend and editing experience. In this file, it appears around lines 2524-2533 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, URL, split, filter, pop, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2529: return decodeURIComponent(lastPath || parsed.hostname.replace(/^www\./, "")) || fallback;; line 2531: return String(value || "").split("/").filter(Boolean).pop() || fallback;.

Important local values include clean at line 2525; parsed at line 2527; lastPath at line 2528. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2524: function displayNameFromUrl(value = "", fallback = "Linked asset") {
2525:   const clean = normalizeLinkTarget(value, { assumeweb: true });
2526:   try {
2527:     const parsed = new URL(clean);
2528:     const lastPath = parsed.pathname.split("/").filter(Boolean).pop();
2529:     return decodeURIComponent(lastPath || parsed.hostname.replace(/^www\./, "")) || fallback;
2530:   } catch {
2531:     return String(value || "").split("/").filter(Boolean).pop() || fallback;
2532:   }
2533: }
```

linkifyRichTextNodes (template-preview.js, lines 2535-2570)

linkifyRichTextNodes part of the private builder frontend and editing experience. In this file, it appears around lines 2535-2570 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createTreeWalker, acceptNode, closest, nextNode, push, forEach, createDocumentFragment, replace, b, append, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2538: if (!node.textContent || !(https?:\V|www\.(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\)/i.test(node.textContent)) return NodeFilter.FILTER_REJECT;; line 2539: return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;; line 2565: return match;.

Important local values include walker at line 2536; nodes at line 2542; node at line 2543; text at line 2550; fragment at line 2551; cursor at line 2552; trailing at line 2555; clean at line 2556; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2535: function linkifyRichTextNodes(root) {
2536:   const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {
2537:     acceptNode(node) {
2538:       if (!node.textContent || !(https?:\V|www\.(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\)/i.test(node.textContent)) return
NodeFilter.FILTER_REJECT;
2539:       return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;
2540:     }
2541:   });
2542:   const nodes = [];
2543:   let node = walker.nextNode();
```

```

2544:   while (node) {
2545:     nodes.push(node);
2546:     node = walker.nextNode();
2547:   }
2548:
2549:   nodes.forEach((textNode) => {
2550:     const text = textNode.textContent || "";
2551:     const fragment = document.createDocumentFragment();
2552:     let cursor = 0;
2553:     text.replace(/\b(https?:\/\/[^\s<]+|www\.[^\s<]+|(?:(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com
\/[^\s<]+)/gi, (match, _url, offset) => {
2554:       if (offset > cursor) fragment.append(document.createTextNode(text.slice(cursor, offset)));
2555:       const trailing = match.match(/[.,;:!?]+\$/)?.[0] || "";
2556:       const clean = trailing ? match.slice(0, -trailing.length) : match;
2557:       const link = document.createElement("a");
2558:       link.href = normalizeLinkTarget(clean, { assumeWeb: true });
2559:       link.target = "_blank";
2560:       link.rel = "noreferrer";
2561:       link.textContent = clean;
2562:       fragment.append(link);
2563:       if (trailing) fragment.append(document.createTextNode(trailing));
2564:       cursor = offset + match.length;
2565:       return match;
2566:     });
2567:     if (cursor < text.length) fragment.append(document.createTextNode(text.slice(cursor)));
2568:     textNode.replaceWith(fragment);
2569:   });
2570: }

```

inlineStyleSignature (template-preview.js, lines 2572-2583)

inlineStyleSignature part of the private builder frontend and editing experience. In this file, it appears around lines 2572-2583 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2574: if (!style) return ""; line 2575: return [.

Important local values include style at line 2573. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2572: function inlineStyleSignature(element) {
2573:   const style = element?.style;
2574:   if (!style) return "";
2575:   return [
2576:     style.fontFamily || "",
2577:     style.fontSize || "",
2578:     style.color || "",
2579:     style.fontWeight || "",
2580:     style.fontStyle || "",
2581:     style.textDecoration || ""
2582:   ].join("|");
2583: }

```

normalizeInlineStyleSpans (template-preview.js, lines 2585-2605)

normalizeInlineStyleSpans validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2585-2605 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, remove, getAttribute, replaceWith, normalize, matches, inlineStyleSignature, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2586: if (!root) return;; line 2591: return;.

Important local values include next at line 2597; remove at line 2600. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2585: function normalizeInlineStyleSpans(root) {
2586:   if (!root) return;

```

```

2587: root.querySelectorAll("span.rich-inline-style, span[style]").forEach((span) => {
2588:   span.style.display = "inline";
2589:   if (!span.textContent) {
2590:     span.remove();
2591:     return;
2592:   }
2593:   if (!span.getAttribute("style")) span.replaceWith(...span.childNodes);
2594: });
2595: root.normalize();
2596: root.querySelectorAll("span.rich-inline-style, span[style]").forEach((span) => {
2597:   let next = span.nextSibling;
2598:   while (next?.nodeType === Node.ELEMENT_NODE && next.matches("span.rich-inline-style, span[style]") &&
inlineStyleSignature(next) === inlineStyleSignature(span)) {
2599:     span.append(...next.childNodes);
2600:     const remove = next;
2601:     next = next.nextSibling;
2602:     remove.remove();
2603:   }
2604: });
2605: }

```

sanitizeRichInlineHtml (template-preview.js, lines 2607-2651)

sanitizeRichInlineHtml validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2607-2651 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, Set, querySelectorAll, forEach, has, replaceWith, getAttribute, trim, normalizeLinkTarget, cleanFontFamily, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2615: return;; line 2650: return template.innerHTML;.

Important local values include template at line 2608; allowedTags at line 2610; href at line 2618; normalizedHref at line 2619; safeHref at line 2620; fontFamily at line 2621; fontPx at line 2622; color at line 2623; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2607: function sanitizeRichInlineHtml(value = "") {
2608:   const template = document.createElement("template");
2609:   template.innerHTML = String(value || "");
2610:   const allowedTags = new Set(["A", "B", "BR", "EM", "I", "SPAN", "STRONG", "U"]);
2611:
2612:   [...template.content.querySelectorAll("*")].forEach((node) => {
2613:     if (!allowedTags.has(node.tagName)) {
2614:       node.replaceWith(...node.childNodes);
2615:       return;
2616:     }
2617:
2618:     const href = node.tagName === "A" ? String(node.getAttribute("href") || "").trim() : "";
2619:     const normalizedHref = normalizeLinkTarget(href, { assumeWeb: true });
2620:     const safeHref = /^(https?:|mailto:|tel:|\/)/i.test(normalizedHref) ? normalizedHref : "";
2621:     const fontFamily = cleanFontFamily(node.style.fontFamily || "");
2622:     const fontPx = normalizeFontPx(parseFloat(node.style.fontSize || ""));
2623:     const color = normalizeTextColor(node.style.color || "");
2624:     const rawFontWeight = node.style.fontWeight || "";
2625:     const fontWeight = /^(bold|[6-9]00)$/i.test(rawFontWeight)
2626:       ? "700"
2627:       : /^(normal|[1-5]00)$/i.test(rawFontWeight)
2628:       ? "400"
2629:       : "";
2630:     const fontStyle = node.style.fontStyle === "italic" ? "italic" : "";
2631:     const textDecoration = node.style.textDecoration.includes("underline") ? "underline" : "";
2632:
2633:     [...node.attributes].forEach((attribute) => node.removeAttribute(attribute.name));
2634:
2635:     if (node.tagName === "A" && safeHref) {
2636:       node.setAttribute("href", safeHref);
2637:       node.setAttribute("target", "_blank");
2638:       node.setAttribute("rel", "noreferrer");
2639:     }
2640:     if (fontFamily) node.style.fontFamily = fontFamily;
2641:     if (fontPx) node.style.fontSize = `${fontPx}px`;
2642:     if (color) node.style.color = color;
2643:     if (fontWeight) node.style.fontWeight = fontWeight;
2644:     if (fontStyle) node.style.fontStyle = fontStyle;
2645:     if (textDecoration) node.style.textDecoration = textDecoration;
2646:   });
2647:
2648:   linkifyRichTextNodes(template.content);
2649:   normalizeInlineStyleSpans(template.content);
2650:   return template.innerHTML;

```

```
2651: }
```

displayTitle (template-preview.js, lines 2653-2662)

displayTitle part of the private builder frontend and editing experience. In this file, it appears around lines 2653-2662 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2661: return replacements[title] || title;.

Important local values include title at line 2654; replacements at line 2655. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2653: function displayTitle(value, fallback = "Untitled item") {
2654:   const title = String(value || fallback).trim();
2655:   const replacements = {
2656:     "Brief": "Overview",
2657:     "Project Brief": "Overview",
2658:     "Design Brief": "Design Overview",
2659:     "Simulation Brief": "Simulation Overview"
2660:   };
2661:   return replacements[title] || title;
2662: }
```

cleanFontFamily (template-preview.js, lines 2664-2671)

cleanFontFamily part of the private builder frontend and editing experience. In this file, it appears around lines 2664-2671 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, replace, trim, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2665: return String(value).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2664: function cleanFontFamily(value = "") {
2665:   return String(value)
2666:     .split(",")
2667:     .map((item) => item.replace(/^[^w\s-]/g, "").trim())
2668:     .filter(Boolean)
2669:     .slice(0, 3)
2670:     .join(", ");
2671: }
```

normalizeFontPx (template-preview.js, lines 2673-2676)

normalizeFontPx validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2673-2676 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2675: return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";

Important local values include size at line 2674. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2673: function normalizeFontPx(value) {
2674:   const size = Number(value);
2675:   return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";
```

```
2676: }
```

normalizeTextColor (template-preview.js, lines 2677-2684)

normalizeTextColor validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2677-2684 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2679: return /^#([0-9a-f]{3}|[0-9a-f]{6})\$/i.test(color) ||.

Important local values include color at line 2678. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2677: function normalizeTextColor(value = "") {
2678:     const color = String(value || "").trim();
2679:     return /^#([0-9a-f]{3}|[0-9a-f]{6})$/i.test(color) ||
2680:         /^rgba?\(/i.test(color) ||
2681:         /^hsla?\(/i.test(color) ||
2682:         ? color
2683:         : "";
2684: }
```

summaryRichPathFromTextPath (template-preview.js, lines 2686-2688)

summaryRichPathFromTextPath part of the private builder frontend and editing experience. In this file, it appears around lines 2686-2688 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2687: return String(pathValue || "").replace(/\.summary\$/, ".summaryRich");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2686: function summaryRichPathFromTextPath(pathValue = "") {
2687:     return String(pathValue || "").replace(/\.summary$/, ".summaryRich");
2688: }
```

overviewFolderFromPath (template-preview.js, lines 2690-2695)

overviewFolderFromPath part of the private builder frontend and editing experience. In this file, it appears around lines 2690-2695 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2691: if (pathValue.includes("design.brief")) return "design-overview";; line 2692: if (pathValue.includes("design.documentation")) return "documentation-overview";; line 2693: if (pathValue.includes("design.simulation")) return "simulation-overview";; line 2694: return "overview";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2690: function overviewFolderFromPath(pathValue = "") {
2691:     if (pathValue.includes("design.brief")) return "design-overview";
2692:     if (pathValue.includes("design.documentation")) return "documentation-overview";
2693:     if (pathValue.includes("design.simulation")) return "simulation-overview";
2694:     return "overview";
2695: }
```

richBlocksFromValue (template-preview.js, lines 2697-2699)

richBlocksFromValue part of the private builder frontend and editing experience. In this file, it appears around lines 2697-2699 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clone, and textBlocksFromPlainText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2698: return rich?.blocks?.length ? clone(rich.blocks) : textBlocksFromPlainText(fallbackText);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2697: function richBlocksFromValue(rich, fallbackText = "") {
2698:   return rich?.blocks?.length ? clone(rich.blocks) : textBlocksFromPlainText(fallbackText);
2699: }
```

richTextStyle (template-preview.js, lines 2700-2714)

richTextStyle part of the private builder frontend and editing experience. In this file, it appears around lines 2700-2714 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, normalizeTextColor, push, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2713: return styles.length ? ` style="\${escapeHtml(styles.join("; ")))}` : "";

Important local values include styles at line 2701; fontFamily at line 2702; fontPx at line 2703; color at line 2704. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2700: function richTextStyle(block = {}) {
2701:   const styles = [];
2702:   const fontFamily = cleanFontFamily(block.fontFamily || "Arial") || "Arial";
2703:   const fontPx = normalizeFontPx(block.fontPx);
2704:   const color = normalizeTextColor(block.color || "");
2705:
2706:   if (fontFamily) styles.push(`font-family: ${fontFamily}`);
2707:   if (fontPx) styles.push(`font-size: ${fontPx}px`);
2708:   if (color) styles.push(`color: ${color}`);
2709:   if (block.bold) styles.push("font-weight: 700");
2710:   if (block.italic) styles.push("font-style: italic");
2711:   if (block.underline) styles.push("text-decoration: underline");
2712:
2713:   return styles.length ? ` style="${escapeHtml(styles.join("; ")))}` : "";
2714: }
```

normalizeCropAspect (template-preview.js, lines 2716-2718)

normalizeCropAspect validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2716-2718 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2717: return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2716: function normalizeCropAspect(value = "") {
2717:   return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";
2718: }
```

normalizeCropZoom (template-preview.js, lines 2720-2723)

normalizeCropZoom validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2720-2723 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2722: return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;

Important local values include zoom at line 2721. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2720: function normalizeCropZoom(value = 1) {
2721:   const zoom = Number(value);
2722:   return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;
2723: }
```

normalizeCropPosition (template-preview.js, lines 2725-2728)

normalizeCropPosition validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2725-2728 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2727: return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;

Important local values include position at line 2726. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2725: function normalizeCropPosition(value = 50) {
2726:   const position = Number(value);
2727:   return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;
2728: }
```

richImageCropStyle (template-preview.js, lines 2730-2738)

richImageCropStyle part of the private builder frontend and editing experience. In this file, it appears around lines 2730-2738 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, push, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2737: return ` style="\${escapeHtml(styles.join("; "));}"`;

Important local values include aspect at line 2731; zoom at line 2732; x at line 2733; y at line 2734; styles at line 2735. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2730: function richImageCropStyle(block = {}) {
2731:   const aspect = normalizeCropAspect(block.cropAspect);
2732:   const zoom = normalizeCropZoom(block.cropZoom);
2733:   const x = normalizeCropPosition(block.cropX);
2734:   const y = normalizeCropPosition(block.cropY);
2735:   const styles = [ `--crop-zoom: ${zoom}`, `--crop-x: ${x}%`, `--crop-y: ${y}%` ];
2736:   if (aspect !== "original") styles.push(`--crop-aspect: ${aspect}`);
2737:   return ` style="${escapeHtml(styles.join("; "));}"`;
2738: }
```


richImageDownloadLink (template-preview.js, lines 2740-2745)

richImageDownloadLink part of the private builder frontend and editing experience. In this file, it appears around lines 2740-2745 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, cleanRichImageTitle, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2744: return `

Important local values include label at line 2741; url at line 2742; target at line 2743. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2740: function richImageDownloadLink(block = {}) {
2741:   const label = escapeHtml(cleanRichImageTitle(block) || block.caption || "Image file");
2742:   const url = block.url || "#";
2743:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(block, url) });
2744:   return `
```

cleanRichImageTitle (template-preview.js, lines 2747-2750)

cleanRichImageTitle part of the private builder frontend and editing experience. In this file, it appears around lines 2747-2750 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2749: return /^pasted image\b/i.test(title) ? "" : title;.

Important local values include title at line 2748. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2747: function cleanRichImageTitle(block = {}) {
2748:   const title = String(block.title || "").trim();
2749:   return /^pasted image\b/i.test(title) ? "" : title;
2750: }
```

renderRichContent (template-preview.js, lines 2752-2787)

renderRichContent renders ui, markup, or display output. In this file, it appears around lines 2752-2787 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, map, includes, richImageDownloadLink, cleanRichImageTitle, normalizeLinkTarget, normalizeCropAspect, richImageCropStyle, escapeHtml, unwrapFormula, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 2754: return `; line 2759: if (block.display === "download") return richImageDownloadLink(block);; line 2762: return `; line 2774: return `

Important local values include blocks at line 2753; align at line 2757; title at line 2760; imageSrc at line 2761; formula at line 2772; size at line 2779; content at line 2780. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2752: function renderRichContent(rich, fallbackText = "") {
2753:   const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
2754:   return `
2755:     <div class="rich-content">
```

```

2756:     ${blocks.map((block) => {
2757:       const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
2758:       if (block.type === "image") {
2759:         if (block.display === "download") return richImageDownloadLink(block);
2760:         const title = cleanRichImageTitle(block);
2761:         const imageSrc = normalizeLinkTarget(block.url, { assumeWeb: true });
2762:         return `
2763:           <figure class="rich-image justify-${align}">
2764:             <span class="rich-image-viewport crop-${normalizeCropAspect(block.cropAspect)} ===
"original" ? "original" : "active">${richImageCropStyle(block)}>
2765:               
2766:             </span>
2767:             ${title || block.caption} ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>`
: ""}${block.caption} ? `<span>${escapeHtml(block.caption)}</span>` : ""}</figcaption>` : ""}
2768:           </figure>
2769:         `;
2770:       }
2771:       if (block.type === "formula") {
2772:         const formula = unwrapFormula(block.formula);
2773:         if (looksLikeWebOrContactText(formula)) {
2774:           return `
```

renderRichFieldContent (template-preview.js, lines 2789-2799)

renderRichFieldContent renders ui, markup, or display output. In this file, it appears around lines 2789-2799 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, map, includes, sanitizeRichInlineHtml, renderInlineMath, richTextStyle, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2791: return blocks.map((block) => {}; line 2792: if (block.type !== "paragraph") return ""; line 2797: return `

Important local values include blocks at line 2790; align at line 2793; content at line 2794. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2789: function renderRichFieldContent(rich, fallbackText = "") {
2790:   const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
2791:   return blocks.map((block) => {
2792:     if (block.type !== "paragraph") return "";
2793:     const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
2794:     const content = block.html
2795:       ? sanitizeRichInlineHtml(block.html)
2796:       : renderInlineMath(block.text || "");
2797:     return `

```

saveSelectedProjectToPortfolio (template-preview.js, lines 2801-2817)

saveSelectedProjectToPortfolio persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2801-2817 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, parseProjectForPortfolio, syncParsedPortfolioGlobalsIntoSaved, filter, Set, map, has, findIndex, max, splice, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2803: if (!project) return false;; line 2816: return true;.

Important local values include project at line 2802; parsedProject at line 2804; nextProjects at line 2806; workingIds at line 2807; workingIndex at line 2809; insertAt at line 2810. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2801: function saveSelectedProjectToPortfolio() {
2802:   const project = selectedProject();
2803:   if (!project) return false;
2804:   const parsedProject = parseProjectForPortfolio(project);
2805:   syncParsedPortfolioGlobalsIntoSaved();
2806:   const nextProjects = (savedPortfolioCatalog.projects || []).filter((item) => item.id !== project.id);
2807:   const workingIds = new Set((catalog.projects || []).map((item) => item.id));
2808:   savedPortfolioCatalog.projects = nextProjects.filter((item) => workingIds.has(item.id));
2809:   const workingIndex = catalog.projects.findIndex((item) => item.id === project.id);
2810:   const insertAt = Math.max(0, workingIndex);
2811:   savedPortfolioCatalog.projects.splice(insertAt, 0, parsedProject);
2812:   markDraftNeedsSave();
2813:   refreshOpenPreviews();
2814:   setStatus(`${project.title} saved into the portfolio preview.`);
2815:   updateBuilderWorkflow();
2816:   return true;
2817: }
```

nextProjects (template-preview.js, lines 2806-2814)

nextProjects part of the private builder frontend and editing experience. In this file, it appears around lines 2806-2814 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, Set, map, has, findIndex, max, splice, markDraftNeedsSave, refreshOpenPreviews, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include nextProjects at line 2806; workingIds at line 2807; workingIndex at line 2809; insertAt at line 2810. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2806:   const nextProjects = (savedPortfolioCatalog.projects || []).filter((item) => item.id !== project.id);
2807:   const workingIds = new Set((catalog.projects || []).map((item) => item.id));
2808:   savedPortfolioCatalog.projects = nextProjects.filter((item) => workingIds.has(item.id));
2809:   const workingIndex = catalog.projects.findIndex((item) => item.id === project.id);
2810:   const insertAt = Math.max(0, workingIndex);
2811:   savedPortfolioCatalog.projects.splice(insertAt, 0, parsedProject);
2812:   markDraftNeedsSave();
2813:   refreshOpenPreviews();
2814:   setStatus(`${project.title} saved into the portfolio preview.`);
```

removeProjectFromPortfolio (template-preview.js, lines 2819-2821)

removeProjectFromPortfolio part of the private builder frontend and editing experience. In this file, it appears around lines 2819-2821 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2819: function removeProjectFromPortfolio(projectId) {
2820:   savedPortfolioCatalog.projects = (savedPortfolioCatalog.projects || []).filter((item) => item.id !==
projectId);
2821: }
```

deleteProjectById (template-preview.js, lines 2823-2836)

deleteProjectById part of the private builder frontend and editing experience. In this file, it appears around lines 2823-2836 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, filter, removeProjectFromPortfolio, closeDialogElement, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2825: if (!project) return;.

Important local values include project at line 2824. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2823: function deleteProjectById(projectId) {
2824:   const project = catalog.projects.find((item) => item.id === projectId);
2825:   if (!project) return;
2826:   catalog.projects = catalog.projects.filter((item) => item.id !== project.id);
2827:   removeProjectFromPortfolio(project.id);
2828:   selectedProjectId = catalog.projects[0]?.id || "";
2829:   activeSectionId = "brief";
2830:   if (projectDialog?.open && project.id === projectWindowTitle.dataset.projectId) {
2831:     closeDialogElement(projectDialog);
2832:   }
2833:   setStatus("Project deleted locally.");
2834:   scheduleAutosave();
2835:   renderAll();
2836: }
```

openDeleteConfirm (template-preview.js, lines 2838-2843)

openDeleteConfirm controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 2838-2843 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2838: function openDeleteConfirm({ title = "Delete project", message, onConfirm }) {
2839:   pendingDeleteAction = onConfirm;
2840:   deleteConfirmTitle.textContent = title;
2841:   deleteConfirmMessage.textContent = message;
2842:   deleteConfirmDialog.showModal();
2843: }
```

requestProjectDelete (template-preview.js, lines 2845-2853)

requestProjectDelete part of the private builder frontend and editing experience. In this file, it appears around lines 2845-2853 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, and deleteProjectById. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2847: if (!project) return;.

Important local values include project at line 2846. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2845: function requestProjectDelete(projectId) {
2846:   const project = catalog.projects.find((item) => item.id === projectId);
2847:   if (!project) return;
2848:   openDeleteConfirm({
2849:     title: "Delete project",
2850:     message: "Are you sure you want to delete this project?",
```

```

2851:     onConfirm: () => deleteProjectById(project.id)
2852:   });
2853: }

```

parserItem (template-preview.js, lines 2855-2866)

parserItem part of the private builder frontend and editing experience. In this file, it appears around lines 2855-2866 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, and clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2865: return item;.

Important local values include item at line 2856. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2855: function parserItem(title, description = "", url = "", meta = "", kind = "text", rich = null, children =
[] ) {
2856:   const item = {
2857:     children: children.filter((child) => child && (child.title || child.description || child.url ||
child.children?.length)),
2858:     description: description || "",
2859:     kind,
2860:     meta: meta || "",
2861:     title: title || "Untitled item",
2862:     url: url || ""
2863:   };
2864:   if (rich?.blocks?.length) item.rich = clone(rich);
2865:   return item;
2866: }

```

canonicalTemplateId (template-preview.js, lines 2868-2870)

canonicalTemplateId part of the private builder frontend and editing experience. In this file, it appears around lines 2868-2870 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2869: return legacyTemplateSkins[id] || id || "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2868: function canonicalTemplateId(id) {
2869:   return legacyTemplateSkins[id] || id || "";
2870: }

```

projectTemplateId (template-preview.js, lines 2872-2874)

projectTemplateId part of the private builder frontend and editing experience. In this file, it appears around lines 2872-2874 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references canonicalTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2873: return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2872: function projectTemplateId(project) {
2873:   return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");
2874: }

```

projectTemplateFor (template-preview.js, lines 2876-2879)

projectTemplateFor part of the private builder frontend and editing experience. In this file, it appears around lines 2876-2879 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId, and find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2878: return templates.find((template) => template.id === templateId) || {};

Important local values include templateId at line 2877. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2876: function projectTemplateFor(project) {
2877:   const templateId = projectTemplateId(project);
2878:   return templates.find((template) => template.id === templateId) || {};
2879: }
```

projectTemplateClass (template-preview.js, lines 2881-2884)

projectTemplateClass part of the private builder frontend and editing experience. In this file, it appears around lines 2881-2884 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2883: return templateId ? `project-template project-template-\${templateId}` : "project-template-white";.

Important local values include templateId at line 2882. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2881: function projectTemplateClass(project) {
2882:   const templateId = projectTemplateId(project);
2883:   return templateId ? `project-template project-template-${templateId}` : "project-template-white";
2884: }
```

responsiveClassName (template-preview.js, lines 2886-2891)

responsiveClassName part of the private builder frontend and editing experience. In this file, it appears around lines 2886-2891 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2887: return String(value || fallback || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2886: function responsiveClassName(value, fallback) {
2887:   return String(value || fallback || "")
2888:     .toLowerCase()
2889:     .replace(/^[a-z0-9]+/g, "-")
2890:     .replace(/^-|-$/g, "") || fallback;
2891: }
```

projectResponsiveClass (template-preview.js, lines 2893-2896)

projectResponsiveClass part of the private builder frontend and editing experience. In this file, it appears around lines 2893-2896 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectResponsiveProfile, and responsiveClassName. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2895: return `responsive-project responsive-density-\${responsiveClassName(profile.density, "balanced")} responsive-card-\${responsiveClassName(profile.cardLayout, "single")}`;.

Important local values include profile at line 2894. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2893: function projectResponsiveClass(project) {
2894:   const profile = projectResponsiveProfile(project);
2895:   return `responsive-project responsive-density-${responsiveClassName(profile.density, "balanced")}
responsive-card-${responsiveClassName(profile.cardLayout, "single")}`;
2896: }
```

responsiveStyleValues (template-preview.js, lines 2898-2906)

responsiveStyleValues part of the private builder frontend and editing experience. In this file, it appears around lines 2898-2906 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectResponsiveProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2900: return {.

Important local values include profile at line 2899. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2898: function responsiveStyleValues(project) {
2899:   const profile = projectResponsiveProfile(project);
2900:   return {
2901:     "--responsive-section-card-min": profile.sectionCardMin,
2902:     "--responsive-content-max": profile.contentMax,
2903:     "--responsive-media-max": profile.mediaMax,
2904:     "--responsive-touch-target": profile.touchTarget
2905:   };
2906: }
```

hasPublicTemplate (template-preview.js, lines 2908-2910)

hasPublicTemplate part of the private builder frontend and editing experience. In this file, it appears around lines 2908-2910 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2909: return Boolean(projectTemplateId(project));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2908: function hasPublicTemplate(project) {
2909:   return Boolean(projectTemplateId(project));
2910: }
```

projectTemplateVisual (template-preview.js, lines 2912-2914)

projectTemplateVisual part of the private builder frontend and editing experience. In this file, it appears around lines 2912-2914 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateFor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2913: return project?.portfolioView?.template?.visual || project?.templateVisual || projectTemplateFor(project).visual || null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2912: function projectTemplateVisual(project) {
2913:   return project?.portfolioView?.template?.visual || project?.templateVisual ||
projectTemplateFor(project).visual || null;
2914: }
```

projectTemplateStyle (template-preview.js, lines 2916-2934)

projectTemplateStyle part of the private builder frontend and editing experience. In this file, it appears around lines 2916-2934 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateVisual, responsiveStyleValues, entries, filter, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2930: return Object.entries(values).

Important local values include visual at line 2917; values at line 2918. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2916: function projectTemplateStyle(project, accent) {
2917:   const visual = projectTemplateVisual(project) || {};
2918:   const values = {
2919:     "--accent": accent,
2920:     "--template-bg": visual.background,
2921:     "--template-panel": visual.panel,
2922:     "--template-accent": visual.accent,
2923:     "--template-hover": visual.hover,
2924:     "--template-click": visual.click,
2925:     "--template-click-text": visual.clickText,
2926:     "--template-line": visual.line,
2927:     "--template-text": visual.text,
2928:     ...responsiveStyleValues(project)
2929:   };
2930:   return Object.entries(values)
2931:     .filter(([ , value]) => value)
2932:     .map(([key, value]) => `${key}: ${value}`)
2933:     .join("; ");
2934: }
```

applyProjectTemplateToElement (template-preview.js, lines 2936-2970)

applyProjectTemplateToElement part of the private builder frontend and editing experience. In this file, it appears around lines 2936-2970 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, startsWith, forEach, remove, projectTemplateClass, split, add, projectResponsiveClass, projectTemplateVisual, responsiveStyleValues, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2937: if (!element) return;.

Important local values include visual at line 2950; values at line 2951. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2936: function applyProjectTemplateToElement(element, project, accent) {
2937:   if (!element) return;
2938:   [...element.classList]
2939:     .filter((className) =>
2940:       className === "project-template" ||
2941:       className === "project-template-white" ||
2942:       className === "responsive-project" ||
2943:       className.startsWith("project-template-") ||
2944:       className.startsWith("responsive-density-") ||
```



```

2945:     className.startsWith("responsive-card-")
2946:   )
2947:   .forEach((className) => element.classList.remove(className));
2948:   projectTemplateClass(project).split(" ").forEach((className) => element.classList.add(className));
2949:   projectResponsiveClass(project).split(" ").forEach((className) => element.classList.add(className));
2950:   const visual = projectTemplateVisual(project) || {};
2951:   const values = {
2952:     "--accent": accent,
2953:     "--template-bg": visual.background,
2954:     "--template-panel": visual.panel,
2955:     "--template-accent": visual.accent,
2956:     "--template-hover": visual.hover,
2957:     "--template-click": visual.click,
2958:     "--template-click-text": visual.clickText,
2959:     "--template-line": visual.line,
2960:     "--template-text": visual.text,
2961:     ...responsiveStyleValues(project)
2962:   };
2963:   Object.entries(values).forEach(([key, value]) => {
2964:     if (value) {
2965:       element.style.setProperty(key, value);
2966:     } else {
2967:       element.style.removeProperty(key);
2968:     }
2969:   });
2970: }

```

resourcesFrom (template-preview.js, lines 2972-2988)

resourcesFrom part of the private builder frontend and editing experience. In this file, it appears around lines 2972-2988 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and parserItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 2973: return (project[key] || []).map((item) => {; line 2975: return parserItem(item.name, item.result || item.method, item.artifact, item.method || item.status || "Test", "test", item.richDescription);; line 2978: return parserItem(item.name, item.revision || item.status, item.artifact, item.status || "PCB", "pcb", item.richDescription);; line 2981: return parserItem(item.title, item.caption, item.url, item.status || "Image", "image", item.richDescription);. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2972: function resourcesFrom(project, key) {
2973:   return (project[key] || []).map((item) => {
2974:     if (key === "tests") {
2975:       return parserItem(item.name, item.result || item.method, item.artifact, item.method || item.status
|| "Test", "test", item.richDescription);
2976:     }
2977:     if (key === "pcbs") {
2978:       return parserItem(item.name, item.revision || item.status, item.artifact, item.status || "PCB",
"pcb", item.richDescription);
2979:     }
2980:     if (key === "media") {
2981:       return parserItem(item.title, item.caption, item.url, item.status || "Image", "image",
item.richDescription);
2982:     }
2983:     if (key === "links") {
2984:       return parserItem(item.label, item.description || "", item.url, "Link", "link",
item.richDescription);
2985:     }
2986:     return parserItem(item.title || item.name || item.label, item.description || item.caption, item.url
|| item.artifact, item.type || item.status || "Document", "document", item.richDescription);
2987:   });
2988: }

```

resourcesFromItems (template-preview.js, lines 2990-3003)

resourcesFromItems part of the private builder frontend and editing experience. In this file, it appears around lines 2990-3003 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and parserItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2991: return (items || []).map((item) => {; line 2994: return parserItem(.

Important local values include url at line 2992; kind at line 2993. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2990: function resourcesFromItems(items = [], fallbackKind = "document") {
2991:   return (items || []).map((item) => {
2992:     const url = item.url || item.artifact || "";
2993:     const kind = item.kind || fallbackKind;
2994:     return parserItem(
2995:       item.title || item.name || item.label || "Untitled item",
2996:       item.description || item.caption || item.result || item.method || "",
2997:       url,
2998:       item.type || item.status || item.method || "",
2999:       kind,
3000:       item.richDescription
3001:     );
3002:   });
3003: }
```

parsedSubsection (template-preview.js, lines 3005-3007)

parsedSubsection part of the private builder frontend and editing experience. In this file, it appears around lines 3005-3007 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parserItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3006: return parserItem(title, description, "", "Subsection", "subsection", rich, items);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3005: function parsedSubsection(title, description = "", items = [], rich = null) {
3006:   return parserItem(title, description, "", "Subsection", "subsection", rich, items);
3007: }
```

richHasContent (template-preview.js, lines 3009-3018)

richHasContent part of the private builder frontend and editing experience. In this file, it appears around lines 3009-3018 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3010: return Boolean(rich?.blocks?.some((block) =>.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3009: function richHasContent(rich) {
3010:   return Boolean(rich?.blocks?.some((block) =>
3011:     block.type === "image" && block.url ||
3012:     block.type === "formula" && block.formula ||
3013:     block.type === "code" && block.code ||
3014:     block.text ||
3015:     block.title ||
3016:     block.caption
3017:   ));
3018: }
```

parsedItemHasContent (template-preview.js, lines 3020-3031)

parsedItemHasContent part of the private builder frontend and editing experience. In this file, it appears around lines 3020-3031 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `richHasContent`, `some`, and `includes`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 3021: `if (!item) return false;;` line 3022: `if (item.kind === "summary") return Boolean(item.description || richHasContent(item.rich));` line 3025: `return Boolean(item.description || item.url || richHasContent(item.rich) || children.some(parsedItemHasContent));` line 3028: `return Boolean(item.title || item.description || item.url || richHasContent(item.rich) || children.some(parsedItemHasContent));`. There are 1 additional return statements in the function body.

Important local values include `children` at line 3023. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3020: function parsedItemHasContent(item) {
3021:   if (!item) return false;
3022:   if (item.kind === "summary") return Boolean(item.description || richHasContent(item.rich));
3023:   const children = item.children || item.items || [];
3024:   if (item.kind === "subsection") {
3025:     return Boolean(item.description || item.url || richHasContent(item.rich) ||
children.some(parsedItemHasContent));
3026:   }
3027:   if (["tool", "language"].includes(item.kind)) {
3028:     return Boolean(item.title || item.description || item.url || richHasContent(item.rich) ||
children.some(parsedItemHasContent));
3029:   }
3030:   return Boolean(item.description || item.url || richHasContent(item.rich) ||
children.some(parsedItemHasContent));
3031: }
```

`parsedSectionHasContent (template-preview.js, lines 3033-3035)`

`parsedSectionHasContent` part of the private builder frontend and editing experience. In this file, it appears around lines 3033-3035 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `richHasContent`, and `some`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3034: `return Boolean(section?.description || richHasContent(section?.rich) || (section?.items || []).some(parsedItemHasContent));`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3033: function parsedSectionHasContent(section) {
3034:   return Boolean(section?.description || richHasContent(section?.rich) || (section?.items ||
[]).some(parsedItemHasContent));
3035: }
```

`responsiveChildren (template-preview.js, lines 3037-3039)`

`responsiveChildren` part of the private builder frontend and editing experience. In this file, it appears around lines 3037-3039 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3038: `return node?.items || node?.children || [];`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3037: function responsiveChildren(node) {
3038:   return node?.items || node?.children || [];
3039: }
```

`richContainsMedia (template-preview.js, lines 3041-3043)`

richContainsMedia part of the private builder frontend and editing experience. In this file, it appears around lines 3041-3043 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3042: return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3041: function richContainsMedia(rich) {
3042:   return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));
3043: }
```

responsiveNodeHasMedia (template-preview.js, lines 3045-3051)

responsiveNodeHasMedia part of the private builder frontend and editing experience. In this file, it appears around lines 3045-3051 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richContainsMedia, responsiveChildren, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3046: if (!node) return false;; line 3047: if (node.kind === "image" || richContainsMedia(node.rich)) return true;; line 3049: if (!/(apng|avif|gif|jpe?g|png|svg|webp)(\?|#|\$)/i.test(url)) return true;; line 3050: return responsiveChildren(node).some(responsiveNodeHasMedia);.

Important local values include url at line 3048. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3045: function responsiveNodeHasMedia(node) {
3046:   if (!node) return false;
3047:   if (node.kind === "image" || richContainsMedia(node.rich)) return true;
3048:   const url = String(node.url || "");
3049:   if (!/(apng|avif|gif|jpe?g|png|svg|webp)(\?|#|$)/i.test(url)) return true;
3050:   return responsiveChildren(node).some(responsiveNodeHasMedia);
3051: }
```

responsiveNodeCount (template-preview.js, lines 3053-3056)

responsiveNodeCount part of the private builder frontend and editing experience. In this file, it appears around lines 3053-3056 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parsedItemHasContent, responsiveChildren, and reduce. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3054: if (!node || !parsedItemHasContent(node)) return 0;; line 3055: return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3053: function responsiveNodeCount(node) {
3054:   if (!node || !parsedItemHasContent(node)) return 0;
3055:   return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);
3056: }
```

responsiveNodeDepth (template-preview.js, lines 3058-3063)

responsiveNodeDepth part of the private builder frontend and editing experience. In this file, it appears around lines 3058-3063 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parsedItemHasContent, responsiveChildren, filter, max, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3059: if (!node || !parsedItemHasContent(node)) return 0;; line 3061: if (!children.length) return 1;; line 3062: return 1 + Math.max(...children.map(responsiveNodeDepth));.

Important local values include children at line 3060. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3058: function responsiveNodeDepth(node) {
3059:   if (!node || !parsedItemHasContent(node)) return 0;
3060:   const children = responsiveChildren(node).filter(parsedItemHasContent);
3061:   if (!children.length) return 1;
3062:   return 1 + Math.max(...children.map(responsiveNodeDepth));
3063: }
```

buildResponsiveProfile (template-preview.js, lines 3065-3096)

buildResponsiveProfile creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 3065-3096 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, parsedSectionHasContent, reduce, responsiveNodeCount, max, map, some, responsiveNodeHasMedia, and hasPublicTemplate. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3081: return {.

Important local values include visibleSections at line 3066; itemCount at line 3067; maxDepth at line 3071; hasMedia at line 3075; density at line 3076. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3065: function buildResponsiveProfile(project, parsedSections = []) {
3066:   const visibleSections = (parsedSections || []).filter((section) => section?.id !== "brief" &&
parsedSectionHasContent(section));
3067:   const itemCount = visibleSections.reduce(
3068:     (count, section) => count + (section.items || []).reduce((sum, item) => sum +
responsiveNodeCount(item), 0),
3069:     0
3070:   );
3071:   const maxDepth = visibleSections.reduce(
3072:     (depth, section) => Math.max(depth, ...(section.items || []).map(responsiveNodeDepth), 1),
3073:     1
3074:   );
3075:   const hasMedia = visibleSections.some((section) => responsiveNodeHasMedia(section));
3076:   const density = itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4
3077:     ? "dense"
3078:     : itemCount <= 6 && visibleSections.length <= 2
3079:     ? "simple"
3080:     : "balanced";
3081:   return {
3082:     version: 1,
3083:     breakpoints: {
3084:       mobile: 640,
3085:       tablet: 900,
3086:       desktop: 1180
3087:     },
3088:     cardLayout: hasPublicTemplate(project) ? "single" : "split",
3089:     contentMax: hasMedia || maxDepth >= 3 ? "1180px" : "980px",
3090:     density,
3091:     mediaMax: hasMedia ? "860px" : "720px",
3092:     sectionCardMin: density === "dense" ? "210px" : density === "simple" ? "280px" : "240px",
3093:     touchTarget: "44px",
3094:     windowMode: "full-screen"
3095:   };
3096: }
```

visibleSections (template-preview.js, lines 3066-3095)

visibleSections part of the private builder frontend and editing experience. In this file, it appears around lines 3066-3095 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, parsedSectionHasContent, reduce, responsiveNodeCount, max, map, some, responsiveNodeHasMedia, and hasPublicTemplate. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3081: return {.

Important local values include visibleSections at line 3066; itemCount at line 3067; maxDepth at line 3071; hasMedia at line 3075; density at line 3076. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3066: const visibleSections = (parsedSections || []).filter((section) => section?.id !== "brief" &&
parsedSectionHasContent(section));
3067: const itemCount = visibleSections.reduce(
3068:   (count, section) => count + (section.items || []).reduce((sum, item) => sum +
responsiveNodeCount(item), 0),
3069:   0
3070: );
3071: const maxDepth = visibleSections.reduce(
3072:   (depth, section) => Math.max(depth, ...(section.items || []).map(responsiveNodeDepth), 1),
3073:   1
3074: );
3075: const hasMedia = visibleSections.some((section) => responsiveNodeHasMedia(section));
3076: const density = itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4
3077:   ? "dense"
3078:   : itemCount <= 6 && visibleSections.length <= 2
3079:   ? "simple"
3080:   : "balanced";
3081: return {
3082:   version: 1,
3083:   breakpoints: {
3084:     mobile: 640,
3085:     tablet: 900,
3086:     desktop: 1180
3087:   },
3088:   cardLayout: hasPublicTemplate(project) ? "single" : "split",
3089:   contentMax: hasMedia || maxDepth >= 3 ? "1180px" : "980px",
3090:   density,
3091:   mediaMax: hasMedia ? "860px" : "720px",
3092:   sectionCardMin: density === "dense" ? "210px" : density === "simple" ? "280px" : "240px",
3093:   touchTarget: "44px",
3094:   windowMode: "full-screen"
3095: };
```

projectResponsiveProfile (template-preview.js, lines 3098-3107)

projectResponsiveProfile part of the private builder frontend and editing experience. In this file, it appears around lines 3098-3107 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references buildResponsiveProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3101: return {; line 3106: return buildResponsiveProfile(project, project?.portfolioView?.sections || []).

Important local values include savedProfile at line 3099. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3098: function projectResponsiveProfile(project) {
3099:   const savedProfile = project?.portfolioView?.responsive;
3100:   if (savedProfile?.version) {
3101:     return {
3102:       ...buildResponsiveProfile(project, project?.portfolioView?.sections || []),
3103:       ...savedProfile
3104:     };
3105:   }
3106:   return buildResponsiveProfile(project, project?.portfolioView?.sections || []);
3107: }
```

parsedSection (template-preview.js, lines 3109-3118)

parsedSection part of the private builder frontend and editing experience. In this file, it appears around lines 3109-3118 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, richHasContent, and clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3117: return section;.

Important local values include section at line 3110. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3109: function parsedSection(id, title, items = [], description = "", rich = null) {
3110:   const section = {
3111:     description,
3112:     id,
3113:     items: items.filter(parsedItemHasContent),
3114:     title
3115:   };
3116:   if (richHasContent(rich)) section.rich = clone(rich);
3117:   return section;
3118: }
```

parsedDesignSection (template-preview.js, lines 3120-3146)

parsedDesignSection part of the private builder frontend and editing experience. In this file, it appears around lines 3120-3146 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isAnalogProject, ensureDesignModel, resourcesFrom, resourcesFromItems, parsedSubsection, and parsedSection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3133: return parsedSection("design", "Design", [; line 3141: return parsedSection("design", "Design", [.

Important local values include design at line 3122; documentationItems at line 3123; boardMediaItems at line 3129. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3120: function parsedDesignSection(project) {
3121:   if (isAnalogProject(project)) {
3122:     const design = ensureDesignModel(project);
3123:     const documentationItems = [
3124:       ...resourcesFrom(project, "documents"),
3125:       ...resourcesFromItems(design.documentation.files, "document"),
3126:       parsedSubsection("References", "", resourcesFromItems(design.documentation.references,
"reference")),
3127:       parsedSubsection("Math / Analysis", "", resourcesFromItems(design.documentation.mathAnalysis,
"analysis"))
3128:     ];
3129:     const boardMediaItems = [
3130:       ...resourcesFrom(project, "pcbs"),
3131:       ...resourcesFrom(project, "media")
3132:     ];
3133:     return parsedSection("design", "Design", [
3134:       parsedSubsection("Design Overview", design.brief.summary || "", [
3135:         ...resourcesFromItems(design.brief.files, "document")
3136:       ], design.brief.summaryRich),
3137:       parsedSubsection("Documentation", design.documentation.summary || "", documentationItems,
design.documentation.summaryRich),
3138:       parsedSubsection("PCB and Visual Evidence", "", boardMediaItems)
3139:     ]);
3140:   }
3141:   return parsedSection("design", "Design", [
3142:     ...resourcesFrom(project, "documents"),
3143:     ...resourcesFrom(project, "pcbs"),
3144:     ...resourcesFrom(project, "media")
3145:   ]);
3146: }
```

parsedSimulationSection (template-preview.js, lines 3148-3156)

parsedSimulationSection part of the private builder frontend and editing experience. In this file, it appears around lines 3148-3156 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isAnalogProject, ensureDesignModel, parsedSection, parsedSubsection, and resourcesFromItems. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3149: if (!isAnalogProject(project)) return null;; line 3151: return parsedSection("simulation", "Simulation", [.

Important local values include design at line 3150. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3148: function parsedSimulationSection(project) {
3149:   if (!isAnalogProject(project)) return null;
3150:   const design = ensureDesignModel(project);
3151:   return parsedSection("simulation", "Simulation", [
3152:     parsedSubsection("Simulation Overview", design.simulation.summary || "", [],
design.simulation.summaryRich),
3153:     parsedSubsection("Simulation Files", "", resourcesFromItems(design.simulation.files, "simulation-
file")),
3154:     parsedSubsection("Simulation Results", "", resourcesFromItems(design.simulation.results, "simulation-
result"))
3155:   ]);
3156: }
```

parsedToolsSection (template-preview.js, lines 3158-3169)

parsedToolsSection part of the private builder frontend and editing experience. In this file, it appears around lines 3158-3169 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, parserItem, toolName, toolDescription, itemTitle, parsedSection, and resourcesFrom. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3162: return parserItem(label, "", "", "Language", "language");; line 3164: return parsedSection("tools", "Tools", [.

Important local values include toolItems at line 3159; languageItems at line 3160; label at line 3161. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3158: function parsedToolsSection(project) {
3159:   const toolItems = (project.tools || []).map((tool) => parserItem(toolName(tool), toolDescription(tool),
"", "Tool", "tool", tool?.richDescription));
3160:   const languageItems = (project.languages || []).map((language) => {
3161:     const label = typeof language === "string" ? language : itemTitle(language);
3162:     return parserItem(label, "", "", "Language", "language");
3163:   });
3164:   return parsedSection("tools", "Tools", [
3165:     ...toolItems,
3166:     ...languageItems,
3167:     ...resourcesFrom(project, "links")
3168:   ]);
3169: }
```

toolItems (template-preview.js, lines 3159-3163)

toolItems part of the private builder frontend and editing experience. In this file, it appears around lines 3159-3163 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, parserItem, toolName, toolDescription, and itemTitle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3162: return parserItem(label, "", "", "Language", "language");.

Important local values include toolItems at line 3159; languageItems at line 3160; label at line 3161. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:


```

3159:   const toolItems = (project.tools || []).map((tool) => parserItem(toolName(tool), toolDescription(tool),
    "", "Tool", "tool", tool?.richDescription));
3160:   const languageItems = (project.languages || []).map((language) => {
3161:     const label = typeof language === "string" ? language : itemTitle(language);
3162:     return parserItem(label, "", "", "Language", "language");
3163:   });

```

languageItems (template-preview.js, lines 3160-3163)

languageItems part of the private builder frontend and editing experience. In this file, it appears around lines 3160-3163 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, itemTitle, and parserItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3162: return parserItem(label, "", "", "Language", "language");.

Important local values include languageItems at line 3160; label at line 3161. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3160:   const languageItems = (project.languages || []).map((language) => {
3161:     const label = typeof language === "string" ? language : itemTitle(language);
3162:     return parserItem(label, "", "", "Language", "language");
3163:   });

```

parseProjectForPortfolio (template-preview.js, lines 3171-3219)

This function converts editable project state into the clean public project object used by previews and the website. It filters empty sections, normalizes overview content, applies appearance templates, gathers resources, and builds responsive profile metadata.

The builder data is messy because it contains drafts, controls, and editor state. The public website needs a clean display model. This parser is that boundary.

It calls or references projectTemplateFor, sectionOptions, flatMap, parsedSection, parserItem, parsedDesignSection, parsedSimulationSection, resourcesFrom, parsedToolsSection, startsWith, and 11 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 8 return statement(s). The most important visible returns are: line 3175: return parsedSection("brief", "Overview", [; line 3179: if (section.id === "design") return parsedDesignSection(project);; line 3180: if (section.id === "simulation") return parsedSimulationSection(project);; line 3181: if (section.id === "tests") return parsedSection("tests", "Tests", resourcesFrom(project, "tests"));. There are 4 additional return statements in the function body.

Important local values include template at line 3172; parsedSections at line 3173; sectionId at line 3184; custom at line 3185. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3171: function parseProjectForPortfolio(project) {
3172:   const template = projectTemplateFor(project);
3173:   const parsedSections = sectionOptions(project).flatMap((section) => {
3174:     if (section.id === "brief") {
3175:       return parsedSection("brief", "Overview", [
3176:         parserItem("Overview", project.summary || "", "", "Overview", "summary", project.summaryRich)
3177:       ]);
3178:     }
3179:     if (section.id === "design") return parsedDesignSection(project);
3180:     if (section.id === "simulation") return parsedSimulationSection(project);
3181:     if (section.id === "tests") return parsedSection("tests", "Tests", resourcesFrom(project, "tests"));
3182:     if (section.id === "tools") return parsedToolsSection(project);
3183:     if (section.id.startsWith("custom:")) {
3184:       const sectionId = section.id.replace("custom:", "");
3185:       const custom = (project.sections || []).find((item) => item.id === sectionId);
3186:       return parsedSection(
3187:         `custom:${sectionId}`,
3188:         custom?.title || section.label,
3189:         (custom?.items || []).map((item) => item.url
3190:           ? parserItem(item.title, item.description, item.url, item.type || item.status || "File",
"file", item.richDescription)
3191:           : parsedSubsection(
3192:             item.title,
3193:             item.description,
3194:             resourcesFromItems(item.children || item.items || item.files || [], "file"),
3195:             item.richDescription
3196:           )),
3197:         custom?.description || "",
3198:         custom?.richDescription

```

```

3199:     });
3200:   }
3201:   return null;
3202: }).filter(parsedSectionHasContent);
3203:
3204: return {
3205:   ...clone(project),
3206:   portfolioView: {
3207:     builtAt: new Date().toISOString(),
3208:     template: {
3209:       group: "Appearance",
3210:       id: projectTemplateId(project),
3211:       label: template.label || project.templateLabel || "",
3212:       visual: template.visual ? clone(template.visual) : projectTemplateVisual(project)
3213:     },
3214:     responsive: buildResponsiveProfile(project, parsedSections),
3215:     title: project.title,
3216:     sections: parsedSections
3217:   }
3218: };
3219: }

```

custom (template-preview.js, lines 3185-3187)

custom part of the private builder frontend and editing experience. In this file, it appears around lines 3185-3187 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and parsedSection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3186: return parsedSection(.

Important local values include custom at line 3185. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3185:     const custom = (project.sections || []).find((item) => item.id === sectionId);
3186:     return parsedSection(
3187:       `custom:${sectionId}`,

```

refreshOpenPreviews (template-preview.js, lines 3221-3230)

refreshOpenPreviews part of the private builder frontend and editing experience. In this file, it appears around lines 3221-3230 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, find, renderProjectWebsitePreview, and renderPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include project at line 3223; savedProject at line 3224. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3221: function refreshOpenPreviews() {
3222:   if (projectPreviewDialog?.open) {
3223:     const project = selectedProject();
3224:     const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3225:     renderProjectWebsitePreview(savedProject || project);
3226:   }
3227:   if (portfolioPreviewDialog?.open) {
3228:     renderPreview();
3229:   }
3230: }

```

savedProject (template-preview.js, lines 3224-3227)

savedProject persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 3224-3227 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and renderProjectWebsitePreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include savedProject at line 3224. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3224:     const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3225:     renderProjectWebsitePreview(savedProject || project);
3226:   }
3227:   if (portfolioPreviewDialog?.open) {
```

saveSelectedProjectAndClose (template-preview.js, lines 3232-3236)

saveSelectedProjectAndClose persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 3232-3236 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveSelectedProjectToPortfolio, and closeDialogElement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3232: function saveSelectedProjectAndClose() {
3233:   if (saveSelectedProjectToPortfolio()) {
3234:     closeDialogElement(projectDialog);
3235:   }
3236: }
```

commitActiveProjectEdits (template-preview.js, lines 3238-3246)

commitActiveProjectEdits part of the private builder frontend and editing experience. In this file, it appears around lines 3238-3246 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveVisibleRichEditors, and saveRichSummary. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3242: return saveRichSummary(); line 3245: return true;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3238: async function commitActiveProjectEdits() {
3239:   saveVisibleRichEditors();
3240:
3241:   if (summaryEditorProjectId) {
3242:     return saveRichSummary();
3243:   }
3244:
3245:   return true;
3246: }
```

saveAllSections (template-preview.js, lines 3248-3259)

saveAllSections persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 3248-3259 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references commitActiveProjectEdits, syncParsedPortfolioGlobalsIntoSaved, map, parseProjectForPortfolio, markDraftNeedsSave, closeDialogElement, refreshOpenPreviews, setStatus, and updateBuilderWorkflow. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3250: if (!(await commitActiveProjectEdits())) return false;; line 3258: return true;.

Important local values include settings at line 3249. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3248: async function saveAllSections(options = {}) {
3249:   const settings = options && options.currentTarget ? {} : options;
3250:   if (!(await commitActiveProjectEdits())) return false;
3251:   syncParsedPortfolioGlobalsIntoSaved();
3252:   savedPortfolioCatalog.projects = (catalog.projects || []).map((project) =>
parseProjectForPortfolio(project));
3253:   markDraftNeedsSave();
3254:   if (projectDialog?.open) closeDialogElement(projectDialog);
3255:   if (settings.refreshOpenPreviews !== false) refreshOpenPreviews();
3256:   setStatus('Saved ${savedPortfolioCatalog.projects.length}
project${savedPortfolioCatalog.projects.length === 1 ? "" : "s"} into the portfolio preview.`);
3257:   updateBuilderWorkflow();
3258:   return true;
3259: }
```

selectedProject (template-preview.js, lines 3261-3263)

selectedProject part of the private builder frontend and editing experience. In this file, it appears around lines 3261-3263 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3262: return catalog.projects.find((project) => project.id === selectedProjectId) || catalog.projects[0];.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3261: function selectedProject() {
3262:   return catalog.projects.find((project) => project.id === selectedProjectId) || catalog.projects[0];
3263: }
```

sectionWindowId (template-preview.js, lines 3265-3269)

sectionWindowId part of the private builder frontend and editing experience. In this file, it appears around lines 3265-3269 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3266: if (["documents", "pcbs", "media"].includes(sectionId)) return "design";; line 3267: if (["links", "highlights", "tools", "languages"].includes(sectionId)) return "tools";; line 3268: return sectionId || "brief";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3265: function sectionWindowId(sectionId) {
3266:   if ([ "documents", "pcbs", "media" ].includes(sectionId)) return "design";
3267:   if ([ "links", "highlights", "tools", "languages" ].includes(sectionId)) return "tools";
3268:   return sectionId || "brief";
3269: }
```

resetProjectWindowScroll (template-preview.js, lines 3271-3279)

resetProjectWindowScroll part of the private builder frontend and editing experience. In this file, it appears around lines 3271-3279 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `requestAnimationFrame`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3271: function resetProjectWindowScroll() {
3272:   projectWindowContent.scrollTop = 0;
3273:   requestAnimationFrame(() => {
3274:     projectWindowContent.scrollTop = 0;
3275:     requestAnimationFrame(() => {
3276:       projectWindowContent.scrollTop = 0;
3277:     });
3278:   });
3279: }
```

openProjectWindow (template-preview.js, lines 3281-3302)

`openProjectWindow` controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3281-3302 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `sectionWindowId`, `removeAttribute`, `remove`, `renderAll`, `add`, `showModal`, `resetProjectWindowScroll`, and `updateDialogWindowButtons`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3281: function openProjectWindow(projectId, sectionId = "brief") {
3282:   selectedProjectId = projectId;
3283:   projectWindowTitle.dataset.projectId = projectId;
3284:   activeSectionId = sectionWindowId(sectionId);
3285:   projectWindowBackStack = [];
3286:   projectWindowForwardStack = [];
3287:   projectTitleMenu.hidden = true;
3288:   projectDialog.removeAttribute("style");
3289:   projectDialog.classList.remove(
3290:     "is-draggable-dialog",
3291:     "is-hidden-dialog",
3292:     "is-maximized-dialog",
3293:     "is-minimized-dialog",
3294:     "is-resized-dialog"
3295:   );
3296:   delete projectDialog.dataset.restoreWindowState;
3297:   renderAll();
3298:   document.body.classList.add("project-window-open");
3299:   if (!projectDialog.open) projectDialog.showModal();
3300:   resetProjectWindowScroll();
3301:   updateDialogWindowButtons(projectDialog);
3302: }
```

categoryByld (template-preview.js, lines 3304-3306)

`categoryByld` part of the private builder frontend and editing experience. In this file, it appears around lines 3304-3306 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `find`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3305: `return catalog.categories.find((category) => category.id === id) || {};`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3304: function categoryById(id) {
3305:   return catalog.categories.find((category) => category.id === id) || {};
3306: }
```

normalizeCategory (template-preview.js, lines 3308-3318)

normalizeCategory validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 3308-3318 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, slugify, and normalizeTextColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3312: return {.

Important local values include label at line 3309; id at line 3310; accent at line 3311. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3308: function normalizeCategory(category = {}) {
3309:   const label = String(category.label || category.title || "New category").trim() || "New category";
3310:   const id = slugify(category.id || label) || "category";
3311:   const accent = normalizeTextColor(category.accent || "") || "#1677a8";
3312:   return {
3313:     accent,
3314:     description: String(category.description || "").trim(),
3315:     id,
3316:     label
3317:   };
3318: }
```

refreshCategoryControls (template-preview.js, lines 3320-3327)

refreshCategoryControls part of the private builder frontend and editing experience. In this file, it appears around lines 3320-3327 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, clone, escapeHtml, join, and renderCategoryDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3320: function refreshCategoryControls() {
3321:   catalog.categories = (catalog.categories || []).map(normalizeCategory);
3322:   savedPortfolioCatalog.categories = clone(catalog.categories || []);
3323:   if (newCategorySelect) {
3324:     newCategorySelect.innerHTML = catalog.categories.map((category) => `
```

linkAttributes (template-preview.js, lines 3329-3332)

linkAttributes part of the private builder frontend and editing experience. In this file, it appears around lines 3329-3332 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, and isWebsiteLinkItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3331: return /^https?:\V/i.test(target) ? ' target="_blank" rel="noreferrer"' : "";

Important local values include target at line 3330. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3329: function linkAttributes(url, item = {}) {
3330:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(item, url) });
```

```
3331:   return /^https?:\/\/i.test(target) ? ' target="_blank" rel="noreferrer" : "";
3332: }
```

isLocalDownloadTarget (template-preview.js, lines 3334-3337)

isLocalDownloadTarget part of the private builder frontend and editing experience. In this file, it appears around lines 3334-3337 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3336: return Boolean(value) && !/^(https?:)?VV/i.test(value) && !/^(mailto:|tel:|#)/i.test(value);.

Important local values include value at line 3335. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3334: function isLocalDownloadTarget(target = "") {
3335:   const value = normalizeLinkTarget(target, { assumeWeb: true });
3336:   return Boolean(value) && !/^(https?:)?VV/i.test(value) && !/^(mailto:|tel:|#)/i.test(value);
3337: }
```

downloadAttribute (template-preview.js, lines 3339-3342)

downloadAttribute part of the private builder frontend and editing experience. In this file, it appears around lines 3339-3342 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, isWebsiteLinkItem, and isLocalDownloadTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3341: return isLocalDownloadTarget(value) ? " download" : "";

Important local values include value at line 3340. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3339: function downloadAttribute(target = "", item = {}) {
3340:   const value = normalizeLinkTarget(target, { assumeWeb: isWebsiteLinkItem(item, target) });
3341:   return isLocalDownloadTarget(value) ? " download" : "";
3342: }
```

resourceLink (template-preview.js, lines 3344-3353)

resourceLink part of the private builder frontend and editing experience. In this file, it appears around lines 3344-3353 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, isWebsiteLinkItem, escapeHtml, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3349: return `

Important local values include rawTarget at line 3345; target at line 3346. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3344: function resourceLink(item, label = item.label || item.title || item.name) {
3345:   const rawTarget = item.url || item.artifact || item.href || item.file || item.path || item.src || "";
3346:   const target = normalizeLinkTarget(rawTarget, { assumeWeb: isWebsiteLinkItem(item, rawTarget) });
3347:
3348:   if (!target || item.status === "planned") {
3349:     return `
```

```

3351:
3352:   return `<a class="resource-link" href="${escapeHtml(target)}"${linkAttributes(target,
item)}${downloadAttribute(target, item)}>${escapeHtml(label)}</a>`;
3353: }

```

pillList (template-preview.js, lines 3355-3360)

pillList part of the private builder frontend and editing experience. In this file, it appears around lines 3355-3360 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3356: return (items || []).map((item) => {}; line 3358: return `<span class="tag \${className}">\${label}</span>`;

Important local values include label at line 3357. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3355: function pillList(items, className = "") {
3356:   return (items || []).map((item) => {
3357:     const label = typeof item === "string" ? item : item.name || item.title || item.label || "";
3358:     return `<span class="tag ${className}">${label}</span>`;
3359:   }).join("");
3360: }

```

evidenceList (template-preview.js, lines 3362-3365)

evidenceList part of the private builder frontend and editing experience. In this file, it appears around lines 3362-3365 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3363: if (!items || !items.length) return `<p class="evidence-empty">\${emptyMessage}</p>`; line 3364: return `<ul>\${items.map(renderItem).join("")}</ul>`;

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3362: function evidenceList(items, renderItem, emptyMessage) {
3363:   if (!items || !items.length) return `<p class="evidence-empty">${emptyMessage}</p>`;
3364:   return `<ul>${items.map(renderItem).join("")}</ul>`;
3365: }

```

detailBlock (template-preview.js, lines 3367-3374)

detailBlock part of the private builder frontend and editing experience. In this file, it appears around lines 3367-3374 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3368: return `

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3367: function detailBlock(title, className, content) {
3368:   return `
3369:     <details class="${className}">
3370:       <summary>${title}</summary>
3371:       ${content}
3372:     </details>
3373:   `;
3374: }

```


mediaGrid (template-preview.js, lines 3376-3398)

mediaGrid part of the private builder frontend and editing experience. In this file, it appears around lines 3376-3398 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3377: if (!items || !items.length) return `<p class="evidence-empty">No project images have been added yet.</p>`; line 3379: return `

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3376: function mediaGrid(items) {
3377:   if (!items || !items.length) return `<p class="evidence-empty">No project images have been added
yet.</p>`;
3378:
3379:   return `
3380:     <div class="media-grid">
3381:       ${items.map((item) => `
3382:         <figure>
3383:           ${item.status === "planned" ? `
3384:             <div class="media-placeholder">${item.title}</div>
3385:             : `
3386:             <a href="${escapeHtml(normalizeLinkTarget(item.url, { assumeWeb: isWebsiteLinkItem(item,
item.url) })))"${linkAttributes(item.url, item)}>
3387:               
3388:             </a>
3389:           `}
3390:           <figcaption>
3391:             <strong>${escapeHtml(item.title)}</strong>
3392:             <span>${escapeHtml(item.caption || "")}</span>
3393:           </figcaption>
3394:         </figure>
3395:       `).join("")}
3396:     </div>
3397:   `;
3398: }
```

itemTitle (template-preview.js, lines 3400-3403)

itemTitle part of the private builder frontend and editing experience. In this file, it appears around lines 3400-3403 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3401: if (typeof item === "string") return item;; line 3402: return item?.title || item?.name || item?.label || "Untitled item";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3400: function itemTitle(item) {
3401:   if (typeof item === "string") return item;
3402:   return item?.title || item?.name || item?.label || "Untitled item";
3403: }
```

itemUrl (template-preview.js, lines 3405-3407)

itemUrl part of the private builder frontend and editing experience. In this file, it appears around lines 3405-3407 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3406: return item?.url || item?.artifact || "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3405: function itemUrl(item) {  
3406:   return item?.url || item?.artifact || "";  
3407: }
```

itemDescription (template-preview.js, lines 3409-3411)

itemDescription part of the private builder frontend and editing experience. In this file, it appears around lines 3409-3411 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3410: return item?.description || item?.caption || item?.result || item?.method || "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3409: function itemDescription(item) {  
3410:   return item?.description || item?.caption || item?.result || item?.method || "";  
3411: }
```

isImageUrl (template-preview.js, lines 3413-3415)

isImageUrl part of the private builder frontend and editing experience. In this file, it appears around lines 3413-3415 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3414: return /\.(png|jpe?g|webp|gif|svg)(\?.*)?\$/i.test(url || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3413: function isImageUrl(url) {  
3414:   return /\.(png|jpe?g|webp|gif|svg)(\?.*)?$/i.test(url || "");  
3415: }
```

portfolioLink (template-preview.js, lines 3417-3421)

portfolioLink part of the private builder frontend and editing experience. In this file, it appears around lines 3417-3421 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, escapeHtml, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3418: if (!url) return `<span>\${label}</span>`; line 3420: return `<a href="\${escapeHtml(target)}"\${linkAttributes(target)}\${downloadAttribute(target)}>\${escapeHtml(label)}</a>`;

Important local values include target at line 3419. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3417: function portfolioLink(url, label) {  
3418:   if (!url) return `<span>${label}</span>`;  
3419:   const target = normalizeLinkTarget(url, { assumeWeb: true });  
3420:   return `<a  
href="${escapeHtml(target)}"${linkAttributes(target)}${downloadAttribute(target)}>${escapeHtml(label)}</a>`;  
3421: }
```

renderPortfolioResourceList (template-preview.js, lines 3423-3444)

renderPortfolioResourceList renders ui, markup, or display output. In this file, it appears around lines 3423-3444 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, itemTitle, itemUrl, itemDescription, portfolioLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3424: return `; line 3433: return `.

Important local values include titleText at line 3430; url at line 3431; description at line 3432. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3423: function renderPortfolioResourceList(title, items, emptyMessage) {
3424:   return `
3425:     <section class="portfolio-preview-section">
3426:       <h3>${title}</h3>
3427:       ${items || []}.length ? `
3428:         <div class="portfolio-resource-list">
3429:           ${items.map((item) => {
3430:             const titleText = itemTitle(item);
3431:             const url = itemUrl(item);
3432:             const description = itemDescription(item);
3433:             return `
3434:               <article class="portfolio-resource">
3435:                 <strong>${portfolioLink(url, titleText)}</strong>
3436:                 ${description ? `<p>${description}</p>` : ""}
3437:               </article>
3438:             `;
3439:           }).join("")}
3440:         </div>
3441:       : `<p class="evidence-empty">${emptyMessage}</p>`
3442:     </section>
3443:   `;
3444: }
```

renderPortfolioMedia (template-preview.js, lines 3446-3468)

renderPortfolioMedia renders ui, markup, or display output. In this file, it appears around lines 3446-3468 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, itemUrl, isImageUrl, itemTitle, itemDescription, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3447: return `; line 3454: return `.

Important local values include url at line 3453. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3446: function renderPortfolioMedia(items) {
3447:   return `
3448:     <section class="portfolio-preview-section">
3449:       <h3>Images</h3>
3450:       ${items || []}.length ? `
3451:         <div class="portfolio-media-grid">
3452:           ${items.map((item) => {
3453:             const url = itemUrl(item);
3454:             return `
3455:               <figure>
3456:                 ${isImageUrl(url) ? `` : `<div class="media-
placeholder">${itemTitle(item)}</div>`}
3457:                 <figcaption>
3458:                   <strong>${itemTitle(item)}</strong>
3459:                   ${itemDescription(item) ? `<span>${itemDescription(item)}</span>` : ""}
3460:                 </figcaption>
3461:               </figure>
3462:             `;
3463:           }).join("")}
3464:         </div>
3465:       : `<p class="evidence-empty">No images have been added yet.</p>`
3466:     </section>
3467:   `;
3468: }
```

renderPortfolioTools (template-preview.js, lines 3470-3486)

renderPortfolioTools renders ui, markup, or display output. In this file, it appears around lines 3470-3486 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, toolName, toolDescription, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3471: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3470: function renderPortfolioTools(project) {
3471:   return `
3472:     <section class="portfolio-preview-section">
3473:       <h3>Tools</h3>
3474:       ${project.tools || []}.length ? `
3475:         <div class="portfolio-tool-list">
3476:           ${project.tools.map((tool) => `
3477:             <article>
3478:               <strong>${toolName(tool)}</strong>
3479:               ${toolDescription(tool) ? `<p>${toolDescription(tool)}</p>` : ""}
3480:             </article>
3481:           `).join("")}
3482:         </div>
3483:       : `<p class="evidence-empty">No tools have been added yet.</p>`
3484:     </section>
3485:   `;
3486: }
```

renderPortfolioCustomSections (template-preview.js, lines 3488-3505)

renderPortfolioCustomSections renders ui, markup, or display output. In this file, it appears around lines 3488-3505 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, portfolioLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3489: return (project.sections || []).map((section) => `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3488: function renderPortfolioCustomSections(project) {
3489:   return (project.sections || []).map((section) => `
3490:     <section class="portfolio-preview-section">
3491:       <h3>${section.title}</h3>
3492:       ${section.description ? `<p>${section.description}</p>` : ""}
3493:       ${section.items || []}.length ? `
3494:         <div class="portfolio-resource-list">
3495:           ${section.items.map((item) => `
3496:             <article class="portfolio-resource">
3497:               <strong>${portfolioLink(item.url, item.title || "Untitled subsection")}</strong>
3498:               ${item.description ? `<p>${item.description}</p>` : ""}
3499:             </article>
3500:           `).join("")}
3501:         </div>
3502:       : `<p class="evidence-empty">No content has been added yet.</p>`
3503:     </section>
3504:   `).join("");
3505: }
```

publicCustomSectionBlocks (template-preview.js, lines 3507-3518)

publicCustomSectionBlocks part of the private builder frontend and editing experience. In this file, it appears around lines 3507-3518 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, detailBlock, renderMultilineInlineText, evidenceList, resourceLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3508: `return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-wide",``.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3507: function publicCustomSectionBlocks(project) {
3508:   return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-
wide", `
3509:     ${section.description ? `
```

renderProjectPortfolioPreview (template-preview.js, lines 3520-3556)

renderProjectPortfolioPreview renders ui, markup, or display output. In this file, it appears around lines 3520-3556 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, find, filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, renderParsedBriefBlock, map, parsedPublicSection, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3521: `if (!project) return `

line 3527: return `; line 3538: return `.`

Important local values include category at line 3522; accent at line 3523; briefSection at line 3525; otherSections at line 3526. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3520: function renderProjectPortfolioPreview(project) {
3521:   if (!project) return `
```

briefSection (template-preview.js, lines 3525-3528)

briefSection part of the private builder frontend and editing experience. In this file, it appears around lines 3525-3528 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, and projectTemplateStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3527: return `.

Important local values include briefSection at line 3525; otherSections at line 3526. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3525:     const briefSection = (project.portfolioView.sections || []).find((section) => section.id ===
"brief");
3526:     const otherSections = (project.portfolioView.sections || []).filter((section) => section.id !==
"brief" && previewSectionHasRenderableContent(section));
3527:     return `
3528:         <article class="portfolio-preview-article ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" style="${projectTemplateStyle(project, accent)}">
```

otherSections (template-preview.js, lines 3526-3528)

otherSections part of the private builder frontend and editing experience. In this file, it appears around lines 3526-3528 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, and projectTemplateStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3527: return `.

Important local values include otherSections at line 3526. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3526:     const otherSections = (project.portfolioView.sections || []).filter((section) => section.id !==
"brief" && previewSectionHasRenderableContent(section));
3527:     return `
3528:         <article class="portfolio-preview-article ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" style="${projectTemplateStyle(project, accent)}">
```

fullProjectPreviewHtmlExact (template-preview.js, lines 3558-3685)

fullProjectPreviewHtmlExact part of the private builder frontend and editing experience. In this file, it appears around lines 3558-3685 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parseProjectForPortfolio, categoryById, normalizeCategory, displayTitle, parsedPortfolioGlobals, stringify, replaceAll, escapeHtml, var, min, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3561: return `<!DOCTYPE html>; line 3601: return `<!DOCTYPE html>.

Important local values include baseHref at line 3559; parsedProject at line 3580; categorySource at line 3581; previewCategory at line 3582; parsedGlobals at line 3586; previewDataObject at line 3587; previewData at line 3599; title at line 3600. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3558: function fullProjectPreviewHtmlExact(project) {
3559:     const baseHref = `${window.location.origin}/`;
3560:     if (!project) {
3561:         return `<!DOCTYPE html>
```

```

3562: <html lang="en">
3563:   <head>
3564:     <meta charset="UTF-8" />
3565:     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
3566:     <base href="{baseHref}" />
3567:     <link rel="stylesheet" href="styles.css" />
3568:     <title>Project preview</title>
3569:   </head>
3570:   <body class="project-isolated-preview">
3571:     <main class="project-isolated-shell">
3572:       <div class="empty-state">
3573:         <h3>No saved project preview</h3>
3574:         <p>Click Save project before viewing the parsed project preview.</p>
3575:       </div>
3576:     </main>
3577:   </body>
3578: </html>;
3579: }
3580: const parsedProject = project.portfolioView ? project : parseProjectForPortfolio(project);
3581: const categorySource = categoryById(parsedProject.category);
3582: const previewCategory = normalizeCategory(categorySource.id ? categorySource : {
3583:   id: parsedProject.category || "project",
3584:   label: displayTitle(parsedProject.category || "Project", "Project")
3585: });
3586: const parsedGlobals = parsedPortfolioGlobals();
3587: const previewDataObject = {
3588:   categories: [previewCategory],
3589:   fieldStyles: parsedGlobals.fieldStyles,
3590:   funFacts: [],
3591:   funFactsRich: null,
3592:   profile: parsedGlobals.profile,
3593:   profileRich: parsedGlobals.profileRich,
3594:   projects: [parsedProject],
3595:   siteContent: parsedGlobals.siteContent,
3596:   siteContentRich: parsedGlobals.siteContentRich,
3597:   siteSections: []
3598: };
3599: const previewData = JSON.stringify(previewDataObject).replaceAll("</", "<\\");
3600: const title = parsedProject.portfolioView?.title || parsedProject.title || "Project preview";
3601: return `<!DOCTYPE html>
3602: <html lang="en">
3603:   <head>
3604:     <meta charset="UTF-8" />
3605:     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
3606:     <base href="{baseHref}" />
3607:     <title>${escapeHtml(title)} | Project preview</title>
3608:     <link rel="stylesheet" href="styles.css" />
3609:     <style>
3610:       body.project-isolated-preview {
3611:         min-height: 100vh;
3612:         margin: 0;
3613:         background: var(--paper, #f8fbff);
3614:         color: var(--ink, #172636);
3615:       }
3616:
3617:       .project-isolated-shell {
3618:         width: min(1180px, calc(100vw - 32px));
3619:         margin: 0 auto;
3620:         padding: 28px 0 36px;
3621:       }
3622:
3623:       .project-isolated-support {
3624:         display: none !important;
3625:       }
3626:
3627:       body.project-isolated-preview .project-grid {
3628:         display: block;
3629:       }
3630:
3631:       body.project-isolated-preview .category-section {
3632:         padding: 0;
3633:         border: 0;
3634:         background: transparent;
3635:       }
3636:
3637:       body.project-isolated-preview .category-heading,
3638:       body.project-isolated-preview .category-description {
3639:         display: none;
3640:       }
3641:
3642:       body.project-isolated-preview .category-projects {
3643:         display: block;
3644:       }
3645:
3646:       body.project-isolated-preview .project-card {
3647:         margin: 0;
3648:       }
3649:
3650:       @media (max-width: 720px) {
3651:         .project-isolated-shell {
3652:           width: min(100vw - 20px, 100%);
3653:           padding: 14px 0 24px;

```

```

3654:     }
3655:   }
3656: </style>
3657: </head>
3658: <body class="project-isolated-preview">
3659:   <main class="project-isolated-shell" aria-label="Isolated project preview">
3660:     <div class="project-isolated-support" aria-hidden="true">
3661:       <label class="search-wrap" for="project-search">
3662:         <span>Search</span>
3663:         <input id="project-search" type="search" tabindex="-1" />
3664:       </label>
3665:       <div id="project-filters">
3666:         <button class="filter-button active" type="button" data-filter="all" tabindex="-1">All</button>
3667:       </div>
3668:       <span id="project-count">0</span>
3669:       <span id="project-track-count">0 Tracks</span>
3670:       <span id="project-track-labels">project categories</span>
3671:       <span id="year">${new Date().getFullYear()}</span>
3672:     </div>
3673:
3674:     <section class="section project-isolated-section" id="projects" aria-labelledby="project-isolated-
title">
3675:       <h1 class="sr-only" id="project-isolated-title">${escapeHtml(title)}</h1>
3676:       <div class="project-grid" id="project-grid" aria-live="polite"></div>
3677:     </section>
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

renderProjectWebsitePreview (template-preview.js, lines 3687-3705)

renderProjectWebsitePreview renders ui, markup, or display output. In this file, it appears around lines 3687-3705 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, applyProjectTemplateToElement, updateProjectPreviewNavigation, add, escapeHtml, querySelector, and fullProjectPreviewHtmlExact. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include category at line 3688; frame at line 3703. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3687: function renderProjectWebsitePreview(project) {
3688:   const category = categoryById(project?.category);
3689:   applyProjectTemplateToElement(projectPreviewDialog, project, category.accent || "#117c7a");
3690:   projectPreviewTitle.textContent = project?.portfolioView?.title || project?.title || "Project preview";
3691:   activeProjectPreviewProject = project || null;
3692:   activeProjectPreviewSectionIndex = -1;
3693:   activeProjectPreviewPath = [];
3694:   activeProjectPreviewForwardStack = [];
3695:   updateProjectPreviewNavigation();
3696:   projectPreviewContent.classList.add("is-website-preview");
3697:   projectPreviewContent.innerHTML = `
3698:     <iframe
3699:       class="portfolio-preview-frame project-preview-frame"
3700:       title="${escapeHtml(project?.title || "Project website preview")}"
3701:     ></iframe>
3702:   `;
3703:   const frame = projectPreviewContent.querySelector("iframe");
3704:   if (frame) frame.srcdoc = fullProjectPreviewHtmlExact(project);
3705: }

```

openProjectPortfolioPreview (template-preview.js, lines 3707-3715)

openProjectPortfolioPreview controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3707-3715 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, find, renderProjectWebsitePreview, add, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include project at line 3708; savedProject at line 3709; previewProject at line 3710. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3707: function openProjectPortfolioPreview() {
3708:   const project = selectedProject();
3709:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3710:   const previewProject = savedProject || project;
3711:   renderProjectWebsitePreview(previewProject);
3712:   document.body.classList.add("full-window-open");
3713:   projectPreviewDialog.showModal();
3714:   projectPreviewDialog.scrollTop = 0;
3715: }
```

savedProject (template-preview.js, lines 3709-3714)

savedProject persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 3709-3714 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, renderProjectWebsitePreview, add, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include savedProject at line 3709; previewProject at line 3710. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3709:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project?.id);
3710:   const previewProject = savedProject || project;
3711:   renderProjectWebsitePreview(previewProject);
3712:   document.body.classList.add("full-window-open");
3713:   projectPreviewDialog.showModal();
3714:   projectPreviewDialog.scrollTop = 0;
```

updateProjectPreviewNavigation (template-preview.js, lines 3717-3721)

updateProjectPreviewNavigation updates ui, state, cache, or stored data. In this file, it appears around lines 3717-3721 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references updateDialogWindowButtons. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3717: function updateProjectPreviewNavigation() {
3718:   projectPreviewBack.hidden = activeProjectPreviewSectionIndex < 0;
3719:   projectPreviewForward.hidden = !activeProjectPreviewForwardStack.length;
3720:   updateDialogWindowButtons(projectPreviewDialog);
3721: }
```

openProjectPreviewNode (template-preview.js, lines 3723-3745)

openProjectPreviewNode controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3723-3745 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, previewSectionHasRenderableContent, previewNodeAtPath, parsedItemHasContent, updateProjectPreviewNavigation, displayTitle, and parsedPublicSectionContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3724: if (!activeProjectPreviewProject?.portfolioView) return;; line 3727: if (!section) return;; line 3729: if (!node) return;; line 3730: if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;;

Important local values include sections at line 3725; section at line 3726; node at line 3728. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3723: function openProjectPreviewNode(sectionIndex, path = [], options = {}) {
3724:   if (!activeProjectPreviewProject?.portfolioView) return;
3725:   const sections = (activeProjectPreviewProject.portfolioView.sections || []).filter((section) =>
section.id !== "brief" && previewSectionHasRenderableContent(section));
3726:   const section = sections[Number(sectionIndex)];
3727:   if (!section) return;
3728:   const node = path.length ? previewNodeAtPath(section, path) : section;
3729:   if (!node) return;
3730:   if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;
3731:
3732:   activeProjectPreviewSectionIndex = Number(sectionIndex);
3733:   activeProjectPreviewPath = path;
3734:   if (!options.preserveForward) activeProjectPreviewForwardStack = [];
3735:   updateProjectPreviewNavigation();
3736:   projectPreviewTitle.textContent = displayTitle(node.title || section.title, "Section");
3737:   projectPreviewContent.innerHTML = `
3738:     <article class="portfolio-preview-article">
3739:       <section class="portfolio-preview-section">
3740:         ${parsedPublicSectionContent(section, Number(sectionIndex), path)}
3741:       </section>
3742:     </article>
3743:   `;
3744:   projectPreviewDialog.scrollTop = 0;
3745: }
```

sections (template-preview.js, lines 3725-3730)

sections part of the private builder frontend and editing experience. In this file, it appears around lines 3725-3730 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, previewSectionHasRenderableContent, previewNodeAtPath, and parsedItemHasContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3727: if (!section) return;; line 3729: if (!node) return;; line 3730: if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;.

Important local values include sections at line 3725; section at line 3726; node at line 3728. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3725:   const sections = (activeProjectPreviewProject.portfolioView.sections || []).filter((section) =>
section.id !== "brief" && previewSectionHasRenderableContent(section));
3726:   const section = sections[Number(sectionIndex)];
3727:   if (!section) return;
3728:   const node = path.length ? previewNodeAtPath(section, path) : section;
3729:   if (!node) return;
3730:   if (path.length ? !parsedItemHasContent(node) : !previewSectionHasRenderableContent(section)) return;
```

projectPreviewBackStep (template-preview.js, lines 3747-3769)

projectPreviewBackStep part of the private builder frontend and editing experience. In this file, it appears around lines 3747-3769 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, slice, renderProjectPortfolioPreview, updateProjectPreviewNavigation, and openProjectPreviewNode. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3748: if (!activeProjectPreviewProject) return;; line 3761: return;.

Important local values include nextPath at line 3767. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3747: function projectPreviewBackStep() {
3748:   if (!activeProjectPreviewProject) return;
3749:   if (!activeProjectPreviewPath.length) {
3750:     if (activeProjectPreviewSectionIndex >= 0) {
3751:       activeProjectPreviewForwardStack.push({
3752:         path: activeProjectPreviewPath.slice(),
3753:         sectionIndex: activeProjectPreviewSectionIndex
3754:       });
3755:     }
```

```

3756:     projectPreviewTitle.textContent = activeProjectPreviewProject.title || "Project preview";
3757:     projectPreviewContent.innerHTML = renderProjectPortfolioPreview(activeProjectPreviewProject);
3758:     activeProjectPreviewSectionIndex = -1;
3759:     activeProjectPreviewPath = [];
3760:     updateProjectPreviewNavigation();
3761:     return;
3762: }
3763: activeProjectPreviewForwardStack.push({
3764:   path: activeProjectPreviewPath.slice(),
3765:   sectionIndex: activeProjectPreviewSectionIndex
3766: });
3767: const nextPath = activeProjectPreviewPath.slice(0, -1);
3768: openProjectPreviewNode(activeProjectPreviewSectionIndex, nextPath, { preserveForward: true });
3769: }

```

projectPreviewForwardStep (template-preview.js, lines 3771-3775)

projectPreviewForwardStep part of the private builder frontend and editing experience. In this file, it appears around lines 3771-3775 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pop, and openProjectPreviewNode. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3772: if (!activeProjectPreviewProject || !activeProjectPreviewForwardStack.length) return;.

Important local values include next at line 3773. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3771: function projectPreviewForwardStep() {
3772:   if (!activeProjectPreviewProject || !activeProjectPreviewForwardStack.length) return;
3773:   const next = activeProjectPreviewForwardStack.pop();
3774:   openProjectPreviewNode(next.sectionIndex, next.path, { preserveForward: true });
3775: }

```

closeOrStepBackProjectPreview (template-preview.js, lines 3777-3783)

closeOrStepBackProjectPreview controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3777-3783 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectPreviewBackStep, and closeProjectPreviewWindow. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3780: return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3777: function closeOrStepBackProjectPreview() {
3778:   if (activeProjectPreviewProject && (activeProjectPreviewSectionIndex >= 0 ||
activeProjectPreviewPath.length)) {
3779:     projectPreviewBackStep();
3780:     return;
3781:   }
3782:   closeProjectPreviewWindow();
3783: }

```

closeProjectPreviewWindow (template-preview.js, lines 3785-3791)

closeProjectPreviewWindow controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3785-3791 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closeDialogElement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3785: function closeProjectPreviewWindow() {
3786:   activeProjectPreviewProject = null;
3787:   activeProjectPreviewSectionIndex = -1;
3788:   activeProjectPreviewPath = [];
3789:   activeProjectPreviewForwardStack = [];
3790:   closeDialogElement(projectPreviewDialog);
3791: }
```

publicProjectCard (template-preview.js, lines 3793-3856)

publicProjectCard part of the private builder frontend and editing experience. In this file, it appears around lines 3793-3856 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parsedPublicProjectCard, categoryById, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, detailBlock, map, itemTitle, join, evidenceList, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3794: if (project.portfolioView) return parsedPublicProjectCard(project);; line 3798: return `

Important local values include category at line 3795; accent at line 3796. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3793: function publicProjectCard(project) {
3794:   if (project.portfolioView) return parsedPublicProjectCard(project);
3795:   const category = categoryById(project.category);
3796:   const accent = category.accent || "#117c7a";
3797:
3798:   return `
3799:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${project.id}" style="${projectTemplateStyle(project, accent)}">
3800:       <div class="project-body">
3801:         <h3>${project.title}</h3>
3802:         <p>${project.summary || ""}</p>
3803:
3804:         ${project.highlights && project.highlights.length ? detailBlock("Project highlights", "project-
drawer", `
3805:           <ul class="highlight-list">
3806:             ${project.highlights.map((item) => `<li>${typeof item === "string" ? item :
itemTitle(item)}</li>`).join("")}
3807:           </ul>
3808:         `) : ""}
3809:
3810:         <div class="evidence-grid" aria-label="${project.title} evidence blocks">
3811:           ${detailBlock("Documents", "evidence-block", evidenceList(project.documents, (item) => `
3812:             <li>
3813:               ${resourceLink(item, item.title)}
3814:               <span>${item.type || "Document"} &mdot; ${item.status || "tracked"}</span>
3815:             </li>
3816:           `, "No document artifact has been added yet.))}
3817:
3818:           ${detailBlock("Tests and results", "evidence-block", evidenceList(project.tests, (item) => `
3819:             <li>
3820:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3821:               <span>${item.method || "Validation"} &mdot; ${item.status || "tracked"}</span>
3822:               ${item.result ? `<p>${item.result}</p>` : ""}
3823:             </li>
3824:           `, "No test artifact has been added yet.))}
3825:
3826:           ${detailBlock("PCBs built", "evidence-block", evidenceList(project.pcb, (item) => `
3827:             <li>
3828:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3829:               <span>${item.revision || "Revision"} &mdot; ${item.status || "tracked"}</span>
3830:             </li>
3831:           `, "No PCB build has been added yet.))}
3832:
3833:           ${detailBlock("Images", "evidence-block evidence-wide", mediaGrid(project.media))}
3834:           ${publicCustomSectionBlocks(project)}
3835:         </div>
3836:
3837:         ${detailBlock("Tools and implementation files", "project-drawer", `
3838:           <div class="project-tooling">
3839:             <div>
3840:               <h4>Tools Used</h4>
3841:               <div class="project-meta">${pillList(project.tools, "tool-tag")}</div>
3842:             </div>
3843:           </div>
3844:         `)
3845:       </div>
3846:     </article>
3847:   `;
```

```

3844:         <h4>Languages</h4>
3845:         <div class="project-meta">${pillList(project.languages, "language-tag")}</div>
3846:       </div>
3847:     </div>
3848:   `)}
3849:
3850:   <div class="resource-list">
3851:     ${project.links || []}.map((item) => resourceLink(item)).join("")
3852:   </div>
3853: </div>
3854: </article>
3855: `;
3856: }

```

renderParsedBriefBlock (template-preview.js, lines 3858-3868)

renderParsedBriefBlock renders ui, markup, or display output. In this file, it appears around lines 3858-3868 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3861: if (!briefItem.rich?.blocks?.length && !briefText) return "";; line 3862: return `.

Important local values include briefItem at line 3859; briefText at line 3860. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3858: function renderParsedBriefBlock(section, fallbackSummary = "") {
3859:   const briefItem = section?.items?.[0] || {};
3860:   const briefText = briefItem.description || fallbackSummary || "";
3861:   if (!briefItem.rich?.blocks?.length && !briefText) return "";
3862:   return `
3863:     <details class="project-brief-default parsed-summary" open>
3864:       <summary>Overview</summary>
3865:       ${renderRichContent(briefItem.rich, briefText)}
3866:     </details>
3867:   `;
3868: }

```

previewPathToString (template-preview.js, lines 3870-3872)

previewPathToString part of the private builder frontend and editing experience. In this file, it appears around lines 3870-3872 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3871: return path.join(".");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3870: function previewPathToString(path = []) {
3871:   return path.join(".");
3872: }

```

previewPathFromString (template-preview.js, lines 3874-3877)

previewPathFromString part of the private builder frontend and editing experience. In this file, it appears around lines 3874-3877 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, filter, and isInteger. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3875: if (!value) return []; line 3876: return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3874: function previewPathFromString(value = "") {
3875:   if (!value) return [];
3876:   return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));
3877: }
```

previewNodeAtPath (template-preview.js, lines 3879-3888)

previewNodeAtPath part of the private builder frontend and editing experience. In this file, it appears around lines 3879-3888 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3884: if (!node) return null;; line 3887: return node;.

Important local values include node at line 3880; children at line 3881. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3879: function previewNodeAtPath(section, path = []) {
3880:   let node = section;
3881:   let children = section?.items || [];
3882:   for (const index of path) {
3883:     node = children[index];
3884:     if (!node) return null;
3885:     children = node.children || [];
3886:   }
3887:   return node;
3888: }
```

previewNodeChildren (template-preview.js, lines 3890-3892)

previewNodeChildren part of the private builder frontend and editing experience. In this file, it appears around lines 3890-3892 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3891: return node?.items || node?.children || [];

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3890: function previewNodeChildren(node) {
3891:   return node?.items || node?.children || [];
3892: }
```

previewSectionHasRenderableContent (template-preview.js, lines 3894-3896)

previewSectionHasRenderableContent part of the private builder frontend and editing experience. In this file, it appears around lines 3894-3896 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasContent, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3895: return Boolean(section?.description || richHasContent(section?.rich) || (section?.items || []).some(parsedItemHasContent));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3894: function previewSectionHasRenderableContent(section) {
3895:   return Boolean(section?.description || richHasContent(section?.rich) || (section?.items ||
[]).some(parsedItemHasContent));
3896: }

```

previewNodeSummary (template-preview.js, lines 3898-3906)

previewNodeSummary part of the private builder frontend and editing experience. In this file, it appears around lines 3898-3906 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, and renderRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3899: if (!rich?.blocks?.length && !text) return "";; line 3900: return `

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3898: function previewNodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.") {
3899:   if (!rich?.blocks?.length && !text) return "";
3900:   return `
3901:     <details class="parsed-summary" open>
3902:       <summary>${escapeHtml(title)}</summary>
3903:       ${renderRichContent(rich, text || "")}
3904:     </details>
3905:   `;
3906: }

```

previewNodeIsOverview (template-preview.js, lines 3908-3911)

previewNodeIsOverview part of the private builder frontend and editing experience. In this file, it appears around lines 3908-3911 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, displayTitle, and endsWith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3910: return item.kind === "summary" || title === "overview" || title.endsWith(" overview");.

Important local values include title at line 3909. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3908: function previewNodeIsOverview(item = {}) {
3909:   const title = normalize(displayTitle(item.title || item.label || ""));
3910:   return item.kind === "summary" || title === "overview" || title.endsWith(" overview");
3911: }

```

previewNodeOverviewDetails (template-preview.js, lines 3913-3931)

previewNodeOverviewDetails part of the private builder frontend and editing experience. In this file, it appears around lines 3913-3931 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, push, renderRichContent, forEach, previewFileList, previewNodeChildren, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3924: if (!blocks.length) return "";; line 3925: return `

Important local values include overviewItems at line 3914; blocks at line 3915; itemFiles at line 3921. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3913: function previewNodeOverviewDetails(node, children = []) {
3914:   const overviewItems = children.filter(previewNodeIsOverview);
3915:   const blocks = [];

```

```

3916:   if (node?.rich?.blocks?.length || node?.description) {
3917:     blocks.push(renderRichContent(node.rich, node.description || ""));
3918:   }
3919:   overviewItems.forEach((item) => {
3920:     if (item.rich?.blocks?.length || item.description) blocks.push(renderRichContent(item.rich,
item.description || ""));
3921:     const itemFiles = previewFileList(previewNodeChildren(item));
3922:     if (itemFiles) blocks.push(itemFiles);
3923:   });
3924:   if (!blocks.length) return "";
3925:   return `
3926:     <details class="parsed-summary section-overview-default" open>
3927:       <summary>Overview</summary>
3928:       ${blocks.join("")}
3929:     </details>
3930:   `;
3931: }

```

previewNodeCard (template-preview.js, lines 3933-3940)

previewNodeCard part of the private builder frontend and editing experience. In this file, it appears around lines 3933-3940 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references displayTitle, previewPathToString, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3935: return `.

Important local values include title at line 3934. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3933: function previewNodeCard(node, sectionIndex, path) {
3934:   const title = displayTitle(node.title);
3935:   return `
3936:     <button class="section-open-card subsection-open-card" type="button" data-preview-section-
index="${sectionIndex}" data-preview-resource-path="${previewPathToString(path)}">
3937:       <span>${escapeHtml(title)}</span>
3938:     </button>
3939:   `;
3940: }

```

previewChildCards (template-preview.js, lines 3942-3950)

previewChildCards part of the private builder frontend and editing experience. In this file, it appears around lines 3942-3950 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, parsedItemHasContent, previewNodeIsOverview, map, previewNodeCard, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3944: if (!visibleChildren.length) return "";; line 3945: return `.

Important local values include visibleChildren at line 3943. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3942: function previewChildCards(children, sectionIndex, basePath = []) {
3943:   const visibleChildren = children.filter((child) => parsedItemHasContent(child) && !child.url &&
!previewNodeIsOverview(child));
3944:   if (!visibleChildren.length) return "";
3945:   return `
3946:     <div class="subsection-grid section-content-grid">
3947:       ${children.map((child, index) => parsedItemHasContent(child) && !child.url &&
!previewNodeIsOverview(child) ? previewNodeCard(child, sectionIndex, [...basePath, index]) : "").join("")}
3948:     </div>
3949:   `;
3950: }

```

previewFileList (template-preview.js, lines 3952-3969)

previewFileList part of the private builder frontend and editing experience. In this file, it appears around lines 3952-3969 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, map, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, downloadAttribute, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3954: if (!files.length) return "";; line 3955: return `

Important local values include files at line 3953. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3952: function previewFileList(children = []) {
3953:   const files = children.filter((child) => child?.url);
3954:   if (!files.length) return "";
3955:   return `
3956:     <div class="section-file-list">
3957:       ${files.map((file) => `
3958:         <a
3959:           class="section-file-link"
3960:           href="${escapeHtml(normalizeLinkTarget(file.url, { assumeweb: isWebsiteLinkItem(file, file.url)
3961:         }})}"
3962:           ${linkAttributes(file.url, file)}
3963:           ${downloadAttribute(file.url, file)}
3964:         >
3965:           ${escapeHtml(file.title || "Open link")}
3966:         </a>
3967:       `).join("")}
3968:     </div>
3969:   `;
```

previewNodeContent (template-preview.js, lines 3971-3983)

previewNodeContent part of the private builder frontend and editing experience. In this file, it appears around lines 3971-3983 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references previewFileList, previewNodeChildren, filter, previewNodesOverview, previewNodeOverviewDetails, and previewChildCards. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3973: if (!isContainer && node.url) return previewFileList([node]);; line 3976: return `

Important local values include isContainer at line 3972; children at line 3974; contentChildren at line 3975. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3971: function previewNodeContent(node, sectionIndex, path = []) {
3972:   const isContainer = !node.kind || node.kind === "subsection";
3973:   if (!isContainer && node.url) return previewFileList([node]);
3974:   const children = previewNodeChildren(node);
3975:   const contentChildren = children.filter((child) => !previewNodeIsOverview(child));
3976:   return `
3977:     <article class="parsed-window-panel section-directory">
3978:       ${previewNodeOverviewDetails(node, children)}
3979:       ${previewChildCards(children, sectionIndex, path)}
3980:       ${previewFileList(contentChildren)}
3981:     </article>
3982:   `;
3983: }
```

parsedPublicSectionContent (template-preview.js, lines 3985-3989)

parsedPublicSectionContent part of the private builder frontend and editing experience. In this file, it appears around lines 3985-3989 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references previewNodeAtPath, and previewNodeContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3987: if (!node) return `

Important local values include node at line 3986. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3985: function parsedPublicSectionContent(section, sectionIndex = 0, path = []) {
3986:   const node = path.length ? previewNodeAtPath(section, path) : section;
3987:   if (!node) return `
```

parsedPublicSection (template-preview.js, lines 3991-3999)

parsedPublicSection part of the private builder frontend and editing experience. In this file, it appears around lines 3991-3999 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, some, escapeHtml, and displayTitle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3994: return `

Important local values include visibleItems at line 3992; hasImages at line 3993. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3991: function parsedPublicSection(section, index = 0) {
3992:   const visibleItems = (section.items || []).filter(parsedItemHasContent);
3993:   const hasImages = visibleItems.some((item) => item.kind === "image");
3994:   return `
3995:     <button class="section-open-card evidence-block ${hasImages ? "evidence-wide" : ""}" type="button"
data-preview-section-index="${index}">
3996:       <span>${escapeHtml(displayTitle(section.title, "Section"))}</span>
3997:     </button>
3998:   `;
3999: }
```

parsedPublicProjectCard (template-preview.js, lines 4001-4019)

parsedPublicProjectCard part of the private builder frontend and editing experience. In this file, it appears around lines 4001-4019 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, find, filter, previewSectionHasRenderableContent, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, renderParsedBriefBlock, map, parsedPublicSection, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4008: return `

Important local values include category at line 4002; accent at line 4003; view at line 4004; briefSection at line 4005; otherSections at line 4006. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4001: function parsedPublicProjectCard(project) {
4002:   const category = categoryById(project.category);
4003:   const accent = category.accent || "#117c7a";
4004:   const view = project.portfolioView;
4005:   const briefSection = (view.sections || []).find((section) => section.id === "brief");
4006:   const otherSections = (view.sections || []).filter((section) => section.id !== "brief" &&
previewSectionHasRenderableContent(section));
4007:
4008:   return `
4009:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${project.id}" style="${projectTemplateStyle(project, accent)}">
4010:       <div class="project-body">
4011:         <h3>${view.title || project.title}</h3>
4012:         ${renderParsedBriefBlock(briefSection, project.summary)}
4013:         <div class="evidence-grid" aria-label="${project.title} parsed project content">
4014:           ${otherSections.map((section, index) => parsedPublicSection(section, index)).join("")}
4015:         </div>
4016:       </div>
4017:     </article>
```

```
4018: `;  
4019: }
```

briefSection (template-preview.js, lines 4005-4006)

briefSection part of the private builder frontend and editing experience. In this file, it appears around lines 4005-4006 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, filter, and previewSectionHasRenderableContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include briefSection at line 4005; otherSections at line 4006. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4005: const briefSection = (view.sections || []).find((section) => section.id === "brief");  
4006: const otherSections = (view.sections || []).filter((section) => section.id !== "brief" &&  
previewSectionHasRenderableContent(section));
```

otherSections (template-preview.js, lines 4006-4006)

otherSections part of the private builder frontend and editing experience. In this file, it appears around lines 4006-4006 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, and previewSectionHasRenderableContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include otherSections at line 4006. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4006: const otherSections = (view.sections || []).filter((section) => section.id !== "brief" &&  
previewSectionHasRenderableContent(section));
```

publicCategorySection (template-preview.js, lines 4021-4036)

publicCategorySection part of the private builder frontend and editing experience. In this file, it appears around lines 4021-4036 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4022: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4021: function publicCategorySection(category, visibleProjects) {  
4022:   return `  
4023:     <section class="category-section ${visibleProjects.length ? "" : "empty-category-section"}" aria-  
labelledby="${category.id}-preview-title">  
4024:       <div class="category-heading">  
4025:         <div>  
4026:           <h3 id="${category.id}-preview-title">${category.label}</h3>  
4027:         </div>  
4028:         ${visibleProjects.length ? `<span>${visibleProjects.length} project${visibleProjects.length === 1  
? "" : "s"}</span>` : ""}  
4029:       </div>  
4030:       ${visibleProjects.length ? `<p class="category-description">${category.description}</p>` : ""}  
4031:       <div class="category-projects">  
4032:         ${visibleProjects.map(publicProjectCard).join("")}  
4033:       </div>  
4034:     </section>
```

```
4035: `;  
4036: }
```

renderedPublicProjects (template-preview.js, lines 4038-4051)

renderedPublicProjects renders ui, markup, or display output. In this file, it appears around lines 4038-4051 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, filter, publicCategorySection, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4040: return `; line 4047: return savedPortfolioCatalog.categories.map((category) => {}); line 4049: return publicCategorySection(category, categoryProjects);.

Important local values include categoryProjects at line 4048. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4038: function renderedPublicProjects() {  
4039:   if (!savedPortfolioCatalog.projects.length) {  
4040:     return `  
4041:       <div class="empty-state">  
4042:         <h3>No parsed projects saved yet.</h3>  
4043:         <p>Open a project, edit it, then click Save to build it into this portfolio preview.</p>  
4044:       </div>  
4045:     `;  
4046:   }  
4047:   return savedPortfolioCatalog.categories.map((category) => {  
4048:     const categoryProjects = savedPortfolioCatalog.projects.filter((project) => project.category ===  
category.id);  
4049:     return publicCategorySection(category, categoryProjects);  
4050:   }).join("");  
4051: }
```

sectionControls (template-preview.js, lines 4053-4060)

sectionControls part of the private builder frontend and editing experience. In this file, it appears around lines 4053-4060 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4054: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4053: function sectionControls(sectionId, uploadKey = "", folder = "") {  
4054:   return `  
4055:     <div class="local-section-controls">  
4056:       <button type="button" title="Edit this section" data-preview-section="${sectionId}">Edit</button>  
4057:       ${uploadKey ? `<button type="button" title="Add file or image" data-preview-upload="${uploadKey}"`  
data-folder="${folder}">+</button>` : ""}  
4058:     </div>  
4059:   `;  
4060: }
```

customSectionBlocks (template-preview.js, lines 4062-4074)

customSectionBlocks part of the private builder frontend and editing experience. In this file, it appears around lines 4062-4074 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, detailBlock, sectionControls, evidenceList, resourceLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4063: return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-wide", `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4062: function customSectionBlocks(project) {
4063:   return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-
wide", `
4064:     ${sectionControls(`custom:${section.id}`, "sections", section.id)}
4065:     ${section.description ? `<p class="evidence-empty">${section.description}</p>` : ""}
4066:     ${evidenceList(section.items || [], (item) => `
4067:       <li>
4068:         ${item.url ? resourceLink(item, item.title) : `<strong>${item.title}</strong>`}
4069:         <span>${item.type || "Section item"} &middot; ${item.status || "tracked"}</span>
4070:         ${item.description ? `<p>${item.description}</p>` : ""}
4071:       </li>
4072:     `, "No content has been added yet.")]
4073:   `)).join("");
4074: }
```

projectCard (template-preview.js, lines 4076-4154)

projectCard part of the private builder frontend and editing experience. In this file, it appears around lines 4076-4154 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, projectTemplateClass, projectResponsiveClass, projectTemplateStyle, detailBlock, map, join, sectionControls, evidenceList, resourceLink, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4080: return `.

Important local values include category at line 4077; accent at line 4078. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4076: function projectCard(project, isSelected = false) {
4077:   const category = categoryById(project.category);
4078:   const accent = category.accent || "#117c7a";
4079:
4080:   return `
4081:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)} ${isSelected ? "preview-project-card" : ""} id="${project.id}"
style="${projectTemplateStyle(project, accent)}">
4082:       <div class="project-body">
4083:         <div class="local-card-controls">
4084:           <button type="button" title="Open project" data-preview-project="${project.id}">Open</button>
4085:           <button type="button" title="Delete project" data-preview-
delete="${project.id}">Delete</button>
4086:         </div>
4087:         <h3>${project.title}</h3>
4088:         <p>${project.summary}</p>
4089:
4090:         ${project.highlights && project.highlights.length ? detailBlock("Project highlights", "project-
drawer", `
4091:           <ul class="highlight-list">
4092:             ${project.highlights.map((item) => `<li>${item}</li>`).join("")}
4093:           </ul>
4094:         `) : ""}
4095:
4096:         <div class="evidence-grid" aria-label="${project.title} evidence blocks">
4097:           ${detailBlock("Documents", "evidence-block", `
4098:             ${sectionControls("documents", "documents", "documents")}
4099:             ${evidenceList(project.documents, (item) => `
4100:               <li>
4101:                 ${resourceLink(item, item.title)}
4102:                 <span>${item.type || "Document"} &middot; ${item.status || "tracked"}</span>
4103:               </li>
4104:             `, "No document artifact has been added yet.")]
4105:           `)}
4106:
4107:           ${detailBlock("Tests and results", "evidence-block", `
4108:             ${sectionControls("tests", "tests", "tests")}
4109:             ${evidenceList(project.tests, (item) => `
4110:               <li>
4111:                 <resourceLink({ url: item.artifact, status: item.status }, item.name)}
4112:                 <span>${item.method || "Validation"} &middot; ${item.status || "tracked"}</span>
4113:                 ${item.result ? `<p>${item.result}</p>` : ""}
4114:               </li>
4115:             `, "No test artifact has been added yet.")]
4116:           `)}
4117:
4118:           ${detailBlock("PCBs built", "evidence-block", `
4119:             ${sectionControls("pcbs", "pcbs", "pcbs")}
4120:             ${evidenceList(project.pcb, (item) => `
```

```

4121:         <li>
4122:             ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
4123:             <span>${item.revision || "Revision"} &middot; ${item.status || "tracked"}</span>
4124:         </li>
4125:     ` , "No PCB build has been added yet.")}
4126: `)}
4127:
4128:     ${detailBlock("Images", "evidence-block evidence-wide", `
4129:         ${sectionControls("media", "media", "images")}
4130:         ${mediaGrid(project.media)}
4131:     `)}
4132:     ${customSectionBlocks(project)}
4133: </div>
4134:
4135:     ${detailBlock("Tools and implementation files", "project-drawer", `
4136:         <div class="project-tooling">
4137:             <div>
4138:                 <h4>Tools Used</h4>
4139:                 <div class="project-meta">${pillList(project.tools, "tool-tag")}</div>
4140:             </div>
4141:             <div>
4142:                 <h4>Languages</h4>
4143:                 <div class="project-meta">${pillList(project.languages, "language-tag")}</div>
4144:             </div>
4145:         </div>
4146:     `)}
4147:
4148:     <div class="resource-list">
4149:         ${project.links || []}.map((item) => resourceLink(item)).join("")
4150:     </div>
4151: </div>
4152: </article>
4153: `;
4154: }

```

buildTemplateProject (template-preview.js, lines 4156-4186)

buildTemplateProject creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4156-4186 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, trim, slugify, clone, and defaultDesignModel. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4162: return {.

Important local values include template at line 4157; title at line 4158; id at line 4159; category at line 4160. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

4156: function buildTemplateProject() {
4157:     const template = templates.find((item) => item.id === templateSelect.value) || null;
4158:     const title = newProjectTitle.value.trim() || "New Hardware Project";
4159:     const id = slugify(title);
4160:     const category = newCategorySelect.value || pendingCreateCategoryId || "analog-mixed-signal";
4161:
4162:     return {
4163:         id,
4164:         title,
4165:         templateId: template?.id || "",
4166:         templateLabel: template?.label || "",
4167:         templateGroup: "Appearance",
4168:         templateVisual: template?.visual ? clone(template.visual) : null,
4169:         category,
4170:         status: "Draft",
4171:         summary: "",
4172:         summaryRich: null,
4173:         focus: [],
4174:         highlights: [],
4175:         documents: [],
4176:         tests: [],
4177:         pcbs: [],
4178:         media: [],
4179:         tools: [],
4180:         languages: [],
4181:         links: [],
4182:         sections: [],
4183:         compileCode: { files: [], activeFileId: "", terminal: "" },
4184:         design: category === "analog-mixed-signal" ? defaultDesignModel() : undefined
4185:     };
4186: }

```

insertProject (template-preview.js, lines 4188-4204)

insertProject part of the private builder frontend and editing experience. In this file, it appears around lines 4188-4204 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, map, and splice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4192: if (placement === "site-start") return [project, ...nextProjects];; line 4193: if (placement === "site-end") return [...nextProjects, project];; line 4200: if (!indexes.length) return [...nextProjects, project];; line 4203: return nextProjects;.

Important local values include nextProjects at line 4189; placement at line 4190; indexes at line 4195; targetIndex at line 4201. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4188: function insertProject(project) {
4189:   const nextProjects = catalog.projects.filter((item) => item.id !== project.id);
4190:   const placement = placementSelect.value;
4191:
4192:   if (placement === "site-start") return [project, ...nextProjects];
4193:   if (placement === "site-end") return [...nextProjects, project];
4194:
4195:   const indexes = nextProjects
4196:     .map((item, index) => ({ item, index }))
4197:     .filter(({ item }) => item.category === project.category)
4198:     .map(({ index }) => index);
4199:
4200:   if (!indexes.length) return [...nextProjects, project];
4201:   const targetIndex = placement === "category-start" ? indexes[0] : indexes[indexes.length - 1] + 1;
4202:   nextProjects.splice(targetIndex, 0, project);
4203:   return nextProjects;
4204: }
```

renderCategoryDropdown (template-preview.js, lines 4206-4213)

renderCategoryDropdown renders ui, markup, or display output. In this file, it appears around lines 4206-4213 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4206: function renderCategoryDropdown() {
4207:   categoryDropdown.innerHTML = (catalog.categories || []).map((category) => `
4208:     <button type="button" data-create-category="${escapeHtml(category.id)}" style="--category-accent:
${escapeHtml(category.accent || "#117c7a")}">
4209:       <span>${escapeHtml(category.label)}</span>
4210:       <small>${escapeHtml(category.description || "Project category")}</small>
4211:     </button>
4212: `).join("");
4213: }
```

toggleCategoryDropdown (template-preview.js, lines 4215-4219)

toggleCategoryDropdown part of the private builder frontend and editing experience. In this file, it appears around lines 4215-4219 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include open at line 4216. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4215: function toggleCategoryDropdown(forceOpen = null) {
4216:   const open = forceOpen === null ? categoryDropdown.hidden : forceOpen;
4217:   categoryDropdown.hidden = !open;
4218:   addProjectButton.setAttribute("aria-expanded", String(open));
4219: }
```

createProjectInCategory (template-preview.js, lines 4221-4233)

createProjectInCategory creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4221-4233 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references buildTemplateProject, insertProject, add, toggleCategoryDropdown, setStatus, categoryById, scheduleAutosave, renderAll, and openProjectWindow. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include project at line 4223. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4221: function createProjectInCategory(categoryId) {
4222:   newCategorySelect.value = categoryId;
4223:   const project = buildTemplateProject();
4224:   catalog.projects = insertProject(project);
4225:   selectedProjectId = project.id;
4226:   activeSectionId = "brief";
4227:   expandedCategories.add(categoryId);
4228:   toggleCategoryDropdown(false);
4229:   setStatus(`Project added locally in ${categoryById(categoryId).label} || categoryId}. Click Save project
to include it in the portfolio preview.`);
4230:   scheduleAutosave();
4231:   renderAll();
4232:   openProjectWindow(project.id, "brief");
4233: }
```

openProjectCreateDialog (template-preview.js, lines 4235-4245)

openProjectCreateDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4235-4245 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, groupedTemplateOptions, toggleCategoryDropdown, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include category at line 4238. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4235: function openProjectCreateDialog(categoryId) {
4236:   pendingCreateCategoryId = categoryId;
4237:   newCategorySelect.value = categoryId;
4238:   const category = categoryById(categoryId);
4239:   projectCreateCategory.textContent = category.label || "Project category";
4240:   templateSelect.innerHTML = groupedTemplateOptions();
4241:   templateSelect.value = "";
4242:   newProjectTitle.value = "";
4243:   toggleCategoryDropdown(false);
4244:   projectCreateDialog.showModal();
4245: }
```

groupedTemplateOptions (template-preview.js, lines 4247-4253)

groupedTemplateOptions part of the private builder frontend and editing experience. In this file, it appears around lines 4247-4253 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4252: return `\${emptyOption}\${options}`;

Important local values include emptyOption at line 4248; options at line 4249. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4247: function groupedTemplateOptions() {
4248:   const emptyOption = `<option value="">No appearance - white project layout</option>`;
4249:   const options = templates
4250:     .map((template) => `<option value="${template.id}">${template.label}</option>`)
4251:     .join("");
4252:   return `${emptyOption}${options}`;
4253: }
```

showTemplatePreview (template-preview.js, lines 4255-4276)

showTemplatePreview controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4255-4276 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setProperty, map, join, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4256: if (!template) return;

Important local values include visual at line 4257; palette at line 4258. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4255: function showTemplatePreview(template) {
4256:   if (!template) return;
4257:   const visual = template.visual || {};
4258:   const palette = visual.palette || ["#0f4c5c", "#7dd3c7", "#ffffff"];
4259:
4260:   templatePreviewGroup.textContent = visual.style ? `Appearance / ${visual.style}` : "Appearance";
4261:   templatePreviewTitle.textContent = template.label;
4262:   templatePreviewDescription.textContent = template.description || "";
4263:   templatePreviewCardTitle.textContent = template.label;
4264:   templatePreviewInteraction.textContent = visual.interaction || "Hover and click styles are shown in the
local builder.";
4265:   templatePreviewCard.style.setProperty("--template-primary", palette[0]);
4266:   templatePreviewCard.style.setProperty("--template-hover", palette[1]);
4267:   templatePreviewCard.style.setProperty("--template-paper", palette[2] || "#ffffff");
4268:   templatePreviewCard.style.setProperty("--template-bg", visual.background || palette[0]);
4269:   templatePreviewCard.style.setProperty("--template-panel", visual.panel || palette[2] || "#ffffff");
4270:   templatePreviewCard.style.setProperty("--template-accent", visual.accent || palette[1]);
4271:   templatePreviewCard.style.setProperty("--template-click", visual.click || palette[2] || "#ffffff");
4272:   templatePreviewCard.style.setProperty("--template-click-text", visual.clickText || visual.text ||
"#172636");
4273:   templateVisual.className = `template-visual template-visual-${visual.style || "schematic"`;
4274:   templatePalette.innerHTML = palette.map((color) => `<span style="background:
${color}">${color}</span>`).join("");
4275:   templateDialog.showModal();
4276: }
```

renderTree (template-preview.js, lines 4278-4319)

renderTree renders ui, markup, or display output. In this file, it appears around lines 4278-4319 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectSearchText, map, filter, includes, projectMatchesSearch, has, escapeHtml, highlightSearchText, join, and updateProjectSearchStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4287: if (cleanQuery && !categoryProjects.length && !categoryHit) return "";; line 4290: return `

Important local values include `cleanQuery` at line 4279; `matchCount` at line 4280; `treeMarkup` at line 4281; `allCategoryProjects` at line 4282; `categoryHit` at line 4283; `categoryProjects` at line 4284; `isExpanded` at line 4289. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4278: function renderTree() {
4279:   const cleanQuery = projectSearchText(projectSearchQuery);
4280:   let matchCount = 0;
4281:   const treeMarkup = catalog.categories.map((category) => {
4282:     const allCategoryProjects = catalog.projects.filter((project) => project.category === category.id);
4283:     const categoryHit = cleanQuery && projectSearchText(`${category.label}
${category.description}`).includes(cleanQuery);
4284:     const categoryProjects = cleanQuery
4285:       ? allCategoryProjects.filter((project) => categoryHit || projectMatchesSearch(project, category,
cleanQuery))
4286:       : allCategoryProjects;
4287:     if (cleanQuery && !categoryProjects.length && !categoryHit) return "";
4288:     matchCount += categoryProjects.length;
4289:     const isExpanded = cleanQuery ? true : expandedCategories.has(category.id);
4290:     return `
4291:       <section class="tree-category">
4292:         <div class="tree-category-heading">
4293:           <button class="tree-category-toggle" type="button" data-toggle-
category="${escapeHtml(category.id)}" aria-expanded="${isExpanded}">
4294:             <span>${highlightSearchText(category.label, projectSearchQuery)}</span>
4295:             <small>${allCategoryProjects.length} project${allCategoryProjects.length === 1 ? "" :
"s"}</small>
4296:           </button>
4297:         </div>
4298:         <div class="tree-projects" ${isExpanded ? "" : "hidden"}>
4299:           ${categoryProjects.length ? categoryProjects.map((project) => `
4300:             <div class="tree-project-row ${project.id === selectedProjectId ? "active" : ""}>
4301:               <button type="button" data-project-id="${escapeHtml(project.id)}">
4302:                 <span>${highlightSearchText(project.title, projectSearchQuery)}</span>
4303:               </button>
4304:               <button class="tree-project-delete" type="button" data-delete-
project="${escapeHtml(project.id)}" aria-label="Delete ${escapeHtml(project.title)}" title="Delete project">
4305:                 &times;
4306:               </button>
4307:             </div>
4308:           `).join("") : cleanQuery ? `<p class="tree-empty">Category matches, but no projects are in it
yet.</p>` : ""}
4309:         </div>
4310:       </section>
4311:     `;
4312:   }).join("");
4313:   projectTree.innerHTML = treeMarkup || `
4314:     <section class="tree-category tree-search-empty">
4315:       <p class="tree-empty">No project, file, tool, or section matched this search.</p>
4316:     </section>
4317:   `;
4318:   updateProjectSearchStatus(cleanQuery ? matchCount : catalog.projects.length);
4319: }
```

summaryWordCount (template-preview.js, lines 4321-4323)

`summaryWordCount` part of the private builder frontend and editing experience. In this file, it appears around lines 4321-4323 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `wordCount`, and `plainTextFromRich`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4322: `return wordCount(project?.summaryRich?.blocks?.length ? plainTextFromRich(project.summaryRich) : project?.summary || "")`;

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4321: function summaryWordCount(project) {
4322:   return wordCount(project?.summaryRich?.blocks?.length ? plainTextFromRich(project.summaryRich) :
project?.summary || "");
4323: }
```

renderSummaryPreview (template-preview.js, lines 4325-4329)

`renderSummaryPreview` renders ui, markup, or display output. In this file, it appears around lines 4325-4329 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderRichContent, and renderMultilineInlineText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4326: if (project.summaryRich?.blocks?.length) return renderRichContent(project.summaryRich, project.summary || ""); line 4327: if (project.summary) return `<p class="rich-paragraph">\${renderMultilineInlineText(project.summary)}</p>`; line 4328: return `<p class="evidence-empty">No project overview has been added yet.</p>`;

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4325: function renderSummaryPreview(project) {
4326:   if (project.summaryRich?.blocks?.length) return renderRichContent(project.summaryRich, project.summary
|| "");
4327:   if (project.summary) return `<p class="rich-
paragraph">${renderMultilineInlineText(project.summary)}</p>`;
4328:   return `<p class="evidence-empty">No project overview has been added yet.</p>`;
4329: }
```

renderSummaryBuilder (template-preview.js, lines 4331-4359)

renderSummaryBuilder renders ui, markup, or display output. In this file, it appears around lines 4331-4359 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references summaryWordCount, and renderSummaryPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4335: return `

Important local values include isEditing at line 4332; summaryWords at line 4333; hasSummary at line 4334. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4331: function renderSummaryBuilder(project) {
4332:   const isEditing = summaryEditorProjectId === project.id;
4333:   const summaryWords = summaryWordCount(project);
4334:   const hasSummary = summaryWords > 0 || Boolean(project.summaryRich?.blocks?.length);
4335:   return `
4336:     <div class="summary-builder wide-field">
4337:       <div class="summary-builder-heading">
4338:         <div>
4339:           <span>Project overview <small>${summaryWords}/1000 words</small></span>
4340:         </div>
4341:         <div class="summary-builder-actions">
4342:           ${isEditing
4343:             ? `<button class="button secondary" type="button" data-summary-cancel="true">Cancel
overview</button>`
4344:             : `<button class="button primary" type="button" data-summary-create="true">${hasSummary ?
"Edit overview" : "Create overview"}</button>`}
4345:           ${hasSummary && !isEditing ? `<button class="button danger-control" type="button" data-summary-
delete="true">Delete overview</button>` : ""}
4346:         </div>
4347:       </div>
4348:       ${isEditing ? `
4349:         <div class="rich-summary-panel">
4350:           <p class="rich-editor-hint">Right-click text or an inserted block for formatting, images,
formulas, code blocks, alignment, and delete controls.</p>
4351:           <div class="rich-summary-editor" data-rich-editor="summary" data-placeholder="Click anywhere
and start writing the project overview." contenteditable="true" spellcheck="true" aria-label="Rich overview
editor"></div>
4352:           <div class="summary-save-actions">
4353:             <button class="button primary" type="button" data-summary-save="true">Save overview</button>
4354:           </div>
4355:         </div>
4356:       ` : `<div class="summary-rendered-preview">${renderSummaryPreview(project)}</div>`}
4357:     </div>
4358:   `;
4359: }
```

applyRichTextBlockStyle (template-preview.js, lines 4361-4372)

applyRichTextBlockStyle part of the private builder frontend and editing experience. In this file, it appears around lines 4361-4372 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, and normalizeTextColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include fontFamily at line 4362; fontPx at line 4363; color at line 4364. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4361: function applyRichTextBlockStyle(block) {
4362:     const fontFamily = cleanFontFamily(block.dataset.fontFamily || "Arial") || "Arial";
4363:     const fontPx = normalizeFontPx(block.dataset.fontPx || "");
4364:     const color = normalizeTextColor(block.dataset.color || "");
4365:
4366:     block.style.fontFamily = fontFamily || "";
4367:     block.style.fontSize = fontPx ? `${fontPx}px` : "";
4368:     block.style.color = color || "";
4369:     block.style.fontWeight = block.dataset.bold === "true" ? "700" : "";
4370:     block.style.fontStyle = block.dataset.italic === "true" ? "italic" : "";
4371:     block.style.textDecoration = block.dataset.underline === "true" ? "underline" : "";
4372: }
```

createRichTextBlock (template-preview.js, lines 4374-4391)

createRichTextBlock creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4374-4391 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, cleanFontFamily, normalizeFontPx, normalizeTextColor, sanitizeRichInlineHtml, and applyRichTextBlockStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4390: return block;.

Important local values include block at line 4375. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4374: function createRichTextBlock(text = "", fontSize = "normal", align = "left", fontFamily = "Arial", fontPx
= "", color = "", bold = false, html = "", italic = false, underline = false) {
4375:     const block = document.createElement("p");
4376:     block.className = `rich-block rich-text-block rich-text-${fontSize} text-${align}`;
4377:     block.dataset.type = "paragraph";
4378:     block.dataset.fontSize = fontSize;
4379:     block.dataset.align = align;
4380:     block.dataset.fontFamily = cleanFontFamily(fontFamily || "Arial") || "Arial";
4381:     block.dataset.fontPx = normalizeFontPx(fontPx);
4382:     block.dataset.color = normalizeTextColor(color);
4383:     block.dataset.bold = bold ? "true" : "false";
4384:     block.dataset.italic = italic ? "true" : "false";
4385:     block.dataset.underline = underline ? "true" : "false";
4386:     block.spellcheck = true;
4387:     if (html) block.innerHTML = sanitizeRichInlineHtml(html);
4388:     else block.textContent = text;
4389:     applyRichTextBlockStyle(block);
4390:     return block;
4391: }
```

richBlockActions (template-preview.js, lines 4393-4401)

richBlockActions part of the private builder frontend and editing experience. In this file, it appears around lines 4393-4401 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4394: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4393: function richBlockActions(label = "block", options = {}) {
4394:   return `
4395:     <div class="rich-block-actions">
4396:       ${options.movable ? `<button class="rich-drag-handle" type="button" draggable="true" data-rich-
drag-handle aria-label="Move ${escapeHtml(label)}>Move</button>` : ""}
4397:       <button type="button" data-rich-block-action="edit" aria-label="Edit
${escapeHtml(label)}>Edit</button>
4398:       <button class="danger-icon" type="button" data-rich-block-action="delete" aria-label="Delete
${escapeHtml(label)}>Delete</button>
4399:     </div>
4400:   `;
4401: }
```

createRichImageBlock (template-preview.js, lines 4403-4431)

createRichImageBlock creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4403-4431 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, cleanRichImageTitle, normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, richBlockActions, richImageCropStyle, escapeHtml, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4430: return figure;.

Important local values include figure at line 4404; align at line 4405; title at line 4406. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4403: function createRichImageBlock(blockData) {
4404:   const figure = document.createElement("figure");
4405:   const align = blockData.align || "center";
4406:   const title = cleanRichImageTitle(blockData);
4407:   figure.className = `rich-block rich-image-block justify-${align}`;
4408:   figure.dataset.type = "image";
4409:   figure.dataset.url = blockData.url || "";
4410:   figure.dataset.title = blockData.title || "";
4411:   figure.dataset.caption = blockData.caption || "";
4412:   figure.dataset.align = align;
4413:   figure.dataset.display = blockData.display === "download" ? "download" : "show";
4414:   figure.dataset.source = blockData.source || "local";
4415:   figure.dataset.cropAspect = normalizeCropAspect(blockData.cropAspect);
4416:   figure.dataset.cropZoom = String(normalizeCropZoom(blockData.cropZoom));
4417:   figure.dataset.cropX = String(normalizeCropPosition(blockData.cropX));
4418:   figure.dataset.cropY = String(normalizeCropPosition(blockData.cropY));
4419:   figure.contentEditable = "false";
4420:   figure.draggable = true;
4421:   figure.title = "Drag image to move it between text.";
4422:   figure.innerHTML = `
4423:     ${richBlockActions("overview image", { movable: true })}
4424:     ${figure.dataset.display === "download" ? `<span class="rich-download-badge">Download only</span>` :
""}
4425:     <span class="rich-image-viewport crop-${figure.dataset.cropAspect === "original" ? "original" :
"active"}"${richImageCropStyle(blockData)}>
4426:       
4427:     </span>
4428:     ${((title || blockData.caption) ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>` :
""}${blockData.caption ? `<span>${escapeHtml(blockData.caption)}</span>` : ""}</figcaption>` : ""}
4429:   `;
4430:   return figure;
4431: }
```

createRichFormulaBlock (template-preview.js, lines 4433-4446)

createRichFormulaBlock creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4433-4446 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, unwrapFormula, richBlockActions, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4445: return block;.

Important local values include block at line 4434; align at line 4435. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4433: function createRichFormulaBlock(blockData) {
4434:   const block = document.createElement("div");
4435:   const align = blockData.align || "center";
4436:   block.className = `rich-block rich-formula-block justify-${align}`;
4437:   block.dataset.type = "formula";
4438:   block.dataset.formula = unwrapFormula(blockData.formula || "");
4439:   block.dataset.align = align;
4440:   block.contentEditable = "false";
4441:   block.innerHTML = `
4442:     ${richBlockActions("formula")}
4443:     <span class="formula-text">${escapeHtml(unwrapFormula(blockData.formula || ""))}</span>
4444:   `;
4445:   return block;
4446: }
```

createRichCodeBlock (template-preview.js, lines 4448-4465)

createRichCodeBlock creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4448-4465 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, normalizeCodePasteMode, normalizeCodeText, normalizeCodeLanguage, detectCodeLanguage, richBlockActions, escapeHtml, codeLanguageLabel, and tokenizedCodeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4464: return block;.

Important local values include block at line 4449; pasteMode at line 4450; code at line 4451; language at line 4452. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4448: function createRichCodeBlock(blockData = {}) {
4449:   const block = document.createElement("figure");
4450:   const pasteMode = normalizeCodePasteMode(blockData.pasteMode || "source");
4451:   const code = normalizeCodeText(blockData.code || "", pasteMode);
4452:   const language = normalizeCodeLanguage(blockData.language || detectCodeLanguage(code));
4453:   block.className = `rich-block rich-code-block rich-code-language-${language}`;
4454:   block.dataset.type = "code";
4455:   block.dataset.language = language;
4456:   block.dataset.pasteMode = pasteMode;
4457:   block.dataset.code = code;
4458:   block.contentEditable = "false";
4459:   block.innerHTML = `
4460:     ${richBlockActions("code block")}
4461:     <figcaption><span>&lt;&gt;</span></figcaption> ${escapeHtml(codeLanguageLabel(language))}<button type="button"
data-rich-block-action="copy-code">Copy code</button></figcaption>
4462:     <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
4463:   `;
4464:   return block;
4465: }
```

refreshRichImageBlock (template-preview.js, lines 4467-4497)

refreshRichImageBlock part of the private builder frontend and editing experience. In this file, it appears around lines 4467-4497 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, remove, add, cleanRichImageTitle, richBlockActions, richImageCropStyle, escapeHtml, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include align at line 4468; title at line 4483. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4467: function refreshRichImageBlock(block, blockData) {
4468:   const align = blockData.align || block.dataset.align || "center";
4469:   block.dataset.url = blockData.url || block.dataset.url || "";
```

```

4470:   block.dataset.title = blockData.title || "";
4471:   block.dataset.caption = blockData.caption || "";
4472:   block.dataset.align = align;
4473:   block.dataset.display = blockData.display === "download" ? "download" : "show";
4474:   block.dataset.source = blockData.source || block.dataset.source || "local";
4475:   block.dataset.cropAspect = normalizeCropAspect(blockData.cropAspect || block.dataset.cropAspect);
4476:   block.dataset.cropZoom = String(normalizeCropZoom(blockData.cropZoom || block.dataset.cropZoom));
4477:   block.dataset.cropX = String(normalizeCropPosition(blockData.cropX ?? block.dataset.cropX));
4478:   block.dataset.cropY = String(normalizeCropPosition(blockData.cropY ?? block.dataset.cropY));
4479:   block.draggable = true;
4480:   block.title = "Drag image to move it between text.";
4481:   block.classList.remove("justify-left", "justify-center", "justify-right");
4482:   block.classList.add(`justify-${align}`);
4483:   const title = cleanRichImageTitle({ title: block.dataset.title });
4484:   block.innerHTML = `
4485:     ${richBlockActions("overview image", { movable: true })}
4486:     ${block.dataset.display === "download" ? `<span class="rich-download-badge">Download only</span>` :
""}
4487:     <span class="rich-image-viewport crop-${block.dataset.cropAspect === "original" ? "original" :
"active"}">${richImageCropStyle({
4488:       cropAspect: block.dataset.cropAspect,
4489:       cropZoom: block.dataset.cropZoom,
4490:       cropX: block.dataset.cropX,
4491:       cropY: block.dataset.cropY
4492:     })}>
4493:       
4494:     </span>
4495:     ${!(title || block.dataset.caption) ? `<figcaption>${title ? `<strong>${escapeHtml(title)}</strong>` :
""}${block.dataset.caption ? `<span>${escapeHtml(block.dataset.caption)}</span>` : ""}</figcaption>` : ""}
4496:   `;
4497: }

```

refreshRichFormulaBlock (template-preview.js, lines 4499-4510)

refreshRichFormulaBlock part of the private builder frontend and editing experience. In this file, it appears around lines 4499-4510 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references unwrapFormula, remove, add, richBlockActions, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include align at line 4500; formula at line 4501. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

4499: function refreshRichFormulaBlock(block, blockData) {
4500:   const align = blockData.align || block.dataset.align || "center";
4501:   const formula = unwrapFormula(blockData.formula || block.dataset.formula || "");
4502:   block.dataset.formula = formula;
4503:   block.dataset.align = align;
4504:   block.classList.remove("justify-left", "justify-center", "justify-right");
4505:   block.classList.add(`justify-${align}`);
4506:   block.innerHTML = `
4507:     ${richBlockActions("formula")}
4508:     <span class="formula-text">${escapeHtml(formula)}</span>
4509:   `;
4510: }

```

refreshRichCodeBlock (template-preview.js, lines 4512-4525)

refreshRichCodeBlock part of the private builder frontend and editing experience. In this file, it appears around lines 4512-4525 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodePasteMode, normalizeCodeText, normalizeCodeLanguage, detectCodeLanguage, richBlockActions, escapeHtml, codeLanguageLabel, and tokenizedCodeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include pasteMode at line 4513; code at line 4514; language at line 4515. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

4512: function refreshRichCodeBlock(block, blockData = {}) {
4513:   const pasteMode = normalizeCodePasteMode(blockData.pasteMode || block.dataset.pasteMode || "source");
4514:   const code = normalizeCodeText(blockData.code || block.dataset.code || "", pasteMode);
4515:   const language = normalizeCodeLanguage(blockData.language || block.dataset.language ||
detectCodeLanguage(code));
4516:   block.dataset.language = language;
4517:   block.dataset.pasteMode = pasteMode;
4518:   block.dataset.code = code;
4519:   block.className = `rich-block rich-code-block rich-code-language-${language}`;
4520:   block.innerHTML = `
4521:     ${richBlockActions("code block")}
4522:     <figcaption><span>&lt;/span> ${escapeHtml(codeLanguageLabel(language))}</figcaption><button type="button"
data-rich-block-action="copy-code">Copy code</button></figcaption>
4523:     <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
4524:   `;
4525: }

```

createRichBlockElement (template-preview.js, lines 4527-4544)

createRichBlockElement creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 4527-4544 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createRichImageBlock, createRichFormulaBlock, createRichCodeBlock, and createRichTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4528: if (block.type === "image") return createRichImageBlock(block);; line 4529: if (block.type === "formula") return createRichFormulaBlock(block);; line 4530: if (block.type === "code") return createRichCodeBlock(block);; line 4532: return createRichTextBlock(

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

4527: function createRichBlockElement(block) {
4528:   if (block.type === "image") return createRichImageBlock(block);
4529:   if (block.type === "formula") return createRichFormulaBlock(block);
4530:   if (block.type === "code") return createRichCodeBlock(block);
4531:
4532:   return createRichTextBlock(
4533:     block.text || "",
4534:     block.fontSize || "normal",
4535:     block.align || "left",
4536:     block.fontFamily || "Arial",
4537:     block.fontPx || "",
4538:     block.color || "",
4539:     Boolean(block.bold),
4540:     block.html || "",
4541:     Boolean(block.italic),
4542:     Boolean(block.underline)
4543:   );
4544: }

```

populateSummaryEditor (template-preview.js, lines 4546-4550)

populateSummaryEditor part of the private builder frontend and editing experience. In this file, it appears around lines 4546-4550 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and populateRichEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4548: if (!editor) return;

Important local values include editor at line 4547. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

4546: function populateSummaryEditor(project) {
4547:   const editor = projectFields.querySelector("[data-rich-editor='summary']");
4548:   if (!editor) return;
4549:   populateRichEditor(editor, project);
4550: }

```


populateStandaloneRichEditor (template-preview.js, lines 4552-4560)

populateStandaloneRichEditor part of the private builder frontend and editing experience. In this file, it appears around lines 4552-4560 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richBlocksFromValue, forEach, append, createRichBlockElement, and createRichTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4553: if (!editor) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4552: function populateStandaloneRichEditor(editor, rich, fallbackText = "") {
4553:   if (!editor) return;
4554:   editor.contentEditable = "true";
4555:   editor.tabIndex = 0;
4556:   editor.spellcheck = true;
4557:   editor.innerHTML = "";
4558:   richBlocksFromValue(rich, fallbackText).forEach((block) =>
editor.append(createRichBlockElement(block)));
4559:   if (!editor.children.length) editor.append(createRichTextBlock(""));
4560: }
```

populateRichEditor (template-preview.js, lines 4562-4592)

This function reconstructs rich text editor blocks from saved data. It turns stored text, images, formulas, and code blocks back into editable DOM elements.

A saved project must reopen in an editable form without losing formatting. This function is the load side of the rich editor.

It calls or references selectedProject, richBlocksFromValue, forEach, append, createRichBlockElement, createRichTextBlock, richBlocksForProject, and getByPath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4563: if (!editor) return;; line 4573: return;; line 4576: if (!project) return;.

Important local values include blocks at line 4580; textPath at line 4585; richPath at line 4586. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4562: function populateRichEditor(editor, project = selectedProject()) {
4563:   if (!editor) return;
4564:   editor.contentEditable = "true";
4565:   editor.tabIndex = 0;
4566:   editor.spellcheck = true;
4567:
4568:   if (editor.dataset.richEditor === "pending-description") {
4569:     editor.innerHTML = "";
4570:     richBlocksFromValue(pendingEditor?.richDescription, pendingEditor?.description || "")
      .forEach((block) => editor.append(createRichBlockElement(block)));
4571:     if (!editor.children.length) editor.append(createRichTextBlock(""));
4572:     return;
4573:   }
4574:
4575:   if (!project) return;
4576:
4577:   editor.innerHTML = "";
4578:
4579:   let blocks = [];
4580:
4581:   if (editor.dataset.richEditor === "summary" && !editor.dataset.richPath) {
4582:     blocks = richBlocksForProject(project);
4583:   } else {
4584:     const textPath = editor.dataset.richTextPath || "";
4585:     const richPath = editor.dataset.richPath || "";
4586:     blocks = richBlocksFromValue(getByPath(project, richPath), getByPath(project, textPath) || "");
4587:   }
4588:
4589:   blocks.forEach((block) => editor.append(createRichBlockElement(block)));
4590:   if (!editor.children.length) editor.append(createRichTextBlock(""));
4591:
4592: }
```

populateRichEditors (template-preview.js, lines 4594-4596)

populateRichEditors part of the private builder frontend and editing experience. In this file, it appears around lines 4594-4596 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, and populateRichEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4594: function populateRichEditors(root = document) {
4595:     root.querySelectorAll("[data-rich-editor]").forEach((editor) => populateRichEditor(editor));
4596: }
```

saveRichEditorToProject (template-preview.js, lines 4598-4658)

This function serializes rich editor DOM blocks back into structured project data. It extracts text formatting, image data, captions, formulas, code blocks, and ordering.

The parser and previews cannot rely on raw contenteditable HTML. This function turns the user's editing surface into predictable data.

It calls or references syncFunFactsFromInput, setStatus, syncSiteRichField, syncProfileRichField, selectedProject, extractRichSummary, plainTextFromRich, wordCount, setByPath, scheduleAutosave, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 4599: if (!editor) return true;; line 4604: return true;; line 4610: return true;; line 4616: return true;. There are 5 additional return statements in the function body.

Important local values include project at line 4624; rich at line 4627; rich at line 4636; plain at line 4637; textPath at line 4648; richPath at line 4649. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4598: function saveRichEditorToProject(editor) {
4599:     if (!editor) return true;
4600:
4601:     if (editor.dataset.richEditor === "fun-facts") {
4602:         syncFunFactsFromInput();
4603:         setStatus("Unsaved local changes.");
4604:         return true;
4605:     }
4606:
4607:     if (editor.dataset.richEditor === "site-field") {
4608:         syncSiteRichField(editor);
4609:         setStatus("Front page text formatting updated locally. Click Save draft before applying to
site.");
4610:         return true;
4611:     }
4612:
4613:     if (editor.dataset.richEditor === "profile-field") {
4614:         syncProfileRichField(editor);
4615:         setStatus("Profile and contact formatting updated locally. Click Save draft before applying to
site.");
4616:         return true;
4617:     }
4618:
4619:     if (editor.dataset.richEditor === "standalone") {
4620:         setStatus("Unsaved dialog changes. Use the dialog save button to keep them.");
4621:         return true;
4622:     }
4623:
4624:     const project = selectedProject();
4625:
4626:     if (editor.dataset.richEditor === "pending-description" && pendingEditor) {
4627:         const rich = extractRichSummary(editor);
4628:         pendingEditor.richDescription = rich;
4629:         pendingEditor.description = plainTextFromRich(rich);
4630:         setStatus("Unsaved local changes.");
4631:         return true;
4632:     }
4633:
4634:     if (!project) return true;
4635:
4636:     const rich = extractRichSummary(editor);
4637:     const plain = plainTextFromRich(rich);
4638:
4639:     if (wordCount(plain) > 1000) {
4640:         setStatus("Overview is limited to 1000 words. Shorten the text before saving.");
4641:         return false;
4642:     }
4643: }
```

```

4642:     }
4643:
4644:     if (editor.dataset.richEditor === "summary" && !editor.dataset.richPath) {
4645:         project.summaryRich = rich;
4646:         project.summary = plain;
4647:     } else {
4648:         const textPath = editor.dataset.richTextPath;
4649:         const richPath = editor.dataset.richPath;
4650:         if (textPath) setByPath(project, textPath, plain);
4651:         if (richPath) setByPath(project, richPath, rich);
4652:     }
4653:
4654:     setStatus("Unsaved local changes.");
4655:     scheduleAutosave();
4656:     schedulePreviewRender();
4657:     return true;
4658: }

```

saveVisibleRichEditors (template-preview.js, lines 4660-4664)

saveVisibleRichEditors persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 4660-4664 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, and saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

4660: function saveVisibleRichEditors() {
4661:     document.querySelectorAll("[data-rich-editor]").forEach((editor) => {
4662:         saveRichEditorToProject(editor);
4663:     });
4664: }

```

currentRichBlock (template-preview.js, lines 4666-4672)

currentRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 4666-4672 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, contains, querySelector, and closest. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4669: if (!node || !editor.contains(node)) return editor.querySelector(".rich-block:last-child");; line 4671: return node.closest(".rich-block") || editor.querySelector(".rich-block:last-child");.

Important local values include selection at line 4667; node at line 4668. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

4666: function currentRichBlock(editor) {
4667:     const selection = window.getSelection();
4668:     let node = selection?.anchorNode;
4669:     if (!node || !editor.contains(node)) return editor.querySelector(".rich-block:last-child");
4670:     if (node.nodeType === Node.TEXT_NODE) node = node.parentElement;
4671:     return node.closest(".rich-block") || editor.querySelector(".rich-block:last-child");
4672: }

```

selectionRangeInsideEditor (template-preview.js, lines 4674-4682)

selectionRangeInsideEditor part of the private builder frontend and editing experience. In this file, it appears around lines 4674-4682 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, getRangeAt, contains, and cloneRange. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4676: if (!selection?.rangeCount) return null;; line 4681: return container && editor.contains(container) ? range.cloneRange() : null;.

Important local values include selection at line 4675; range at line 4677; container at line 4678. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4674: function selectionRangeInsideEditor(editor) {
4675:   const selection = window.getSelection();
4676:   if (!selection?.rangeCount) return null;
4677:   const range = selection.getRangeAt(0);
4678:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4679:     ? range.commonAncestorContainer.parentElement
4680:     : range.commonAncestorContainer;
4681:   return container && editor.contains(container) ? range.cloneRange() : null;
4682: }
```

captureRichSelection (template-preview.js, lines 4684-4688)

captureRichSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4684-4688 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectionRangeInsideEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4687: return range;.

Important local values include range at line 4685. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4684: function captureRichSelection(editor) {
4685:   const range = selectionRangeInsideEditor(editor);
4686:   if (range) activeRichSelectionRange = range;
4687:   return range;
4688: }
```

rememberRichFormattingSelection (template-preview.js, lines 4690-4700)

rememberRichFormattingSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4690-4700 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectionRangeInsideEditor, rangeBelongsToEditor, and cloneRange. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4691: if (!editor) return null;; line 4695: if (!range) return null;; line 4699: return range;.

Important local values include range at line 4692. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4690: function rememberRichFormattingSelection(editor = activeSummaryEditor) {
4691:   if (!editor) return null;
4692:   const range = selectionRangeInsideEditor(editor)
4693:     || (rangeBelongsToEditor(activeTextSelectionRange, editor) ? activeTextSelectionRange.cloneRange() :
null)
4694:     || (rangeBelongsToEditor(activeRichSelectionRange, editor) ? activeRichSelectionRange.cloneRange() :
null);
4695:   if (!range) return null;
4696:   activeSummaryEditor = editor;
4697:   activeRichSelectionRange = range.cloneRange();
4698:   if (!range.collapsed) activeTextSelectionRange = range.cloneRange();
4699:   return range;
4700: }
```

restoreRichSelection (template-preview.js, lines 4702-4713)

restoreRichSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4702-4713 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, getSelection, removeAllRanges, addRange, cloneRange, and showPersistentTextSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4703: if (!activeRichSelectionRange) return false;; line 4707: if (!container || !editor.contains(container)) return false;; line 4712: return true;;

Important local values include container at line 4704; selection at line 4708. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4702: function restoreRichSelection(editor) {
4703:   if (!activeRichSelectionRange) return false;
4704:   const container = activeRichSelectionRange.commonAncestorContainer.nodeType === Node.TEXT_NODE
4705:     ? activeRichSelectionRange.commonAncestorContainer.parentElement
4706:     : activeRichSelectionRange.commonAncestorContainer;
4707:   if (!container || !editor.contains(container)) return false;
4708:   const selection = window.getSelection();
4709:   selection.removeAllRanges();
4710:   selection.addRange(activeRichSelectionRange.cloneRange());
4711:   showPersistentTextSelection(activeRichSelectionRange);
4712:   return true;
4713: }
```

showPersistentTextSelection (template-preview.js, lines 4715-4745)

showPersistentTextSelection controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4715-4745 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references set, Highlight, cloneRange, closest, createElement, setAttribute, append, replaceChildren, getClientRects, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4716: if (!range || range.collapsed) return;; line 4720: return;; line 4736: if (rect.width < 1 || rect.height < 1) return;;

Important local values include container at line 4723; editor at line 4726; host at line 4727; marker at line 4737. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4715: function showPersistentTextSelection(range) {
4716:   if (!range || range.collapsed) return;
4717:   if (window.CSS?.highlights && typeof window.Highlight === "function") {
4718:     window.CSS.highlights.set(persistentSelectionHighlightName, new
window.Highlight(range.cloneRange()));
4719:     if (persistentSelectionOverlay) persistentSelectionOverlay.hidden = true;
4720:     return;
4721:   }
4722:
4723:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4724:     ? range.commonAncestorContainer.parentElement
4725:     : range.commonAncestorContainer;
4726:   const editor = container?.closest?("[data-rich-editor]") || activeSummaryEditor;
4727:   const host = editor?.closest("dialog") || document.body;
4728:   if (!persistentSelectionOverlay) {
4729:     persistentSelectionOverlay = document.createElement("div");
4730:     persistentSelectionOverlay.className = "persistent-selection-overlay";
4731:     persistentSelectionOverlay.setAttribute("aria-hidden", "true");
4732:   }
4733:   if (persistentSelectionOverlay.parentElement !== host) host.append(persistentSelectionOverlay);
4734:   persistentSelectionOverlay.replaceChildren();
4735:   [...range.getClientRects()].forEach((rect) => {
4736:     if (rect.width < 1 || rect.height < 1) return;
4737:     const marker = document.createElement("span");
4738:     marker.style.left = `${rect.left}px`;
4739:     marker.style.top = `${rect.top}px`;
4740:     marker.style.width = `${rect.width}px`;
4741:     marker.style.height = `${rect.height}px`;
4742:     persistentSelectionOverlay.append(marker);
4743:   });
4744:   persistentSelectionOverlay.hidden = !persistentSelectionOverlay.childElementCount;
4745: }
```

clearPersistentTextSelection (template-preview.js, lines 4747-4756)

clearPersistentTextSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4747-4756 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceChildren. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4753: if (!clearRanges) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4747: function clearPersistentTextSelection(clearRanges = false) {
4748:   window.CSS?.highlights?.delete?.(persistentSelectionHighlightName);
4749:   if (persistentSelectionOverlay) {
4750:     persistentSelectionOverlay.replaceChildren();
4751:     persistentSelectionOverlay.hidden = true;
4752:   }
4753:   if (!clearRanges) return;
4754:   activeRichSelectionRange = null;
4755:   activeTextSelectionRange = null;
4756: }
```

refreshPersistentSelectionHighlight (template-preview.js, lines 4758-4765)

refreshPersistentSelectionHighlight part of the private builder frontend and editing experience. In this file, it appears around lines 4758-4765 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references rangeBelongsToEditor, and showPersistentTextSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include range at line 4759. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4758: function refreshPersistentSelectionHighlight() {
4759:   const range = rangeBelongsToEditor(activeTextSelectionRange, activeSummaryEditor)
4760:     ? activeTextSelectionRange
4761:     : rangeBelongsToEditor(activeRichSelectionRange, activeSummaryEditor)
4762:       ? activeRichSelectionRange
4763:       : null;
4764:   if (range) showPersistentTextSelection(range);
4765: }
```

rangeBelongsToEditor (template-preview.js, lines 4767-4773)

rangeBelongsToEditor part of the private builder frontend and editing experience. In this file, it appears around lines 4767-4773 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4768: if (!range || range.collapsed || !editor) return false;; line 4772: return Boolean(container && editor.contains(container));.

Important local values include container at line 4769. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4767: function rangeBelongsToEditor(range, editor) {
4768:   if (!range || range.collapsed || !editor) return false;
4769:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4770:     ? range.commonAncestorContainer.parentElement
```

```

4771:         : range.commonAncestorContainer;
4772:     return Boolean(container && editor.contains(container));
4773: }

```

editorFromRange (template-preview.js, lines 4775-4781)

editorFromRange part of the private builder frontend and editing experience. In this file, it appears around lines 4775-4781 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4776: if (!range || range.collapsed) return null;; line 4780: return container?.closest?("[data-rich-editor]") || null;.

Important local values include container at line 4777. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

4775: function editorFromRange(range) {
4776:   if (!range || range.collapsed) return null;
4777:   const container = range.commonAncestorContainer.nodeType === Node.TEXT_NODE
4778:     ? range.commonAncestorContainer.parentElement
4779:     : range.commonAncestorContainer;
4780:   return container?.closest?("[data-rich-editor]") || null;
4781: }

```

pointInsideRange (template-preview.js, lines 4783-4793)

pointInsideRange part of the private builder frontend and editing experience. In this file, it appears around lines 4783-4793 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getClientRects, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4784: if (!range || range.collapsed) return false;; line 4785: return [...range.getClientRects()].some((rect) =>.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

4783: function pointInsideRange(range, x, y) {
4784:   if (!range || range.collapsed) return false;
4785:   return [...range.getClientRects()].some((rect) =>
4786:     rect.width > 0 &&
4787:     rect.height > 0 &&
4788:     x >= rect.left &&
4789:     x <= rect.right &&
4790:     y >= rect.top &&
4791:     y <= rect.bottom
4792:   );
4793: }

```

richEditorFromContextEvent (template-preview.js, lines 4795-4805)

richEditorFromContextEvent part of the private builder frontend and editing experience. In this file, it appears around lines 4795-4805 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references rangeBelongsToEditor, pointInsideRange, and editorFromRange. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 4797: if (directEditor) return directEditor;; line 4802: return editorFromRange(savedRange);; line 4804: return null;.

Important local values include directEditor at line 4796; savedRange at line 4798. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4795: function richEditorFromContextEvent(event) {
4796:   const directEditor = event.target.closest?("[data-rich-editor]");
4797:   if (directEditor) return directEditor;
4798:   const savedRange = rangeBelongsToEditor(activeTextSelectionRange, activeSummaryEditor)
4799:     ? activeTextSelectionRange
4800:     : activeRichSelectionRange;
4801:   if (pointInsideRange(savedRange, event.clientX, event.clientY)) {
4802:     return editorFromRange(savedRange);
4803:   }
4804:   return null;
4805: }
```

displayRichFontName (template-preview.js, lines 4807-4812)

displayRichFontName part of the private builder frontend and editing experience. In this file, it appears around lines 4807-4812 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, replace, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4808: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4807: function displayRichFontName(value = "") {
4808:   return String(value || "")
4809:     .split(",")[0]
4810:     .replace(/^[\'\\"]|['\"]$/g, "")
4811:     .trim() || "Arial";
4812: }
```

rgbColorToHex (template-preview.js, lines 4814-4823)

rgbColorToHex part of the private builder frontend and editing experience. In this file, it appears around lines 4814-4823 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, slice, split, map, join, match, min, toString, and padStart. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4816: if (/^[0-9a-f]{6}\$/i.test(color)) return color.toLowerCase();; line 4818: return `\${color.slice(1).split("").map((part) => part + part).join("")}`.toLowerCase();; line 4821: if (!match) return color;; line 4822: return `\${match.slice(1, 4).map((part) => Math.min(255, Number(part)).toString(16).padStart(2, "0")).join("")}`;.

Important local values include color at line 4815; match at line 4820. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4814: function rgbColorToHex(value = "") {
4815:   const color = String(value || "").trim();
4816:   if (/^[0-9a-f]{6}$/i.test(color)) return color.toLowerCase();
4817:   if (/^[0-9a-f]{3}$/i.test(color)) {
4818:     return `${color.slice(1).split("").map((part) => part + part).join("")}`.toLowerCase();
4819:   }
4820:   const match = color.match(/^rgba?\s*(\d+)\D+(\d+)\D+(\d+)\D+/i);
4821:   if (!match) return color;
4822:   return `${match.slice(1, 4).map((part) => Math.min(255, Number(part)).toString(16).padStart(2,
"0")).join("")}`;
4823: }
```

textNodesInRichSelection (template-preview.js, lines 4825-4844)

textNodesInRichSelection part of the private builder frontend and editing experience. In this file, it appears around lines 4825-4844 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createTreeWalker, acceptNode, trim, closest, intersectsNode, nextNode, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 4828: if (!node.textContent || (!includeWhitespace && !node.textContent.trim())) return NodeFilter.FILTER_REJECT;; line 4829: if (node.parentElement?.closest("[contenteditable='false']")) return NodeFilter.FILTER_REJECT;; line 4831: return range.intersectsNode(node) ? NodeFilter.FILTER_ACCEPT : NodeFilter.FILTER_REJECT;; line 4833: return NodeFilter.FILTER_REJECT;. There are 1 additional return statements in the function body.

Important local values include walker at line 4826; nodes at line 4837; node at line 4838. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4825: function textNodesInRichSelection(range, editor, includeWhitespace = false) {
4826:   const walker = document.createTreeWalker(editor, NodeFilter.SHOW_TEXT, {
4827:     acceptNode(node) {
4828:       if (!node.textContent || (!includeWhitespace && !node.textContent.trim())) return
NodeFilter.FILTER_REJECT;
4829:       if (node.parentElement?.closest("[contenteditable='false']")) return NodeFilter.FILTER_REJECT;
4830:       try {
4831:         return range.intersectsNode(node) ? NodeFilter.FILTER_ACCEPT : NodeFilter.FILTER_REJECT;
4832:       } catch {
4833:         return NodeFilter.FILTER_REJECT;
4834:       }
4835:     }
4836:   });
4837:   const nodes = [];
4838:   let node = walker.nextNode();
4839:   while (node) {
4840:     nodes.push(node);
4841:     node = walker.nextNode();
4842:   }
4843:   return nodes;
4844: }
```

richBlockFromRange (template-preview.js, lines 4846-4853)

richBlockFromRange part of the private builder frontend and editing experience. In this file, it appears around lines 4846-4853 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4847: if (!range || !editor) return null;; line 4852: return block && editor.contains(block) ? block : null;.

Important local values include container at line 4848; block at line 4851. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4846: function richBlockFromRange(range, editor) {
4847:   if (!range || !editor) return null;
4848:   const container = range.startContainer?.nodeType === Node.TEXT_NODE
4849:     ? range.startContainer.parentElement
4850:     : range.startContainer;
4851:   const block = container?.closest?(".rich-block");
4852:   return block && editor.contains(block) ? block : null;
4853: }
```

uniformSelectionValue (template-preview.js, lines 4855-4861)

uniformSelectionValue part of the private builder frontend and editing experience. In this file, it appears around lines 4855-4861 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, trim, filter, every, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4857: if (!cleanValues.length) return "";; line 4858: return cleanValues.every((value) => value.toLowerCase() === cleanValues[0].toLowerCase()).

Important local values include `cleanValues` at line 4856. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4855: function uniformSelectionValue(values) {
4856:   const cleanValues = values.map((value) => String(value || "").trim()).filter(Boolean);
4857:   if (!cleanValues.length) return "";
4858:   return cleanValues.every((value) => value.toLowerCase() === cleanValues[0].toLowerCase())
4859:     ? cleanValues[0]
4860:     : "";
4861: }
```

richSelectionStyleState (template-preview.js, lines 4863-4871)

`richSelectionStyleState` part of the private builder frontend and editing experience. In this file, it appears around lines 4863-4871 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `textNodesInRichSelection`, `map`, `getComputedStyle`, `uniformSelectionValue`, `rgbColorToHex`, `displayRichFontName`, and `round`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4866: `return {`.

Important local values include `textNodes` at line 4864; `styles` at line 4865. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4863: function richSelectionStyleState(range, editor) {
4864:   const textNodes = textNodesInRichSelection(range, editor);
4865:   const styles = textNodes.map((node) => getComputedStyle(node.parentElement || editor));
4866:   return {
4867:     color: uniformSelectionValue(styles.map((style) => rgbColorToHex(style.color))),
4868:     font: uniformSelectionValue(styles.map((style) => displayRichFontName(style.fontFamily))),
4869:     size: uniformSelectionValue(styles.map((style) => String(Math.round(parseFloat(style.fontSize) ||
16))))
4870:   };
4871: }
```

closeSelectionInspectorOptions (template-preview.js, lines 4873-4877)

`closeSelectionInspectorOptions` controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4873-4877 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `querySelectorAll`, and `forEach`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4873: function closeSelectionInspectorOptions(except = "") {
4874:   textSelectionInspector?.querySelectorAll("[data-selection-options]").forEach((options) => {
4875:     if (options.dataset.selectionOptions !== except) options.hidden = true;
4876:   });
4877: }
```

positionSelectionInspector (template-preview.js, lines 4879-4900)

`positionSelectionInspector` part of the private builder frontend and editing experience. In this file, it appears around lines 4879-4900 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `getBoundingClientRect`, `closest`, `min`, and `max`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4880: `if (!textSelectionInspector || selectionInspectorWasMoved) return;`

Important local values include `rect` at line 4881; `hostDialog` at line 4882; `hostRect` at line 4883; `bounds` at line 4884; `inspectorRect` at line 4890; `inspectorWidth` at line 4891; `inspectorHeight` at line 4892; `left` at line 4893; plus 2 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4879: function positionSelectionInspector(range) {
4880:   if (!textSelectionInspector || selectionInspectorWasMoved) return;
4881:   const rect = range.getBoundingClientRect();
4882:   const hostDialog = textSelectionInspector.closest("dialog");
4883:   const hostRect = hostDialog?.getBoundingClientRect();
4884:   const bounds = {
4885:     bottom: Math.min(window.innerHeight - 8, (hostRect?.bottom ?? window.innerHeight) - 8),
4886:     left: Math.max(8, (hostRect?.left ?? 0) + 8),
4887:     right: Math.min(window.innerWidth - 8, (hostRect?.right ?? window.innerWidth) - 8),
4888:     top: Math.max(8, (hostRect?.top ?? 0) + 8)
4889:   };
4890:   const inspectorRect = textSelectionInspector.getBoundingClientRect();
4891:   const inspectorWidth = Math.min(inspectorRect.width || 310, bounds.right - bounds.left);
4892:   const inspectorHeight = Math.min(inspectorRect.height || 260, bounds.bottom - bounds.top);
4893:   const left = Math.max(bounds.left, Math.min(rect.left, bounds.right - inspectorWidth));
4894:   const preferredTop = rect.bottom + 10;
4895:   const top = preferredTop + inspectorHeight <= bounds.bottom
4896:     ? preferredTop
4897:     : Math.max(bounds.top, rect.top - inspectorHeight - 10);
4898:   textSelectionInspector.style.left = `${left}px`;
4899:   textSelectionInspector.style.top = `${top}px`;
4900: }
```

showTextSelectionInspector (template-preview.js, lines 4902-4924)

`showTextSelectionInspector` controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4902-4924 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references to `String`, `trim`, `closest`, `append`, `cloneRange`, `showPersistentTextSelection`, `richSelectionStyleState`, `slice`, and `positionSelectionInspector`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 4903: `if (!textSelectionInspector || !editor || !range || range.collapsed) return;`; line 4905: `if (!selectedText) return;`

Important local values include `selectedText` at line 4904; `inspectorHost` at line 4906; `state` at line 4912. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4902: function showTextSelectionInspector(editor, range) {
4903:   if (!textSelectionInspector || !editor || !range || range.collapsed) return;
4904:   const selectedText = range.toString().trim();
4905:   if (!selectedText) return;
4906:   const inspectorHost = editor.closest("dialog") || document.body;
4907:   if (textSelectionInspector.parentElement !== inspectorHost)
inspectorHost.append(textSelectionInspector);
4908:   activeSummaryEditor = editor;
4909:   activeRichSelectionRange = range.cloneRange();
4910:   activeTextSelectionRange = range.cloneRange();
4911:   showPersistentTextSelection(range);
4912:   const state = richSelectionStyleState(range, editor);
4913:   selectionTextPreview.textContent = selectedText.length > 120 ? `${selectedText.slice(0, 117)}...` :
selectedText;
4914:   selectionTextPreview.title = selectedText;
4915:   selectionFontInput.value = state.font;
4916:   selectionColorInput.value = state.color;
4917:   selectionSizeInput.value = state.size;
4918:   selectionFontInput.placeholder = state.font ? "" : "Mixed";
4919:   selectionColorInput.placeholder = state.color ? "" : "Mixed";
4920:   selectionSizeInput.placeholder = state.size ? "" : "Mixed";
4921:   if (state.color && /^[0-9a-f]{6}$/i.test(state.color)) selectionColorPicker.value = state.color;
4922:   textSelectionInspector.hidden = false;
4923:   positionSelectionInspector(range);
4924: }
```

hideTextSelectionInspector (template-preview.js, lines 4926-4931)

`hideTextSelectionInspector` controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 4926-4931 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closeSelectionInspectorOptions. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 4927: if (!textSelectionInspector) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
4926: function hideTextSelectionInspector() {
4927:   if (!textSelectionInspector) return;
4928:   textSelectionInspector.hidden = true;
4929:   closeSelectionInspectorOptions();
4930:   selectionInspectorWasMoved = false;
4931: }
```

refreshTextSelectionInspector (template-preview.js, lines 4933-4953)

refreshTextSelectionInspector part of the private builder frontend and editing experience. In this file, it appears around lines 4933-4953 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references matches, getSelection, showTextSelectionInspector, hideTextSelectionInspector, and getRangeAt. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4934: if (!textSelectionInspector || selectionInspectorPointerInside || textSelectionInspector.matches(":focus-within")) return;; line 4939: return;; line 4942: return;; line 4950: return;.

Important local values include selection at line 4935; range at line 4944; node at line 4945; editor at line 4947. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
4933: function refreshTextSelectionInspector() {
4934:   if (!textSelectionInspector || selectionInspectorPointerInside ||
textSelectionInspector.matches(":focus-within")) return;
4935:   const selection = window.getSelection();
4936:   if (!selection?.rangeCount || selection.isCollapsed) {
4937:     if (activeTextSelectionRange && !activeTextSelectionRange.collapsed && activeSummaryEditor) {
4938:       showTextSelectionInspector(activeSummaryEditor, activeTextSelectionRange);
4939:       return;
4940:     }
4941:     hideTextSelectionInspector();
4942:     return;
4943:   }
4944:   const range = selection.getRangeAt(0);
4945:   let node = range.commonAncestorContainer;
4946:   if (node.nodeType === Node.TEXT_NODE) node = node.parentElement;
4947:   const editor = node?.closest?("[data-rich-editor]");
4948:   if (!editor) {
4949:     hideTextSelectionInspector();
4950:     return;
4951:   }
4952:   showTextSelectionInspector(editor, range);
4953: }
```

scheduleTextSelectionInspectorRefresh (template-preview.js, lines 4955-4958)

scheduleTextSelectionInspectorRefresh part of the private builder frontend and editing experience. In this file, it appears around lines 4955-4958 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

4955: function scheduleTextSelectionInspectorRefresh(delay = 0) {
4956:   clearTimeout(selectionInspectorRefreshTimer);
4957:   selectionInspectorRefreshTimer = setTimeout(refreshTextSelectionInspector, delay);
4958: }

```

applySelectionInspectorFormat (template-preview.js, lines 4960-4990)

This function applies formatting chosen in the floating selection inspector to the currently highlighted text.

It is what makes font, size, and color controls actually affect only the selected text instead of changing a whole block.

It calls or references cloneRange, cleanFontFamily, applyRichInlineCommand, normalizeTextColor, rgbColorToHex, normalizeFontPx, closeSelectionInspectorOptions, requestAnimationFrame, and showTextSelectionInspector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 4961: if (!activeSummaryEditor || !activeTextSelectionRange) return;; line 4965: if (!font) return;; line 4971: if (!color) return;; line 4979: if (!size) return;.

Important local values include font at line 4964; color at line 4970; normalized at line 4972; size at line 4978. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

4960: function applySelectionInspectorFormat(kind, rawValue) {
4961:   if (!activeSummaryEditor || !activeTextSelectionRange) return;
4962:   activeRichSelectionRange = activeTextSelectionRange.cloneRange();
4963:   if (kind === "font") {
4964:     const font = cleanFontFamily(rawValue);
4965:     if (!font) return;
4966:     selectionFontInput.value = font;
4967:     applyRichInlineCommand(activeSummaryEditor, "fontName", font);
4968:   }
4969:   if (kind === "color") {
4970:     const color = normalizeTextColor(rawValue);
4971:     if (!color) return;
4972:     const normalized = rgbColorToHex(color);
4973:     selectionColorInput.value = normalized;
4974:     if (/^[0-9a-f]{6}$/i.test(normalized)) selectionColorPicker.value = normalized;
4975:     applyRichInlineCommand(activeSummaryEditor, "foreColor", normalized);
4976:   }
4977:   if (kind === "size") {
4978:     const size = normalizeFontPx(rawValue);
4979:     if (!size) return;
4980:     selectionSizeInput.value = size;
4981:     applyRichInlineCommand(activeSummaryEditor, "fontSize", size);
4982:   }
4983:   closeSelectionInspectorOptions();
4984:   requestAnimationFrame(() => {
4985:     if (activeRichSelectionRange && activeSummaryEditor) {
4986:       activeTextSelectionRange = activeRichSelectionRange.cloneRange();
4987:       showTextSelectionInspector(activeSummaryEditor, activeRichSelectionRange);
4988:     }
4989:   });
4990: }

```

renderSelectionInspectorOptions (template-preview.js, lines 4992-5007)

renderSelectionInspectorOptions renders ui, markup, or display output. In this file, it appears around lines 4992-5007 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include fontOptions at line 4993; sizeOptions at line 4994. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

4992: function renderSelectionInspectorOptions() {
4993:   const fontOptions = textSelectionInspector?.querySelector("[data-selection-options='font']");
4994:   const sizeOptions = textSelectionInspector?.querySelector("[data-selection-options='size']");
4995:   if (fontOptions) {
4996:     fontOptions.innerHTML = commonRichFonts.map((font) => `<button type="button" data-selection-
value="font" data-value="${escapeHtml(font)}" style="font-family:
${escapeHtml(font)}">${escapeHtml(font)}</button>`).join("");
4997:   }
4998:   if (sizeOptions) {
4999:     sizeOptions.innerHTML = commonRichSizes.map((size) => `<button type="button" data-selection-

```

```

value="size" data-value="${size}">${size} px</button>`).join("");
5000:   }
5001:   if (selectionColorSwatches) {
5002:     selectionColorSwatches.innerHTML = commonRichColors.map((color) => `<button type="button" data-
selection-value="color" data-value="${color}" style="--selection-swatch: ${color}" aria-label="Use
${color}"></button>`).join("");
5003:   }
5004:   if (richContextColorSwatches) {
5005:     richContextColorSwatches.innerHTML = commonRichColors.map((color) => `<button type="button" data-
rich-color-value="${color}" style="--selection-swatch: ${color}" aria-label="Use ${color}"></button>`).join("");
5006:   }
5007: }

```

applyStyleToRichSelection (template-preview.js, lines 5009-5055)

applyStyleToRichSelection part of the private builder frontend and editing experience. In this file, it appears around lines 5009-5055 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textNodesInRichSelection, every, getComputedStyle, includes, forEach, max, min, splitText, createElement, cleanFontFamily, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5011: if (!nodes.length) return null;; line 5029: if (start === end) return;; line 5047: if (!formattedNodes.length) return null;; line 5054: return nextRange;.

Important local values include nodes at line 5010; makeBold at line 5012; makeItalic at line 5015; makeUnderline at line 5018; formattedNodes at line 5021; length at line 5024; start at line 5025; end at line 5026; plus 5 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5009: function applyStyleToRichSelection(editor, range, command, value = "") {
5010:   const nodes = textNodesInRichSelection(range, editor, true);
5011:   if (!nodes.length) return null;
5012:   const makeBold = command === "bold"
5013:     ? !nodes.every((node) => Number.parseInt(getComputedStyle(node.parentElement || editor).fontWeight,
10) >= 600)
5014:     : false;
5015:   const makeItalic = command === "italic"
5016:     ? !nodes.every((node) => getComputedStyle(node.parentElement || editor).fontStyle === "italic")
5017:     : false;
5018:   const makeUnderline = command === "underline"
5019:     ? !nodes.every((node) => getComputedStyle(node.parentElement ||
editor).textDecorationLine.includes("underline"))
5020:     : false;
5021:   const formattedNodes = [];
5022:
5023:   nodes.forEach((node) => {
5024:     const length = node.textContent?.length || 0;
5025:     let start = node === range.startContainer ? range.startOffset : 0;
5026:     let end = node === range.endContainer ? range.endOffset : length;
5027:     start = Math.max(0, Math.min(start, length));
5028:     end = Math.max(start, Math.min(end, length));
5029:     if (start === end) return;
5030:
5031:     if (end < length) node.splitText(end);
5032:     const selectedNode = start > 0 ? node.splitText(start) : node;
5033:     const wrapper = document.createElement("span");
5034:     wrapper.className = "rich-inline-style";
5035:     wrapper.style.display = "inline";
5036:     if (command === "fontName") wrapper.style.fontFamily = cleanFontFamily(value);
5037:     if (command === "foreColor") wrapper.style.color = normalizeTextColor(value);
5038:     if (command === "fontSize") wrapper.style.fontSize = `${normalizeFontPx(value)}px`;
5039:     if (command === "bold") wrapper.style.fontWeight = makeBold ? "700" : "400";
5040:     if (command === "italic") wrapper.style.fontStyle = makeItalic ? "italic" : "normal";
5041:     if (command === "underline") wrapper.style.textDecoration = makeUnderline ? "underline" : "none";
5042:     selectedNode.replaceWith(wrapper);
5043:     wrapper.append(selectedNode);
5044:     formattedNodes.push(selectedNode);
5045:   });
5046:
5047:   if (!formattedNodes.length) return null;
5048:   normalizeInlineStyleSpans(editor);
5049:   const nextRange = document.createRange();
5050:   const firstNode = formattedNodes[0];
5051:   const lastNode = formattedNodes[formattedNodes.length - 1];
5052:   nextRange.setStart(firstNode, 0);
5053:   nextRange.setEnd(lastNode, lastNode.textContent?.length || 0);
5054:   return nextRange;
5055: }

```

applyRichInlineCommand (template-preview.js, lines 5057-5103)

applyRichInlineCommand part of the private builder frontend and editing experience. In this file, it appears around lines 5057-5103 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectionRangeInsideEditor, cloneRange, rangeBelongsToEditor, applyStyleToRichSelection, showPersistentTextSelection, richBlockFromRange, selectedRichBlock, currentRichBlock, setStatus, selectRichBlock, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5058: if (!editor) return;; line 5083: return;; line 5098: return;;.

Important local values include liveRange at line 5059; savedRange at line 5060; appliedToSelection at line 5068; nextRange at line 5070; block at line 5080. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5057: function applyRichInlineCommand(editor, command, value = "") {
5058:   if (!editor) return;
5059:   const liveRange = selectionRangeInsideEditor(editor);
5060:   const savedRange = liveRange && !liveRange.collapsed
5061:     ? liveRange.cloneRange()
5062:     : rangeBelongsToEditor(activeTextSelectionRange, editor)
5063:       ? activeTextSelectionRange.cloneRange()
5064:       : rangeBelongsToEditor(activeRichSelectionRange, editor)
5065:         ? activeRichSelectionRange.cloneRange()
5066:         : null;
5067:
5068:   let appliedToSelection = false;
5069:   if (savedRange) {
5070:     const nextRange = applyStyleToRichSelection(editor, savedRange, command, value);
5071:     if (nextRange) {
5072:       activeRichSelectionRange = nextRange.cloneRange();
5073:       activeTextSelectionRange = nextRange.cloneRange();
5074:       showPersistentTextSelection(nextRange);
5075:       appliedToSelection = true;
5076:     }
5077:   }
5078:
5079:   if (!appliedToSelection) {
5080:     const block = richBlockFromRange(savedRange, editor) || selectedRichBlock(editor) ||
currentRichBlock(editor);
5081:     if (!block || block.dataset.type !== "paragraph") {
5082:       setStatus("Highlight text or click inside a text line before formatting.");
5083:       return;
5084:     }
5085:     selectRichBlock(block);
5086:     if (command === "fontName") updateCurrentTextBlock(editor, { fontFamily: value });
5087:     if (command === "foreColor") updateCurrentTextBlock(editor, { color: value });
5088:     if (command === "fontSize") updateCurrentTextBlock(editor, { fontPx: value });
5089:     if (command === "bold") updateCurrentTextBlock(editor, { bold: block.dataset.bold !== "true" });
5090:     if (command === "italic") {
5091:       updateCurrentTextBlock(editor, { italic: block.dataset.italic !== "true" });
5092:     }
5093:     if (command === "underline") {
5094:       updateCurrentTextBlock(editor, { underline: block.dataset.underline !== "true" });
5095:     }
5096:     saveRichEditorToProject(editor);
5097:     setStatus("Formatting applied and saved locally for the current text line.");
5098:     return;
5099:   }
5100:
5101:   saveRichEditorToProject(editor);
5102:   setStatus("Unsaved local changes. Click the section save button to keep this formatting.");
5103: }
```

selectRichBlock (template-preview.js, lines 5105-5113)

selectRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5105-5113 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, remove, add, closest, and indexOf. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include editor at line 5110. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5105: function selectRichBlock(block) {
5106:   document.querySelectorAll(".rich-block.selected").forEach((item) => item.classList.remove("selected"));
5107:   activeSummaryBlock = block || null;
5108:   if (block) {
5109:     block.classList.add("selected");
5110:     const editor = block.closest("[data-rich-editor]");
5111:     if (editor) editor.dataset.activeRichBlockIndex = String([...editor.children].indexOf(block));
5112:   }
5113: }
```

selectedRichBlock (template-preview.js, lines 5115-5123)

selectedRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5115-5123 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isInteger, matches, contains, and currentRichBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5116: if (!editor) return null;; line 5119: return editor.children[index];; line 5121: if (activeSummaryBlock && editor.contains(activeSummaryBlock)) return activeSummaryBlock;; line 5122: return currentRichBlock(editor);.

Important local values include index at line 5117. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5115: function selectedRichBlock(editor) {
5116:   if (!editor) return null;
5117:   const index = Number(editor.dataset.activeRichBlockIndex);
5118:   if (Number.isInteger(index) && index >= 0 && editor.children[index]?.matches(".rich-block")) {
5119:     return editor.children[index];
5120:   }
5121:   if (activeSummaryBlock && editor.contains(activeSummaryBlock)) return activeSummaryBlock;
5122:   return currentRichBlock(editor);
5123: }
```

syncRichContextMenuControls (template-preview.js, lines 5125-5144)

syncRichContextMenuControls keeps two places aligned, such as local data and target files. In this file, it appears around lines 5125-5144 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getComputedStyle, displayRichFontName, round, normalizeTextColor, restoreRichSelection, closest, queryCommandState, some, and setAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5126: if (!block || block.dataset.type !== "paragraph") return;.

Important local values include computed at line 5128; family at line 5129; size at line 5130; color at line 5131; bold at line 5133. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5125: function syncRichContextMenuControls(block) {
5126:   if (!block || block.dataset.type !== "paragraph") return;
5127:
5128:   const computed = getComputedStyle(block);
5129:   const family = displayRichFontName(computed.fontFamily) || "Arial";
5130:   const size = String(Math.round(parseFloat(computed.fontSize) || 16));
5131:   const color = normalizeTextColor(block.dataset.color || "") || "#17202a";
5132:   restoreRichSelection(block.closest("[data-rich-editor]"));
5133:   const bold = document.queryCommandState("bold") || block.dataset.bold === "true";
5134:
5135:   if (richFontSelect) {
5136:     richFontSelect.value = [...richFontSelect.options].some((option) => option.value === family ||
option.textContent === family)
5137:       ? family
5138:       : "Arial";
5139:   }
5140:
5141:   if (richFontSize) richFontSize.value = size;
5142:   if (richColorInput) richColorInput.value = color;
5143:   if (richBoldButton) richBoldButton.setAttribute("aria-pressed", bold ? "true" : "false");
5144: }
```


focusTextBlock (template-preview.js, lines 5146-5158)

focusTextBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5146-5158 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectRichBlock, closest, focus, createRange, selectNodeContents, collapse, getSelection, removeAllRanges, addRange, and captureRichSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5147: if (!block) return;.

Important local values include editor at line 5149; range at line 5151; selection at line 5154. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5146: function focusTextBlock(block) {
5147:   if (!block) return;
5148:   selectRichBlock(block);
5149:   const editor = block.closest("[data-rich-editor]");
5150:   editor?.focus();
5151:   const range = document.createRange();
5152:   range.selectNodeContents(block);
5153:   range.collapse(false);
5154:   const selection = window.getSelection();
5155:   selection.removeAllRanges();
5156:   selection.addRange(range);
5157:   captureRichSelection(editor);
5158: }
```

setTextBlockCaretOffset (template-preview.js, lines 5160-5174)

setTextBlockCaretOffset part of the private builder frontend and editing experience. In this file, it appears around lines 5160-5174 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectRichBlock, closest, focus, max, min, createRange, setStart, collapse, getSelection, removeAllRanges, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5161: if (!block) return;.

Important local values include editor at line 5163; textNode at line 5165; safeOffset at line 5166; range at line 5167; selection at line 5170. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5160: function setTextBlockCaretOffset(block, offset) {
5161:   if (!block) return;
5162:   selectRichBlock(block);
5163:   const editor = block.closest("[data-rich-editor]");
5164:   editor?.focus();
5165:   const textNode = block.firstChild || block;
5166:   const safeOffset = Math.max(0, Math.min(offset, textNode.textContent?.length || 0));
5167:   const range = document.createRange();
5168:   range.setStart(textNode, safeOffset);
5169:   range.collapse(true);
5170:   const selection = window.getSelection();
5171:   selection.removeAllRanges();
5172:   selection.addRange(range);
5173:   captureRichSelection(editor);
5174: }
```

textBlockCaretOffset (template-preview.js, lines 5176-5183)

textBlockCaretOffset part of the private builder frontend and editing experience. In this file, it appears around lines 5176-5183 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, contains, getRangeAt, cloneRange, selectNodeContents, setEnd, and toString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 5178: if (!selection?.rangeCount || !block.contains(selection.anchorNode)) return block.textContent.length;; line 5182: return range.toString().length;.

Important local values include selection at line 5177; range at line 5179. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5176: function textBlockCaretOffset(block) {
5177:   const selection = window.getSelection();
5178:   if (!selection?.rangeCount || !block.contains(selection.anchorNode)) return block.textContent.length;
5179:   const range = selection.getRangeAt(0).cloneRange();
5180:   range.selectNodeContents(block);
5181:   range.setEnd(selection.anchorNode, selection.anchorOffset);
5182:   return range.toString().length;
5183: }
```

matchingTextBlock (template-preview.js, lines 5185-5198)

matchingTextBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5185-5198 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createRichTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5186: return createRichTextBlock(.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
5185: function matchingTextBlock(block, text = "") {
5186:   return createRichTextBlock(
5187:     text,
5188:     block?.dataset.fontSize || "normal",
5189:     block?.dataset.align || "left",
5190:     block?.dataset.fontFamily || "Arial",
5191:     block?.dataset.fontPx || "",
5192:     block?.dataset.color || "",
5193:     block?.dataset.bold === "true",
5194:     "",
5195:     block?.dataset.italic === "true",
5196:     block?.dataset.underline === "true"
5197:   );
5198: }
```

richInsertionRange (template-preview.js, lines 5200-5221)

richInsertionRange part of the private builder frontend and editing experience. In this file, it appears around lines 5200-5221 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectionRangeInsideEditor, collapse, restoreRichSelection, querySelectorAll, at, createRange, and selectNodeContents. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5204: return currentRange;; line 5210: return restored;; line 5218: return range;; line 5220: return null;.

Important local values include currentRange at line 5201; restored at line 5207; lastText at line 5213; range at line 5215. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5200: function richInsertionRange(editor) {
5201:   const currentRange = selectionRangeInsideEditor(editor);
5202:   if (currentRange) {
5203:     currentRange.collapse(true);
5204:     return currentRange;
5205:   }
5206:   if (restoreRichSelection(editor)) {
5207:     const restored = selectionRangeInsideEditor(editor);
5208:     if (restored) {
5209:       restored.collapse(true);
5210:       return restored;
5211:     }
5212:   }
5213:   return null;
5214: }
```

```

5212:   }
5213:   const lastText = [...editor.querySelectorAll(".rich-text-block")].at(-1);
5214:   if (lastText) {
5215:     const range = document.createRange();
5216:     range.selectNodeContents(lastText);
5217:     range.collapse(false);
5218:     return range;
5219:   }
5220:   return null;
5221: }

```

insertRichBlockAtCursor (template-preview.js, lines 5223-5257)

insertRichBlockAtCursor part of the private builder frontend and editing experience. In this file, it appears around lines 5223-5257 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cloneRange, richInsertionRange, selectedRichBlock, contains, collapse, createRange, setStart, setEnd, extractContents, after, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include range at line 5224; rangeNode at line 5225; current at line 5228; nextText at line 5229; tail at line 5233; trailingContent at line 5236; reference at line 5242. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5223: function insertRichBlockAtCursor(editor, blockElement, insertionRange = null) {
5224:   const range = insertionRange?.cloneRange() || richInsertionRange(editor);
5225:   const rangeNode = range?.startContainer?.nodeType === Node.TEXT_NODE
5226:     ? range.startContainer.parentElement
5227:     : range?.startContainer;
5228:   const current = rangeNode?.closest?(".rich-block") || selectedRichBlock(editor);
5229:   let nextText = null;
5230:
5231:   if (range && current?.dataset.type === "paragraph" && current.contains(range.startContainer)) {
5232:     range.collapse(true);
5233:     const tail = document.createRange();
5234:     tail.setStart(range.startContainer, range.startOffset);
5235:     tail.setEnd(current, current.childNodes.length);
5236:     const trailingContent = tail.extractContents();
5237:     current.after(blockElement);
5238:     nextText = matchingTextBlock(current, "");
5239:     nextText.append(trailingContent);
5240:     blockElement.after(nextText);
5241:   } else if (range?.startContainer === editor) {
5242:     const reference = editor.children[range.startOffset] || null;
5243:     editor.insertBefore(blockElement, reference);
5244:     nextText = createRichTextBlock("");
5245:     blockElement.after(nextText);
5246:   } else if (current) {
5247:     current.after(blockElement);
5248:     nextText = matchingTextBlock(current, "");
5249:     blockElement.after(nextText);
5250:   } else {
5251:     editor.append(blockElement);
5252:     nextText = createRichTextBlock("");
5253:     editor.append(nextText);
5254:   }
5255:
5256:   focusTextBlock(nextText);
5257: }

```

insertRichBlockAfterCursor (template-preview.js, lines 5259-5261)

insertRichBlockAfterCursor part of the private builder frontend and editing experience. In this file, it appears around lines 5259-5261 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references insertRichBlockAtCursor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
5259: function insertRichBlockAfterCursor(editor, blockElement) {
5260:   insertRichBlockAtCursor(editor, blockElement);
5261: }
```

cleanupEmptyRichTextBlocks (template-preview.js, lines 5263-5275)

cleanupEmptyRichTextBlocks part of the private builder frontend and editing experience. In this file, it appears around lines 5263-5275 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, richElementPlainText, remove, querySelector, append, and createRichTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 5264: if (!editor) return;; line 5267: if (hasVisibleText) return;.

Important local values include hasVisibleText at line 5266; previous at line 5268; next at line 5269; keepTypingBlockAfterImage at line 5270; isOnlyBlock at line 5271. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5263: function cleanupEmptyRichTextBlocks(editor) {
5264:   if (!editor) return;
5265:   [...editor.querySelectorAll(":scope > .rich-text-block")].forEach((block) => {
5266:     const hasVisibleText = richElementPlainText(block);
5267:     if (hasVisibleText) return;
5268:     const previous = block.previousElementSibling;
5269:     const next = block.nextElementSibling;
5270:     const keepTypingBlockAfterImage = previous?.dataset.type === "image" && !next;
5271:     const isOnlyBlock = editor.children.length <= 1;
5272:     if (!isOnlyBlock && !keepTypingBlockAfterImage) block.remove();
5273:   });
5274:   if (!editor.querySelector(":scope > .rich-block")) editor.append(createRichTextBlock(""));
5275: }
```

normalizeRichEditorStructure (template-preview.js, lines 5277-5307)

normalizeRichEditorStructure validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 5277-5307 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references forEach, trim, remove, createRichTextBlock, replaceWith, matches, sanitizeRichInlineHtml, querySelectorAll, add, removeAttribute, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5278: if (!editor) return;; line 5283: return;; line 5287: return;; line 5290: if (node.nodeType !== Node.ELEMENT_NODE || node.matches(".rich-block")) return;.

Important local values include paragraph at line 5285; paragraph at line 5291. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5277: function normalizeRichEditorStructure(editor) {
5278:   if (!editor) return;
5279:   [...editor.childNodes].forEach((node) => {
5280:     if (node.nodeType === Node.TEXT_NODE) {
5281:       if (!node.textContent.trim()) {
5282:         node.remove();
5283:         return;
5284:       }
5285:       const paragraph = createRichTextBlock(node.textContent);
5286:       node.replaceWith(paragraph);
5287:       return;
5288:     }
5289:
5290:     if (node.nodeType !== Node.ELEMENT_NODE || node.matches(".rich-block")) return;
5291:     const paragraph = createRichTextBlock("");
5292:     paragraph.innerHTML = sanitizeRichInlineHtml(node.innerHTML || node.textContent || "");
5293:     node.replaceWith(paragraph);
5294:   });
5295:
5296:   editor.querySelectorAll(".rich-text-block").forEach((block) => {
5297:     block.dataset.type = "paragraph";
```

```

5298:     block.dataset.fontSize ||= "normal";
5299:     block.dataset.align ||= "left";
5300:     block.dataset.fontFamily ||= "Arial";
5301:     block.classList.add("rich-block");
5302:     block.removeAttribute("contenteditable");
5303:     applyRichTextBlockStyle(block);
5304:   });
5305:
5306:   if (!editor.querySelector(":scope > .rich-block")) editor.append(createRichTextBlock(""));
5307: }

```

richElementPlainText (template-preview.js, lines 5309-5313)

richElementPlainText part of the private builder frontend and editing experience. In this file, it appears around lines 5309-5313 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cloneNode, querySelectorAll, forEach, replaceWith, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5312: return String(cloneElement.textContent || "").trim();.

Important local values include cloneElement at line 5310. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5309: function richElementPlainText(element) {
5310:   const cloneElement = element.cloneNode(true);
5311:   cloneElement.querySelectorAll("br").forEach((lineBreak) => lineBreak.replaceWith("\n"));
5312:   return String(cloneElement.textContent || "").trim();
5313: }

```

updateCurrentTextBlock (template-preview.js, lines 5315-5357)

updateCurrentTextBlock updates ui, state, cache, or stored data. In this file, it appears around lines 5315-5357 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedRichBlock, remove, add, call, cleanFontFamily, normalizeFontPx, normalizeTextColor, applyRichTextBlockStyle, and saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5317: if (!block || block.dataset.type !== "paragraph") return;.

Important local values include block at line 5316. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5315: function updateCurrentTextBlock(editor, updates) {
5316:   const block = selectedRichBlock(editor);
5317:   if (!block || block.dataset.type !== "paragraph") return;
5318:
5319:   if (updates.fontSize) {
5320:     block.dataset.fontSize = updates.fontSize;
5321:     block.classList.remove("rich-text-small", "rich-text-normal", "rich-text-large");
5322:     block.classList.add(`rich-text-${updates.fontSize}`);
5323:   }
5324:
5325:   if (updates.align) {
5326:     block.dataset.align = updates.align;
5327:     block.classList.remove("text-left", "text-center", "text-right");
5328:     block.classList.add(`text-${updates.align}`);
5329:   }
5330:
5331:   if (Object.prototype.hasOwnProperty.call(updates, "fontFamily")) {
5332:     block.dataset.fontFamily = cleanFontFamily(updates.fontFamily);
5333:   }
5334:
5335:   if (Object.prototype.hasOwnProperty.call(updates, "fontPx")) {
5336:     block.dataset.fontPx = normalizeFontPx(updates.fontPx);
5337:   }
5338:
5339:   if (Object.prototype.hasOwnProperty.call(updates, "color")) {
5340:     block.dataset.color = normalizeTextColor(updates.color);
5341:   }
5342:
5343:   if (Object.prototype.hasOwnProperty.call(updates, "bold")) {

```

```

5344:         block.dataset.bold = updates.bold ? "true" : "false";
5345:     }
5346:
5347:     if (Object.prototype.hasOwnProperty.call(updates, "italic")) {
5348:         block.dataset.italic = updates.italic ? "true" : "false";
5349:     }
5350:
5351:     if (Object.prototype.hasOwnProperty.call(updates, "underline")) {
5352:         block.dataset.underline = updates.underline ? "true" : "false";
5353:     }
5354:
5355:     applyRichTextBlockStyle(block);
5356:     saveRichEditorToProject(editor);
5357: }

```

extractRichSummary (template-preview.js, lines 5359-5416)

extractRichSummary part of the private builder frontend and editing experience. In this file, it appears around lines 5359-5416 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeRichEditorStructure, normalizeInlineStyleSpans, querySelectorAll, map, normalizeCropAspect, normalizeCropPosition, normalizeCropZoom, unwrapFormula, normalizeCodeText, querySelector, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 8 return statement(s). The most important visible returns are: line 5365: return {; line 5380: return {; line 5388: if (!code) return null;; line 5389: return {. There are 4 additional return statements in the function body.

Important local values include blocks at line 5362; align at line 5363; code at line 5387; text at line 5396. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5359: function extractRichSummary(editor) {
5360:     normalizeRichEditorStructure(editor);
5361:     normalizeInlineStyleSpans(editor);
5362:     const blocks = [...editor.querySelectorAll(".rich-block")].map((element) => {
5363:         const align = element.dataset.align || "left";
5364:         if (element.dataset.type === "image") {
5365:             return {
5366:                 align,
5367:                 caption: element.dataset.caption || "",
5368:                 cropAspect: normalizeCropAspect(element.dataset.cropAspect),
5369:                 cropX: normalizeCropPosition(element.dataset.cropX),
5370:                 cropY: normalizeCropPosition(element.dataset.cropY),
5371:                 cropZoom: normalizeCropZoom(element.dataset.cropZoom),
5372:                 display: element.dataset.display === "download" ? "download" : "show",
5373:                 source: element.dataset.source || "local",
5374:                 title: element.dataset.title || "",
5375:                 type: "image",
5376:                 url: element.dataset.url || ""
5377:             };
5378:         }
5379:         if (element.dataset.type === "formula") {
5380:             return {
5381:                 align,
5382:                 formula: unwrapFormula(element.dataset.formula || element.textContent || ""),
5383:                 type: "formula"
5384:             };
5385:         }
5386:         if (element.dataset.type === "code") {
5387:             const code = normalizeCodeText(element.dataset.code || element.querySelector("code")?.textContent || "", element.dataset.pasteMode || "source");
5388:             if (!code) return null;
5389:             return {
5390:                 code,
5391:                 language: normalizeCodeLanguage(element.dataset.language || detectCodeLanguage(code)),
5392:                 pasteMode: normalizeCodePasteMode(element.dataset.pasteMode),
5393:                 type: "code"
5394:             };
5395:         }
5396:         const text = richElementPlainText(element);
5397:         if (!text) return null;
5398:         if (isFormulaOnly(text)) {
5399:             return { align: element.dataset.align || "center", formula: unwrapFormula(text), type: "formula" };
5400:         }
5401:         return {
5402:             align,
5403:             bold: element.dataset.bold === "true",
5404:             color: element.dataset.color || "",
5405:             fontFamily: element.dataset.fontFamily || "",
5406:             fontPx: element.dataset.fontPx || "",
5407:             fontSize: element.dataset.fontSize || "normal",
5408:             html: sanitizeRichInlineHtml(element.innerHTML),
5409:             italic: element.dataset.italic === "true",

```

```

5410:         text,
5411:         underline: element.dataset.underline === "true",
5412:         type: "paragraph"
5413:     });
5414: }).filter(Boolean);
5415: return { blocks };
5416: }

```

uploadSummaryImageFile (template-preview.js, lines 5418-5458)

uploadSummaryImageFile part of the private builder frontend and editing experience. In this file, it appears around lines 5418-5458 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, isAllowedUpload, setStatus, call, withExtension, readFileAsDataURL, fetch, stringify, json, normalizeCropAspect, and 2 more. Endpoint references found inside it are /api/upload. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5421: if (!projectId || !file) return null;; line 5425: return null;; line 5443: return null;; line 5445: return {.

Important local values include project at line 5419; projectId at line 5420; validation at line 5422; displayTitle at line 5427; fileName at line 5428; data at line 5429; response at line 5430; result at line 5440. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5418: async function uploadSummaryImageFile(file, options = {}) {
5419:   const project = selectedProject();
5420:   const projectId = options.projectId || project?.id;
5421:   if (!projectId || !file) return null;
5422:   const validation = isAllowedUpload("media", file.name, file.type);
5423:   if (!validation.ok) {
5424:     setStatus(validation.message);
5425:     return null;
5426:   }
5427:   const displayTitle = Object.prototype.hasOwnProperty.call(options, "title") ? options.title :
file.name;
5428:   const fileName = withExtension(displayTitle || file.name, file.name);
5429:   const data = await readFileAsDataURL(file);
5430:   const response = await fetch("/api/upload", {
5431:     method: "POST",
5432:     headers: { "Content-Type": "application/json" },
5433:     body: JSON.stringify({
5434:       projectId,
5435:       section: options.folder || "summary",
5436:       fileName,
5437:       data
5438:     })
5439:   });
5440:   const result = await response.json();
5441:   if (!response.ok) {
5442:     setStatus(result.error || "Image upload failed.");
5443:     return null;
5444:   }
5445:   return {
5446:     align: options.align || "center",
5447:     caption: options.caption || "",
5448:     cropAspect: normalizeCropAspect(options.cropAspect),
5449:     cropX: normalizeCropPosition(options.cropX),
5450:     cropY: normalizeCropPosition(options.cropY),
5451:     cropZoom: normalizeCropZoom(options.cropZoom),
5452:     display: options.display === "download" ? "download" : "show",
5453:     source: "local",
5454:     title: displayTitle || "",
5455:     type: "image",
5456:     url: result.url
5457:   };
5458: }

```

updateSummaryImageDialogVisibility (template-preview.js, lines 5460-5465)

updateSummaryImageDialogVisibility updates ui, state, cache, or stored data. In this file, it appears around lines 5460-5465 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include source at line 5461; isLocal at line 5462. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5460: function updateSummaryImageDialogVisibility() {
5461:   const source = summaryImageSource?.value || "local";
5462:   const isLocal = source === "local";
5463:   document.querySelector(".summary-image-local-field").hidden = !isLocal;
5464:   document.querySelector(".summary-image-url-field").hidden = isLocal;
5465: }
```

openSummaryImageDialog (template-preview.js, lines 5467-5546)

openSummaryImageDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 5467-5546 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `querySelector`, `normalizeCropAspect`, `normalizeCropZoom`, `updateSummaryImageDialogVisibility`, `dialogValue`, `normalizeLinkTarget`, `trim`, `setStatus`, `urlLooksLikeDirectImage`, `normalizeCropPosition`, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 5483: if (!saved) return null;; line 5493: return null;; line 5498: return null;; line 5500: return {. There are 3 additional return statements in the function body.

Important local values include heading at line 5468; submitButton at line 5469; saved at line 5482; file at line 5484; source at line 5485; url at line 5486; existingUrl at line 5489; nextUrl at line 5490; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5467: async function openSummaryImageDialog(existingBlock = null, options = {}) {
5468:   const heading = summaryImageDialog.querySelector("h2");
5469:   const submitButton = summaryImageDialog.querySelector("button[type='submit']");
5470:   if (heading) heading.textContent = existingBlock ? "Edit image" : "Add image to overview";
5471:   if (submitButton) submitButton.textContent = existingBlock ? "Save image" : "Add image";
5472:   summaryImageSource.value = existingBlock?.dataset.source || "local";
5473:   summaryImageFile.value = "";
5474:   summaryImageUrl.value = existingBlock?.dataset.source && existingBlock?.dataset.source !== "local" ?
existingBlock?.dataset.url || "" : "";
5475:   summaryImageTitle.value = existingBlock?.dataset.title || "";
5476:   summaryImageCaption.value = existingBlock?.dataset.caption || "";
5477:   summaryImageAlign.value = existingBlock?.dataset.align || "center";
5478:   summaryImageDisplay.value = existingBlock?.dataset.display || "show";
5479:   summaryImageCrop.value = normalizeCropAspect(existingBlock?.dataset.cropAspect);
5480:   summaryImageZoom.value = String(normalizeCropZoom(existingBlock?.dataset.cropZoom));
5481:   updateSummaryImageDialogVisibility();
5482:   const saved = await dialogValue(summaryImageDialog);
5483:   if (!saved) return null;
5484:   const file = summaryImageFile.files[0];
5485:   const source = summaryImageSource.value || "local";
5486:   const url = normalizeLinkTarget(summaryImageUrl.value.trim(), { assumeWeb: true });
5487:
5488:   if (source !== "local") {
5489:     const existingUrl = existingBlock?.dataset.url || "";
5490:     const nextUrl = url || existingUrl;
5491:     if (!nextUrl) {
5492:       setStatus("Paste an image URL or share link first.");
5493:       return null;
5494:     }
5495:     const display = summaryImageDisplay.value === "download" ? "download" : "show";
5496:     if (display === "show" && !urlLooksLikeDirectImage(nextUrl)) {
5497:       setStatus("Use a direct image URL for visible images, or choose Downloadable file only for share
links and non-image URLs.");
5498:       return null;
5499:     }
5500:     return {
5501:       align: summaryImageAlign.value,
5502:       caption: summaryImageCaption.value.trim(),
5503:       cropAspect: summaryImageCrop.value,
5504:       cropX: normalizeCropPosition(existingBlock?.dataset.cropX),
5505:       cropY: normalizeCropPosition(existingBlock?.dataset.cropY),
5506:       cropZoom: normalizeCropZoom(summaryImageZoom.value),
5507:       display,
5508:       source,
5509:       title: summaryImageTitle.value.trim() || displayNameFromUrl(nextUrl, source === "drive" ? "Google
Drive image" : "Web image"),
5510:       type: "image",
5511:       url: nextUrl
5512:     };
5513:   }
```



```

5514:
5515:   if (!file && !existingBlock) {
5516:     setStatus("Choose an image first.");
5517:     return null;
5518:   }
5519:   if (!file && existingBlock) {
5520:     return {
5521:       align: summaryImageAlign.value,
5522:       caption: summaryImageCaption.value.trim(),
5523:       cropAspect: summaryImageCrop.value,
5524:       cropX: normalizeCropPosition(existingBlock.dataset.cropX),
5525:       cropY: normalizeCropPosition(existingBlock.dataset.cropY),
5526:       cropZoom: normalizeCropZoom(summaryImageZoom.value),
5527:       display: summaryImageDisplay.value === "download" ? "download" : "show",
5528:       source: existingBlock.dataset.source || "local",
5529:       title: summaryImageTitle.value.trim() || existingBlock.dataset.title || "",
5530:       type: "image",
5531:       url: existingBlock.dataset.url || ""
5532:     };
5533:   }
5534:   return uploadSummaryImageFile(file, {
5535:     align: summaryImageAlign.value,
5536:     caption: summaryImageCaption.value.trim(),
5537:     cropAspect: summaryImageCrop.value,
5538:     cropX: 50,
5539:     cropY: 50,
5540:     cropZoom: normalizeCropZoom(summaryImageZoom.value),
5541:     display: summaryImageDisplay.value === "download" ? "download" : "show",
5542:     folder: options.folder || "summary",
5543:     projectId: options.projectId || "",
5544:     title: summaryImageTitle.value.trim() || file.name
5545:   });
5546: }

```

openSummaryFormulaDialog (template-preview.js, lines 5548-5563)

openSummaryFormulaDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 5548-5563 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, dialogValue, unwrapFormula, trim, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5556: if (!saved) return null;; line 5560: return null;; line 5562: return { align: summaryFormulaAlign.value, formula, type: "formula" };.

Important local values include heading at line 5549; submitButton at line 5550; saved at line 5555; formula at line 5557. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5548: async function openSummaryFormulaDialog(existingBlock = null) {
5549:   const heading = summaryFormulaDialog.querySelector("h2");
5550:   const submitButton = summaryFormulaDialog.querySelector("button[type='submit']");
5551:   if (heading) heading.textContent = existingBlock ? "Edit formula" : "Add formula";
5552:   if (submitButton) submitButton.textContent = existingBlock ? "Save formula" : "Add formula";
5553:   summaryFormulaInput.value = existingBlock?.dataset.formula || "";
5554:   summaryFormulaAlign.value = existingBlock?.dataset.align || "center";
5555:   const saved = await dialogValue(summaryFormulaDialog);
5556:   if (!saved) return null;
5557:   const formula = unwrapFormula(summaryFormulaInput.value.trim());
5558:   if (!formula) {
5559:     setStatus("Paste or type a formula first.");
5560:     return null;
5561:   }
5562:   return { align: summaryFormulaAlign.value, formula, type: "formula" };
5563: }

```

refreshSummaryCodeDialogPreview (template-preview.js, lines 5565-5575)

refreshSummaryCodeDialogPreview part of the private builder frontend and editing experience. In this file, it appears around lines 5565-5575 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references detectCodeLanguage, normalizeCodeLanguage, normalizeCodeText, and renderRichCodeBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5566: if (!summaryCodePreview || !summaryCodeInput) return;.

Important local values include rawCode at line 5567; language at line 5568; code at line 5571. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5565: function refreshSummaryCodeDialogPreview() {
5566:   if (!summaryCodePreview || !summaryCodeInput) return;
5567:   const rawCode = summaryCodeInput.value || "";
5568:   const language = summaryCodeLanguage?.value === "auto"
5569:     ? detectCodeLanguage(rawCode)
5570:     : normalizeCodeLanguage(summaryCodeLanguage?.value || "javascript");
5571:   const code = normalizeCodeText(rawCode, summaryCodePasteMode?.value || "source");
5572:   summaryCodePreview.innerHTML = code
5573:     ? renderRichCodeBlock({ code, language, pasteMode: summaryCodePasteMode?.value || "source" })
5574:     : `
```

beautifyCodeClient (template-preview.js, lines 5577-5592)

beautifyCodeClient part of the private builder frontend and editing experience. In this file, it appears around lines 5577-5592 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, normalizeCodeLanguage, includes, split, map, trim, max, repeat, match, join, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5582: return normalized.split("\n").map((rawLine) => {; line 5584: if (!line) return "";; line 5590: return nextLine;.

Important local values include normalized at line 5578; lang at line 5579; depth at line 5581; line at line 5583; nextLine at line 5586; opens at line 5587; closes at line 5588. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5577: function beautifyCodeClient(code = "", language = "javascript") {
5578:   const normalized = String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ");
5579:   const lang = normalizeCodeLanguage(language);
5580:   if (["c", "cpp", "java", "javascript", "verilog", "systemverilog"].includes(lang)) {
5581:     let depth = 0;
5582:     return normalized.split("\n").map((rawLine) => {
5583:       const line = rawLine.trim();
5584:       if (!line) return "";
5585:       if (/^[{}]\]/.test(line)) depth = Math.max(0, depth - 1);
5586:       const nextLine = `${" ".repeat(depth)}${line}`;
5587:       const opens = (line.match(/[{\[\/g) || []).length;
5588:       const closes = (line.match(/[}\]\/g) || []).length;
5589:       depth = Math.max(0, depth + opens - closes);
5590:       return nextLine;
5591:     }).join("\n").trimEnd();
5592:   }
```

beautifySummaryCodeDialog (template-preview.js, lines 5599-5618)

beautifySummaryCodeDialog part of the private builder frontend and editing experience. In this file, it appears around lines 5599-5618 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references detectCodeLanguage, normalizeCodeLanguage, fetch, now, stringify, json, Error, beautifyCodeClient, and refreshSummaryCodeDialogPreview. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5600: if (!summaryCodeInput) return;.

Important local values include language at line 5601; response at line 5605; result at line 5611. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5599: async function beautifySummaryCodeDialog() {
5600:   if (!summaryCodeInput) return;
5601:   const language = summaryCodeLanguage?.value === "auto"
5602:     ? detectCodeLanguage(summaryCodeInput.value || "")
5603:     : normalizeCodeLanguage(summaryCodeLanguage?.value || "javascript");
5604:   try {
```

```

5605:     const response = await fetch(`/api/code/beautify?t=${Date.now()}`, {
5606:       method: "POST",
5607:       cache: "no-store",
5608:       headers: { "Content-Type": "application/json" },
5609:       body: JSON.stringify({ language, code: summaryCodeInput.value || "" })
5610:     });
5611:     const result = await response.json();
5612:     if (!response.ok || !result.ok) throw new Error(result.error || "Beautifier failed.");
5613:     summaryCodeInput.value = result.code || summaryCodeInput.value;
5614:   } catch {
5615:     summaryCodeInput.value = beautifyCodeClient(summaryCodeInput.value || "", language);
5616:   }
5617:   refreshSummaryCodeDialogPreview();
5618: }

```

openSummaryCodeDialog (template-preview.js, lines 5620-5640)

openSummaryCodeDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 5620-5640 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, detectCodeLanguage, refreshSummaryCodeDialogPreview, dialogValue, normalizeCodePasteMode, normalizeCodeText, setStatus, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5631: if (!saved) return null;; line 5636: return null;; line 5639: return { code, language, pasteMode, type: "code" };;

Important local values include heading at line 5621; submitButton at line 5622; defaultCode at line 5625; saved at line 5630; pasteMode at line 5632; code at line 5633; language at line 5638. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5620: async function openSummaryCodeDialog(existingBlock = null, options = {}) {
5621:   const heading = summaryCodeDialog.querySelector("h2");
5622:   const submitButton = summaryCodeDialog.querySelector("button[type='submit']");
5623:   if (heading) heading.textContent = existingBlock ? "Edit code block" : "Add code block";
5624:   if (submitButton) submitButton.textContent = existingBlock ? "Save code" : "Add code";
5625:   const defaultCode = options.code || "";
5626:   summaryCodeLanguage.value = existingBlock?.dataset.language || (defaultCode ?
detectCodeLanguage(defaultCode) : "auto");
5627:   summaryCodePasteMode.value = existingBlock?.dataset.pasteMode || options.pasteMode || "source";
5628:   summaryCodeInput.value = existingBlock?.dataset.code || defaultCode;
5629:   refreshSummaryCodeDialogPreview();
5630:   const saved = await dialogValue(summaryCodeDialog);
5631:   if (!saved) return null;
5632:   const pasteMode = normalizeCodePasteMode(summaryCodePasteMode.value);
5633:   const code = normalizeCodeText(summaryCodeInput.value, pasteMode);
5634:   if (!code) {
5635:     setStatus("Paste or type code before saving the block.");
5636:     return null;
5637:   }
5638:   const language = summaryCodeLanguage.value === "auto" ? detectCodeLanguage(code) :
normalizeCodeLanguage(summaryCodeLanguage.value);
5639:   return { code, language, pasteMode, type: "code" };
5640: }

```

configureSummaryContextMenu (template-preview.js, lines 5642-5665)

configureSummaryContextMenu part of the private builder frontend and editing experience. In this file, it appears around lines 5642-5665 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, querySelectorAll, forEach, and has. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include plainActions at line 5643; action at line 5653; textOnly at line 5660. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5642: function configureSummaryContextMenu(mode = "rich", options = {}) {
5643:   const plainActions = new Set([
5644:     "copy-text",
5645:     "paste-text",

```

```

5646:     "cut-text",
5647:     "select-all-text",
5648:     "toggle-bold",
5649:     "toggle-italic",
5650:     "toggle-underline"
5651:   });
5652:   summaryContextMenu.querySelectorAll("[data-rich-action]").forEach((button) => {
5653:     const action = button.dataset.richAction;
5654:     button.hidden = mode === "plain" ? !plainActions.has(action) : false;
5655:   });
5656:   summaryContextMenu.querySelectorAll(".rich-menu-field").forEach((field) => {
5657:     field.hidden = false;
5658:   });
5659:   if (mode === "rich") {
5660:     const textOnly = Boolean(options.textOnly);
5661:     summaryContextMenu.querySelectorAll("[data-rich-action='add-image'], [data-rich-action='add-
formula'], [data-rich-action='add-code'], [data-rich-action='paste-code']").forEach((button) => {
5662:       button.hidden = textOnly;
5663:     });
5664:   }
5665: }

```

applyPlainTextControlFormat (template-preview.js, lines 5667-5701)

applyPlainTextControlFormat part of the private builder frontend and editing experience. In this file, it appears around lines 5667-5701 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call, cleanFontFamily, normalizeFontPx, normalizeTextColor, rgbColorToHex, storePlainControlStyle, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5668: if (!control) return;.

Important local values include font at line 5670; size at line 5675; color at line 5680. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

5667: function applyPlainTextControlFormat(control, updates = {}) {
5668:   if (!control) return;
5669:   if (Object.prototype.hasOwnProperty.call(updates, "fontFamily")) {
5670:     const font = cleanFontFamily(updates.fontFamily);
5671:     control.dataset.builderFontFamily = font;
5672:     control.style.fontFamily = font || "";
5673:   }
5674:   if (Object.prototype.hasOwnProperty.call(updates, "fontPx")) {
5675:     const size = normalizeFontPx(updates.fontPx);
5676:     control.dataset.builderFontPx = size;
5677:     control.style.fontSize = size ? `${size}px` : "";
5678:   }
5679:   if (Object.prototype.hasOwnProperty.call(updates, "color")) {
5680:     const color = normalizeTextColor(updates.color);
5681:     control.dataset.builderColor = color;
5682:     control.style.color = color || "";
5683:     if (richColorInput && color && /^[0-9a-f]{6}$/i.test(rgbColorToHex(color))) {
5684:       richColorInput.value = rgbColorToHex(color);
5685:     }
5686:   }
5687:   if (Object.prototype.hasOwnProperty.call(updates, "bold")) {
5688:     control.dataset.builderBold = updates.bold ? "true" : "false";
5689:     control.style.fontWeight = updates.bold ? "700" : "";
5690:   }
5691:   if (Object.prototype.hasOwnProperty.call(updates, "italic")) {
5692:     control.dataset.builderItalic = updates.italic ? "true" : "false";
5693:     control.style.fontStyle = updates.italic ? "italic" : "";
5694:   }
5695:   if (Object.prototype.hasOwnProperty.call(updates, "underline")) {
5696:     control.dataset.builderUnderline = updates.underline ? "true" : "false";
5697:     control.style.textDecoration = updates.underline ? "underline" : "";
5698:   }
5699:   storePlainControlStyle(control);
5700:   setStatus("Formatting applied and saved locally for this field.");
5701: }

```

syncPlainContextMenuControls (template-preview.js, lines 5703-5719)

syncPlainContextMenuControls keeps two places aligned, such as local data and target files. In this file, it appears around lines 5703-5719 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `getComputedStyle`, `displayRichFontName`, `round`, `rgbColorToHex`, `some`, and `setAttribute`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5704: `if (!control) return;`.

Important local values include `computed` at line 5705; `family` at line 5706; `size` at line 5707; `color` at line 5708. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5703: function syncPlainContextMenuControls(control) {
5704:   if (!control) return;
5705:   const computed = getComputedStyle(control);
5706:   const family = displayRichFontName(control.dataset.builderFontFamily || computed.fontFamily) ||
"Arial";
5707:   const size = String(Math.round(parseFloat(control.dataset.builderFontPx || computed.fontSize) || 16));
5708:   const color = rgbColorToHex(control.dataset.builderColor || computed.color || "#17202a");
5709:   if (richFontSelect) {
5710:     richFontSelect.value = [...richFontSelect.options].some((option) => option.value === family ||
option.textContent === family)
5711:     ? family
5712:     : "Arial";
5713:   }
5714:   if (richFontSize) richFontSize.value = [...richFontSize.options].some((option) => option.value === size
|| option.textContent === size)
5715:   ? size
5716:   : "16";
5717:   if (richColorInput && /^[0-9a-f]{6}$/i.test(color)) richColorInput.value = color;
5718:   if (richBoldButton) richBoldButton.setAttribute("aria-pressed", control.dataset.builderBold === "true"
? "true" : "false");
5719: }
```

plainTextControlFromContextEvent (template-preview.js, lines 5721-5728)

`plainTextControlFromContextEvent` part of the private builder frontend and editing experience. In this file, it appears around lines 5721-5728 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `closest`, `toLowerCase`, and `includes`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5723: `if (!control || control.closest("#summary-context-menu")) return null;`; line 5725: `if (control.disabled || control.readOnly) return null;`; line 5726: `if ([ "button", "checkbox", "color", "file", "hidden", "image", "radio", "range", "reset", "submit"].includes(type)) return null;`; line 5727: `return control;`.

Important local values include `control` at line 5722; `type` at line 5724. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5721: function plainTextControlFromContextEvent(event) {
5722:   const control = event.target.closest?(".textarea, input");
5723:   if (!control || control.closest("#summary-context-menu")) return null;
5724:   const type = String(control.type || "text").toLowerCase();
5725:   if (control.disabled || control.readOnly) return null;
5726:   if ([ "button", "checkbox", "color", "file", "hidden", "image", "radio", "range", "reset",
"submit"].includes(type)) return null;
5727:   return control;
5728: }
```

handlePlainTextAction (template-preview.js, lines 5730-5778)

`handlePlainTextAction` handles an event, request, command, or user action. In this file, it appears around lines 5730-5778 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `focus`, `max`, `min`, `slice`, `writeText`, `setRangeText`, `dispatchEvent`, `Event`, `readText`, `select`, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 5732: `if (!control) return;`; line 5748: `return;`; line 5757: `return;`; line 5763: `return;`. There are 3 additional return statements in the function body.

Important local values include control at line 5731; selection at line 5734; start at line 5738; end at line 5739; text at line 5742; text at line 5752. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5730: async function handlePlainTextAction(action) {
5731:   const control = activePlainTextControl;
5732:   if (!control) return;
5733:   control.focus({ preventScroll: true });
5734:   const selection = activePlainTextSelection || {
5735:     start: control.selectionStart ?? 0,
5736:     end: control.selectionEnd ?? control.value.length
5737:   };
5738:   const start = Math.max(0, Math.min(selection.start, control.value.length));
5739:   const end = Math.max(start, Math.min(selection.end, control.value.length));
5740:
5741:   if (action === "copy-text" || action === "cut-text") {
5742:     const text = control.value.slice(start, end) || control.value;
5743:     if (text) await navigator.clipboard.writeText(text);
5744:     if (action === "cut-text" && end > start) {
5745:       control.setRangeText("", start, end, "start");
5746:       control.dispatchEvent(new Event("input", { bubbles: true }));
5747:     }
5748:     return;
5749:   }
5750:
5751:   if (action === "paste-text") {
5752:     const text = await navigator.clipboard.readText();
5753:     if (text) {
5754:       control.setRangeText(text, start, end, "end");
5755:       control.dispatchEvent(new Event("input", { bubbles: true }));
5756:     }
5757:     return;
5758:   }
5759:
5760:   if (action === "select-all-text") {
5761:     control.select();
5762:     activePlainTextSelection = { start: 0, end: control.value.length };
5763:     return;
5764:   }
5765:
5766:   if (action === "toggle-bold") {
5767:     applyPlainTextControlFormat(control, { bold: control.dataset.builderBold !== "true" });
5768:     return;
5769:   }
5770:   if (action === "toggle-italic") {
5771:     applyPlainTextControlFormat(control, { italic: control.dataset.builderItalic !== "true" });
5772:     return;
5773:   }
5774:   if (action === "toggle-underline") {
5775:     applyPlainTextControlFormat(control, { underline: control.dataset.builderUnderline !== "true" });
5776:     return;
5777:   }
5778: }
```

handleRichAction (template-preview.js, lines 5780-5878)

handleRichAction handles an event, request, command, or user action. In this file, it appears around lines 5780-5878 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references applyRichInlineCommand, updateCurrentTextBlock, restoreRichSelection, getSelection, toString, selectedRichBlock, writeText, readText, insertPlainTextIntoRichEditor, saveRichEditorToProject, and 20 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 5781: if (!editor) return;; line 5801: return;; line 5810: return;; line 5824: return;. There are 1 additional return statements in the function body.

Important local values include selectedText at line 5797; block at line 5798; text at line 5799; text at line 5805; selection at line 5814; selectedText at line 5815; range at line 5818; block at line 5827; plus 9 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5780: async function handleRichAction(action, editor) {
5781:   if (!editor) return;
5782:   activeSummaryEditor = editor;
5783:   if (action === "toggle-bold") {
5784:     applyRichInlineCommand(editor, "bold");
5785:   }
5786:   if (action === "toggle-italic") {
5787:     applyRichInlineCommand(editor, "italic");
5788:   }
```

```

5789:   if (action === "toggle-underline") {
5790:     applyRichInlineCommand(editor, "underline");
5791:   }
5792:   if (action === "align-left") updateCurrentTextBlock(editor, { align: "left" });
5793:   if (action === "align-center") updateCurrentTextBlock(editor, { align: "center" });
5794:   if (action === "align-right") updateCurrentTextBlock(editor, { align: "right" });
5795:   if (action === "copy-text") {
5796:     restoreRichSelection(editor);
5797:     const selectedText = window.getSelection()?.toString() || "";
5798:     const block = selectedRichBlock(editor);
5799:     const text = selectedText || block?.textContent || "";
5800:     if (text) await navigator.clipboard.writeText(text);
5801:     return;
5802:   }
5803:
5804:   if (action === "paste-text") {
5805:     const text = await navigator.clipboard.readText();
5806:     if (text) {
5807:       insertPlainTextIntoRichEditor(editor, text);
5808:       saveRichEditorToProject(editor);
5809:     }
5810:     return;
5811:   }
5812:   if (action === "cut-text") {
5813:     restoreRichSelection(editor);
5814:     const selection = window.getSelection();
5815:     const selectedText = selection?.toString() || "";
5816:     if (selectedText && selection.rangeCount) {
5817:       await navigator.clipboard.writeText(selectedText);
5818:       const range = selection.getRangeAt(0);
5819:       range.deleteContents();
5820:       captureRichSelection(editor);
5821:       cleanupEmptyRichTextBlocks(editor);
5822:       saveRichEditorToProject(editor);
5823:     }
5824:     return;
5825:   }
5826:   if (action === "select-all-text") {
5827:     const block = selectedRichBlock(editor);
5828:     const target = block?.dataset.type === "paragraph" ? block : editor;
5829:     const range = document.createRange();
5830:     range.selectNodeContents(target);
5831:     const selection = window.getSelection();
5832:     selection.removeAllRanges();
5833:     selection.addRange(range);
5834:     activeRichSelectionRange = range.cloneRange();
5835:     activeTextSelectionRange = range.cloneRange();
5836:     showPersistentTextSelection(range);
5837:     showTextSelectionInspector(editor, range);
5838:     return;
5839:   }
5840:   if (action === "add-image") {
5841:     const imageBlock = await openSummaryImageDialog(null, {
5842:       folder: editor.dataset.richFolder || "summary",
5843:       projectId: editor.dataset.richProjectId || ""
5844:     });
5845:     if (imageBlock) {
5846:       insertRichBlockAfterCursor(editor, createRichImageBlock(imageBlock));
5847:       saveRichEditorToProject(editor);
5848:     }
5849:   }
5850:   if (action === "add-formula") {
5851:     const formulaBlock = await openSummaryFormulaDialog();
5852:     if (formulaBlock) {
5853:       insertRichBlockAfterCursor(editor, createRichFormulaBlock(formulaBlock));
5854:       saveRichEditorToProject(editor);
5855:     }
5856:   }
5857:   if (action === "add-code") {
5858:     const selectedText = window.getSelection()?.toString() || "";
5859:     const codeBlock = await openSummaryCodeDialog(null, { code: selectedText, pasteMode: "source" });
5860:     if (codeBlock) {
5861:       insertRichBlockAfterCursor(editor, createRichCodeBlock(codeBlock));
5862:       saveRichEditorToProject(editor);
5863:     }
5864:   }
5865:   if (action === "paste-code") {
5866:     const text = await navigator.clipboard.readText();
5867:     const codeBlock = await openSummaryCodeDialog(null, { code: text, pasteMode: "source" });
5868:     if (codeBlock) {
5869:       insertRichBlockAfterCursor(editor, createRichCodeBlock(codeBlock));
5870:       saveRichEditorToProject(editor);
5871:     }
5872:   }
5873:   if (action === "edit-block") {
5874:     await editSelectedRichBlock(editor);
5875:     saveRichEditorToProject(editor);
5876:   }
5877:   if (action === "delete-block") deleteSelectedRichBlock(editor);
5878: }

```

hideSummaryContextMenu (template-preview.js, lines 5880-5884)

hideSummaryContextMenu controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 5880-5884 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
5880: function hideSummaryContextMenu() {  
5881:   summaryContextMenu.hidden = true;  
5882:   activePlainTextControl = null;  
5883:   activePlainTextSelection = null;  
5884: }
```

editSelectedRichBlock (template-preview.js, lines 5886-5909)

editSelectedRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5886-5909 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedRichBlock, selectRichBlock, openSummaryImageDialog, refreshRichImageBlock, openSummaryFormulaDialog, refreshRichFormulaBlock, openSummaryCodeDialog, refreshRichCodeBlock, focusTextBlock, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 5888: if (!block) return;; line 5893: return;; line 5898: return;; line 5903: return;.

Important local values include block at line 5887; updated at line 5891; updated at line 5896; updated at line 5901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5886: async function editSelectedRichBlock(editor) {  
5887:   const block = selectedRichBlock(editor);  
5888:   if (!block) return;  
5889:   selectRichBlock(block);  
5890:   if (block.dataset.type === "image") {  
5891:     const updated = await openSummaryImageDialog(block);  
5892:     if (updated) refreshRichImageBlock(block, updated);  
5893:     return;  
5894:   }  
5895:   if (block.dataset.type === "formula") {  
5896:     const updated = await openSummaryFormulaDialog(block);  
5897:     if (updated) refreshRichFormulaBlock(block, updated);  
5898:     return;  
5899:   }  
5900:   if (block.dataset.type === "code") {  
5901:     const updated = await openSummaryCodeDialog(block);  
5902:     if (updated) refreshRichCodeBlock(block, updated);  
5903:     return;  
5904:   }  
5905:   if (block.dataset.type === "paragraph") {  
5906:     focusTextBlock(block);  
5907:     setStatus("Text block selected. Type directly in the block to edit it.");  
5908:   }  
5909: }
```

copyRichCodeBlock (template-preview.js, lines 5911-5918)

copyRichCodeBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5911-5918 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, writeText, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5912: if (!block || block.dataset.type !== "code") return false;; line 5914: if (!code) return false;; line 5917: return true;.

Important local values include code at line 5913. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5911: async function copyRichCodeBlock(block) {
5912:   if (!block || block.dataset.type !== "code") return false;
5913:   const code = block.dataset.code || block.querySelector("code")?.textContent || "";
5914:   if (!code) return false;
5915:   await navigator.clipboard.writeText(code);
5916:   setStatus("Code copied to clipboard.");
5917:   return true;
5918: }
```

removeRichBlock (template-preview.js, lines 5920-5931)

removeRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5920-5931 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references matches, remove, querySelector, append, createRichTextBlock, and focusTextBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5921: if (!block) return;.

Important local values include nextFocus at line 5922. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5920: function removeRichBlock(block, editor) {
5921:   if (!block) return;
5922:   const nextFocus = block.previousElementSibling?.matches(".rich-text-block")
5923:     ? block.previousElementSibling
5924:     : block.nextElementSibling?.matches(".rich-text-block")
5925:     ? block.nextElementSibling
5926:     : null;
5927:   block.remove();
5928:   activeSummaryBlock = null;
5929:   if (!editor.querySelector(".rich-block")) editor.append(createRichTextBlock(""));
5930:   focusTextBlock(nextFocus || editor.querySelector(".rich-text-block"));
5931: }
```

deleteSelectedRichBlock (template-preview.js, lines 5933-5948)

deleteSelectedRichBlock part of the private builder frontend and editing experience. In this file, it appears around lines 5933-5948 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedRichBlock, openDeleteConfirm, and removeRichBlock. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5935: if (!block) return;.

Important local values include block at line 5934; label at line 5936. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5933: function deleteSelectedRichBlock(editor) {
5934:   const block = selectedRichBlock(editor);
5935:   if (!block) return;
5936:   const label = block.dataset.type === "image"
5937:     ? "this overview image"
5938:     : block.dataset.type === "formula"
5939:     ? "this formula"
5940:     : block.dataset.type === "code"
5941:     ? "this code block"
5942:     : "this text block";
5943:   openDeleteConfirm({
5944:     title: "Delete overview block",
5945:     message: `Are you sure you want to delete ${label}?`,
5946:     onConfirm: () => removeRichBlock(block, editor)
5947:   });
5948: }
```

deleteProjectSummary (template-preview.js, lines 5950-5966)

deleteProjectSummary part of the private builder frontend and editing experience. In this file, it appears around lines 5950-5966 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references openDeleteConfirm, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5951: if (!project) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
5950: function deleteProjectSummary(project) {
5951:   if (!project) return;
5952:   openDeleteConfirm({
5953:     title: "Delete project overview",
5954:     message: "Are you sure you want to delete the current project overview?",
5955:     onConfirm: () => {
5956:       project.summary = "";
5957:       project.summaryRich = null;
5958:       summaryEditorProjectId = "";
5959:       activeSummaryEditor = null;
5960:       activeSummaryBlock = null;
5961:       setStatus("Project overview deleted in the editor. Click Save project to update the portfolio
preview.");
5962:       scheduleAutosave();
5963:       renderAll();
5964:     }
5965:   });
5966: }
```

saveRichSummary (template-preview.js, lines 5968-5987)

saveRichSummary persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 5968-5987 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, querySelector, extractRichSummary, plainTextFromRich, wordCount, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 5971: if (!project || !editor) return true;; line 5976: return false;; line 5986: return true;.

Important local values include project at line 5969; editor at line 5970; rich at line 5972; plain at line 5973. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
5968: async function saveRichSummary() {
5969:   const project = selectedProject();
5970:   const editor = projectFields.querySelector("[data-rich-editor='summary']");
5971:   if (!project || !editor) return true;
5972:   const rich = extractRichSummary(editor);
5973:   const plain = plainTextFromRich(rich);
5974:   if (wordCount(plain) > 1000) {
5975:     setStatus("Project overview is limited to 1000 words. Shorten the text before saving.");
5976:     return false;
5977:   }
5978:   project.summaryRich = rich;
5979:   project.summary = plain;
5980:   summaryEditorProjectId = "";
5981:   activeSummaryEditor = null;
5982:   activeSummaryBlock = null;
5983:   setStatus("Overview saved in the editor. Click Save project to include this version in the portfolio
preview.");
5984:   scheduleAutosave();
5985:   renderAll();
5986:   return true;
5987: }
```

renderFields (template-preview.js, lines 5989-6004)

renderFields renders ui, markup, or display output. In this file, it appears around lines 5989-6004 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderSummaryBuilder, requestAnimationFrame, and populateSummaryEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 5993: return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
5989: function renderFields(project) {
5990:   if (!project) {
5991:     projectFields.hidden = false;
5992:     projectFields.innerHTML = `
```

sectionOptions (template-preview.js, lines 6006-6016)

sectionOptions part of the private builder frontend and editing experience. In this file, it appears around lines 6006-6016 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, filter, and isAnalogProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6012: return [.

Important local values include custom at line 6007. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6006: function sectionOptions(project) {
6007:   const custom = (project.sections || []).map((section) => ({
6008:     id: `custom:${section.id}`,
6009:     label: section.title,
6010:     folder: section.id
6011:   }));
6012:   return [
6013:     ...standardSections.filter((section) => !section.analogOnly || isAnalogProject(project)),
6014:     ...custom
6015:   ];
6016: }
```

custom (template-preview.js, lines 6007-6011)

custom part of the private builder frontend and editing experience. In this file, it appears around lines 6007-6011 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include custom at line 6007. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6007:   const custom = (project.sections || []).map((section) => ({
6008:     id: `custom:${section.id}`,
6009:     label: section.title,
6010:     folder: section.id
6011:   }));

```

ensureSiteSections (template-preview.js, lines 6018-6021)

ensureSiteSections part of the private builder frontend and editing experience. In this file, it appears around lines 6018-6021 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clone. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

6018: function ensureSiteSections() {
6019:   catalog.siteSections = catalog.siteSections || [];
6020:   savedPortfolioCatalog.siteSections = savedPortfolioCatalog.siteSections || clone(catalog.siteSections
|| []);
6021: }

```

siteSectionHasContent (template-preview.js, lines 6023-6032)

siteSectionHasContent part of the private builder frontend and editing experience. In this file, it appears around lines 6023-6032 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasContent, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6024: return Boolean(.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

6023: function siteSectionHasContent(section) {
6024:   return Boolean(
6025:     section?.description ||
6026:     richHasContent(section?.richDescription) ||
6027:     section?.backgroundImage ||
6028:     (section?.links || []).some((link) => link.label || link.url) ||
6029:     (section?.assets || []).some((asset) => asset.title || asset.url) ||
6030:     (section?.subsections || []).some((item) => item.title || item.description ||
richHasContent(item.richDescription) || (item.links || []).length)
6031:   );
6032: }

```

siteSectionRenderable (template-preview.js, lines 6034-6036)

siteSectionRenderable part of the private builder frontend and editing experience. In this file, it appears around lines 6034-6036 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references siteSectionHasContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6035: return section?.visible !== false && siteSectionHasContent(section);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

6034: function siteSectionRenderable(section) {
6035:   return section?.visible !== false && siteSectionHasContent(section);
6036: }

```

siteSectionLinkCount (template-preview.js, lines 6038-6044)

siteSectionLinkCount part of the private builder frontend and editing experience. In this file, it appears around lines 6038-6044 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flatMap, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6039: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6038: function siteSectionLinkCount(section) {
6039:   return [
6040:     ...(section.links || []),
6041:     ...(section.assets || []),
6042:     ...(section.subsections || []).flatMap((item) => item.links || [])
6043:   ].filter((item) => item.label || item.url).length;
6044: }
```

renderSiteSectionList (template-preview.js, lines 6046-6094)

renderSiteSectionList renders ui, markup, or display output. In this file, it appears around lines 6046-6094 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSiteSections, map, url, siteSectionLinkCount, richHasContent, renderRichContent, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include sections at line 6048. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6046: function renderSiteSectionList() {
6047:   ensureSiteSections();
6048:   const sections = catalog.siteSections || [];
6049:   siteSectionList.innerHTML = sections.length ? sections.map((section) => `
6050:     <article class="site-section-item ${section.visible === false ? "is-hidden-section" : ""}"
6051:     style="${section.backgroundImage ? `--site-section-bg: url('${section.backgroundImage}');` : ""}">
6052:       <div>
6053:         <strong>${section.title || "Untitled section"}</strong>
6054:         <span>${section.visible === false ? "Hidden from website" : "Visible on website"} .
6055:         ${section.subsections || []}.length> subsection${(section.subsections || []).length === 1 ? "" : "s"} .
6056:         ${siteSectionLinkCount(section)} link${(siteSectionLinkCount(section) === 1 ? "" : "s")}</span>
6057:         <div>
6058:           <strong>${section.title || "Untitled subsection"}</strong>
6059:           <span>${section.description || richHasContent(section.richDescription) ?
6060:             renderRichContent(section.richDescription, section.description || "") : ""}</span>
6061:           <div>
6062:             <div class="site-subsection-list">
6063:               ${section.subsections || []}.length ? `
6064:                 <div class="site-subsection-item">
6065:                   <div>
6066:                     <strong>${item.title || "Untitled subsection"}</strong>
6067:                     <span>${item.description || richHasContent(item.richDescription) ?
6068:                       renderRichContent(item.richDescription, item.description || "") : ""}</span>
6069:                     <div>
6070:                       <div class="site-section-actions">
6071:                         <button type="button" data-site-subsection-edit="${section.id}" data-
6072:                         index="${index}">Edit</button>
6073:                         <button class="danger-icon" type="button" data-site-subsection-delete="${section.id}"
6074:                         data-index="${index}">Delete</button>
6075:                       </div>
6076:                     </div>
6077:                   </div>
6078:                 </div>
6079:               </div>
6080:             </div>
6081:             <div class="site-link-list">
6082:               ${section.links || []}.length ? `
6083:                 <div class="site-link-item">
6084:                   <div>
6085:                     <strong>${link.title || "Untitled link"}</strong>
6086:                     <span>${link.description || richHasContent(link.richDescription) ?
6087:                       renderRichContent(link.richDescription, link.description || "") : ""}</span>
6088:                     <div>
6089:                       <div class="site-section-actions">
6090:                         <button type="button" data-site-link-edit="${section.id}" data-
6091:                         index="${index}">Edit</button>
6092:                         <button class="danger-icon" type="button" data-site-link-delete="${section.id}"
6093:                         data-index="${index}">Delete</button>
6094:                       </div>
6095:                     </div>
6096:                   </div>
6097:                 </div>
6098:               </div>
6099:             </div>
6100:           </div>
6101:         </div>
6102:       </div>
6103:     </div>
6104:   `).join("")}
6105: }
```

```

6078:         <button type="button" data-site-link-delete="${section.id}" data-
index="${index}">Delete</button>
6079:     </span>
6080:     `).join("")}}
6081: </div>
6082: ` : ""}
6083: <div class="site-section-actions">
6084:     <button type="button" data-site-section-toggle="${section.id}">${section.visible === false ?
"Show" : "Hide"}</button>
6085:     <button type="button" data-site-section-edit="${section.id}">Edit</button>
6086:     <button type="button" data-site-subsection-add="${section.id}">Add subsection</button>
6087:     <button type="button" data-site-link-add="${section.id}">Add link</button>
6088:     <button type="button" data-site-section-bg="${section.id}">Background</button>
6089:     ${section.backgroundImage ? `<button type="button" data-site-section-bg-
clear="${section.id}">Clear bg</button>` : ""}
6090:     <button class="danger-icon" type="button" data-site-section-
delete="${section.id}">Delete</button>
6091: </div>
6092: </article>
6093: `).join("") : `<p class="evidence-empty">No extra portfolio areas added yet.</p>`;
6094: }

```

siteSectionDialogValue (template-preview.js, lines 6096-6118)

siteSectionDialogValue part of the private builder frontend and editing experience. In this file, it appears around lines 6096-6118 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references populateStandaloneRichEditor, dialogValue, trim, setStatus, extractRichSummary, plainTextFromRich, and slugify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 6105: if (!saved) return null;; line 6109: return null;; line 6112: return {.

Important local values include saved at line 6104; title at line 6106; richDescription at line 6111. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6096: async function siteSectionDialogValue(section = {}) {
6097:   sectionDialogTitle.textContent = section.id ? "Edit section" : "Create section";
6098:   sectionTitle.placeholder = "Professional profile, Personal background, Sports...";
6099:   sectionDescription.dataset.placeholder = "Short section overview shown on the website";
6100:   sectionDescription.dataset.richFolder = section.id || "new-portfolio-section";
6101:   sectionDescription.dataset.richProjectId = "_site-sections";
6102:   sectionTitle.value = section.title || "";
6103:   populateStandaloneRichEditor(sectionDescription, section.richDescription, section.description || "");
6104:   const saved = await dialogValue(sectionDialog);
6105:   if (!saved) return null;
6106:   const title = sectionTitle.value.trim();
6107:   if (!title) {
6108:     setStatus("Type a section title before saving.");
6109:     return null;
6110:   }
6111:   const richDescription = extractRichSummary(sectionDescription);
6112:   return {
6113:     description: plainTextFromRich(richDescription, "\n").trim(),
6114:     id: section.id || slugify(title),
6115:     richDescription,
6116:     title
6117:   };
6118: }

```

siteSubsectionDialogValue (template-preview.js, lines 6120-6143)

siteSubsectionDialogValue part of the private builder frontend and editing experience. In this file, it appears around lines 6120-6143 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references populateStandaloneRichEditor, dialogValue, trim, setStatus, extractRichSummary, plainTextFromRich, and slugify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 6129: if (!saved) return null;; line 6133: return null;; line 6136: return {.

Important local values include saved at line 6128; title at line 6130; richDescription at line 6135. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6120: async function siteSubsectionDialogValue(subsection = {}) {
6121:   sectionDialogTitle.textContent = subsection.id ? "Edit subsection" : "Create subsection";
6122:   sectionTitle.placeholder = "Sports, Hobbies, Resume links, Origin...";
6123:   sectionDescription.dataset.placeholder = "Short subsection text shown inside the portfolio area";
6124:   sectionDescription.dataset.richFolder = subsection.id || "new-portfolio-subsection";
6125:   sectionDescription.dataset.richProjectId = "_site-sections";
6126:   sectionTitle.value = subsection.title || "";
6127:   populateStandaloneRichEditor(sectionDescription, subsection.richDescription, subsection.description ||
  "");
6128:   const saved = await dialogValue(sectionDialog);
6129:   if (!saved) return null;
6130:   const title = sectionTitle.value.trim();
6131:   if (!title) {
6132:     setStatus("Type a subsection title before saving.");
6133:     return null;
6134:   }
6135:   const richDescription = extractRichSummary(sectionDescription);
6136:   return {
6137:     description: plainTextFromRich(richDescription, "\n").trim(),
6138:     id: subsection.id || slugify(title),
6139:     links: subsection.links || [],
6140:     richDescription,
6141:     title
6142:   };
6143: }
```

dialogLinkValue (template-preview.js, lines 6145-6169)

dialogLinkValue part of the private builder frontend and editing experience. In this file, it appears around lines 6145-6169 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references removeEventListener, resolve, trim, setStatus, normalizeLinkTarget, addEventListener, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 6146: return new Promise((resolve) => {; line 6155: return;; line 6162: return;;

Important local values include onClose at line 6151; label at line 6157; url at line 6158. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6145: function dialogLinkValue(link = {}) {
6146:   return new Promise((resolve) => {
6147:     siteLinkEyebrow.textContent = link.url ? "Edit portfolio link" : "Add portfolio link";
6148:     siteLinkTitle.textContent = link.url ? "Edit link" : "Add link";
6149:     siteLinkLabel.value = link.label || "";
6150:     siteLinkUrl.value = link.url || "";
6151:     const onClose = () => {
6152:       siteLinkDialog.removeEventListener("close", onClose);
6153:       if (siteLinkDialog.returnValue !== "save") {
6154:         resolve(null);
6155:         return;
6156:       }
6157:       const label = siteLinkLabel.value.trim();
6158:       const url = siteLinkUrl.value.trim();
6159:       if (!label || !url) {
6160:         setStatus("Type both a link label and URL before saving.");
6161:         resolve(null);
6162:         return;
6163:       }
6164:       resolve({ ...link, label, url: normalizeLinkTarget(url, { assumeweb: true }) });
6165:     };
6166:     siteLinkDialog.addEventListener("close", onClose);
6167:     siteLinkDialog.showModal();
6168:   });
6169: }
```

onClose (template-preview.js, lines 6151-6165)

onClose part of the private builder frontend and editing experience. In this file, it appears around lines 6151-6165 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references removeEventListener, resolve, trim, setStatus, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6155: return;; line 6162: return;.

Important local values include onClose at line 6151; label at line 6157; url at line 6158. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6151:     const onClose = () => {
6152:       siteLinkDialog.removeEventListener("close", onClose);
6153:       if (siteLinkDialog.returnValue !== "save") {
6154:         resolve(null);
6155:         return;
6156:       }
6157:       const label = siteLinkLabel.value.trim();
6158:       const url = siteLinkUrl.value.trim();
6159:       if (!label || !url) {
6160:         setStatus("Type both a link label and URL before saving.");
6161:         resolve(null);
6162:         return;
6163:       }
6164:       resolve({ ...link, label, url: normalizeLinkTarget(url, { assumeWeb: true }) });
6165:     };
```

addSiteSection (template-preview.js, lines 6171-6191)

addSiteSection part of the private builder frontend and editing experience. In this file, it appears around lines 6171-6191 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSiteSections, siteSectionDialogValue, slugify, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6174: if (!input) return;.

Important local values include input at line 6173; id at line 6175. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6171: async function addSiteSection() {
6172:   ensureSiteSections();
6173:   const input = await siteSectionDialogValue();
6174:   if (!input) return;
6175:   const id = input.id || slugify(input.title);
6176:   catalog.siteSections.push({
6177:     id,
6178:     title: input.title,
6179:     description: input.description,
6180:     richDescription: input.richDescription,
6181:     visible: false,
6182:     backgroundImage: "",
6183:     links: [],
6184:     assets: [],
6185:     subsections: []
6186:   });
6187:   markDraftNeedsSave();
6188:   setStatus("Portfolio area added locally.");
6189:   scheduleAutosave();
6190:   renderAll();
6191: }
```

editSiteSection (template-preview.js, lines 6193-6206)

editSiteSection part of the private builder frontend and editing experience. In this file, it appears around lines 6193-6206 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSiteSections, find, siteSectionDialogValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6196: if (!section) return;; line 6198: if (!input) return;.

Important local values include section at line 6195; input at line 6197. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6193: async function editSiteSection(sectionId) {
6194:   ensureSiteSections();
6195:   const section = catalog.siteSections.find((item) => item.id === sectionId);
6196:   if (!section) return;
6197:   const input = await siteSectionDialogValue(section);
6198:   if (!input) return;
6199:   section.title = input.title;
6200:   section.description = input.description;
6201:   section.richDescription = input.richDescription;
6202:   markDraftNeedsSave();
6203:   setStatus("Portfolio area edited locally.");
6204:   scheduleAutosave();
6205:   renderAll();
6206: }
```

addSiteSubsection (template-preview.js, lines 6208-6219)

addSiteSubsection part of the private builder frontend and editing experience. In this file, it appears around lines 6208-6219 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, siteSubsectionDialogValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6210: if (!section) return;; line 6212: if (!input) return;.

Important local values include section at line 6209; input at line 6211. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6208: async function addSiteSubsection(sectionId) {
6209:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6210:   if (!section) return;
6211:   const input = await siteSubsectionDialogValue();
6212:   if (!input) return;
6213:   section.subsections = section.subsections || [];
6214:   section.subsections.push(input);
6215:   markDraftNeedsSave();
6216:   setStatus("Portfolio subsection added locally.");
6217:   scheduleAutosave();
6218:   renderAll();
6219: }
```

section (template-preview.js, lines 6209-6218)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6209-6218 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, siteSubsectionDialogValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6210: if (!section) return;; line 6212: if (!input) return;.

Important local values include section at line 6209; input at line 6211. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6209:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6210:   if (!section) return;
6211:   const input = await siteSubsectionDialogValue();
6212:   if (!input) return;
6213:   section.subsections = section.subsections || [];
6214:   section.subsections.push(input);
6215:   markDraftNeedsSave();
6216:   setStatus("Portfolio subsection added locally.");
```

```
6217:   scheduleAutosave();
6218:   renderAll();
```

editSiteSubsection (template-preview.js, lines 6221-6232)

editSiteSubsection part of the private builder frontend and editing experience. In this file, it appears around lines 6221-6232 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, siteSubsectionDialogValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6224: if (!section || !subsection) return;; line 6226: if (!input) return;.

Important local values include section at line 6222; subsection at line 6223; input at line 6225. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6221: async function editSiteSubsection(sectionId, index) {
6222:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6223:   const subsection = section?.subsections?.[Number(index)];
6224:   if (!section || !subsection) return;
6225:   const input = await siteSubsectionDialogValue(subsection);
6226:   if (!input) return;
6227:   section.subsections[Number(index)] = input;
6228:   markDraftNeedsSave();
6229:   setStatus("Portfolio subsection edited locally.");
6230:   scheduleAutosave();
6231:   renderAll();
6232: }
```

section (template-preview.js, lines 6222-6231)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6222-6231 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, siteSubsectionDialogValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6224: if (!section || !subsection) return;; line 6226: if (!input) return;.

Important local values include section at line 6222; subsection at line 6223; input at line 6225. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6222:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6223:   const subsection = section?.subsections?.[Number(index)];
6224:   if (!section || !subsection) return;
6225:   const input = await siteSubsectionDialogValue(subsection);
6226:   if (!input) return;
6227:   section.subsections[Number(index)] = input;
6228:   markDraftNeedsSave();
6229:   setStatus("Portfolio subsection edited locally.");
6230:   scheduleAutosave();
6231:   renderAll();
```

deleteSiteSubsection (template-preview.js, lines 6234-6248)

deleteSiteSubsection part of the private builder frontend and editing experience. In this file, it appears around lines 6234-6248 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, splice, markDraftNeedsSave, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6237: if (!section || !subsection) return;.

Important local values include section at line 6235; subsection at line 6236. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6234: function deleteSiteSubsection(sectionId, index) {
6235:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6236:   const subsection = section?.subsections?.[Number(index)];
6237:   if (!section || !subsection) return;
6238:   openDeleteConfirm({
6239:     title: "Delete subsection",
6240:     message: `Are you sure you want to delete "${subsection.title}"?`,
6241:     onConfirm: () => {
6242:       section.subsections.splice(Number(index), 1);
6243:       markDraftNeedsSave();
6244:       scheduleAutosave();
6245:       renderAll();
6246:     }
6247:   });
6248: }
```

section (template-preview.js, lines 6235-6247)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6235-6247 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, splice, markDraftNeedsSave, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6237: if (!section || !subsection) return;.

Important local values include section at line 6235; subsection at line 6236. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6235:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6236:   const subsection = section?.subsections?.[Number(index)];
6237:   if (!section || !subsection) return;
6238:   openDeleteConfirm({
6239:     title: "Delete subsection",
6240:     message: `Are you sure you want to delete "${subsection.title}"?`,
6241:     onConfirm: () => {
6242:       section.subsections.splice(Number(index), 1);
6243:       markDraftNeedsSave();
6244:       scheduleAutosave();
6245:       renderAll();
6246:     }
6247:   });
```

addSiteSectionLink (template-preview.js, lines 6250-6261)

addSiteSectionLink part of the private builder frontend and editing experience. In this file, it appears around lines 6250-6261 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, dialogLinkValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6252: if (!section) return;; line 6254: if (!input) return;.

Important local values include section at line 6251; input at line 6253. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6250: async function addSiteSectionLink(sectionId) {
6251:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6252:   if (!section) return;
6253:   const input = await dialogLinkValue();
6254:   if (!input) return;
6255:   section.links = section.links || [];
6256:   section.links.push(input);
6257:   markDraftNeedsSave();
6258:   setStatus("Portfolio link added locally.");
6259:   scheduleAutosave();
6260:   renderAll();
```

```
6261: }
```

section (template-preview.js, lines 6251-6260)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6251-6260 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, dialogLinkValue, push, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6252: if (!section) return;; line 6254: if (!input) return;.

Important local values include section at line 6251; input at line 6253. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6251: const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6252: if (!section) return;
6253: const input = await dialogLinkValue();
6254: if (!input) return;
6255: section.links = section.links || [];
6256: section.links.push(input);
6257: markDraftNeedsSave();
6258: setStatus("Portfolio link added locally.");
6259: scheduleAutosave();
6260: renderAll();
```

editSiteSectionLink (template-preview.js, lines 6263-6274)

editSiteSectionLink part of the private builder frontend and editing experience. In this file, it appears around lines 6263-6274 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, dialogLinkValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6266: if (!section || !link) return;; line 6268: if (!input) return;.

Important local values include section at line 6264; link at line 6265; input at line 6267. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6263: async function editSiteSectionLink(sectionId, index) {
6264:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6265:   const link = section?.links?.[Number(index)];
6266:   if (!section || !link) return;
6267:   const input = await dialogLinkValue(link);
6268:   if (!input) return;
6269:   section.links[Number(index)] = input;
6270:   markDraftNeedsSave();
6271:   setStatus("Portfolio link edited locally.");
6272:   scheduleAutosave();
6273:   renderAll();
6274: }
```

section (template-preview.js, lines 6264-6273)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6264-6273 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, dialogLinkValue, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6266: if (!section || !link) return;; line 6268: if (!input) return;.

Important local values include section at line 6264; link at line 6265; input at line 6267. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6264:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6265:   const link = section?.links?.[Number(index)];
6266:   if (!section || !link) return;
6267:   const input = await dialogLinkValue(link);
6268:   if (!input) return;
6269:   section.links[Number(index)] = input;
6270:   markDraftNeedsSave();
6271:   setStatus("Portfolio link edited locally.");
6272:   scheduleAutosave();
6273:   renderAll();

```

deleteSiteSectionLink (template-preview.js, lines 6276-6284)

deleteSiteSectionLink part of the private builder frontend and editing experience. In this file, it appears around lines 6276-6284 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, splice, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6278: if (!section?.links?.[Number(index)]) return;.

Important local values include section at line 6277. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6276: function deleteSiteSectionLink(sectionId, index) {
6277:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6278:   if (!section?.links?.[Number(index)]) return;
6279:   section.links.splice(Number(index), 1);
6280:   markDraftNeedsSave();
6281:   setStatus("Portfolio link deleted locally.");
6282:   scheduleAutosave();
6283:   renderAll();
6284: }

```

section (template-preview.js, lines 6277-6283)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6277-6283 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, splice, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6278: if (!section?.links?.[Number(index)]) return;.

Important local values include section at line 6277. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6277:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6278:   if (!section?.links?.[Number(index)]) return;
6279:   section.links.splice(Number(index), 1);
6280:   markDraftNeedsSave();
6281:   setStatus("Portfolio link deleted locally.");
6282:   scheduleAutosave();
6283:   renderAll();

```

toggleSiteSectionVisibility (template-preview.js, lines 6286-6294)

toggleSiteSectionVisibility part of the private builder frontend and editing experience. In this file, it appears around lines 6286-6294 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6288: if (!section) return;.

Important local values include section at line 6287. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6286: function toggleSiteSectionVisibility(sectionId) {
6287:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6288:   if (!section) return;
6289:   section.visible = section.visible === false;
6290:   markDraftNeedsSave();
6291:   setStatus(section.visible ? "Portfolio area will show on the website preview." : "Portfolio area hidden
from the website preview.");
6292:   scheduleAutosave();
6293:   renderAll();
6294: }
```

section (template-preview.js, lines 6287-6293)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6287-6293 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6288: if (!section) return;.

Important local values include section at line 6287. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6287:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6288:   if (!section) return;
6289:   section.visible = section.visible === false;
6290:   markDraftNeedsSave();
6291:   setStatus(section.visible ? "Portfolio area will show on the website preview." : "Portfolio area hidden
from the website preview.");
6292:   scheduleAutosave();
6293:   renderAll();
```

deleteSiteSection (template-preview.js, lines 6296-6310)

deleteSiteSection part of the private builder frontend and editing experience. In this file, it appears around lines 6296-6310 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, filter, markDraftNeedsSave, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6298: if (!section) return;.

Important local values include section at line 6297. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6296: function deleteSiteSection(sectionId) {
6297:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6298:   if (!section) return;
6299:   openDeleteConfirm({
6300:     title: "Delete portfolio area",
6301:     message: `Are you sure you want to delete "${section.title}"?`,
6302:     onConfirm: () => {
6303:       catalog.siteSections = (catalog.siteSections || []).filter((item) => item.id !== sectionId);
6304:       savedPortfolioCatalog.siteSections = (savedPortfolioCatalog.siteSections || []).filter((item) =>
item.id !== sectionId);
6305:       markDraftNeedsSave();
6306:       scheduleAutosave();
6307:       renderAll();
6308:     }
6309:   });
6310: }
```

section (template-preview.js, lines 6297-6309)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6297-6309 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, openDeleteConfirm, filter, markDraftNeedsSave, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6298: if (!section) return;.

Important local values include section at line 6297. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6297: const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6298: if (!section) return;
6299: openDeleteConfirm({
6300:   title: "Delete portfolio area",
6301:   message: `Are you sure you want to delete "${section.title}"?`,
6302:   onConfirm: () => {
6303:     catalog.siteSections = (catalog.siteSections || []).filter((item) => item.id !== sectionId);
6304:     savedPortfolioCatalog.siteSections = (savedPortfolioCatalog.siteSections || []).filter((item) =>
item.id !== sectionId);
6305:     markDraftNeedsSave();
6306:     scheduleAutosave();
6307:     renderAll();
6308:   }
6309: });
```

setSiteSectionBackground (template-preview.js, lines 6312-6343)

setSiteSectionBackground part of the private builder frontend and editing experience. In this file, it appears around lines 6312-6343 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, chooseFile, startsWith, setStatus, readFileAsDataURL, fetch, now, stringify, json, markDraftNeedsSave, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 6314: if (!section) return;; line 6316: if (!file) return;; line 6319: return;; line 6336: return;.

Important local values include section at line 6313; file at line 6315; data at line 6321; response at line 6322; result at line 6333. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6312: async function setSiteSectionBackground(sectionId) {
6313:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6314:   if (!section) return;
6315:   const file = await chooseFile();
6316:   if (!file) return;
6317:   if (!file.type.startsWith("image/")) {
6318:     setStatus("Choose an image file for a section background.");
6319:     return;
6320:   }
6321:   const data = await readFileAsDataURL(file);
6322:   const response = await fetch(`/api/upload?t=${Date.now()}`, {
6323:     method: "POST",
6324:     cache: "no-store",
6325:     headers: { "Content-Type": "application/json" },
6326:     body: JSON.stringify({
6327:       projectId: "_site-sections",
6328:       section: section.id,
6329:       fileName: file.name,
6330:       data
6331:     })
6332:   });
6333:   const result = await response.json();
6334:   if (!response.ok) {
6335:     setStatus(result.error || "Background upload failed.");
6336:     return;
6337:   }
6338:   section.backgroundImage = result.url;
6339:   markDraftNeedsSave();
6340:   setStatus("Portfolio area background saved locally.");
6341:   scheduleAutosave();
6342:   renderAll();
6343: }
```

section (template-preview.js, lines 6313-6320)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6313-6320 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, chooseFile, startsWith, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 6314: if (!section) return;; line 6316: if (!file) return;; line 6319: return;.

Important local values include section at line 6313; file at line 6315. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6313: const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6314: if (!section) return;
6315: const file = await chooseFile();
6316: if (!file) return;
6317: if (!file.type.startsWith("image/")) {
6318:   setStatus("Choose an image file for a section background.");
6319:   return;
6320: }
```

clearSiteSectionBackground (template-preview.js, lines 6345-6353)

clearSiteSectionBackground part of the private builder frontend and editing experience. In this file, it appears around lines 6345-6353 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6347: if (!section) return;.

Important local values include section at line 6346. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6345: function clearSiteSectionBackground(sectionId) {
6346:   const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6347:   if (!section) return;
6348:   section.backgroundImage = "";
6349:   markDraftNeedsSave();
6350:   setStatus("Portfolio area background cleared locally.");
6351:   scheduleAutosave();
6352:   renderAll();
6353: }
```

section (template-preview.js, lines 6346-6352)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6346-6352 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, markDraftNeedsSave, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6347: if (!section) return;.

Important local values include section at line 6346. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6346: const section = (catalog.siteSections || []).find((item) => item.id === sectionId);
6347: if (!section) return;
6348: section.backgroundImage = "";
6349: markDraftNeedsSave();
6350: setStatus("Portfolio area background cleared locally.");
```



```
6351:   scheduleAutosave();
6352:   renderAll();
```

renderSectionTabs (template-preview.js, lines 6355-6369)

renderSectionTabs renders ui, markup, or display output. In this file, it appears around lines 6355-6369 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionOptions, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6358: return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6355: function renderSectionTabs(project) {
6356:   if (!project) {
6357:     sectionTabs.innerHTML = "";
6358:     return;
6359:   }
6360:
6361:   sectionTabs.innerHTML = `
6362:     ${sectionOptions(project).map((section) => `
6363:       <button class="${section.id === activeSectionId ? "active" : ""}" type="button" data-section-
id="${section.id}">
6364:         ${section.label}
6365:       </button>
6366:     `).join("")}
6367:     <button type="button" data-add-section="true">Add section</button>
6368:   `;
6369: }
```

renderTextArray (template-preview.js, lines 6371-6389)

renderTextArray renders ui, markup, or display output. In this file, it appears around lines 6371-6389 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, join, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6373: return `.

Important local values include items at line 6372. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6371: function renderTextArray(project, key, label) {
6372:   const items = project[key] || [];
6373:   return `
6374:     <div class="section-actions">
6375:       <button class="button primary" type="button" data-open-editor="text-array" data-key="${key}" data-
mode="add">Add ${label}</button>
6376:     </div>
6377:     <div class="builder-items">
6378:       ${items.map((item, index) => `
6379:         <article class="builder-item">
6380:           <div>
6381:             <strong>${typeof item === "string" ? item : item.title || item.name || item.label ||
""}</strong>
6382:           </div>
6383:           <button type="button" data-open-editor="text-array" data-key="${key}" data-mode="edit" data-
index="${index}">Edit</button>
6384:           <button class="danger-icon" type="button" data-delete-array="${key}" data-
index="${index}">Delete</button>
6385:         </article>
6386:       `).join("")} || <p class="evidence-empty">No ${label.toLowerCase()} added yet.</p>`
6387:     </div>
6388:   `;
6389: }
```

toolName (template-preview.js, lines 6391-6393)

toolName part of the private builder frontend and editing experience. In this file, it appears around lines 6391-6393 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6392: return typeof item === "string" ? item : item.name || item.title || item.label || "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6391: function toolName(item) {
6392:   return typeof item === "string" ? item : item.name || item.title || item.label || "";
6393: }
```

toolDescription (template-preview.js, lines 6395-6397)

toolDescription part of the private builder frontend and editing experience. In this file, it appears around lines 6395-6397 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6396: return typeof item === "string" ? "" : item.description || item.usedFor || "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6395: function toolDescription(item) {
6396:   return typeof item === "string" ? "" : item.description || item.usedFor || "";
6397: }
```

renderToolsSection (template-preview.js, lines 6399-6420)

renderToolsSection renders ui, markup, or display output. In this file, it appears around lines 6399-6420 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, toolName, richHasContent, renderRichContent, toolDescription, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6401: return `

Important local values include items at line 6400. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6399: function renderToolsSection(project) {
6400:   const items = project.tools || [];
6401:   return `
6402:     <div class="section-actions">
6403:       <button class="button primary" type="button" data-open-editor="tool" data-mode="add">Add
tool</button>
6404:     </div>
6405:     <div class="builder-items tool-items">
6406:       ${items.map((item, index) => `
6407:         <article class="builder-item tool-item">
6408:           <div>
6409:             <strong>${toolName(item)} || "Untitled tool"</strong>
6410:           </div>
6411:           ${richHasContent(item?.richDescription)
6412:             ? renderRichContent(item.richDescription, toolDescription(item))
6413:             : `<p>${toolDescription(item)} || "No usage description added yet."</p>`}
6414:           <button type="button" data-open-editor="tool" data-mode="edit" data-
index="${index}">Edit</button>
6415:           <button class="danger-icon" type="button" data-delete-array="tools" data-
index="${index}">Delete</button>
```

```

6416:         </article>
6417:     `).join("") || `
```

renderObjectSection (template-preview.js, lines 6422-6450)

renderObjectSection renders ui, markup, or display output. In this file, it appears around lines 6422-6450 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, richHasContent, renderRichContent, renderMultilineInlineText, join, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6424: return `; line 6434: return `.

Important local values include items at line 6423; title at line 6431; url at line 6432; description at line 6433. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6422: function renderObjectSection(project, key, folder, label) {
6423:     const items = project[key] || [];
6424:     return `
6425:         <div class="section-actions">
6426:             <button class="button primary" type="button" data-open-editor="object" data-key="${key}" data-
mode="add">Add text item</button>
6427:             ${folder ? `<button class="button secondary" type="button" data-upload="${key}" data-
folder="${folder}">Add file or image</button>` : ""}
6428:         </div>
6429:         <div class="builder-items">
6430:             ${items.map((item, index) => {
6431:                 const title = item.title || item.name || item.label || "Untitled item";
6432:                 const url = item.url || item.artifact || "";
6433:                 const description = item.description || item.caption || item.result || item.method || "";
6434:                 return `
6435:                     <article class="builder-item">
6436:                         <div>
6437:                             <strong>${title}</strong>
6438:                             <span>${url || "No file attached"}</span>
6439:                             ${richHasContent(item.richDescription)
6440:                                 ? renderRichContent(item.richDescription, description)
6441:                                 : description ? `<p>${renderMultilineInlineText(description)}</p>` : ""}
6442:                         </div>
6443:                         <button type="button" data-open-editor="object" data-key="${key}" data-mode="edit" data-
index="${index}">Edit</button>
6444:                         <button class="danger-icon" type="button" data-delete-item="${key}" data-
index="${index}">Delete</button>
6445:                     </article>
6446:                 `;
6447:             }).join("") || `
```

designArray (template-preview.js, lines 6452-6458)

designArray part of the private builder frontend and editing experience. In this file, it appears around lines 6452-6458 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureDesignModel, getByPath, isArray, and setByPath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6455: if (Array.isArray(items)) return items;; line 6457: return getByPath(project, pathValue);.

Important local values include items at line 6454. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6452: function designArray(project, pathValue) {
6453:     ensureDesignModel(project);
6454:     const items = getByPath(project, pathValue);
6455:     if (Array.isArray(items)) return items;
6456:     setByPath(project, pathValue, []);
6457:     return getByPath(project, pathValue);

```

6458: }

makeDesignItem (template-preview.js, lines 6460-6466)

makeDesignItem part of the private builder frontend and editing experience. In this file, it appears around lines 6460-6466 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 6461: if (kind === "reference") return { title, description, url, type: "Reference", status: url ? "linked" : "draft" }; line 6462: if (kind === "analysis") return { title, description, type: "Math / Analysis", status: "draft" }; line 6463: if (kind === "simulation-file") return { title, description, url, type: "Simulation file", status: url ? "uploaded" : "draft" }; line 6464: if (kind === "simulation-result") return { title, description, url, type: "Simulation result", status: url ? "uploaded" : "draft" };. There are 1 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6460: function makeDesignItem(kind, title, description = "", url = "") {
6461:   if (kind === "reference") return { title, description, url, type: "Reference", status: url ? "linked" :
"draft" };
6462:   if (kind === "analysis") return { title, description, type: "Math / Analysis", status: "draft" };
6463:   if (kind === "simulation-file") return { title, description, url, type: "Simulation file", status: url
? "uploaded" : "draft" };
6464:   if (kind === "simulation-result") return { title, description, url, type: "Simulation result", status:
url ? "uploaded" : "draft" };
6465:   return { title, description, url, type: "Document", status: url ? "uploaded" : "draft" };
6466: }
```

renderDesignObjectSection (template-preview.js, lines 6468-6496)

renderDesignObjectSection renders ui, markup, or display output. In this file, it appears around lines 6468-6496 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references designArray, map, richHasContent, renderRichContent, renderMultilineInlineText, join, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6470: return `; line 6480: return `.

Important local values include items at line 6469; title at line 6477; url at line 6478; description at line 6479. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6468: function renderDesignObjectSection(project, pathValue, folder, label, kind = "document") {
6469:   const items = designArray(project, pathValue);
6470:   return `
6471:     <div class="section-actions">
6472:       <button class="button primary" type="button" data-open-editor="design-object" data-design-
path="${pathValue}" data-design-kind="${kind}" data-mode="add">Add ${label}</button>
6473:       ${folder ? `<button class="button secondary" type="button" data-upload-design="${pathValue}" data-
folder="${folder}" data-design-kind="${kind}">Add file</button>` : ""}
6474:     </div>
6475:     <div class="builder-items">
6476:       ${items.map((item, index) => {
6477:         const title = item.title || item.name || item.label || "Untitled item";
6478:         const url = item.url || item.artifact || "";
6479:         const description = item.description || item.caption || item.result || item.method || "";
6480:         return `
6481:           <article class="builder-item">
6482:             <div>
6483:               <strong>${title}</strong>
6484:               <span>${url || "No file attached"}</span>
6485:               ${richHasContent(item.richDescription)
6486:                 ? renderRichContent(item.richDescription, description)
6487:                 : description ? `<p>${renderMultilineInlineText(description)}</p>` : ""}
6488:             </div>
6489:             <button type="button" data-open-editor="design-object" data-design-path="${pathValue}" data-
design-kind="${kind}" data-mode="edit" data-index="${index}">Edit</button>
6490:             <button class="danger-icon" type="button" data-delete-design-item="${pathValue}" data-
index="${index}">Delete</button>
6491:           </article>`
```

```

6492:         `;
6493:     }).join("") || `
```

renderDesignSummaryField (template-preview.js, lines 6498-6530)

renderDesignSummaryField renders ui, markup, or display output. In this file, it appears around lines 6498-6530 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getByPath, summaryRichPathFromTextPath, escapeHtml, and overviewFolderFromPath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6504: return `.

Important local values include textValue at line 6499; richPath at line 6500; richValue at line 6501; hasValue at line 6502. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6498: function renderDesignSummaryField(project, pathValue, label, placeholder) {
6499:     const textValue = getByPath(project, pathValue) || "";
6500:     const richPath = summaryRichPathFromTextPath(pathValue);
6501:     const richValue = getByPath(project, richPath);
6502:     const hasValue = Boolean(textValue || richValue?.blocks?.length);
6503:
6504:     return `
6505:     <div class="wide-field design-summary-field rich-design-summary-field">
6506:         <div class="summary-builder-heading">
6507:             <span>
6508:                 ${label}
6509:                 ${hasValue ? `<button class="inline-danger-button" type="button" data-clear-design-
summary="${pathValue}">Clear overview</button>` : ""}
6510:             </span>
6511:         </div>
6512:
6513:         <div class="rich-summary-panel">
6514:             <p class="rich-editor-hint">Right-click to add images, code blocks, apply bold, color, font,
numeric size, alignment, or formulas.</p>
6515:
6516:             <div
6517:                 class="rich-summary-editor"
6518:                 data-rich-editor="section-overview"
6519:                 data-rich-text-path="${escapeHtml(pathValue)}"
6520:                 data-rich-path="${escapeHtml(richPath)}"
6521:                 data-rich-folder="${escapeHtml(overviewFolderFromPath(pathValue))}"
6522:                 data-placeholder="${escapeHtml(placeholder || "")}"
6523:                 contenteditable="true"
6524:                 spellcheck="true"
6525:                 aria-label="${escapeHtml(label)} rich editor"
6526:             ></div>
6527:         </div>
6528:     </div>
6529: `;
6530: }

```

designPathHasContent (template-preview.js, lines 6532-6536)

designPathHasContent part of the private builder frontend and editing experience. In this file, it appears around lines 6532-6536 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getByPath, isArray, some, itemTitle, richHasContent, and summaryRichPathFromTextPath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6534: if (Array.isArray(value)) return value.some((item) => itemTitle(item) || item?.description || item?.url || item?.artifact || richHasContent(item?.richDescription));; line 6535: return Boolean(value || richHasContent(getByPath(project, summaryRichPathFromTextPath(pathValue)))).

Important local values include value at line 6533. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6532: function designPathHasContent(project, pathValue) {
6533:     const value = getByPath(project, pathValue);

```

```

6534:   if (Array.isArray(value)) return value.some((item) => itemTitle(item) || item?.description || item?.url
|| item?.artifact || richHasContent(item?.richDescription));
6535:   return Boolean(value || richHasContent(getByPath(project, summaryRichPathFromTextPath(pathValue))));
6536: }

```

renderEmptyDynamicSectionPrompt (template-preview.js, lines 6538-6548)

renderEmptyDynamicSectionPrompt renders ui, markup, or display output. In this file, it appears around lines 6538-6548 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6539: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

6538: function renderEmptyDynamicSectionPrompt(label = "section") {
6539:   return `
6540:     <div class="compile-code-empty">
6541:       <h3>No ${escapeHtml(label)} content added yet</h3>
6542:       <p>Create your own sections and subsections with the Add section button. Empty predefined groups
stay hidden so the project structure is controlled by you.</p>
6543:       <div class="section-actions">
6544:         <button class="button primary" type="button" data-add-section="true">Add section</button>
6545:       </div>
6546:     </div>
6547:   `;
6548: }

```

renderAnalogDesignWorkspace (template-preview.js, lines 6550-6593)

renderAnalogDesignWorkspace renders ui, markup, or display output. In this file, it appears around lines 6550-6593 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureDesignModel, designPathHasContent, push, renderDesignSummaryField, renderDesignObjectSection, renderObjectSection, join, renderEmptyDynamicSectionPrompt, and renderPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6587: return `.

Important local values include cards at line 6552. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6550: function renderAnalogDesignWorkspace(project) {
6551:   ensureDesignModel(project);
6552:   const cards = [];
6553:   if (designPathHasContent(project, "design.brief.summary") || designPathHasContent(project,
"design.brief.files")) {
6554:     cards.push(`
6555:       <section class="design-workspace-card">
6556:         <h3>Design Overview</h3>
6557:         ${renderDesignSummaryField(project, "design.brief.summary", "Design overview", "State the analog
design objective, topology, constraints, and design decisions.")}
6558:         ${renderDesignObjectSection(project, "design.brief.files", "design-brief", "brief file",
"document")}
6559:       </section>
6560:     `);
6561:   }
6562:   if (
6563:     designPathHasContent(project, "design.documentation.summary") ||
6564:     designPathHasContent(project, "design.documentation.files") ||
6565:     designPathHasContent(project, "design.documentation.references") ||
6566:     designPathHasContent(project, "design.documentation.mathAnalysis")
6567:   ) {
6568:     cards.push(`
6569:       <section class="design-workspace-card">
6570:         <h3>Documentation</h3>
6571:         ${renderDesignSummaryField(project, "design.documentation.summary", "Documentation overview",
"Summarize schematics, notes, datasheets, calculations, and supporting documents.")}
6572:         ${renderDesignObjectSection(project, "design.documentation.files", "documentation", "document",
"document")}
6573:         ${designPathHasContent(project, "design.documentation.references") ?

```

```

`<h4>References</h4>${renderDesignObjectSection(project, "design.documentation.references", "", "reference",
"reference")}` : ""}
6574:     ${designPathHasContent(project, "design.documentation.mathAnalysis") ? `<h4>Math /
Analysis</h4>${renderDesignObjectSection(project, "design.documentation.mathAnalysis", "", "analysis item",
"analysis")}` : ""}
6575:   </section>
6576: `);
6577:   }
6578:   if ((project.pcbcs || []).length || (project.media || []).length) {
6579:     cards.push(`
6580:       <section class="design-workspace-card">
6581:         <h3>PCB and Visual Evidence</h3>
6582:         ${((project.pcbcs || []).length ? renderObjectSection(project, "pcbcs", "pcbcs", "PCBs") : "")}
6583:         ${((project.media || []).length ? renderObjectSection(project, "media", "images", "Images") : "")}
6584:       </section>
6585:     `);
6586:   }
6587:   return `
6588:     <div class="section-stack analog-design-workspace">
6589:       ${cards.join("")} || renderEmptyDynamicSectionPrompt("design")
6590:       ${renderPendingEditor()}
6591:     </div>
6592:   `;
6593: }

```

renderAnalogSimulationWorkspace (template-preview.js, lines 6595-6628)

renderAnalogSimulationWorkspace renders ui, markup, or display output. In this file, it appears around lines 6595-6628 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureDesignModel, designPathHasContent, push, renderDesignSummaryField, renderDesignObjectSection, join, renderEmptyDynamicSectionPrompt, and renderPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6622: return `.

Important local values include cards at line 6597. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6595: function renderAnalogSimulationWorkspace(project) {
6596:   ensureDesignModel(project);
6597:   const cards = [];
6598:   if (designPathHasContent(project, "design.simulation.summary")) {
6599:     cards.push(`
6600:       <section class="design-workspace-card">
6601:         <h3>Simulation Overview</h3>
6602:         ${renderDesignSummaryField(project, "design.simulation.summary", "Simulation overview", "Explain
what was simulated, why it mattered, and what the results proved.")}
6603:       </section>
6604:     `);
6605:   }
6606:   if (designPathHasContent(project, "design.simulation.files")) {
6607:     cards.push(`
6608:       <section class="design-workspace-card">
6609:         <h3>Simulation Files</h3>
6610:         ${renderDesignObjectSection(project, "design.simulation.files", "simulations", "simulation file",
"simulation-file")}
6611:       </section>
6612:     `);
6613:   }
6614:   if (designPathHasContent(project, "design.simulation.results")) {
6615:     cards.push(`
6616:       <section class="design-workspace-card">
6617:         <h3>Simulation Results</h3>
6618:         ${renderDesignObjectSection(project, "design.simulation.results", "simulation-results",
"simulation result", "simulation-result")}
6619:       </section>
6620:     `);
6621:   }
6622:   return `
6623:     <div class="section-stack analog-design-workspace">
6624:       ${cards.join("")} || renderEmptyDynamicSectionPrompt("simulation")
6625:       ${renderPendingEditor()}
6626:     </div>
6627:   `;
6628: }

```

pendingEditorUsesRichDescription (template-preview.js, lines 6630-6632)

pendingEditorUsesRichDescription part of the private builder frontend and editing experience. In this file, it appears around lines 6630-6632 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6631: return Boolean(editor && editor.type !== "text-array");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6630: function pendingEditorUsesRichDescription(editor = pendingEditor) {
6631:   return Boolean(editor && editor.type !== "text-array");
6632: }
```

pendingEditorFolder (template-preview.js, lines 6634-6641)

pendingEditorFolder part of the private builder frontend and editing experience. In this file, it appears around lines 6634-6641 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 6635: if (!editor) return "content";; line 6636: if (editor.type === "custom-section") return `custom-\${slugify(editor.sectionId || "section")}-overview`; line 6637: if (editor.type === "custom") return `custom-\${slugify(editor.sectionId || "section")}-subsection`; line 6638: if (editor.type === "design-object") return `design-\${slugify(editor.kind || "content")}`;. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
6634: function pendingEditorFolder(editor = pendingEditor) {
6635:   if (!editor) return "content";
6636:   if (editor.type === "custom-section") return `custom-${slugify(editor.sectionId ||
"section")}-overview`;
6637:   if (editor.type === "custom") return `custom-${slugify(editor.sectionId || "section")}-subsection`;
6638:   if (editor.type === "design-object") return `design-${slugify(editor.kind || "content")}`;
6639:   if (editor.type === "object") return slugify(editor.key || "content");
6640:   return slugify(editor.type || "content");
6641: }
```

renderPendingRichDescriptionEditor (template-preview.js, lines 6643-6666)

renderPendingRichDescriptionEditor renders ui, markup, or display output. In this file, it appears around lines 6643-6666 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, and pendingEditorFolder. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6649: return `.

Important local values include label at line 6644. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6643: function renderPendingRichDescriptionEditor(editor = pendingEditor) {
6644:   const label = editor?.type === "tool"
6645:     ? "What it was used for"
6646:     : editor?.type === "custom-section"
6647:       ? "Section content"
6648:       : "Content";
6649:   return `
6650:     <div class="pending-rich-description rich-design-summary-field">
6651:       <div class="summary-builder-heading"><span>${escapeHtml(label)}</span></div>
6652:       <div class="rich-summary-panel">
6653:         <p class="rich-editor-hint">Right-click to add images, code blocks, apply bold, color, font,
```



```

numeric size, alignment, or formulas.</p>
6654:     <div>
6655:         class="rich-summary-editor"
6656:         data-rich-editor="pending-description"
6657:         data-rich-folder="${escapeHtml(pendingEditorFolder(editor))}"
6658:         data-placeholder="Type or paste the complete content here."
6659:         contenteditable="true"
6660:         spellcheck="true"
6661:         aria-label="${escapeHtml(label)} rich editor"
6662:     ></div>
6663: </div>
6664: </div>
6665: `;
6666: }

```

renderPendingEditor (template-preview.js, lines 6668-6692)

renderPendingEditor renders ui, markup, or display output. In this file, it appears around lines 6668-6692 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderPendingRichDescriptionEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6669: if (!pendingEditor) return "";; line 6674: return `.

Important local values include title at line 6670; isTool at line 6671; isSimpleText at line 6672; isSectionDetails at line 6673. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6668: function renderPendingEditor() {
6669:   if (!pendingEditor) return "";
6670:   const title = pendingEditor.title || "";
6671:   const isTool = pendingEditor.type === "tool";
6672:   const isSimpleText = pendingEditor.type === "text-array";
6673:   const isSectionDetails = pendingEditor.type === "custom-section";
6674:   return `
6675:     <form class="pending-editor" id="pending-editor">
6676:       <div>
6677:         <h3>${pendingEditor.mode === "edit" ? "Edit" : "Add"} ${isTool ? "tool" : isSimpleText ? "item" :
isSectionDetails ? "section" : "subsection"}</h3>
6678:       </div>
6679:       <div class="${isTool ? "tool-editor-grid" : "pending-editor-grid"}">
6680:         <label>
6681:           <span>${isTool ? "Tool name" : "Title"}</span>
6682:           <input name="title" type="text" value="${title}" required>
6683:         </label>
6684:         ${isSimpleText ? "" : renderPendingRichDescriptionEditor(pendingEditor)}
6685:       </div>
6686:       <div class="pending-editor-actions">
6687:         <button class="button primary" type="submit">Save</button>
6688:         <button class="button secondary" type="button" data-cancel-pending="true">Cancel</button>
6689:       </div>
6690:     </form>
6691:   `;
6692: }

```

openPendingEditor (template-preview.js, lines 6694-6701)

openPendingEditor controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 6694-6701 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderSectionContent, selectedProject, requestAnimationFrame, populateRichEditors, querySelector, and scrollIntoView. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

6694: function openPendingEditor(config) {
6695:   pendingEditor = config;
6696:   renderSectionContent(selectedProject());
6697:   requestAnimationFrame(() => {
6698:     populateRichEditors(sectionContent);

```

```

6699:     document.querySelector("#pending-editor")?.scrollIntoView({ behavior: "smooth", block: "nearest" });
6700:   });
6701: }

```

savePendingEditor (template-preview.js, lines 6703-6785)

savePendingEditor persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 6703-6785 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, trim, querySelector, extractRichSummary, plainTextFromRich, setStatus, push, makeItemForSection, designArray, makeDesignItem, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 6705: if (!project || !pendingEditor) return;; line 6712: return;; line 6763: if (!section) return;; line 6775: if (!section) return;.

Important local values include project at line 6704; title at line 6706; richEditor at line 6707; richDescription at line 6708; description at line 6709; nextTool at line 6717; key at line 6726; existing at line 6729; plus 10 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6703: function savePendingEditor(form) {
6704:   const project = selectedProject();
6705:   if (!project || !pendingEditor) return;
6706:   const title = form.elements.title.value.trim();
6707:   const richEditor = form.querySelector("[data-rich-editor='pending-description']");
6708:   const richDescription = richEditor ? extractRichSummary(richEditor) : null;
6709:   const description = richEditor ? plainTextFromRich(richDescription) : "";
6710:   if (!title) {
6711:     setStatus("Type a title before saving.");
6712:     return;
6713:   }
6714:
6715:   if (pendingEditor.type === "tool") {
6716:     project.tools = project.tools || [];
6717:     const nextTool = { name: title, description, richDescription };
6718:     if (pendingEditor.mode === "edit") {
6719:       project.tools[Number(pendingEditor.index)] = nextTool;
6720:     } else {
6721:       project.tools.push(nextTool);
6722:     }
6723:   }
6724:
6725:   if (pendingEditor.type === "object") {
6726:     const key = pendingEditor.key;
6727:     project[key] = project[key] || [];
6728:     if (pendingEditor.mode === "edit") {
6729:       const existing = project[key][Number(pendingEditor.index)] || {};
6730:       const existingUrl = existing.url || existing.artifact || "";
6731:       const nextItem = makeItemForSection(key, title, description, existingUrl);
6732:       project[key][Number(pendingEditor.index)] = { ...existing, ...nextItem, richDescription };
6733:     } else {
6734:       const nextItem = makeItemForSection(key, title, description);
6735:       project[key].push({ ...nextItem, richDescription });
6736:     }
6737:   }
6738:
6739:   if (pendingEditor.type === "design-object") {
6740:     const items = designArray(project, pendingEditor.path);
6741:     const kind = pendingEditor.kind || "document";
6742:     if (pendingEditor.mode === "edit") {
6743:       const existing = items[Number(pendingEditor.index)] || {};
6744:       const existingUrl = existing.url || existing.artifact || "";
6745:       items[Number(pendingEditor.index)] = { ...existing, ...makeDesignItem(kind, title, description,
existingUrl), richDescription };
6746:     } else {
6747:       items.push({ ...makeDesignItem(kind, title, description), richDescription });
6748:     }
6749:   }
6750:
6751:   if (pendingEditor.type === "text-array") {
6752:     const key = pendingEditor.key;
6753:     project[key] = project[key] || [];
6754:     if (pendingEditor.mode === "edit") {
6755:       project[key][Number(pendingEditor.index)] = title;
6756:     } else {
6757:       project[key].push(title);
6758:     }
6759:   }
6760:
6761:   if (pendingEditor.type === "custom") {
6762:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);

```

```

6763:     if (!section) return;
6764:     section.items = section.items || [];
6765:     const nextItem = { title, description, richDescription, type: "Text", status: "draft" };
6766:     if (pendingEditor.mode === "edit") {
6767:       section.items[Number(pendingEditor.index)] = { ...section.items[Number(pendingEditor.index)],
...nextItem };
6768:     } else {
6769:       section.items.push(nextItem);
6770:     }
6771:   }
6772:
6773:   if (pendingEditor.type === "custom-section") {
6774:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
6775:     if (!section) return;
6776:     section.title = title;
6777:     section.description = description;
6778:     section.richDescription = richDescription;
6779:   }
6780:
6781:   pendingEditor = null;
6782:   setStatus("Saved in the project editor. Click Save project to include this version in the portfolio
preview.");
6783:   scheduleAutosave();
6784:   renderAll();
6785: }

```

section (template-preview.js, lines 6762-6765)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6762-6765 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6763: if (!section) return;.

Important local values include section at line 6762; nextItem at line 6765. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6762:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
6763:     if (!section) return;
6764:     section.items = section.items || [];
6765:     const nextItem = { title, description, richDescription, type: "Text", status: "draft" };

```

section (template-preview.js, lines 6774-6779)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6774-6779 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6775: if (!section) return;.

Important local values include section at line 6774. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6774:     const section = (project.sections || []).find((item) => item.id === pendingEditor.sectionId);
6775:     if (!section) return;
6776:     section.title = title;
6777:     section.description = description;
6778:     section.richDescription = richDescription;
6779:   }

```

openEditorFromButton (template-preview.js, lines 6787-6879)

openEditorFromButton controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 6787-6879 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, openPendingEditor, toolName, toolDescription, designArray, and find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 6789: if (!project) return;; line 6804: return;; line 6819: return;; line 6836: return;. There are 2 additional return statements in the function body.

Important local values include project at line 6788; type at line 6790; mode at line 6791; index at line 6792; item at line 6795; key at line 6808; item at line 6809; pathValue at line 6823; plus 7 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6787: function openEditorFromButton(button) {
6788:   const project = selectedProject();
6789:   if (!project) return;
6790:   const type = button.dataset.openEditor;
6791:   const mode = button.dataset.mode || "add";
6792:   const index = button.dataset.index;
6793:
6794:   if (type === "tool") {
6795:     const item = mode === "edit" ? project.tools?.[Number(index)] : {};
6796:     openPendingEditor({
6797:       type,
6798:       mode,
6799:       index,
6800:       title: toolName(item),
6801:       description: toolDescription(item),
6802:       richDescription: item?.richDescription || null
6803:     });
6804:     return;
6805:   }
6806:
6807:   if (type === "object") {
6808:     const key = button.dataset.key;
6809:     const item = mode === "edit" ? project[key]?.[Number(index)] : {};
6810:     openPendingEditor({
6811:       type,
6812:       mode,
6813:       key,
6814:       index,
6815:       title: item?.title || item?.name || item?.label || "",
6816:       description: item?.description || item?.caption || item?.result || item?.method || "",
6817:       richDescription: item?.richDescription || null
6818:     });
6819:     return;
6820:   }
6821:
6822:   if (type === "design-object") {
6823:     const pathValue = button.dataset.designPath;
6824:     const items = designArray(project, pathValue);
6825:     const item = mode === "edit" ? items?.[Number(index)] : {};
6826:     openPendingEditor({
6827:       type,
6828:       mode,
6829:       path: pathValue,
6830:       kind: button.dataset.designKind || "document",
6831:       index,
6832:       title: item?.title || item?.name || item?.label || "",
6833:       description: item?.description || item?.caption || item?.result || item?.method || "",
6834:       richDescription: item?.richDescription || null
6835:     });
6836:     return;
6837:   }
6838:
6839:
6840:   if (type === "custom") {
6841:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6842:     const item = mode === "edit" ? section?.items?.[Number(index)] : {};
6843:     openPendingEditor({
6844:       type,
6845:       mode,
6846:       sectionId: button.dataset.sectionId,
6847:       index,
6848:       title: item?.title || "",
6849:       description: item?.description || "",
6850:       richDescription: item?.richDescription || null
6851:     });
6852:     return;
6853:   }
6854:
6855:   if (type === "text-array") {
6856:     const key = button.dataset.key;
6857:     const item = mode === "edit" ? project[key]?.[Number(index)] : "";
6858:     openPendingEditor({
6859:       type,
```

```

6860:         mode,
6861:         key,
6862:         index,
6863:         title: typeof item === "string" ? item : item?.title || item?.name || item?.label || ""
6864:     });
6865:     return;
6866: }
6867:
6868: if (type === "custom-section") {
6869:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6870:     openPendingEditor({
6871:         type,
6872:         mode: "edit",
6873:         sectionId: button.dataset.sectionId,
6874:         title: section?.title || "",
6875:         description: section?.description || "",
6876:         richDescription: section?.richDescription || null
6877:     });
6878: }
6879: }

```

section (template-preview.js, lines 6841-6842)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6841-6842 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include section at line 6841; item at line 6842. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6841:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6842:     const item = mode === "edit" ? section?.items?.[Number(index)] : {};

```

section (template-preview.js, lines 6869-6877)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6869-6877 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and openPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include section at line 6869. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6869:     const section = (project.sections || []).find((item) => item.id === button.dataset.sectionId);
6870:     openPendingEditor({
6871:         type,
6872:         mode: "edit",
6873:         sectionId: button.dataset.sectionId,
6874:         title: section?.title || "",
6875:         description: section?.description || "",
6876:         richDescription: section?.richDescription || null
6877:     });

```

builderPreviewContent (template-preview.js, lines 6881-6888)

builderPreviewContent creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 6881-6888 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasContent, renderRichContent, renderMultilineInlineText, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6887: return `<div class="builder-item-preview">\${content}</div>`;.

Important local values include content at line 6882. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6881: function builderPreviewContent(rich, text = "", emptyText = "No content has been added yet.") {
6882:   const content = richHasContent(rich)
6883:   ? renderRichContent(rich, text || "")
6884:   : text
6885:   ? `
```

renderCustomSection (template-preview.js, lines 6890-6933)

renderCustomSection renders ui, markup, or display output. In this file, it appears around lines 6890-6933 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, builderPreviewContent, map, escapeHtml, richHasContent, join, and renderPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6892: if (!section) return `

Important local values include section at line 6891. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6890: function renderCustomSection(project, sectionId) {
6891:   const section = (project.sections || []).find((item) => item.id === sectionId);
6892:   if (!section) return `
```

```

6927:         <button class="danger-icon" type="button" data-delete-custom-item="${section.id}" data-
index="${index}">Delete</button>
6928:     </article>
6929:     `).join("") || `<p class="evidence-empty">No subsections added yet.</p>`
6930: </div>
6931:     ${renderPendingEditor()}
6932: `;
6933: }

```

section (template-preview.js, lines 6891-6892)

section part of the private builder frontend and editing experience. In this file, it appears around lines 6891-6892 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6892: if (!section) return `<p class="evidence-empty">Section not found.</p>`;

Important local values include section at line 6891. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6891: const section = (project.sections || []).find((item) => item.id === sectionId);
6892: if (!section) return `<p class="evidence-empty">Section not found.</p>`;

```

compileLanguageOptions (template-preview.js, lines 6935-6940)

compileLanguageOptions part of the compile code language/tool/build/run flow. In this file, it appears around lines 6935-6940 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodeLanguage, map, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6937: return supportedCodeLanguages.map((language) => `

Important local values include normalized at line 6936. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6935: function compileLanguageOptions(selected = "javascript") {
6936:   const normalized = normalizeCodeLanguage(selected);
6937:   return supportedCodeLanguages.map((language) => `
6938:     <option value="${escapeHtml(language.id)}"${language.id === normalized ? " selected" :
""}>${escapeHtml(language.label)}</option>
6939:   `).join("");
6940: }

```

compileRoleOptions (template-preview.js, lines 6942-6947)

compileRoleOptions part of the compile code language/tool/build/run flow. In this file, it appears around lines 6942-6947 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCompileFileRole, map, escapeHtml, compileFileRoleLabel, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6944: return ["design", "testbench"].map((role) => `

Important local values include normalized at line 6943. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6942: function compileRoleOptions(selected = "design") {
6943:   const normalized = normalizeCompileFileRole(selected, "verilog");
6944:   return ["design", "testbench"].map((role) => `

```

```

6945:     <option value="${escapeHtml(role)}"${role === normalized ? " selected" :
""}>${escapeHtml(compileFileRoleLabel(role))}</option>
6946:   `).join("");
6947: }

```

hdlTestbenchTemplate (template-preview.js, lines 6949-6973)

hdlTestbenchTemplate part of the private builder frontend and editing experience. In this file, it appears around lines 6949-6973 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugify, replace, dut, clk, reset_n, done, dumpfile, dumpvars, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6953: return [.

Important local values include tbName at line 6950; dutName at line 6951; commentPrefix at line 6952. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6949: function hdlTestbenchTemplate(language = "systemverilog", designName = "design") {
6950:   const tbName = `tb_${slugify(designName || "design").replace(/-/g, "_") || "design"}`;
6951:   const dutName = slugify(designName || "design").replace(/-/g, "_") || "design";
6952:   const commentPrefix = language === "verilog" ? "//" : "/*";
6953:   return [
6954:     `timescale 1ns/1ps`,
6955:     ``,
6956:     `module ${tbName};`,
6957:     `  ${commentPrefix} Declare testbench signals here.`,
6958:     `  ${commentPrefix} Example: reg clk = 0; reg reset_n = 0; wire done;`,
6959:     ``,
6960:     `  ${commentPrefix} Instantiate the design under test and connect the signals.`,
6961:     `  ${commentPrefix} Example: ${dutName} dut (.clk(clk), .reset_n(reset_n), .done(done));`,
6962:     ``,
6963:     `  initial begin`,
6964:     `    $dumpfile("waveform.vcd");`,
6965:     `    $dumpvars(0, ${tbName});`,
6966:     ``,
6967:     `    ${commentPrefix} Drive stimulus over time here.`,
6968:     `    #100;`,
6969:     `    $finish;`,
6970:     `  end`,
6971:     `endmodule`
6972:   ].join("\n");
6973: }

```

firstHdlModuleName (template-preview.js, lines 6975-6978)

firstHdlModuleName part of the private builder frontend and editing experience. In this file, it appears around lines 6975-6978 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 6977: return match?.[1] || "design";.

Important local values include match at line 6976. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

6975: function firstHdlModuleName(code = "") {
6976:   const match = String(code || "").match(/\\bmodule\\s+([A-Za-z_][\\w$]*)/);
6977:   return match?.[1] || "design";
6978: }

```

compileAppendDestinations (template-preview.js, lines 6980-7026)

compileAppendDestinations part of the compile code language/tool/build/run flow. In this file, it appears around lines 6980-7026 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `isAnalogProject`, `ensureDesignModel`, `forEach`, `push`, `summaryRichPathFromTextPath`, and `walkItems`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 6981: `if (!project) return []`; line 7025: `return destinations`;

Important local values include `destinations` at line 6982; `walkItems` at line 7007; `itemPath` at line 7009; `itemTitle` at line 7010. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
6980: function compileAppendDestinations(project) {
6981:   if (!project) return [];
6982:   const destinations = [{
6983:     id: "project-overview",
6984:     label: "Project overview",
6985:     richPath: "summaryRich",
6986:     textPath: "summary"
6987:   }];
6988:
6989:   if (isAnalogProject(project)) {
6990:     ensureDesignModel(project);
6991:     [
6992:       ["design-overview", "Design overview", "design.brief.summary"],
6993:       ["documentation-overview", "Documentation overview", "design.documentation.summary"],
6994:       ["simulation-overview", "Simulation overview", "design.simulation.summary"]
6995:     ].forEach(([id, label, textPath]) => {
6996:       destinations.push({ id, label, richPath: summaryRichPathFromTextPath(textPath), textPath });
6997:     });
6998:   }
6999:
7000:   (project.sections || []).forEach((section, index) => {
7001:     destinations.push({
7002:       id: `custom-${section.id || index}`,
7003:       label: `${section.title || `Custom section ${index + 1}`} overview`,
7004:       richPath: `sections.${index}.richDescription`,
7005:       textPath: `sections.${index}.description`
7006:     });
7007:     const walkItems = (items = [], prefix = `sections.${index}.items`, labelPrefix = section.title || `Custom section ${index + 1}`) => {
7008:       (items || []).forEach((item, itemIndex) => {
7009:         const itemPath = `${prefix}.${itemIndex}`;
7010:         const itemTitle = item.title || `Subsection ${itemIndex + 1}`;
7011:         if (!item.url) {
7012:           destinations.push({
7013:             id: `custom-${section.id || index}-${itemPath}`,
7014:             label: `${labelPrefix} / ${itemTitle} overview`,
7015:             richPath: `${itemPath}.richDescription`,
7016:             textPath: `${itemPath}.description`
7017:           });
7018:         }
7019:         walkItems(item.children || item.items || [], `${itemPath}.children`, `${labelPrefix} / ${itemTitle}`);
7020:       });
7021:     };
7022:     walkItems(section.items || []);
7023:   });
7024:
7025:   return destinations;
7026: }
```

compileAppendDestinationOptions (template-preview.js, lines 7028-7036)

`compileAppendDestinationOptions` part of the compile code language/tool/build/run flow. In this file, it appears around lines 7028-7036 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `compileAppendDestinations`, `some`, `map`, `escapeHtml`, and `join`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7033: `return destinations.map((destination) => ``.

Important local values include `destinations` at line 7029; `selected` at line 7030. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7028: function compileAppendDestinationOptions(project, selectedId = "") {
7029:   const destinations = compileAppendDestinations(project);
7030:   const selected = destinations.some((destination) => destination.id === selectedId)
7031:     ? selectedId
7032:     : destinations[0]?.id || "";
7033:   return destinations.map((destination) => `
```

```

7034:     <option value="`${escapeHtml(destination.id)}${destination.id === selected ? " selected" :
""}>${escapeHtml(destination.label)}</option>
7035:   `).join("");
7036: }

```

activeCompileFile (template-preview.js, lines 7038-7041)

activeCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 7038-7041 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, and find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7040: return workspace.files.find((file) => file.id === workspace.activeFileId) || workspace.files[0] || null;.

Important local values include workspace at line 7039. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7038: function activeCompileFile(project) {
7039:   const workspace = ensureCompileCode(project);
7040:   return workspace.files.find((file) => file.id === workspace.activeFileId) || workspace.files[0] ||
null;
7041: }

```

compileLogTimestamp (template-preview.js, lines 7043-7047)

compileLogTimestamp part of the compile code language/tool/build/run flow. In this file, it appears around lines 7043-7047 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toISOString, isNaN, getTime, and toLocaleTimeString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7045: if (Number.isNaN(date.getTime())) return "";; line 7046: return date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit", second: "2-digit" });.

Important local values include date at line 7044. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7043: function compileLogTimestamp(value = new Date().toISOString()) {
7044:   const date = new Date(value);
7045:   if (Number.isNaN(date.getTime())) return "";
7046:   return date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit", second: "2-digit" });
7047: }

```

addCompileMessage (template-preview.js, lines 7049-7062)

addCompileMessage part of the compile code language/tool/build/run flow. In this file, it appears around lines 7049-7062 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, slugify, now, random, toString, slice, toISOString, includes, and updateCompileMessagesPanel. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7051: if (!workspace || !text) return;.

Important local values include workspace at line 7050. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7049: function addCompileMessage(project, text, level = "info") {
7050:   const workspace = ensureCompileCode(project);
7051:   if (!workspace || !text) return;
7052:   workspace.messages = [

```

```

7053:     {
7054:       id: slugify(`message-${Date.now()}-${Math.random().toString(16).slice(2)}`),
7055:       at: new Date().toISOString(),
7056:       level: ["info", "success", "warning", "error"].includes(level) ? level : "info",
7057:       text: String(text)
7058:     },
7059:     ...(workspace.messages || [])
7060:   ].slice(0, 80);
7061:   updateCompileMessagesPanel(project);
7062: }

```

renderCompileMessages (template-preview.js, lines 7064-7073)

renderCompileMessages renders ui, markup, or display output. In this file, it appears around lines 7064-7073 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, escapeHtml, compileLogTimestamp, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7066: if (!messages.length) return `<p class="compile-message-empty">No messages yet.</p>`; line 7067: return messages.map((message) => `

Important local values include messages at line 7065. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7064: function renderCompileMessages(workspace) {
7065:   const messages = Array.isArray(workspace?.messages) ? workspace.messages : [];
7066:   if (!messages.length) return `<p class="compile-message-empty">No messages yet.</p>`;
7067:   return messages.map((message) => `
7068:     <div class="compile-message compile-message-${escapeHtml(message.level || "info")}">
7069:       <time>${escapeHtml(compileLogTimestamp(message.at))}</time>
7070:       <span>${escapeHtml(message.text)}</span>
7071:     </div>
7072:   `).join("");
7073: }

```

updateCompileTerminalPanel (template-preview.js, lines 7075-7084)

updateCompileTerminalPanel updates ui, state, cache, or stored data. In this file, it appears around lines 7075-7084 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, ensureCompileCode, querySelector, and requestAnimationFrame. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7079: if (!terminalElement) return;.

Important local values include workspace at line 7076; terminal at line 7077; terminalElement at line 7078. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7075: function updateCompileTerminalPanel(project, file = activeCompileFile(project)) {
7076:   const workspace = ensureCompileCode(project);
7077:   const terminal = file?.lastResult?.terminal || workspace?.terminal || compileTerminalStatus ||
7078:   "Compiler output will appear here after you save or run a source file.";
7079:   const terminalElement = sectionContent.querySelector("[data-compile-terminal]");
7080:   if (!terminalElement) return;
7081:   terminalElement.textContent = terminal;
7082:   requestAnimationFrame(() => {
7083:     terminalElement.scrollTop = terminalElement.scrollHeight;
7084:   });
7085: }

```

updateCompileMessagesPanel (template-preview.js, lines 7086-7091)

updateCompileMessagesPanel updates ui, state, cache, or stored data. In this file, it appears around lines 7086-7091 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `ensureCompileCode`, `querySelector`, and `renderCompileMessages`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7089: `if (!log) return;`

Important local values include `workspace` at line 7087; `log` at line 7088. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7086: function updateCompileMessagesPanel(project) {
7087:   const workspace = ensureCompileCode(project);
7088:   const log = sectionContent.querySelector("[data-compile-messages]");
7089:   if (!log) return;
7090:   log.innerHTML = renderCompileMessages(workspace);
7091: }
```

showCompileOutput (template-preview.js, lines 7093-7105)

`showCompileOutput` part of the compile code language/tool/build/run flow. In this file, it appears around lines 7093-7105 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `activeCompileFile`, `updateCompileTerminalPanel`, `querySelector`, `add`, `scrollIntoView`, `requestAnimationFrame`, and `remove`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7097: `if (!panel || !terminal) return;`

Important local values include `panel` at line 7095; `terminal` at line 7096. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7093: function showCompileOutput(project, file = activeCompileFile(project)) {
7094:   updateCompileTerminalPanel(project, file);
7095:   const panel = sectionContent.querySelector("[data-compile-terminal-panel]");
7096:   const terminal = sectionContent.querySelector("[data-compile-terminal]");
7097:   if (!panel || !terminal) return;
7098:   panel.classList.add("is-focused");
7099:   panel.scrollIntoView({ behavior: "smooth", block: "center" });
7100:   requestAnimationFrame(() => {
7101:     terminal.focus?({ preventScroll: true });
7102:     terminal.scrollTop = terminal.scrollHeight;
7103:   });
7104:   window.setTimeout(() => panel.classList.remove("is-focused"), 1400);
7105: }
```

activeCompileWaveform (template-preview.js, lines 7107-7110)

`activeCompileWaveform` part of the compile code language/tool/build/run flow. In this file, it appears around lines 7107-7110 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `activeCompileFile`, and `ensureCompileCode`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7109: `return file?.lastResult?.waveform || file?.waveform || workspace?.waveform || null;`

Important local values include `workspace` at line 7108. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7107: function activeCompileWaveform(project, file = activeCompileFile(project)) {
7108:   const workspace = ensureCompileCode(project);
7109:   return file?.lastResult?.waveform || file?.waveform || workspace?.waveform || null;
7110: }
```

scopeSignalValueAt (template-preview.js, lines 7112-7117)

`scopeSignalValueAt` part of the private builder frontend and editing experience. In this file, it appears around lines 7112-7117 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 7114: if (value === "1") return "1";; line 7115: if (value === "0") return "0";; line 7116: return "x";.

Important local values include value at line 7113. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7112: function scopeSignalValueAt(changes = [], index = 0) {
7113:   const value = String(changes[index]?.value ?? "x").toLowerCase();
7114:   if (value === "1") return "1";
7115:   if (value === "0") return "0";
7116:   return "x";
7117: }
```

renderScalarWavePath (template-preview.js, lines 7119-7140)

renderScalarWavePath renders ui, markup, or display output. In this file, it appears around lines 7119-7140 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, sort, max, min, forEach, xFor, yFor, scopeSignalValueAt, and toFixed. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7139: return path;.

Important local values include normalized at line 7120; top at line 7123; bottom at line 7124; middle at line 7125; yFor at line 7126; xFor at line 7127; path at line 7128; x at line 7130; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7119: function renderScalarWavePath(changes = [], maxTime = 1, rowY = 0, left = 150, width = 680) {
7120:   const normalized = (Array.isArray(changes) && changes.length ? changes : [{ time: 0, value: "x" }])
7121:     .map((change) => ({ time: Number(change.time) || 0, value: String(change.value ?? "x") }))
7122:     .sort((a, b) => a.time - b.time);
7123:   const top = rowY + 5;
7124:   const bottom = rowY + 23;
7125:   const middle = rowY + 14;
7126:   const yFor = (value) => value === "1" ? top : value === "0" ? bottom : middle;
7127:   const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) || 0) / Math.max(1, maxTime))) *
width;
7128:   let path = "";
7129:   normalized.forEach((change, index) => {
7130:     const x = xFor(change.time);
7131:     const y = yFor(scopeSignalValueAt(normalized, index));
7132:     if (index === 0) {
7133:       path += `M ${x.toFixed(1)} ${y.toFixed(1)}`;
7134:     } else {
7135:       path += `H ${x.toFixed(1)} V ${y.toFixed(1)}`;
7136:     }
7137:   });
7138:   path += `H ${xFor((left + width).toFixed(1))}`;
7139:   return path;
7140: }
```

yFor (template-preview.js, lines 7126-7137)

yFor part of the private builder frontend and editing experience. In this file, it appears around lines 7126-7137 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max, min, forEach, xFor, scopeSignalValueAt, and toFixed. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include yFor at line 7126; xFor at line 7127; path at line 7128; x at line 7130; y at line 7131. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7126:   const yFor = (value) => value === "1" ? top : value === "0" ? bottom : middle;
7127:   const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) || 0) / Math.max(1, maxTime))) *
width;
7128:   let path = "";
7129:   normalized.forEach((change, index) => {
7130:     const x = xFor(change.time);
7131:     const y = yFor(scopeSignalValueAt(normalized, index));
7132:     if (index === 0) {
7133:       path += `M ${x.toFixed(1)} ${y.toFixed(1)}`;
7134:     } else {
7135:       path += `H ${x.toFixed(1)} V ${y.toFixed(1)}`;
7136:     }
7137:   });
```

xFor (template-preview.js, lines 7127-7137)

xFor part of the private builder frontend and editing experience. In this file, it appears around lines 7127-7137 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max, min, forEach, yFor, scopeSignalValueAt, and toFixed. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include xFor at line 7127; path at line 7128; x at line 7130; y at line 7131. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7127:   const xFor = (time) => left + Math.max(0, Math.min(1, (Number(time) || 0) / Math.max(1, maxTime))) *
width;
7128:   let path = "";
7129:   normalized.forEach((change, index) => {
7130:     const x = xFor(change.time);
7131:     const y = yFor(scopeSignalValueAt(normalized, index));
7132:     if (index === 0) {
7133:       path += `M ${x.toFixed(1)} ${y.toFixed(1)}`;
7134:     } else {
7135:       path += `H ${x.toFixed(1)} V ${y.toFixed(1)}`;
7136:     }
7137:   });
```

renderBusWaveLabels (template-preview.js, lines 7142-7152)

renderBusWaveLabels renders ui, markup, or display output. In this file, it appears around lines 7142-7152 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, map, sort, slice, max, min, toFixed, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7147: return normalized.map((change, index) => {; line 7150: return `<text x="\${Math.min(left + width - 70, x + 4).toFixed(1)}" y="\${rowY + 19}" class="scope-value-label">\${escapeHtml(value)}</text>\${index ? `<line x1="\${x.toFixed(1)}" x2="\${x.toFixed(1)}" y1="\${rowY + 5}" y2="\${r`

Important local values include normalized at line 7143; x at line 7148; value at line 7149. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7142: function renderBusWaveLabels(changes = [], maxTime = 1, rowY = 0, left = 150, width = 680) {
7143:   const normalized = (Array.isArray(changes) && changes.length ? changes : [{ time: 0, value: "x" }])
7144:     .map((change) => ({ time: Number(change.time) || 0, value: String(change.value ?? "x") }))
7145:     .sort((a, b) => a.time - b.time)
7146:     .slice(0, 12);
7147:   return normalized.map((change, index) => {
7148:     const x = left + Math.max(0, Math.min(1, change.time / Math.max(1, maxTime))) * width;
7149:     const value = change.value.length > 12 ? `${change.value.slice(0, 12)}...` : change.value;
7150:     return `<text x="${Math.min(left + width - 70, x + 4).toFixed(1)}" y="${rowY + 19}" class="scope-
value-label">${escapeHtml(value)}</text>${index ? `<line x1="${x.toFixed(1)}" x2="${x.toFixed(1)}" y1="${rowY +
5}" y2="${rowY + 25}" class="scope-transition" />` : ""}`;
7151:   }).join("");
7152: }
```

renderHdlScope (template-preview.js, lines 7154-7203)

renderHdlScope renders ui, markup, or display output. In this file, it appears around lines 7154-7203 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, slice, max, flatMap, map, round, toFixed, join, escapeHtml, renderScalarWavePath, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 7157: return `; line 7173: return `<line x1="\${x.toFixed(1)}" x2="\${x.toFixed(1)}" y1="18" y2="\${height - 18}" class="scope-grid" /><text x="\${x.toFixed(1)}" y="14" class="scope-time-label">\${label}</text>`; line 7180: return `; line 7190: return `.

Important local values include signals at line 7155; left at line 7164; width at line 7165; rowHeight at line 7166; top at line 7167; height at line 7168; maxTime at line 7169; grid at line 7170; plus 7 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7154: function renderHdlScope(waveform) {
7155:   const signals = Array.isArray(waveform?.signals) ? waveform.signals.slice(0, 16) : [];
7156:   if (!signals.length) {
7157:     return `
7158:       <div class="compile-scope-empty">
7159:         <strong>No waveform scope yet.</strong>
7160:         <span>Run a Verilog/SystemVerilog design with a testbench that calls <code>$dumpfile</code> and
<code>$dumpvars</code>.</span>
7161:       </div>
7162:     `;
7163:   }
7164:   const left = 165;
7165:   const width = 760;
7166:   const rowHeight = 34;
7167:   const top = 28;
7168:   const height = top + signals.length * rowHeight + 28;
7169:   const maxTime = Math.max(1, Number(waveform.maxTime) || Math.max(...signals.flatMap((signal) =>
(signal.changes || []).map((change) => Number(change.time) || 0)), 1));
7170:   const grid = [0, 0.25, 0.5, 0.75, 1].map((ratio) => {
7171:     const x = left + ratio * width;
7172:     const label = Math.round(maxTime * ratio);
7173:     return `<line x1="${x.toFixed(1)}" x2="${x.toFixed(1)}" y1="18" y2="${height - 18}" class="scope-
grid" /><text x="${x.toFixed(1)}" y="14" class="scope-time-label">${label}</text>`;
7174:   }).join("");
7175:   const rows = signals.map((signal, index) => {
7176:     const y = top + index * rowHeight;
7177:     const name = String(signal.name || signal.reference || `signal_${index + 1}`);
7178:     const changes = Array.isArray(signal.changes) ? signal.changes : [];
7179:     const scalar = Number(signal.width || 1) <= 1;
7180:     return `
7181:       <g class="scope-row">
7182:         <text x="12" y="${y + 20}" class="scope-signal-label">${escapeHtml(name.length > 24 ?
`${name.slice(0, 24)}...` : name)}</text>
7183:         <line x1="${left}" x2="${left + width}" y1="${y + 27}" y2="${y + 27}" class="scope-row-line" />
7184:         ${scalar
7185:           ? `<path d="${renderScalarWavePath(changes, maxTime, y, left, width)}" class="scope-wave" />`
7186:           : `<line x1="${left}" x2="${left + width}" y1="${y + 15}" y2="${y + 15}" class="scope-bus"
/><${renderBusWaveLabels(changes, maxTime, y, left, width)}>`
7187:         }
7188:       `;
7189:   }).join("");
7190:   return
7191:     <div class="compile-scope-meta">
7192:       <span>${escapeHtml(waveform.source || "waveform.vcd")}</span>
7193:       <span>${signals.length} signal${signals.length === 1 ? "" : "s"}</span>
7194:       <span>0 to ${escapeHtml(maxTime)} ${escapeHtml(waveform.timeScale || "ticks")}</span>
7195:     </div>
7196:     <div class="compile-scope-scroll">
7197:       <svg class="compile-scope-svg" viewBox="0 0 ${left + width + 20} ${height}" role="img" aria-
label="HDL waveform scope">
7198:         ${grid}
7199:         ${rows}
7200:       </svg>
7201:     </div>
7202:   `;
7203: }
```

renderCompileScopePanel (template-preview.js, lines 7205-7217)

renderCompileScopePanel renders ui, markup, or display output. In this file, it appears around lines 7205-7217 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, activeCompileWaveform, isHdlLanguage, and renderHdlScope. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7207: if (!isHdlLanguage(file?.language) && !waveform) return "";; line 7208: return `.

Important local values include waveform at line 7206. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7205: function renderCompileScopePanel(project, file = activeCompileFile(project)) {
7206:   const waveform = activeCompileWaveform(project, file);
7207:   if (!isHdlLanguage(file?.language) && !waveform) return "";
7208:   return `
7209:     <section class="compile-scope-panel" data-compile-scope-panel aria-label="HDL signal scope">
7210:       <div class="compile-terminal-heading">
7211:         <span class="compile-terminal-title"><span aria-hidden="true" class="scope-dot"></span> Signal
scope</span>
7212:         <button type="button" data-compile-show-scope>Show scope</button>
7213:       </div>
7214:       ${renderHdlScope(waveform)}
7215:     </section>
7216:   `;
7217: }
```

showCompileScope (template-preview.js, lines 7219-7228)

showCompileScope part of the compile code language/tool/build/run flow. In this file, it appears around lines 7219-7228 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, querySelector, addCompileMessage, add, scrollIntoView, and remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7223: return;.

Important local values include panel at line 7220. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7219: function showCompileScope(project, file = activeCompileFile(project)) {
7220:   const panel = sectionContent.querySelector("[data-compile-scope-panel]");
7221:   if (!panel) {
7222:     addCompileMessage(project, "No HDL scope is available for the active source yet.", "warning");
7223:     return;
7224:   }
7225:   panel.classList.add("is-focused");
7226:   panel.scrollIntoView({ behavior: "smooth", block: "center" });
7227:   window.setTimeout(() => panel.classList.remove("is-focused"), 1400);
7228: }
```

copyCompileOutput (template-preview.js, lines 7230-7245)

copyCompileOutput part of the compile code language/tool/build/run flow. In this file, it appears around lines 7230-7245 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, ensureCompileCode, trim, addCompileMessage, writeText, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7235: return;.

Important local values include workspace at line 7231; text at line 7232. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7230: async function copyCompileOutput(project, file = activeCompileFile(project)) {
7231:   const workspace = ensureCompileCode(project);
7232:   const text = file?.lastResult?.terminal || workspace?.terminal || compileTerminalStatus || "";
7233:   if (!text.trim()) {
```



```

7234:     addCompileMessage(project, "There is no terminal output to copy yet.", "warning");
7235:     return;
7236: }
7237: try {
7238:     await navigator.clipboard.writeText(text);
7239:     addCompileMessage(project, "Terminal output copied to the clipboard.", "success");
7240:     setStatus("Terminal output copied.");
7241: } catch {
7242:     addCompileMessage(project, "Terminal output could not be copied by this browser.", "warning");
7243:     setStatus("Terminal output could not be copied.");
7244: }
7245: }

```

clearCompileOutput (template-preview.js, lines 7247-7256)

clearCompileOutput part of the compile code language/tool/build/run flow. In this file, it appears around lines 7247-7256 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references activeCompileFile, ensureCompileCode, updateCompileTerminalPanel, addCompileMessage, setStatus, and scheduleAutosave. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include workspace at line 7248. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7247: function clearCompileOutput(project, file = activeCompileFile(project)) {
7248:     const workspace = ensureCompileCode(project);
7249:     if (file?.lastResult) file.lastResult.terminal = "";
7250:     if (workspace) workspace.terminal = "";
7251:     compileTerminalStatus = "";
7252:     updateCompileTerminalPanel(project, file);
7253:     addCompileMessage(project, "Terminal output cleared.", "info");
7254:     setStatus("Terminal output cleared.");
7255:     scheduleAutosave();
7256: }

```

richBlocksForCompileAppend (template-preview.js, lines 7258-7265)

richBlocksForCompileAppend part of the compile code language/tool/build/run flow. In this file, it appears around lines 7258-7265 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getByPath, richBlocksFromValue, filter, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 7261: return richBlocksFromValue(existingRich, existingText).filter((block) => {; line 7262: if (block.type !== "paragraph") return true;; line 7263: return Boolean(String(block.text || "").trim() || block.html);.

Important local values include existingRich at line 7259; existingText at line 7260. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7258: function richBlocksForCompileAppend(project, destination) {
7259:     const existingRich = getByPath(project, destination.richPath);
7260:     const existingText = getByPath(project, destination.textPath) || "";
7261:     return richBlocksFromValue(existingRich, existingText).filter((block) => {
7262:         if (block.type !== "paragraph") return true;
7263:         return Boolean(String(block.text || "").trim() || block.html);
7264:     });
7265: }

```

codeBlockForCompileFile (template-preview.js, lines 7267-7274)

codeBlockForCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 7267-7274 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizeCodeText`, `normalizeCodeLanguage`, and `detectCodeLanguage`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7268: `return {`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
7267: function codeBlockForCompileFile(file) {
7268:   return {
7269:     code: normalizeCodeText(file?.code || "", "source"),
7270:     language: normalizeCodeLanguage(file?.language || detectCodeLanguage(file?.code || "", file?.fileName
|| "")),
7271:     pasteMode: "source",
7272:     type: "code"
7273:   };
7274: }
```

compileFileDirtyLabel (template-preview.js, lines 7276-7283)

`compileFileDirtyLabel` part of the compile code language/tool/build/run flow. In this file, it appears around lines 7276-7283 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 7277: `if (!file) return ""`; line 7278: `if (file.dirty) return "Unsaved"`; line 7279: `if (file.lastResult?.ok === true) return "Compiled"`; line 7280: `if (file.lastResult?.ok === false) return "Needs fix"`; line 7281: `if (file.savedAt) return "Saved"`; line 7282: `return "Draft"`. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
7276: function compileFileDirtyLabel(file) {
7277:   if (!file) return "";
7278:   if (file.dirty) return "Unsaved";
7279:   if (file.lastResult?.ok === true) return "Compiled";
7280:   if (file.lastResult?.ok === false) return "Needs fix";
7281:   if (file.savedAt) return "Saved";
7282:   return "Draft";
7283: }
```

renderCompileCodeSection (template-preview.js, lines 7285-7408)

This function draws the Compile Code workspace inside a project. It builds the file explorer, editor, language controls, compile/build/run/simulate/scope buttons, terminal panel, message log, and append-code controls.

The builder needs an IDE-like surface inside each project. This renderer turns saved compile workspace state into the screen the user edits and runs.

It calls or references `ensureCompileCode`, `activeCompileFile`, `escapeHtml`, `map`, `match`, `replace`, `compileFileDirtyLabel`, `join`, `defaultCodeFileName`, `compileLanguageOptions`, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7289: `return ``.

Important local values include `workspace` at line 7286; `activeFile` at line 7287; `terminal` at line 7288. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7285: function renderCompileCodeSection(project) {
7286:   const workspace = ensureCompileCode(project);
7287:   const activeFile = activeCompileFile(project);
7288:   const terminal = activeFile?.lastResult?.terminal || workspace.terminal || compileTerminalStatus ||
"Compiler output will appear here after you save or run a source file.";
7289:   return `
7290:     <div class="compile-code-workspace">
7291:       <aside class="compile-code-sidebar" aria-label="Compile code files">
7292:         <div class="compile-code-sidebar-heading">
7293:           <h3>Compile Code</h3>
7294:           <small>${workspace.files.length} source file${workspace.files.length === 1 ? "" : "s"}</small>
7295:         </div>
7296:         <div class="compile-code-workspace-tree" aria-label="Project workspace tree">
7297:           <details open>
7298:             <summary>${escapeHtml(project.title || "Project workspace")}</summary>
```

```

7299:         <div class="compile-code-tree-files">
7300:             ${workspace.files.map((file) => `
7301:                 <button class="compile-code-tree-file${file.id} === workspace.activeFileId ? " is-active"
: ""}] type="button" data-compile-select="${escapeHtml(file.id)}">
7302:                     <span class="compile-file-extension">${escapeHtml((file.fileName.match(/\.[^.]?$/)[0]
|| "src").replace(".", ""))}</span>
7303:                     <span>${escapeHtml(file.fileName)}</span>
7304:                     <small>${escapeHtml(compileFileDirtyLabel(file))}</small>
7305:                 </button>
7306:             `).join("") || `<p>No source files yet.</p>`}
7307:         </div>
7308:     </details>
7309: </div>
7310: <div class="compile-code-actions">
7311:     <button type="button" data-compile-add>Add code file</button>
7312:     <button type="button" data-compile-add-testbench>Add testbench</button>
7313:     <button type="button" data-compile-import>Import file</button>
7314:     <button type="button" data-compile-tools>Check compilers</button>
7315: </div>
7316: </aside>
7317:
7318: <section class="compile-code-main" aria-label="Compile code editor">
7319:     ${activeFile ?
7320:         <div class="compile-code-meta-grid">
7321:             <label>
7322:                 <span>Title</span>
7323:                 <input type="text" data-compile-field="title" value="${escapeHtml(activeFile.title || "")}"
placeholder="Source title" />
7324:             </label>
7325:             <label>
7326:                 <span>File name</span>
7327:                 <input type="text" data-compile-field="fileName" value="${escapeHtml(activeFile.fileName ||
"" )}" placeholder="${escapeHtml(defaultCodeFileName(activeFile.language))}" />
7328:             </label>
7329:             <label>
7330:                 <span>Language</span>
7331:                 <select data-compile-
field="language">${compileLanguageOptions(activeFile.language)}</select>
7332:             </label>
7333:             <label>
7334:                 <span>${isHdlLanguage(activeFile.language) ? "HDL role" : "Source role"}</span>
7335:                 ${isHdlLanguage(activeFile.language)
? `<select data-compile-field="role">${compileRoleOptions(activeFile.role)}</select>`
: `<input type="text" value="Program source" disabled />`}
7336:             </label>
7337:             <label>
7338:                 <div class="compile-code-editor-grid compile-code-editor-grid-single">
7339:                     <section class="compile-code-preview-panel compile-code-active-panel" aria-label="Active
syntax highlighted code editor">
7340:                         <div class="compile-code-preview-heading">
7341:                             <span>${escapeHtml(codeLanguageLabel(activeFile.language))} active editor</span>
7342:                         </div>
7343:                         <div class="compile-code-active-editor">
7344:                             <pre class="compile-code-preview" aria-hidden="true"><code data-compile-
preview>${tokenizedCodeHtml(activeFile.code || "", activeFile.language)}</code></pre>
7345:                             <textarea data-compile-field="code" data-compile-active-editor spellcheck="false"
wrap="off" rows="18" placeholder="Type or paste code here">${escapeHtml(activeFile.code || "")}</textarea>
7346:                         </div>
7347:                     </section>
7348:                 </div>
7349:                 <label class="compile-stdin-field">
7350:                     <span>Program input, optional</span>
7351:                     <textarea data-compile-field="stdin" rows="4" placeholder="Input passed to stdin when the
code runs">${escapeHtml(activeFile.stdin || "")}</textarea>
7352:                 </label>
7353:                 <div class="compile-code-command-bar">
7354:                     <button type="button" data-compile-save>Save source</button>
7355:                     <button type="button" data-compile-beautify>Beautify</button>
7356:                     <button type="button" data-compile-compile>Compile</button>
7357:                     <button type="button" data-compile-build>Build project</button>
7358:                     ${isHdlLanguage(activeFile.language)
? `<button type="button" data-compile-simulate>Simulate</button>`
: `<button type="button" data-compile-run>Run</button>`}
7359:                     <button class="compile-output-button" type="button" data-compile-show-output title="Open the
terminal output for this source"><span class="compile-command-icon" aria-hidden="true">&gt;</span><span>Show
output</span></button>
7360:                     ${isHdlLanguage(activeFile.language) ? `<button class="compile-output-button" type="button"
data-compile-show-scope title="Open the HDL waveform scope"><span class="compile-command-icon" aria-
hidden="true"></span><span>Show scope</span></button>` : ""}
7361:                     <button type="button" data-compile-install>Install tools</button>
7362:                     <button class="danger-icon" type="button" data-compile-delete>Delete source</button>
7363:                 </div>
7364:                 <section class="compile-append-panel" aria-label="Append code to project section">
7365:                     <div>
7366:                         <strong>Append code to project</strong>
7367:                         <span>Insert the active source as a formatted code block into this project.</span>
7368:                     </div>
7369:                     <label>
7370:                         <span>Destination</span>
7371:                         <select data-compile-append-target>${compileAppendDestinationOptions(project,
workspace.appendTargetId)}</select>
7372:                     </label>

```

```

7377:         <div class="compile-append-actions">
7378:             <button type="button" data-compile-append-code>Append code to project</button>
7379:         </div>
7380:     </section>
7381:     `
7382:     <div class="compile-code-empty">
7383:         <h3>No compile file selected</h3>
7384:         <p>Add a source file for C, C++, SystemVerilog, LTspice, Java, JavaScript, Python, or
HTML.</p>
7385:     </div>
7386: `}
7387: <section class="compile-terminal-panel" data-compile-terminal-panel aria-label="Compiler terminal
output">
7388:     <div class="compile-terminal-heading">
7389:         <span class="compile-terminal-title"><span aria-hidden="true" class="terminal-dot"></span>
OMB Local Terminal</span>
7390:         <span class="compile-terminal-controls">
7391:             <button type="button" data-compile-copy-output>Copy output</button>
7392:             <button type="button" data-compile-clear-output>Clear output</button>
7393:         </span>
7394:     </div>
7395:     <pre class="compile-terminal" data-compile-terminal tabindex="0" role="log" aria-
live="polite">${escapeHtml(terminal)}</pre>
7396:     </section>
7397:     ${renderCompileScopePanel(project, activeFile)}
7398:     <section class="compile-messages-panel" aria-label="Compile messages log">
7399:         <div class="compile-terminal-heading">
7400:             <span>Messages log</span>
7401:             <button type="button" data-compile-clear-messages>Clear log</button>
7402:         </div>
7403:         <div class="compile-messages-log" data-compile-
messages>${renderCompileMessages(workspace)}</div>
7404:     </section>
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

renderSectionContent (template-preview.js, lines 7410-7472)

renderSectionContent renders ui, markup, or display output. In this file, it appears around lines 7410-7472 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionOptions, find, startsWith, renderCustomSection, replace, requestAnimationFrame, populateRichEditors, isAnalogProject, renderAnalogDesignWorkspace, renderObjectSection, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 7414: return;; line 7423: return;; line 7429: return;; line 7435: if (isAnalogProject(project)) return renderAnalogDesignWorkspace(project);. There are 1 additional return statements in the function body.

Important local values include section at line 7417; isOverview at line 7418; renderers at line 7432; cards at line 7436. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7410: function renderSectionContent(project) {
7411:     if (!project) {
7412:         sectionContent.hidden = true;
7413:         sectionContent.innerHTML = "";
7414:         return;
7415:     }
7416:
7417:     const section = sectionOptions(project).find((item) => item.id === activeSectionId) ||
standardSections[0];
7418:     const isOverview = section.id === "brief";
7419:     projectFields.hidden = !isOverview;
7420:     sectionContent.hidden = isOverview;
7421:     if (isOverview) {
7422:         sectionContent.innerHTML = "";
7423:         return;
7424:     }
7425:
7426:     if (section.id.startsWith("custom:")) {
7427:         sectionContent.innerHTML = renderCustomSection(project, section.id.replace("custom:", ""));
7428:         requestAnimationFrame(() => populateRichEditors(sectionContent));
7429:         return;
7430:     }
7431:
7432:     const renderers = {
7433:         brief: () => "",
7434:         design: () => {
7435:             if (isAnalogProject(project)) return renderAnalogDesignWorkspace(project);
7436:             const cards = [
7437:                 (project.documents || []).length ? `<section><h3>Documents</h3>${renderObjectSection(project,
"documents", "documents", "Documents")}</section>` : "",
7438:                 (project.pcbS || []).length ? `<section><h3>PCBs Built</h3>${renderObjectSection(project, "pcbS",
"pcbS", "PCBs")}</section>` : "",

```

```

7439:         (project.media || []).length ? `<section><h3>Images</h3>${renderObjectSection(project, "media",
"images", "Images")}</section>` : ""
7440:       ].filter(Boolean);
7441:       return `
7442:         <div class="section-stack">
7443:           ${cards.join("") || renderEmptyDynamicSectionPrompt("design")}
7444:           ${renderPendingEditor()}
7445:         </div>
7446:       `;
7447:     },
7448:     simulation: () => isAnalogProject(project) ? renderAnalogSimulationWorkspace(project) : "",
7449:     "compile-code": () => renderCompileCodeSection(project),
7450:     tests: () => `${renderObjectSection(project, "tests", "tests", "Tests")}${renderPendingEditor()}`,
7451:     tools: () => `
7452:       <div class="section-stack">
7453:         <section>
7454:           <h3>Tools Used</h3>
7455:           ${renderToolsSection(project)}
7456:         </section>
7457:         <section>
7458:           <h3>Languages</h3>
7459:           ${renderTextArray(project, "languages", "Language")}
7460:         </section>
7461:         <section>
7462:           <h3>Links</h3>
7463:           ${renderObjectSection(project, "links", "", "Links")}
7464:         </section>
7465:         ${renderPendingEditor()}
7466:       </div>
7467:     `;
7468:   };
7469:
7470:   sectionContent.innerHTML = renderers[section.id]();
7471:   requestAnimationFrame(() => populateRichEditors(sectionContent));
7472: }

```

fullPortfolioPreviewHtml (template-preview.js, lines 7474-7476)

fullPortfolioPreviewHtml part of the private builder frontend and editing experience. In this file, it appears around lines 7474-7476 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fullPortfolioPreviewHtmlExact. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7475: return fullPortfolioPreviewHtmlExact();.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

7474: function fullPortfolioPreviewHtml() {
7475:   return fullPortfolioPreviewHtmlExact();
7476: }

```

previewValue (template-preview.js, lines 7478-7480)

previewValue part of the private builder frontend and editing experience. In this file, it appears around lines 7478-7480 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7479: return source && Object.prototype.hasOwnProperty.call(source, key) ? source[key] : fallback;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

7478: function previewValue(source, key, fallback) {
7479:   return source && Object.prototype.hasOwnProperty.call(source, key) ? source[key] : fallback;
7480: }

```

fullPortfolioPreviewHtmlExact (template-preview.js, lines 7482-7702)

This function builds the local preview HTML that should match the public website layout. It combines profile data, fun facts, sections, projects, contacts, AI placement, and generated project markup into one preview frame.

The user needs to see exactly what will be pushed before publishing. This function is the truth test between builder state and website output.

It calls or references `parsedPortfolioGlobals`, `normalizeSiteContent`, `previewValue`, `normalizeProfile`, `normalizeFieldStyles`, `stringify`, `replaceAll`, `escapeHtml`, `plainFieldStyleAttribute`, `renderRichFieldContent`, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7506: `return `<!DOCTYPE html>`.`

Important local values include `baseHref` at line 7483; `parsedGlobals` at line 7484; `siteContent` at line 7485; `profile` at line 7486; `fieldStyles` at line 7487; `profileRich` at line 7488; `siteContentRich` at line 7489; `ownerName` at line 7490; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7482: function fullPortfolioPreviewHtmlExact(dataOverride = null) {
7483:   const baseHref = `${window.location.origin}/`;
7484:   const parsedGlobals = dataOverride ? null : parsedPortfolioGlobals();
7485:   const siteContent = normalizeSiteContent(previewValue(dataOverride, "siteContent",
parsedGlobals?.siteContent || {}));
7486:   const profile = normalizeProfile(previewValue(dataOverride, "profile", parsedGlobals?.profile || {}));
7487:   const fieldStyles = normalizeFieldStyles(previewValue(dataOverride, "fieldStyles",
parsedGlobals?.fieldStyles || {}));
7488:   const profileRich = previewValue(dataOverride, "profileRich", parsedGlobals?.profileRich || {});
7489:   const siteContentRich = previewValue(dataOverride, "siteContentRich", parsedGlobals?.siteContentRich ||
{});
7490:   const ownerName = profile.displayName || "Portfolio";
7491:   const portfolioLabel = profile.portfolioLabel || "Portfolio";
7492:   const previewDataObject = {
7493:     categories: previewValue(dataOverride, "categories", parsedGlobals?.categories || []),
7494:     fieldStyles,
7495:     funFacts: previewValue(dataOverride, "funFacts", parsedGlobals?.funFacts || []),
7496:     funFactsRich: previewValue(dataOverride, "funFactsRich", parsedGlobals?.funFactsRich || null),
7497:     profile,
7498:     profileRich,
7499:     projects: previewValue(dataOverride, "projects", savedPortfolioCatalog.projects || []),
7500:     siteContent,
7501:     siteContentRich,
7502:     siteSections: previewValue(dataOverride, "siteSections", parsedGlobals?.siteSections || [])
7503:   };
7504:   const previewData = JSON.stringify(previewDataObject).replaceAll("</", "<\\");
7505:
7506:   return `<!DOCTYPE html>
7507: <html lang="en">
7508:   <head>
7509:     <meta charset="UTF-8" />
7510:     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7511:     <base href=${baseHref} />
7512:     <meta name="description" content="A portfolio website built with OMB Portfolio Builder for projects,
documents, diagrams, source code, tests, links, and profile information." />
7513:     <meta name="robots" content="index, follow" />
7514:     <meta name="author" content=${escapeHtml(ownerName)} />
7515:     <meta name="portfolio-ai-endpoint" content="https://omb-portfolio-ai.maurice-baraza-
otieno.workers.dev/api/portfolio-ai" />
7516:     <title${escapeHtml(portfolioLabel)} | ${escapeHtml(portfolioLabel)}</title>
7517:     <link rel="stylesheet" href="styles.css" />
7518:   </head>
7519:   <body>
7520:     <header class="site-header" id="top">
7521:       <a class="brand" href="#top" aria-label="Go to top">
7522:         <span class="brand-lockup" aria-hidden="true">
7523:           <img class="brand-icon" alt="" hidden />
7524:           <span class="omb-engraving">${escapeHtml(profile.brandText || "Portfolio")}</span>
7525:         </span>
7526:         <span>
7527:           <strong${plainFieldStyleAttribute(fieldStyles, "profile-display-
name")}>${renderRichFieldContent(profileRich.displayName, ownerName)}</strong>
7528:           <small${plainFieldStyleAttribute(fieldStyles, "profile-portfolio-
label")}>${renderRichFieldContent(profileRich.portfolioLabel, portfolioLabel)}</small>
7529:         </span>
7530:       </a>
7531:
7532:       <div class="header-right">
7533:         <nav class="site-nav" aria-label="Primary navigation">
7534:           <a href="#ask-ai">Ask AI</a>
7535:           <a href="#projects">Projects</a>
7536:           <a href="#resume">Resume</a>
7537:           <a href="#process">Process</a>
7538:           <a href="#contact">Contact</a>
7539:         </nav>
7540:         <a class="header-avatar" href="#contact" aria-label="Go to contact information" hidden>
7541:           <img alt="" />
7542:         </a>
7543:       </div>
7544:     </header>
```

```

7545:
7546:     <main>
7547:       <section class="hero" aria-labelledby="hero-title">
7548:         <img class="hero-image" alt="" hidden />
7549:         <div class="hero-shade" aria-hidden="true"></div>
7550:         <div class="hero-content">
7551:           <p class="eyebrow">${plainFieldStyleAttribute(fieldStyles, "site-hero-
renderRichFieldContent(siteContentRich.heroEyebrow, siteContent.heroEyebrow)}</p>
7552:           <h1 id="hero-title">${plainFieldStyleAttribute(fieldStyles, "site-hero-
title")}>${renderRichFieldContent(siteContentRich.heroTitle, siteContent.heroTitle)}</h1>
7553:           <p class="hero-copy">${plainFieldStyleAttribute(fieldStyles, "site-hero-
copy")}>${renderRichFieldContent(siteContentRich.heroCopy, siteContent.heroCopy)}</p>
7554:           <div class="hero-actions">
7555:             <a class="button primary" href="#projects">View projects</a>
7556:             <a class="button secondary" href="#resume" hidden>View resume</a>
7557:             <a class="button secondary" href="#" target="_blank" rel="noreferrer" hidden>GitHub
profile</a>
7558:             <a class="ai-jump-link" href="#ask-ai" aria-label="Jump to the portfolio AI assistant">
7559:               <svg viewBox="0 0 24 24" aria-hidden="true">
7560:                 <path d="M12 3v3m-5 7H4m16 0h-3M7.5 7.5 5.4 5.4m11.1 2.1 2.1-2.1M8 10h8v7a2 2 0 0 1-2
2h-4a2 2 0 0 1-2-2z" />
7561:                 <path d="M10 13h.01M14 13h.01M10 16h4" />
7562:               </svg>
7563:               <span>Ask AI</span>
7564:             </a>
7565:           </div>
7566:         </div>
7567:       </section>
7568:
7569:       <section class="fun-facts-section" id="fun-facts-callout" aria-label="Fun facts" hidden></section>
7570:
7571:       <section class="builder-download-section" aria-label="Download portfolio builder">
7572:         <a class="builder-download-link" href="https://github.com/otienomaurice/omb-portfolio-
builder/releases/latest/download/OMB-Portfolio-Builder-Setup-latest-x64.exe" download>
7573:           <svg viewBox="0 0 24 24" aria-hidden="true">
7574:             <path d="M12 3v11m0 0 4-4m-4 4-4-4" />
7575:             <path d="M5 17v2h14v-2" />
7576:           </svg>
7577:           <span>Download builder application</span>
7578:         </a>
7579:       </section>
7580:
7581:       <section class="summary-band" aria-label="Portfolio highlights">
7582:         <div class="metric">
7583:           <strong id="project-count">0</strong>
7584:           <span>presented projects</span>
7585:         </div>
7586:         <div class="metric">
7587:           <strong id="project-track-count">0 Tracks</strong>
7588:           <span id="project-track-labels">project categories</span>
7589:         </div>
7590:         <div class="metric">
7591:           <strong>Artifacts</strong>
7592:           <span>source, diagrams, and reports</span>
7593:         </div>
7594:       </section>
7595:
7596:       <section class="section" id="projects" aria-labelledby="projects-title">
7597:         <div class="section-heading">
7598:           <div>
7599:             <p class="eyebrow">Selected work</p>
7600:             <h2 id="projects-title">Engineering Projects</h2>
7601:           </div>
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

renderPreview (template-preview.js, lines 7704-7706)

renderPreview renders ui, markup, or display output. In this file, it appears around lines 7704-7706 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fullPortfolioPreviewHtmlExact. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

7704: function renderPreview() {
7705:   portfolioPreviewFrame.srcdoc = fullPortfolioPreviewHtmlExact();
7706: }

```

openPortfolioPreview (template-preview.js, lines 7708-7713)

openPortfolioPreview controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 7708-7713 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveAllSections, renderPreview, add, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7709: if (!(await saveAllSections({ refreshOpenPreviews: false }))) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
7708: async function openPortfolioPreview() {
7709:   if (!(await saveAllSections({ refreshOpenPreviews: false }))) return;
7710:   renderPreview();
7711:   document.body.classList.add("full-window-open");
7712:   portfolioPreviewDialog.showModal();
7713: }
```

renderAll (template-preview.js, lines 7715-7732)

renderAll renders ui, markup, or display output. In this file, it appears around lines 7715-7732 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSiteSections, selectedProject, sectionOptions, some, renderFunFactsEditor, renderSiteContentEditor, renderProfileEditor, renderSiteSectionList, renderTree, renderFields, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include project at line 7718. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7715: function renderAll() {
7716:   ensureSiteSections();
7717:   if (!selectedProjectId && catalog.projects.length) selectedProjectId = catalog.projects[0].id;
7718:   const project = selectedProject();
7719:   if (project && !sectionOptions(project).some((section) => section.id === activeSectionId)) {
7720:     activeSectionId = "brief";
7721:   }
7722:   renderFunFactsEditor();
7723:   renderSiteContentEditor();
7724:   renderProfileEditor();
7725:   renderSiteSectionList();
7726:   renderTree();
7727:   renderFields(project);
7728:   renderSectionTabs(project);
7729:   renderSectionContent(project);
7730:   if (portfolioPreviewDialog?.open) renderPreview();
7731:   updateBuilderWorkflow();
7732: }
```

updateProjectField (template-preview.js, lines 7734-7768)

updateProjectField updates ui, state, cache, or stored data. In this file, it appears around lines 7734-7768 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, split, map, trim, filter, limitWords, slugify, includes, setStatus, scheduleAutosave, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7736: if (!project) return;; line 7751: if (!nextId) return;.

Important local values include project at line 7735; needsChromeRender at line 7738; statusMessage at line 7739; limited at line 7743; nextId at line 7750. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7734: function updateProjectField(field, value) {
7735:   const project = selectedProject();
7736:   if (!project) return;
7737:
7738:   let needsChromeRender = false;
7739:   let statusMessage = "Unsaved local changes.";
7740:   if (field === "focus") {
7741:     project.focus = value.split(",").map((item) => item.trim()).filter(Boolean);
7742:   } else if (field === "summary") {
7743:     const limited = limitWords(value, 1000);
7744:     project.summary = limited;
7745:     if (limited !== value && document.activeElement?.dataset?.field === "summary") {
7746:       document.activeElement.value = limited;
7747:       statusMessage = "Project overview is limited to 1000 words.";
7748:     }
7749:   } else if (field === "id") {
7750:     const nextId = slugify(value);
7751:     if (!nextId) return;
7752:     project.id = nextId;
7753:     selectedProjectId = nextId;
7754:     needsChromeRender = true;
7755:   } else {
7756:     project[field] = value;
7757:     needsChromeRender = ["title", "category", "status"].includes(field);
7758:   }
7759:
7760:   if (field === "title") projectWindowTitle.textContent = value || "Untitled project";
7761:   setStatus(statusMessage);
7762:   scheduleAutosave();
7763:   if (needsChromeRender) {
7764:     scheduleChromeRender();
7765:   } else {
7766:     schedulePreviewRender();
7767:   }
7768: }
```

renameSelectedProject (template-preview.js, lines 7770-7775)

renameSelectedProject part of the private builder frontend and editing experience. In this file, it appears around lines 7770-7775 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, and beginTitleRename. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7772: if (!project) return;.

Important local values include project at line 7771. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7770: function renameSelectedProject() {
7771:   const project = selectedProject();
7772:   if (!project) return;
7773:   projectTitleMenu.hidden = true;
7774:   beginTitleRename();
7775: }
```

projectMetaRows (template-preview.js, lines 7777-7813)

projectMetaRows part of the private builder frontend and editing experience. In this file, it appears around lines 7777-7813 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references categoryById, projectTemplateFor, find, flatMap, filter, and toLocaleString. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7778: if (!project) return [];; line 7799: return [.

Important local values include category at line 7779; template at line 7780; savedProject at line 7781; design at line 7782; designFileCount at line 7783; uploadedFileCount at line 7791. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7777: function projectMetaRows(project) {
7778:   if (!project) return [];
7779:   const category = categoryById(project.category);
7780:   const template = projectTemplateFor(project);
7781:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project.id);
7782:   const design = project.design || {};
7783:   const designFileCount = [
7784:     ...(design.brief?.files || []),
7785:     ...(design.documentation?.files || []),
7786:     ...(design.documentation?.references || []),
7787:     ...(design.documentation?.mathAnalysis || []),
7788:     ...(design.simulation?.files || []),
7789:     ...(design.simulation?.results || [])
7790:   ].length;
7791:   const uploadedFileCount = [
7792:     ...(project.documents || []),
7793:     ...(project.tests || []),
7794:     ...(project.pcbns || []),
7795:     ...(project.media || []),
7796:     ...(project.links || []),
7797:     ...(project.sections || []).flatMap((section) => section.items || [])
7798:   ].filter((item) => item.url || item.artifact).length + designFileCount;
7799:   return [
7800:     ["Title", project.title || "Untitled project"],
7801:     ["Project ID / folder", project.id || "Not assigned"],
7802:     ["Category", category.label || project.category || "Not assigned"],
7803:     ["Status", project.status || "Draft"],
7804:     ["Appearance", template.label || project.templateLabel || "No appearance - white project layout"],
7805:     ["Appearance ID", project.templateId || "None"],
7806:     ["Portfolio preview", savedProject ? "Saved into parsed portfolio preview" : "Not yet saved into
portfolio preview"],
7807:     ["Custom sections", String((project.sections || []).length)],
7808:     ["Project files / links", String(uploadedFileCount)],
7809:     ["Tools", String((project.tools || []).length)],
7810:     ["Languages", String((project.languages || []).length)],
7811:     ["Last parsed", savedProject?.portfolioView?.builtAt ? new
Date(savedProject.portfolioView.builtAt).toLocaleString() : "Not parsed yet"]
7812:   ];
7813: }
```

savedProject (template-preview.js, lines 7781-7782)

savedProject persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 7781-7782 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include savedProject at line 7781; design at line 7782. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7781:   const savedProject = (savedPortfolioCatalog.projects || []).find((item) => item.id === project.id);
7782:   const design = project.design || {};
```

openProjectMetaDetails (template-preview.js, lines 7815-7827)

openProjectMetaDetails controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 7815-7827 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, projectMetaRows, map, join, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7817: if (!project) return;.

Important local values include project at line 7816. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7815: function openProjectMetaDetails() {
7816:   const project = selectedProject();
```

```

7817:   if (!project) return;
7818:   projectTitleMenu.hidden = true;
7819:   projectMetaTitle.textContent = project.title || "Project details";
7820:   projectMetaGrid.innerHTML = projectMetaRows(project).map(([label, value]) => `
7821:     <article>
7822:       <span>${label}</span>
7823:       <strong>${value}</strong>
7824:     </article>
7825:   `).join("");
7826:   if (!projectMetaDialog.open) projectMetaDialog.showModal();
7827: }

```

selectTitleText (template-preview.js, lines 7829-7835)

selectTitleText part of the private builder frontend and editing experience. In this file, it appears around lines 7829-7835 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, createRange, selectNodeContents, removeAllRanges, and addRange. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include selection at line 7830; range at line 7831. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7829: function selectTitleText() {
7830:   const selection = window.getSelection();
7831:   const range = document.createRange();
7832:   range.selectNodeContents(projectWindowTitle);
7833:   selection.removeAllRanges();
7834:   selection.addRange(range);
7835: }

```

beginTitleRename (template-preview.js, lines 7837-7846)

beginTitleRename part of the private builder frontend and editing experience. In this file, it appears around lines 7837-7846 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, focus, selectTitleText, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7839: if (!project) return;.

Important local values include project at line 7838. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7837: function beginTitleRename() {
7838:   const project = selectedProject();
7839:   if (!project) return;
7840:   originalTitleDraft = project.title || "";
7841:   projectWindowTitle.contentEditable = "true";
7842:   projectWindowTitle.focus();
7843:   selectTitleText();
7844:   titleRenameActions.hidden = false;
7845:   setStatus("Edit the title, then click Save title.");
7846: }

```

endTitleRename (template-preview.js, lines 7848-7865)

endTitleRename part of the private builder frontend and editing experience. In this file, it appears around lines 7848-7865 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, trim, getSelection, removeAllRanges, setStatus, scheduleAutosave, and scheduleChromeRender. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7850: if (!project) return;.

Important local values include project at line 7849; nextTitle at line 7851. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7848: function endTitleRename(saveChanges) {
7849:   const project = selectedProject();
7850:   if (!project) return;
7851:   const nextTitle = projectWindowTitle.textContent.trim();
7852:   projectWindowTitle.contentEditable = "false";
7853:   titleRenameActions.hidden = true;
7854:   window.getSelection()?.removeAllRanges();
7855:   if (saveChanges && nextTitle) {
7856:     project.title = nextTitle;
7857:     projectWindowTitle.textContent = nextTitle;
7858:     setStatus("Project title saved in the editor. Click Save project to include this version in the
portfolio preview.");
7859:     scheduleAutosave();
7860:     scheduleChromeRender();
7861:   } else {
7862:     projectWindowTitle.textContent = originalTitleDraft || project.title || "Untitled project";
7863:     setStatus("Title edit canceled.");
7864:   }
7865: }
```

makeItemForSection (template-preview.js, lines 7867-7874)

makeItemForSection part of the private builder frontend and editing experience. In this file, it appears around lines 7867-7874 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 7868: if (key === "documents") return { title, type: "Document", url, status: url ? "uploaded" : "draft" };; line 7869: if (key === "tests") return { name: title, method: description || "Validation notes", status: url ? "uploaded" : "draft", artifact: url, result: description };; line 7870: if (key === "pcbs") return { name: title, revision: "Rev A", status: url ? "uploaded" : "draft", artifact: url };; line 7871: if (key === "media") return { title, url, status: url ? "uploaded" : "draft", caption: description };. There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
7867: function makeItemForSection(key, title, description = "", url = "") {
7868:   if (key === "documents") return { title, type: "Document", url, status: url ? "uploaded" : "draft" };
7869:   if (key === "tests") return { name: title, method: description || "Validation notes", status: url ?
"uploaded" : "draft", artifact: url, result: description };
7870:   if (key === "pcbs") return { name: title, revision: "Rev A", status: url ? "uploaded" : "draft",
artifact: url };
7871:   if (key === "media") return { title, url, status: url ? "uploaded" : "draft", caption: description };
7872:   if (key === "links") return { label: title, url };
7873:   return { title, description, url, status: url ? "uploaded" : "draft" };
7874: }
```

addTextItem (template-preview.js, lines 7876-7891)

addTextItem part of the private builder frontend and editing experience. In this file, it appears around lines 7876-7891 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, prompt, includes, push, makeItemForSection, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7879: if (!project || !title) return;.

Important local values include project at line 7877; title at line 7878; description at line 7885. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7876: function addTextItem(key) {
7877:   const project = selectedProject();
```

```

7878:   const title = prompt("Type a title for this item.");
7879:   if (!project || !title) return;
7880:
7881:   project[key] = project[key] || [];
7882:   if (["highlights", "tools", "languages"].includes(key)) {
7883:     project[key].push(title);
7884:   } else {
7885:     const description = prompt("Add a short description if you want.") || "";
7886:     project[key].push(makeItemForSection(key, title, description));
7887:   }
7888:   setStatus("Item added in the editor. Click Save project to include this version in the portfolio
preview.");
7889:   scheduleAutosave();
7890:   renderAll();
7891: }

```

editObjectItem (template-preview.js, lines 7893-7914)

editObjectItem part of the private builder frontend and editing experience. In this file, it appears around lines 7893-7914 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, prompt, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 7896: if (!item) return;; line 7900: if (!nextTitle) return;.

Important local values include project at line 7894; item at line 7895; currentTitle at line 7898; nextTitle at line 7899; nextDescription at line 7901. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

7893: function editObjectItem(key, index) {
7894:   const project = selectedProject();
7895:   const item = project?.[key]?.[index];
7896:   if (!item) return;
7897:
7898:   const currentTitle = item.title || item.name || item.label || "";
7899:   const nextTitle = prompt("Edit title.", currentTitle);
7900:   if (!nextTitle) return;
7901:   const nextDescription = prompt("Edit description.", item.description || item.caption || item.result ||
item.method || "") || "";
7902:
7903:   if ("title" in item) item.title = nextTitle;
7904:   if ("name" in item) item.name = nextTitle;
7905:   if ("label" in item) item.label = nextTitle;
7906:   if ("caption" in item) item.caption = nextDescription;
7907:   if ("result" in item) item.result = nextDescription;
7908:   if ("description" in item) item.description = nextDescription;
7909:   if (!("caption" in item) && !("result" in item) && !("description" in item)) item.description =
nextDescription;
7910:
7911:   setStatus("Item edited in the editor. Click Save project to include this version in the portfolio
preview.");
7912:   scheduleAutosave();
7913:   renderAll();
7914: }

```

chooseFile (template-preview.js, lines 7916-7922)

chooseFile part of the private builder frontend and editing experience. In this file, it appears around lines 7916-7922 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references resolve, and click. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7917: return new Promise((resolve) => {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

7916: function chooseFile() {
7917:   return new Promise((resolve) => {
7918:     filePicker.value = "";
7919:     filePicker.onchange = () => resolve(filePicker.files[0] || null);
7920:     filePicker.click();
7921:   });
7922: }

```

newCompileFile (template-preview.js, lines 7924-7942)

newCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 7924-7942 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodeLanguage, safeClientCodeFileName, defaultCodeFileName, normalizeCompileFileRole, inferCompileFileRole, slugify, now, random, toString, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7928: return {.

Important local values include normalized at line 7925; fileName at line 7926; role at line 7927. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7924: function newCompileFile(language = "javascript", seed = {}) {
7925:   const normalized = normalizeCodeLanguage(language);
7926:   const fileName = safeClientCodeFileName(seed.fileName || defaultCodeFileName(normalized), normalized);
7927:   const role = normalizeCompileFileRole(seed.role || inferCompileFileRole(fileName, seed.code || "",
normalized), normalized);
7928:   return {
7929:     id: slugify(`${fileName}-${Date.now()}-${Math.random().toString(16).slice(2)}`),
7930:     title: seed.title || fileName,
7931:     fileName,
7932:     language: normalized,
7933:     role,
7934:     code: String(seed.code || ""),
7935:     stdin: "",
7936:     savedPath: "",
7937:     savedAt: "",
7938:     waveform: null,
7939:     lastResult: null,
7940:     dirty: true
7941:   };
7942: }
```

activeCompileWorkspaceAndFile (template-preview.js, lines 7944-7947)

activeCompileWorkspaceAndFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 7944-7947 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, ensureCompileCode, and activeCompileFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7946: return { workspace, file: activeCompileFile(project) };

Important local values include workspace at line 7945. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7944: function activeCompileWorkspaceAndFile(project = selectedProject()) {
7945:   const workspace = ensureCompileCode(project);
7946:   return { workspace, file: activeCompileFile(project) };
7947: }
```

updateCompilePreview (template-preview.js, lines 7949-7952)

updateCompilePreview updates ui, state, cache, or stored data. In this file, it appears around lines 7949-7952 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and tokenizedCodeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include preview at line 7950. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7949: function updateCompilePreview(file) {
7950:   const preview = sectionContent.querySelector("[data-compile-preview]");
7951:   if (preview && file) preview.innerHTML = tokenizedCodeHtml(file.code || "", file.language);
7952: }
```

syncCompileEditorScroll (template-preview.js, lines 7954-7960)

syncCompileEditorScroll part of the compile code language/tool/build/run flow. In this file, it appears around lines 7954-7960 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and querySelector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7957: if (!preview || !textarea) return;

Important local values include editor at line 7955; preview at line 7956. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7954: function syncCompileEditorScroll(textarea) {
7955:   const editor = textarea?.closest(".compile-code-active-editor");
7956:   const preview = editor?.querySelector(".compile-code-preview");
7957:   if (!preview || !textarea) return;
7958:   preview.scrollTop = textarea.scrollTop;
7959:   preview.scrollLeft = textarea.scrollLeft;
7960: }
```

compilePayload (template-preview.js, lines 7962-7984)

compilePayload part of the compile code language/tool/build/run flow. In this file, it appears around lines 7962-7984 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, inferCompileFileRole, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7964: return {.

Important local values include workspace at line 7963. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7962: function compilePayload(project, file, options = {}) {
7963:   const workspace = ensureCompileCode(project);
7964:   return {
7965:     projectId: project.id,
7966:     fileId: file.id,
7967:     title: file.title,
7968:     fileName: file.fileName,
7969:     language: file.language,
7970:     role: file.role || inferCompileFileRole(file.fileName, file.code, file.language),
7971:     code: file.code,
7972:     stdin: file.stdin || "",
7973:     workspaceFiles: (workspace.files || []).map((workspaceFile) => ({
7974:       id: workspaceFile.id,
7975:       title: workspaceFile.title,
7976:       fileName: workspaceFile.fileName,
7977:       language: workspaceFile.language,
7978:       role: workspaceFile.role || inferCompileFileRole(workspaceFile.fileName, workspaceFile.code,
workspaceFile.language),
7979:       code: workspaceFile.code || ""
7980:     })),
7981:     action: options.action || "",
7982:     forceRebuild: options.forceRebuild === true
7983:   };
7984: }
```

selectedCompileAppendDestination (template-preview.js, lines 7986-7990)

selectedCompileAppendDestination part of the compile code language/tool/build/run flow. In this file, it appears around lines 7986-7990 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `ensureCompileCode`, `compileAppendDestinations`, and `find`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 7989: `return destinations.find((destination) => destination.id === workspace.appendTargetId) || destinations[0] || null;`.

Important local values include `workspace` at line 7987; `destinations` at line 7988. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7986: function selectedCompileAppendDestination(project) {
7987:   const workspace = ensureCompileCode(project);
7988:   const destinations = compileAppendDestinations(project);
7989:   return destinations.find((destination) => destination.id === workspace.appendTargetId) ||
destinations[0] || null;
7990: }
```

appendCompileCodeToProject (template-preview.js, lines 7992-8019)

This function inserts a compile workspace source file into a chosen project section or subsection as a formatted code block.

Successful code needs a clean path from experimentation into portfolio evidence. This function moves code from the IDE workspace into the public-facing project narrative.

It calls or references `codeBlockForCompileFile`, `trim`, `setStatus`, `addCompileMessage`, `selectedCompileAppendDestination`, `richBlocksForCompileAppend`, `push`, `setByPath`, `plainTextFromRich`, `toISOString`, and 3 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 7997: `return false;`; line 8004: `return false;`; line 8018: `return true;`.

Important local values include `block` at line 7993; `destination` at line 8000; `blocks` at line 8007. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
7992: function appendCompileCodeToProject(project, file) {
7993:   const block = codeBlockForCompileFile(file);
7994:   if (!block.code.trim()) {
7995:     setStatus("Type or import code before appending it to the project.");
7996:     addCompileMessage(project, "Append stopped because the active source is empty.", "warning");
7997:     return false;
7998:   }
7999:
8000:   const destination = selectedCompileAppendDestination(project);
8001:   if (!destination) {
8002:     setStatus("No project destination is available for this code block.");
8003:     addCompileMessage(project, "Append stopped because no destination section is available.", "warning");
8004:     return false;
8005:   }
8006:
8007:   const blocks = richBlocksForCompileAppend(project, destination);
8008:   blocks.push(block);
8009:   setByPath(project, destination.richPath, { blocks });
8010:   setByPath(project, destination.textPath, plainTextFromRich({ blocks }));
8011:
8012:   file.lastAppendedAt = new Date().toISOString();
8013:   addCompileMessage(project, `${file.fileName || file.title} || "Source code" appended to
${destination.label}.`, "success");
8014:   setStatus(`Code appended to ${destination.label}. Save project to include it in the portfolio
preview.`);
8015:   scheduleAutosave();
8016:   schedulePreviewRender();
8017:   updateCompileMessagesPanel(project);
8018:   return true;
8019: }
```

saveCompileFile (template-preview.js, lines 8021-8038)

`saveCompileFile` persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 8021-8038 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `fetch`, `now`, `stringify`, `compilePayload`, `json`, `Error`, `toISOString`, and `addCompileMessage`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8022: if (!project || !file) return null;; line 8037: return result.saved;.

Important local values include response at line 8023; result at line 8029. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8021: async function saveCompileFile(project, file) {
8022:   if (!project || !file) return null;
8023:   const response = await fetch(`/api/code/save?t=${Date.now()}`, {
8024:     method: "POST",
8025:     cache: "no-store",
8026:     headers: { "Content-Type": "application/json" },
8027:     body: JSON.stringify(compilePayload(project, file))
8028:   });
8029:   const result = await response.json();
8030:   if (!response.ok || !result.ok) throw new Error(result.error || "Code could not be saved.");
8031:   file.savedPath = result.saved?.sourcePath || "";
8032:   file.savedAt = result.saved?.savedAt || new Date().toISOString();
8033:   file.fileName = result.saved?.fileName || file.fileName;
8034:   file.language = result.saved?.language || file.language;
8035:   file.dirty = false;
8036:   addCompileMessage(project, `Saved ${file.fileName || "source file"}`, "info");
8037:   return result.saved;
8038: }
```

importCompileFile (template-preview.js, lines 8040-8063)

importCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 8040-8063 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureCompileCode, chooseFile, flatMap, split, pop, toLowerCase, includes, setStatus, text, detectCodeLanguage, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8043: if (!file) return;; line 8048: return;.

Important local values include workspace at line 8041; file at line 8042; allowedExtensions at line 8044; extension at line 8045; code at line 8050; language at line 8051; profile at line 8052; sourceFile at line 8053. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8040: async function importCompileFile(project) {
8041:   const workspace = ensureCompileCode(project);
8042:   const file = await chooseFile();
8043:   if (!file) return;
8044:   const allowedExtensions = supportedCodeLanguages.flatMap((item) => item.extensions || []);
8045:   const extension = `.${String(file.name || "").split(".").pop().toLowerCase()}`;
8046:   if (!allowedExtensions.includes(extension)) {
8047:     setStatus("Choose a supported source file: C, C++, Verilog, SystemVerilog, LTspice, Java, JavaScript,
Python, or HTML.");
8048:     return;
8049:   }
8050:   const code = await file.text();
8051:   const language = detectCodeLanguage(code, file.name);
8052:   const profile = codeLanguageProfile(language);
8053:   const sourceFile = newCompileFile(profile.id, {
8054:     fileName: file.name,
8055:     title: file.name.replace(/\.([^.]+)$/, ""),
8056:     code
8057:   });
8058:   workspace.files.push(sourceFile);
8059:   workspace.activeFileId = sourceFile.id;
8060:   setStatus(`Imported ${file.name}. Click Save source before compiling.`);
8061:   scheduleAutosave();
8062:   renderSectionContent(project);
8063: }
```

beautifyCompileFile (template-preview.js, lines 8065-8085)

beautifyCompileFile part of the compile code language/tool/build/run flow. In this file, it appears around lines 8065-8085 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, stringify, compilePayload, json, Error, setStatus, scheduleAutosave, and renderSectionContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage

keys detected.

It has 1 return statement(s). The most important visible returns are: line 8066: if (!file) return;.

Important local values include response at line 8068; result at line 8074. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8065: async function beautifyCompileFile(project, file) {
8066:   if (!file) return;
8067:   try {
8068:     const response = await fetch(`/api/code/beautify?t=${Date.now()}`, {
8069:       method: "POST",
8070:       cache: "no-store",
8071:       headers: { "Content-Type": "application/json" },
8072:       body: JSON.stringify(compilePayload(project, file))
8073:     });
8074:     const result = await response.json();
8075:     if (!response.ok || !result.ok) throw new Error(result.error || "Beautifier failed.");
8076:     file.code = result.code || file.code;
8077:     file.language = result.language || file.language;
8078:     file.dirty = true;
8079:     setStatus("Code beautified. Review it, then save source.");
8080:     scheduleAutosave();
8081:     renderSectionContent(project);
8082:   } catch (error) {
8083:     setStatus(error.message || "Code beautifier failed.");
8084:   }
8085: }
```

compileActiveFile (template-preview.js, lines 8087-8145)

This function packages the current compile workspace, saves the active file state, calls the backend compile endpoint, then updates terminal output, messages, waveform data, dirty flags, and UI state.

It is the frontend half of the Compile button. Without it, the editor would not connect to server-side compilers or immediately show output.

It calls or references ensureCompileCode, isHdlLanguage, addCompileMessage, updateCompileTerminalPanel, updateCompileMessagesPanel, saveCompileFile, fetch, now, stringify, compilePayload, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8088: if (!file) return;.

Important local values include workspace at line 8089; action at line 8090; actionLabel at line 8091; response at line 8102; result at line 8111; compileResult at line 8112. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8087: async function compileActiveFile(project, file, options = {}) {
8088:   if (!file) return;
8089:   const workspace = ensureCompileCode(project);
8090:   const action = options.action || (isHdlLanguage(file.language) ? "simulate" : "run");
8091:   const actionLabel = action === "compile" ? "Compile" : action === "build" ? "Build project" : action
=== "simulate" ? "Simulate" : "Run";
8092:   file.lastResult = {
8093:     ok: null,
8094:     terminal: `${actionLabel} started. Please wait...`
8095:   };
8096:   workspace.terminal = file.lastResult.terminal;
8097:   addCompileMessage(project, `${actionLabel} started for ${file.fileName || file.title || "source
file"}`, "info");
8098:   updateCompileTerminalPanel(project, file);
8099:   updateCompileMessagesPanel(project);
8100:   try {
8101:     await saveCompileFile(project, file);
8102:     const response = await fetch(`/api/code/compile?t=${Date.now()}`, {
8103:       method: "POST",
8104:       cache: "no-store",
8105:       headers: { "Content-Type": "application/json" },
8106:       body: JSON.stringify(compilePayload(project, file, {
8107:         action,
8108:         forceRebuild: options.forceRebuild === true || action === "build"
8109:       })))
8110:   };
8111:   const result = await response.json();
8112:   const compileResult = result.result || {};
8113:   file.lastResult = {
8114:     ok: Boolean(result.ok),
8115:     language: compileResult.language || file.language,
8116:     terminal: compileResult.terminal || result.error || "No compiler output was returned.",
8117:     waveform: compileResult.waveform || null,
8118:     finishedAt: new Date().toISOString()
8119:   };
8120:   file.waveform = compileResult.waveform || null;
8121:   ensureCompileCode(project).waveform = compileResult.waveform || null;
```

```

8122:     if (compileResult.saved) {
8123:         file.savedPath = compileResult.saved.sourcePath || file.savedPath;
8124:         file.savedAt = compileResult.saved.savedAt || file.savedAt;
8125:     }
8126:     file.dirty = false;
8127:     ensureCompileCode(project).terminal = file.lastResult.terminal;
8128:     updateCompileTerminalPanel(project, file);
8129:     addCompileMessage(project, result.ok ? `${actionLabel} succeeded for ${file.fileName}.` :
`${actionLabel} completed with errors for ${file.fileName}.`, result.ok ? "success" : "error");
8130:     setStatus(result.ok ? `${actionLabel} succeeded.` : `${actionLabel} completed with errors.`);
8131:     scheduleAutosave();
8132:     renderSectionContent(project);
8133:   } catch (error) {
8134:     file.lastResult = {
8135:       ok: false,
8136:       terminal: error.message || "Compile/run failed.",
8137:       finishedAt: new Date().toISOString()
8138:     };
8139:     ensureCompileCode(project).terminal = file.lastResult.terminal;
8140:     updateCompileTerminalPanel(project, file);
8141:     addCompileMessage(project, `${actionLabel} failed for ${file.fileName || "source file"}.`, "error");
8142:     setStatus(error.message || `${actionLabel} failed.`);
8143:     renderSectionContent(project);
8144:   }
8145: }

```

checkCompileTools (template-preview.js, lines 8147-8162)

checkCompileTools part of the compile code language/tool/build/run flow. In this file, it appears around lines 8147-8162 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fetch, now, json, Error, entries, map, join, ensureCompileCode, renderSectionContent, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8154: return `\${status.ready ? "READY" : "MISSING"} \${status.label} (\${id}) - \${toolText}`;

Important local values include response at line 8149; result at line 8150; lines at line 8152; toolText at line 8153. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8147: async function checkCompileTools(project) {
8148:   try {
8149:     const response = await fetch(`${api}/code/tools?t=${Date.now()}`, { cache: "no-store" });
8150:     const result = await response.json();
8151:     if (!response.ok || !result.ok) throw new Error(result.error || "Compiler status failed.");
8152:     const lines = Object.entries(result.tools?.languages || {}).map(([id, status]) => {
8153:       const toolText = Object.entries(status.tools || {}).map(([tool, value]) => `${tool}:
${value}`).join(", ") || "no compiler required";
8154:       return `${status.ready ? "READY" : "MISSING"} ${status.label} (${id}) - ${toolText}`;
8155:     });
8156:     compileTerminalStatus = [`Compile workspace: ${result.tools?.compileRoot || ""}`,
...lines].join("\n");
8157:     ensureCompileCode(project).terminal = compileTerminalStatus;
8158:     renderSectionContent(project);
8159:   } catch (error) {
8160:     setStatus(error.message || "Compiler status failed.");
8161:   }
8162: }

```

installCompileTools (template-preview.js, lines 8164-8187)

installCompileTools part of the compile code language/tool/build/run flow. In this file, it appears around lines 8164-8187 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references codeLanguageLabel, renderSectionContent, fetch, now, stringify, json, toISOString, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8165: if (!file) return;

Important local values include response at line 8169; result at line 8175. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8164: async function installCompileTools(project, file) {
8165:   if (!file) return;
8166:   file.lastResult = { ok: null, terminal: `Installing tools for ${codeLanguageLabel(file.language)}. This
can take several minutes...` };
8167:   renderSectionContent(project);
8168:   try {
8169:     const response = await fetch(`/api/code/install-tools?t=${Date.now()}`, {
8170:       method: "POST",
8171:       cache: "no-store",
8172:       headers: { "Content-Type": "application/json" },
8173:       body: JSON.stringify({ language: file.language })
8174:     });
8175:     const result = await response.json();
8176:     file.lastResult = {
8177:       ok: Boolean(result.ok),
8178:       terminal: result.terminal || result.error || "Tool installation finished.",
8179:       finishedAt: new Date().toISOString()
8180:     };
8181:     setStatus(result.ok ? "Compiler tool installation finished." : "Compiler tool installation could not
complete.");
8182:     renderSectionContent(project);
8183:   } catch (error) {
8184:     file.lastResult = { ok: false, terminal: error.message || "Tool installation failed." };
8185:     renderSectionContent(project);
8186:   }
8187: }
```

readFileAsDataURL (template-preview.js, lines 8189-8196)

readFileAsDataURL loads data from disk, network, browser storage, or a service. In this file, it appears around lines 8189-8196 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references FileReader, resolve, and readAsDataURL. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8190: return new Promise((resolve, reject) => {.

Important local values include reader at line 8191. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8189: function readFileAsDataURL(file) {
8190:   return new Promise((resolve, reject) => {
8191:     const reader = new FileReader();
8192:     reader.onload = () => resolve(reader.result);
8193:     reader.onerror = reject;
8194:     reader.readAsDataURL(file);
8195:   });
8196: }
```

uploadProfileAsset (template-preview.js, lines 8198-8242)

uploadProfileAsset part of the private builder frontend and editing experience. In this file, it appears around lines 8198-8242 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references syncProfileFromInputs, chooseFile, Set, has, startsWith, setStatus, includes, extensionFor, toLowerCase, readFileAsDataURL, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 8201: if (!file) return;; line 8206: return;; line 8213: return;; line 8232: return;.

Important local values include profile at line 8199; file at line 8200; imageKinds at line 8203; allowedResume at line 8210; data at line 8217; response at line 8218; result at line 8229. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8198: async function uploadProfileAsset(kind) {
8199:   const profile = syncProfileFromInputs();
8200:   const file = await chooseFile();
8201:   if (!file) return;
8202:
8203:   const imageKinds = new Set(["profileImage", "heroImage", "brandImage"]);
```

```

8204:   if (imageKinds.has(kind) && !file.type.startsWith("image/")) {
8205:     setStatus("Choose an image file for this profile asset.");
8206:     return;
8207:   }
8208:
8209:   if (kind === "resumeUrl") {
8210:     const allowedResume = [".pdf", ".doc", ".docx"].includes(extensionFor(file.name).toLowerCase());
8211:     if (!allowedResume) {
8212:       setStatus("Choose a PDF, DOC, or DOCX resume file.");
8213:       return;
8214:     }
8215:   }
8216:
8217:   const data = await readFileAsDataURL(file);
8218:   const response = await fetch(`/api/upload?t=${Date.now()}`, {
8219:     method: "POST",
8220:     cache: "no-store",
8221:     headers: { "Content-Type": "application/json" },
8222:     body: JSON.stringify({
8223:       projectId: "_site-profile",
8224:       section: kind,
8225:       fileName: file.name,
8226:       data
8227:     })
8228:   });
8229:   const result = await response.json();
8230:   if (!response.ok) {
8231:     setStatus(result.error || "Profile asset upload failed.");
8232:     return;
8233:   }
8234:
8235:   profile[kind] = result.url;
8236:   catalog.profile = normalizeProfile(profile);
8237:   markDraftNeedsSave();
8238:   setStatus("Profile asset saved locally. Click Save draft before applying to site.");
8239:   scheduleAutosave();
8240:   schedulePreviewRender();
8241:   renderProfileEditor();
8242: }

```

extensionFor (template-preview.js, lines 8244-8247)

extensionFor part of the private builder frontend and editing experience. In this file, it appears around lines 8244-8247 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8246: return match ? match[1] : "";

Important local values include match at line 8245. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8244: function extensionFor(fileName) {
8245:   const match = String(fileName || "").match(/\.([a-z0-9]+)$/i);
8246:   return match ? match[1] : "";
8247: }

```

withExtension (template-preview.js, lines 8249-8253)

withExtension part of the private builder frontend and editing experience. In this file, it appears around lines 8249-8253 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and extensionFor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8251: if (!trimmed) return fallbackFileName;; line 8252: return /^[a-z0-9]+\$/.test(trimmed) ? trimmed : `\${trimmed}\${extensionFor(fallbackFileName)}`;

Important local values include trimmed at line 8250. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8249: function withExtension(name, fallbackFileName) {

```

```

8250:   const trimmed = String(name || "").trim();
8251:   if (!trimmed) return fallbackFileName;
8252:   return /\.[a-z0-9]+\$/i.test(trimmed) ? trimmed : `${trimmed}${extensionFor(fallbackFileName)}`;
8253: }

```

uploadProfile (template-preview.js, lines 8255-8286)

uploadProfile part of the private builder frontend and editing experience. In this file, it appears around lines 8255-8286 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8285: return profiles[key] || profiles.sections;

Important local values include anyProjectFile at line 8256; profiles at line 8263. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8255: function uploadProfile(key) {
8256:   const anyProjectFile = {
8257:     label: "project file",
8258:     accept: "",
8259:     extensions: [],
8260:     mimeStarts: [],
8261:     allFiles: true
8262:   };
8263:   const profiles = {
8264:     media: {
8265:       ...anyProjectFile,
8266:       label: "image or file"
8267:     },
8268:     pcbs: {
8269:       ...anyProjectFile,
8270:       label: "PCB artifact"
8271:     },
8272:     tests: {
8273:       ...anyProjectFile,
8274:       label: "test artifact"
8275:     },
8276:     documents: {
8277:       ...anyProjectFile,
8278:       label: "document"
8279:     },
8280:     sections: {
8281:       ...anyProjectFile,
8282:       label: "section file"
8283:     }
8284:   };
8285:   return profiles[key] || profiles.sections;
8286: }

```

allowedFileMessage (template-preview.js, lines 8288-8291)

allowedFileMessage part of the private builder frontend and editing experience. In this file, it appears around lines 8288-8291 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8289: if (profile.allFiles) return `Add any local file type for this \${profile.label}.`; line 8290: return `Add a \${profile.label}: \${profile.extensions.join(", ")}`;

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

8288: function allowedFileMessage(profile) {
8289:   if (profile.allFiles) return `Add any local file type for this ${profile.label}.`;
8290:   return `Add a ${profile.label}: ${profile.extensions.join(", ")}`;
8291: }

```

isAllowedUpload (template-preview.js, lines 8293-8306)

isAllowedUpload part of the private builder frontend and editing experience. In this file, it appears around lines 8293-8306 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uploadProfile, extensionFor, toLowerCase, includes, startsWith, some, and allowedFileMessage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 8295: if (profile.allFiles) return { ok: true, message: "" }; line 8298: return { ok: false, message: "Images can only accept image files. Add PDFs under Documents, Tests, or a custom section instead." }; line 8302: return {

Important local values include profile at line 8294; extension at line 8296; allowedByExtension at line 8300; allowedByMime at line 8301. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8293: function isAllowedUpload(key, fileName, mimeType = "") {
8294:   const profile = uploadProfile(key);
8295:   if (profile.allFiles) return { ok: true, message: "" };
8296:   const extension = extensionFor(fileName).toLowerCase();
8297:   if (key === "media" && !profile.extensions.includes(extension) && !mimeType.startsWith("image/")) {
8298:     return { ok: false, message: "Images can only accept image files. Add PDFs under Documents, Tests, or
a custom section instead." };
8299:   }
8300:   const allowedByExtension = profile.extensions.includes(extension);
8301:   const allowedByMime = profile.mimeStarts.some((prefix) => mimeType.startsWith(prefix));
8302:   return {
8303:     ok: allowedByExtension || allowedByMime,
8304:     message: `${fileName} || "This file" is not a supported ${profile.label}.
${allowedFileMessage(profile)}.`
8305:   };
8306: }
```

validateAssetForSection (template-preview.js, lines 8308-8317)

validateAssetForSection validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 8308-8317 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isAllowedUpload, normalizeLinkTarget, extensionFromUrl, and extensionFor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 8310: return isAllowedUpload(key, asset.fileName || asset.file?.name || "", asset.file?.type || ""); line 8314: return { ok: true, message: "" }; line 8316: return isAllowedUpload(key, url || asset.title || "", "");

Important local values include url at line 8312. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8308: function validateAssetForSection(key, asset) {
8309:   if (asset.source === "local") {
8310:     return isAllowedUpload(key, asset.fileName || asset.file?.name || "", asset.file?.type || "");
8311:   }
8312:   const url = normalizeLinkTarget(asset.url || "", { assumeWeb: true });
8313:   if (!extensionFromUrl(url) && !extensionFor(asset.title || "")) {
8314:     return { ok: true, message: "" };
8315:   }
8316:   return isAllowedUpload(key, url || asset.title || "", "");
8317: }
```

dialogValue (template-preview.js, lines 8319-8328)

dialogValue part of the private builder frontend and editing experience. In this file, it appears around lines 8319-8328 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references removeEventListener, resolve, addEventListener, and showModal. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8320: `return new Promise((resolve) => {`.

Important local values include `onClose` at line 8321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8319: function dialogValue(dialog) {
8320:   return new Promise((resolve) => {
8321:     const onClose = () => {
8322:       dialog.removeEventListener("close", onClose);
8323:       resolve(dialog.returnValue === "save");
8324:     };
8325:     dialog.addEventListener("close", onClose);
8326:     dialog.showModal();
8327:   });
8328: }
```

onClose (template-preview.js, lines 8321-8324)

`onClose` part of the private builder frontend and editing experience. In this file, it appears around lines 8321-8324 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `removeEventListener`, and `resolve`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include `onClose` at line 8321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8321:   const onClose = () => {
8322:     dialog.removeEventListener("close", onClose);
8323:     resolve(dialog.returnValue === "save");
8324:   };
```

openAssetDialog (template-preview.js, lines 8330-8392)

`openAssetDialog` controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8330-8392 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `uploadProfile`, `updateAssetDialogVisibility`, `dialogValue`, `trim`, `setStatus`, `withExtension`, `isAllowedUpload`, `normalizeLinkTarget`, `displayNameFromUrl`, `validateAssetForSection`, and 2 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 8344: `if (!saved) return null;`; line 8355: `return null;`; line 8362: `return null;`; line 8367: `return null;`. There are 2 additional return statements in the function body.

Important local values include `profile` at line 8331; `saved` at line 8343; `file` at line 8346; `url` at line 8347; `fileName` at line 8348; `title` at line 8349; `validation` at line 8359; `validation` at line 8371; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8330: async function openAssetDialog(key) {
8331:   const profile = uploadProfile(key);
8332:   assetDialogTitle.textContent = `Add ${profile.label}`;
8333:   assetSource.value = "local";
8334:   assetFile.value = "";
8335:   assetFile.accept = profile.accept;
8336:   assetUrl.value = "";
8337:   assetTitle.value = "";
8338:   assetCaption.value = "";
8339:   captionSource.value = "text";
8340:   captionFile.value = "";
8341:   updateAssetDialogVisibility();
8342:
8343:   const saved = await dialogValue(assetDialog);
8344:   if (!saved) return null;
8345:
8346:   let file = null;
8347:   let url = assetUrl.value.trim();
8348:   let fileName = "";
```



```

8349: let title = assetTitle.value.trim();
8350:
8351: if (assetSource.value === "local") {
8352:   file = assetFile.files[0];
8353:   if (!file) {
8354:     setStatus("Choose a local file first.");
8355:     return null;
8356:   }
8357:   fileName = withExtension(title, file.name);
8358:   title = title || file.name;
8359:   const validation = isAllowedUpload(key, fileName, file.type);
8360:   if (!validation.ok) {
8361:     setStatus(validation.message);
8362:     return null;
8363:   }
8364: } else {
8365:   if (!url) {
8366:     setStatus("Paste a web or Google Drive link first.");
8367:     return null;
8368:   }
8369:   url = normalizeLinkTarget(url, { assumeWeb: true });
8370:   title = title || displayNameFromUrl(url, "Linked asset");
8371:   const validation = validateAssetForSection(key, { source: assetSource.value, url, title });
8372:   if (!validation.ok) {
8373:     setStatus(validation.message);
8374:     return null;
8375:   }
8376: }
8377:
8378: let caption = assetCaption.value.trim();
8379: if (captionSource.value === "file" && captionFile.files[0]) {
8380:   caption = await captionFile.files[0].text();
8381: }
8382:
8383: return {
8384:   caption,
8385:   file,
8386:   fileName,
8387:   kind: file ? "file" : inferUrlAssetKind(url, assetSource.value),
8388:   source: assetSource.value,
8389:   title,
8390:   url
8391: };
8392: }

```

uploadToSection (template-preview.js, lines 8394-8454)

uploadToSection part of the private builder frontend and editing experience. In this file, it appears around lines 8394-8454 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, openAssetDialog, validateAssetForSection, setStatus, labelForUrlAssetKind, readFileAsDataURL, fetch, stringify, json, find, and 4 more. Endpoint references found inside it are /api/upload. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 8396: if (!project) return;; line 8399: if (!asset) return;; line 8404: return;; line 8426: return;. There are 1 additional return statements in the function body.

Important local values include project at line 8395; asset at line 8398; validation at line 8401; sectionFolder at line 8407; url at line 8408; type at line 8409; data at line 8412; response at line 8413; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8394: async function uploadToSection(key, folder, customSectionId = "", customItemIndex = null) {
8395:   const project = selectedProject();
8396:   if (!project) return;
8397:
8398:   const asset = await openAssetDialog(key);
8399:   if (!asset) return;
8400:
8401:   const validation = validateAssetForSection(key, asset);
8402:   if (!validation.ok) {
8403:     setStatus(validation.message);
8404:     return;
8405:   }
8406:
8407:   const sectionFolder = customSectionId || folder || key;
8408:   let url = asset.url;
8409:   let type = asset.source === "local" ? "File" : labelForUrlAssetKind(asset.kind);
8410:
8411:   if (asset.file) {
8412:     const data = await readFileAsDataURL(asset.file);
8413:     const response = await fetch("/api/upload", {

```

```

8414:     method: "POST",
8415:     headers: { "Content-Type": "application/json" },
8416:     body: JSON.stringify({
8417:       projectId: project.id,
8418:       section: sectionFolder,
8419:       fileName: asset.fileName,
8420:       data
8421:     })
8422:   });
8423:   const result = await response.json();
8424:   if (!response.ok) {
8425:     setStatus(result.error || "Upload failed.");
8426:     return;
8427:   }
8428:   url = result.url;
8429:   type = asset.file.type || "File";
8430: }
8431:
8432: if (customSectionId) {
8433:   const section = (project.sections || []).find((item) => item.id === customSectionId);
8434:   const uploadedItem = { title: asset.title, description: asset.caption, type, url, status:
asset.source === "local" ? "uploaded" : "linked" };
8435:   if (customItemIndex !== null && customItemIndex !== "") {
8436:     const parent = section?.items?.[Number(customItemIndex)];
8437:     if (!parent || parent.url) return;
8438:     parent.children = parent.children || [];
8439:     parent.children.push(uploadedItem);
8440:   } else {
8441:     section.items = section.items || [];
8442:     section.items.push(uploadedItem);
8443:   }
8444: } else {
8445:   project[key] = project[key] || [];
8446:   project[key].push(makeItemForSection(key, asset.title, asset.caption, url));
8447: }
8448:
8449: setStatus(asset.source === "local"
8450:   ? `Saved ${asset.fileName} to ${url}. Click Save project to include it in the portfolio preview.`
8451:   : `Linked ${asset.title}. Click Save project to include it in the portfolio preview.`);
8452: scheduleAutosave();
8453: renderAll();
8454: }

```

section (template-preview.js, lines 8433-8434)

section part of the private builder frontend and editing experience. In this file, it appears around lines 8433-8434 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include section at line 8433; uploadedItem at line 8434. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8433:   const section = (project.sections || []).find((item) => item.id === customSectionId);
8434:   const uploadedItem = { title: asset.title, description: asset.caption, type, url, status:
asset.source === "local" ? "uploaded" : "linked" };

```

uploadToDesignSection (template-preview.js, lines 8456-8502)

uploadToDesignSection part of the private builder frontend and editing experience. In this file, it appears around lines 8456-8502 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, openAssetDialog, validateAssetForSection, setStatus, labelForUrlAssetKind, readFileAsDataURL, fetch, stringify, json, designArray, and 4 more. Endpoint references found inside it are /api/upload. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 8458: if (!project) return;; line 8461: if (!asset) return;; line 8466: return;; line 8486: return;;

Important local values include project at line 8457; asset at line 8460; validation at line 8463; url at line 8469; type at line 8470; data at line 8472; response at line 8473; result at line 8483; plus 1 more local variable(s). These variables show what the function is assembling,

checking, or handing to another layer.

Relevant source excerpt:

```
8456: async function uploadToDesignSection(pathValue, folder, kind = "document") {
8457:   const project = selectedProject();
8458:   if (!project) return;
8459:
8460:   const asset = await openAssetDialog(kind === "simulation-result" ? "tests" : "sections");
8461:   if (!asset) return;
8462:
8463:   const validation = validateAssetForSection(kind === "simulation-result" ? "tests" : "sections", asset);
8464:   if (!validation.ok) {
8465:     setStatus(validation.message);
8466:     return;
8467:   }
8468:
8469:   let url = asset.url;
8470:   let type = asset.source === "local" ? "File" : labelForUrlAssetKind(asset.kind);
8471:   if (asset.file) {
8472:     const data = await readFileAsDataURL(asset.file);
8473:     const response = await fetch("/api/upload", {
8474:       method: "POST",
8475:       headers: { "Content-Type": "application/json" },
8476:       body: JSON.stringify({
8477:         projectId: project.id,
8478:         section: folder || "design",
8479:         fileName: asset.fileName,
8480:         data
8481:       })
8482:     });
8483:     const result = await response.json();
8484:     if (!response.ok) {
8485:       setStatus(result.error || "Upload failed.");
8486:       return;
8487:     }
8488:     url = result.url;
8489:     type = asset.file.type || "File";
8490:   }
8491:
8492:   const items = designArray(project, pathValue);
8493:   items.push({
8494:     ...makeDesignItem(kind, asset.title, asset.caption, url),
8495:     type
8496:   });
8497:   setStatus(asset.source === "local"
8498:     ? `Saved ${asset.fileName} to ${url}. Click Save project to include it in the portfolio preview.`
8499:     : `Linked ${asset.title}. Click Save project to include it in the portfolio preview.`);
8500:   scheduleAutosave();
8501:   renderAll();
8502: }
```

openSectionDialog (template-preview.js, lines 8504-8523)

openSectionDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8504-8523 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references dialogValue, trim, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 8511: if (!saved) return null;; line 8516: return null;; line 8519: return {}.

Important local values include saved at line 8510; title at line 8513. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8504: async function openSectionDialog() {
8505:   sectionDialogTitle.textContent = "Create a section";
8506:   sectionTitle.placeholder = "Design, Measurements, Firmware...";
8507:   sectionDescription.placeholder = "Optional description shown to recruiters";
8508:   sectionTitle.value = "";
8509:   sectionDescription.value = "";
8510:   const saved = await dialogValue(sectionDialog);
8511:   if (!saved) return null;
8512:
8513:   const title = sectionTitle.value.trim();
8514:   if (!title) {
8515:     setStatus("Type a section title before saving.");
8516:     return null;
8517:   }
8518:
8519:   return {
```

```

8520:     description: sectionDescription.value.trim(),
8521:     title
8522:   };
8523: }

```

openCategoryDialog (template-preview.js, lines 8525-8531)

openCategoryDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8525-8531 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references showModal, and focus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

8525: function openCategoryDialog() {
8526:   categoryTitle.value = "";
8527:   categoryDescription.value = "";
8528:   categoryAccent.value = "#1677a8";
8529:   categoryDialog.showModal();
8530:   setTimeout(() => categoryTitle.focus(), 0);
8531: }

```

saveCategoryFromDialog (template-preview.js, lines 8533-8559)

saveCategoryFromDialog persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 8533-8559 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, setStatus, slugify, Set, map, has, normalizeCategory, add, refreshCategoryControls, renderAll, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8537: return;.

Important local values include label at line 8534; baseId at line 8539; ids at line 8540; id at line 8541; suffix at line 8542; category at line 8547. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8533: function saveCategoryFromDialog() {
8534:   const label = categoryTitle.value.trim();
8535:   if (!label) {
8536:     setStatus("Type a category name before saving.");
8537:     return;
8538:   }
8539:   const baseId = slugify(label);
8540:   const ids = new Set((catalog.categories || []).map((category) => category.id));
8541:   let id = baseId;
8542:   let suffix = 2;
8543:   while (ids.has(id)) {
8544:     id = `${baseId}-${suffix}`;
8545:     suffix += 1;
8546:   }
8547:   const category = normalizeCategory({
8548:     accent: categoryAccent.value,
8549:     description: categoryDescription.value,
8550:     id,
8551:     label
8552:   });
8553:   catalog.categories = [...(catalog.categories || []), category];
8554:   expandedCategories.add(category.id);
8555:   refreshCategoryControls();
8556:   renderAll();
8557:   scheduleAutosave();
8558:   setStatus(`Category "${category.label}" added locally. It is now available under Add new project.`);
8559: }

```

addCustomSection (template-preview.js, lines 8561-8582)

addCustomSection part of the private builder frontend and editing experience. In this file, it appears around lines 8561-8582 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `selectedProject`, `openSectionDialog`, `slugify`, `Set`, `map`, `has`, `push`, `setStatus`, `scheduleAutosave`, and `renderAll`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8563: `if (!project) return;`; line 8566: `if (!sectionInput) return;`.

Important local values include `project` at line 8562; `sectionInput` at line 8565; `id` at line 8568; `existingIds` at line 8569; `suffix` at line 8570. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8561: async function addCustomSection() {
8562:   const project = selectedProject();
8563:   if (!project) return;
8564:
8565:   const sectionInput = await openSectionDialog();
8566:   if (!sectionInput) return;
8567:
8568:   let id = slugify(sectionInput.title);
8569:   const existingIds = new Set((project.sections || []).map((section) => section.id));
8570:   let suffix = 2;
8571:   while (existingIds.has(id)) {
8572:     id = `${slugify(sectionInput.title)}-${suffix}`;
8573:     suffix += 1;
8574:   }
8575:
8576:   project.sections = project.sections || [];
8577:   project.sections.push({ id, title: sectionInput.title, description: sectionInput.description, items: []
});
8578:   activeSectionId = `custom:${id}`;
8579:   setStatus("New section added in the editor. Click Save project to include this version in the portfolio
preview.");
8580:   scheduleAutosave();
8581:   renderAll();
8582: }
```

addCustomItem (template-preview.js, lines 8584-8596)

`addCustomItem` part of the private builder frontend and editing experience. In this file, it appears around lines 8584-8596 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `selectedProject`, `find`, `prompt`, `push`, `setStatus`, `scheduleAutosave`, and `renderAll`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8588: `if (!section || !title) return;`.

Important local values include `project` at line 8585; `section` at line 8586; `title` at line 8587; `description` at line 8589. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8584: function addCustomItem(sectionId) {
8585:   const project = selectedProject();
8586:   const section = (project.sections || []).find((item) => item.id === sectionId);
8587:   const title = prompt("Type a subsection title.");
8588:   if (!section || !title) return;
8589:   const description = prompt("Paste or type a text description if you want.") || "";
8590:
8591:   section.items = section.items || [];
8592:   section.items.push({ title, description, type: "Text", status: "draft" });
8593:   setStatus("Subsection added locally.");
8594:   scheduleAutosave();
8595:   renderAll();
8596: }
```

section (template-preview.js, lines 8586-8589)

`section` part of the private builder frontend and editing experience. In this file, it appears around lines 8586-8589 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `find`, and `prompt`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8588: if (!section || !title) return;.

Important local values include section at line 8586; title at line 8587; description at line 8589. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8586: const section = (project.sections || []).find((item) => item.id === sectionId);
8587: const title = prompt("Type a subsection title.");
8588: if (!section || !title) return;
8589: const description = prompt("Paste or type a text description if you want.") || "";
```

editCustomItem (template-preview.js, lines 8598-8609)

editCustomItem part of the private builder frontend and editing experience. In this file, it appears around lines 8598-8609 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references selectedProject, find, prompt, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8601: if (!item) return;; line 8603: if (!title) return;.

Important local values include project at line 8599; item at line 8600; title at line 8602. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8598: function editCustomItem(sectionId, index) {
8599:   const project = selectedProject();
8600:   const item = (project.sections || []).find((section) => section.id === sectionId)?.items?.[index];
8601:   if (!item) return;
8602:   const title = prompt("Edit subsection title.", item.title || "");
8603:   if (!title) return;
8604:   item.title = title;
8605:   item.description = prompt("Edit description.", item.description || "") || "";
8606:   setStatus("Subsection edited locally.");
8607:   scheduleAutosave();
8608:   renderAll();
8609: }
```

item (template-preview.js, lines 8600-8608)

item part of the private builder frontend and editing experience. In this file, it appears around lines 8600-8608 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, prompt, setStatus, scheduleAutosave, and renderAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8601: if (!item) return;; line 8603: if (!title) return;.

Important local values include item at line 8600; title at line 8602. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8600: const item = (project.sections || []).find((section) => section.id === sectionId)?.items?.[index];
8601: if (!item) return;
8602: const title = prompt("Edit subsection title.", item.title || "");
8603: if (!title) return;
8604: item.title = title;
8605: item.description = prompt("Edit description.", item.description || "") || "";
8606: setStatus("Subsection edited locally.");
8607: scheduleAutosave();
8608: renderAll();
```

catalogForSave (template-preview.js, lines 8611-8633)

catalogForSave part of the private builder frontend and editing experience. In this file, it appears around lines 8611-8633 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `parsedPortfolioGlobals`, and `clone`. Endpoint references found inside it are `/api/apply-catalog`. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 8614: `return {}`; line 8620: `return {}`.

Important local values include `parsedGlobals` at line 8612. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8611: function catalogForSave(endpoint) {
8612:   const parsedGlobals = parsedPortfolioGlobals();
8613:   if (endpoint === "/api/apply-catalog") {
8614:     return {
8615:       ...savedPortfolioCatalog,
8616:       ...parsedGlobals,
8617:       projects: clone(savedPortfolioCatalog.projects || [])
8618:     };
8619:   }
8620:   return {
8621:     ...catalog,
8622:     categories: parsedGlobals.categories,
8623:     fieldStyles: parsedGlobals.fieldStyles,
8624:     funFacts: parsedGlobals.funFacts,
8625:     funFactsRich: parsedGlobals.funFactsRich,
8626:     profile: parsedGlobals.profile,
8627:     profileRich: parsedGlobals.profileRich,
8628:     siteContent: parsedGlobals.siteContent,
8629:     siteContentRich: parsedGlobals.siteContentRich,
8630:     siteSections: clone(catalog.siteSections || []),
8631:     projects: clone(catalog.projects || [])
8632:   };
8633: }
```

saveCatalog (template-preview.js, lines 8635-8697)

This function saves the builder catalog through the backend, coordinates parser output, and updates local saved state after successful persistence.

Most builder edits mean nothing until the catalog is saved. This function is the local persistence checkpoint before previewing or publishing.

It calls or references `catalogForSave`, `showBuilderError`, `setStatus`, `setSaveState`, `fetch`, `now`, `stringify`, `json`, and `showPublishResult`. Endpoint references found inside it are `/api/apply-catalog`, and `/api/save-draft`. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 8645: `return;;`; line 8655: `return;;`; line 8675: `return;;`; line 8683: `return;;`.

Important local values include `targetCatalog` at line 8636; applying at line 8662; response at line 8665; result at line 8671. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8635: async function saveCatalog(endpoint, message) {
8636:   const targetCatalog = catalogForSave(endpoint);
8637:   if (endpoint === "/api/apply-catalog" && !draftSavedSinceChanges) {
8638:     showBuilderError(
8639:       "Save draft required",
8640:       "Please click Save draft before applying this portfolio to the live site.",
8641:       "Apply to site was stopped before commit and push. Save the draft, then click Apply to site again."
8642:     );
8643:     setStatus("Apply to site stopped. Save draft first.");
8644:     setSaveState("dirty", "Save draft needed");
8645:     return;
8646:   }
8647:   if (endpoint === "/api/apply-catalog" && !(targetCatalog.projects || []).length) {
8648:     showBuilderError(
8649:       "No saved project preview",
8650:       "Save a project or use Save all sections before applying to the live site.",
8651:       "Apply to site was stopped before commit and push."
8652:     );
8653:     setStatus("Apply to site stopped. Save a project or use Save all sections first.");
8654:     setSaveState("dirty", "Preview needs projects");
8655:     return;
8656:   }
8657:
8658:   try {
8659:     clearTimeout(autosaveTimer);
8660:     saveDraftButton.disabled = true;
8661:     applyCatalogButton.disabled = true;
8662:     const applying = endpoint === "/api/apply-catalog";
8663:     setStatus(applying ? "Applying site changes..." : "Saving draft...");
8664:     setSaveState(applying ? "publishing" : "saving", applying ? "Publishing..." : "Saving...");
8665:     const response = await fetch(`${endpoint}?t=${Date.now()}`, {
8666:       method: "POST",
```

```

8667:     cache: "no-store",
8668:     headers: { "Content-Type": "application/json" },
8669:     body: JSON.stringify({ catalog: targetCatalog });
8670: });
8671: const result = await response.json();
8672: if (!response.ok) {
8673:     setStatus(result.error || "Save failed.");
8674:     setSaveState("error", endpoint === "/api/apply-catalog" ? "Publish failed" : "Save failed");
8675:     return;
8676: }
8677: if (result.publish) {
8678:     setStatus(result.publish.pushed
8679:         ? `${message}: ${result.file}. Pushed to ${result.publish.branch}.`
8680:         : `${message}: ${result.file}. Git push failed: ${result.publish.error}`);
8681:     setSaveState(result.publish.pushed ? "published" : "error", result.publish.pushed ? "Site applied"
: "Push failed");
8682:     showPublishResult(result);
8683:     return;
8684: }
8685: if (endpoint === "/api/save-draft") {
8686:     draftSavedSinceChanges = true;
8687:     setSaveState("saved", "Draft saved");
8688: }
8689: setStatus(`${message}: ${result.file}`);
8690: } catch {
8691:     setStatus("Save failed. Make sure the local server window is still running.");
8692:     setSaveState("error", "Save failed");
8693: } finally {
8694:     saveDraftButton.disabled = false;
8695:     applyCatalogButton.disabled = false;
8696: }
8697: }

```

loadData (template-preview.js, lines 8699-8745)

This function loads templates, projects, saved site content, profile data, categories, and current builder state into the frontend.

It is the startup data path for the builder UI. If it fails, panels may appear empty even when files exist on disk.

It calls or references all, fetch, now, json, call, normalizeFunFacts, clone, normalizeFieldStyles, normalizeProfile, normalizeSiteContent, and 13 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include catalogData at line 8704; templateData at line 8705. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8699: async function loadData() {
8700:     const [catalogResponse, templateResponse] = await Promise.all([
8701:         fetch(`/api/catalog?t=${Date.now()}`, { cache: "no-store" }),
8702:         fetch(`/api/templates?t=${Date.now()}`, { cache: "no-store" })
8703:     ]);
8704:     const catalogData = await catalogResponse.json();
8705:     const templateData = await templateResponse.json();
8706:
8707:     catalog = catalogData.catalog;
8708:     catalog.funFacts = Object.prototype.hasOwnProperty.call(catalog, "funFacts")
8709:         ? normalizeFunFacts(catalog.funFacts)
8710:         : clone(defaultFunFacts);
8711:     catalog.funFactsRich = catalog.funFactsRich || null;
8712:     catalog.fieldStyles = normalizeFieldStyles(catalog.fieldStyles || {});
8713:     catalog.siteContentRich = catalog.siteContentRich || {};
8714:     catalog.profileRich = catalog.profileRich || {};
8715:     catalog.profile = normalizeProfile(catalog.profile || {});
8716:     catalog.siteContent = normalizeSiteContent(catalog.siteContent || {});
8717:     catalog.siteSections = catalog.siteSections || [];
8718:     catalog.projects = (catalog.projects || []).filter((project) => !starterProjectIds.has(project.id));
8719:     catalog.projects.forEach((project) => {
8720:         if (isAnalogProject(project)) ensureDesignModel(project);
8721:         ensureCompileCode(project);
8722:     });
8723:     savedPortfolioCatalog = {
8724:         categories: clone(catalog.categories || []),
8725:         fieldStyles: clone(catalog.fieldStyles || {}),
8726:         funFacts: clone(catalog.funFacts || []),
8727:         funFactsRich: clone(catalog.funFactsRich || null),
8728:         profile: clone(normalizeProfile(catalog.profile || {})),
8729:         profileRich: clone(catalog.profileRich || {}),
8730:         siteContent: clone(catalog.siteContent || defaultSiteContent),
8731:         siteContentRich: clone(catalog.siteContentRich || {}),
8732:         projects: [],
8733:         siteSections: clone(catalog.siteSections || [])
8734:     };
8735:     templates = templateData.templates || [];
8736:     setStatus(`Loaded ${catalogData.source} catalog. Builder controls are local-only.`);

```



```

8737:   draftSavedSinceChanges = true;
8738:   setSaveState("loaded", "Loaded");
8739:
8740:   templateSelect.innerHTML = groupedTemplateOptions();
8741:   refreshCategoryControls();
8742:   selectedProjectId = catalog.projects[0]?.id || "";
8743:   expandedCategories = new Set(catalog.categories.map((category) => category.id));
8744:   renderAll();
8745: }

```

prepareBuilderGuideTopicLinks (template-preview.js, lines 8758-8768)

prepareBuilderGuideTopicLinks part of the private builder frontend and editing experience. In this file, it appears around lines 8758-8768 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, querySelector, setAttribute, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8761: if (!summary) return;.

Important local values include summary at line 8760. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8758: function prepareBuilderGuideTopicLinks() {
8759:   builderGuideSections?.querySelectorAll("[data-guide-section]").forEach((section) => {
8760:     const summary = section.querySelector("summary");
8761:     if (!summary) return;
8762:     summary.setAttribute("role", "link");
8763:     summary.setAttribute("tabindex", "0");
8764:     summary.setAttribute("aria-label", `Open ${summary.textContent.trim()}`);
8765:     summary.title = "Open this guide topic";
8766:     section.open = false;
8767:   });
8768: }

```

updateBuilderGuideStats (template-preview.js, lines 8770-8849)

updateBuilderGuideStats updates ui, state, cache, or stored data. In this file, it appears around lines 8770-8849 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, entries, forEach, querySelector, guideSectionText, join, toLowerCase, filterBuilderGuide, trim, querySelectorAll, and 10 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 8771: if (!builderGuideDialog) return;; line 8791: return []; line 8799: if (!builderGuideSections) return;.

Important local values include projectCount at line 8772; categoryCount at line 8773; parsedCount at line 8774; targetLabel at line 8775; values at line 8778; node at line 8785; query at line 8800; sections at line 8801; plus 5 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

8770: function updateBuilderGuideStats() {
8771:   if (!builderGuideDialog) return;
8772:   const projectCount = catalog.projects?.length || 0;
8773:   const categoryCount = catalog.categories?.length || 0;
8774:   const parsedCount = savedPortfolioCatalog.projects?.length || 0;
8775:   const targetLabel = currentPublishTarget?.remote
8776:     ? `Target: ${currentPublishTarget.remote.replace(/^https:\/\/github\.com\/\//i, "")}`
8777:     : "Target: not connected";
8778:   const values = {
8779:     projects: `Projects: ${projectCount} (${parsedCount} parsed)`,
8780:     categories: `Categories: ${categoryCount}`,
8781:     draft: draftSavedSinceChanges ? "Draft: saved" : "Draft: unsaved changes",
8782:     target: targetLabel
8783:   };
8784:   Object.entries(values).forEach(([key, value]) => {
8785:     const node = builderGuideDialog.querySelector(`[data-guide-stat='${key}']`);
8786:     if (node) node.textContent = value;
8787:   });
8788: }
8789:
8790: function guideSectionText(section) {

```

```

8791:   return [
8792:     section.querySelector("summary")?.textContent || "",
8793:     section.textContent || "",
8794:     section.dataset.guidSection || ""
8795:   ].join(" ").toLowerCase();
8796: }
8797:
8798: function filterBuilderGuide() {
8799:   if (!builderGuideSections) return;
8800:   const query = String(builderGuideSearch?.value || "").trim().toLowerCase();
8801:   const sections = [...builderGuideSections.querySelectorAll("[data-guide-section]")];
8802:   let visibleCount = 0;
8803:   sections.forEach((section) => {
8804:     const topicMatch = activeBuilderGuideTopic === "all" || (section.dataset.guidSection ||
8805:       "").split(/s+/).includes(activeBuilderGuideTopic);
8806:     const queryMatch = !query || guideSectionText(section).includes(query);
8807:     const visible = topicMatch && queryMatch;
8808:     section.hidden = !visible;
8809:     if (visible) {
8810:       visibleCount += 1;
8811:       section.open = false;
8812:     }
8813:   });
8814:   if (builderGuideResults) {
8815:     const topicText = activeBuilderGuideTopic === "all" ? "all topics" : activeBuilderGuideTopic;
8816:     builderGuideResults.textContent = visibleCount
8817:       ? `Showing ${visibleCount} guide topic${visibleCount === 1 ? "" : "s"} for ${topicText}${query ? `
8818:         matching "${query}"` : ""}.`
8819:       : `No guide topics match "${query}"`;
8820:   }
8821: }
8822:
8823: function setBuilderGuideTopic(topic = "all") {
8824:   activeBuilderGuideTopic = topic || "all";
8825:   builderGuideDialog?.querySelectorAll("[data-guide-topic]").forEach((button) => {
8826:     button.setAttribute("aria-pressed", button.dataset.guidTopic === activeBuilderGuideTopic ? "true" :
8827:       "false");
8828:   });
8829:   filterBuilderGuide();
8830: }
8831:
8832: function closeGuideAndFocus(target) {
8833:   closeDialogElement(builderGuideDialog);
8834:   requestAnimationFrame(() => {
8835:     target?.scrollIntoView?({ behavior: "smooth", block: "center" });
8836:     if (target?.focus) target.focus({ preventScroll: true });
8837:   });
8838: }
8839:
8840: function handleBuilderGuideAction(action = "") {
8841:   if (action === "profile") closeGuideAndFocus(profileDisplayNameInput);
8842:   if (action === "site-content") closeGuideAndFocus(siteHeroTitleInput);
8843:   if (action === "fun-facts") closeGuideAndFocus(funFactsInput);
8844:   if (action === "projects") closeGuideAndFocus(projectTree);
8845:   if (action === "preview") {
8846:     closeDialogElement(builderGuideDialog);
8847:     openPortfolioPreviewButton?.click();
8848:   }
8849:   if (action === "target") {
8850:     closeDialogElement(builderGuideDialog);
8851:     publishTargetOpen?.click();
8852:   }
8853: }

```

guideSectionText (template-preview.js, lines 8790-8796)

guideSectionText part of the private builder frontend and editing experience. In this file, it appears around lines 8790-8796 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, join, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8791: return [

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

8790: function guideSectionText(section) {
8791:   return [
8792:     section.querySelector("summary")?.textContent || "",
8793:     section.textContent || "",
8794:     section.dataset.guidSection || ""
8795:   ].join(" ").toLowerCase();
8796: }

```

filterBuilderGuide (template-preview.js, lines 8798-8819)

filterBuilderGuide part of the private builder frontend and editing experience. In this file, it appears around lines 8798-8819 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, querySelectorAll, forEach, split, includes, and guideSectionText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8799: if (!builderGuideSections) return;.

Important local values include query at line 8800; sections at line 8801; visibleCount at line 8802; topicMatch at line 8804; queryMatch at line 8805; visible at line 8806; topicText at line 8814. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8798: function filterBuilderGuide() {
8799:   if (!builderGuideSections) return;
8800:   const query = String(builderGuideSearch?.value || "").trim().toLowerCase();
8801:   const sections = [...builderGuideSections.querySelectorAll("[data-guide-section]")];
8802:   let visibleCount = 0;
8803:   sections.forEach((section) => {
8804:     const topicMatch = activeBuilderGuideTopic === "all" || (section.dataset.guideSection ||
8805:   "").split(/\s+/).includes(activeBuilderGuideTopic);
8806:     const queryMatch = !query || guideSectionText(section).includes(query);
8807:     const visible = topicMatch && queryMatch;
8808:     section.hidden = !visible;
8809:     if (visible) {
8810:       visibleCount += 1;
8811:       section.open = false;
8812:     }
8813:   });
8814:   if (builderGuideResults) {
8815:     const topicText = activeBuilderGuideTopic === "all" ? "all topics" : activeBuilderGuideTopic;
8816:     builderGuideResults.textContent = visibleCount
8817:       ? `Showing ${visibleCount} guide topic${visibleCount === 1 ? "" : "s"} for ${topicText}${query ? `
8818:   matching "${query}"` : ""}` : `No guide topics match "${query}"`;
8819:   }
```

setBuilderGuideTopic (template-preview.js, lines 8821-8827)

setBuilderGuideTopic part of the private builder frontend and editing experience. In this file, it appears around lines 8821-8827 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, setAttribute, and filterBuilderGuide. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
8821: function setBuilderGuideTopic(topic = "all") {
8822:   activeBuilderGuideTopic = topic || "all";
8823:   builderGuideDialog?.querySelectorAll("[data-guide-topic]").forEach((button) => {
8824:     button.setAttribute("aria-pressed", button.dataset.guideTopic === activeBuilderGuideTopic ? "true" :
8825:   "false");
8826:   });
8827:   filterBuilderGuide();
8828: }
```

closeGuideAndFocus (template-preview.js, lines 8829-8835)

closeGuideAndFocus controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8829-8835 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closeDialogElement, requestAnimationFrame, and focus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
8829: function closeGuideAndFocus(target) {
8830:   closeDialogElement(builderGuideDialog);
8831:   requestAnimationFrame(() => {
8832:     target?.scrollIntoView?({ behavior: "smooth", block: "center" });
8833:     if (target?.focus) target.focus({ preventScroll: true });
8834:   });
8835: }
```

handleBuilderGuideAction (template-preview.js, lines 8837-8854)

handleBuilderGuideAction handles an event, request, command, or user action. In this file, it appears around lines 8837-8854 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closeGuideAndFocus, closeDialogElement, and click. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
8837: function handleBuilderGuideAction(action = "") {
8838:   if (action === "profile") closeGuideAndFocus(profileDisplayNameInput);
8839:   if (action === "site-content") closeGuideAndFocus(siteHeroTitleInput);
8840:   if (action === "fun-facts") closeGuideAndFocus(funFactsInput);
8841:   if (action === "projects") closeGuideAndFocus(projectTree);
8842:   if (action === "preview") {
8843:     closeDialogElement(builderGuideDialog);
8844:     openPortfolioPreviewButton?.click();
8845:   }
8846:   if (action === "target") {
8847:     closeDialogElement(builderGuideDialog);
8848:     publishTargetOpen?.click();
8849:   }
8850:   if (action === "updates") {
8851:     closeDialogElement(builderGuideDialog);
8852:     appUpdateCheckButton?.click();
8853:   }
8854: }
```

openBuilderGuideTopic (template-preview.js, lines 8856-8878)

openBuilderGuideTopic controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 8856-8878 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, trim, replaceChildren, cloneNode, add, append, createElement, showModal, requestAnimationFrame, focus, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 8857: if (!section || !builderGuideTopicDialog || !builderGuideTopicTitle || !builderGuideTopicBody) return;.

Important local values include summary at line 8858; source at line 8859; title at line 8860; copy at line 8864; empty at line 8868. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
8856: function openBuilderGuideTopic(section) {
8857:   if (!section || !builderGuideTopicDialog || !builderGuideTopicTitle || !builderGuideTopicBody) return;
8858:   const summary = section.querySelector("summary");
8859:   const source = section.querySelector(":scope > div");
8860:   const title = summary?.textContent?.trim() || "Builder guide topic";
8861:   builderGuideTopicTitle.textContent = title;
8862:   builderGuideTopicBody.replaceChildren();
8863:   if (source) {
8864:     const copy = source.cloneNode(true);
8865:     copy.classList.add("builder-guide-topic-copy");
8866:     builderGuideTopicBody.append(copy);
8867:   } else {
```

```

8868:     const empty = document.createElement("p");
8869:     empty.textContent = "This guide topic does not have written instructions yet.";
8870:     builderGuideTopicBody.append(empty);
8871:   }
8872:   if (!builderGuideTopicDialog.open) builderGuideTopicDialog.showModal();
8873:   requestAnimationFrame(() => {
8874:     builderGuideTopicBody.scrollTop = 0;
8875:     builderGuideTopicBody.focus({ preventScroll: true });
8876:     updateDialogWindowButtons(builderGuideTopicDialog);
8877:   });
8878: }

```

handleRichEditorContextMenu (template-preview.js, lines 9312-9372)

handleRichEditorContextMenu handles an event, request, command, or user action. In this file, it appears around lines 9312-9372 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richEditorFromContextEvent, preventDefault, stopPropagation, closest, clearPersistentTextSelection, hideTextSelectionInspector, configureSummaryContextMenu, selectionRangeInsideEditor, cloneRange, contains, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9316: if (!editor && !textBlock) return;.

Important local values include editor at line 9313; textBlock at line 9314; nextEditor at line 9320; liveRange at line 9333; savedRange at line 9334; savedContainer at line 9337; menuHost at line 9352; hostRect at line 9359; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

9312: function handleRichEditorContextMenu(event) {
9313:   const editor = richEditorFromContextEvent(event);
9314:   const textBlock = event.target.closest?(".rich-text-block");
9315:
9316:   if (!editor && !textBlock) return;
9317:
9318:   event.preventDefault();
9319:   event.stopPropagation();
9320:   const nextEditor = editor || textBlock.closest("[data-rich-editor]");
9321:   if (activeSummaryEditor && nextEditor && activeSummaryEditor !== nextEditor) {
9322:     activeRichSelectionRange = null;
9323:     activeTextSelectionRange = null;
9324:     clearPersistentTextSelection(true);
9325:     summaryContextMenu.hidden = true;
9326:   }
9327:   hideTextSelectionInspector();
9328:
9329:   activePlainTextControl = null;
9330:   activePlainTextSelection = null;
9331:   activeSummaryEditor = nextEditor;
9332:   configureSummaryContextMenu("rich", { textOnly: activeSummaryEditor.dataset.richTextOnly === "true"
});
9333:   const liveRange = selectionRangeInsideEditor(activeSummaryEditor);
9334:   const savedRange = activeTextSelectionRange && !activeTextSelectionRange.collapsed
9335:     ? activeTextSelectionRange.cloneRange()
9336:     : null;
9337:   const savedContainer = savedRange?.commonAncestorContainer?.nodeType === Node.TEXT_NODE
9338:     ? savedRange.commonAncestorContainer.parentElement
9339:     : savedRange?.commonAncestorContainer;
9340:   if (liveRange && !liveRange.collapsed) {
9341:     activeRichSelectionRange = liveRange.cloneRange();
9342:     activeTextSelectionRange = liveRange.cloneRange();
9343:   } else if (savedRange && savedContainer && activeSummaryEditor.contains(savedContainer)) {
9344:     activeRichSelectionRange = savedRange.cloneRange();
9345:     restoreRichSelection(activeSummaryEditor);
9346:   } else {
9347:     captureRichSelection(activeSummaryEditor);
9348:   }
9349:   selectRichBlock(event.target.closest?(".rich-block") || currentRichBlock(activeSummaryEditor));
9350:   syncRichContextMenuControls(activeSummaryBlock);
9351:
9352:   const menuHost = activeSummaryEditor.closest("dialog") || document.body;
9353:   if (summaryContextMenu.parentElement !== menuHost) menuHost.append(summaryContextMenu);
9354:
9355:   summaryContextMenu.style.left = `${event.clientX}px`;
9356:   summaryContextMenu.style.top = `${event.clientY}px`;
9357:   summaryContextMenu.style.maxHeight = "";
9358:   summaryContextMenu.hidden = false;
9359:   const hostRect = menuHost.getBoundingClientRect();
9360:   const bounds = {
9361:     bottom: Math.min(window.innerHeight - 8, hostRect.bottom - 8),
9362:     left: Math.max(8, hostRect.left + 8),
9363:     right: Math.min(window.innerWidth - 8, hostRect.right - 8),

```

```

9364:         top: Math.max(8, hostRect.top + 8)
9365:     };
9366:     summaryContextMenu.style.maxHeight = `${Math.max(180, bounds.bottom - bounds.top)}px`;
9367:     const menuRect = summaryContextMenu.getBoundingClientRect();
9368:     const left = Math.max(bounds.left, Math.min(event.clientX, bounds.right - menuRect.width));
9369:     const top = Math.max(bounds.top, Math.min(event.clientY, bounds.bottom - menuRect.height));
9370:     summaryContextMenu.style.left = `${left}px`;
9371:     summaryContextMenu.style.top = `${top}px`;
9372: }

```

handlePlainTextContextMenu (template-preview.js, lines 9398-9433)

handlePlainTextContextMenu handles an event, request, command, or user action. In this file, it appears around lines 9398-9433 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references plainTextControlFromContextEvent, preventDefault, hideTextSelectionInspector, configureSummaryContextMenu, syncPlainTextMenuControls, closest, append, getBoundingClientRect, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9400: if (!control) return;.

Important local values include control at line 9399; menuHost at line 9414; hostRect at line 9420; bounds at line 9421; menuRect at line 9428; left at line 9429; top at line 9430. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

9398: function handlePlainTextContextMenu(event) {
9399:     const control = plainTextControlFromContextEvent(event);
9400:     if (!control) return;
9401:
9402:     event.preventDefault();
9403:     hideTextSelectionInspector();
9404:     activeSummaryEditor = null;
9405:     activeSummaryBlock = null;
9406:     activePlainTextControl = control;
9407:     activePlainTextSelection = {
9408:         start: control.selectionStart ?? 0,
9409:         end: control.selectionEnd ?? control.value.length
9410:     };
9411:     configureSummaryContextMenu("plain");
9412:     syncPlainTextMenuControls(control);
9413:
9414:     const menuHost = control.closest("dialog") || document.body;
9415:     if (summaryContextMenu.parentElement !== menuHost) menuHost.append(summaryContextMenu);
9416:     summaryContextMenu.style.left = `${event.clientX}px`;
9417:     summaryContextMenu.style.top = `${event.clientY}px`;
9418:     summaryContextMenu.style.maxHeight = "";
9419:     summaryContextMenu.hidden = false;
9420:     const hostRect = menuHost.getBoundingClientRect();
9421:     const bounds = {
9422:         bottom: Math.min(window.innerHeight - 8, hostRect.bottom - 8),
9423:         left: Math.max(8, hostRect.left + 8),
9424:         right: Math.min(window.innerWidth - 8, hostRect.right - 8),
9425:         top: Math.max(8, hostRect.top + 8)
9426:     };
9427:     summaryContextMenu.style.maxHeight = `${Math.max(140, bounds.bottom - bounds.top)}px`;
9428:     const menuRect = summaryContextMenu.getBoundingClientRect();
9429:     const left = Math.max(bounds.left, Math.min(event.clientX, bounds.right - menuRect.width));
9430:     const top = Math.max(bounds.top, Math.min(event.clientY, bounds.bottom - menuRect.height));
9431:     summaryContextMenu.style.left = `${left}px`;
9432:     summaryContextMenu.style.top = `${top}px`;
9433: }

```

handleRichEditorPaste (template-preview.js, lines 9437-9480)

This function controls paste behavior inside rich editors. It decides whether clipboard content is an image, plain text, HTML, code, or formula-like text, then inserts it using the builder's block model.

Pasting is where editors often become messy. This function keeps pasted content usable while preventing links from being misread as formulas and images from landing in broken positions.

It calls or references closest, find, startsWith, preventDefault, setStatus, uploadSummaryImageFile, getAsFile, insertRichBlockAtCursor, createRichImageBlock, saveRichEditorToProject, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 9439: if (!editor) return;; line 9447: return;; line 9463: return;; line 9471: return;. There are 1 additional return statements in the function body.

Important local values include editor at line 9438; imageItem at line 9441; imageBlock at line 9450; pastedHtml at line 9466; pastedText at line 9474. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9437: async function handleRichEditorPaste(event) {
9438:   const editor = event.target.closest("[data-rich-editor]");
9439:   if (!editor) return;
9440:
9441:   const imageItem = [...(event.clipboardData?.items || [])].find((item) =>
item.type.startsWith("image/"));
9442:
9443:   if (imageItem) {
9444:     event.preventDefault();
9445:     if (editor.dataset.richTextOnly === "true") {
9446:       setStatus("Images can only be pasted into overview editors, not title, profile, or contact
fields.");
9447:       return;
9448:     }
9449:
9450:     const imageBlock = await uploadSummaryImageFile(imageItem.getAsFile(), {
9451:       align: "center",
9452:       display: "show",
9453:       folder: editor.dataset.richFolder || "summary",
9454:       projectId: editor.dataset.richProjectId || "",
9455:       title: ""
9456:     });
9457:
9458:     if (imageBlock) {
9459:       insertRichBlockAtCursor(editor, createRichImageBlock(imageBlock));
9460:       saveRichEditorToProject(editor);
9461:     }
9462:
9463:     return;
9464:   }
9465:
9466:   const pastedHtml = event.clipboardData?.getData("text/html") || "";
9467:   if (pastedHtml) {
9468:     event.preventDefault();
9469:     insertHtmlIntoRichEditor(editor, pastedHtml);
9470:     saveRichEditorToProject(editor);
9471:     return;
9472:   }
9473:
9474:   const pastedText = event.clipboardData?.getData("text/plain") || "";
9475:   if (!pastedText) return;
9476:
9477:   event.preventDefault();
9478:   insertPlainTextIntoRichEditor(editor, pastedText);
9479:   saveRichEditorToProject(editor);
9480: }
```

restoreRichInsertionPoint (template-preview.js, lines 9494-9503)

restoreRichInsertionPoint part of the private builder frontend and editing experience. In this file, it appears around lines 9494-9503 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references restoreRichSelection, focusTextBlock, selectedRichBlock, querySelector, focus, selectionRangeInsideEditor, and deleteContents. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 9499: if (!range) return null;; line 9502: return range;.

Important local values include range at line 9498. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9494: function restoreRichInsertionPoint(editor) {
9495:   if (!restoreRichSelection(editor)) focusTextBlock(selectedRichBlock(editor) ||
editor.querySelector(".rich-text-block"));
9496:   editor.focus({ preventScroll: true });
9497:
9498:   const range = selectionRangeInsideEditor(editor);
9499:   if (!range) return null;
9500:
9501:   range.deleteContents();
9502:   return range;
9503: }
```

insertFragmentIntoRichEditor (template-preview.js, lines 9505-9521)

insertFragmentIntoRichEditor part of the private builder frontend and editing experience. In this file, it appears around lines 9505-9521 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references restoreRichInsertionPoint, insertNode, createRange, setStartAfter, setStart, collapse, getSelection, removeAllRanges, addRange, and captureRichSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9507: if (!range || !fragment) return;.

Important local values include range at line 9506; lastInsertedNode at line 9509; nextRange at line 9512; selection at line 9517. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9505: function insertFragmentIntoRichEditor(editor, fragment) {
9506:   const range = restoreRichInsertionPoint(editor);
9507:   if (!range || !fragment) return;
9508:
9509:   const lastInsertedNode = fragment.lastChild;
9510:   range.insertNode(fragment);
9511:
9512:   const nextRange = document.createRange();
9513:   if (lastInsertedNode) nextRange.setStartAfter(lastInsertedNode);
9514:   else nextRange.setStart(range.startContainer, range.startOffset);
9515:   nextRange.collapse(true);
9516:
9517:   const selection = window.getSelection();
9518:   selection.removeAllRanges();
9519:   selection.addRange(nextRange);
9520:   captureRichSelection(editor);
9521: }
```

insertPlainTextIntoRichEditor (template-preview.js, lines 9523-9533)

insertPlainTextIntoRichEditor part of the private builder frontend and editing experience. In this file, it appears around lines 9523-9533 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, createDocumentFragment, split, forEach, append, createElement, createTextNode, and insertFragmentIntoRichEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9525: if (!textToInsert) return;.

Important local values include textToInsert at line 9524; fragment at line 9527. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9523: function insertPlainTextIntoRichEditor(editor, pastedText) {
9524:   const textToInsert = String(pastedText || "").replace(/\r\n?/g, "\n");
9525:   if (!textToInsert) return;
9526:
9527:   const fragment = document.createDocumentFragment();
9528:   textToInsert.split("\n").forEach((line, index) => {
9529:     if (index > 0) fragment.append(document.createElement("br"));
9530:     if (line) fragment.append(document.createTextNode(line));
9531:   });
9532:   insertFragmentIntoRichEditor(editor, fragment);
9533: }
```

insertHtmlIntoRichEditor (template-preview.js, lines 9535-9543)

insertHtmlIntoRichEditor part of the private builder frontend and editing experience. In this file, it appears around lines 9535-9543 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, createElement, sanitizeRichInlineHtml, insertFragmentIntoRichEditor, and cloneNode. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include `normalizedHtml` at line 9536; `template` at line 9540. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9535: function insertHtmlIntoRichEditor(editor, pastedHtml) {
9536:     const normalizedHtml = String(pastedHtml || "")
9537:     .replace(/<\/(?:p|div|li|h[1-6]|tr)>/gi, "<br>")
9538:     .replace(/<(?:p|div|li|h[1-6]|tr|td|th)[^>]*>/gi, "")
9539:     .replace(/<br\s*\/*>\s*<br\s*\/*>\s*{2,}/gi, "<br><br>");
9540:     const template = document.createElement("template");
9541:     template.innerHTML = sanitizeRichInlineHtml(normalizedHtml);
9542:     insertFragmentIntoRichEditor(editor, template.content.cloneNode(true));
9543: }
```

handleRichEditorKeydown (template-preview.js, lines 9565-9610)

`handleRichEditorKeydown` handles an event, request, command, or user action. In this file, it appears around lines 9565-9610 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `template-preview.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `closest`, `toLowerCase`, `preventDefault`, `applyRichInlineCommand`, `execCommand`, `captureRichSelection`, `selectionRangeInsideEditor`, `contains`, `deleteContents`, and 13 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 9567: `if (!editor) return;;` line 9577: `return;;` line 9580: `if (event.key !== "Enter") return;;` line 9583: `if (!block) return;.` There are 2 additional return statements in the function body.

Important local values include `editor` at line 9566; `command` at line 9571; `block` at line 9582; `range` at line 9592; `tail` at line 9595; `trailingContent` at line 9598; `nextBlock` at line 9599; `nextRange` at line 9602; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9565: function handleRichEditorKeydown(event) {
9566:     const editor = event.target.closest("[data-rich-editor]");
9567:     if (!editor) return;
9568:
9569:     if ((event.ctrlKey || event.metaKey) && ["b", "i", "u"].includes(event.key.toLowerCase())) {
9570:         event.preventDefault();
9571:         const command = event.key.toLowerCase() === "b"
9572:             ? "bold"
9573:             : event.key.toLowerCase() === "i"
9574:             ? "italic"
9575:             : "underline";
9576:         applyRichInlineCommand(editor, command);
9577:         return;
9578:     }
9579:
9580:     if (event.key !== "Enter") return;
9581:
9582:     const block = event.target.closest(".rich-text-block");
9583:     if (!block) return;
9584:
9585:     event.preventDefault();
9586:     if (event.shiftKey) {
9587:         document.execCommand("insertLineBreak", false);
9588:         captureRichSelection(editor);
9589:         return;
9590:     }
9591:
9592:     const range = selectionRangeInsideEditor(editor);
9593:     if (!range || !block.contains(range.startContainer)) return;
9594:     range.deleteContents();
9595:     const tail = document.createRange();
9596:     tail.setStart(range.startContainer, range.startOffset);
9597:     tail.setEnd(block, block.childNodes.length);
9598:     const trailingContent = tail.extractContents();
9599:     const nextBlock = matchingTextBlock(block, "");
9600:     nextBlock.append(trailingContent);
9601:     block.after(nextBlock);
9602:     const nextRange = document.createRange();
9603:     nextRange.selectNodeContents(nextBlock);
9604:     nextRange.collapse(true);
9605:     const selection = window.getSelection();
9606:     selection.removeAllRanges();
9607:     selection.addRange(nextRange);
9608:     captureRichSelection(editor);
9609:     setStatus("Unsaved local changes.");
9610: }
```

handleRichEditorFocus (template-preview.js, lines 9632-9637)

handleRichEditorFocus handles an event, request, command, or user action. In this file, it appears around lines 9632-9637 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, execCommand, and captureRichSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9634: if (!editor) return;.

Important local values include editor at line 9633. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9632: function handleRichEditorFocus(event) {
9633:   const editor = event.target.closest("[data-rich-editor]");
9634:   if (!editor) return;
9635:   document.execCommand("defaultParagraphSeparator", false, "p");
9636:   captureRichSelection(editor);
9637: }
```

handleRichEditorInput (template-preview.js, lines 9639-9644)

handleRichEditorInput handles an event, request, command, or user action. In this file, it appears around lines 9639-9644 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, captureRichSelection, and setStatus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9641: if (!editor) return;.

Important local values include editor at line 9640. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9639: function handleRichEditorInput(event) {
9640:   const editor = event.target.closest("[data-rich-editor]");
9641:   if (!editor) return;
9642:   captureRichSelection(editor);
9643:   setStatus("Unsaved local changes.");
9644: }
```

handleRichEditorBeforeInput (template-preview.js, lines 9646-9653)

handleRichEditorBeforeInput handles an event, request, command, or user action. In this file, it appears around lines 9646-9653 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and clearPersistentTextSelection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9648: if (!editor) return;.

Important local values include editor at line 9647; inputType at line 9649. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9646: function handleRichEditorBeforeInput(event) {
9647:   const editor = event.target.closest("[data-rich-editor]");
9648:   if (!editor) return;
9649:   const inputType = event.inputType || "";
9650:   if (!/insertFromPaste|insertFromDrop/i.test(inputType) && /^(insert|delete|history)/i.test(inputType))
{
9651:     clearPersistentTextSelection(true);
9652:   }
9653: }
```

finishTextSelectionGesture (template-preview.js, lines 9728-9753)

finishTextSelectionGesture part of the private builder frontend and editing experience. In this file, it appears around lines 9728-9753 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSelection, selectionRangeInsideEditor, cloneRange, showPersistentTextSelection, rangeBelongsToEditor, clearPersistentTextSelection, and refreshTextSelectionInspector. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9729: if (event?.button === 2 || event?.target?.closest?("#text-selection-inspector, #summary-context-menu")) return;.

Important local values include selection at line 9733; node at line 9734; editor at line 9736; range at line 9737. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9728: function finishTextSelectionGesture(event) {
9729:   if (event?.button === 2 || event?.target?.closest?("#text-selection-inspector, #summary-context-
menu")) return;
9730:   selectionGestureActive = false;
9731:   clearTimeout(selectionGestureFinishTimer);
9732:   selectionGestureFinishTimer = setTimeout(() => {
9733:     const selection = window.getSelection();
9734:     let node = selection?.anchorNode;
9735:     if (node?.nodeType === Node.TEXT_NODE) node = node.parentElement;
9736:     const editor = node?.closest?("[data-rich-editor]");
9737:     const range = editor ? selectionRangeInsideEditor(editor) : null;
9738:     if (editor && range && !range.collapsed) {
9739:       activeSummaryEditor = editor;
9740:       activeRichSelectionRange = range.cloneRange();
9741:       activeTextSelectionRange = range.cloneRange();
9742:       selectionGestureProducedRange = true;
9743:       showPersistentTextSelection(range);
9744:     } else if (!selectionGestureProducedRange) {
9745:       if (editor && rangeBelongsToEditor(activeTextSelectionRange, editor)) {
9746:         showPersistentTextSelection(activeTextSelectionRange);
9747:       } else {
9748:         clearPersistentTextSelection(true);
9749:       }
9750:     }
9751:     refreshTextSelectionInspector();
9752:   }, 40);
9753: }
```

endSelectionInspectorDrag (template-preview.js, lines 9814-9819)

endSelectionInspectorDrag part of the private builder frontend and editing experience. In this file, it appears around lines 9814-9819 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9815: if (!activeSelectionInspectorDrag) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
9814: function endSelectionInspectorDrag() {
9815:   if (!activeSelectionInspectorDrag) return;
9816:   activeSelectionInspectorDrag.target?.releasePointerCapture?.(activeSelectionInspectorDrag.pointerId);
9817:   textSelectionInspector?.classList.remove("is-dragging");
9818:   activeSelectionInspectorDrag = null;
9819: }
```

handleSelectionInspectorTextField (template-preview.js, lines 9843-9849)

handleSelectionInspectorTextField handles an event, request, command, or user action. In this file, it appears around lines 9843-9849 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references preventDefault, and applySelectionInspectorFormat. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9844: if (event.type === "keydown" && event.key !== "Enter") return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
9843: function handleSelectionInspectorTextField(event) {
9844:   if (event.type === "keydown" && event.key !== "Enter") return;
9845:   if (event.type === "keydown") event.preventDefault();
9846:   if (event.target === selectionFontInput) applySelectionInspectorFormat("font", event.target.value);
9847:   if (event.target === selectionColorInput) applySelectionInspectorFormat("color", event.target.value);
9848:   if (event.target === selectionSizeInput) applySelectionInspectorFormat("size", event.target.value);
9849: }
```

scheduleSelectionInspectorTextField (template-preview.js, lines 9851-9859)

scheduleSelectionInspectorTextField part of the private builder frontend and editing experience. In this file, it appears around lines 9851-9859 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, applySelectionInspectorFormat, normalizeTextColor, and normalizeFontPx. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include target at line 9853. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9851: function scheduleSelectionInspectorTextField(event) {
9852:   clearTimeout(selectionInspectorInputTimer);
9853:   const target = event.target;
9854:   selectionInspectorInputTimer = setTimeout(() => {
9855:     if (target === selectionFontInput && cleanFontFamily(target.value))
applySelectionInspectorFormat("font", target.value);
9856:     if (target === selectionColorInput && normalizeTextColor(target.value))
applySelectionInspectorFormat("color", target.value);
9857:     if (target === selectionSizeInput && normalizeFontPx(target.value))
applySelectionInspectorFormat("size", target.value);
9858:   }, 350);
9859: }
```

focusAdjacentImageText (template-preview.js, lines 9871-9882)

focusAdjacentImageText part of the private builder frontend and editing experience. In this file, it appears around lines 9871-9882 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, matches, matchingTextBlock, before, after, focusTextBlock, and saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9873: if (!editor) return;.

Important local values include editor at line 9872; sibling at line 9874; textBlock at line 9875. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9871: function focusAdjacentImageText(block, direction) {
9872:   const editor = block?.closest("[data-rich-editor]");
9873:   if (!editor) return;
9874:   const sibling = direction === "before" ? block.previousElementSibling : block.nextElementSibling;
9875:   const textBlock = sibling?.matches(".rich-text-block") ? sibling : matchingTextBlock(block, "");
9876:   if (textBlock !== sibling) {
9877:     if (direction === "before") block.before(textBlock);
9878:     else block.after(textBlock);
9879:   }
9880:   focusTextBlock(textBlock);
9881:   saveRichEditorToProject(editor);
9882: }
```

handleRichCaretClick (template-preview.js, lines 9884-9889)

handleRichCaretClick handles an event, request, command, or user action. In this file, it appears around lines 9884-9889 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and focusAdjacentImageText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 9886: if (!caretButton) return false;; line 9888: return true;.

Important local values include caretButton at line 9885. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9884: function handleRichCaretClick(event) {
9885:   const caretButton = event.target.closest("[data-rich-caret]");
9886:   if (!caretButton) return false;
9887:   focusAdjacentImageText(caretButton.closest(".rich-image-block"), caretButton.dataset.richCaret);
9888:   return true;
9889: }
```

handleRichDragStart (template-preview.js, lines 9891-9910)

handleRichDragStart handles an event, request, command, or user action. In this file, it appears around lines 9891-9910 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, preventDefault, selectRichBlock, add, clearData, and setData. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9898: return;.

Important local values include handle at line 9892; imageBlock at line 9893; isControl at line 9894; block at line 9895. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9891: function handleRichDragStart(event) {
9892:   const handle = event.target.closest("[data-rich-drag-handle]");
9893:   const imageBlock = event.target.closest(".rich-image-block");
9894:   const isControl = event.target.closest("button, input, select, textarea, a, .rich-block-actions, [data-rich-caret]");
9895:   const block = handle?.closest(".rich-block") || (!isControl ? imageBlock : null);
9896:   if (!block) {
9897:     if (imageBlock) event.preventDefault();
9898:     return;
9899:   }
9900:   activeRichDragBlock = block;
9901:   selectRichBlock(activeRichDragBlock);
9902:   activeRichDragBlock.classList.add("is-dragging");
9903:   if (event.dataTransfer) {
9904:     event.dataTransfer.clearData();
9905:     event.dataTransfer.effectAllowed = "move";
9906:     event.dataTransfer.dropEffect = "move";
9907:     event.dataTransfer.setData("text/x-rich-block", "move");
9908:     event.dataTransfer.setData("text/plain", activeRichDragBlock.dataset.title || activeRichDragBlock.dataset.type || "image");
9909:   }
9910: }
```

handleRichDragOver (template-preview.js, lines 9912-9919)

handleRichDragOver handles an event, request, command, or user action. In this file, it appears around lines 9912-9919 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, preventDefault, and updateRichDropMarker. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 9913: if (!activeRichDragBlock) return;; line 9915: if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;.

Important local values include editor at line 9914. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9912: function handleRichDragOver(event) {
9913:   if (!activeRichDragBlock) return;
9914:   const editor = event.target.closest("[data-rich-editor]");
9915:   if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;
9916:   event.preventDefault();
9917:   if (event.dataTransfer) event.dataTransfer.dropEffect = "move";
9918:   updateRichDropMarker(editor, activeRichDragBlock, event.clientX, event.clientY, event.target);
9919: }
```

richRangeFromPoint (template-preview.js, lines 9921-9933)

richRangeFromPoint part of the private builder frontend and editing experience. In this file, it appears around lines 9921-9933 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references caretRangeFromPoint, caretPositionFromPoint, createRange, setStart, and collapse. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 9922: if (document.caretRangeFromPoint) return document.caretRangeFromPoint(x, y);; line 9929: return range;; line 9932: return null;.

Important local values include position at line 9924; range at line 9926. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9921: function richRangeFromPoint(x, y) {
9922:   if (document.caretRangeFromPoint) return document.caretRangeFromPoint(x, y);
9923:   if (document.caretPositionFromPoint) {
9924:     const position = document.caretPositionFromPoint(x, y);
9925:     if (position) {
9926:       const range = document.createRange();
9927:       range.setStart(position.offsetNode, position.offset);
9928:       range.collapse(true);
9929:       return range;
9930:     }
9931:   }
9932:   return null;
9933: }
```

richDropLocation (template-preview.js, lines 9935-9950)

richDropLocation part of the private builder frontend and editing experience. In this file, it appears around lines 9935-9950 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references validDropRangeForEditor, richRangeFromPoint, contains, and elementFromPoint. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9943: return {.

Important local values include previousPointerEvents at line 9936; dropRange at line 9939; pointTarget at line 9940. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9935: function richDropLocation(editor, movingBlock, x, y, eventTarget = null) {
9936:   const previousPointerEvents = movingBlock?.style.pointerEvents || "";
9937:   if (movingBlock) movingBlock.style.pointerEvents = "none";
9938:   try {
9939:     const dropRange = validDropRangeForEditor(editor, movingBlock, richRangeFromPoint(x, y));
9940:     const pointTarget = eventTarget && !movingBlock?.contains(eventTarget)
9941:       ? eventTarget
9942:       : document.elementFromPoint(x, y);
9943:     return {
9944:       dropRange,
9945:       target: pointTarget?.closest?(".rich-block") || null
9946:     };
9947:   } finally {
9948:     if (movingBlock) movingBlock.style.pointerEvents = previousPointerEvents;
9949:   }
9950: }
```

validDropRangeForEditor (template-preview.js, lines 9952-9960)

validDropRangeForEditor part of the private builder frontend and editing experience. In this file, it appears around lines 9952-9960 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, and closest. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 9956: if (!range || !rangeNode || !editor.contains(rangeNode)) return null;; line 9957: if (movingBlock?.contains(rangeNode)) return null;; line 9958: if (rangeNode.closest("[contenteditable='false']")) return null;; line 9959: return range;.

Important local values include rangeNode at line 9953. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9952: function validDropRangeForEditor(editor, movingBlock, range) {
9953:   const rangeNode = range?.startContainer?.nodeType === Node.TEXT_NODE
9954:     ? range.startContainer.parentElement
9955:     : range?.startContainer;
9956:   if (!range || !rangeNode || !editor.contains(rangeNode)) return null;
9957:   if (movingBlock?.contains(rangeNode)) return null;
9958:   if (rangeNode.closest("[contenteditable='false']")) return null;
9959:   return range;
9960: }
```

ensureRichDropMarker (template-preview.js, lines 9962-9971)

ensureRichDropMarker part of the private builder frontend and editing experience. In this file, it appears around lines 9962-9971 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, setAttribute, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 9970: return activeRichDropMarker;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
9962: function ensureRichDropMarker() {
9963:   if (!activeRichDropMarker) {
9964:     activeRichDropMarker = document.createElement("div");
9965:     activeRichDropMarker.className = "rich-drop-marker";
9966:     activeRichDropMarker.setAttribute("aria-hidden", "true");
9967:     document.body.append(activeRichDropMarker);
9968:   }
9969:   activeRichDropMarker.hidden = false;
9970:   return activeRichDropMarker;
9971: }
```

hideRichDropMarker (template-preview.js, lines 9973-9975)

hideRichDropMarker controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 9973-9975 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
9973: function hideRichDropMarker() {
9974:   if (activeRichDropMarker) activeRichDropMarker.hidden = true;
9975: }
```

updateRichDropMarker (template-preview.js, lines 9977-10006)

updateRichDropMarker updates ui, state, cache, or stored data. In this file, it appears around lines 9977-10006 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureRichDropMarker, richDropLocation, getClientRects, getBoundingClientRect, max, min, and contains. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 9978: if (!editor || !movingBlock) return;; line 9993: return;.

Important local values include marker at line 9979; rangeRect at line 9982; anchor at line 9983; anchorRect at line 9986; rect at line 9987; targetRect at line 9996; imageRect at line 9999; beforeTarget at line 10000. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
9977: function updateRichDropMarker(editor, movingBlock, x, y, eventTarget = null) {
9978:   if (!editor || !movingBlock) return;
9979:   const marker = ensureRichDropMarker();
9980:   const { dropRange, target } = richDropLocation(editor, movingBlock, x, y, eventTarget);
9981:   if (dropRange) {
9982:     const rangeRect = dropRange.getClientRects()[0];
9983:     const anchor = dropRange.startContainer.nodeType === Node.TEXT_NODE
9984:       ? dropRange.startContainer.parentElement
9985:       : dropRange.startContainer;
9986:     const anchorRect = anchor?.getBoundingClientRect?();
9987:     const rect = rangeRect || anchorRect || editor.getBoundingClientRect();
9988:     marker.className = "rich-drop-marker caret";
9989:     marker.style.left = `${Math.max(rect.left, Math.min(x, rect.right || rect.left))}px`;
9990:     marker.style.top = `${rect.top}px`;
9991:     marker.style.width = "2px";
9992:     marker.style.height = `${Math.max(22, rect.height || 22)}px`;
9993:     return;
9994:   }
9995:
9996:   const targetRect = (target && editor.contains(target) && target !== movingBlock)
9997:     ? target.getBoundingClientRect()
9998:     : editor.getBoundingClientRect();
9999:   const imageRect = movingBlock.getBoundingClientRect();
10000:   const beforeTarget = !target || y < targetRect.top + targetRect.height / 2;
10001:   marker.className = "rich-drop-marker block";
10002:   marker.style.left = `${targetRect.left}px`;
10003:   marker.style.top = `${beforeTarget ? targetRect.top : targetRect.bottom}px`;
10004:   marker.style.width = `${Math.min(targetRect.width, Math.max(160, imageRect.width ||
targetRect.width))}px`;
10005:   marker.style.height = "3px";
10006: }
```

moveRichBlockToPoint (template-preview.js, lines 10008-10029)

moveRichBlockToPoint part of the private builder frontend and editing experience. In this file, it appears around lines 10008-10029 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richDropLocation, remove, insertRichBlockAtCursor, contains, append, getBoundingClientRect, before, after, cleanupEmptyRichTextBlocks, focusAdjacentImageText, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 10009: if (!editor || !movingBlock) return;; line 10013: if (target === movingBlock) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
10008: function moveRichBlockToPoint(editor, movingBlock, x, y, eventTarget = null, options = {}) {
10009:   if (!editor || !movingBlock) return;
10010:   const { dropRange, target } = richDropLocation(editor, movingBlock, x, y, eventTarget);
10011:
10012:   movingBlock.classList.remove("is-dragging");
10013:   if (target === movingBlock) return;
10014:
10015:   if (dropRange) {
10016:     insertRichBlockAtCursor(editor, movingBlock, dropRange);
10017:   } else if (!target || !editor.contains(target)) {
10018:     editor.append(movingBlock);
10019:   } else if (y < target.getBoundingClientRect().top + target.getBoundingClientRect().height / 2) {
10020:     target.before(movingBlock);
10021:   } else {
```



```

10022:     target.after(movingBlock);
10023:   }
10024:
10025:   cleanupEmptyRichTextBlocks(editor);
10026:   if (options.focusAfter !== false) focusAdjacentImageText(movingBlock, "after");
10027:   if (options.commit !== false) saveRichEditorToProject(editor);
10028:   if (options.focusAfter === false) selectRichBlock(movingBlock);
10029: }

```

handleRichDrop (template-preview.js, lines 10031-10039)

handleRichDrop handles an event, request, command, or user action. In this file, it appears around lines 10031-10039 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, preventDefault, moveRichBlockToPoint, and hideRichDropMarker. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 10032: if (!activeRichDragBlock) return;; line 10034: if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;.

Important local values include editor at line 10033. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

10031: function handleRichDrop(event) {
10032:   if (!activeRichDragBlock) return;
10033:   const editor = event.target.closest("[data-rich-editor]");
10034:   if (!editor || editor !== activeRichDragBlock.closest("[data-rich-editor]")) return;
10035:   event.preventDefault();
10036:   moveRichBlockToPoint(editor, activeRichDragBlock, event.clientX, event.clientY, event.target);
10037:   hideRichDropMarker();
10038:   activeRichDragBlock = null;
10039: }

```

imageMoveDragControl (template-preview.js, lines 10041-10043)

imageMoveDragControl part of the private builder frontend and editing experience. In this file, it appears around lines 10041-10043 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10042: return target.closest("button, input, select, textarea, a, .rich-block-actions, [data-rich-caret]");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

10041: function imageMoveDragControl(target) {
10042:   return target.closest("button, input, select, textarea, a, .rich-block-actions, [data-rich-caret]");
10043: }

```

beginRichImageMoveDrag (template-preview.js, lines 10045-10061)

beginRichImageMoveDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10045-10061 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, and imageMoveDragControl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 10046: if (event.button !== 0) return;; line 10048: if (!block || imageMoveDragControl(event.target)) return;; line 10049: if (event.target.closest(".rich-image-viewport.crop-active")) return;; line 10051: if (!editor) return;.

Important local values include block at line 10047; editor at line 10050. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10045: function beginRichImageMoveDrag(event) {
10046:   if (event.button !== 0) return;
10047:   const block = event.target.closest(".rich-image-block");
10048:   if (!block || !imageMoveDragControl(event.target)) return;
10049:   if (event.target.closest(".rich-image-viewport.crop-active")) return;
10050:   const editor = block.closest("[data-rich-editor]");
10051:   if (!editor) return;
10052:   activeRichImageMoveDrag = {
10053:     block,
10054:     editor,
10055:     moved: false,
10056:     pointerId: event.pointerId,
10057:     startX: event.clientX,
10058:     startY: event.clientY,
10059:   };
10060:   block.setPointerCapture?.(event.pointerId);
10061: }
```

moveRichImageMoveDrag (template-preview.js, lines 10063-10077)

moveRichImageMoveDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10063-10077 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references hypot, selectRichBlock, add, updateRichDropMarker, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 10064: if (!activeRichImageMoveDrag) return;; line 10067: if (!drag.moved && distance < 6) return;.

Important local values include drag at line 10065; distance at line 10066. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10063: function moveRichImageMoveDrag(event) {
10064:   if (!activeRichImageMoveDrag) return;
10065:   const drag = activeRichImageMoveDrag;
10066:   const distance = Math.hypot(event.clientX - drag.startX, event.clientY - drag.startY);
10067:   if (!drag.moved && distance < 6) return;
10068:   if (!drag.moved) {
10069:     drag.moved = true;
10070:     activeRichDragBlock = drag.block;
10071:     selectRichBlock(drag.block);
10072:     drag.block.classList.add("is-dragging");
10073:     document.body.classList.add("rich-image-dragging");
10074:   }
10075:   updateRichDropMarker(drag.editor, drag.block, event.clientX, event.clientY, event.target);
10076:   event.preventDefault();
10077: }
```

endRichImageMoveDrag (template-preview.js, lines 10079-10092)

endRichImageMoveDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10079-10092 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references moveRichBlockToPoint, hideRichDropMarker, preventDefault, and remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10080: if (!activeRichImageMoveDrag) return;.

Important local values include drag at line 10081. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10079: function endRichImageMoveDrag(event) {
10080:   if (!activeRichImageMoveDrag) return;
10081:   const drag = activeRichImageMoveDrag;
10082:   drag.block.releasePointerCapture?.(drag.pointerId);
10083:   if (drag.moved) {
10084:     moveRichBlockToPoint(drag.editor, drag.block, event.clientX, event.clientY);
10085:     hideRichDropMarker();
10086:     event.preventDefault();
10087:   }
10088:   drag.block.classList.remove("is-dragging");
```

```

10089: document.body.classList.remove("rich-image-dragging");
10090: activeRichImageMoveDrag = null;
10091: activeRichDragBlock = null;
10092: }

```

beginImageCropDrag (template-preview.js, lines 10094-10109)

beginImageCropDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10094-10109 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, normalizeCropPosition, max, getBoundingClientRect, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10096: if (!viewport || event.button !== 0) return;.

Important local values include viewport at line 10095; block at line 10097. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

10094: function beginImageCropDrag(event) {
10095:   const viewport = event.target.closest(".rich-image-viewport.crop-active");
10096:   if (!viewport || event.button !== 0) return;
10097:   const block = viewport.closest(".rich-image-block");
10098:   activeImageCropDrag = {
10099:     block,
10100:     editor: block.closest("[data-rich-editor]"),
10101:     startX: event.clientX,
10102:     startY: event.clientY,
10103:     cropX: normalizeCropPosition(block.dataset.cropX),
10104:     cropY: normalizeCropPosition(block.dataset.cropY),
10105:     width: Math.max(1, viewport.getBoundingClientRect().width),
10106:     height: Math.max(1, viewport.getBoundingClientRect().height)
10107:   };
10108:   event.preventDefault();
10109: }

```

moveImageCropDrag (template-preview.js, lines 10111-10121)

moveImageCropDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10111-10121 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCropPosition, querySelector, and setProperty. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10112: if (!activeImageCropDrag) return;.

Important local values include nextX at line 10114; nextY at line 10115; viewport at line 10118. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

10111: function moveImageCropDrag(event) {
10112:   if (!activeImageCropDrag) return;
10113:   const { block, startX, startY, cropX, cropY, width, height } = activeImageCropDrag;
10114:   const nextX = normalizeCropPosition(cropX - ((event.clientX - startX) / width) * 100);
10115:   const nextY = normalizeCropPosition(cropY - ((event.clientY - startY) / height) * 100);
10116:   block.dataset.cropX = String(nextX);
10117:   block.dataset.cropY = String(nextY);
10118:   const viewport = block.querySelector(".rich-image-viewport");
10119:   viewport?.style.setProperty("--crop-x", `${nextX}%`);
10120:   viewport?.style.setProperty("--crop-y", `${nextY}%`);
10121: }

```

endImageCropDrag (template-preview.js, lines 10123-10127)

endImageCropDrag part of the private builder frontend and editing experience. In this file, it appears around lines 10123-10127 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10124: if (!activeImageCropDrag) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
10123: function endImageCropDrag() {  
10124:   if (!activeImageCropDrag) return;  
10125:   saveRichEditorToProject(activeImageCropDrag.editor);  
10126:   activeImageCropDrag = null;  
10127: }
```

handleStandaloneRichClick (template-preview.js, lines 10211-10226)

handleStandaloneRichClick handles an event, request, command, or user action. In this file, it appears around lines 10211-10226 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references handleRichCaretClick, closest, selectRichBlock, copyRichCodeBlock, editSelectedRichBlock, deleteSelectedRichBlock, and saveRichEditorToProject. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 10212: if (handleRichCaretClick(event)) return;; line 10216: if (!blockAction) return;; line 10221: return;.

Important local values include blockAction at line 10213; richBlock at line 10214; editor at line 10217. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10211: async function handleStandaloneRichClick(event) {  
10212:   if (handleRichCaretClick(event)) return;  
10213:   const blockAction = event.target.closest("[data-rich-block-action]");  
10214:   const richBlock = event.target.closest(".rich-block");  
10215:   if (richBlock) selectRichBlock(richBlock);  
10216:   if (!blockAction) return;  
10217:   const editor = blockAction.closest("[data-rich-editor]");  
10218:   selectRichBlock(blockAction.closest(".rich-block"));  
10219:   if (blockAction.dataset.richBlockAction === "copy-code") {  
10220:     await copyRichCodeBlock(blockAction.closest(".rich-block"));  
10221:     return;  
10222:   }  
10223:   if (blockAction.dataset.richBlockAction === "edit") await editSelectedRichBlock(editor);  
10224:   if (blockAction.dataset.richBlockAction === "delete") deleteSelectedRichBlock(editor);  
10225:   saveRichEditorToProject(editor);  
10226: }
```

hasDataset (template-preview.js, lines 10323-10327)

hasDataset part of the private builder frontend and editing experience. In this file, it appears around lines 10323-10327 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so template-preview.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references call, and addCustomSection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 10326: return;.

Important local values include hasDataset at line 10323. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
10323: const hasDataset = (key) => Object.prototype.hasOwnProperty.call(button.dataset, key);  
10324: if (button.dataset.addSection) {  
10325:   await addCustomSection();  
10326:   return;  
10327: }
```

Representative opening snippet

```
1: const builderGuideOpen = document.querySelector("#builder-guide-open");  
2: const builderGuideDialog = document.querySelector("#builder-guide-dialog");  
3: const builderGuideClose = document.querySelector("#builder-guide-close");  
4: const builderGuideSearch = document.querySelector("#builder-guide-search");  
5: const builderGuideResults = document.querySelector("#builder-guide-results");
```

```

6: const builderGuideSections = document.querySelector("#builder-guide-sections");
7: const builderGuideTopicDialog = document.querySelector("#builder-guide-topic-dialog");
8: const builderGuideTopicTitle = document.querySelector("#builder-guide-topic-title");
9: const builderGuideTopicBody = document.querySelector("#builder-guide-topic-body");
10: const builderGuideTopicClose = document.querySelector("#builder-guide-topic-close");
11: const templateSelect = document.querySelector("#template-select");
12: const newCategorySelect = document.querySelector("#new-category-select");
13: const placementSelect = document.querySelector("#placement-select");
14: const newProjectTitle = document.querySelector("#new-project-title");
15: const addProjectButton = document.querySelector("#add-project");
16: const addCategoryButton = document.querySelector("#add-category");
17: const addSiteSectionButton = document.querySelector("#add-site-section");
18: const funFactsInput = document.querySelector("#fun-facts-input");
19: const funFactsCount = document.querySelector("#fun-facts-count");
20: const saveFunFactsButton = document.querySelector("#save-fun-facts");
21: const clearFunFactsButton = document.querySelector("#clear-fun-facts");
22: const siteHeroEyebrowInput = document.querySelector("#site-hero-eyebrow");
23: const siteHeroTitleInput = document.querySelector("#site-hero-title");
24: const siteHeroCopyInput = document.querySelector("#site-hero-copy");
25: const saveSiteContentButton = document.querySelector("#save-site-content");
26: const profileDisplayNameInput = document.querySelector("#profile-display-name");
27: const profilePortfolioLabelInput = document.querySelector("#profile-portfolio-label");
28: const profileContactIntroInput = document.querySelector("#profile-contact-intro");
29: const profileEmailInput = document.querySelector("#profile-email");
30: const profilePhoneInput = document.querySelector("#profile-phone");
31: const profileGithubInput = document.querySelector("#profile-github");
32: const profileLinkedinInput = document.querySelector("#profile-linkedin");
33: const profileWebsiteInput = document.querySelector("#profile-website");
34: const saveProfileContentButton = document.querySelector("#save-profile-content");
35: const profileImageUploadButton = document.querySelector("#profile-image-upload");
36: const profileHeroUploadButton = document.querySelector("#profile-hero-upload");
37: const profileResumeUploadButton = document.querySelector("#profile-resume-upload");
38: const profileBrandUploadButton = document.querySelector("#profile-brand-upload");
39: const profileAssetStatus = document.querySelector("#profile-asset-status");
40: const categoryDropdown = document.querySelector("#category-dropdown");
41: const siteSectionList = document.querySelector("#site-section-list");
42: const projectCreateDialog = document.querySelector("#project-create-dialog");
43: const projectCreateForm = document.querySelector("#project-create-form");
44: const projectCreateCategory = document.querySelector("#project-create-category");
45: const projectCreateCancel = document.querySelector("#project-create-cancel");
46: const categoryDialog = document.querySelector("#category-dialog");
47: const categoryForm = document.querySelector("#category-form");
48: const categoryTitle = document.querySelector("#category-title");
49: const categoryDescription = document.querySelector("#category-description");
50: const categoryAccent = document.querySelector("#category-accent");
51: const categoryCancel = document.querySelector("#category-cancel");
52: const projectTree = document.querySelector("#project-tree");
53: const projectSearchInput = document.querySelector("#project-search");
54: const projectSearchClear = document.querySelector("#project-search-clear");
55: const projectSearchStatus = document.querySelector("#project-search-status");
56: const projectDialog = document.querySelector("#project-dialog");
57: const projectWindowTitle = document.querySelector("#project-window-title");
58: const projectWindowClose = document.querySelector("#project-window-close");
59: const projectWindowDelete = document.querySelector("#project-window-delete");
60: const viewProjectPreviewButton = document.querySelector("#view-project-preview");
61: const saveProjectButton = document.querySelector("#save-project");
62: const saveProjectCloseButton = document.querySelector("#save-project-close");
63: const projectTitleMenu = document.querySelector("#project-title-menu");
64: const titleRenameActions = document.querySelector("#title-rename-actions");
65: const titleRenameSave = document.querySelector("#title-rename-save");
66: const titleRenameCancel = document.querySelector("#title-rename-cancel");
67: const projectMetaDialog = document.querySelector("#project-meta-dialog");
68: const projectMetaTitle = document.querySelector("#project-meta-title");
69: const projectMetaGrid = document.querySelector("#project-meta-grid");
70: const projectMetaClose = document.querySelector("#project-meta-close");
71: const projectFields = document.querySelector("#project-fields");
72: const sectionTabs = document.querySelector("#section-tabs");

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: template-preview.html

Item	Explanation
File	template-preview.html
Category	Website/builder UI source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	92,731 bytes
Extension	.html
MIME type	text/html
Text lines	1,349

Why this file exists

- Builder HTML shell loaded inside the Electron app and local preview runtime.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Installer at line 11	Windows setup, update, shortcut, or installation behavior. Evidence: <link rel="shortcut icon" href="assets/favicon.ico" />
Rich text at line 64	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Rich text at line 79	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Rich text at line 94	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Rich text at line 118	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Rich text at line 133	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Rich text at line 148	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Rich text at line 163	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Rich text at line 178	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Network request at line 192	Frontend-backend communication or public web/API calls. Evidence: data-placeholder="https://github.com/username"
Rich text at line 193	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Network request at line 207	Frontend-backend communication or public web/API calls. Evidence: data-placeholder="https://linkedin.com/in/username"
Rich text at line 208	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Network request at line 222	Frontend-backend communication or public web/API calls. Evidence: data-placeholder="https://example.com"
Rich text at line 223	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Rich text at line 259	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: contenteditable="true"
Browser DOM at line 391	Builder or website interface behavior in the browser/Electron renderer. Evidence: Resume should be a public-safe PDF, not a private editable document.

Item	Explanation
Installer at line 420	Windows setup, update, shortcut, or installation behavior. Evidence: All project creation starts from the main Add project control. This keeps project creation consistent and prevents hidden category-specific add buttons from creating projects with incomplete metadata.
Security or auth at line 432	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Only categories that contain projects show project rows. Empty categories stay as clean category headers until a project is added.
Rich text at line 462	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <p>Rich text fields behave like document editors. Click where you want to type, highlight the exact words you want to edit, then apply formatting to that selection. Formatting should not change an entire paragraph unless
Rich text at line 463	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <p>The builder stores the text as structured content so the portfolio can render it cleanly later. That means the editor may standardize pasted formatting, but it should preserve intentional choices such as bold text, fo
Rich text at line 464	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <p>Front-page, profile, contact, fun facts, and project overview fields now use the same selection-level text behavior. Highlight the exact words you want to edit, then use the right-click menu or floating selection insp
Rich text at line 466	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: Right-click highlighted text in any builder text field to copy, cut, paste, select all, change font, change size, change color, and apply bold, italic, or underline.
Rich text at line 467	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: The default typing font is Arial.
Rich text at line 468	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: If selected text has mixed font, size, or color, the inspector fields are blank so you know the selection is mixed.
Rich text at line 469	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: Use Enter for a new paragraph. Pasted line breaks remain line breaks.
Rich text at line 470	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: Typing and wrapping should not create fake new paragraphs; only hard enters and pasted line breaks do.
Rich text at line 471	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: Use the Code block command or Paste as code for source snippets so they render differently from general text.
Rich text at line 477	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: Keep color changes readable on the selected project appearance.
Rich text at line 478	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: If a pasted paragraph looks strange, clear the formatting and reapply only the formatting you need.
Rich text at line 480	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <p>If the selection menu disappears, click back inside the rich editor and highlight the text again. Before saving, read the section in the project preview to make sure line breaks, lists, and emphasized words still look
Rich text at line 484	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <summary>6. Links, formulas, code, and pasted content</summary>
Cloudflare or AI at line 487	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <p>Use this section when pasting content from ChatGPT, documentation, GitHub, LinkedIn, LTspice notes, equations, source code, or lab references. The builder tries to recognize the content, but you should still check the
Compiler or simulator at line 491	Part of the Compile Code workspace or language/tool detection. Evidence: Use Code block or Paste as code for C, C++, SystemVerilog, LTspice, Java, JavaScript, Python, and HTML snippets.
Rich text at line 492	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: Choose Basic paste when you want the builder to clean extra trailing whitespace, or Keep source spacing when indentation is important.
Compiler or simulator at line 495	Part of the Compile Code workspace or language/tool detection. Evidence: For Verilog and SystemVerilog, use Simulate to run the simulator. Simulation requires a Testbench file; plain Compile can still check or build HDL design files.
Security or auth at line 496	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Use Add testbench to create a starter testbench with \$dumpfile and \$dumpvars. Those waveform calls feed the signal scope, where HDL signals can be viewed over time.
Network request at line 503	Frontend-backend communication or public web/API calls. Evidence: Use full links when possible: https://github.com/user/repo is clearer than only github.com.
Compiler or simulator at line 507	Part of the Compile Code workspace or language/tool detection. Evidence: Use Compile Code to test C, C++, Java, JavaScript, Python, HTML checks, and any installed Verilog, SystemVerilog, or LTspice command-line tools before deciding what to publish later.
Security or auth at line 508	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Use Show output for terminal logs and Show scope for HDL timing waveforms after a successful testbench simulation.

Item	Explanation
Rich text at line 510	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: If a link is accidentally treated as a formula, remove the formula block and paste it again as normal text or link content.
Cloudflare or AI at line 512	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <p>When pasting from ChatGPT, review formulas and code before saving. If the content is a link, it should render as a clickable link, not as a formula. If the content is source code, it should render inside a dark code b
Security or auth at line 519	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <p>Images are best for visual evidence: schematics, PCB layouts, test benches, oscilloscope captures, simulation plots, block diagrams, timing diagrams, microscope photos, assembled boards, or setup photos. Files are bes
Rich text at line 521	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: Paste an image into a rich editor to place it below the cursor line.
Rich text at line 522	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: Use Add image when you need source selection, rename, caption, alignment, crop, or visible/download-only control.
Compiler or simulator at line 535	Part of the Compile Code workspace or language/tool detection. Evidence: <p>Project evidence is saved under docs/project-folder/section-or-subsection/filename. For example, a SystemVerilog design file could be saved as docs/laser-link/design/filter.sv. The upload status shows the generated pa
Security or auth at line 581	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <summary>10. Publishing target and authentication</summary>
Security or auth at line 583	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <p>Publishing target controls where Apply to site can push. A target is saved only after write access is verified, because the builder should never publish to a website unless the current user has permission to update th
Security or auth at line 584	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <p>This protects your portfolio from accidental pushes and also lets another user install the builder for their own site without getting access to your website. The target describes where the public portfolio will live;
Security or auth at line 588	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Click Authenticate target to verify write access for that repository and branch.
Git command at line 589	Repository state, publishing, branch checks, or release automation. Evidence: Complete GitHub/Git Credential Manager sign-in only when prompted.
Security or auth at line 590	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Use Load from target only after authentication succeeds and only when you want to import that website repository into this builder.
Security or auth at line 595	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Authenticate target does not import content; Load from target does not authenticate a new target.
Git command at line 596	Repository state, publishing, branch checks, or release automation. Evidence: If authentication fails, read the message before retrying; it usually explains whether Git, credentials, branch state, or repository access caused the failure.
Security or auth at line 598	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <p>GitHub authorization is cached per repository and branch. Daily reuse avoids repeated prompts, and repeated successful authorization can extend trust for that exact target. If the target changes, authenticate again be
Git command at line 605	Repository state, publishing, branch checks, or release automation. Evidence: The builder can be used offline for most editing work. Publishing is the only part that needs the network and a Git target with write access.
Git command at line 611	Repository state, publishing, branch checks, or release automation. Evidence: If offline, keep editing and saving drafts. Publish when the network and Git target are available.
Security or auth at line 617	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Use Apply to site only after the preview looks correct and the target is authenticated.
Installer at line 625	Windows setup, update, shortcut, or installation behavior. Evidence: <p>Use Check updates to manually look for a newer builder. Updates come from GitHub Releases after changes are merged into the release branch and a new installer is built. Use the Preferences menu to switch between light
Installer at line 629	Windows setup, update, shortcut, or installation behavior. Evidence: The app downloads the installer, closes the current builder, updates the existing install, and reopens the app.
Installer at line 631	Windows setup, update, shortcut, or installation behavior. Evidence: If an old uninstaller fails, setup stops the builder process and continues updating the same folder.
Installer at line 636	Windows setup, update, shortcut, or installation behavior. Evidence: If a release is paused, wait for the next version instead of forcing the old installer.
Installer at line 637	Windows setup, update, shortcut, or installation behavior. Evidence: If Windows blocks a process from closing, use Task Manager as Administrator to end the stuck installer or old builder process.
Installer at line 638	Windows setup, update, shortcut, or installation behavior. Evidence: If the app opens but looks stale, close it fully and reopen it from the desktop shortcut.

Item	Explanation
Installer at line 640	Windows setup, update, shortcut, or installation behavior. Evidence: <p>If the app cannot update, close any old OMB Builder process in Task Manager and reopen the app from the desktop shortcut. Avoid installing another copy in a different folder; duplicate copies can make Windows shortcut
Browser DOM at line 646	Builder or website interface behavior in the browser/Electron renderer. Evidence: <p>Anything pushed to the portfolio repository can become public. Treat the portfolio as a public engineering document. If a file would be unsafe in a public GitHub repository, it should not be uploaded or published thro
Security or auth at line 647	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <p>Good portfolio evidence is public-safe, clearly labeled, and useful to a visitor. Private credentials, private lab data, personal identity documents, and unpublished confidential work should stay out of the portfolio.
Security or auth at line 649	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Do not upload private keys, passwords, private tokens, or private lab data.
Security or auth at line 652	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Keep authentication local. The builder should not commit credentials.
Security or auth at line 657	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Check code files for tokens, API keys, Wi-Fi credentials, private comments, or machine-specific paths.
Cloudflare or AI at line 660	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: Use Security report to review app download counts, current publishing authorization status, and whether visitor logging requires Cloudflare analytics.
Security or auth at line 662	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <p>If another user installs the builder, they should connect their own target unless you explicitly authorize access to your target. The builder should help people create portfolios, but it should not give anyone silent
Git command at line 669	Repository state, publishing, branch checks, or release automation. Evidence: The routine is intentionally repetitive because publishing is safer when every project has been parsed and every section has been previewed before Git pushes anything.
Security or auth at line 684	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: For publishing target changes, authenticate before loading or applying site data.
Rich text at line 755	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: Accent color
Rich text at line 756	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <input id="category-accent" type="color" value="#1677a8" />
Rich text at line 780	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <button type="button" data-title-action="paste">Paste as title</button>
Git command at line 868	Repository state, publishing, branch checks, or release automation. Evidence: <input id="publish-target-repository" type="text" placeholder="https://github.com/USERNAME/USERNAME.github.io.git" />
Security or auth at line 871	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Authenticate target verifies that this computer can write to the selected GitHub Pages repository. Load from target imports the latest compatible website data from that authenticated repository into this builder. Authent
Security or auth at line 874	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <button class="button primary" id="publish-target-auth" type="submit">Authenticate target</button>

Representative opening snippet

```

1: <!DOCTYPE html>
2: <html lang="en">
3:   <head>
4:     <meta charset="UTF-8" />
5:     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6:     <meta name="robots" content="noindex, nofollow" />
7:     <title>Local Portfolio Builder</title>
8:     <link rel="icon" type="image/png" sizes="16x16" href="assets/favicon-16.png" />
9:     <link rel="icon" type="image/png" sizes="32x32" href="assets/favicon-32.png" />
10:    <link rel="icon" type="image/png" sizes="192x192" href="assets/favicon-192.png" />
11:    <link rel="shortcut icon" href="assets/favicon.ico" />
12:    <link rel="apple-touch-icon" sizes="180x180" href="assets/apple-touch-icon.png" />
13:    <link rel="stylesheet" href="styles.css" />
14:    <link rel="stylesheet" href="template-preview.css" />
15:  </head>
16:  <body class="template-preview-page">
17:    <header class="template-header">
18:      <a class="brand" href="index.html#top" aria-label="Back to portfolio">
19:        <span class="brand-lockup" aria-hidden="true">
20:          
21:          <span class="omb-engraving">OMB</span>
22:        </span>
23:      </span>
24:      <strong>Local Portfolio Builder</strong>
25:      <small>Build, preview, save locally, then publish when ready</small>

```

```

26:         </span>
27:     </a>
28:     <div class="builder-header-actions">
29:         <span class="builder-save-state" id="builder-save-state" data-state="loaded">Loaded</span>
30:         <a class="button secondary" href="index.html#projects" target="_blank"
rel="noreferrer">Portfolio</a>
31:         <button class="button secondary" id="publish-target-open" type="button">Publishing
target</button>
32:         <button class="button secondary" id="security-report-open" type="button">Security
report</button>
33:         <button class="button secondary" id="app-update-check" type="button">Check updates</button>
34:         <button class="button primary" id="save-draft" type="button">Save draft</button>
35:         <button class="button secondary" id="apply-catalog" type="button">Apply to site</button>
36:     </div>
37: </header>
38:
39: <main class="builder-workspace">
40:     <aside class="builder-sidebar" aria-labelledby="library-title">
41:         <button class="builder-guide-panel" id="builder-guide-open" type="button" aria-label="Open
builder guide">
42:             <span class="builder-guide-icon" aria-hidden="true"></span>
43:             <span>
44:                 <strong id="library-title">Portfolio Builder</strong>
45:                 <small>Click for a quick build guide</small>
46:             </span>
47:         </button>
48:
49:         <div class="builder-panel site-section-panel">
50:             <div class="panel-title-row">
51:                 <h2>Front page text</h2>
52:                 <button class="button secondary compact-button" id="save-site-content"
type="button">Save</button>
53:             </div>
54:             <div class="front-page-text-fields">
55:                 <label>
56:                     <span>Small label</span>
57:                     <div
58:                         class="rich-summary-editor front-rich-field"
59:                         id="site-hero-eyebrow"
60:                         data-rich-editor="site-field"
61:                         data-site-field="heroEyebrow"
62:                         data-rich-text-only="true"
63:                         data-placeholder="Engineering portfolio"
64:                         contenteditable="true"
65:                         spellcheck="true"
66:                         role="textbox"
67:                         aria-label="Small front page label"
68:                     ></div>
69:                 </label>
70:                 <label>
71:                     <span>Main headline</span>
72:                     <div

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: template-preview.css

Item	Explanation
File	template-preview.css
Category	Website/builder UI source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	131,453 bytes
Extension	.css
MIME type	text/css
Text lines	5,749

Why this file exists

- Builder application CSS for dialogs, windows, rich editors, compile code, dark mode, guide windows, and local previews.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Rich text at line 34	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #16384a;
Rich text at line 36	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 37	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;
Rich text at line 43	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #f5c56d;
Rich text at line 44	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #5f4207;
Rich text at line 50	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #93c5fd;
Rich text at line 51	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #164780;
Rich text at line 57	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #8eefac;
Rich text at line 58	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #14532d;
Rich text at line 63	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #fca5a5;
Rich text at line 64	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #7f1d1d;
Rich text at line 70	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #d6effa;
Rich text at line 71	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #16384a;
Rich text at line 74	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color 140ms ease,
Rich text at line 75	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: background-color 140ms ease,
Rich text at line 76	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color 140ms ease,
Rich text at line 83	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #b8e6f7;
Rich text at line 84	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #16384a;
Rich text at line 91	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #ffffff;
Rich text at line 92	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #16384a;

Item	Explanation
Rich text at line 148	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 149	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 1.18rem;
Rich text at line 173	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);
Rich text at line 174	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.72rem;
Rich text at line 175	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;
Rich text at line 184	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 186	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font: inherit;
Rich text at line 187	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 700;
Rich text at line 191	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #4b91db;
Rich text at line 198	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);
Rich text at line 199	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 205	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: inherit;
Rich text at line 222	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 223	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.96rem;
Rich text at line 241	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #16384a;
Rich text at line 243	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.75rem;
Rich text at line 244	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;
Rich text at line 266	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #16384a;
Rich text at line 278	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 279	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 1rem;
Rich text at line 283	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);
Rich text at line 284	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 292	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border: 2px solid currentColor;
Rich text at line 304	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: background: currentColor;
Rich text at line 309	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: box-shadow: 0 7px 0 currentColor;
Rich text at line 338	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #16384a;
Rich text at line 340	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 341	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;
Rich text at line 356	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);
Rich text at line 357	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 358	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;
Rich text at line 366	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 368	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font: inherit;
Rich text at line 369	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.9rem;
Rich text at line 378	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #16384a;
Rich text at line 380	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font: inherit;
Rich text at line 381	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.8rem;
Rich text at line 382	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;
Rich text at line 388	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #ffffff;
Rich text at line 390	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #1f6ed4;
Rich text at line 395	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);

Item	Explanation
Rich text at line 396	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.82rem;
Rich text at line 433	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 435	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.92rem;
Rich text at line 436	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 950;
Rich text at line 456	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #087f9b;
Rich text at line 457	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 950;
Rich text at line 471	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #16384a;
Rich text at line 473	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.7rem;
Rich text at line 474	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 950;
Rich text at line 490	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #ffffff;
Rich text at line 496	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #ffffff;
Rich text at line 500	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: rgba(255, 255, 255, 0.7);
Rich text at line 542	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);
Rich text at line 543	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.86rem;
Rich text at line 555	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #2b5064;
Rich text at line 556	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.86rem;
Rich text at line 582	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #fafdff;
Rich text at line 583	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 601	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);

CSS selectors and visual hooks

Item	Explanation
Line 1	.template-preview-page
Line 5	.template-header
Line 20	.section-actions
Line 27	.builder-save-state
Line 42	.builder-save-state[data-state="dirty"]
Line 49	.builder-save-state[data-state="publishing"]
Line 56	.builder-save-state[data-state="published"]
Line 62	.builder-save-state[data-state="error"]
Line 69	.template-preview-page button
Line 82	.template-preview-page button:hover
Line 90	.template-preview-page button:active
Line 98	.builder-workspace
Line 107	.builder-inspector
Line 116	.template-preview
Line 123	.builder-panel
Line 129	.project-library-heading
Line 137	.project-library-heading > div
Line 143	.project-library-heading h2

Item	Explanation
Line 147	.project-library-heading h2
Line 153	.project-library-tools
Line 165	.project-search-field
Line 172	.workflow-card span
Line 179	.project-search-field input
Line 190	.project-search-field input:focus
Line 196	#project-search-status
Line 203	.tree-category-toggle mark
Line 210	.workflow-card
Line 219	.workflow-card strong
Line 228	.workflow-actions
Line 234	.workflow-metrics span
Line 247	.workflow-actions .button
Line 253	.template-preview > .section-heading
Line 257	.builder-guide-panel
Line 273	.builder-guide-panel small
Line 277	.builder-guide-panel strong
Line 282	.builder-guide-panel small
Line 287	.builder-guide-icon
Line 297	.builder-guide-icon::after
Line 307	.builder-guide-icon::before
Line 312	.builder-guide-icon::after
Line 318	.builder-guide-dialog-content
Line 328	.builder-guide-actions
Line 334	.builder-guide-live span
Line 344	.builder-guide-tools
Line 353	.builder-guide-tools label
Line 361	.builder-guide-tools input
Line 373	.builder-guide-actions button
Line 387	.builder-guide-actions button:hover
Line 393	.builder-guide-results
Line 399	.builder-guide-sections
Line 416	.builder-guide-section
Line 424	.builder-guide-section[hidden]
Line 428	.builder-guide-section summary
Line 444	.builder-guide-section summary::-webkit-details-marker
Line 448	.builder-guide-section summary::before
Line 462	.builder-guide-section summary::after
Line 480	.builder-guide-section[open] summary
Line 485	.builder-guide-section[open] summary::before
Line 489	.builder-guide-section summary:hover

Item	Explanation
Line 495	.builder-guide-section summary: hover::after
Line 499	.builder-guide-section summary: hover::after
Line 504	.builder-guide-section > div
Line 513	.builder-guide-topic-dialog
Line 517	.builder-guide-topic-content
Line 522	.builder-guide-topic-body
Line 534	.builder-guide-topic-copy
Line 540	.builder-guide-topic-body p
Line 550	.builder-guide-topic-body ul
Line 561	.builder-guide-topic-body li
Line 565	.publish-result-box
Line 570	.publish-result-box p
Line 574	.publish-result-box pre
Line 588	.publish-target-form
Line 595	.publish-target-form label
Line 600	.publish-target-field-note
Line 606	.publish-target-form input
Line 616	.publish-target-current
Line 627	.publish-target-current div
Line 633	.publish-target-current strong
Line 637	.publish-target-current span
Line 641	.publish-target-readiness
Line 648	.publish-target-readiness strong
Line 653	.publish-target-readiness ul
Line 661	.publish-target-readiness li.ready
Line 665	.publish-target-readiness li.blocked
Line 669	.publish-target-tool-actions
Line 675	.publish-target-note
Line 682	.profile-setup-fields
Line 687	.profile-setup-fields label: first-child
Line 691	.profile-asset-actions
Line 697	.profile-asset-status
Line 704	.app-update-dialog
Line 708	.security-report-dialog
Line 712	.app-update-content
Line 717	.security-report-body
Line 727	.security-report-body p
Line 734	.app-update-body p: first-child
Line 739	.security-report-grid
Line 745	.security-report-card
Line 756	.security-report-card h3

Item	Explanation
Line 763	.security-report-card strong
Line 769	.security-report-card ul
Line 779	.security-report-assets
Line 787	.security-report-assets li
Line 797	.security-report-assets li:last-child
Line 801	.security-report-assets span:first-child
Line 808	.app-update-release-meta
Line 816	.app-update-dialog-actions
Line 820	.app-update-dialog-actions [hidden]
Line 824	.portfolio-preview-button
Line 834	.portfolio-preview-icon
Line 842	.portfolio-preview-icon::before
Line 854	.portfolio-preview-icon::after
Line 867	.save-all-icon
Line 876	.save-all-icon::after
Line 881	.save-all-icon::before
Line 890	.save-all-icon::after
Line 902	.builder-panel p
Line 906	.builder-panel h1
Line 911	.builder-panel h2

Representative opening snippet

```

1: .template-preview-page {
2:   background: #eef8fc;
3: }
4:
5: .template-header {
6:   position: sticky;
7:   top: 0;
8:   z-index: 20;
9:   display: flex;
10:  align-items: center;
11:  justify-content: space-between;
12:  gap: 18px;
13:  padding: 12px clamp(18px, 4vw, 56px);
14:  border-bottom: 1px solid rgba(201, 222, 234, 0.84);
15:  background: rgba(214, 239, 250, 0.96);
16:  backdrop-filter: blur(16px);
17: }
18:
19: .builder-header-actions,
20: .section-actions {
21:   display: flex;
22:   flex-wrap: wrap;
23:   gap: 10px;
24:   align-items: center;
25: }
26:
27: .builder-save-state {
28:   display: inline-flex;
29:   align-items: center;
30:   min-height: 34px;
31:   padding: 0 12px;
32:   border: 1px solid #9bd4ea;
33:   border-radius: 999px;
34:   color: #16384a;
35:   background: #f5fbfe;
36:   font-size: 0.78rem;
37:   font-weight: 900;
38:   letter-spacing: 0.01em;
39:   white-space: nowrap;
40: }
41:

```



```
42: .builder-save-state[data-state="dirty"] {
43:   border-color: #f5c56d;
44:   color: #5f4207;
45:   background: #fff7dc;
46: }
47:
48: .builder-save-state[data-state="saving"],
49: .builder-save-state[data-state="publishing"] {
50:   border-color: #93c5fd;
51:   color: #164780;
52:   background: #dbeafe;
53: }
54:
55: .builder-save-state[data-state="saved"],
56: .builder-save-state[data-state="published"] {
57:   border-color: #86efac;
58:   color: #14532d;
59:   background: #dcfce7;
60: }
61:
62: .builder-save-state[data-state="error"] {
63:   border-color: #fca5a5;
64:   color: #7f1d1d;
65:   background: #fee2e2;
66: }
67:
68: .template-preview-page .button,
69: .template-preview-page button {
70:   border-color: #d6effa;
71:   color: #16384a;
72:   background: #d6effa;
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: script.js

Item	Explanation
File	script.js
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	158,556 bytes
Extension	.js
MIME type	application/javascript
Text lines	3,710

Why this file exists

- Public website JavaScript for navigation, portfolio rendering, search, AI UI, and interactive project windows.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

API endpoints mentioned or called

Item	Explanation
/api/portfolio-ai	Line 1492. Handles AI assistant questions.
/api/portfolio-ai	Line 1493. Handles AI assistant questions.

Important implementation signals

Item	Explanation
Browser DOM at line 1	Builder or website interface behavior in the browser/Electron renderer. Evidence: const grid = document.querySelector("#project-grid");
Browser DOM at line 2	Builder or website interface behavior in the browser/Electron renderer. Evidence: const searchInput = document.querySelector("#project-search");
Browser DOM at line 3	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectFilters = document.querySelector("#project-filters");
Browser DOM at line 4	Builder or website interface behavior in the browser/Electron renderer. Evidence: let filterButtons = [...document.querySelectorAll(".filter-button")];
Browser DOM at line 5	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectCount = document.querySelector("#project-count");
Browser DOM at line 6	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectTrackCount = document.querySelector("#project-track-count");
Browser DOM at line 7	Builder or website interface behavior in the browser/Electron renderer. Evidence: const projectTrackLabels = document.querySelector("#project-track-labels");
Browser DOM at line 8	Builder or website interface behavior in the browser/Electron renderer. Evidence: const year = document.querySelector("#year");
Browser DOM at line 9	Builder or website interface behavior in the browser/Electron renderer. Evidence: const funFactsCallout = document.querySelector("#fun-facts-callout");
Browser DOM at line 10	Builder or website interface behavior in the browser/Electron renderer. Evidence: const heroEyebrow = document.querySelector(".hero-content .eyebrow");

Item	Explanation
Browser DOM at line 11	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const heroTitle = document.querySelector("#hero-title");</code>
Browser DOM at line 12	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const heroCopy = document.querySelector(".hero-copy");</code>
Browser DOM at line 13	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const brandName = document.querySelector(".brand strong");</code>
Browser DOM at line 14	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const brandSubtitle = document.querySelector(".brand small");</code>
Browser DOM at line 15	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const brandIcon = document.querySelector(".brand-icon");</code>
Browser DOM at line 16	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const brandText = document.querySelector(".omb-engraving");</code>
Browser DOM at line 17	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const headerAvatar = document.querySelector(".header-avatar");</code>
Browser DOM at line 18	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const headerAvatarImage = document.querySelector(".header-avatar img");</code>
Browser DOM at line 19	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const heroImage = document.querySelector(".hero-image");</code>
Browser DOM at line 20	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const resumeSection = document.querySelector("#resume");</code>
Browser DOM at line 21	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactBand = document.querySelector("#contact");</code>
Browser DOM at line 22	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const profilePhoto = document.querySelector(".profile-photo");</code>
Browser DOM at line 23	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactTitle = document.querySelector("#contact-title");</code>
Browser DOM at line 24	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactIntro = document.querySelector(".contact-intro");</code>
Browser DOM at line 25	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactDetails = document.querySelector(".contact-details");</code>
Browser DOM at line 26	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const contactLinks = document.querySelector(".contact-links");</code>
Browser DOM at line 27	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const footerOwner = document.querySelector(".site-footer span");</code>
Browser DOM at line 29	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantForm = document.querySelector("#ai-assistant-form");</code>
Browser DOM at line 30	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantPanel = document.querySelector("#ask-ai");</code>
Browser DOM at line 31	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantInput = document.querySelector("#ai-assistant-input");</code>
Browser DOM at line 32	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantLog = document.querySelector("#ai-assistant-log");</code>
Browser DOM at line 33	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const aiAssistantStatus = document.querySelector("#ai-assistant-status");</code>
Cloudflare or AI at line 34	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>const aiClearChatButton = document.querySelector("#ai-clear-chat");</code>
Cloudflare or AI at line 57	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <code>let assistantChatHistory = [];</code>
Compiler or simulator at line 73	Part of the Compile Code workspace or language/tool detection. Evidence: <code>{ id: "verilog", label: "Verilog", aliases: ["verilog", "v"] },</code>
Compiler or simulator at line 74	Part of the Compile Code workspace or language/tool detection. Evidence: <code>{ id: "systemverilog", label: "SystemVerilog", aliases: ["systemverilog", "system verilog", "sv"] },</code>
Compiler or simulator at line 76	Part of the Compile Code workspace or language/tool detection. Evidence: <code>{ id: "java", label: "Java", aliases: ["java"] },</code>

Item	Explanation
Compiler or simulator at line 78	Part of the Compile Code workspace or language/tool detection. Evidence: { id: "python", label: "Python", aliases: ["python", "py"] },
Compiler or simulator at line 104	Part of the Compile Code workspace or language/tool detection. Evidence: "embedded-wireless-node": "appearance-deep-navy-cyan-click",
Browser DOM at line 175	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.add("is-draggable-dialog");
Browser DOM at line 184	Builder or website interface behavior in the browser/Electron renderer. Evidence: if (dialog.querySelector("[data-section-dialog-resize-handle]")) return;
Browser DOM at line 186	Builder or website interface behavior in the browser/Electron renderer. Evidence: const handle = document.createElement("div");
Browser DOM at line 188	Builder or website interface behavior in the browser/Electron renderer. Evidence: handle.dataset.sectionDialogResizeHandle = direction;
Browser DOM at line 197	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.add("is-resized-dialog");
Browser DOM at line 255	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.add("is-dragging-dialog");
Browser DOM at line 273	Builder or website interface behavior in the browser/Electron renderer. Evidence: activeSectionDialogDrag.dialog.classList.remove("is-dragging-dialog");
Browser DOM at line 281	Builder or website interface behavior in the browser/Electron renderer. Evidence: if (!handle !dialog?.open dialog.classList.contains("is-minimized-dialog")) return;
Browser DOM at line 285	Builder or website interface behavior in the browser/Electron renderer. Evidence: direction: handle.dataset.sectionDialogResizeHandle,
Browser DOM at line 293	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.add("is-resizing-dialog");
Browser DOM at line 309	Builder or website interface behavior in the browser/Electron renderer. Evidence: activeSectionDialogResize.dialog.classList.remove("is-resizing-dialog");
Browser DOM at line 314	Builder or website interface behavior in the browser/Electron renderer. Evidence: const minimized = dialog.classList.contains("is-minimized-dialog");
Browser DOM at line 315	Builder or website interface behavior in the browser/Electron renderer. Evidence: const button = dialog.querySelector(".section-view-minimize");
Browser DOM at line 324	Builder or website interface behavior in the browser/Electron renderer. Evidence: dialog.classList.toggle("is-minimized-dialog");
Browser DOM at line 331	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointerdown", beginSectionDialogResize);
Browser DOM at line 332	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointerdown", beginSectionDialogDrag);
Browser DOM at line 333	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointermove", moveSectionDialogResize);
Browser DOM at line 334	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointermove", moveSectionDialogDrag);
Browser DOM at line 335	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointerup", endSectionDialogResize);
Browser DOM at line 336	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointerup", endSectionDialogDrag);
Browser DOM at line 337	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointercancel", endSectionDialogResize);
Browser DOM at line 338	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("pointercancel", endSectionDialogDrag);
Browser DOM at line 339	Builder or website interface behavior in the browser/Electron renderer. Evidence: window.addEventListener("resize", () => {
Browser DOM at line 340	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.querySelectorAll(".section-view-dialog.is-draggable-dialog[open]").forEach((dialog) => {
Rich text at line 370	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: function normalizeCodePasteMode(value = "") {

Item	Explanation
Rich text at line 374	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: function normalizeCodeText(value = "", pasteMode = "source") {
Rich text at line 376	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/t/g, " ").replace(/s+\$g, "");
Compiler or simulator at line 400	Part of the Compile Code workspace or language/tool detection. Evidence: if (/b(always_ff always_comb always_latch logic interface covergroup assert s+property typedef s+enum class s+w+)\b/.test(code)) return "systemverilog";
Compiler or simulator at line 401	Part of the Compile Code workspace or language/tool detection. Evidence: if (/b(module endmodule always assign reg wire initial posedge negedge)\b/.test(code)) return "verilog";
Cloudflare or AI at line 402	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: if (/^s*\.(tran ac dc op model subckt ends param)\b/im.test(code) /bVw+ s+ w+ s+ w+ s+(?:DC SIN PULSE)?/i.test(code)) return "Itspice";
Compiler or simulator at line 403	Part of the Compile Code workspace or language/tool detection. Evidence: if (/b(def elif import s+w+ from s+w+ s+import self None True False)\b/.test(code)) return "python";
Compiler or simulator at line 404	Part of the Compile Code workspace or language/tool detection. Evidence: if (/b(public s+class private s+ protected s+ static s+void s+main System\.out)\b/.test(code)) return "java";
Security or auth at line 412	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function tokenizedCodeHtml(code = "", language = "javascript") {
Security or auth at line 414	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const tokens = [];
Security or auth at line 417	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const token = `@@CODE_TOKEN_\${tokens.length}@`;
Security or auth at line 418	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: tokens.push(`<span class="\${className}">\${match}</span>`);
Security or auth at line 419	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: return token;
Security or auth at line 424	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: protect(/<!--[\s\S]*?-->/g, "code-token-comment");
Security or auth at line 425	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: protect(/(<V?)([a-zA-Z0-9:-]+)([\s\S]?)(V?>)/g, "code-token-tag");
Compiler or simulator at line 427	Part of the Compile Code workspace or language/tool detection. Evidence: const commentPattern = ["python", "Itspice"].includes(normalizeCodeLanguage(language))
Security or auth at line 430	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: protect(commentPattern, "code-token-comment");

Important constants and variables

Item	Explanation
grid	Line 1: const grid = document.querySelector("#project-grid");
searchInput	Line 2: const searchInput = document.querySelector("#project-search");
projectFilters	Line 3: const projectFilters = document.querySelector("#project-filters");
filterButtons	Line 4: let filterButtons = [...document.querySelectorAll(".filter-button")];
projectCount	Line 5: const projectCount = document.querySelector("#project-count");
projectTrackCount	Line 6: const projectTrackCount = document.querySelector("#project-track-count");
projectTrackLabels	Line 7: const projectTrackLabels = document.querySelector("#project-track-labels");
year	Line 8: const year = document.querySelector("#year");
funFactsCallout	Line 9: const funFactsCallout = document.querySelector("#fun-facts-callout");
heroEyebrow	Line 10: const heroEyebrow = document.querySelector(".hero-content .eyebrow");
heroTitle	Line 11: const heroTitle = document.querySelector("#hero-title");
heroCopy	Line 12: const heroCopy = document.querySelector(".hero-copy");
brandName	Line 13: const brandName = document.querySelector(".brand strong");
brandSubtitle	Line 14: const brandSubtitle = document.querySelector(".brand small");

Item	Explanation
brandIcon	Line 15: const brandIcon = document.querySelector(".brand-icon");
brandText	Line 16: const brandText = document.querySelector(".omb-engraving");
headerAvatar	Line 17: const headerAvatar = document.querySelector(".header-avatar");
headerAvatarImage	Line 18: const headerAvatarImage = document.querySelector(".header-avatar img");
heroImage	Line 19: const heroImage = document.querySelector(".hero-image");
resumeSection	Line 20: const resumeSection = document.querySelector("#resume");
contactBand	Line 21: const contactBand = document.querySelector("#contact");
profilePhoto	Line 22: const profilePhoto = document.querySelector(".profile-photo");
contactTitle	Line 23: const contactTitle = document.querySelector("#contact-title");
contactIntro	Line 24: const contactIntro = document.querySelector(".contact-intro");
contactDetails	Line 25: const contactDetails = document.querySelector(".contact-details");
contactLinks	Line 26: const contactLinks = document.querySelector(".contact-links");
footerOwner	Line 27: const footerOwner = document.querySelector(".site-footer span");
searchWrap	Line 28: const searchWrap = searchInput?.closest(".search-wrap");
aiAssistantForm	Line 29: const aiAssistantForm = document.querySelector("#ai-assistant-form");
aiAssistantPanel	Line 30: const aiAssistantPanel = document.querySelector("#ask-ai");
aiAssistantInput	Line 31: const aiAssistantInput = document.querySelector("#ai-assistant-input");
aiAssistantLog	Line 32: const aiAssistantLog = document.querySelector("#ai-assistant-log");
aiAssistantStatus	Line 33: const aiAssistantStatus = document.querySelector("#ai-assistant-status");
aiClearChatButton	Line 34: const aiClearChatButton = document.querySelector("#ai-clear-chat");
categories	Line 36: let categories = [];
projects	Line 37: let projects = [];
siteSections	Line 38: let siteSections = [];
funFacts	Line 39: let funFacts = [];
funFactsRich	Line 40: let funFactsRich = null;
fieldStyles	Line 41: let fieldStyles = {};
siteContent	Line 42: let siteContent = null;
siteContentRich	Line 43: let siteContentRich = {};
profile	Line 44: let profile = null;
profileRich	Line 45: let profileRich = {};
activeFilter	Line 46: let activeFilter = "all";
activeSectionDialogDrag	Line 47: let activeSectionDialogDrag = null;
activeSectionDialogResize	Line 48: let activeSectionDialogResize = null;
sectionDialogDragEnabled	Line 49: let sectionDialogDragEnabled = false;
isApplyingSectionRoute	Line 50: let isApplyingSectionRoute = false;
searchPanel	Line 51: let searchPanel = null;
searchStatus	Line 52: let searchStatus = null;
searchLimitSelect	Line 53: let searchLimitSelect = null;
currentSearchResults	Line 54: let currentSearchResults = [];
assistantSourceCounter	Line 55: let assistantSourceCounter = 0;
assistantSourceMap	Line 56: let assistantSourceMap = new Map();

Item	Explanation
assistantChatHistory	Line 57: let assistantChatHistory = [];
searchableEntries	Line 58: let searchableEntries = [];
searchResultLimit	Line 59: let searchResultLimit = 10;
searchHighlightTimer	Line 60: let searchHighlightTimer = 0;
searchHighlightName	Line 61: const searchHighlightName = "portfolio-search-highlight";
sectionRouteKey	Line 63: const sectionRouteKey = "portfolio-section";
sectionRouteParams	Line 64: const sectionRouteParams = {
supportedCodeLanguages	Line 70: const supportedCodeLanguages = [
legacyTemplateSkins	Line 82: const legacyTemplateSkins = {
defaultSiteContent	Line 108: const defaultSiteContent = {
defaultProfile	Line 114: const defaultProfile = {
items	Line 137: const items = Array.isArray(value) ? value : String(value "").split(/\r?\n/);
facts	Line 146: const facts = normalizeFunFacts(funFacts);
hasRichFacts	Line 147: const hasRichFacts = Boolean(funFactsRich?.blocks?.length);
rect	Line 162: const rect = dialog.getBoundingClientRect();
margin	Line 163: const margin = 12;
maxLeft	Line 164: const maxLeft = Math.max(margin, window.innerWidth - rect.width - margin);
maxTop	Line 165: const maxTop = Math.max(margin, window.innerHeight - rect.height - margin);
rect	Line 173: const rect = dialog.getBoundingClientRect();
position	Line 174: const position = clampSectionDialogPosition(rect.left, rect.top, dialog);
handle	Line 186: const handle = document.createElement("div");
rect	Line 196: const rect = dialog.getBoundingClientRect();
margin	Line 204: const margin = 12;
minWidth	Line 205: const minWidth = Math.min(320, Math.max(180, window.innerWidth - margin * 2));
minHeight	Line 206: const minHeight = Math.min(170, Math.max(120, window.innerHeight - margin * 2));

JSON/YAML-style keys detected

Item	Explanation
skin-light-blue	Line 83: "skin-light-blue": "appearance-light-blue-red-click",
skin-clean-white	Line 84: "skin-clean-white": "appearance-white-blue-click",
skin-deep-navy	Line 85: "skin-deep-navy": "appearance-deep-navy-cyan-click",
skin-red-warm	Line 86: "skin-red-warm": "appearance-warm-red-pale-click",
skin-emerald-instrument	Line 87: "skin-emerald-instrument": "appearance-emerald-mint-click",
skin-amber-power	Line 88: "skin-amber-power": "appearance-amber-cream-click",
skin-violet-mixed	Line 89: "skin-violet-mixed": "appearance-violet-lilac-click",
skin-graphite-asic	Line 90: "skin-graphite-asic": "appearance-graphite-white-click",
analog-opamp-topology	Line 91: "analog-opamp-topology": "appearance-light-blue-red-click",
analog-power-charger	Line 92: "analog-power-charger": "appearance-amber-cream-click",
analog-filter-front-end	Line 93: "analog-filter-front-end": "appearance-light-blue-red-click",
analog-sensor-interface	Line 94: "analog-sensor-interface": "appearance-emerald-mint-click",
analog-mixed-signal-timing	Line 95: "analog-mixed-signal-timing": "appearance-violet-lilac-click",

Item	Explanation
digital-fpga-pipeline	Line 96: "digital-fpga-pipeline": "appearance-deep-navy-cyan-click",
digital-asic-block	Line 97: "digital-asic-block": "appearance-graphite-white-click",
digital-verification-suite	Line 98: "digital-verification-suite": "appearance-graphite-white-click",
digital-interface-controller	Line 99: "digital-interface-controller": "appearance-deep-navy-cyan-click",
digital-signal-processing	Line 100: "digital-signal-processing": "appearance-violet-lilac-click",
embedded-stm32-sensor	Line 101: "embedded-stm32-sensor": "appearance-emerald-mint-click",
embedded-rtos-control	Line 102: "embedded-rtos-control": "appearance-graphite-white-click",
embedded-power-monitor	Line 103: "embedded-power-monitor": "appearance-amber-cream-click",
embedded-wireless-node	Line 104: "embedded-wireless-node": "appearance-deep-navy-cyan-click",
embedded-motor-control	Line 105: "embedded-motor-control": "appearance-violet-lilac-click"
Brief	Line 792: "Brief": "Overview",
Project Brief	Line 793: "Project Brief": "Overview",
Design Brief	Line 794: "Design Brief": "Design Overview",
Simulation Brief	Line 795: "Simulation Brief": "Simulation Overview"
--responsive-section-card-min	Line 1029: "--responsive-section-card-min": profile.sectionCardMin,
--responsive-content-max	Line 1030: "--responsive-content-max": profile.contentMax,
--responsive-media-max	Line 1031: "--responsive-media-max": profile.mediaMax,
--responsive-touch-target	Line 1032: "--responsive-touch-target": profile.touchTarget
--accent	Line 1047: "--accent": accent,
--template-bg	Line 1048: "--template-bg": visual.background,
--template-panel	Line 1049: "--template-panel": visual.panel,
--template-accent	Line 1050: "--template-accent": visual.accent,
--template-hover	Line 1051: "--template-hover": visual.hover,
--template-click	Line 1052: "--template-click": visual.click,
--template-click-text	Line 1053: "--template-click-text": visual.clickText,
--template-line	Line 1054: "--template-line": visual.line,
--template-text	Line 1055: "--template-text": visual.text,
--accent	Line 1080: "--accent": accent,
--template-bg	Line 1081: "--template-bg": visual.background,
--template-panel	Line 1082: "--template-panel": visual.panel,
--template-accent	Line 1083: "--template-accent": visual.accent,
--template-hover	Line 1084: "--template-hover": visual.hover,
--template-click	Line 1085: "--template-click": visual.click,
--template-click-text	Line 1086: "--template-click-text": visual.clickText,
--template-line	Line 1087: "--template-line": visual.line,
--template-text	Line 1088: "--template-text": visual.text,

Event handlers and user interactions

Item	Explanation
Line 331	document.addEventListener("pointerdown", beginSectionDialogResize);
Line 332	document.addEventListener("pointerdown", beginSectionDialogDrag);

Item	Explanation
Line 333	<code>document.addEventListener("pointermove", moveSectionDialogResize);</code>
Line 334	<code>document.addEventListener("pointermove", moveSectionDialogDrag);</code>
Line 335	<code>document.addEventListener("pointerup", endSectionDialogResize);</code>
Line 336	<code>document.addEventListener("pointerup", endSectionDialogDrag);</code>
Line 337	<code>document.addEventListener("pointercancel", endSectionDialogResize);</code>
Line 338	<code>document.addEventListener("pointercancel", endSectionDialogDrag);</code>
Line 339	<code>window.addEventListener("resize", () => {</code>
Line 1214	<code>button.addEventListener("click", () => {</code>
Line 3272	<code>searchLimitSelect.addEventListener("change", () => {</code>
Line 3276	<code>searchPanel.addEventListener("mousedown", (event) => event.preventDefault());</code>
Line 3277	<code>searchPanel.addEventListener("click", (event) => {</code>
Line 3434	<code>dialog.querySelector(".section-view-close").addEventListener("click", () => closeSectionDialog(dialog));</code>
Line 3435	<code>dialog.addEventListener("cancel", () => clearSectionRouteInHistory());</code>
Line 3436	<code>dialog.addEventListener("close", () => {</code>
Line 3445	<code>dialog.querySelector(".section-view-back").addEventListener("click", () => {</code>
Line 3457	<code>dialog.querySelector(".section-view-forward").addEventListener("click", () => {</code>
Line 3464	<code>dialog.querySelector("#section-view-content").addEventListener("click", (event) => {</code>
Line 3562	<code>window.addEventListener("popstate", (event) => {</code>
Line 3567	<code>searchInput.addEventListener("input", handleSearchInput);</code>
Line 3568	<code>searchInput.addEventListener("focus", showSearchPanelIfNeeded);</code>
Line 3569	<code>searchInput.addEventListener("keydown", (event) => {</code>
Line 3580	<code>document.addEventListener("click", (event) => {</code>
Line 3586	<code>document.addEventListener("click", (event) => {</code>
Line 3599	<code>aiAssistantForm?.addEventListener("submit", (event) => {</code>
Line 3606	<code>aiClearChatButton?.addEventListener("click", clearAssistantChat);</code>
Line 3607	<code>window.addEventListener("resize", updateAssistantPanelGrowth);</code>
Line 3609	<code>aiAssistantLog?.addEventListener("click", (event) => {</code>
Line 3616	<code>grid.addEventListener("click", (event) => {</code>

Function-by-function detail

normalize (script.js, lines 132-134)

normalize validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 132-134 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 133: `return String(value || "").toLowerCase();`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

132: function normalize(value) {
133:   return String(value || "").toLowerCase();
134: }
```

normalizeFunFacts (script.js, lines 136-142)

normalizeFunFacts validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 136-142 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, split, map, replace, trim, filter, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 138: return items.

Important local values include items at line 137. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
136: function normalizeFunFacts(value) {
137:   const items = Array.isArray(value) ? value : String(value || "").split(/\r?\n/);
138:   return items
139:     .map((item) => String(item || "").replace(/\s+/g, " ").trim())
140:     .filter(Boolean)
141:     .slice(0, 10);
142: }
```

renderFunFacts (script.js, lines 144-159)

renderFunFacts renders ui, markup, or display output. In this file, it appears around lines 144-159 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeFunFacts, renderRichContent, slice, join, map, and renderInlineMath. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 145: if (!funFactsCallout) return;.

Important local values include facts at line 146; hasRichFacts at line 147. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
144: function renderFunFacts() {
145:   if (!funFactsCallout) return;
146:   const facts = normalizeFunFacts(funFacts);
147:   const hasRichFacts = Boolean(funFactsRich?.blocks?.length);
148:   funFactsCallout.hidden = !facts.length && !hasRichFacts;
149:   funFactsCallout.innerHTML = facts.length || hasRichFacts ? `
150:     <div class="fun-facts-window">
151:       <span class="fun-facts-label">Fun fact:</span>
152:       <div class="fun-facts-lines">
153:         ${hasRichFacts
154:           ? renderRichContent({ blocks: funFactsRich.blocks.slice(0, 3) }, facts.join("\n"))
155:           : facts.map((fact) => `<p class="fun-fact-line">${renderInlineMath(fact)}</p>`).join("")}
156:       </div>
157:     </div>
158:   ` : "";
159: }
```

clampSectionDialogPosition (script.js, lines 161-170)

clampSectionDialogPosition part of the public website runtime. In this file, it appears around lines 161-170 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, max, and min. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 166: return {.

Important local values include rect at line 162; margin at line 163; maxLeft at line 164; maxTop at line 165. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

161: function clampSectionDialogPosition(left, top, dialog) {
162:   const rect = dialog.getBoundingClientRect();
163:   const margin = 12;
164:   const maxLeft = Math.max(margin, window.innerWidth - rect.width - margin);
165:   const maxTop = Math.max(margin, window.innerHeight - rect.height - margin);
166:   return {
167:     left: Math.min(Math.max(left, margin), maxLeft),
168:     top: Math.min(Math.max(top, margin), maxTop)
169:   };
170: }

```

anchorSectionDialog (script.js, lines 172-181)

anchorSectionDialog part of the public website runtime. In this file, it appears around lines 172-181 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getBoundingClientRect, clampSectionDialogPosition, and add. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include rect at line 173; position at line 174. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

172: function anchorSectionDialog(dialog) {
173:   const rect = dialog.getBoundingClientRect();
174:   const position = clampSectionDialogPosition(rect.left, rect.top, dialog);
175:   dialog.classList.add("is-draggable-dialog");
176:   dialog.style.left = `${position.left}px`;
177:   dialog.style.top = `${position.top}px`;
178:   dialog.style.right = "auto";
179:   dialog.style.bottom = "auto";
180:   dialog.style.margin = "0";
181: }

```

ensureSectionResizeHandles (script.js, lines 183-192)

ensureSectionResizeHandles part of the public website runtime. In this file, it appears around lines 183-192 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, forEach, createElement, setAttribute, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 184: if (dialog.querySelector("[data-section-dialog-resize-handle]")) return;.

Important local values include handle at line 186. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

183: function ensureSectionResizeHandles(dialog) {
184:   if (dialog.querySelector("[data-section-dialog-resize-handle]")) return;
185:   ["n", "e", "s", "w", "ne", "nw", "se", "sw"].forEach((direction) => {
186:     const handle = document.createElement("div");
187:     handle.className = `dialog-resize-handle resize-${direction}`;
188:     handle.dataset.sectionDialogResizeHandle = direction;
189:     handle.setAttribute("aria-hidden", "true");
190:     dialog.append(handle);
191:   });
192: }

```

anchorSectionDialogForResize (script.js, lines 194-201)

anchorSectionDialogForResize part of the public website runtime. In this file, it appears around lines 194-201 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `anchorSectionDialog`, `getBoundingClientRect`, and `add`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include `rect` at line 196. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
194: function anchorSectionDialogForResize(dialog) {
195:   anchorSectionDialog(dialog);
196:   const rect = dialog.getBoundingClientRect();
197:   dialog.classList.add("is-resized-dialog");
198:   dialog.style.width = `${rect.width}px`;
199:   dialog.style.height = `${rect.height}px`;
200:   dialog.style.maxHeight = "none";
201: }
```

sectionDialogResizeBounds (script.js, lines 203-230)

`sectionDialogResizeBounds` part of the public website runtime. In this file, it appears around lines 203-230 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `min`, `max`, and `includes`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 229: `return { height, left, top, width };`.

Important local values include `margin` at line 204; `minWidth` at line 205; `minHeight` at line 206; `direction` at line 207; `dx` at line 209; `dy` at line 210; `right` at line 219; `bottom` at line 224. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
203: function sectionDialogResizeBounds(state, event) {
204:   const margin = 12;
205:   const minWidth = Math.min(320, Math.max(180, window.innerWidth - margin * 2));
206:   const minHeight = Math.min(170, Math.max(120, window.innerHeight - margin * 2));
207:   const direction = state.direction;
208:   let { left, top, width, height } = state.rect;
209:   const dx = event.clientX - state.startX;
210:   const dy = event.clientY - state.startY;
211:
212:   if (direction.includes("e")) {
213:     width = Math.min(Math.max(state.rect.width + dx, minWidth), window.innerWidth - left - margin);
214:   }
215:   if (direction.includes("s")) {
216:     height = Math.min(Math.max(state.rect.height + dy, minHeight), window.innerHeight - top - margin);
217:   }
218:   if (direction.includes("w")) {
219:     const right = state.rect.left + state.rect.width;
220:     left = Math.min(Math.max(state.rect.left + dx, margin), right - minWidth);
221:     width = right - left;
222:   }
223:   if (direction.includes("n")) {
224:     const bottom = state.rect.top + state.rect.height;
225:     top = Math.min(Math.max(state.rect.top + dy, margin), bottom - minHeight);
226:     height = bottom - top;
227:   }
228:
229:   return { height, left, top, width };
230: }
```

sectionDialogFromDragEvent (script.js, lines 232-239)

`sectionDialogFromDragEvent` part of the public website runtime. In this file, it appears around lines 232-239 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `closest`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 233: `if (event.target.closest("[data-section-dialog-resize-handle]")) return null;`; line 235: `if (!handle) return null;`; line 236: `if (event.target.closest("button, a, input, textarea, select, label, summary")) return null;`; line 238: `return dialog?.open ? dialog : null;`.

Important local values include handle at line 234; dialog at line 237. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
232: function sectionDialogFromDragEvent(event) {
233:   if (event.target.closest("[data-section-dialog-resize-handle]")) return null;
234:   const handle = event.target.closest("[data-section-dialog-drag='true']");
235:   if (!handle) return null;
236:   if (event.target.closest("button, a, input, textarea, select, label, summary")) return null;
237:   const dialog = handle.closest("dialog");
238:   return dialog?.open ? dialog : null;
239: }
```

beginSectionDialogDrag (script.js, lines 241-257)

beginSectionDialogDrag part of the public website runtime. In this file, it appears around lines 241-257 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionDialogFromDragEvent, anchorSectionDialog, getBoundingClientRect, add, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 242: if (event.button !== 0) return;; line 244: if (!dialog) return;.

Important local values include dialog at line 243; rect at line 246. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
241: function beginSectionDialogDrag(event) {
242:   if (event.button !== 0) return;
243:   const dialog = sectionDialogFromDragEvent(event);
244:   if (!dialog) return;
245:   anchorSectionDialog(dialog);
246:   const rect = dialog.getBoundingClientRect();
247:   activeSectionDialogDrag = {
248:     dialog,
249:     offsetX: event.clientX - rect.left,
250:     offsetY: event.clientY - rect.top,
251:     pointerId: event.pointerId,
252:     pointerTarget: event.target
253:   };
254:   event.target.setPointerCapture?.(event.pointerId);
255:   dialog.classList.add("is-dragging-dialog");
256:   event.preventDefault();
257: }
```

moveSectionDialogDrag (script.js, lines 259-268)

moveSectionDialogDrag part of the public website runtime. In this file, it appears around lines 259-268 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clampSectionDialogPosition. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 260: if (!activeSectionDialogDrag) return;.

Important local values include next at line 261. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
259: function moveSectionDialogDrag(event) {
260:   if (!activeSectionDialogDrag) return;
261:   const next = clampSectionDialogPosition(
262:     event.clientX - activeSectionDialogDrag.offsetX,
263:     event.clientY - activeSectionDialogDrag.offsetY,
264:     activeSectionDialogDrag.dialog
265:   );
266:   activeSectionDialogDrag.dialog.style.left = `${next.left}px`;
267:   activeSectionDialogDrag.dialog.style.top = `${next.top}px`;
268: }
```

endSectionDialogDrag (script.js, lines 270-275)

endSectionDialogDrag part of the public website runtime. In this file, it appears around lines 270-275 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 271: if (!activeSectionDialogDrag) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
270: function endSectionDialogDrag() {
271:   if (!activeSectionDialogDrag) return;
272:   activeSectionDialogDrag.pointerTarget?.releasePointerCapture?.(activeSectionDialogDrag.pointerId);
273:   activeSectionDialogDrag.dialog.classList.remove("is-dragging-dialog");
274:   activeSectionDialogDrag = null;
275: }
```

beginSectionDialogResize (script.js, lines 277-295)

beginSectionDialogResize part of the public website runtime. In this file, it appears around lines 277-295 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closest, contains, anchorSectionDialogForResize, getBoundingClientRect, add, and preventDefault. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 278: if (event.button !== 0) return;; line 281: if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog")) return;.

Important local values include handle at line 279; dialog at line 280. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
277: function beginSectionDialogResize(event) {
278:   if (event.button !== 0) return;
279:   const handle = event.target.closest("[data-section-dialog-resize-handle]");
280:   const dialog = handle?.closest("dialog");
281:   if (!handle || !dialog?.open || dialog.classList.contains("is-minimized-dialog")) return;
282:   anchorSectionDialogForResize(dialog);
283:   activeSectionDialogResize = {
284:     dialog,
285:     direction: handle.dataset.sectionDialogResizeHandle,
286:     pointerId: event.pointerId,
287:     pointerTarget: event.target,
288:     rect: dialog.getBoundingClientRect(),
289:     startX: event.clientX,
290:     startY: event.clientY
291:   };
292:   event.target.setPointerCapture?.(event.pointerId);
293:   dialog.classList.add("is-resizing-dialog");
294:   event.preventDefault();
295: }
```

moveSectionDialogResize (script.js, lines 297-304)

moveSectionDialogResize part of the public website runtime. In this file, it appears around lines 297-304 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionDialogResizeBounds. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 298: if (!activeSectionDialogResize) return;.

Important local values include next at line 299. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
297: function moveSectionDialogResize(event) {
298:   if (!activeSectionDialogResize) return;
299:   const next = sectionDialogResizeBounds(activeSectionDialogResize, event);
300:   activeSectionDialogResize.dialog.style.left = `${next.left}px`;
301:   activeSectionDialogResize.dialog.style.top = `${next.top}px`;
302:   activeSectionDialogResize.dialog.style.width = `${next.width}px`;
303:   activeSectionDialogResize.dialog.style.height = `${next.height}px`;
304: }
```

endSectionDialogResize (script.js, lines 306-311)

endSectionDialogResize part of the public website runtime. In this file, it appears around lines 306-311 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 307: if (!activeSectionDialogResize) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
306: function endSectionDialogResize() {
307:   if (!activeSectionDialogResize) return;
308:   activeSectionDialogResize.pointerTarget?.releasePointerCapture?.(activeSectionDialogResize.pointerId);
309:   activeSectionDialogResize.dialog.classList.remove("is-resizing-dialog");
310:   activeSectionDialogResize = null;
311: }
```

updateSectionDialogMinimize (script.js, lines 313-320)

updateSectionDialogMinimize updates ui, state, cache, or stored data. In this file, it appears around lines 313-320 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references contains, querySelector, and setAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 316: if (!button) return;.

Important local values include minimized at line 314; button at line 315. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
313: function updateSectionDialogMinimize(dialog) {
314:   const minimized = dialog.classList.contains("is-minimized-dialog");
315:   const button = dialog.querySelector(".section-view-minimize");
316:   if (!button) return;
317:   button.textContent = minimized ? "+" : "-";
318:   button.title = minimized ? "Restore window" : "Minimize window";
319:   button.setAttribute("aria-label", minimized ? "Restore window" : "Minimize window");
320: }
```

toggleSectionDialogMinimized (script.js, lines 322-326)

toggleSectionDialogMinimized part of the public website runtime. In this file, it appears around lines 322-326 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references anchorSectionDialog, toggle, and updateSectionDialogMinimize. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
322: function toggleSectionDialogMinimized(dialog) {
```

```

323:   anchorSectionDialog(dialog);
324:   dialog.classList.toggle("is-minimized-dialog");
325:   updateSectionDialogMinimize(dialog);
326: }

```

enableSectionDialogDrag (script.js, lines 328-347)

enableSectionDialogDrag part of the public website runtime. In this file, it appears around lines 328-347 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references addEventListener, querySelectorAll, forEach, getBoundingClientRect, and clampSectionDialogPosition. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 329: if (sectionDialogDragEnabled) return;.

Important local values include rect at line 341; next at line 342. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

328: function enableSectionDialogDrag() {
329:   if (sectionDialogDragEnabled) return;
330:   sectionDialogDragEnabled = true;
331:   document.addEventListener("pointerdown", beginSectionDialogResize);
332:   document.addEventListener("pointerdown", beginSectionDialogDrag);
333:   document.addEventListener("pointermove", moveSectionDialogResize);
334:   document.addEventListener("pointermove", moveSectionDialogDrag);
335:   document.addEventListener("pointerup", endSectionDialogResize);
336:   document.addEventListener("pointerup", endSectionDialogDrag);
337:   document.addEventListener("pointercancel", endSectionDialogResize);
338:   document.addEventListener("pointercancel", endSectionDialogDrag);
339:   window.addEventListener("resize", () => {
340:     document.querySelectorAll(".section-view-dialog.is-draggable-dialog[open]").forEach((dialog) => {
341:       const rect = dialog.getBoundingClientRect();
342:       const next = clampSectionDialogPosition(rect.left, rect.top, dialog);
343:       dialog.style.left = `${next.left}px`;
344:       dialog.style.top = `${next.top}px`;
345:     });
346:   });
347: }

```

escapeHtml (script.js, lines 349-356)

escapeHtml part of the public website runtime. In this file, it appears around lines 349-356 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replaceAll. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 350: return String(value || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

349: function escapeHtml(value) {
350:   return String(value || "")
351:     .replaceAll("&", "&amp;")
352:     .replaceAll("<", "&lt;")
353:     .replaceAll(">", "&gt;")
354:     .replaceAll("'", "&quot;")
355:     .replaceAll('"', "&#039;");
356: }

```

normalizeCodeLanguage (script.js, lines 358-364)

normalizeCodeLanguage validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 358-364 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, replace, find, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 363: return match?.id || "javascript";.

Important local values include clean at line 359; match at line 360. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
358: function normalizeCodeLanguage(value = "") {
359:   const clean = String(value || "").trim().toLowerCase().replace(/[_-]+/g, " ");
360:   const match = supportedCodeLanguages.find((language) =>
361:     language.id === clean || language.aliases.includes(clean)
362:   );
363:   return match?.id || "javascript";
364: }
```

codeLanguageLabel (script.js, lines 366-368)

codeLanguageLabel part of the public website runtime. In this file, it appears around lines 366-368 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and normalizeCodeLanguage. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 367: return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label || "Code";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
366: function codeLanguageLabel(value = "") {
367:   return supportedCodeLanguages.find((language) => language.id === normalizeCodeLanguage(value))?.label
|| "Code";
368: }
```

normalizeCodePasteMode (script.js, lines 370-372)

normalizeCodePasteMode validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 370-372 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 371: return value === "source" ? "source" : "basic";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
370: function normalizeCodePasteMode(value = "") {
371:   return value === "source" ? "source" : "basic";
372: }
```

normalizeCodeText (script.js, lines 374-383)

normalizeCodeText validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 374-383 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, normalizeCodePasteMode, split, map, trimEnd, join, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 376: if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, " ").replace(/\s+\$/g, "");; line 377: return normalized.

Important local values include normalized at line 375. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
374: function normalizeCodeText(value = "", pasteMode = "source") {
375:   const normalized = String(value || "").replace(/\r\n?/g, "\n").replace(/\u00a0/g, " ");
376:   if (normalizeCodePasteMode(pasteMode) === "source") return normalized.replace(/\t/g, "
").replace(/\s+$/g, "");
377:   return normalized
378:     .split("\n")
379:     .map((line) => line.trimEnd())
380:     .join("\n")
381:     .replace(/\n{4,}/g, "\n\n\n")
382:     .trim();
383: }
```

sourceLooksCpp (script.js, lines 385-389)

sourceLooksCpp part of the public website runtime. In this file, it appears around lines 385-389 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 387: return
/##include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text) ||.

Important local values include text at line 386. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
385: function sourceLooksCpp(source = "") {
386:   const text = String(source || "");
387:   return
/##include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text)
||
388:   /\b(std::|cout|cin|cerr|namespace|template\s*<|class\s+\w+|new\s+\w|delete\s+|constexpr|nullptr|using
\s+namespace)\b/.test(text);
389: }
```

sourceLooksC (script.js, lines 391-395)

sourceLooksC part of the public website runtime. In this file, it appears around lines 391-395 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 393: return
/##include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||.

Important local values include text at line 392. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
391: function sourceLooksC(source = "") {
392:   const text = String(source || "");
393:   return /##include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||
394:   /\b(printf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text);
395: }
```

detectCodeLanguage (script.js, lines 397-410)

detectCodeLanguage part of the public website runtime. In this file, it appears around lines 397-410 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It has 9 return statement(s). The most important visible returns are: line 399: if (*/<V?[a-z][\s\S]*?>/i*.test(code)) || *</!doctype\s+html/i*.test(code)) return "html"; line 400: if (*/\b(always_ff|always_comb)|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+w+)\b/i*.test(code)) return "systemverilog"; line 401: if (*/\b(module|endmodule)|always|assign|reg|wire|initial|posedge|negedge)\b/i*.test(code)) return "verilog"; line 402: if (*/^\s*(?:tran|ac|dc|op|model|subckt|endsubckt|param)\b/im*.test(code)) || */\b(Vw|Vs+W+S+W+S+S+(?:DC|SIN|PULSE)?/i*.test(code)) return "Ispice";. There are 5 additional return statements in the function body.

Relevant source excerpt:

```

397: function detectCodeLanguage(value = "") {
398:   const code = String(value || "");
399:   if (/<\/?[a-z][\s\S]*?>/i.test(code) || /<!doctype\s+html/i.test(code)) return "html";
400:   if (/^(\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|
class\s+w+)\b/\.test(code)) return "systemverilog";
401:   if (/^(\b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/\.test(code)) return
"verilog";
402:   if (/^(\s*\.?(tran|ac|dc|op|model|subckt|ends|param)\b/im.test(code) ||
/\bV\w+\s+\w+\s+\w+\s+(?:DC|SIN|PULSE)?/i.test(code)) return "ltspice";
403:   if (/^(\b(def|elif|import\s+w+|from\s+w+\s+import|self|None|True|False)\b/\.test(code)) return "python";
404:   if (/^(\b(public\s+class|private\s+|protected\s+|static\s+void\s+main|System\.out)\b/\.test(code)) return
"java";
405:   if (sourceLooksCpp(code) || sourceLooksC(code) ||
/^(\s*#include|main\s*\{|\puts\s*\(|fprintf|sizeof\s*\(|\b/\.test(code)) {
406:     return sourceLooksCpp(code) ? "cpp" : "c";
407:   }
408:   if (/^(\b(const|let|var|function|=>|console\.log|document\.|window\.)\b/\.test(code)) return "javascript";
409:   return "javascript";
410: }

```

tokenizedCodeHtml part of the public website runtime. In this file, it appears around lines 412-454 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

It calls or references `escapeHtml`, `replace`, `push`, `protect`, `includes`, `normalizeCodeLanguage`, `b`, and `CODE_TOKEN_`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

Important local values include `html` at line 413; `tokens` at line 414; `protect` at line 415; `token` at line 417; `commentPattern` at line 427; `keywordMap` at line 439; `keywords` at line 450. These variables show what the function is assembling, checking, or handing to another layer.

```

412: function tokenizedCodeHtml(code = "", language = "javascript") {
413:   let html = escapeHtml(code);
414:   const tokens = [];
415:   const protect = (pattern, className) => {
416:     html = html.replace(pattern, (match) => {
417:       const token = `@@CODE_TOKEN_${tokens.length}@@`;
418:       tokens.push(``);
419:       return token;
420:     });
421:   };
422:
423:   if (language === "html") {
424:     protect(/&lt;!--[\s\S]*--&gt;/g, "code-token-comment");
425:     protect(/(&lt;\/?)([a-zA-Z0-9:-]+)([\s\S]?)(\/?&gt;)/g, "code-token-tag");
426:   } else {
427:     const commentPattern = ["python", "lisp"].includes(normalizeCodeLanguage(language))
428:       ? /\/*[\s\S]*?\*/|\{\{\/\{\/[\^\\n]*#\#[^\\n]*\/g
429:       : /\/*[\s\S]*?\*/|\{\{\/\{\/[\^\\n]*\/g;
430:     protect(commentPattern, "code-token-comment");
431:     protect(/"(?:\\.|[\^\\])*"|'(?:\\.|[\^\\])*'|(?:\\/|[\^\\])*\/g, "code-token-string");

```

```

432:     if ([ "c", "cpp" ].includes(normalizeCodeLanguage(language))) {
433:         protect(/#\s*\w+[\^n]*/gm, "code-token-preprocessor");
434:     }
435: }
436:
437: protect(/\b(?:0x[\da-f]+|\d+(?:\.\d+)?(?:e[+-]?[d+])?)\b/gi, "code-token-number");
438:
439: const keywordMap = {
440:     c: "_Alignas|_Alignof|_Atomic|_Bool|_Complex|_Generic|_Imaginary|_Noreturn|_Static_assert|_Thread_loc
al|auto|break|case|char|const|continue|default|do|double|else|enum|extern|float|for|goto|if|inline|int|long|regi
ster|restrict|return|short|signed|sizeof|static|struct|switch|typedef|union|unsigned|void|volatile|while",
441:     cpp: "alignas|alignof|and|and_eq|asm|auto|bitand|bitor|bool|break|case|catch|char|char_t|char16_t|ch
ar32_t|class|compl|concept|const|consteval|constexpr|constinit|const_cast|continue|co_await|co_return|co_yield|d
ecltype|default|delete|do|double|dynamic_cast|else|enum|explicit|export|extern|false|float|for|friend|if|inline|
int|long|mutable|namespace|new|noexcept|not|not_eq|nullptr|operator|or|or_eq|private|protected|public|register|r
einterpret_cast|requires|return|short|signed|sizeof|static|static_assert|static_cast|struct|switch|template|this
|thread_local|throw|true|try|typedef|typeid|typename|union|unsigned|using|virtual|void|volatile|wchar_t|while|xor
|xor_eq",
442:     verilog: "always|and|assign|begin|buf|case|casex|casez|deassign|default|defparam|disable|edge|else|en
d|endcase|endfunction|endmodule|endprimitive|endspecify|endtable|endtask|event|for|force|forever|fork|function|g
enerate|genvar|if|initial|inout|input|integer|join|module|nand|negedge|nor|not|or|output|parameter|posedge|primi
tive|reg|release|repeat|signed|specify|supply0|supply1|table|task|tri|wand|while|wire|wor|xnor|xor",
443:     systemverilog: "always|always_comb|always_ff|always_latch|assign|automatic|begin|bit|case|class|clock
ing|covergroup|default|disable|do|else|end|endcase|endclass|endclocking|endfunction|endmodule|endpackage|endtask
|enum|for|forever|function|generate|genvar|if|initial|input|int|interface|logic|module|negedge|output|package|pa
rameter|posedge|reg|return|signed|task|typedef|wire",
444:     ltspice: "ac|dc|end|ends|four|func|global|ic|include|lib|meas|model|nodeset|op|options|param|plot|pro
be|save|step|subckt|temp|tran",
445:     java: "abstract|assert|boolean|break|byte|case|catch|char|class|const|continue|default|do|double|else
|enum|extends|final|finally|float|for|if|implements|import|instanceof|int|interface|long|native|new|null|package
|private|protected|public|return|short|static|strictfp|super|switch|synchronized|this|throw|throws|transient|try
|void|volatile|while",
446:     javascript: "async|await|break|case|catch|class|const|continue|debugger|default|delete|do|else|export
|extends|false|finally|for|from|function|if|import|in|instanceof|let|new|null|of|return|static|super|switch|this
|throw|true|try|typeof|undefined|var|void|while|yield",
447:     python: "and|as|assert|async|await|break|class|continue|def|del|elif|else|except|False|finally|for|fr
om|global|if|import|in|is|lambda|None|nonlocal|not|or|pass|raise|return|True|try|while|with|yield",
448:     html: "html|head|body|main|section|article|div|span|script|style|link|meta|title|header|footer|nav|bu
tton|input|form|label|img|a|p|pre|code"
449: };
450: const keywords = keywordMap[normalizeCodeLanguage(language)] || keywordMap.javascript;
451: html = html.replace(new RegExp(`\\b(${keywords})\\b`, "g"), "g"), '<span class="code-token-
keyword">$1</span>');
452: html = html.replace(/@@CODE_TOKEN_(\d+)@@/g, (_match, index) => tokens[Number(index)] || "");
453: return html;
454: }

```

protect (script.js, lines 415-421)

protect part of the public website runtime. In this file, it appears around lines 415-421 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 419: return token;.

Important local values include protect at line 415; token at line 417. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

415:     const protect = (pattern, className) => {
416:         html = html.replace(pattern, (match) => {
417:             const token = `@@CODE_TOKEN_${tokens.length}@@`;
418:             tokens.push(`<span class="${className}">${match}</span>`);
419:             return token;
420:         });
421:     };

```

renderRichCodeBlock (script.js, lines 456-465)

renderRichCodeBlock renders ui, markup, or display output. In this file, it appears around lines 456-465 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCodeLanguage, detectCodeLanguage, normalizeCodeText, escapeHtml, codeLanguageLabel, and tokenizedCodeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 459: return `.

Important local values include language at line 457; code at line 458. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
456: function renderRichCodeBlock(block = {}) {
457:   const language = normalizeCodeLanguage(block.language || detectCodeLanguage(block.code || ""));
458:   const code = normalizeCodeText(block.code || "", block.pasteMode || "source");
459:   return `
460:     <figure class="rich-code-block rich-code-language-${language}">
461:       <figcaption><span>&lt;&gt;</span> ${escapeHtml(codeLanguageLabel(language))}</figcaption>
462:       <pre><code>${tokenizedCodeHtml(code, language)}</code></pre>
463:     </figure>
464:   `;
465: }
```

renderPlainMultiline (script.js, lines 467-469)

renderPlainMultiline renders ui, markup, or display output. In this file, it appears around lines 467-469 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 468: return escapeHtml(value).replace(/\r\n?|\n/g, "
");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
467: function renderPlainMultiline(value = "") {
468:   return escapeHtml(value).replace(/\r\n?|\n/g, "<br>");
469: }
```

normalizeSiteContent (script.js, lines 471-477)

normalizeSiteContent validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 471-477 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 472: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
471: function normalizeSiteContent(content = {}) {
472:   return {
473:     heroCopy: String(content.heroCopy || "").trim() || defaultSiteContent.heroCopy,
474:     heroEyebrow: String(content.heroEyebrow || "").trim() || defaultSiteContent.heroEyebrow,
475:     heroTitle: String(content.heroTitle || "").trim() || defaultSiteContent.heroTitle
476:   };
477: }
```

normalizeProfile (script.js, lines 479-495)

normalizeProfile validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 479-495 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 480: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
479: function normalizeProfile(profileValue = {}) {
480:   return {
481:     brandImage: String(profileValue.brandImage || "").trim(),
482:     brandText: String(profileValue.brandText || "").trim() || String(profileValue.displayName ||
"").trim() || defaultProfile.brandText,
483:     contactIntro: String(profileValue.contactIntro || "").trim(),
484:     displayName: String(profileValue.displayName || "").trim(),
485:     email: String(profileValue.email || "").trim(),
486:     githubUrl: String(profileValue.githubUrl || "").trim(),
487:     heroImage: String(profileValue.heroImage || "").trim(),
488:     linkedinUrl: String(profileValue.linkedinUrl || "").trim(),
489:     phone: String(profileValue.phone || "").trim(),
490:     portfolioLabel: String(profileValue.portfolioLabel || "").trim() || defaultProfile.portfolioLabel,
491:     profileImage: String(profileValue.profileImage || "").trim(),
492:     resumeUrl: String(profileValue.resumeUrl || "").trim(),
493:     websiteUrl: String(profileValue.websiteUrl || "").trim()
494:   };
495: }
```

mailComposeLink (script.js, lines 497-500)

mailComposeLink part of the public website runtime. In this file, it appears around lines 497-500 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 499: return clean ? `https://mail.google.com/mail/?view=cm&fs=1&to=\${encodeURIComponent(clean)}` : "";

Important local values include clean at line 498. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
497: function mailComposeLink(email = "") {
498:   const clean = String(email || "").trim();
499:   return clean ? `https://mail.google.com/mail/?view=cm&fs=1&to=${encodeURIComponent(clean)}` : "";
500: }
```

phoneLink (script.js, lines 502-505)

phoneLink part of the public website runtime. In this file, it appears around lines 502-505 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 504: return clean ? `tel:\${clean.replace(/[\^d+]/g, "")}` : "";

Important local values include clean at line 503. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
502: function phoneLink(phone = "") {
503:   const clean = String(phone || "").trim();
504:   return clean ? `tel:${clean.replace(/[\^d+]/g, "")}` : "";
505: }
```

renderSiteContent (script.js, lines 507-515)

renderSiteContent renders ui, markup, or display output. In this file, it appears around lines 507-515 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeSiteContent, renderRichFieldContent, and applyPlainFieldStyle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include content at line 508. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
507: function renderSiteContent() {
508:   const content = normalizeSiteContent(siteContent || {});
509:   if (heroEyebrow) heroEyebrow.innerHTML = renderRichFieldContent(siteContentRich.heroEyebrow,
content.heroEyebrow);
510:   if (heroTitle) heroTitle.innerHTML = renderRichFieldContent(siteContentRich.heroTitle,
content.heroTitle);
511:   if (heroCopy) heroCopy.innerHTML = renderRichFieldContent(siteContentRich.heroCopy, content.heroCopy);
512:   applyPlainFieldStyle(heroEyebrow, "site-hero-eyebrow");
513:   applyPlainFieldStyle(heroTitle, "site-hero-title");
514:   applyPlainFieldStyle(heroCopy, "site-hero-copy");
515: }
```

renderProfile (script.js, lines 517-627)

renderProfile renders ui, markup, or display output. In this file, it appears around lines 517-627 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeProfile, mailComposeLink, phoneLink, normalizeLinkTarget, filter, renderRichFieldContent, applyPlainFieldStyle, slice, toUpperCase, setAttribute, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 622: return `

Important local values include current at line 518; displayName at line 519; label at line 520; emailLink at line 521; callLink at line 522; links at line 523; heroGithubLink at line 569; resumeObject at line 581; plus 2 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
517: function renderProfile() {
518:   const current = normalizeProfile(profile || {});
519:   const displayName = current.displayName || "Portfolio";
520:   const label = current.portfolioLabel || "Portfolio";
521:   const emailLink = mailComposeLink(current.email);
522:   const callLink = phoneLink(current.phone);
523:   const links = [
524:     current.email ? { label: "Email", url: emailLink, external: true } : null,
525:     current.phone ? { label: "Call", url: callLink } : null,
526:     current.githubUrl ? { label: "GitHub", url: normalizeLinkTarget(current.githubUrl, { assumeWeb: true
}), external: true } : null,
527:     current.linkedinUrl ? { label: "LinkedIn", url: normalizeLinkTarget(current.linkedinUrl, { assumeWeb:
true }}, external: true } : null,
528:     current.websiteUrl ? { label: "Website", url: normalizeLinkTarget(current.websiteUrl, { assumeWeb:
true }}, external: true } : null,
529:     current.resumeUrl ? { label: "Resume", url: current.resumeUrl, external: false } : null
530:   ].filter(Boolean);
531:
532:   document.title = `${displayName} | ${label}`;
533:   if (brandName) brandName.innerHTML = renderRichFieldContent(profileRich.displayName, displayName);
534:   if (brandSubtitle) brandSubtitle.innerHTML = renderRichFieldContent(profileRich.portfolioLabel, label);
535:   applyPlainFieldStyle(brandName, "profile-display-name");
536:   applyPlainFieldStyle(brandSubtitle, "profile-portfolio-label");
537:   if (brandText) brandText.textContent = current.brandText || displayName.slice(0, 3).toUpperCase() ||
"PORT";
538:   if (brandIcon) {
539:     if (current.brandImage) {
540:       brandIcon.src = current.brandImage;
541:       brandIcon.hidden = false;
542:     } else {
543:       brandIcon.hidden = true;
544:     }
545:   }
546:
547:   if (headerAvatar && headerAvatarImage) {
548:     headerAvatar.hidden = !current.profileImage;
549:     if (current.profileImage) {
550:       headerAvatarImage.src = current.profileImage;
551:       headerAvatarImage.alt = displayName;
552:       headerAvatar.setAttribute("aria-label", `Go to ${displayName} contact information`);
553:     }
554:   }
555:
556:   if (heroImage) {
557:     if (current.heroImage) {
```

```

558:         heroImage.hidden = false;
559:         heroImage.src = current.heroImage;
560:         heroImage.alt = `${displayName} portfolio background`;
561:     } else {
562:         heroImage.hidden = true;
563:     }
564: }
565:
566: document.querySelectorAll('a[href="#resume"]').forEach((link) => {
567:     link.hidden = !current.resumeUrl;
568: });
569: const heroGithubLink = [...document.querySelectorAll(".hero-actions a")]
570:     .find((link) => /github/i.test(link.textContent || ""));
571: if (heroGithubLink) {
572:     heroGithubLink.hidden = !current.githubUrl;
573:     if (current.githubUrl) heroGithubLink.href = normalizeLinkTarget(current.githubUrl, { assumeWeb: true
});
574: }
575:
576: if (resumeSection) {
577:     resumeSection.hidden = !current.resumeUrl;
578:     resumeSection.querySelectorAll("a").forEach((link) => {
579:         link.href = current.resumeUrl || "#";
580:     });
581:     const resumeObject = resumeSection.querySelector("object");
582:     if (resumeObject) resumeObject.data = current.resumeUrl || "";
583:     const fallbackLink = resumeSection.querySelector("object a");
584:     if (fallbackLink) fallbackLink.href = current.resumeUrl || "#";
585: }
586:
587: if (profilePhoto) {
588:     profilePhoto.hidden = !current.profileImage;
589:     if (current.profileImage) {
590:         profilePhoto.src = current.profileImage;
591:         profilePhoto.alt = `Portrait of ${displayName}`;
592:     }
593: }
594: if (contactTitle) contactTitle.innerHTML = renderRichFieldContent(profileRich.displayName,
displayName);
595: applyPlainFieldStyle(contactTitle, "profile-display-name");
596: if (contactIntro) {
597:     contactIntro.innerHTML = renderRichFieldContent(
598:         profileRich.contactIntro,
599:         current.contactIntro || (current.displayName ? `${displayName}'s contact information and public
links` : "Add contact details in the builder.")
600:     );
601: }
602: applyPlainFieldStyle(contactIntro, "profile-contact-intro");
603: if (contactDetails) {
604:     contactDetails.innerHTML = [
605:         current.email ? `` : "",
606:         current.phone ? `` : ""
607:     \\].filter\\(Boolean\\).join\\(""\\);
608: }
609: if \\(contactLinks\\) {
610:     contactLinks.innerHTML = links.map\\(\\(link\\) => {
611:         const styleId = link.label === "GitHub"
612:             ? "profile-github"
613:             : link.label === "LinkedIn"
614:             ? "profile-linkedin"
615:             : link.label === "Website"
616:             ? "profile-website"
617:             : link.label === "Email"
618:             ? "profile-email"
619:             : link.label === "Call"
620:             ? "profile-phone"
621:             : "";
622:         return ``;
623:     }\\\).join\\\(""\\\);
624: }
625: if \\\(footerOwner\\\) footerOwner.innerHTML = `&copy; <span id="year">\\\${new Date\\\(\\\).getFullYear\\\(\\\)}</span>
\\\${displayName}`;
626: if \\\(contactBand\\\) contactBand.hidden = !links.length && !current.profileImage && !current.contactIntro
&& !current.displayName;
627: }

```

textBlocksFromPlainText (script.js, lines 629-637)

textBlocksFromPlainText part of the public website runtime. In this file, it appears around lines 629-637 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 630: return [{}].

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
629: function textBlocksFromPlainText(text) {
630:   return [{
631:     align: "left",
632:     fontFamily: "Arial",
633:     fontSize: "normal",
634:     text: String(text || ""),
635:     type: "paragraph"
636:   }];
637: }
```

unwrapFormula (script.js, lines 639-652)

unwrapFormula part of the public website runtime. In this file, it appears around lines 639-652 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and match. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 649: if (match) return match[group].trim(); line 651: return text;.

Important local values include text at line 640; wrappers at line 641; match at line 648. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
639: function unwrapFormula(value) {
640:   const text = String(value || "").trim();
641:   const wrappers = [
642:     [/^\\$\\$([\\s\\S]+)\\$\\$/g, 1],
643:     [/^\\$\\$([\\s\\S]+)\\$\\$/g, 1],
644:     [/^\\$\\$([\\s\\S]+)\\$\\$/g, 1],
645:     [/^\\$\\$([\\s\\S]+)\\$\\$/g, 1]
646:   ];
647:   for (const [pattern, group] of wrappers) {
648:     const match = text.match(pattern);
649:     if (match) return match[group].trim();
650:   }
651:   return text;
652: }
```

renderInlineMath (script.js, lines 654-663)

renderInlineMath renders ui, markup, or display output. In this file, it appears around lines 654-663 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, replace, b, match, slice, and normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 657: return withMath.replace(/b(https?:\\V[\\s<]+|www\\. [\\s<]+|(? :github|linkedin|gitlab|bitbucket|drive|docs)\\. comV[\\s<]+)/gi, (match) => {; line 661: return `<a href="\${href}" target="\_blank" rel="noreferrer">\${clean}</a>\${trailing}`;.

Important local values include escaped at line 655; withMath at line 656; trailing at line 658; clean at line 659; href at line 660. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
654: function renderInlineMath(text) {
655:   const escaped = escapeHtml(text);
656:   const withMath = escaped.replace(/\\$\\$([\\s\\S]+)\\$\\$/g, '<span class="rich-inline-formula">$1</span>');
657:   return withMath.replace(/b(https?:\\V[\\s<]+|www\\. [\\s<]+|(? :github|linkedin|gitlab|bitbucket|drive|docs)\\. comV[\\s<]+)/gi, (match) => {
658:     const trailing = match.match(/[,;:!?]+$/)?.[0] || "";
659:     const clean = trailing ? match.slice(0, -trailing.length) : match;
660:     const href = normalizeLinkTarget(clean, { assumeWeb: true });
661:     return `<a href="${href}" target="_blank" rel="noreferrer">${clean}</a>${trailing}`;
```

```
662:   });
663: }
```

renderMultilineInlineText (script.js, lines 665-667)

renderMultilineInlineText renders ui, markup, or display output. In this file, it appears around lines 665-667 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderInlineMath, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 666: return renderInlineMath(text).replace(/\r\n?|\n/g, "
");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
665: function renderMultilineInlineText(text = "") {
666:   return renderInlineMath(text).replace(/\r\n?|\n/g, "<br>");
667: }
```

looksLikeBareWebAddress (script.js, lines 669-676)

looksLikeBareWebAddress part of the public website runtime. In this file, it appears around lines 669-676 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, split, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 671: if (!clean || /\s/.test(clean) || /^(.?.\?V|#|mailto:|tel:)/i.test(clean)) return false;; line 672: if (!/[a-z][a-z0-9+.-]*/i.test(clean)) return false;; line 674: if (!/[a-z0-9-]+(\.[a-z0-9-]+)+\$/i.test(host)) return false;; line 675: return !/\.(pdf|docx?|xlsx?|pptx?|zip|7z|rar|png|jpe?g|gif|svg|webp|txt|md|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb)\$/i.test(host);.

Important local values include clean at line 670; host at line 673. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
669: function looksLikeBareWebAddress(value = "") {
670:   const clean = String(value || "").trim();
671:   if (!clean || /\s/.test(clean) || /^(.?.\?V|#|mailto:|tel:)/i.test(clean)) return false;
672:   if (!/[a-z][a-z0-9+.-]*/i.test(clean)) return false;
673:   const host = clean.split(/[/?#]/)[0].toLowerCase();
674:   if (!/[a-z0-9-]+(\.[a-z0-9-]+)+$/i.test(host)) return false;
675:   return !/\.(pdf|docx?|xlsx?|pptx?|zip|7z|rar|png|jpe?g|gif|svg|webp|txt|md|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb)$/i.test(host);
676: }
```

looksLikeWebOrContactText (script.js, lines 678-686)

looksLikeWebOrContactText part of the public website runtime. In this file, it appears around lines 678-686 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, b, looksLikeBareWebAddress, split, some, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 680: if (!clean) return false;; line 681: if (/^(?:https?:V|www\.)[\s<>"]+/i.test(clean)) return true;; line 682: if (/^(b[a-z0-9._%+~]@[a-z0-9-]+\.[a-z]{2,}\b/i.test(clean)) return true;; line 683: if (/^(b(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com|[\s<>"]+/i.test(clean)) return true;. There are 2 additional return statements in the function body.

Important local values include clean at line 679. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
678: function looksLikeWebOrContactText(value = "") {
```

```

679:   const clean = String(value || "").trim();
680:   if (!clean) return false;
681:   if (/^b(?:https?:\/\/|www\.)[^s<>'"]+/i.test(clean)) return true;
682:   if (/^b[a-z0-9._%+~]@[a-z0-9.-]+\.[a-z]{2,}\b/i.test(clean)) return true;
683:   if (/^b(?:github|linkedin|gitlab|bitbucket|drive|docs)\.com\/[^s<>'"]+/i.test(clean)) return true;
684:   if (looksLikeBareWebAddress(clean)) return true;
685:   return clean.split(/\s+/).some((word) => looksLikeBareWebAddress(word.replace(/^[('"]+|['"]+$/g,
  "")));
686: }

```

normalizeLinkTarget (script.js, lines 688-695)

normalizeLinkTarget validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 688-695 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and looksLikeBareWebAddress. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 690: if (!clean) return ""; line 691: if (/^V/i.test(clean)) return `https:\${clean}`; line 692: if (/^www\./i.test(clean)) return `https://\${clean}`; line 693: if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://\${clean}`. There are 1 additional return statements in the function body.

Important local values include clean at line 689. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

688: function normalizeLinkTarget(target = "", options = {}) {
689:   const clean = String(target || "").trim();
690:   if (!clean) return "";
691:   if (/^V/i.test(clean)) return `https:${clean}`;
692:   if (/^www\./i.test(clean)) return `https://${clean}`;
693:   if (options.assumeWeb && looksLikeBareWebAddress(clean)) return `https://${clean}`;
694:   return clean;
695: }

```

isWebsiteLinkItem (script.js, lines 697-705)

isWebsiteLinkItem part of the public website runtime. In this file, it appears around lines 697-705 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, filter, join, toLowerCase, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 699: if (/^(https?:)?V/i.test(clean) || /^www\./i.test(clean)) return true; line 704: return /\b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText);

Important local values include clean at line 698; typeText at line 700. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

697: function isWebsiteLinkItem(item = {}, target = "") {
698:   const clean = String(target || "").trim();
699:   if (/^(https?:)?V/i.test(clean) || /^www\./i.test(clean)) return true;
700:   const typeText = [item.kind, item.type, item.status, item.meta, item.label, item.title]
701:     .filter(Boolean)
702:     .join(" ")
703:     .toLowerCase();
704:   return /\b(link|website|web link|url|external|github|linkedin|drive|social|profile)\b/.test(typeText);
705: }

```

linkifyRichTextNodes (script.js, lines 707-742)

linkifyRichTextNodes part of the public website runtime. In this file, it appears around lines 707-742 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createTreeWalker, acceptNode, closest, nextNode, push, forEach, createDocumentFragment, replace, b, append, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 710: if (!node.textContent || !(https?:\\|www\\.|(?github|linkedin|gitlab|bitbucket|drive|docs)\\.com\\)/i.test(node.textContent)) return NodeFilter.FILTER_REJECT;; line 711: return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;; line 737: return match;

Important local values include walker at line 708; nodes at line 714; node at line 715; text at line 722; fragment at line 723; cursor at line 724; trailing at line 727; clean at line 728; plus 1 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
707: function linkifyRichTextNodes(root) {
708:   const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {
709:     acceptNode(node) {
710:       if (!node.textContent ||
!(https?:\\|www\\.|(?github|linkedin|gitlab|bitbucket|drive|docs)\\.com\\)/i.test(node.textContent)) return
NodeFilter.FILTER_REJECT;
711:       return node.parentElement?.closest("a") ? NodeFilter.FILTER_REJECT : NodeFilter.FILTER_ACCEPT;
712:     }
713:   });
714:   const nodes = [];
715:   let node = walker.nextNode();
716:   while (node) {
717:     nodes.push(node);
718:     node = walker.nextNode();
719:   }
720:
721:   nodes.forEach((textNode) => {
722:     const text = textNode.textContent || "";
723:     const fragment = document.createDocumentFragment();
724:     let cursor = 0;
725:     text.replace(/\\b(https?:\\|www\\.|(?github|linkedin|gitlab|bitbucket|drive|docs)\\.com
\\|(?\\s+)|gi, (match, _url, offset) => {
726:       if (offset > cursor) fragment.append(document.createTextNode(text.slice(cursor, offset)));
727:       const trailing = match.match(/[,.;:!?]+$/)?.[0] || "";
728:       const clean = trailing ? match.slice(0, -trailing.length) : match;
729:       const link = document.createElement("a");
730:       link.href = normalizeLinkTarget(clean, { assumeWeb: true });
731:       link.target = "_blank";
732:       link.rel = "noreferrer";
733:       link.textContent = clean;
734:       fragment.append(link);
735:       if (trailing) fragment.append(document.createTextNode(trailing));
736:       cursor = offset + match.length;
737:       return match;
738:     });
739:     if (cursor < text.length) fragment.append(document.createTextNode(text.slice(cursor)));
740:     textNode.replaceWith(fragment);
741:   });
742: }
```

sanitizeRichInlineHtml (script.js, lines 744-787)

sanitizeRichInlineHtml validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 744-787 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, Set, querySelectorAll, forEach, has, replaceWith, getAttribute, trim, normalizeLinkTarget, cleanFontFamily, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 752: return;; line 786: return template.innerHTML;.

Important local values include template at line 745; allowedTags at line 747; href at line 755; normalizedHref at line 756; safeHref at line 757; fontFamily at line 758; fontPx at line 759; color at line 760; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
744: function sanitizeRichInlineHtml(value = "") {
745:   const template = document.createElement("template");
746:   template.innerHTML = String(value || "");
747:   const allowedTags = new Set(["A", "B", "BR", "EM", "I", "SPAN", "STRONG", "U"]);
748:
749:   [...template.content.querySelectorAll("*")].forEach((node) => {
750:     if (!allowedTags.has(node.tagName)) {
751:       node.replaceWith(...node.childNodes);

```

```

752:     return;
753: }
754:
755: const href = node.tagName === "A" ? String(node.getAttribute("href") || "").trim() : "";
756: const normalizedHref = normalizeLinkTarget(href, { assumeWeb: true });
757: const safeHref = /^(https?:|mailto:|tel:|#|\/)/i.test(normalizedHref) ? normalizedHref : "";
758: const fontFamily = cleanFontFamily(node.style.fontFamily || "");
759: const fontPx = normalizeFontPx(parseFloat(node.style.fontSize || ""));
760: const color = normalizeTextColor(node.style.color || "");
761: const rawFontWeight = node.style.fontWeight || "";
762: const fontWeight = /^(bold|[6-9]00)$/i.test(rawFontWeight)
763:   ? "700"
764:   : /^(normal|[1-5]00)$/i.test(rawFontWeight)
765:   ? "400"
766:   : "";
767: const fontStyle = node.style.fontStyle === "italic" ? "italic" : "";
768: const textDecoration = node.style.textDecoration.includes("underline") ? "underline" : "";
769:
770: [...node.attributes].forEach((attribute) => node.removeAttribute(attribute.name));
771:
772: if (node.tagName === "A" && safeHref) {
773:   node.setAttribute("href", safeHref);
774:   node.setAttribute("target", "_blank");
775:   node.setAttribute("rel", "noreferrer");
776: }
777: if (fontFamily) node.style.fontFamily = fontFamily;
778: if (fontPx) node.style.fontSize = `${fontPx}px`;
779: if (color) node.style.color = color;
780: if (fontWeight) node.style.fontWeight = fontWeight;
781: if (fontStyle) node.style.fontStyle = fontStyle;
782: if (textDecoration) node.style.textDecoration = textDecoration;
783: });
784:
785: linkifyRichTextNodes(template.content);
786: return template.innerHTML;
787: }

```

displayTitle (script.js, lines 789-798)

displayTitle part of the public website runtime. In this file, it appears around lines 789-798 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 797: return replacements[title] || title;.

Important local values include title at line 790; replacements at line 791. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

789: function displayTitle(value, fallback = "Untitled item") {
790:   const title = String(value || fallback).trim();
791:   const replacements = {
792:     "Brief": "Overview",
793:     "Project Brief": "Overview",
794:     "Design Brief": "Design Overview",
795:     "Simulation Brief": "Simulation Overview"
796:   };
797:   return replacements[title] || title;
798: }

```

cleanFontFamily (script.js, lines 800-807)

cleanFontFamily part of the public website runtime. In this file, it appears around lines 800-807 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, replace, trim, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 801: return String(value).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

800: function cleanFontFamily(value = "") {

```

```

801:   return String(value)
802:     .split(",")
803:     .map((item) => item.replace(/^[^w\s-]/g, "").trim())
804:     .filter(Boolean)
805:     .slice(0, 3)
806:     .join(", ");
807: }

```

normalizeFontPx (script.js, lines 809-812)

normalizeFontPx validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 809-812 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 811: return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";

Important local values include size at line 810. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

809: function normalizeFontPx(value) {
810:   const size = Number(value);
811:   return Number.isFinite(size) && size >= 8 && size <= 24 ? String(size) : "";
812: }

```

normalizeTextColor (script.js, lines 813-820)

normalizeTextColor validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 813-820 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 815: return /^[0-9a-f]{3}||[0-9a-f]{6}\$/i.test(color) ||.

Important local values include color at line 814. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

813: function normalizeTextColor(value = "") {
814:   const color = String(value || "").trim();
815:   return /^[0-9a-f]{3}||[0-9a-f]{6}$/i.test(color) ||
816:     /^rgba?\(/i.test(color) ||
817:     /^hsla?\(/i.test(color)
818:     ? color
819:     : "";
820: }

```

rgbColorToHex (script.js, lines 822-830)

rgbColorToHex part of the public website runtime. In this file, it appears around lines 822-830 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, toLowerCase, match, map, max, min, toString, padStart, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 824: if (/^[0-9a-f]{6}\$/i.test(color)) return color.toLowerCase();; line 826: if (shortHex) return `#\${shortHex[1]}\${shortHex[1]}\${shortHex[2]}\${shortHex[2]}\${shortHex[3]}\${shortHex[3]}`.toLowerCase();; line 828: if (!rgb) return color;; line 829: return `#\${[rgb[1], rgb[2], rgb[3]].map((part) => Math.max(0, Math.min(255, Number(part))).toString(16).padStart(2, "0")).join("")}`;

Important local values include color at line 823; shortHex at line 825; rgb at line 827. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
822: function rgbColorToHex(value = "") {
823:   const color = String(value || "").trim();
824:   if (/^[0-9a-f]{6}$/i.test(color)) return color.toLowerCase();
825:   const shortHex = color.match(/^[0-9a-f]{1}([0-9a-f])([0-9a-f])$/i);
826:   if (shortHex) return
`#${shortHex[1]}${shortHex[2]}${shortHex[2]}${shortHex[2]}${shortHex[3]}${shortHex[3]}`.toLowerCase();
827:   const rgb = color.match(/^rgba?\s*((\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*)/i);
828:   if (!rgb) return color;
829:   return `#${[rgb[1], rgb[2], rgb[3]].map((part) => Math.max(0, Math.min(255,
Number(part))).toString(16).padStart(2, "0"))}.join("")}`;
830: }
```

normalizePlainFieldStyle (script.js, lines 832-844)

normalizePlainFieldStyle validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 832-844 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, normalizeTextColor, and rgbColorToHex. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 843: return normalized;.

Important local values include normalized at line 833; fontFamily at line 834; fontPx at line 835; color at line 836. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
832: function normalizePlainFieldStyle(style = {}) {
833:   const normalized = {};
834:   const fontFamily = cleanFontFamily(style.fontFamily || "");
835:   const fontPx = normalizeFontPx(style.fontPx || style.fontSize || "");
836:   const color = normalizeTextColor(style.color || "");
837:   if (fontFamily) normalized.fontFamily = fontFamily;
838:   if (fontPx) normalized.fontPx = fontPx;
839:   if (color) normalized.color = rgbColorToHex(color);
840:   if (style.bold === true || style.bold === "true") normalized.bold = true;
841:   if (style.italic === true || style.italic === "true") normalized.italic = true;
842:   if (style.underline === true || style.underline === "true") normalized.underline = true;
843:   return normalized;
844: }
```

normalizeFieldStyles (script.js, lines 846-852)

normalizeFieldStyles validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 846-852 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fromEntries, entries, map, normalizePlainFieldStyle, filter, and keys. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 847: return Object.fromEntries(.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
846: function normalizeFieldStyles(styles = {}) {
847:   return Object.fromEntries(
848:     Object.entries(styles || {}).
849:       .map(([fieldId, style]) => [fieldId, normalizePlainFieldStyle(style)])
850:       .filter(([ , style]) => Object.keys(style).length)
851:   );
852: }
```

plainFieldStyleToCss (script.js, lines 854-864)

plainFieldStyleToCss part of the public website runtime. In this file, it appears around lines 854-864 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizePlainFieldStyle`, `push`, and `join`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 863: `return rules.join("; ");`.

Important local values include `normalized` at line 855; `rules` at line 856. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
854: function plainFieldStyleToCss(style = {}) {
855:   const normalized = normalizePlainFieldStyle(style);
856:   const rules = [];
857:   if (normalized.fontFamily) rules.push(`font-family: ${normalized.fontFamily}`);
858:   if (normalized.fontPx) rules.push(`font-size: ${normalized.fontPx}px`);
859:   if (normalized.color) rules.push(`color: ${normalized.color}`);
860:   if (normalized.bold) rules.push("font-weight: 700");
861:   if (normalized.italic) rules.push("font-style: italic");
862:   if (normalized.underline) rules.push("text-decoration: underline");
863:   return rules.join("; ");
864: }
```

applyPlainFieldStyle (script.js, lines 866-875)

`applyPlainFieldStyle` part of the public website runtime. In this file, it appears around lines 866-875 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizePlainFieldStyle`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 867: `if (!element) return;`.

Important local values include `style` at line 868. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
866: function applyPlainFieldStyle(element, fieldId = "") {
867:   if (!element) return;
868:   const style = normalizePlainFieldStyle(fieldStyles[fieldId] || {});
869:   element.style.fontFamily = style.fontFamily || "";
870:   element.style.fontSize = style.fontPx ? `${style.fontPx}px` : "";
871:   element.style.color = style.color || "";
872:   element.style.fontWeight = style.bold ? "700" : "";
873:   element.style.fontStyle = style.italic ? "italic" : "";
874:   element.style.textDecoration = style.underline ? "underline" : "";
875: }
```

plainFieldStyleAttribute (script.js, lines 877-880)

`plainFieldStyleAttribute` part of the public website runtime. In this file, it appears around lines 877-880 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `plainFieldStyleToCss`, and `escapeHtml`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 879: `return css ? `style="${escapeHtml(css)}"` : "";`.

Important local values include `css` at line 878. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
877: function plainFieldStyleAttribute(fieldId = "") {
878:   const css = plainFieldStyleToCss(fieldStyles[fieldId] || {});
879:   return css ? `style="${escapeHtml(css)}"` : "";
880: }
```

richTextStyle (script.js, lines 882-896)

`richTextStyle` part of the public website runtime. In this file, it appears around lines 882-896 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references cleanFontFamily, normalizeFontPx, normalizeTextColor, push, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 895: return styles.length ? `style="\${escapeHtml(styles.join(";"))}"` : "";

Important local values include styles at line 883; fontFamily at line 884; fontPx at line 885; color at line 886. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
882: function richTextStyle(block = {}) {
883:   const styles = [];
884:   const fontFamily = cleanFontFamily(block.fontFamily || "Arial") || "Arial";
885:   const fontPx = normalizeFontPx(block.fontPx);
886:   const color = normalizeTextColor(block.color || "");
887:
888:   if (fontFamily) styles.push(`font-family: ${fontFamily}`);
889:   if (fontPx) styles.push(`font-size: ${fontPx}px`);
890:   if (color) styles.push(`color: ${color}`);
891:   if (block.bold) styles.push("font-weight: 700");
892:   if (block.italic) styles.push("font-style: italic");
893:   if (block.underline) styles.push("text-decoration: underline");
894:
895:   return styles.length ? `style="${escapeHtml(styles.join(";"))}"` : "";
896: }
```

normalizeCropAspect (script.js, lines 898-900)

normalizeCropAspect validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 898-900 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 899: return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
898: function normalizeCropAspect(value = "") {
899:   return ["1 / 1", "4 / 3", "16 / 9", "3 / 4"].includes(value) ? value : "original";
900: }
```

normalizeCropZoom (script.js, lines 902-905)

normalizeCropZoom validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 902-905 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 904: return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;

Important local values include zoom at line 903. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
902: function normalizeCropZoom(value = 1) {
903:   const zoom = Number(value);
904:   return Number.isFinite(zoom) ? Math.min(3, Math.max(1, zoom)) : 1;
905: }
```

normalizeCropPosition (script.js, lines 907-910)

normalizeCropPosition validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 907-910 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isFinite, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 909: return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;.

Important local values include position at line 908. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
907: function normalizeCropPosition(value = 50) {
908:   const position = Number(value);
909:   return Number.isFinite(position) ? Math.min(100, Math.max(0, position)) : 50;
910: }
```

richImageCropStyle (script.js, lines 912-921)

richImageCropStyle part of the public website runtime. In this file, it appears around lines 912-921 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeCropAspect, normalizeCropZoom, normalizeCropPosition, push, escapeHtml, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 920: return ` style="\${escapeHtml(styles.join("; ")))}`;.

Important local values include aspect at line 913; styles at line 914. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
912: function richImageCropStyle(block = {}) {
913:   const aspect = normalizeCropAspect(block.cropAspect);
914:   const styles = [
915:     `--crop-zoom: ${normalizeCropZoom(block.cropZoom)}`,
916:     `--crop-x: ${normalizeCropPosition(block.cropX)}%`,
917:     `--crop-y: ${normalizeCropPosition(block.cropY)}%`
918:   ];
919:   if (aspect !== "original") styles.push(`--crop-aspect: ${aspect}`);
920:   return ` style="${escapeHtml(styles.join("; ")))}`;
921: }
```

richImageDownloadLink (script.js, lines 923-940)

richImageDownloadLink part of the public website runtime. In this file, it appears around lines 923-940 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, cleanRichImageTitle, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 928: return `

Important local values include label at line 924; url at line 925; target at line 926. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
923: function richImageDownloadLink(block = {}) {
924:   const label = escapeHtml(cleanRichImageTitle(block) || block.caption || "Image file");
925:   const url = block.url || "#";
926:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(block, url) });
927: }
```

```

928:     return `
929:     <p class="rich-download-only">
930:       <a
931:         class="resource-link"
932:         href="${escapeHtml(target)}"
933:         ${linkAttributes(target, block)}
934:         ${downloadAttribute(target, block)}
935:       >
936:         ${label}
937:       </a>
938:     </p>
939:   `;
940: }

```

cleanRichImageTitle (script.js, lines 942-945)

cleanRichImageTitle part of the public website runtime. In this file, it appears around lines 942-945 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 944: return /^pasted image\b/i.test(title) ? "" : title;.

Important local values include title at line 943. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

942: function cleanRichImageTitle(block = {}) {
943:   const title = String(block.title || "").trim();
944:   return /^pasted image\b/i.test(title) ? "" : title;
945: }

```

renderRichContent (script.js, lines 947-982)

renderRichContent renders ui, markup, or display output. In this file, it appears around lines 947-982 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, map, includes, richImageDownloadLink, cleanRichImageTitle, normalizeLinkTarget, normalizeCropAspect, richImageCropStyle, escapeHtml, unwrapFormula, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 949: return `; line 954: if (block.display === "download") return richImageDownloadLink(block);; line 957: return `; line 969: return `<p class="rich-paragraph rich-text-normal text-\${align}">\${renderInlineMath(formula)}</p>`. There are 3 additional return statements in the function body.

Important local values include blocks at line 948; align at line 952; title at line 955; imageSrc at line 956; formula at line 967; size at line 974; content at line 975. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

947: function renderRichContent(rich, fallbackText = "") {
948:   const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
949:   return `
950:     <div class="rich-content">
951:       ${blocks.map((block) => {
952:         const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
953:         if (block.type === "image") {
954:           if (block.display === "download") return richImageDownloadLink(block);
955:           const title = cleanRichImageTitle(block);
956:           const imageSrc = normalizeLinkTarget(block.url, { assumeWeb: true });
957:           return
958:             <figure class="rich-image justify-${align}">
959:               <span class="rich-image-viewport crop-${normalizeCropAspect(block.cropAspect) ===
"original" ? "original" : "active"}"${richImageCropStyle(block)}>
960:                 
961:               </span>
962:               ${title || block.caption} ? <figcaption>${title} ?
<strong>${escapeHtml(title)}</strong>` : ""}${block.caption ? `<span>${escapeHtml(block.caption)}</span>` : ""}
</figcaption>` : ""}
963:             </figure>
964:           `;
965:         }
966:         if (block.type === "formula") {
967:           const formula = unwrapFormula(block.formula);

```

```

968:         if (looksLikeWebOrContactText(formula)) {
969:             return `
```

renderRichFieldContent (script.js, lines 984-994)

renderRichFieldContent renders ui, markup, or display output. In this file, it appears around lines 984-994 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, map, includes, sanitizeRichInlineHtml, renderInlineMath, richTextStyle, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 986: return blocks.map((block) => {; line 987: if (block.type !== "paragraph") return "";; line 992: return `

Important local values include blocks at line 985; align at line 988; content at line 989. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

984: function renderRichFieldContent(rich, fallbackText = "") {
985:   const blocks = rich?.blocks?.length ? rich.blocks : textBlocksFromPlainText(fallbackText);
986:   return blocks.map((block) => {
987:     if (block.type !== "paragraph") return "";
988:     const align = ["left", "center", "right"].includes(block.align) ? block.align : "left";
989:     const content = block.html
990:       ? sanitizeRichInlineHtml(block.html)
991:       : renderInlineMath(block.text || "");
992:     return `

```

slugLabel (script.js, lines 996-999)

slugLabel part of the public website runtime. In this file, it appears around lines 996-999 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 998: return category ? category.label : value;.

Important local values include category at line 997. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

996: function slugLabel(value) {
997:   const category = categories.find((item) => item.id === value);
998:   return category ? category.label : value;
999: }

```

canonicalTemplateld (script.js, lines 1001-1003)

canonicalTemplateld part of the public website runtime. In this file, it appears around lines 1001-1003 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1002: return legacyTemplateSkins[id] || id || "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1001: function canonicalTemplateId(id) {  
1002:   return legacyTemplateSkins[id] || id || "";  
1003: }
```

projectTemplateId (script.js, lines 1005-1007)

projectTemplateId part of the public website runtime. In this file, it appears around lines 1005-1007 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references canonicalTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1006: return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1005: function projectTemplateId(project) {  
1006:   return canonicalTemplateId(project?.portfolioView?.template?.id || project?.templateId || "");  
1007: }
```

projectTemplateClass (script.js, lines 1009-1012)

projectTemplateClass part of the public website runtime. In this file, it appears around lines 1009-1012 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1011: return templateId ? `project-template project-template-\${templateId}` : "project-template-white";.

Important local values include templateId at line 1010. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1009: function projectTemplateClass(project) {  
1010:   const templateId = projectTemplateId(project);  
1011:   return templateId ? `project-template project-template-${templateId}` : "project-template-white";  
1012: }
```

responsiveClassName (script.js, lines 1014-1019)

responsiveClassName part of the public website runtime. In this file, it appears around lines 1014-1019 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1015: return String(value || fallback || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1014: function responsiveClassName(value, fallback) {
1015:   return String(value || fallback || "")
1016:     .toLowerCase()
1017:     .replace(/^[a-z0-9]+/g, "-")
1018:     .replace(/^-|-$/g, "") || fallback;
1019: }
```

projectResponsiveClass (script.js, lines 1021-1024)

projectResponsiveClass part of the public website runtime. In this file, it appears around lines 1021-1024 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectResponsiveProfile, and responsiveClassName. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1023: return `responsive-project responsive-density-\${responsiveClassName(profile.density, "balanced")} responsive-card-\${responsiveClassName(profile.cardLayout, "single")}`;

Important local values include profile at line 1022. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1021: function projectResponsiveClass(project) {
1022:   const profile = projectResponsiveProfile(project);
1023:   return `responsive-project responsive-density-${responsiveClassName(profile.density, "balanced")}
responsive-card-${responsiveClassName(profile.cardLayout, "single")}`;
1024: }
```

responsiveStyleValues (script.js, lines 1026-1034)

responsiveStyleValues part of the public website runtime. In this file, it appears around lines 1026-1034 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectResponsiveProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1028: return {.

Important local values include profile at line 1027. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1026: function responsiveStyleValues(project) {
1027:   const profile = projectResponsiveProfile(project);
1028:   return {
1029:     "--responsive-section-card-min": profile.sectionCardMin,
1030:     "--responsive-content-max": profile.contentMax,
1031:     "--responsive-media-max": profile.mediaMax,
1032:     "--responsive-touch-target": profile.touchTarget
1033:   };
1034: }
```

hasPublicTemplate (script.js, lines 1036-1038)

hasPublicTemplate part of the public website runtime. In this file, it appears around lines 1036-1038 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateId. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1037: return Boolean(projectTemplateId(project));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1036: function hasPublicTemplate(project) {  
1037:   return Boolean(projectTemplateId(project));  
1038: }
```

projectTemplateVisual (script.js, lines 1040-1042)

projectTemplateVisual part of the public website runtime. In this file, it appears around lines 1040-1042 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1041: return project?.portfolioView?.template?.visual || project?.templateVisual || null;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1040: function projectTemplateVisual(project) {  
1041:   return project?.portfolioView?.template?.visual || project?.templateVisual || null;  
1042: }
```

projectTemplateStyle (script.js, lines 1044-1062)

projectTemplateStyle part of the public website runtime. In this file, it appears around lines 1044-1062 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectTemplateVisual, responsiveStyleValues, entries, filter, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1058: return Object.entries(values).

Important local values include visual at line 1045; values at line 1046. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1044: function projectTemplateStyle(project, accent) {  
1045:   const visual = projectTemplateVisual(project) || {};  
1046:   const values = {  
1047:     "--accent": accent,  
1048:     "--template-bg": visual.background,  
1049:     "--template-panel": visual.panel,  
1050:     "--template-accent": visual.accent,  
1051:     "--template-hover": visual.hover,  
1052:     "--template-click": visual.click,  
1053:     "--template-click-text": visual.clickText,  
1054:     "--template-line": visual.line,  
1055:     "--template-text": visual.text,  
1056:     ...responsiveStyleValues(project)  
1057:   };  
1058:   return Object.entries(values)  
1059:     .filter(([key, value]) => value)  
1060:     .map(([key, value]) => `${key}: ${value}`)  
1061:     .join("; ");  
1062: }
```

applyProjectTemplateToElement (script.js, lines 1064-1098)

applyProjectTemplateToElement part of the public website runtime. In this file, it appears around lines 1064-1098 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, startsWith, forEach, remove, projectTemplateClass, split, add, projectResponsiveClass, projectTemplateVisual, responsiveStyleValues, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1065: if (!element) return;.

Important local values include visual at line 1078; values at line 1079. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1064: function applyProjectTemplateToElement(element, project, accent) {
1065:   if (!element) return;
1066:   [...element.classList]
1067:     .filter((className) =>
1068:       className === "project-template" ||
1069:       className === "project-template-white" ||
1070:       className === "responsive-project" ||
1071:       className.startsWith("project-template-") ||
1072:       className.startsWith("responsive-density-") ||
1073:       className.startsWith("responsive-card-")
1074:     )
1075:     .forEach((className) => element.classList.remove(className));
1076:   projectTemplateClass(project).split(" ").forEach((className) => element.classList.add(className));
1077:   projectResponsiveClass(project).split(" ").forEach((className) => element.classList.add(className));
1078:   const visual = projectTemplateVisual(project) || {};
1079:   const values = {
1080:     "--accent": accent,
1081:     "--template-bg": visual.background,
1082:     "--template-panel": visual.panel,
1083:     "--template-accent": visual.accent,
1084:     "--template-hover": visual.hover,
1085:     "--template-click": visual.click,
1086:     "--template-click-text": visual.clickText,
1087:     "--template-line": visual.line,
1088:     "--template-text": visual.text,
1089:     ...responsiveStyleValues(project)
1090:   };
1091:   Object.entries(values).forEach(([key, value]) => {
1092:     if (value) {
1093:       element.style.setProperty(key, value);
1094:     } else {
1095:       element.style.removeProperty(key);
1096:     }
1097:   });
1098: }
```

parsedItemTerms (script.js, lines 1100-1110)

parsedItemTerms part of the public website runtime. In this file, it appears around lines 1100-1110 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flatMap. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1101: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1100: function parsedItemTerms(item) {
1101:   return [
1102:     item?.title,
1103:     item?.description,
1104:     item?.meta,
1105:     item?.url,
1106:     item?.kind,
1107:     ...(item?.rich?.blocks || []).flatMap((block) => [block.text, block.formula, block.title,
block.caption]),
1108:     ...(item?.children || []).flatMap(parsedItemTerms)
1109:   ];
1110: }
```

flattenProject (script.js, lines 1112-1165)

flattenProject part of the public website runtime. In this file, it appears around lines 1112-1165 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugLabel, electronicsSearchKeywords, flatMap, map, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1138: return [.

Important local values include categoryLabel at line 1113; electronicsKeywords at line 1114; richSummaryTerms at line 1117; parsedChildTerms at line 1123; design at line 1126; designTerms at line 1127. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1112: function flattenProject(project) {
1113:   const categoryLabel = slugLabel(project.category);
1114:   const electronicsKeywords = window.electronicsSearchKeywords
1115:     ? window.electronicsSearchKeywords(project, categoryLabel)
1116:     : [];
1117:   const richSummaryTerms = (project.summaryRich?.blocks || []).flatMap((block) => [
1118:     block.text,
1119:     block.formula,
1120:     block.title,
1121:     block.caption
1122:   ]);
1123:   const parsedChildTerms = (project.portfolioView?.sections || []).flatMap((section) =>
1124:     (section.items || []).flatMap(parsedItemTerms)
1125:   );
1126:   const design = project.design || {};
1127:   const designTerms = [
1128:     design.brief?.summary,
1129:     ...(design.brief?.files || []).flatMap((item) => [item.title, item.description, item.url]),
1130:     design.documentation?.summary,
1131:     ...(design.documentation?.files || []).flatMap((item) => [item.title, item.description, item.url]),
1132:     ...(design.documentation?.references || []).flatMap((item) => [item.title, item.description,
item.url]),
1133:     ...(design.documentation?.mathAnalysis || []).flatMap((item) => [item.title, item.description]),
1134:     design.simulation?.summary,
1135:     ...(design.simulation?.files || []).flatMap((item) => [item.title, item.description, item.url]),
1136:     ...(design.simulation?.results || []).flatMap((item) => [item.title, item.description, item.url])
1137:   ];
1138:   return [
1139:     project.id,
1140:     project.title,
1141:     categoryLabel,
1142:     project.status,
1143:     project.summary,
1144:     ...richSummaryTerms,
1145:     ...parsedChildTerms,
1146:     ...designTerms,
1147:     ...(project.focus || []),
1148:     ...(project.highlights || []),
1149:     ...(project.tools || []).map((item) => typeof item === "string" ? item : [item.name, item.title,
item.label, item.description].filter(Boolean).join(" ")),
1150:     ...(project.languages || []),
1151:     ...(project.documents || []).flatMap((item) => [item.title, item.type, item.status, item.url]),
1152:     ...(project.tests || []).flatMap((item) => [item.name, item.method, item.status, item.result,
item.artifact]),
1153:     ...(project.pcbns || []).flatMap((item) => [item.name, item.revision, item.status, item.artifact]),
1154:     ...(project.media || []).flatMap((item) => [item.title, item.caption, item.url]),
1155:     ...(project.sections || []).flatMap((section) => [
1156:       section.title,
1157:       section.description,
1158:       ...(section.items || []).flatMap((item) => [item.title, item.description, item.type, item.status,
item.url])
1159:     ]),
1160:     ...(project.links || []).flatMap((item) => [item.label, item.url]),
1161:     ...electronicsKeywords
1162:   ]
1163:   .map(normalize)
1164:   .join(" ");
1165: }
```

richSummaryTerms (script.js, lines 1117-1126)

richSummaryTerms part of the public website runtime. In this file, it appears around lines 1117-1126 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flatMap. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include richSummaryTerms at line 1117; parsedChildTerms at line 1123; design at line 1126. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1117:   const richSummaryTerms = (project.summaryRich?.blocks || []).flatMap((block) => [
1118:     block.text,
1119:     block.formula,
```

```

1120:     block.title,
1121:     block.caption
1122:   });
1123:   const parsedChildTerms = (project.portfolioView?.sections || []).flatMap((section) =>
1124:     (section.items || []).flatMap(parsedItemTerms)
1125:   );
1126:   const design = project.design || {};

```

parsedChildTerms (script.js, lines 1123-1126)

parsedChildTerms part of the public website runtime. In this file, it appears around lines 1123-1126 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flatMap. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include parsedChildTerms at line 1123; design at line 1126. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1123:   const parsedChildTerms = (project.portfolioView?.sections || []).flatMap((section) =>
1124:     (section.items || []).flatMap(parsedItemTerms)
1125:   );
1126:   const design = project.design || {};

```

projectMatches (script.js, lines 1167-1169)

projectMatches part of the public website runtime. In this file, it appears around lines 1167-1169 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flattenProject, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1168: return !query || flattenProject(project).includes(query);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1167: function projectMatches(project, query) {
1168:   return !query || flattenProject(project).includes(query);
1169: }

```

projectVisible (script.js, lines 1171-1174)

projectVisible part of the public website runtime. In this file, it appears around lines 1171-1174 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references projectMatches. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1173: return categoryMatch && projectMatches(project, query);.

Important local values include categoryMatch at line 1172. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1171: function projectVisible(project, query) {
1172:   const categoryMatch = activeFilter === "all" || project.category === activeFilter;
1173:   return categoryMatch && projectMatches(project, query);
1174: }

```

normalizeCategory (script.js, lines 1176-1184)

normalizeCategory validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 1176-1184 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, normalizeTextColor, toLowerCase, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1178: return {.

Important local values include label at line 1177. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1176: function normalizeCategory(category = {}) {
1177:   const label = String(category.label || category.title || category.id || "Project category").trim() ||
"Project category";
1178:   return {
1179:     accent: normalizeTextColor(category.accent || "") || "#1677a8",
1180:     description: String(category.description || "").trim(),
1181:     id: String(category.id || label.toLowerCase().replace(/^[^a-z0-9]+/g, "-").replace(/^-|-$/g, "")) ||
"category"),
1182:     label
1183:   };
1184: }
```

hydrateProjectCategories (script.js, lines 1186-1201)

hydrateProjectCategories part of the public website runtime. In this file, it appears around lines 1186-1201 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, Set, forEach, trim, has, split, filter, charAt, toUpperCase, slice, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1191: if (!id || knownIds.has(id)) return;; line 1200: return normalized;.

Important local values include normalized at line 1187; knownIds at line 1188; id at line 1190; label at line 1192. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1186: function hydrateProjectCategories(rawCategories = [], rawProjects = []) {
1187:   const normalized = rawCategories.map(normalizeCategory);
1188:   const knownIds = new Set(normalized.map((category) => category.id));
1189:   rawProjects.forEach((project) => {
1190:     const id = String(project?.category || "").trim();
1191:     if (!id || knownIds.has(id)) return;
1192:     const label = id
1193:       .split(/[-_\s]+/)
1194:       .filter(Boolean)
1195:       .map((part) => part.charAt(0).toUpperCase() + part.slice(1))
1196:       .join(" ") || "Project category";
1197:     normalized.push(normalizeCategory({ id, label, description: "Project category" }));
1198:     knownIds.add(id);
1199:   });
1200:   return normalized;
1201: }
```

setActiveFilter (script.js, lines 1203-1209)

setActiveFilter part of the public website runtime. In this file, it appears around lines 1203-1209 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some, forEach, and toggle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1203: function setActiveFilter(filter = "all") {
1204:   activeFilter = filter;
1205:   if (activeFilter !== "all" && !categories.some((category) => category.id === activeFilter)) {
1206:     activeFilter = "all";
1207:   }
1208:   filterButtons.forEach((item) => item.classList.toggle("active", item.dataset.filter === activeFilter));
1209: }
```

bindFilterButtons (script.js, lines 1211-1220)

bindFilterButtons part of the public website runtime. In this file, it appears around lines 1211-1220 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelectorAll, forEach, addEventListener, setActiveFilter, renderProjects, and updateSearchDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1211: function bindFilterButtons() {
1212:   filterButtons = [...document.querySelectorAll(".filter-button")];
1213:   filterButtons.forEach((button) => {
1214:     button.addEventListener("click", () => {
1215:       setActiveFilter(button.dataset.filter || "all");
1216:       renderProjects();
1217:       updateSearchDropdown();
1218:     });
1219:   });
1220: }
```

renderCategoryFilters (script.js, lines 1222-1241)

renderCategoryFilters renders ui, markup, or display output. In this file, it appears around lines 1222-1241 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, join, escapeHtml, bindFilterButtons, and setActiveFilter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1222: function renderCategoryFilters() {
1223:   if (projectTrackCount) {
1224:     projectTrackCount.textContent = `${categories.length} Track${categories.length === 1 ? "" : "s"}`;
1225:   }
1226:   if (projectTrackLabels) {
1227:     projectTrackLabels.textContent = categories.length
1228:       ? categories.map((category) => category.label).join(", ")
1229:       : "project categories";
1230:   }
1231:   if (projectFilters) {
1232:     projectFilters.innerHTML = `
1233:     <button class="filter-button active" type="button" data-filter="all">All</button>
1234:     ${categories.map((category) => `
1235:     <button class="filter-button" type="button" data-
filter="${escapeHtml(category.id)}">${escapeHtml(category.label)}</button>
1236:     `).join("")}
1237:     `;
1238:   }
1239:   bindFilterButtons();
1240:   setActiveFilter(activeFilter);
1241: }
```

flattenSearchText (script.js, lines 1243-1251)

flattenSearchText part of the public website runtime. In this file, it appears around lines 1243-1251 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flat, filter, map, stringify, join, replace, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1244: return values.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1243: function flattenSearchText(values = []) {
1244:   return values
1245:     .flat(Infinity)
1246:     .filter((value) => value !== null && value !== undefined)
1247:     .map((value) => typeof value === "object" ? JSON.stringify(value) : String(value))
1248:     .join(" ")
1249:     .replace(/\s+/g, " ")
1250:     .trim();
1251: }
```

richTextTerms (script.js, lines 1253-1264)

richTextTerms part of the public website runtime. In this file, it appears around lines 1253-1264 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flatMap. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1254: return (rich?.blocks || []).flatMap((block) => [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1253: function richTextTerms(rich) {
1254:   return (rich?.blocks || []).flatMap((block) => [
1255:     block.text,
1256:     block.html,
1257:     block.formula,
1258:     block.code,
1259:     block.language,
1260:     block.title,
1261:     block.caption,
1262:     block.url
1263:   ]);
1264: }
```

addSearchEntry (script.js, lines 1266-1276)

addSearchEntry part of the public website runtime. In this file, it appears around lines 1266-1276 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flattenSearchText, normalize, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1268: if (!entry.title && !text) return null;; line 1275: return storedEntry;.

Important local values include text at line 1267; storedEntry at line 1269. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1266: function addSearchEntry(entries, entry) {
1267:   const text = flattenSearchText([entry.title, entry.type, entry.context, entry.text, entry.url]);
1268:   if (!entry.title && !text) return null;
1269:   const storedEntry = {
1270:     ...entry,
1271:     normalizedText: normalize(text),
1272:     text
```

```

1273:   };
1274:   entries.push(storedEntry);
1275:   return storedEntry;
1276: }

```

searchSnippet (script.js, lines 1278-1286)

searchSnippet part of the public website runtime. In this file, it appears around lines 1278-1286 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, trim, normalize, indexOf, slice, max, and min. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1280: if (!clean) return ""; line 1282: if (index < 0) return clean.slice(0, 150); line 1285: return `\${start ? "...": ""}\${clean.slice(start, end)}\${end < clean.length ? "...": ""}`;

Important local values include clean at line 1279; index at line 1281; start at line 1283; end at line 1284. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1278: function searchSnippet(text = "", query = "") {
1279:   const clean = String(text || "").replace(/\s+/g, " ").trim();
1280:   if (!clean) return "";
1281:   const index = normalize(clean).indexOf(normalize(query));
1282:   if (index < 0) return clean.slice(0, 150);
1283:   const start = Math.max(0, index - 54);
1284:   const end = Math.min(clean.length, index + query.length + 80);
1285:   return `${start ? "...": ""}${clean.slice(start, end)}${end < clean.length ? "...": ""}`;
1286: }

```

entryScore (script.js, lines 1288-1314)

entryScore part of the public website runtime. In this file, it appears around lines 1288-1314 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, trim, split, filter, startsWith, includes, and forEach. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1290: if (!normalizedQuery) return 0; line 1313: return score;

Important local values include normalizedQuery at line 1289; title at line 1291; type at line 1292; context at line 1293; text at line 1294; words at line 1295; score at line 1296. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1288: function entryScore(entry, query) {
1289:   const normalizedQuery = normalize(query).trim();
1290:   if (!normalizedQuery) return 0;
1291:   const title = normalize(entry.title);
1292:   const type = normalize(entry.type);
1293:   const context = normalize(entry.context);
1294:   const text = entry.normalizedText || "";
1295:   const words = normalizedQuery.split(/\s+/).filter(Boolean);
1296:   let score = 0;
1297:
1298:   if (title === normalizedQuery) score += 160;
1299:   if (title.startsWith(normalizedQuery)) score += 120;
1300:   if (title.includes(normalizedQuery)) score += 90;
1301:   if (context.includes(normalizedQuery)) score += 55;
1302:   if (type.includes(normalizedQuery)) score += 35;
1303:   if (text.includes(normalizedQuery)) score += 45;
1304:   words.forEach((word) => {
1305:     if (title.includes(word)) score += 16;
1306:     if (context.includes(word)) score += 10;
1307:     if (text.includes(word)) score += 6;
1308:   });
1309:
1310:   if (score > 0 && entry.kind === "file") score += 18;
1311:   if (score > 0 && entry.kind === "section") score += 14;
1312:   if (score > 0 && entry.kind === "project") score += 12;
1313:   return score;
1314: }

```

entryMatches (script.js, lines 1316-1318)

entryMatches part of the public website runtime. In this file, it appears around lines 1316-1318 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references entryScore. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1317: return entryScore(entry, query) > 0;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1316: function entryMatches(entry, query) {
1317:   return entryScore(entry, query) > 0;
1318: }
```

fileIsBrowserSearchable (script.js, lines 1320-1323)

fileIsBrowserSearchable part of the public website runtime. In this file, it appears around lines 1320-1323 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, and toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1322: return /\.txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|spice|cir|net|asc|sch|kicad_sch|kicad_pcb\$/i.test(clean);.

Important local values include clean at line 1321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1320: function fileIsBrowserSearchable(url = "") {
1321:   const clean = String(url || "").split(/[?#]/)[0].toLowerCase();
1322:   return /\.txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhd|?|spice|cir|net|asc|sch|k
icad_sch|kicad_pcb$/i.test(clean);
1323: }
```

indexSearchableFileText (script.js, lines 1325-1338)

indexSearchableFileText part of the public website runtime. In this file, it appears around lines 1325-1338 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileIsBrowserSearchable, startsWith, fetch, text, flattenSearchText, slice, normalize, and updateSearchDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1326: if (!entry.url || !fileIsBrowserSearchable(entry.url)) return;; line 1327: if (!/^https?:?\/\//i.test(entry.url) && !entry.url.startsWith(window.location.origin)) return;; line 1330: if (!response.ok) return;.

Important local values include response at line 1329; text at line 1331. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1325: async function indexSearchableFileText(entry) {
1326:   if (!entry.url || !fileIsBrowserSearchable(entry.url)) return;
1327:   if (!/^https?:?\/\//i.test(entry.url) && !entry.url.startsWith(window.location.origin)) return;
1328:   try {
1329:     const response = await fetch(entry.url);
1330:     if (!response.ok) return;
1331:     const text = await response.text();
1332:     entry.text = flattenSearchText([entry.text, text.slice(0, 60000)]);
1333:     entry.normalizedText = normalize(entry.text);
1334:     updateSearchDropdown();
1335:   } catch {
1336:     // Files remain searchable by title, path, caption, and catalog text if direct text fetch is
unavailable.
1337:   }
1338: }
```

addProjectSearchEntries (script.js, lines 1340-1414)

addProjectSearchEntries part of the public website runtime. In this file, it appears around lines 1340-1414 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugLabel, richTextTerms, electronicsSearchKeywords, addSearchEntry, filter, sectionHasRenderableContent, forEach, displayTitle, parsedItemTerms, fileNameFromUrl, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include categoryLabel at line 1341; baseContext at line 1342; projectTerms at line 1343; sections at line 1364; walkItems at line 1376; nextPath at line 1378; title at line 1379; url at line 1380; plus 5 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1340: function addProjectSearchEntries(entries, project) {
1341:   const categoryLabel = slugLabel(project.category);
1342:   const baseContext = `${categoryLabel} / ${project.title}`;
1343:   const projectTerms = [
1344:     project.id,
1345:     project.summary,
1346:     richTextTerms(project.summaryRich),
1347:     project.focus,
1348:     project.highlights,
1349:     project.tools,
1350:     project.languages,
1351:     project.links,
1352:     window.electronicsSearchKeywords ? window.electronicsSearchKeywords(project, categoryLabel) : []
1353:   ];
1354:
1355:   addSearchEntry(entries, {
1356:     context: categoryLabel,
1357:     kind: "project",
1358:     projectId: project.id,
1359:     title: project.title,
1360:     type: "Project",
1361:     text: projectTerms
1362:   });
1363:
1364:   const sections = (project.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
1365:   sections.forEach((section, sectionIndex) => {
1366:     addSearchEntry(entries, {
1367:       context: baseContext,
1368:       kind: "section",
1369:       projectId: project.id,
1370:       sectionIndex: String(sectionIndex),
1371:       title: displayTitle(section.title, "Section"),
1372:       type: "Project section",
1373:       text: [section.description, richTextTerms(section.rich), parsedItemTerms(section)]
1374:     });
1375:
1376:     const walkItems = (items = [], path = [], parentTitles = []) => {
1377:       items.forEach((item, index) => {
1378:         const nextPath = [...path, index];
1379:         const title = displayTitle(item.title || item.name || item.label || fileNameFromUrl(item.url ||
"", "Project item");
1380:         const url = item.url(item);
1381:         const context = [baseContext, displayTitle(section.title, "Section"),
...parentTitles].filter(Boolean).join(" / ");
1382:         const entry = {
1383:           context,
1384:           kind: url ? "file" : "subsection",
1385:           projectId: project.id,
1386:           resourcePath: pathToString(nextPath),
1387:           sectionIndex: String(sectionIndex),
1388:           title,
1389:           type: url ? "File or uploaded asset" : "Subsection",
1390:           url,
1391:           text: [item.description, item.meta, item.kind, item.type, url, richTextTerms(item.rich),
parsedItemTerms(item)]
1392:         };
1393:         const storedEntry = addSearchEntry(entries, entry);
1394:         if (url && storedEntry) indexSearchableFileText(storedEntry);
1395:         walkItems(nodeChildren(item), nextPath, [...parentTitles, title]);
1396:       });
1397:     };
1398:     walkItems(section.items || []);
1399:   });
1400:
1401:   uniqueDownloads(collectDownloadsFromValue(project, "Project file", [])).forEach((download) => {
1402:     const entry = {
```



```

1403:     context: baseContext,
1404:     kind: "file",
1405:     projectId: project.id,
1406:     title: download.title || fileNameFromUrl(download.url),
1407:     type: download.type || "Uploaded file",
1408:     url: download.url,
1409:     text: [download.description, download.status, download.url]
1410:   });
1411:   const storedEntry = addSearchEntry(entries, entry);
1412:   if (storedEntry) indexSearchableFileText(storedEntry);
1413: });
1414: }

```

sections (script.js, lines 1364-1399)

sections part of the public website runtime. In this file, it appears around lines 1364-1399 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, sectionHasRenderableContent, forEach, addSearchEntry, displayTitle, richTextTerms, parsedItemTerms, fileNameFromUrl, itemUrl, join, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include sections at line 1364; walkItems at line 1376; nextPath at line 1378; title at line 1379; url at line 1380; context at line 1381; entry at line 1382; storedEntry at line 1393. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1364:   const sections = (project.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
1365:   sections.forEach((section, sectionIndex) => {
1366:     addSearchEntry(entries, {
1367:       context: baseContext,
1368:       kind: "section",
1369:       projectId: project.id,
1370:       sectionIndex: String(sectionIndex),
1371:       title: displayTitle(section.title, "Section"),
1372:       type: "Project section",
1373:       text: [section.description, richTextTerms(section.rich), parsedItemTerms(section)]
1374:     });
1375:
1376:     const walkItems = (items = [], path = [], parentTitles = []) => {
1377:       items.forEach((item, index) => {
1378:         const nextPath = [...path, index];
1379:         const title = displayTitle(item.title || item.name || item.label || fileNameFromUrl(item.url ||
""), "Project item");
1380:         const url = itemUrl(item);
1381:         const context = [baseContext, displayTitle(section.title, "Section"),
...parentTitles].filter(Boolean).join(" / ");
1382:         const entry = {
1383:           context,
1384:           kind: url ? "file" : "subsection",
1385:           projectId: project.id,
1386:           resourcePath: pathToString(nextPath),
1387:           sectionIndex: String(sectionIndex),
1388:           title,
1389:           type: url ? "File or uploaded asset" : "Subsection",
1390:           url,
1391:           text: [item.description, item.meta, item.kind, item.type, url, richTextTerms(item.rich),
parsedItemTerms(item)]
1392:         };
1393:         const storedEntry = addSearchEntry(entries, entry);
1394:         if (url && storedEntry) indexSearchableFileText(storedEntry);
1395:         walkItems(nodeChildren(item), nextPath, [...parentTitles, title]);
1396:       });
1397:     };
1398:     walkItems(section.items || []);
1399:   });

```

addSiteSectionSearchEntries (script.js, lines 1416-1463)

addSiteSectionSearchEntries part of the public website runtime. In this file, it appears around lines 1416-1463 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `normalizeProfile`, `splice`, `forEach`, `addSearchEntry`, `indexSearchableFileText`, `filter`, and `richTextTerms`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

Important local values include `current` at line 1417; `pageSections` at line 1418; `resumeTextEntry` at line 1436. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1416: function addSiteSectionSearchEntries(entries) {
1417:   const current = normalizeProfile(profile || {});
1418:   const pageSections = [
1419:     { id: "top", title: heroTitle?.textContent || "Front page", type: "Page section", text:
[heroEyebrow?.textContent, heroCopy?.textContent] },
1420:     { id: "projects", title: "Engineering Projects", type: "Directory", text: "project directory
categories files sections" },
1421:     { id: "process", title: "Process", type: "Page section", text: "engineering process design testing
documentation" },
1422:     { id: "contact", title: "Contact", type: "Page section", text: [current.displayName, current.email,
current.phone, current.githubUrl, current.linkedinUrl, current.websiteUrl, current.contactIntro] },
1423:   ];
1424:   if (current.resumeUrl) {
1425:     pageSections.splice(2, 0, { id: "resume", title: "Resume", type: "Page section", text: "resume
professional profile document" });
1426:   }
1427:   pageSections.forEach((section) => addSearchEntry(entries, {
1428:     domTarget: section.id,
1429:     kind: "page",
1430:     title: section.title,
1431:     type: section.type,
1432:     text: section.text
1433:   }));
1434:
1435:   if (current.resumeUrl) {
1436:     const resumeTextEntry = addSearchEntry(entries, {
1437:       context: "Professional Profile / Resume",
1438:       kind: "file",
1439:       title: "Resume",
1440:       type: "Resume",
1441:       url: current.resumeUrl,
1442:       text: [current.displayName, current.portfolioLabel, current.contactIntro, current.resumeUrl]
1443:     });
1444:     if (resumeTextEntry) indexSearchableFileText(resumeTextEntry);
1445:   }
1446:
1447:   (siteSections || []).filter(siteSectionRenderable).forEach((section) => {
1448:     addSearchEntry(entries, {
1449:       domTarget: section.id,
1450:       kind: "page",
1451:       title: section.title || "Portfolio section",
1452:       type: "Portfolio section",
1453:       text: [section.description, richTextTerms(section.richDescription), section.links, section.assets]
1454:     });
1455:     (section.subsections || []).forEach((item) => addSearchEntry(entries, {
1456:       domTarget: section.id,
1457:       kind: "page",
1458:       title: item.title || "Portfolio subsection",
1459:       type: "Portfolio subsection",
1460:       text: [item.description, richTextTerms(item.richDescription), item.links]
1461:     }));
1462:   });
1463: }
```

rebuildSearchIndex (script.js, lines 1465-1470)

This function builds the searchable index used by the public website. It collects project titles, sections, rich text, files, profile content, public sections, and other searchable terms into structured entries.

A real search box needs an index rather than scanning random DOM text every time. This function makes search fast and able to jump to specific content.

It calls or references `addSiteSectionSearchEntries`, `forEach`, and `addProjectSearchEntries`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include `entries` at line 1466. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1465: function rebuildSearchIndex() {
1466:   const entries = [];
```

```

1467:   addSiteSectionSearchEntries(entries);
1468:   projects.forEach((project) => addProjectSearchEntries(entries, project));
1469:   searchableEntries = entries;
1470: }

```

searchResultsFor (script.js, lines 1472-1486)

This function ranks search entries for the visitor's query. It compares normalized terms, phrase matches, entry scores, and searchable text to return the closest results.

It makes search behave more like a small search engine instead of a simple browser find box.

It calls or references normalize, trim, Set, map, entryScore, filter, sort, localeCompare, join, has, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 1474: if (!normalizedQuery) return []; line 1476: return searchableEntries; line 1482: if (seen.has(key)) return false;; line 1484: return true;.

Important local values include normalizedQuery at line 1473; seen at line 1475; key at line 1481. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1472: function searchResultsFor(query) {
1473:   const normalizedQuery = normalize(query).trim();
1474:   if (!normalizedQuery) return [];
1475:   const seen = new Set();
1476:   return searchableEntries
1477:     .map((entry) => ({ ...entry, score: entryScore(entry, normalizedQuery) }))
1478:     .filter((entry) => entry.score > 0)
1479:     .sort((a, b) => b.score - a.score || a.title.localeCompare(b.title))
1480:     .filter((entry) => {
1481:       const key = [entry.kind, entry.projectId, entry.sectionIndex, entry.resourcePath, entry.domTarget,
entry.url, entry.title].join("|");
1482:       if (seen.has(key)) return false;
1483:       seen.add(key);
1484:       return true;
1485:     });
1486: }

```

assistantEndpoint (script.js, lines 1488-1494)

assistantEndpoint part of the public website runtime. In this file, it appears around lines 1488-1494 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, querySelector, and includes. Endpoint references found inside it are /api/portfolio-ai. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 1489: if (typeof window.OMB_AI_ENDPOINT === "string") return window.OMB_AI_ENDPOINT.trim(); line 1491: if (metaEndpoint.trim()) return String(metaEndpoint).trim(); line 1492: if (!["localhost", "127.0.0.1"].includes(window.location.hostname)) return "/api/portfolio-ai"; line 1493: return "/api/portfolio-ai";.

Important local values include metaEndpoint at line 1490. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1488: function assistantEndpoint() {
1489:   if (typeof window.OMB_AI_ENDPOINT === "string") return window.OMB_AI_ENDPOINT.trim();
1490:   const metaEndpoint = document.querySelector('meta[name="portfolio-ai-endpoint"]')?.content || "";
1491:   if (metaEndpoint.trim()) return String(metaEndpoint).trim();
1492:   if (!["localhost", "127.0.0.1"].includes(window.location.hostname)) return "/api/portfolio-ai";
1493:   return "/api/portfolio-ai";
1494: }

```

assistantSourceLabel (script.js, lines 1496-1505)

assistantSourceLabel part of the public website runtime. In this file, it appears around lines 1496-1505 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, trim, filter, slice, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1498: if (result.kind === "project" || result.kind === "page") return title;; line 1504: return [title, specificContext].filter(Boolean).join(" - ");.

Important local values include title at line 1497; contextParts at line 1499; specificContext at line 1503. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1496: function assistantSourceLabel(result = {}) {
1497:   const title = result.title || "Portfolio item";
1498:   if (result.kind === "project" || result.kind === "page") return title;
1499:   const contextParts = String(result.context || "")
1500:     .split("/")
1501:     .map((item) => item.trim())
1502:     .filter(Boolean);
1503:   const specificContext = contextParts.slice(-2).join(" / ");
1504:   return [title, specificContext].filter(Boolean).join(" - ");
1505: }
```

assistantProjectSummary (script.js, lines 1507-1522)

assistantProjectSummary part of the public website runtime. In this file, it appears around lines 1507-1522 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugLabel, filter, sectionHasRenderableContent, map, and displayTitle. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1512: return {.

Important local values include categoryLabel at line 1508; sections at line 1509. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1507: function assistantProjectSummary(project = {}) {
1508:   const categoryLabel = slugLabel(project.category);
1509:   const sections = (project.portfolioView?.sections || [])
1510:     .filter((section) => section.id !== "brief" && sectionHasRenderableContent(section))
1511:     .map((section) => displayTitle(section.title, "Section"));
1512:   return {
1513:     category: categoryLabel,
1514:     focus: project.focus || [],
1515:     id: project.id,
1516:     links: project.links || [],
1517:     sections,
1518:     summary: project.summary || "",
1519:     title: project.portfolioView?.title || project.title,
1520:     tools: project.tools || []
1521:   };
1522: }
```

assistantProjectTitleAcronyms (script.js, lines 1524-1535)

assistantProjectTitleAcronyms part of the public website runtime. In this file, it appears around lines 1524-1535 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, split, filter, Set, min, add, slice, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1534: return [...acronyms].filter((item) => item.length >= 2);.

Important local values include words at line 1525; acronyms at line 1528. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1524: function assistantProjectTitleAcronyms(title = "") {
1525:   const words = normalize(title)
1526:     .split(/\s+/)
1527:     .filter((word) => word.length > 2);
1528:   const acronyms = new Set();
1529:   for (let start = 0; start < words.length; start += 1) {
1530:     for (let length = 2; length <= Math.min(5, words.length - start); length += 1) {
1531:       acronyms.add(words.slice(start, start + length).map((word) => word[0]).join(""));
1532:     }
1533:   }
```

```
1534:   return [...acronyms].filter((item) => item.length >= 2);
1535: }
```

assistantProjectIsNamedInQuestion (script.js, lines 1537-1553)

assistantProjectIsNamedInQuestion part of the public website runtime. In this file, it appears around lines 1537-1553 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, Set, includes, split, assistantProjectTitleAcronyms, some, has, filter, and min. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 1539: if (!clean) return false;; line 1543: if (!title) return false;; line 1544: if (clean.includes(title)) return true;; line 1548: if (acronymMatch) return true;. There are 2 additional return statements in the function body.

Important local values include clean at line 1538; fillerWords at line 1540; rawTitle at line 1541; title at line 1542; cleanWords at line 1545; acronymMatch at line 1546; words at line 1549; matches at line 1551. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1537: function assistantProjectIsNamedInQuestion(question = "", project = {}) {
1538:   const clean = normalize(question);
1539:   if (!clean) return false;
1540:   const fillerWords = new Set(["with", "from", "into", "using", "project", "design", "system",
"systems"]);
1541:   const rawTitle = project.portfolioView?.title || project.title || project.id;
1542:   const title = normalize(rawTitle);
1543:   if (!title) return false;
1544:   if (clean.includes(title)) return true;
1545:   const cleanWords = new Set(clean.split(/\s+/));
1546:   const acronymMatch = assistantProjectTitleAcronyms(rawTitle)
1547:     .some((acronym) => cleanWords.has(acronym));
1548:   if (acronymMatch) return true;
1549:   const words = title.split(/\s+/).filter((word) => word.length > 3 && !fillerWords.has(word));
1550:   if (!words.length) return false;
1551:   const matches = words.filter((word) => clean.includes(word)).length;
1552:   return words.length <= 2 ? matches === words.length : matches >= Math.min(3, words.length);
1553: }
```

assistantNamedProjectIds (script.js, lines 1555-1559)

assistantNamedProjectIds part of the public website runtime. In this file, it appears around lines 1555-1559 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, assistantProjectIsNamedInQuestion, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1556: return projects.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1555: function assistantNamedProjectIds(question = "") {
1556:   return projects
1557:     .filter((project) => assistantProjectIsNamedInQuestion(question, project))
1558:     .map((project) => project.id);
1559: }
```

assistantProjectTitleMatches (script.js, lines 1561-1563)

assistantProjectTitleMatches part of the public website runtime. In this file, it appears around lines 1561-1563 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantNamedProjectIds. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1562: return assistantNamedProjectIds(question).length > 0;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1561: function assistantProjectTitleMatches(question = "") {
1562:   return assistantNamedProjectIds(question).length > 0;
1563: }
```

assistantUrlKind (script.js, lines 1565-1577)

assistantUrlKind part of the public website runtime. In this file, it appears around lines 1565-1577 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, toLowerCase, and fileIsBrowserSearchable. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 10 return statement(s). The most important visible returns are: line 1567: if (!clean) return "unknown";; line 1568: if (/github\.com|raw\.githubusercontent\.com|gist\.githubusercontent\.com/i.test(clean)) return "github";; line 1569: if (/linkedin\.com/i.test(clean)) return "linkedin";; line 1570: if (/\.png|jpe?g|gif|webp|svg)\$/i.test(clean)) return "image";. There are 6 additional return statements in the function body.

Important local values include clean at line 1566. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1565: function assistantUrlKind(url = "") {
1566:   const clean = String(url || "").split(/[?#]/)[0].toLowerCase();
1567:   if (!clean) return "unknown";
1568:   if (/github\.com|raw\.githubusercontent\.com|gist\.githubusercontent\.com/i.test(clean)) return
"github";
1569:   if (/linkedin\.com/i.test(clean)) return "linkedin";
1570:   if (/\.png|jpe?g|gif|webp|svg)$/i.test(clean)) return "image";
1571:   if (/\.pdf$/i.test(clean)) return "pdf";
1572:   if (/\.zip|7z|rar)$/i.test(clean)) return "archive";
1573:   if (/\.xlsx?|csv$/i.test(clean)) return "spreadsheet";
1574:   if (fileIsBrowserSearchable(clean)) return "text";
1575:   if (/^https?:\/\//i.test(clean)) return "webpage";
1576:   return "file";
1577: }
```

assistantAbsoluteUrl (script.js, lines 1579-1658)

assistantAbsoluteUrl part of the public website runtime. In this file, it appears around lines 1579-1658 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, looksLikeBareWebAddress, URL, assistantLinkRecordsFromValue, uniqueDownloads, collectDownloadsFromValue, forEach, push, assistantUrlKind, fileNameFromUrl, and 14 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 1581: if (!target) return "";; line 1583: return new URL(target, window.location.href).href;; line 1585: return target;; line 1594: if (!rawUrl) return;;. There are 5 additional return statements in the function body.

Important local values include target at line 1580; records at line 1590; downloads at line 1591; rawUrl at line 1593; text at line 1607; matches at line 1608; records at line 1622; add at line 1623; plus 6 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1579: function assistantAbsoluteUrl(url = "") {
1580:   const target = normalizeLinkTarget(url, { assumeWeb: looksLikeBareWebAddress(url) ||
/^https?:\/\//i.test(url) });
1581:   if (!target) return "";
1582:   try {
1583:     return new URL(target, window.location.href).href;
1584:   } catch {
1585:     return target;
1586:   }
1587: }
1588:
1589: function assistantLinkRecordsFromValue(value, context = "") {
1590:   const records = [];
1591:   const downloads = uniqueDownloads(collectDownloadsFromValue(value, "Portfolio asset", []));
```

```

1592:   downloads.forEach((download) => {
1593:     const rawUrl = download.url || download.href || download.path || "";
1594:     if (!rawUrl) return;
1595:     records.push({
1596:       context,
1597:       description: download.description || "",
1598:       kind: download.kind || assistantUrlKind(rawUrl),
1599:       label: download.title || download.label || fileNameFromUrl(rawUrl),
1600:       url: assistantAbsoluteUrl(rawUrl)
1601:     });
1602:   });
1603:   return records;
1604: }
1605:
1606: function assistantUrlsFromTextValue(value, context = "") {
1607:   const text = flattenSearchText([value]);
1608:   const matches = text.match(/(?:https?:\/\/(www\.)?[\s<>"]+\/gi) || [];
1609:   return [...new Set(matches.map((url) => url.replace(/[,;:!?]+$/g, "")))]
1610:     .slice(0, 16)
1611:     .map((url) => ({
1612:       context,
1613:       description: "",
1614:       kind: assistantUrlKind(url),
1615:       label: /github\.com\/i.test(url) ? "GitHub repository or profile" : fileNameFromUrl(url),
1616:       type: "Public link",
1617:       url: assistantAbsoluteUrl(url)
1618:     }));
1619: }
1620:
1621: function assistantPublicProfileLinks() {
1622:   const records = [];
1623:   const add = (item = {}, context = "") => {
1624:     const rawUrl = item.url || item.href || item.path || "";
1625:     if (!rawUrl) return;
1626:     const label = item.label || item.title || fileNameFromUrl(rawUrl);
1627:     records.push({
1628:       context,
1629:       kind: item.kind || assistantUrlKind(rawUrl),
1630:       label,
1631:       text: item.description || "",
1632:       url: assistantAbsoluteUrl(rawUrl)
1633:     });
1634:   };
1635:
1636:   const current = normalizeProfile(profile || {});
1637:   if (current.resumeUrl) add({ label: "Resume", kind: "resume", url: current.resumeUrl }, "Professional Profile");
1638:   if (current.githubUrl) add({ label: "GitHub", kind: "github", url: current.githubUrl }, "Professional Profile");
1639:   if (current.linkedinUrl) add({ label: "LinkedIn", kind: "linkedin", url: current.linkedinUrl }, "Professional Profile");
1640:   if (current.websiteUrl) add({ label: "Website", kind: "webpage", url: current.websiteUrl }, "Professional Profile");
1641:
1642:   (siteSections || []).forEach((section) => {
1643:     (section.links || []).forEach((link) => add(link, section.title || "Portfolio section"));
1644:   });
1645:
1646:   return records.filter((record, index, array) => (
1647:     record.url && array.findIndex((item) => item.url === record.url && item.label === record.label) ===
1648:     index
1649:   ));
1650: }
1651:
1652: function assistantProjectEvidence(project = {}) {
1653:   const downloads = uniqueDownloads(collectDownloadsFromValue(project, "Project asset", []));
1654:   const downloadRecords = downloads.slice(0, 26).map((download) => {
1655:     const rawUrl = download.url || "";
1656:     return {
1657:       description: download.description || "",
1658:       kind: assistantUrlKind(rawUrl),
1659:       label: download.title || download.label || fileNameFromUrl(rawUrl),

```

assistantLinkRecordsFromValue (script.js, lines 1589-1604)

assistantLinkRecordsFromValue part of the public website runtime. In this file, it appears around lines 1589-1604 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, collectDownloadsFromValue, forEach, push, assistantUrlKind, fileNameFromUrl, and assistantAbsoluteUrl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1594: if (!rawUrl) return;; line 1603: return records;.

Important local values include records at line 1590; downloads at line 1591; rawUrl at line 1593. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1589: function assistantLinkRecordsFromValue(value, context = "") {
1590:   const records = [];
1591:   const downloads = uniqueDownloads(collectDownloadsFromValue(value, "Portfolio asset", []));
1592:   downloads.forEach((download) => {
1593:     const rawUrl = download.url || download.href || download.path || "";
1594:     if (!rawUrl) return;
1595:     records.push({
1596:       context,
1597:       description: download.description || "",
1598:       kind: download.kind || assistantUrlKind(rawUrl),
1599:       label: download.title || download.label || fileNameFromUrl(rawUrl),
1600:       url: assistantAbsoluteUrl(rawUrl)
1601:     });
1602:   });
1603:   return records;
1604: }
```

assistantUrlsFromTextValue (script.js, lines 1606-1619)

assistantUrlsFromTextValue part of the public website runtime. In this file, it appears around lines 1606-1619 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references flattenSearchText, match, b, Set, map, replace, slice, assistantUrlKind, fileNameFromUrl, and assistantAbsoluteUrl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1609: return [...new Set(matches.map((url) => url.replace(/[,;:!?]+\$/g, "")))].

Important local values include text at line 1607; matches at line 1608. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1606: function assistantUrlsFromTextValue(value, context = "") {
1607:   const text = flattenSearchText([value]);
1608:   const matches = text.match(/b(?:https?:\/\/|www\.)[^s<"]+\/gi) || [];
1609:   return [...new Set(matches.map((url) => url.replace(/[,;:!?]+$/g, "")))]
1610:     .slice(0, 16)
1611:     .map((url) => ({
1612:       context,
1613:       description: "",
1614:       kind: assistantUrlKind(url),
1615:       label: /github\.com\/i.test(url) ? "GitHub repository or profile" : fileNameFromUrl(url),
1616:       type: "Public link",
1617:       url: assistantAbsoluteUrl(url)
1618:     }));
1619: }
```

assistantPublicProfileLinks (script.js, lines 1621-1649)

assistantPublicProfileLinks part of the public website runtime. In this file, it appears around lines 1621-1649 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileNameFromUrl, push, assistantUrlKind, assistantAbsoluteUrl, normalizeProfile, add, forEach, filter, and findIndex. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1625: if (!rawUrl) return;; line 1646: return records.filter((record, index, array) => (.

Important local values include records at line 1622; add at line 1623; rawUrl at line 1624; label at line 1626; current at line 1636. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1621: function assistantPublicProfileLinks() {
1622:   const records = [];
1623:   const add = (item = {}, context = "") => {
1624:     const rawUrl = item.url || item.href || item.path || "";
```



```

1625:     if (!rawUrl) return;
1626:     const label = item.label || item.title || fileNameFromUrl(rawUrl);
1627:     records.push({
1628:       context,
1629:       kind: item.kind || assistantUrlKind(rawUrl),
1630:       label,
1631:       text: item.description || "",
1632:       url: assistantAbsoluteUrl(rawUrl)
1633:     });
1634:   };
1635:
1636:   const current = normalizeProfile(profile || {});
1637:   if (current.resumeUrl) add({ label: "Resume", kind: "resume", url: current.resumeUrl }, "Professional Profile");
1638:   if (current.githubUrl) add({ label: "GitHub", kind: "github", url: current.githubUrl }, "Professional Profile");
1639:   if (current.linkedinUrl) add({ label: "LinkedIn", kind: "linkedin", url: current.linkedinUrl }, "Professional Profile");
1640:   if (current.websiteUrl) add({ label: "Website", kind: "webpage", url: current.websiteUrl }, "Professional Profile");
1641:
1642:   (siteSections || []).forEach((section) => {
1643:     (section.links || []).forEach((link) => add(link, section.title || "Portfolio section"));
1644:   });
1645:
1646:   return records.filter((record, index, array) => (
1647:     record.url && array.findIndex((item) => item.url === record.url && item.label === record.label) ===
index
1648:   ));
1649: }

```

assistantProjectEvidence (script.js, lines 1651-1667)

assistantProjectEvidence part of the public website runtime. In this file, it appears around lines 1651-1667 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, collectDownloadsFromValue, slice, map, assistantUrlKind, fileNameFromUrl, assistantAbsoluteUrl, assistantUrlsFromTextValue, filter, and findIndex. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1655: return {}; line 1664: return [...downloadRecords, ...textLinkRecords].filter((record, index, array) => (

Important local values include downloads at line 1652; downloadRecords at line 1653; rawUrl at line 1654; textLinkRecords at line 1663. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1651: function assistantProjectEvidence(project = {}) {
1652:   const downloads = uniqueDownloads(collectDownloadsFromValue(project, "Project asset", []));
1653:   const downloadRecords = downloads.slice(0, 26).map((download) => {
1654:     const rawUrl = download.url || "";
1655:     return {
1656:       description: download.description || "",
1657:       kind: assistantUrlKind(rawUrl),
1658:       label: download.title || download.label || fileNameFromUrl(rawUrl),
1659:       type: download.type || "",
1660:       url: assistantAbsoluteUrl(rawUrl)
1661:     };
1662:   });
1663:   const textLinkRecords = assistantUrlsFromTextValue([project.summary, project.links,
project.portfolioView], project.portfolioView?.title || project.title);
1664:   return [...downloadRecords, ...textLinkRecords].filter((record, index, array) => (
1665:     record.url && array.findIndex((item) => item.url === record.url) === index
1666:   ));
1667: }

```

assistantQuestionIsCasual (script.js, lines 1669-1675)

assistantQuestionIsCasual part of the public website runtime. In this file, it appears around lines 1669-1675 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1671: return
/^(hi|hello|hey|good\s+(morning|afternoon|evening))|yo|sup|thanks|thank\s+you|ok|okay)\b/.test(clean).

Important local values include clean at line 1670. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1669: function assistantQuestionIsCasual(question = "") {
1670:   const clean = normalize(question);
1671:   return
1672:   /^(hi|hello|hey|good\s+(morning|afternoon|evening))|yo|sup|thanks|thank\s+you|ok|okay)\b/.test(clean)
1673:   || /^(how\s+are\s+you|what\s+can\s+you\s+help.*|what\s+do\s+you\s+do|who\s+are\s+you|help|help\s+me|c
1674:   an\s+you\s+help|nice\s+to\s+meet\s+you)\?\?\$/ .test(clean)
1675:   || /^(what'?s|what\s+is|do\s+you\s+know)\s+my\s+name\?\?\$/ .test(clean)
1676:   || /who\s+am\s+i\?\?\$/ .test(clean);
1677: }
```

assistantQuestionHasPortfolioIntent (script.js, lines 1677-1682)

assistantQuestionHasPortfolioIntent part of the public website runtime. In this file, it appears around lines 1677-1682 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, assistantProjectTitleMatches, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1679: return assistantProjectTitleMatches(question).

Important local values include clean at line 1678. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1677: function assistantQuestionHasPortfolioIntent(question = "") {
1678:   const clean = normalize(question);
1679:   return assistantProjectTitleMatches(question)
1680:   || /\b(maurice|otieno|portfolio|resume|github|linkedin|contact|email|phone|project|projects|repo|repo
1681:   sitory|repositories|file|files|document|documents|artifact|artifacts|link|links|download|open|show|where|built|b
1682:   uild|created|designed|implemented)\b/.test(clean)
1683:   || /\b(your|his|maurice's)\s+(project|projects|resume|github|linkedin|portfolio|work|email|phone|cont
1684:   act|repo|repository|files?|documents?|links?)\b/.test(clean);
1685: }
```

assistantConversationSuggestsPortfolioContext (script.js, lines 1684-1688)

assistantConversationSuggestsPortfolioContext part of the public website runtime. In this file, it appears around lines 1684-1688 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice, some, b, and normalize. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1685: return history.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1684: function assistantConversationSuggestsPortfolioContext(history = []) {
1685:   return history
1686:   .slice(-4)
1687:   .some((item) => /\b(maurice|otieno|portfolio|project|resume|github|linkedin|electronics|engineering)\
1688:   b/.test(normalize(item?.content)));
1689: }
```

assistantQuestionAllowsPublicLookup (script.js, lines 1690-1695)

assistantQuestionAllowsPublicLookup part of the public website runtime. In this file, it appears around lines 1690-1695 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantQuestionIntent, normalize, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1692: return intent === "general_engineering".

Important local values include clean at line 1691. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1690: function assistantQuestionAllowsPublicLookup(question = "", intent = assistantQuestionIntent(question)) {
1691:   const clean = normalize(question);
1692:   return intent === "general_engineering"
1693:   || intent === "general_knowledge"
1694:   || /\b(github|linkedin|repo|repository|repositories|public\s+link|website|web\s+page|profile|resume|u
ploded\s+file|document|source\s+code)\b/.test(clean);
1695: }
```

assistantQuestionLooksConceptual (script.js, lines 1697-1702)

assistantQuestionLooksConceptual part of the public website runtime. In this file, it appears around lines 1697-1702 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, assistantQuestionIsEngineeringRelated, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1699: if (!assistantQuestionIsEngineeringRelated(question)) return false;; line 1700: return /^(what|what\s+is|what\s+are|what's|define|explain|describe|how|why|compare|differentiate)\b/.test(clean).

Important local values include clean at line 1698. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1697: function assistantQuestionLooksConceptual(question = "") {
1698:   const clean = normalize(question);
1699:   if (!assistantQuestionIsEngineeringRelated(question)) return false;
1700:   return
/^(what|what\s+is|what\s+are|what's|define|explain|describe|how|why|compare|differentiate)\b/.test(clean)
1701:   || /\b(definition|meaning|basics|overview|introduction|difference\s+between|how\s+does|how\s+do|why\s
+does|why\s+do)\b/.test(clean);
1702: }
```

assistantQuestionIntent (script.js, lines 1704-1717)

assistantQuestionIntent part of the public website runtime. In this file, it appears around lines 1704-1717 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantQuestionIsCasual, normalize, assistantQuestionHasPortfolioIntent, assistantConversationSuggestsPortfolioContext, b, assistantQuestionIsEngineeringRelated, and assistantQuestionLooksConceptual. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 1705: if (assistantQuestionIsCasual(question)) return "general_conversation";; line 1707: return "general_conversation";; line 1711: return "general_conversation";; line 1713: if (!hasPortfolioIntent && assistantQuestionIsEngineeringRelated(question)) return "general_engineering";. There are 3 additional return statements in the function body.

Important local values include hasPortfolioIntent at line 1709. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1704: function assistantQuestionIntent(question = "", history = assistantChatHistory) {
1705:   if (assistantQuestionIsCasual(question)) return "general_conversation";
1706:   if (/^(what'?s|what\s+is|do\s+you\s+know)\s+my\s+name\??$/i.test(normalize(question))) ||
/who\s+am\s+i\??$/i.test(normalize(question))) {
1707:     return "general_conversation";
1708:   }
1709:   const hasPortfolioIntent = assistantQuestionHasPortfolioIntent(question);
1710:   if (!hasPortfolioIntent && assistantConversationSuggestsPortfolioContext(history) &&
/\b(name|you|your|he|his|h|him|that|this|it)\b/.test(normalize(question))) {
1711:     return "general_conversation";
```

```

1712:   }
1713:   if (!hasPortfolioIntent && assistantQuestionIsEngineeringRelated(question)) return
"general_engineering";
1714:   if (!hasPortfolioIntent && assistantQuestionLooksConceptual(question)) return "general_engineering";
1715:   if (hasPortfolioIntent) return "portfolio_specific";
1716:   return "general_knowledge";
1717: }

```

assistantQuestionTokens (script.js, lines 1719-1730)

assistantQuestionTokens part of the public website runtime. In this file, it appears around lines 1719-1730 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, normalize, split, map, trim, filter, and has. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1726: return normalize(question).

Important local values include stopWords at line 1720. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1719: function assistantQuestionTokens(question = "") {
1720:   const stopWords = new Set([
1721:     "about", "again", "also", "and", "are", "can", "could", "does", "for", "from", "give", "have", "his",
1722:     "into", "link", "links", "maurice", "me", "my", "open", "otieno", "please", "portfolio", "project",
1723:     "projects", "show", "tell", "that", "the", "their", "this", "what", "where", "which", "who", "with",
1724:     "work", "your"
1725:   ]);
1726:   return normalize(question)
1727:     .split(/^[a-z0-9+#+.]+/i)
1728:     .map((token) => token.trim())
1729:     .filter((token) => token.length > 2 && !stopWords.has(token));
1730: }

```

assistantSpecificSourceTokens (script.js, lines 1732-1739)

assistantSpecificSourceTokens part of the public website runtime. In this file, it appears around lines 1732-1739 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, assistantQuestionTokens, filter, and has. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1738: return assistantQuestionTokens(question).filter((token) => !genericTokens.has(token));.

Important local values include genericTokens at line 1733. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1732: function assistantSpecificSourceTokens(question = "") {
1733:   const genericTokens = new Set([
1734:     "built", "build", "created", "designed", "did", "done", "engineering", "file", "files", "hardware",
1735:     "item", "items", "section", "sections", "system", "systems", "thing", "things", "tool", "tools",
1736:     "use", "used", "using", "website"
1737:   ]);
1738:   return assistantQuestionTokens(question).filter((token) => !genericTokens.has(token));
1739: }

```

assistantPromptTargetsProjectLanding (script.js, lines 1741-1746)

assistantPromptTargetsProjectLanding part of the public website runtime. In this file, it appears around lines 1741-1746 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1743: return `/b(open|show|view|go\s+to|take\s+me\s+to)\b/.test(clean)`.

Important local values include `clean` at line 1742. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1741: function assistantPromptTargetsProjectLanding(question = "") {
1742:   const clean = normalize(question);
1743:   return /b(open|show|view|go\s+to|take\s+me\s+to)\b/.test(clean)
1744:     && /b(project|overview|page)\b/.test(clean)
1745:     && !/b(code|source|repo|repository|github|file|files|document|documents|test|tests|result|results|pc
b|schematic|simulation|tool|tools|resume|linkedin)\b/.test(clean);
1746: }
```

assistantEntryRelationScore (script.js, lines 1748-1762)

`assistantEntryRelationScore` part of the public website runtime. In this file, it appears around lines 1748-1762 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `assistantSpecificSourceTokens`, `normalize`, `filter`, `join`, `includes`, and `b`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1761: return `relation`;

Important local values include `tokens` at line 1749; `haystack` at line 1750; `tokenHits` at line 1751; `relation` at line 1752. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1748: function assistantEntryRelationScore(result = {}, question = "") {
1749:   const tokens = assistantSpecificSourceTokens(question);
1750:   const haystack = normalize([result.title, result.context, result.type, result.text,
result.url].filter(Boolean).join(" "));
1751:   const tokenHits = tokens.filter((token) => haystack.includes(token)).length;
1752:   let relation = tokenHits * 30;
1753:
1754:   if (tokens.length && tokenHits === tokens.length) relation += 25;
1755:   if (Number(result.score || 0) >= 120) relation += 15;
1756:   if (Number(result.score || 0) >= 80) relation += 8;
1757:   if (result.kind === "project") relation += 12;
1758:   if (result.kind === "file" && /b(file|files|download|document|documents|code|source|github|repo|reposit
ory|test|tests|result|results|pcb|schematic|simulation)\b/.test(normalize(question))) {
1759:     relation += 16;
1760:   }
1761:   return relation;
1762: }
```

assistantNamedProjectTopicTokens (script.js, lines 1764-1776)

`assistantNamedProjectTopicTokens` part of the public website runtime. In this file, it appears around lines 1764-1776 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `assistantNamedProjectIds`, `Set`, `map`, `find`, `filter`, `forEach`, `normalize`, `split`, `add`, `assistantProjectTitleAcronyms`, and 2 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1775: return `assistantSpecificSourceTokens(question).filter((token) => !projectVocabulary.has(token))`;

Important local values include `projectVocabulary` at line 1765; `title` at line 1770. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1764: function assistantNamedProjectTopicTokens(question = "", namedIds = assistantNamedProjectIds(question)) {
1765:   const projectVocabulary = new Set();
1766:   namedIds
1767:     .map((id) => projects.find((project) => project.id === id))
1768:     .filter(Boolean)
1769:     .forEach((project) => {
1770:       const title = project.portfolioView?.title || project.title || project.id;
1771:       normalize(title).split(/\s+/).filter((word) => word.length > 2).forEach((word) =>
projectVocabulary.add(word));
1772:     });
1773:   return assistantSpecificSourceTokens(question).filter((token) => !projectVocabulary.has(token));
1774: }
```

```

1772:     assistantProjectTitleAcronyms(title).forEach((acronym) => projectVocabulary.add(acronym));
1773:     projectVocabulary.add(normalize(project.id));
1774:   });
1775:   return assistantSpecificSourceTokens(question).filter((token) => !projectVocabulary.has(token));
1776: }

```

assistantSourcesForDisplay (script.js, lines 1778-1813)

assistantSourcesForDisplay part of the public website runtime. In this file, it appears around lines 1778-1813 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantQuestionIntent, assistantSpecificSourceTokens, assistantNamedProjectIds, Set, assistantNamedProjectTopicTokens, b, normalize, assistantPromptTargetsProjectLanding, map, assistantEntryRelationScore, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 1779: if (intent !== "portfolio_specific") return []; line 1789: if (!namedProjectIds.size && !pageLinkIntent && tokens.length === 0) return []; line 1794: if (!result) return false;; line 1795: if (namedProjectIds.size && !namedProjectIds.has(result.projectId)) return false;. There are 8 additional return statements in the function body.

Important local values include tokens at line 1780; namedProjectIdList at line 1781; namedProjectIds at line 1782; namedProjectTopicTokens at line 1783; asksForLinks at line 1784; asksForProjectLanding at line 1785; pageLinkIntent at line 1786; asksForSpecificArtifact at line 1787; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1778: function assistantSourcesForDisplay(question = "", results = [], intent =
assistantQuestionIntent(question)) {
1779:   if (intent !== "portfolio_specific") return [];
1780:   const tokens = assistantSpecificSourceTokens(question);
1781:   const namedProjectIdList = assistantNamedProjectIds(question);
1782:   const namedProjectIds = new Set(namedProjectIdList);
1783:   const namedProjectTopicTokens = assistantNamedProjectTopicTokens(question, namedProjectIdList);
1784:   const asksForLinks = /\b(open|show|where|link|links|github|linkedin|resume|download|file|files|repo|rep
ository|source\s+code)\b/.test(normalize(question));
1785:   const asksForProjectLanding = assistantPromptTargetsProjectLanding(question);
1786:   const pageLinkIntent = /\b(resume|github|linkedin|contact|email|phone)\b/.test(normalize(question));
1787:   const asksForSpecificArtifact = /\b(code|source|repo|repository|github|file|files|document|documents|te
st|tests|result|results|pcb|schematic|simulation|tool|tools)\b/.test(normalize(question));
1788:
1789:   if (!namedProjectIds.size && !pageLinkIntent && tokens.length === 0) return [];
1790:
1791:   const filtered = results
1792:     .map((result) => ({ ...result, relation: assistantEntryRelationScore(result, question) }))
1793:     .filter((result) => {
1794:       if (!result) return false;
1795:       if (namedProjectIds.size && !namedProjectIds.has(result.projectId)) return false;
1796:       if (asksForProjectLanding) return result.kind === "project";
1797:       const haystack = normalize([result.title, result.context, result.type, result.text,
result.url].filter(Boolean).join(" "));
1798:       const tokenHits = tokens.filter((token) => haystack.includes(token)).length;
1799:       const topicHits = namedProjectTopicTokens.filter((token) => haystack.includes(token)).length;
1800:       if (namedProjectIds.size && namedProjectTopicTokens.length && result.kind !== "project" &&
topicHits === 0) return false;
1801:       if (namedProjectIds.size && namedProjectTopicTokens.length && result.kind === "project" &&
asksForSpecificArtifact) return false;
1802:       if (namedProjectIds.size && result.kind === "project") return true;
1803:       if (pageLinkIntent && tokenHits >= 1) return true;
1804:       if (!tokens.length) return false;
1805:       if (tokenHits >= Math.min(2, tokens.length)) return true;
1806:       if (asksForLinks && tokenHits >= 1 && result.relation >= 35) return true;
1807:       return result.relation >= 55;
1808:     });
1809:
1810:   return filtered
1811:     .sort((a, b) => b.relation - a.relation || Number(b.score || 0) - Number(a.score || 0))
1812:     .slice(0, asksForLinks ? 4 : 2);
1813: }

```

assistantContextForQuestion (script.js, lines 1815-1890)

This function chooses which portfolio context should accompany an AI question. It identifies project names, profile links, evidence files, public URLs, and intent signals related to the visitor's prompt.

The assistant should not attach generic portfolio links to every answer. This function decides what context is actually related.

It calls or references `assistantQuestionIntent`, includes, `searchResultsFor`, `slice`, `Set`, `map`, `filter`, `assistantNamedProjectIds`, `find`, `b`, and 10 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1833: `if (!wantsPublicCode) return 0;`; line 1836: `return bGithub - aGithub;`; line 1860: `return {`.

Important local values include `directResults` at line 1816; `projectIds` at line 1819; `matchedProjects` at line 1823; `includePortfolioLinkContext` at line 1827; `wantsPublicCode` at line 1828; `publicProfileFetches` at line 1829; `aGithub` at line 1834; `bGithub` at line 1835; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1815: function assistantContextForQuestion(question = "", intent = assistantQuestionIntent(question),
knownResults = null) {
1816:   const directResults = ["general_engineering", "general_knowledge",
"general_conversation"].includes(intent)
1817:   ? []
1818:   : (knownResults || searchResultsFor(question)).slice(0, 8);
1819:   const projectIds = [...new Set([
1820:     ...directResults.map((item) => item.projectId).filter(Boolean),
1821:     ...assistantNamedProjectIds(question)
1822:   ]]);
1823:   const matchedProjects = projectIds
1824:     .map((id) => projects.find((project) => project.id === id))
1825:     .filter(Boolean)
1826:     .slice(0, 5);
1827:   const includePortfolioLinkContext = intent === "portfolio_specific";
1828:   const wantsPublicCode = /\b(github|repo|repository|repositories|source\s+code|code|snippet|firmware|ver
ilog|systemverilog|python|javascript)\b/.test(normalize(question));
1829:   const publicProfileFetches = includePortfolioLinkContext
1830:     ? assistantPublicProfileLinks()
1831:     .map((item) => ({ ...item, question }));
1832:     .sort((a, b) => {
1833:       if (!wantsPublicCode) return 0;
1834:       const aGithub = /github/i.test(`${a.kind} || ""` ? 1 : 0;
1835:       const bGithub = /github/i.test(`${b.kind} || ""` ? 1 : 0;
1836:       return bGithub - aGithub;
1837:     });
1838:   : [];
1839:   const resultFetches = directResults.map((result) => ({
1840:     context: result.context,
1841:     kind: assistantUrlKind(result.url),
1842:     title: result.title,
1843:     type: result.type,
1844:     url: assistantAbsoluteUrl(result.url || ""),
1845:     question
1846:   }));
1847:   const projectFetches = matchedProjects.flatMap((project) => assistantProjectEvidence(project)
1848:     .filter((item) => ["text", "webpage", "github", "linkedin", "resume_text"].includes(item.kind))
1849:     .slice(0, 6)
1850:     .map((item) => ({ ...item, question })));
1851:   const sourceFetches = [
1852:     ...(wantsPublicCode ? publicProfileFetches : []),
1853:     ...resultFetches,
1854:     ...(wantsPublicCode ? [] : publicProfileFetches),
1855:     ...projectFetches
1856:   ].filter((item) => item.url);
1857:   const uniqueSourceFetches = sourceFetches.filter((item, index, array) => (
1858:     array.findIndex((candidate) => candidate.url === item.url) === index
1859:   )).slice(0, 14);
1860:   return {
1861:     categories: categories.map((category) => ({ id: category.id, label: category.label })),
1862:     intent,
1863:     profile: normalizeProfile(profile || {}),
1864:     knowledgeManifest: {
1865:       publicProfiles: includePortfolioLinkContext ? assistantPublicProfileLinks() : [],
1866:       publicSourcePolicy: "Only public profile and project links shown in the portfolio are used. GitHub
repository links can be expanded into repository metadata, README text, and selected public source files when
the visitor asks for code.",
1867:       matchedProjectEvidence: matchedProjects.map((project) => ({
1868:         evidence: assistantProjectEvidence(project),
1869:         projectId: project.id,
1870:         title: project.portfolioView?.title || project.title
1871:       })),
1872:       note: "Image records include filenames, captions, descriptions, and neighboring portfolio text. Use
a vision-capable backend before claiming visual details not present in captions or text."
1873:     },
1874:     portfolioContextPolicy: intent !== "portfolio_specific"
1875:       ? "Answer from general knowledge first. Do not lead with the portfolio owner's project context
unless the visitor explicitly asks to connect the concept to the portfolio."
1876:       : "Answer from the configured portfolio context first. Do not invent project details outside the
supplied context.",
1877:     projects: matchedProjects.map(assistantProjectSummary),
1878:     question,
1879:     results: directResults.map((result) => ({
1880:       context: result.context,
1881:       kind: result.kind,
1882:       text: searchSnippet(result.text, question),
```

```

1883:         title: result.title,
1884:         type: result.type,
1885:         url: result.url
1886:     })),
1887:     sourceFetches: uniqueSourceFetches,
1888:     siteSections: siteSections.filter(siteSectionRenderable).map((section) => ({ id: section.id, title:
section.title })))
1889: };
1890: }

```

assistantRemember (script.js, lines 1892-1901)

assistantRemember part of the public website runtime. In this file, it appears around lines 1892-1901 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, push, and slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1895: if (!cleanContent) return;.

Important local values include cleanRole at line 1893; cleanContent at line 1894. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1892: function assistantRemember(role, content = "") {
1893:   const cleanRole = role === "assistant" ? "assistant" : "user";
1894:   const cleanContent = String(content || "").trim();
1895:   if (!cleanContent) return;
1896:   assistantChatHistory.push({
1897:     role: cleanRole,
1898:     content: cleanContent.slice(0, 1400)
1899:   });
1900:   assistantChatHistory = assistantChatHistory.slice(-10);
1901: }

```

assistantConversationForRequest (script.js, lines 1903-1912)

assistantConversationForRequest part of the public website runtime. In this file, it appears around lines 1903-1912 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice, map, and push. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1911: return history;.

Important local values include history at line 1904. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

1903: function assistantConversationForRequest(currentQuestion = "", priorConversation = assistantChatHistory)
{
1904:   const history = priorConversation.slice(-8).map((item) => ({
1905:     role: item.role === "assistant" ? "assistant" : "user",
1906:     content: String(item.content || "").slice(0, 1200)
1907:   }));
1908:   if (currentQuestion) {
1909:     history.push({ role: "user", content: String(currentQuestion).slice(0, 1200) });
1910:   }
1911:   return history;
1912: }

```

assistantFallbackResults (script.js, lines 1914-1972)

assistantFallbackResults part of the public website runtime. In this file, it appears around lines 1914-1972 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references assistantQuestionIntent, normalize, trim, assistantNamedProjectIds, searchResultsFor, filter, includes, slice, map, find, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are

no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 1915: if (intent !== "portfolio_specific") return []; line 1917: if (!clean) return []; line 1922: if (focusedResults.length) return focusedResults.slice(0, 6); line 1924: if (results.length) return results.slice(0, 6);. There are 5 additional return statements in the function body.

Important local values include clean at line 1916; namedProjectIds at line 1918; results at line 1919; focusedResults at line 1921; category at line 1941. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1914: function assistantFallbackResults(question = "", intent = assistantQuestionIntent(question)) {
1915:   if (intent !== "portfolio_specific") return [];
1916:   const clean = normalize(question).trim();
1917:   if (!clean) return [];
1918:   const namedProjectIds = assistantNamedProjectIds(question);
1919:   let results = searchResultsFor(clean);
1920:   if (namedProjectIds.length && results.length) {
1921:     const focusedResults = results.filter((result) => namedProjectIds.includes(result.projectId));
1922:     if (focusedResults.length) return focusedResults.slice(0, 6);
1923:   }
1924:   if (results.length) return results.slice(0, 6);
1925:
1926:   if (namedProjectIds.length) {
1927:     return namedProjectIds
1928:       .map((id) => projects.find((project) => project.id === id))
1929:       .filter(Boolean)
1930:       .map((project) => ({
1931:         context: slugLabel(project.category),
1932:         kind: "project",
1933:         projectId: project.id,
1934:         text: flattenSearchText([project.summary, project.focus, project.tools, project.languages]),
1935:         title: project.portfolioView?.title || project.title,
1936:         type: "Project"
1937:       })))
1938:     .slice(0, 6);
1939:   }
1940:
1941:   const category = categories.find((item) => clean.includes(normalize(item.label)) ||
clean.includes(normalize(item.id)));
1942:   if (category) {
1943:     return projects
1944:       .filter((project) => project.category === category.id)
1945:       .map((project) => ({
1946:         context: category.label,
1947:         kind: "project",
1948:         projectId: project.id,
1949:         text: flattenSearchText([project.summary, project.focus, project.tools, project.languages]),
1950:         title: project.portfolioView?.title || project.title,
1951:         type: "Project"
1952:       })))
1953:     .slice(0, 6);
1954:   }
1955:
1956:   if (/resume|professional|contact|email|phone|github|linkedin|profile/.test(clean)) {
1957:     return searchResultsFor("resume contact github professional profile").slice(0, 6);
1958:   }
1959:
1960:   if (/project|work|portfolio|hardware|design/.test(clean)) {
1961:     return projects.slice(0, 6).map((project) => ({
1962:       context: slugLabel(project.category),
1963:       kind: "project",
1964:       projectId: project.id,
1965:       text: project.summary || "",
1966:       title: project.portfolioView?.title || project.title,
1967:       type: "Project"
1968:     })));
1969:   }
1970:
1971:   return [];
1972: }
```

assistantQuestionIsEngineeringRelated (script.js, lines 1974-1977)

assistantQuestionIsEngineeringRelated part of the public website runtime. In this file, it appears around lines 1974-1977 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1976: return /\b(op\s*\amp;|amplifier|analog|mixed\s*signal|adc|dac|filter|vco|oscillator|pwm|charger|rectifier|regulator|buck|boost|ldo|mosfet|bjt|transistor|diode|pcb|schematic|layout|ground|noise|frequency|g

ain|phase|tspice|.

Important local values include clean at line 1975. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1974: function assistantQuestionIsEngineeringRelated(question = "") {
1975:   const clean = normalize(question);
1976:   return /\b(op\s*amp|amplifier|analog|mixed\s*signal|adc|dac|filter|vco|oscillator|pwm|charger|rectifier
|regulator|buck|boost|ldo|mosfet|bjt|transistor|diode|pcb|schematic|layout|ground|noise|frequency|gain|phase|lts
pice|kicad|vivado|verilog|vhdl|fpga|asic|stm32|mcu|microcontroller|embedded|firmware|rtos|i2c|spi|uart|sensor|co
ntrol|signal|circuit|electronics|hardware|power)\b/.test(clean);
1977: }
```

assistantGeneralEngineeringAnswer (script.js, lines 1979-2091)

assistantGeneralEngineeringAnswer part of the public website runtime. In this file, it appears around lines 1979-2091 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, assistantQuestionIsEngineeringRelated, join, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 1981: if (!assistantQuestionIsEngineeringRelated(question)) return "";; line 1984: return [; line 2000: return [; line 2013: return [. There are 5 additional return statements in the function body.

Important local values include clean at line 1980. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
1979: function assistantGeneralEngineeringAnswer(question = "") {
1980:   const clean = normalize(question);
1981:   if (!assistantQuestionIsEngineeringRelated(question)) return "";
1982:
1983:   if (/b(emb|bedded\s+systems?)\b/.test(clean)) {
1984:     return [
1985:       "Embedded systems is a field of engineering focused on computers that are built into larger
products, machines, instruments, or control systems.",
1986:       "",
1987:       "Unlike a laptop or desktop computer, an embedded system is usually designed for a specific job. It
combines hardware, firmware, sensors, actuators, communication interfaces, and timing constraints so the
product can make measurements, control outputs, or respond to events reliably.",
1988:       "",
1989:       "Common examples include:",
1990:       "- A microcontroller controlling a motor drive.",
1991:       "- A sensor node collecting temperature, pressure, or motion data.",
1992:       "- A medical device running real-time measurement firmware.",
1993:       "- An automotive controller reading sensors and driving actuators.",
1994:       "",
1995:       "The important engineering ideas are timing, interrupts, power use, peripheral interfaces,
reliability, and how the firmware proves the hardware is behaving correctly."
1996:     ].join("\n");
1997:   }
1998:
1999:   if (/b(op\s*amp|amplifier|fully differential|differential)\b/.test(clean)) {
2000:     return [
2001:       "A useful way to think about an op amp is as a high-gain error-correcting block. With negative
feedback, the circuit around it sets the useful behavior: gain, filtering, buffering, summing, integration, or
differential conversion.",
2002:       "",
2003:       "For a fully differential op amp:",
2004:       "- The signal is carried on two opposite-phase outputs instead of one output referenced to
ground.",
2005:       "- The differential output improves noise immunity and dynamic range.",
2006:       "- A common-mode feedback loop is normally needed to hold the average output voltage at the desired
common-mode level.",
2007:       "",
2008:       "In portfolio terms, this connects most naturally to analog and mixed-signal design, especially
filters, VCO control paths, ADC front ends, and precision measurement circuits."
2009:     ].join("\n");
2010:   }
2011:
2012:   if (/b(pwm|vco|oscillator|frequency)\b/.test(clean)) {
2013:     return [
2014:       "PWM-driven VCO work is about converting a digital timing signal into an analog control quantity
and then into a frequency.",
2015:       "",
2016:       "The usual signal chain is:",
2017:       "- PWM duty cycle encodes the control value.",
2018:       "- A low-pass filter turns the PWM waveform into an average voltage.",
2019:       "- The VCO maps that voltage to oscillation frequency.",
2020:       "- Tests compare duty cycle, control voltage, frequency range, ripple, and stability.",
```

```

2021:     "",
2022:     "That is a mixed-signal problem because digital timing, analog filtering, oscillator behavior, and
measurement all interact."
2023:   ].join("\n");
2024: }
2025:
2026: if (/b(fpga|asic|verilog|vhdl|vivado|digital)\b/.test(clean)) {
2027:   return [
2028:     "FPGA and ASIC work both implement digital hardware, but the engineering tradeoffs are different.",
2029:     "",
2030:     "- FPGA design is reconfigurable, fast to test, and useful for prototyping or deployment when
flexibility matters.",
2031:     "- ASIC design targets a fixed silicon implementation, so verification, timing closure, power,
area, and design-for-test become much more permanent decisions.",
2032:     "- Verilog or VHDL describes hardware structure and behavior; it is not software running line by
line.",
2033:     "",
2034:     "For a recruiter, strong evidence includes block diagrams, RTL, simulation waveforms, timing
reports, verification plans, and notes explaining design tradeoffs."
2035:   ].join("\n");
2036: }
2037:
2038: if (/b(stm32|mcu|microcontroller|embedded|firmware|rtos|i2c|spi|uart)\b/.test(clean)) {
2039:   return [
2040:     "Embedded work is strongest when the firmware is tied clearly to hardware behavior.",
2041:     "",
2042:     "A good explanation usually covers:",
2043:     "- The MCU role in the system.",
2044:     "- The peripherals used, such as ADC, PWM, timers, I2C, SPI, UART, or GPIO.",
2045:     "- How timing, interrupts, sampling, power, and fault handling were managed.",
2046:     "- What was measured to prove the system worked.",
2047:     "",
2048:     "STM32CubeIDE is useful because it helps configure peripherals, generate startup code, debug
firmware, and manage the build flow around STM32 devices."
2049:   ].join("\n");
2050: }
2051:
2052: if (/b(pcb|layout|kicad|schematic|ground|noise)\b/.test(clean)) {
2053:   return [
2054:     "PCB design is where the schematic becomes a physical electrical system.",
2055:     "",
2056:     "The important ideas are:",
2057:     "- Current returns matter as much as current paths.",
2058:     "- Grounding, decoupling, trace width, component placement, and loop area affect noise and
stability.",
2059:     "- Analog, power, and switching sections should be arranged so noisy currents do not corrupt
sensitive measurements.",
2060:     "- A good portfolio PCB section should show schematic intent, layout decisions, board files, bring-
up notes, and test results.",
2061:     "",
2062:     "KiCad evidence is strongest when the design files are paired with clear measurements or bring-up
photos."
2063:   ].join("\n");
2064: }
2065:
2066: if (/b(charger|rectifier|ac\s*to\s*dc|regulator|buck|boost|ldo|power)\b/.test(clean)) {
2067:   return [
2068:     "An AC-to-DC charger or power supply usually has a few major stages.",
2069:     "",
2070:     "- Input protection and filtering handle transients and noise.",
2071:     "- Rectification converts AC into pulsating DC.",
2072:     "- Bulk capacitance smooths the rectified waveform.",
2073:     "- Regulation sets the desired output voltage or current.",
2074:     "- Load testing checks ripple, thermal behavior, regulation, and fault response.",
2075:     "",
2076:     "For portfolio presentation, the best proof is a clear schematic, expected waveforms, measured
output behavior, and a short explanation of component choices."
2077:   ].join("\n");
2078: }
2079:
2080: return [
2081:   "I can answer this as a general electronics or hardware-engineering question, even if it is not
directly tied to one saved project yet.",
2082:   "",
2083:   "The best way to approach it is:",
2084:   "- Define the signal or energy flow.",
2085:   "- Identify the active devices, passive networks, and control points.",
2086:   "- State what should be measured to prove the design works.",
2087:   "- Connect the explanation back to a project artifact: schematic, code, test data, PCB, or
simulation.",
2088:   "",
2089:   "If you ask about a specific circuit, tool, waveform, or failure mode, I can make the explanation
more concrete."
2090: ].join("\n");
2091: }

```

assistantLocalAnswer (script.js, lines 2093-2188)

This function builds a local fallback answer when the remote AI endpoint is unavailable or unnecessary.

The chat should still respond to greetings, basic portfolio questions, and common electronics prompts instead of failing silently when the backend is down.

It calls or references assistantQuestionIntent, normalizeProfile, normalize, b, join, assistantGeneralEngineeringAnswer, replace, Set, map, filter, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 11 return statement(s). The most important visible returns are: line 2098: return "Hi, what can I do for you?"; line 2101: return `I am an AI agent for \${ownerName}'s portfolio. I know the portfolio owner is \${ownerName}, but I do not know a visitor's personal name unless they tell me.`; line 2104: return `I am an AI agent for \${ownerName}'s portfolio. I can help with projects, resume links, public links, files, and related engineering questions.`; line 2106: return [. There are 7 additional return statements in the function body.

Important local values include ownerName at line 2094; clean at line 2096; generalAnswer at line 2118; generalAnswer at line 2128; projectIds at line 2152; matchedProjects at line 2153; projectNames at line 2156; fileCount at line 2157; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2093: function assistantLocalAnswer(question = "", results = [], intent = assistantQuestionIntent(question)) {
2094:   const ownerName = normalizeProfile(profile || {}).displayName || "the portfolio owner";
2095:   if (intent === "general_conversation") {
2096:     const clean = normalize(question);
2097:     if (/^(hi|hello|hey|yo|sup)\b/.test(clean)) {
2098:       return "Hi, what can I do for you?";
2099:     }
2100:     if (/^(what'?s|what\s+is|do\s+you\s+know)\s+my\s+name\??$/i.test(clean) ||
2101: /who\s+am\s+i\??$/i.test(clean)) {
2102:       return `I am an AI agent for ${ownerName}'s portfolio. I know the portfolio owner is ${ownerName},
but I do not know a visitor's personal name unless they tell me.`;
2103:     }
2104:     if (/^(who are you|what are you)\b/.test(clean)) {
2105:       return `I am an AI agent for ${ownerName}'s portfolio. I can help with projects, resume links,
public links, files, and related engineering questions.`;
2106:     }
2107:     return [
2108:       "I am here with you. I can help explore ${ownerName}'s portfolio, explain projects, summarize
files, open relevant sections, or answer related engineering questions.",
2109:       "",
2110:       "You can ask things like:",
2111:       "- What is embedded systems?",
2112:       "- Show me a project.",
2113:       "- What tools were used?",
2114:       "- Explain the difference between FPGA and ASIC design."
2115:     ].join("\n");
2116:   }
2117:   if (intent === "general_engineering") {
2118:     const generalAnswer = assistantGeneralEngineeringAnswer(question);
2119:     if (generalAnswer) return generalAnswer;
2120:     return [
2121:       "I can answer that as a general engineering question, but I need a little more detail to make the
answer useful.",
2122:       "",
2123:       "Try asking about a specific circuit, device, tool, protocol, waveform, or failure mode."
2124:     ].join("\n");
2125:   }
2126:   if (intent === "general_knowledge") {
2127:     const generalAnswer = assistantGeneralEngineeringAnswer(question);
2128:     if (generalAnswer) return generalAnswer;
2129:     return [
2130:       "I can answer that as a general question from the local fallback, but the live model will usually
give a deeper answer when it is reachable.",
2131:       "",
2132:       "Short answer:",
2133:       "- ${question.replace(/\?+$/, "")} is a topic I can discuss by breaking it into definition,
purpose, main parts, and practical examples.",
2134:       "- If the question connects to Maurice's portfolio, I will use the saved project context and links
as evidence.",
2135:       "- If it is broader than the portfolio, I will answer it as a general engineering or technology
concept first."
2136:     ].join("\n");
2137:   }
2138: }
2139:
2140: if (!projects.length) return "The project catalog is still loading. Try again in a moment.";
2141: if (!results.length) {
2142:   return [
2143:     "I could not find a strong portfolio match for \"${question}\".",
2144:     "",
2145:     "Try asking about:",
2146:     "- A project name or category.",
2147:     "- A tool such as LTspice, KiCad, Vivado, STM32CubeIDE, or GitHub.",
2148:     "- A hardware topic such as op amps, PCB testing, VCOs, embedded firmware, FPGA, ASIC, or power
circuits."
2149:   ].join("\n");
2150: }
2151:
```

```

2152: const projectIds = [...new Set(results.map((item) => item.projectId).filter(Boolean))];
2153: const matchedProjects = projectIds
2154:   .map((id) => projects.find((project) => project.id === id))
2155:   .filter(Boolean);
2156: const projectNames = matchedProjects.map((project) => project.portfolioView?.title ||
project.title).slice(0, 3);
2157: const fileCount = results.filter((item) => item.kind === "file").length;
2158: const sectionCount = results.filter((item) => item.kind === "section" || item.kind ===
"subsection").length;
2159: const pageCount = results.filter((item) => item.kind === "page").length;
2160: const parts = [];
2161:
2162: if (projectNames.length) {
2163:   parts.push(`The strongest portfolio match is ${projectNames.join(projectNames.length > 2 ? ", " : "
and ")}.`);
2164: } else if (pageCount) {
2165:   parts.push("The strongest matches are in the profile, resume, contact, or portfolio information
sections.");
2166: } else {
2167:   parts.push(`I found ${results.length} relevant portfolio item${results.length === 1 ? "" : "s"}.`);
2168: }
2169:
2170: parts.push("");
2171: parts.push("What I found:");
2172: if (sectionCount || fileCount) {
2173:   parts.push(`- ${sectionCount} section match${sectionCount === 1 ? "" : "es"}.`);
2174:   if (fileCount) parts.push(`- ${fileCount} file or link match${fileCount === 1 ? "" : "es"}.`);
2175: }
2176: if (pageCount) parts.push(`- ${pageCount} page or profile match${pageCount === 1 ? "" : "es"}.`);
2177: if (projectNames.length) {
2178:   parts.push(`- Best project candidate${projectNames.length === 1 ? "" : "s"}: ${projectNames.join(",
")}.`);
2179: }
2180:
2181: const firstSnippet = searchSnippet(results[0].text, question);
2182: if (firstSnippet) {
2183:   parts.push("");
2184:   parts.push("Why it matched:");
2185:   parts.push(firstSnippet);
2186: }
2187: return parts.join("\n");
2188: }

```

assistantSourcesMarkup (script.js, lines 2190-2208)

assistantSourcesMarkup part of the public website runtime. In this file, it appears around lines 2190-2208 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice, map, set, escapeHtml, assistantSourceLabel, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2192: if (!sources.length) return "";; line 2196: return `; line 2200: return `.

Important local values include sources at line 2191; sourceButtons at line 2193; id at line 2194. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2190: function assistantSourcesMarkup(results = []) {
2191:   const sources = results.slice(0, 5);
2192:   if (!sources.length) return "";
2193:   const sourceButtons = sources.map((source) => {
2194:     const id = String(assistantSourceCounter++);
2195:     assistantSourceMap.set(id, source);
2196:     return `
2197:       <button type="button" data-ai-source-
id="${id}">${escapeHtml(assistantSourceLabel(source))}</button>
2198:     `;
2199:   }).join("");
2200:   return `
2201:     <div class="ai-source-panel">
2202:       <p class="ai-source-heading">Relevant portfolio links</p>
2203:       <div class="ai-source-list" aria-label="Assistant sources">
2204:         ${sourceButtons}
2205:       </div>
2206:     </div>
2207:   `;
2208: }

```

setAssistantStatus (script.js, lines 2210-2214)

setAssistantStatus part of the public website runtime. In this file, it appears around lines 2210-2214 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2211: if (!aiAssistantStatus) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2210: function setAssistantStatus(message, state = "ready") {
2211:   if (!aiAssistantStatus) return;
2212:   aiAssistantStatus.textContent = message;
2213:   aiAssistantStatus.dataset.state = state;
2214: }
```

assistantDesktopDockedHeight (script.js, lines 2216-2221)

assistantDesktopDockedHeight part of the public website runtime. In this file, it appears around lines 2216-2221 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getComputedStyle, getPropertyValue, trim, endsWith, min, and max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2217: if (!aiAssistantPanel) return 360;; line 2219: if (value.endsWith("px")) return Number.parseFloat(value) || 360;; line 2220: return Math.min(Math.max(window.innerHeight * 0.38, 340), 390);.

Important local values include value at line 2218. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2216: function assistantDesktopDockedHeight() {
2217:   if (!aiAssistantPanel) return 360;
2218:   const value = getComputedStyle(aiAssistantPanel).getPropertyValue("--ai-console-rest-height").trim();
2219:   if (value.endsWith("px")) return Number.parseFloat(value) || 360;
2220:   return Math.min(Math.max(window.innerHeight * 0.38, 340), 390);
2221: }
```

updateAssistantPanelGrowth (script.js, lines 2223-2246)

updateAssistantPanelGrowth updates ui, state, cache, or stored data. In this file, it appears around lines 2223-2246 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references matchMedia, removeProperty, remove, assistantDesktopDockedHeight, getBoundingClientRect, querySelector, querySelectorAll, reduce, min, round, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2224: if (!aiAssistantPanel || !aiAssistantLog || !aiAssistantForm) return;; line 2229: return;.

Important local values include isDesktop at line 2225; dockedHeight at line 2232; formHeight at line 2233; clearHeight at line 2234; titleHeight at line 2235; chromeHeight at line 2236; messages at line 2237; messageGap at line 2238; plus 4 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2223: function updateAssistantPanelGrowth() {
2224:   if (!aiAssistantPanel || !aiAssistantLog || !aiAssistantForm) return;
2225:   const isDesktop = window.matchMedia("(min-width: 901px)").matches;
2226:   if (!isDesktop) {
2227:     aiAssistantPanel.style.removeProperty("--ai-console-live-height");
2228:     aiAssistantPanel.classList.remove("ai-assistant-panel-expanded");
2229:     return;
2230:   }
2231: }
```

```

2232:   const dockedHeight = assistantDesktopDockedHeight();
2233:   const formHeight = aiAssistantForm.getBoundingClientRect().height || 46;
2234:   const clearHeight = aiClearChatButton?.getBoundingClientRect().height || 38;
2235:   const titleHeight = document.querySelector("#ai-assistant-title")?.getBoundingClientRect().height ||
34;
2236:   const chromeHeight = titleHeight + formHeight + clearHeight + 92;
2237:   const messages = [...aiAssistantLog.querySelectorAll(".ai-message")];
2238:   const messageGap = messages.length > 1 ? (messages.length - 1) * 10 : 0;
2239:   const messageHeight = messages.reduce((sum, message) => sum + message.getBoundingClientRect().height,
0) + messageGap;
2240:   const contentHeight = messageHeight + chromeHeight;
2241:   const maxHeight = Math.min(window.innerHeight * 0.76, 720);
2242:   const nextHeight = Math.round(Math.max(dockedHeight, Math.min(contentHeight, maxHeight)));
2243:
2244:   aiAssistantPanel.style.setProperty("--ai-console-live-height", `${nextHeight}px`);
2245:   aiAssistantPanel.classList.toggle("ai-assistant-panel-expanded", nextHeight > dockedHeight + 12);
2246: }

```

renderAssistantInline (script.js, lines 2248-2250)

renderAssistantInline renders ui, markup, or display output. In this file, it appears around lines 2248-2250 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderInlineMath, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2249: return renderInlineMath(value).replace(/*{([\^*]+)}*/g, "\$1");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2248: function renderAssistantInline(value = "") {
2249:   return renderInlineMath(value).replace(/\*{([\^*]+)}\*/g, "<strong>$1</strong>");
2250: }

```

renderAssistantAnswerContent (script.js, lines 2252-2304)

renderAssistantAnswerContent renders ui, markup, or display output. In this file, it appears around lines 2252-2304 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace, split, push, map, renderAssistantInline, join, escapeHtml, forEach, trim, match, and 3 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 7 return statement(s). The most important visible returns are: line 2258: if (!bulletItems.length) return;; line 2263: if (!codeBlock) return;; line 2279: return;; line 2283: return;. There are 3 additional return statements in the function body.

Important local values include lines at line 2253; html at line 2254; bulletItems at line 2255; codeBlock at line 2256; flushBullets at line 2257; flushCode at line 2262; language at line 2264; trimmed at line 2270; plus 2 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2252: function renderAssistantAnswerContent(content = "") {
2253:   const lines = String(content || "").replace(/\r\n?/g, "\n").split("\n");
2254:   const html = [];
2255:   let bulletItems = [];
2256:   let codeBlock = null;
2257:   const flushBullets = () => {
2258:     if (!bulletItems.length) return;
2259:     html.push(`<ul>${bulletItems.map((item) =>
<li>${renderAssistantInline(item)}</li>`).join("")}</ul>`);
2260:     bulletItems = [];
2261:   };
2262:   const flushCode = () => {
2263:     if (!codeBlock) return;
2264:     const language = codeBlock.language ? ` data-language="${escapeHtml(codeBlock.language)}"` : "";
2265:     html.push(`<pre class="ai-code-
block"${language}><code>${escapeHtml(codeBlock.lines.join("\n"))}</code></pre>`);
2266:     codeBlock = null;
2267:   };
2268:
2269:   lines.forEach((line) => {
2270:     const trimmed = line.trim();

```

```

2271:     const fence = trimmed.match(/^```([a-z0-9_+.#-]*)\s*$/i);
2272:     if (fence) {
2273:       if (codeBlock) {
2274:         flushCode();
2275:       } else {
2276:         flushBullets();
2277:         codeBlock = { language: fence[1] || "", lines: [] };
2278:       }
2279:       return;
2280:     }
2281:     if (codeBlock) {
2282:       codeBlock.lines.push(line);
2283:       return;
2284:     }
2285:     if (!trimmed) {
2286:       flushBullets();
2287:       return;
2288:     }
2289:     const bullet = trimmed.match(/^[-*\u2022]\s+(.+)/u);
2290:     if (bullet) {
2291:       bulletItems.push(bullet[1]);
2292:       return;
2293:     }
2294:     flushBullets();
2295:     if (/^[^!?]{2,64}$/.test(trimmed)) {
2296:       html.push(`<p><strong>${renderAssistantInline(trimmed.slice(0, -1))}</strong></p>`);
2297:     } else {
2298:       html.push(`<p>${renderAssistantInline(trimmed)}</p>`);
2299:     }
2300:   });
2301:   flushCode();
2302:   flushBullets();
2303:   return `<div class="ai-answer-content">${html.join("")} || "<p>I am ready.</p>"</div>`;
2304: }

```

flushBullets (script.js, lines 2257-2261)

flushBullets part of the public website runtime. In this file, it appears around lines 2257-2261 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references push, map, renderAssistantInline, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2258: if (!bulletItems.length) return;.

Important local values include flushBullets at line 2257. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2257:   const flushBullets = () => {
2258:     if (!bulletItems.length) return;
2259:     html.push(`<ul>${bulletItems.map((item) =>
`<li>${renderAssistantInline(item)}</li>`).join("")}</ul>`);
2260:     bulletItems = [];
2261:   };

```

flushCode (script.js, lines 2262-2267)

flushCode part of the public website runtime. In this file, it appears around lines 2262-2267 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, push, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2263: if (!codeBlock) return;.

Important local values include flushCode at line 2262; language at line 2264. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2262:   const flushCode = () => {
2263:     if (!codeBlock) return;
2264:     const language = codeBlock.language ? ` data-language="${escapeHtml(codeBlock.language)}" ` : "";
2265:     html.push(`<pre class="ai-code-block"${language}><code>${escapeHtml(codeBlock.lines.join("\n"))}</code></pre>`);

```



```
2266:     codeBlock = null;
2267:   };
```

appendAssistantMessage (script.js, lines 2306-2317)

appendAssistantMessage part of the public website runtime. In this file, it appears around lines 2306-2317 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, renderAssistantAnswerContent, assistantSourcesMarkup, renderMultilineInlineText, append, and updateAssistantPanelGrowth. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2307: if (!aiAssistantLog) return;; line 2316: return article;.

Important local values include article at line 2308. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2306: function appendAssistantMessage(role, content, sources = []) {
2307:   if (!aiAssistantLog) return;
2308:   const article = document.createElement("article");
2309:   article.className = `ai-message ai-message-${role}`;
2310:   article.innerHTML = role === "assistant"
2311:     ? `${renderAssistantAnswerContent(content)}${assistantSourcesMarkup(sources)}`
2312:     : `<p>${renderMultilineInlineText(content)}</p>`;
2313:   aiAssistantLog.append(article);
2314:   updateAssistantPanelGrowth();
2315:   aiAssistantLog.scrollTop = aiAssistantLog.scrollHeight;
2316:   return article;
2317: }
```

scrollAssistantAnswerToStart (script.js, lines 2319-2329)

scrollAssistantAnswerToStart part of the public website runtime. In this file, it appears around lines 2319-2329 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max, and requestAnimationFrame. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2320: if (!aiAssistantLog || !article) return;.

Important local values include scrollToAnswer at line 2321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2319: function scrollAssistantAnswerToStart(article) {
2320:   if (!aiAssistantLog || !article) return;
2321:   const scrollToAnswer = () => {
2322:     aiAssistantLog.scrollTop = Math.max(0, article.offsetTop - aiAssistantLog.offsetTop - 4);
2323:   };
2324:   if (typeof window.requestAnimationFrame === "function") {
2325:     window.requestAnimationFrame(scrollToAnswer);
2326:   } else {
2327:     window.setTimeout(scrollToAnswer, 0);
2328:   }
2329: }
```

scrollToAnswer (script.js, lines 2321-2323)

scrollToAnswer part of the public website runtime. In this file, it appears around lines 2321-2323 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include `scrollToAnswer` at line 2321. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2321: const scrollToAnswer = () => {
2322:   aiAssistantLog.scrollTop = Math.max(0, article.offsetTop - aiAssistantLog.offsetTop - 4);
2323: };
```

appendAssistantPendingMessage (script.js, lines 2331-2335)

`appendAssistantPendingMessage` part of the public website runtime. In this file, it appears around lines 2331-2335 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `appendAssistantMessage`, and `add`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2334: `return article`;

Important local values include `article` at line 2332. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2331: function appendAssistantPendingMessage() {
2332:   const article = appendAssistantMessage("assistant", "Thinking", []);
2333:   article?.classList.add("ai-message-pending");
2334:   return article;
2335: }
```

replaceAssistantMessage (script.js, lines 2337-2346)

`replaceAssistantMessage` part of the public website runtime. In this file, it appears around lines 2337-2346 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `appendAssistantMessage`, `renderAssistantAnswerContent`, `assistantSourcesMarkup`, `updateAssistantPanelGrowth`, and `scrollAssistantAnswerToStart`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2340: `return`;

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2337: function replaceAssistantMessage(article, content, sources = []) {
2338:   if (!article) {
2339:     appendAssistantMessage("assistant", content, sources);
2340:     return;
2341:   }
2342:   article.className = "ai-message ai-message-assistant";
2343:   article.innerHTML = `${renderAssistantAnswerContent(content)}${assistantSourcesMarkup(sources)}`;
2344:   updateAssistantPanelGrowth();
2345:   scrollAssistantAnswerToStart(article);
2346: }
```

setAssistantBusy (script.js, lines 2348-2352)

`setAssistantBusy` part of the public website runtime. In this file, it appears around lines 2348-2352 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `querySelector`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no `local/session` storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include `submitButton` at line 2349. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2348: function setAssistantBusy(isBusy) {
2349:   const submitButton = aiAssistantForm?.querySelector('button[type="submit"]');
2350:   if (aiAssistantInput) aiAssistantInput.disabled = isBusy;
2351:   if (submitButton) submitButton.disabled = isBusy;
2352: }
```

clearAssistantChat (script.js, lines 2354-2363)

clearAssistantChat part of the public website runtime. In this file, it appears around lines 2354-2363 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Map, setAssistantStatus, updateAssistantPanelGrowth, and focus. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2355: if (!aiAssistantLog) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2354: function clearAssistantChat() {
2355:   if (!aiAssistantLog) return;
2356:   assistantSourceCounter = 0;
2357:   assistantSourceMap = new Map();
2358:   assistantChatHistory = [];
2359:   aiAssistantLog.innerHTML = "";
2360:   setAssistantStatus("Chat cleared");
2361:   updateAssistantPanelGrowth();
2362:   aiAssistantInput?.focus();
2363: }
```

assistantRemoteTimeoutMs (script.js, lines 2365-2374)

assistantRemoteTimeoutMs part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 2365-2374 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, isArray, some, toLowerCase, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2370: return 45000;; line 2372: if (options.allowWebSearch) return 30000;; line 2373: return 18000;.

Important local values include clean at line 2366; sourceFetches at line 2367; usesGitHub at line 2368. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2365: function assistantRemoteTimeoutMs(question = "", context = {}, options = {}) {
2366:   const clean = normalize(question);
2367:   const sourceFetches = Array.isArray(context.sourceFetches) ? context.sourceFetches : [];
2368:   const usesGitHub = sourceFetches.some((source) => String(source.kind || "").toLowerCase() === "github"
|| /github\.com/i.test(source.url || ""));
2369:   if (usesGitHub || /\b(github|repo|repository|repositories|source\s+code|code|snippet|firmware|verilog|s
ystemverilog|python|javascript|circuit\s+file)\b/.test(clean)) {
2370:     return 45000;
2371:   }
2372:   if (options.allowWebSearch) return 30000;
2373:   return 18000;
2374: }
```

askRemoteAssistant (script.js, lines 2376-2411)

This function sends the user's question and selected context to the configured AI endpoint, waits with a timeout, and returns the remote answer if available.

It is the browser side of the website AI backend call. It lets the public site use Cloudflare/OpenAI/Workers AI without exposing implementation details in the UI.

It calls or references assistantEndpoint, AbortController, abort, assistantRemoteTimeoutMs, fetch, stringify, Error, json, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage

keys detected.

It has 2 return statement(s). The most important visible returns are: line 2378: if (!endpoint) return null;; line 2410: return answer ? { answer, model: data.model || "", usedWebSearch: Boolean(data.usedWebSearch) } : null;.

Important local values include endpoint at line 2377; controller at line 2379; didTimeout at line 2380; timer at line 2381; data at line 2408; answer at line 2409. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2376: async function askRemoteAssistant(question, context, options = {}) {
2377:   const endpoint = assistantEndpoint();
2378:   if (!endpoint) return null;
2379:   const controller = new AbortController();
2380:   let didTimeout = false;
2381:   const timer = setTimeout(() => {
2382:     didTimeout = true;
2383:     controller.abort();
2384:   }, assistantRemoteTimeoutMs(question, context, options));
2385:   let response;
2386:   try {
2387:     response = await fetch(endpoint, {
2388:       method: "POST",
2389:       headers: { "Content-Type": "application/json" },
2390:       signal: controller.signal,
2391:       body: JSON.stringify({
2392:         context,
2393:         conversation: options.conversation || [],
2394:         intent: options.intent || context.intent || "portfolio_specific",
2395:         allowWebSearch: Boolean(options.allowWebSearch),
2396:         question,
2397:         mode: "portfolio-plus-general-knowledge",
2398:         responseType: "chatgpt-like-answer-first-links-second"
2399:       })
2400:     });
2401:   } catch (error) {
2402:     if (didTimeout) throw new Error("AI request timed out while reading public sources.");
2403:     throw error;
2404:   } finally {
2405:     clearTimeout(timer);
2406:   }
2407:   if (!response.ok) throw new Error("AI service unavailable");
2408:   const data = await response.json();
2409:   const answer = String(data.answer || data.message || "").trim();
2410:   return answer ? { answer, model: data.model || "", usedWebSearch: Boolean(data.usedWebSearch) } : null;
2411: }
```

answerAssistantQuestion (script.js, lines 2413-2458)

This function coordinates the full chat response flow: classify the question, build context, try the remote assistant, fall back locally if needed, and render the response.

It is the public website's AI conversation orchestrator. It keeps the interface conversational while still grounding answers in portfolio context when appropriate.

It calls or references trim, slice, appendAssistantMessage, assistantRemember, appendAssistantPendingMessage, setAssistantBusy, setAssistantStatus, assistantEndpoint, assistantQuestionIntent, assistantFallbackResults, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2415: if (!cleanQuestion) return;; line 2439: return;;

Important local values include cleanQuestion at line 2414; priorConversation at line 2416; pendingMessage at line 2419; intent at line 2424; rawSources at line 2425; sources at line 2426; context at line 2427; conversation at line 2428; plus 3 more local variable(s). These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2413: async function answerAssistantQuestion(question = "") {
2414:   const cleanQuestion = String(question || "").trim();
2415:   if (!cleanQuestion) return;
2416:   const priorConversation = assistantChatHistory.slice();
2417:   appendAssistantMessage("user", cleanQuestion);
2418:   assistantRemember("user", cleanQuestion);
2419:   const pendingMessage = appendAssistantPendingMessage();
2420:   setAssistantBusy(true);
2421:   try {
2422:     setAssistantStatus(assistantEndpoint() ? "Composing AI answer" : "Composing local answer",
"working");
2423:
2424:     const intent = assistantQuestionIntent(cleanQuestion, priorConversation);
2425:     const rawSources = assistantFallbackResults(cleanQuestion, intent);
2426:     const sources = assistantSourcesForDisplay(cleanQuestion, rawSources, intent);
2427:     const context = assistantContextForQuestion(cleanQuestion, intent, rawSources);
2428:     const conversation = assistantConversationForRequest(cleanQuestion, priorConversation);
2429:     try {
```

```

2430:     const remoteAnswer = await askRemoteAssistant(cleanQuestion, context, {
2431:       allowWebSearch: assistantQuestionAllowsPublicLookup(cleanQuestion, intent),
2432:       conversation,
2433:       intent
2434:     });
2435:     if (remoteAnswer?.answer) {
2436:       replaceAssistantMessage(pendingMessage, remoteAnswer.answer, sources);
2437:       assistantRemember("assistant", remoteAnswer.answer);
2438:       setAssistantStatus(remoteAnswer.model ? `AI answer ready: ${remoteAnswer.model}` : "AI answer
ready");
2439:       return;
2440:     }
2441:   } catch (error) {
2442:     setAssistantStatus(error?.message || "Local answer used", "error");
2443:   }
2444:
2445:   const fallbackAnswer = assistantLocalAnswer(cleanQuestion, sources, intent);
2446:   replaceAssistantMessage(pendingMessage, fallbackAnswer, sources);
2447:   assistantRemember("assistant", fallbackAnswer);
2448:   if (!assistantEndpoint()) setAssistantStatus("Local portfolio intelligence");
2449: } catch {
2450:   const errorAnswer = "I had trouble processing that question. Try rephrasing it with a project name,
tool name, or electronics topic.";
2451:   replaceAssistantMessage(pendingMessage, errorAnswer, []);
2452:   assistantRemember("assistant", errorAnswer);
2453:   setAssistantStatus("Assistant error", "error");
2454: } finally {
2455:   setAssistantBusy(false);
2456:   aiAssistantInput?.focus();
2457: }
2458: }

```

linkAttributes (script.js, lines 2459-2462)

linkAttributes part of the public website runtime. In this file, it appears around lines 2459-2462 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, and isWebsiteLinkItem. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2461: return /^https?:\V/i.test(target) ? ' target="_blank" rel="noreferrer"' : "";

Important local values include target at line 2460. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2459: function linkAttributes(url, item = {}) {
2460:   const target = normalizeLinkTarget(url, { assumeWeb: isWebsiteLinkItem(item, url) });
2461:   return /^https?:\V/i.test(target) ? ' target="_blank" rel="noreferrer"' : "";
2462: }

```

isLocalDownloadTarget (script.js, lines 2464-2467)

isLocalDownloadTarget part of the public website runtime. In this file, it appears around lines 2464-2467 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2466: return Boolean(value) && !/^(https?:)\V/i.test(value) && !/^(mailto:|tel:|#)/i.test(value);

Important local values include value at line 2465. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2464: function isLocalDownloadTarget(target = "") {
2465:   const value = normalizeLinkTarget(target, { assumeWeb: true });
2466:   return Boolean(value) && !/^(https?:)\V/i.test(value) && !/^(mailto:|tel:|#)/i.test(value);
2467: }

```

downloadAttribute (script.js, lines 2469-2472)

downloadAttribute part of the public website runtime. In this file, it appears around lines 2469-2472 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalizeLinkTarget, isWebsiteLinkItem, and isLocalDownloadTarget. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2471: return isLocalDownloadTarget(value) ? " download" : "";

Important local values include value at line 2470. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2469: function downloadAttribute(target = "", item = {}) {
2470:   const value = normalizeLinkTarget(target, { assumeWeb: isWebsiteLinkItem(item, target) });
2471:   return isLocalDownloadTarget(value) ? " download" : "";
2472: }
```

fileNameFromUrl (script.js, lines 2474-2481)

fileNameFromUrl part of the public website runtime. In this file, it appears around lines 2474-2481 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, and pop. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2477: return decodeURIComponent(clean); line 2479: return clean;

Important local values include clean at line 2475. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2474: function fileNameFromUrl(url = "") {
2475:   const clean = String(url || "").split(/[?#]/)[0].split("/").pop() || "Download file";
2476:   try {
2477:     return decodeURIComponent(clean);
2478:   } catch {
2479:     return clean;
2480:   }
2481: }
```

resourceLink (script.js, lines 2483-2501)

resourceLink part of the public website runtime. In this file, it appears around lines 2483-2501 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileNameFromUrl, normalizeLinkTarget, isWebsiteLinkItem, escapeHtml, linkAttributes, and downloadAttribute. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2488: return `

Important local values include rawTarget at line 2484; target at line 2485. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2483: function resourceLink(item = {}, label = item.label || item.title || item.name ||
fileNameFromUrl(item.url || item.artifact)) {
2484:   const rawTarget = item.url || item.artifact || item.href || item.file || item.path || item.src || "";
2485:   const target = normalizeLinkTarget(rawTarget, { assumeWeb: isWebsiteLinkItem(item, rawTarget) });
2486:
2487:   if (!target || item.status === "planned") {
2488:     return `
```

```

2492:     <a
2493:         class="resource-link"
2494:         href="${escapeHtml(target)}"
2495:         ${linkAttributes(target, item)}
2496:         ${downloadAttribute(target, item)}
2497:     >
2498:         ${escapeHtml(label || fileNameFromUrl(target))}
2499:     </a>
2500: `;
2501: }

```

pillList (script.js, lines 2503-2510)

pillList part of the public website runtime. In this file, it appears around lines 2503-2510 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2504: return (items || []); line 2507: return label ? `<span class="tag \${className}">\${label}</span>` : "".

Important local values include label at line 2506. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2503: function pillList(items, className = "") {
2504:     return (items || [])
2505:         .map((item) => {
2506:             const label = typeof item === "string" ? item : item.name || item.title || item.label || "";
2507:             return label ? `<span class="tag ${className}">${label}</span>` : "";
2508:         })
2509:         .join("");
2510: }

```

evidenceList (script.js, lines 2512-2518)

evidenceList part of the public website runtime. In this file, it appears around lines 2512-2518 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2514: return `<p class="evidence-empty">\${emptyMessage}</p>`; line 2517: return `<ul>\${items.map(renderItem).join("")}</ul>`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2512: function evidenceList(items, renderItem, emptyMessage) {
2513:     if (!items || !items.length) {
2514:         return `<p class="evidence-empty">${emptyMessage}</p>`;
2515:     }
2516:
2517:     return `<ul>${items.map(renderItem).join("")}</ul>`;
2518: }

```

detailBlock (script.js, lines 2520-2527)

detailBlock part of the public website runtime. In this file, it appears around lines 2520-2527 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2521: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2520: function detailBlock(title, className, content) {
2521:   return `
2522:     <details class="${className}">
2523:       <summary>${title}</summary>
2524:       ${content}
2525:     </details>
2526:   `;
2527: }
```

mediaGrid (script.js, lines 2529-2549)

mediaGrid part of the public website runtime. In this file, it appears around lines 2529-2549 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2531: return `

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2529: function mediaGrid(items) {
2530:   if (!items || !items.length) {
2531:     return `
```

siteSectionHasContent (script.js, lines 2551-2560)

siteSectionHasContent part of the public website runtime. In this file, it appears around lines 2551-2560 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2552: return Boolean(

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2551: function siteSectionHasContent(section) {
2552:   return Boolean(
2553:     section?.description ||
2554:     section?.richDescription?.blocks?.length ||
2555:     section?.backgroundImage ||
2556:     (section?.links || []).some((link) => link.label || link.url) ||
2557:     (section?.assets || []).some((asset) => asset.title || asset.url) ||
2558:     (section?.subsections || []).some((item) => item.title || item.description ||
item.richDescription?.blocks?.length || (item.links || []).length)
2559:   );
2560: }
```


siteSectionRenderable (script.js, lines 2562-2564)

siteSectionRenderable part of the public website runtime. In this file, it appears around lines 2562-2564 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references siteSectionHasContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2563: return section?.visible !== false && siteSectionHasContent(section);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2562: function siteSectionRenderable(section) {  
2563:   return section?.visible !== false && siteSectionHasContent(section);  
2564: }
```

renderDynamicLinks (script.js, lines 2566-2570)

renderDynamicLinks renders ui, markup, or display output. In this file, it appears around lines 2566-2570 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, map, resourceLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2568: if (!links.length) return "";; line 2569: return `<div class="resource-list dynamic-link-list">\${links.map((item) => resourceLink(item, item.label || item.title || "Open")).join("")}</div>`.

Important local values include links at line 2567. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2566: function renderDynamicLinks(items = []) {  
2567:   const links = items.filter((item) => item?.url && (item.label || item.title));  
2568:   if (!links.length) return "";  
2569:   return `<div class="resource-list dynamic-link-list">${links.map((item) => resourceLink(item,  
item.label || item.title || "Open")).join("")}</div>`;  
2570: }
```

renderSiteSections (script.js, lines 2572-2609)

renderSiteSections renders ui, markup, or display output. In this file, it appears around lines 2572-2609 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, filter, map, url, escapeHtml, renderRichContent, renderDynamicLinks, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2574: if (!mount) return;; line 2580: return `

Important local values include mount at line 2573; visibleSections at line 2575; style at line 2577; links at line 2578; subsections at line 2579. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2572: function renderSiteSections() {  
2573:   const mount = document.querySelector("#dynamic-sections");  
2574:   if (!mount) return;  
2575:   const visibleSections = (siteSections || []).filter(siteSectionRenderable);  
2576:   mount.innerHTML = visibleSections.map((section) => {  
2577:     const style = section.backgroundImage ? ` style="--section-bg:  
url('${escapeHtml(section.backgroundImage)}')'` : "";  
2578:     const links = [...(section.links || []), ...(section.assets || [])];  
2579:     const subsections = (section.subsections || []).filter((item) => item.title || item.description ||  
item.richDescription?.blocks?.length || (item.links || []).length);  
2580:     return `  
2581:       <section class="section dynamic-section" id="${escapeHtml(section.id || "")}"${style}>  
2582:         <div class="dynamic-section-surface">
```

```

2583:         <div class="section-heading">
2584:             <div>
2585:                 <h2>${escapeHtml(section.title || "Untitled section")}</h2>
2586:             </div>
2587:         </div>
2588:         ${section.description || section.richDescription?.blocks?.length
2589:         ? `<div class="dynamic-section-copy">${renderRichContent(section.richDescription,
section.description || "")}</div>`
2590:         : ""}
2591:         ${subsections.length ? `
2592:         <div class="dynamic-section-grid">
2593:             ${subsections.map((item) => `
2594:                 <article class="dynamic-section-card">
2595:                     <h3>${escapeHtml(item.title || "Untitled")}</h3>
2596:                     ${item.description || item.richDescription?.blocks?.length
2597:                     ? renderRichContent(item.richDescription, item.description || "")
2598:                     : ""}
2599:                     ${renderDynamicLinks(item.links || [])}
2600:                 </article>
2601:             `).join("")}
2602:         </div>
2603:         ` : ""}
2604:         ${renderDynamicLinks(links)}
2605:     </div>
2606: </section>
2607: `;
2608: }).join("");
2609: }

```

subsections (script.js, lines 2579-2581)

subsections part of the public website runtime. In this file, it appears around lines 2579-2581 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2580: return `.

Important local values include subsections at line 2579. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2579:     const subsections = (section.subsections || []).filter((item) => item.title || item.description ||
item.richDescription?.blocks?.length || (item.links || []).length);
2580:     return `
2581:     <section class="section dynamic-section" id="${escapeHtml(section.id || "")}"${style}>

```

customSectionBlocks (script.js, lines 2611-2622)

customSectionBlocks part of the public website runtime. In this file, it appears around lines 2611-2622 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, detailBlock, evidenceList, resourceLink, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2612: return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-wide", `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2611: function customSectionBlocks(project) {
2612:     return (project.sections || []).map((section) => detailBlock(section.title, "evidence-block evidence-
wide", `
2613:     ${section.description ? `<p class="evidence-empty">${section.description}</p>` : ""}
2614:     ${evidenceList(section.items || [], (item) => `
2615:         <li>
2616:             ${item.url ? resourceLink(item, item.title) : `<strong>${item.title}</strong>`}
2617:             <span>${item.type || "Section item"} &middot; ${item.status || "tracked">`</span>
2618:             ${item.description ? `<p>${item.description}</p>` : ""}
2619:         </li>
2620:     `, "No content has been added yet.")}
2621:     `)).join("");
2622: }

```

itemUrl (script.js, lines 2623-2625)

itemUrl part of the public website runtime. In this file, it appears around lines 2623-2625 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2624: return item.url || item.artifact || item.href || item.file || item.path || item.src || "";

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2623: function itemUrl(item = {}) {  
2624:   return item.url || item.artifact || item.href || item.file || item.path || item.src || "";  
2625: }
```

itemLabel (script.js, lines 2627-2630)

itemLabel part of the public website runtime. In this file, it appears around lines 2627-2630 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references itemUrl, and fileNameFromUrl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2629: return item.title || item.label || item.name || item.caption || fileNameFromUrl(url) || fallback;

Important local values include url at line 2628. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2627: function itemLabel(item = {}, fallback = "Download file") {  
2628:   const url = itemUrl(item);  
2629:   return item.title || item.label || item.name || item.caption || fileNameFromUrl(url) || fallback;  
2630: }
```

normalizeDownloadAsset (script.js, lines 2632-2643)

normalizeDownloadAsset validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 2632-2643 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references itemUrl, and itemLabel. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2634: if (!url || item.status === "planned") return null;; line 2636: return {.

Important local values include url at line 2633. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2632: function normalizeDownloadAsset(item = {}, fallbackType = "File") {  
2633:   const url = itemUrl(item);  
2634:   if (!url || item.status === "planned") return null;  
2635:  
2636:   return {  
2637:     title: itemLabel(item),  
2638:     url,  
2639:     status: item.status || "uploaded",  
2640:     type: item.type || item.kind || item.meta || fallbackType,  
2641:     description: item.description || item.caption || item.summary || ""  
2642:   };  
2643: }
```

collectRichDownloads (script.js, lines 2645-2659)

collectRichDownloads part of the public website runtime. In this file, it appears around lines 2645-2659 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references forEach, push, cleanRichImageTitle, and fileNameFromUrl. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2647: if (!block?.url) return;; line 2658: return output;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2645: function collectRichDownloads(rich, fallbackType = "File", output = []) {
2646:   (rich?.blocks || []).forEach((block) => {
2647:     if (!block?.url) return;
2648:
2649:     output.push({
2650:       title: cleanRichImageTitle(block) || block.title || block.caption ||
fileNameFromUrl(block.url),
2651:       url: block.url,
2652:       status: "uploaded",
2653:       type: block.type === "image" ? "Image" : fallbackType,
2654:       description: block.caption || ""
2655:     });
2656:   });
2657:   return output;
2658: }
2659: }
```

collectDownloadsFromValue (script.js, lines 2661-2689)

collectDownloadsFromValue part of the public website runtime. In this file, it appears around lines 2661-2689 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, forEach, normalizeDownloadAsset, push, and collectRichDownloads. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2662: if (!value) return output;; line 2666: return output;; line 2669: if (typeof value !== "object") return output;; line 2688: return output;.

Important local values include asset at line 2671. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2661: function collectDownloadsFromValue(value, fallbackType = "File", output = []) {
2662:   if (!value) return output;
2663:
2664:   if (Array.isArray(value)) {
2665:     value.forEach((item) => collectDownloadsFromValue(item, fallbackType, output));
2666:     return output;
2667:   }
2668:
2669:   if (typeof value !== "object") return output;
2670:
2671:   const asset = normalizeDownloadAsset(value, fallbackType);
2672:   if (asset) output.push(asset);
2673:
2674:   collectRichDownloads(value.rich, fallbackType, output);
2675:   collectRichDownloads(value.summaryRich, fallbackType, output);
2676:
2677:   [
2678:     "files",
2679:     "assets",
2680:     "items",
2681:     "children",
2682:     "links",
2683:     "results",
2684:     "references",
2685:     "mathAnalysis"
2686:   ].forEach((key) => collectDownloadsFromValue(value[key], fallbackType, output));
2687:
2688:   return output;
2689: }
```

uniqueDownloads (script.js, lines 2691-2700)

uniqueDownloads part of the public website runtime. In this file, it appears around lines 2691-2700 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Set, filter, has, and add. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2694: return items.filter((item) => {; line 2696: if (!item.url || seen.has(key)) return false;; line 2698: return true;;

Important local values include seen at line 2692; key at line 2695. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2691: function uniqueDownloads(items = []) {
2692:   const seen = new Set();
2693:
2694:   return items.filter((item) => {
2695:     const key = `${item.url}:${item.title}`;
2696:     if (!item.url || seen.has(key)) return false;
2697:     seen.add(key);
2698:     return true;
2699:   });
2700: }
```

renderDownloadBlock (script.js, lines 2702-2717)

renderDownloadBlock renders ui, markup, or display output. In this file, it appears around lines 2702-2717 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, detailBlock, escapeHtml, map, resourceLink, renderMultilineInlineText, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2704: if (!downloads.length) return "";; line 2706: return detailBlock(escapeHtml(title), "evidence-block evidence-wide download-evidence-block", `

Important local values include downloads at line 2703. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2702: function renderDownloadBlock(title, items = []) {
2703:   const downloads = uniqueDownloads(items);
2704:   if (!downloads.length) return "";
2705:
2706:   return detailBlock(escapeHtml(title), "evidence-block evidence-wide download-evidence-block", `
2707:     <ul class="download-list">
2708:       ${downloads.map((item) => `
2709:         <li>
2710:           ${resourceLink(item, item.title)}
2711:           <span>${escapeHtml(item.type || "File")} &middot; ${escapeHtml(item.status ||
"uploaded")}</span>
2712:           ${item.description ? `<p>${renderMultilineInlineText(item.description)}</p>` : ""}
2713:         </li>
2714:       `).join("")}
2715:     </ul>
2716:   `);
2717: }
```

renderCollectedDownloadSections (script.js, lines 2719-2760)

renderCollectedDownloadSections renders ui, markup, or display output. In this file, it appears around lines 2719-2760 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, collectDownloadsFromValue, push, renderDownloadBlock, addBlock, forEach, displayTitle, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no

local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2759: return blocks.join("");.

Important local values include blocks at line 2720; addBlock at line 2722; downloads at line 2723. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2719: function renderCollectedDownloadSections(project) {
2720:   const blocks = [];
2721:
2722:   const addBlock = (title, source, fallbackType = "File") => {
2723:     const downloads = uniqueDownloads(collectDownloadsFromValue(source, fallbackType, []));
2724:     if (downloads.length) blocks.push(renderDownloadBlock(title, downloads));
2725:   };
2726:
2727:   addBlock("Overview downloads", [
2728:     { summaryRich: project.summaryRich },
2729:     project.design?.brief?.files,
2730:     project.design?.brief?.summaryRich
2731:   ], "Overview file");
2732:
2733:   addBlock("Design downloads", [
2734:     project.design?.documentation?.files,
2735:     project.design?.documentation?.references,
2736:     project.design?.documentation?.mathAnalysis
2737:   ], "Design file");
2738:
2739:   addBlock("Simulation downloads", [
2740:     project.design?.simulation?.files,
2741:     project.design?.simulation?.results
2742:   ], "Simulation file");
2743:
2744:   addBlock("Project documents", project.documents, "Document");
2745:   addBlock("Test artifacts", project.tests, "Test artifact");
2746:   addBlock("PCB artifacts", project.pcb, "PCB artifact");
2747:   addBlock("Images and media", project.media, "Image");
2748:
2749:   (project.sections || []).forEach((section) => {
2750:     addBlock(`${displayTitle(section.title, "Custom section")} downloads`, section.items ||
section.files || [], section.title || "Section file");
2751:   });
2752:
2753:   (project.portfolioView?.sections || []).
2754:     .filter((section) => section.id !== "brief")
2755:     .forEach((section) => {
2756:       addBlock(`${displayTitle(section.title, "Section")} downloads`, section.items || [],
section.title || "Section file");
2757:     });
2758:
2759:   return blocks.join("");
2760: }
```

renderParsedBriefBlock (script.js, lines 2761-2771)

renderParsedBriefBlock renders ui, markup, or display output. In this file, it appears around lines 2761-2771 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2764: if (!briefItem.rich?.blocks?.length && !briefText) return "";; line 2765: return `.

Important local values include briefItem at line 2762; briefText at line 2763. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2761: function renderParsedBriefBlock(section, fallbackSummary = "") {
2762:   const briefItem = section?.items?.[0] || {};
2763:   const briefText = briefItem.description || fallbackSummary || "";
2764:   if (!briefItem.rich?.blocks?.length && !briefText) return "";
2765:   return `
2766:     <details class="project-brief-default parsed-summary" open>
2767:       <summary>Overview</summary>
2768:       ${renderRichContent(briefItem.rich, briefText)}
2769:     </details>
2770: `;
2771: }
```

pathToString (script.js, lines 2773-2775)

pathToString part of the public website runtime. In this file, it appears around lines 2773-2775 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2774: return path.join(".");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2773: function pathToString(path = []) {  
2774:   return path.join(".");  
2775: }
```

pathFromString (script.js, lines 2777-2780)

pathFromString part of the public website runtime. In this file, it appears around lines 2777-2780 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, filter, and isInteger. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2778: if (!value) return []; line 2779: return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2777: function pathFromString(value = "") {  
2778:   if (!value) return [];  
2779:   return value.split(".").map((item) => Number(item)).filter((item) => Number.isInteger(item));  
2780: }
```

sectionRouteState (script.js, lines 2782-2789)

sectionRouteState part of the public website runtime. In this file, it appears around lines 2782-2789 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2783: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2782: function sectionRouteState(projectId, sectionIndex, resourcePath = "") {  
2783:   return {  
2784:     appWindow: sectionRouteKey,  
2785:     projectId: String(projectId || ""),  
2786:     resourcePath: String(resourcePath || ""),  
2787:     sectionIndex: String(sectionIndex ?? "")  
2788:   };  
2789: }
```

canUseSectionHistory (script.js, lines 2791-2793)

canUseSectionHistory part of the public website runtime. In this file, it appears around lines 2791-2793 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2792: return Boolean(window.history && typeof window.history.pushState === "function" && typeof window.history.replaceState === "function");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2791: function canUseSectionHistory() {
2792:   return Boolean(window.history && typeof window.history.pushState === "function" && typeof
window.history.replaceState === "function");
2793: }
```

sectionBaseUrl (script.js, lines 2795-2799)

sectionBaseUrl part of the public website runtime. In this file, it appears around lines 2795-2799 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URL, values, forEach, and delete. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2798: return url;.

Important local values include url at line 2796. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2795: function sectionBaseUrl() {
2796:   const url = new URL(window.location.href);
2797:   Object.values(sectionRouteParams).forEach((param) => url.searchParams.delete(param));
2798:   return url;
2799: }
```

sectionRouteUrl (script.js, lines 2801-2809)

sectionRouteUrl part of the public website runtime. In this file, it appears around lines 2801-2809 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sectionBaseUrl, set, and delete. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2808: return url;.

Important local values include url at line 2802. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2801: function sectionRouteUrl(projectId, sectionIndex, resourcePath = "") {
2802:   const url = sectionBaseUrl();
2803:   url.searchParams.set(sectionRouteParams.project, String(projectId || ""));
2804:   url.searchParams.set(sectionRouteParams.section, String(sectionIndex ?? ""));
2805:   if (resourcePath) url.searchParams.set(sectionRouteParams.path, String(resourcePath));
2806:   else url.searchParams.delete(sectionRouteParams.path);
2807:   url.hash = "projects";
2808:   return url;
2809: }
```

sectionRouteFromState (script.js, lines 2811-2819)

sectionRouteFromState part of the public website runtime. In this file, it appears around lines 2811-2819 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2812: if (!state || state.appWindow !== sectionRouteKey) return null;; line 2813: if (!state.projectId || state.sectionIndex === "") return null;; line 2814: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2811: function sectionRouteFromState(state) {
2812:   if (!state || state.appWindow !== sectionRouteKey) return null;
2813:   if (!state.projectId || state.sectionIndex === "") return null;
2814:   return {
2815:     projectId: String(state.projectId),
2816:     resourcePath: String(state.resourcePath || ""),
2817:     sectionIndex: String(state.sectionIndex)
2818:   };
2819: }
```

sectionRouteFromLocation (script.js, lines 2821-2831)

sectionRouteFromLocation part of the public website runtime. In this file, it appears around lines 2821-2831 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references URLSearchParams, and get. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2825: if (!projectId || sectionIndex === null) return null;; line 2826: return {.

Important local values include params at line 2822; projectId at line 2823; sectionIndex at line 2824. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2821: function sectionRouteFromLocation() {
2822:   const params = new URLSearchParams(window.location.search);
2823:   const projectId = params.get(sectionRouteParams.project);
2824:   const sectionIndex = params.get(sectionRouteParams.section);
2825:   if (!projectId || sectionIndex === null) return null;
2826:   return {
2827:     projectId,
2828:     resourcePath: params.get(sectionRouteParams.path) || "",
2829:     sectionIndex
2830:   };
2831: }
```

sameSectionRoute (script.js, lines 2833-2838)

sameSectionRoute part of the public website runtime. In this file, it appears around lines 2833-2838 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2834: return Boolean(left && right &&.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2833: function sameSectionRoute(left, right) {
2834:   return Boolean(left && right &&
2835:     left.projectId === right.projectId &&
2836:     String(left.sectionIndex) === String(right.sectionIndex) &&
2837:     String(left.resourcePath || "") === String(right.resourcePath || ""));
2838: }
```

syncSectionRouteToHistory (script.js, lines 2840-2851)

syncSectionRouteToHistory keeps two places aligned, such as local data and target files. In this file, it appears around lines 2840-2851 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `canUseSectionHistory`, `sectionRouteState`, `sectionRouteUrl`, `sectionRouteFromState`, `sectionRouteFromLocation`, and `sameSectionRoute`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2841: `if (isApplyingSectionRoute || mode === "none" || !canUseSectionHistory()) return;`

Important local values include `route` at line 2842; `url` at line 2843; `currentRoute` at line 2844; `method` at line 2845. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2840: function syncSectionRouteToHistory(projectId, sectionIndex, resourcePath = "", mode = "push") {
2841:   if (isApplyingSectionRoute || mode === "none" || !canUseSectionHistory()) return;
2842:   const route = sectionRouteState(projectId, sectionIndex, resourcePath);
2843:   const url = sectionRouteUrl(projectId, sectionIndex, resourcePath);
2844:   const currentRoute = sectionRouteFromState(history.state) || sectionRouteFromLocation();
2845:   const method = mode === "replace" || sameSectionRoute(currentRoute, route) ? "replaceState" :
"pushState";
2846:   try {
2847:     history[method](route, "", url);
2848:   } catch {
2849:     // Preview iframes and restricted browser contexts may reject history writes.
2850:   }
2851: }
```

clearSectionRouteInHistory (script.js, lines 2853-2860)

`clearSectionRouteInHistory` part of the public website runtime. In this file, it appears around lines 2853-2860 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `canUseSectionHistory`, `replaceState`, and `sectionBaseUrl`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2854: `if (isApplyingSectionRoute || !canUseSectionHistory()) return;`

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2853: function clearSectionRouteInHistory() {
2854:   if (isApplyingSectionRoute || !canUseSectionHistory()) return;
2855:   try {
2856:     history.replaceState({ appWindow: "portfolio-base" }, "", sectionBaseUrl());
2857:   } catch {
2858:     // The visible dialog is already closed; URL cleanup is best effort.
2859:   }
2860: }
```

initializeSectionHistoryState (script.js, lines 2862-2871)

`initializeSectionHistoryState` part of the public website runtime. In this file, it appears around lines 2862-2871 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `canUseSectionHistory`, `sectionRouteFromLocation`, `sectionRouteFromState`, `replaceState`, `sectionRouteState`, and `sectionRouteUrl`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2863: `if (!canUseSectionHistory()) return;`

Important local values include `route` at line 2864. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2862: function initializeSectionHistoryState() {
2863:   if (!canUseSectionHistory()) return;
2864:   const route = sectionRouteFromLocation() || sectionRouteFromState(history.state);
2865:   try {
2866:     if (route) history.replaceState(sectionRouteState(route.projectId, route.sectionIndex,
route.resourcePath), "", sectionRouteUrl(route.projectId, route.sectionIndex, route.resourcePath));
2867:     else if (!history.state?.appWindow) history.replaceState({ appWindow: "portfolio-base" }, "",
window.location.href);

```

```

2868:   } catch {
2869:     // Browser history state is progressive enhancement for the portfolio windows.
2870:   }
2871: }

```

nodeAtPath (script.js, lines 2873-2882)

nodeAtPath part of the public website runtime. In this file, it appears around lines 2873-2882 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2878: if (!node) return null;; line 2881: return node;.

Important local values include node at line 2874; children at line 2875. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2873: function nodeAtPath(section, path = []) {
2874:   let node = section;
2875:   let children = section?.items || [];
2876:   for (const index of path) {
2877:     node = children[index];
2878:     if (!node) return null;
2879:     children = node.children || [];
2880:   }
2881:   return node;
2882: }

```

nodeChildren (script.js, lines 2884-2886)

nodeChildren part of the public website runtime. In this file, it appears around lines 2884-2886 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2885: return node?.items || node?.children || [];

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2884: function nodeChildren(node) {
2885:   return node?.items || node?.children || [];
2886: }

```

richHasRenderableContent (script.js, lines 2888-2897)

richHasRenderableContent part of the public website runtime. In this file, it appears around lines 2888-2897 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2889: return Boolean(rich?.blocks?.some((block) =>.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2888: function richHasRenderableContent(rich) {
2889:   return Boolean(rich?.blocks?.some((block) =>
2890:     block.type === "image" && block.url ||
2891:     block.type === "formula" && block.formula ||
2892:     block.type === "code" && block.code ||
2893:     block.text ||
2894:     block.title ||
2895:     block.caption

```

```
2896:   });
2897: }
```

nodeHasRenderableContent (script.js, lines 2899-2910)

nodeHasRenderableContent part of the public website runtime. In this file, it appears around lines 2899-2910 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasRenderableContent, nodeChildren, some, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 2900: if (!node) return false;; line 2901: if (node.kind === "summary") return Boolean(node.description || richHasRenderableContent(node.rich));; line 2904: return Boolean(node.description || node.url || richHasRenderableContent(node.rich) || children.some(nodeHasRenderableContent));; line 2907: return Boolean(node.title || node.description || node.url || richHasRenderableContent(node.rich) || children.some(nodeHasRenderableContent));. There are 1 additional return statements in the function body.

Important local values include children at line 2902. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2899: function nodeHasRenderableContent(node) {
2900:   if (!node) return false;
2901:   if (node.kind === "summary") return Boolean(node.description || richHasRenderableContent(node.rich));
2902:   const children = nodeChildren(node);
2903:   if (node.kind === "subsection") {
2904:     return Boolean(node.description || node.url || richHasRenderableContent(node.rich) ||
children.some(nodeHasRenderableContent));
2905:   }
2906:   if (["tool", "language"].includes(node.kind)) {
2907:     return Boolean(node.title || node.description || node.url || richHasRenderableContent(node.rich) ||
children.some(nodeHasRenderableContent));
2908:   }
2909:   return Boolean(node.description || node.url || richHasRenderableContent(node.rich) ||
children.some(nodeHasRenderableContent));
2910: }
```

sectionHasRenderableContent (script.js, lines 2912-2914)

sectionHasRenderableContent part of the public website runtime. In this file, it appears around lines 2912-2914 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richHasRenderableContent, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2913: return Boolean(section?.description || richHasRenderableContent(section?.rich) || (section?.items || []).some(nodeHasRenderableContent));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2912: function sectionHasRenderableContent(section) {
2913:   return Boolean(section?.description || richHasRenderableContent(section?.rich) || (section?.items ||
[]).some(nodeHasRenderableContent));
2914: }
```

responsiveChildren (script.js, lines 2916-2918)

responsiveChildren part of the public website runtime. In this file, it appears around lines 2916-2918 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2917: return node?.items || node?.children || [];

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2916: function responsiveChildren(node) {  
2917:   return node?.items || node?.children || [];  
2918: }
```

richContainsMedia (script.js, lines 2920-2922)

richContainsMedia part of the public website runtime. In this file, it appears around lines 2920-2922 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2921: return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2920: function richContainsMedia(rich) {  
2921:   return Boolean(rich?.blocks?.some((block) => block.type === "image" && block.url));  
2922: }
```

responsiveNodeHasMedia (script.js, lines 2924-2930)

responsiveNodeHasMedia part of the public website runtime. In this file, it appears around lines 2924-2930 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references richContainsMedia, responsiveChildren, and some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 2925: if (!node) return false;; line 2926: if (node.kind === "image" || richContainsMedia(node.rich)) return true;; line 2928: if (/^(apng|avif|gif|jpe?g|png|svg|webp)(\?|#|\$)/i.test(url)) return true;; line 2929: return responsiveChildren(node).some(responsiveNodeHasMedia);.

Important local values include url at line 2927. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2924: function responsiveNodeHasMedia(node) {  
2925:   if (!node) return false;  
2926:   if (node.kind === "image" || richContainsMedia(node.rich)) return true;  
2927:   const url = String(node.url || "");  
2928:   if (/^(apng|avif|gif|jpe?g|png|svg|webp)(\?|#|$)/i.test(url)) return true;  
2929:   return responsiveChildren(node).some(responsiveNodeHasMedia);  
2930: }
```

responsiveNodeCount (script.js, lines 2932-2935)

responsiveNodeCount part of the public website runtime. In this file, it appears around lines 2932-2935 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references nodeHasRenderableContent, responsiveChildren, and reduce. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2933: if (!node || !nodeHasRenderableContent(node)) return 0;; line 2934: return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2932: function responsiveNodeCount(node) {  
2933:   if (!node || !nodeHasRenderableContent(node)) return 0;
```

```

2934:   return 1 + responsiveChildren(node).reduce((count, child) => count + responsiveNodeCount(child), 0);
2935: }

```

responsiveNodeDepth (script.js, lines 2937-2942)

responsiveNodeDepth part of the public website runtime. In this file, it appears around lines 2937-2942 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references nodeHasRenderableContent, responsiveChildren, filter, max, and map. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2938: if (!node || !nodeHasRenderableContent(node)) return 0;; line 2940: if (!children.length) return 1;; line 2941: return 1 + Math.max(...children.map(responsiveNodeDepth));.

Important local values include children at line 2939. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2937: function responsiveNodeDepth(node) {
2938:   if (!node || !nodeHasRenderableContent(node)) return 0;
2939:   const children = responsiveChildren(node).filter(nodeHasRenderableContent);
2940:   if (!children.length) return 1;
2941:   return 1 + Math.max(...children.map(responsiveNodeDepth));
2942: }

```

inferResponsiveProfile (script.js, lines 2944-2976)

inferResponsiveProfile part of the public website runtime. In this file, it appears around lines 2944-2976 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, sectionHasRenderableContent, reduce, responsiveNodeCount, max, map, some, responsiveNodeHasMedia, and hasPublicTemplate. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2961: return {.

Important local values include sections at line 2945; visibleSections at line 2946; itemCount at line 2947; maxDepth at line 2951; hasMedia at line 2955; density at line 2956. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

2944: function inferResponsiveProfile(project) {
2945:   const sections = project?.portfolioView?.sections || [];
2946:   const visibleSections = sections.filter((section) => section?.id !== "brief" &&
sectionHasRenderableContent(section));
2947:   const itemCount = visibleSections.reduce(
2948:     (count, section) => count + (section.items || []).reduce((sum, item) => sum +
responsiveNodeCount(item), 0),
2949:     0
2950:   );
2951:   const maxDepth = visibleSections.reduce(
2952:     (depth, section) => Math.max(depth, ...(section.items || []).map(responsiveNodeDepth), 1),
2953:     1
2954:   );
2955:   const hasMedia = visibleSections.some((section) => responsiveNodeHasMedia(section));
2956:   const density = itemCount >= 18 || visibleSections.length >= 6 || maxDepth >= 4
2957:     ? "dense"
2958:     : itemCount <= 6 && visibleSections.length <= 2
2959:     ? "simple"
2960:     : "balanced";
2961:   return {
2962:     version: 1,
2963:     breakpoints: {
2964:       mobile: 640,
2965:       tablet: 900,
2966:       desktop: 1180
2967:     },
2968:     cardLayout: hasPublicTemplate(project) ? "single" : "split",
2969:     contentMax: hasMedia || maxDepth >= 3 ? "1180px" : "980px",
2970:     density,
2971:     mediaMax: hasMedia ? "860px" : "720px",
2972:     sectionCardMin: density === "dense" ? "210px" : density === "simple" ? "280px" : "240px",
2973:     touchTarget: "44px",
2974:     windowMode: "full-screen"
2975:   };

```

```
2976: }
```

projectResponsiveProfile (script.js, lines 2978-2982)

projectResponsiveProfile part of the public website runtime. In this file, it appears around lines 2978-2982 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references inferResponsiveProfile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2980: if (savedProfile?.version) return { ...inferResponsiveProfile(project), ...savedProfile }; line 2981: return inferResponsiveProfile(project);.

Important local values include savedProfile at line 2979. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2978: function projectResponsiveProfile(project) {
2979:   const savedProfile = project?.portfolioView?.responsive;
2980:   if (savedProfile?.version) return { ...inferResponsiveProfile(project), ...savedProfile };
2981:   return inferResponsiveProfile(project);
2982: }
```

nodeSummary (script.js, lines 2984-2992)

nodeSummary part of the public website runtime. In this file, it appears around lines 2984-2992 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, and renderRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 2985: if (!rich?.blocks?.length && !text) return ""; line 2986: return `

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2984: function nodeSummary(title, rich, text, emptyMessage = "No summary has been added yet.") {
2985:   if (!rich?.blocks?.length && !text) return "";
2986:   return `
2987:     <details class="parsed-summary" open>
2988:       <summary>${escapeHtml(title)}</summary>
2989:       ${renderRichContent(rich, text || "")}
2990:     </details>
2991:   `;
2992: }
```

nodeIsOverview (script.js, lines 2994-2997)

nodeIsOverview part of the public website runtime. In this file, it appears around lines 2994-2997 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, displayTitle, and endsWith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2996: return item.kind === "summary" || title === "overview" || title.endsWith(" overview");.

Important local values include title at line 2995. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2994: function nodeIsOverview(item = {}) {
2995:   const title = normalize(displayTitle(item.title || item.label || ""));
2996:   return item.kind === "summary" || title === "overview" || title.endsWith(" overview");
2997: }
```

nodeOverviewDetails (script.js, lines 2999-3017)

nodeOverviewDetails part of the public website runtime. In this file, it appears around lines 2999-3017 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, push, renderRichContent, forEach, renderInlineSectionFiles, nodeChildren, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3010: if (!blocks.length) return ""; line 3011: return `

Important local values include overviewItems at line 3000; blocks at line 3001; itemFiles at line 3007. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
2999: function nodeOverviewDetails(node, children = []) {
3000:   const overviewItems = children.filter(nodeIsOverview);
3001:   const blocks = [];
3002:   if (node?.rich?.blocks?.length || node?.description) {
3003:     blocks.push(renderRichContent(node.rich, node.description || ""));
3004:   }
3005:   overviewItems.forEach((item) => {
3006:     if (item.rich?.blocks?.length || item.description) blocks.push(renderRichContent(item.rich,
item.description || ""));
3007:     const itemFiles = renderInlineSectionFiles(nodeChildren(item));
3008:     if (itemFiles) blocks.push(itemFiles);
3009:   });
3010:   if (!blocks.length) return "";
3011:   return `
3012:     <details class="parsed-summary section-overview-default" open>
3013:       <summary>Overview</summary>
3014:       ${blocks.join("")}
3015:     </details>
3016: `;
3017: }
```

parsedNodeCard (script.js, lines 3019-3032)

parsedNodeCard part of the public website runtime. In this file, it appears around lines 3019-3032 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references displayTitle, pathToString, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3021: return `

Important local values include title at line 3020. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3019: function parsedNodeCard(node, projectId, sectionIndex, path) {
3020:   const title = displayTitle(node.title);
3021:   return `
3022:     <button
3023:       class="section-open-card subsection-open-card"
3024:       type="button"
3025:       data-section-project="${projectId}"
3026:       data-section-index="${sectionIndex}"
3027:       data-resource-path="${pathToString(path)}"
3028:     >
3029:       <span>${escapeHtml(title)}</span>
3030:     </button>
3031: `;
3032: }
```

parsedChildCards (script.js, lines 3034-3042)

parsedChildCards part of the public website runtime. In this file, it appears around lines 3034-3042 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, nodeHasRenderableContent, nodeIsOverview, map, parsedNodeCard, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3036: if (!visibleChildren.length) return "";; line 3037: return `.

Important local values include visibleChildren at line 3035. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3034: function parsedChildCards(children, projectId, sectionIndex, basePath = []) {
3035:   const visibleChildren = children.filter((child) => nodeHasRenderableContent(child) && !child.url &&
!nodeIsOverview(child));
3036:   if (!visibleChildren.length) return "";
3037:   return `
3038:     <div class="subsection-grid section-content-grid">
3039:       ${children.map((child, index) => nodeHasRenderableContent(child) && !child.url &&
!nodeIsOverview(child) ? parsedNodeCard(child, projectId, sectionIndex, [...basePath, index]) : "").join("")}
3040:     </div>
3041:   `;
3042: }
```

parsedNodeContent (script.js, lines 3044-3058)

parsedNodeContent part of the public website runtime. In this file, it appears around lines 3044-3058 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderInlineSectionFiles, nodeChildren, filter, nodeIsOverview, nodeOverviewDetails, and parsedChildCards. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3046: if (!isContainer && node.url) return renderInlineSectionFiles([node]);; line 3051: return `.

Important local values include isContainer at line 3045; children at line 3048; contentChildren at line 3049. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3044: function parsedNodeContent(node, projectId, sectionIndex, path = []) {
3045:   const isContainer = !node.kind || node.kind === "subsection";
3046:   if (!isContainer && node.url) return renderInlineSectionFiles([node]);
3047:
3048:   const children = nodeChildren(node);
3049:   const contentChildren = children.filter((child) => !nodeIsOverview(child));
3050:
3051:   return `
3052:   <article class="parsed-window-panel section-directory">
3053:     ${nodeOverviewDetails(node, children)}
3054:     ${parsedChildCards(children, projectId, sectionIndex, path)}
3055:     ${renderInlineSectionFiles(contentChildren)}
3056:   </article>
3057:   `;
3058: }
```

parsedSectionContent (script.js, lines 3060-3064)

parsedSectionContent part of the public website runtime. In this file, it appears around lines 3060-3064 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references nodeAtPath, and parsedNodeContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3062: if (!node) return `<p class="evidence-empty">This section could not be found.</p>`;; line 3063: return parsedNodeContent(node, projectId, sectionIndex, path);.

Important local values include node at line 3061. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3060: function parsedSectionContent(section, projectId, sectionIndex, path = []) {
3061:   const node = path.length ? nodeAtPath(section, path) : section;
3062:   if (!node) return `<p class="evidence-empty">This section could not be found.</p>`;
```

```

3063:   return parsedNodeContent(node, projectId, sectionIndex, path);
3064: }

```

parsedSection (script.js, lines 3066-3080)

`parsedSection` part of the public website runtime. In this file, it appears around lines 3066-3080 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `filter`, `some`, `escapeHtml`, and `displayTitle`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3070: `return ``.

Important local values include `visibleItems` at line 3067; `hasImages` at line 3068. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3066: function parsedSection(section, index, project) {
3067:   const visibleItems = (section.items || []).filter(nodeHasRenderableContent);
3068:   const hasImages = visibleItems.some((item) => item.kind === "image");
3069:
3070:   return `
3071:     <button
3072:       class="section-open-card evidence-block ${hasImages ? "evidence-wide" : ""}"
3073:       type="button"
3074:       data-section-project="${projectId}"
3075:       data-section-index="${index}"
3076:     >
3077:       <span>${escapeHtml(displayTitle(section.title, "Section"))}</span>
3078:     </button>
3079:   `;
3080: }

```

projectCard (script.js, lines 3082-3164)

`projectCard` part of the public website runtime. In this file, it appears around lines 3082-3164 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so `script.js` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `find`, `filter`, `sectionHasRenderableContent`, `projectTemplateClass`, `projectResponsiveClass`, `projectTemplateStyle`, `renderParsedBriefBlock`, `map`, `parsedSection`, `join`, and 7 more. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3090: `return ``; line 3106: `return ``.

Important local values include `category` at line 3084; `accent` at line 3085; `sections` at line 3086; `briefSection` at line 3087; `otherSections` at line 3088; `category` at line 3103; `accent` at line 3104. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3082: function projectCard(project) {
3083:   if (project.portfolioView) {
3084:     const category = categories.find((item) => item.id === project.category) || {};
3085:     const accent = category.accent || "#117c7a";
3086:     const sections = project.portfolioView.sections || [];
3087:     const briefSection = sections.find((section) => section.id === "brief");
3088:     const otherSections = sections.filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
3089:
3090:     return `
3091:       <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${projectId}" style="${projectTemplateStyle(project, accent)}">
3092:         <div class="project-body">
3093:           <h3>${project.portfolioView.title || project.title}</h3>
3094:           ${renderParsedBriefBlock(briefSection, project.summary)}
3095:           <div class="evidence-grid" aria-label="${project.title} parsed project content">
3096:             ${otherSections.map((section, index) => parsedSection(section, index, project)).join("")}
3097:           </div>
3098:         </div>
3099:       </article>
3100:     `;
3101:   }
3102:
3103:   const category = categories.find((item) => item.id === project.category) || {};
3104:   const accent = category.accent || "#117c7a";

```

```

3105:
3106:   return `
3107:     <article class="project-card catalog-card ${projectTemplateClass(project)}
${projectResponsiveClass(project)}" id="${project.id}" style="${projectTemplateStyle(project, accent)}">
3108:       <div class="project-body">
3109:         <h3>${project.title}</h3>
3110:         <p class="rich-paragraph">${renderMultilineInlineText(project.summary)}</p>
3111:
3112:         ${project.highlights && project.highlights.length ? detailBlock("Project highlights", "project-
drawer", `
3113:           <ul class="highlight-list">
3114:             ${project.highlights.map((item) => `<li>${item}</li>`).join("")}
3115:           </ul>
3116:         `) : ""}
3117:
3118:         <div class="evidence-grid" aria-label="${project.title} evidence blocks">
3119:           ${detailBlock("Documents", "evidence-block", evidenceList(project.documents, (item) => `
3120:             <li>
3121:               ${resourceLink(item, item.title)}
3122:               <span>${item.type || "Document"} &middot; ${item.status || "tracked"}</span>
3123:             </li>
3124:           `, "No document artifact has been added yet.))}
3125:
3126:           ${detailBlock("Tests and results", "evidence-block", evidenceList(project.tests, (item) => `
3127:             <li>
3128:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3129:               <span>${item.method || "Validation"} &middot; ${item.status || "tracked"}</span>
3130:               ${item.result ? `<p>${item.result}</p>` : ""}
3131:             </li>
3132:           `, "No test artifact has been added yet.))}
3133:
3134:           ${detailBlock("PCBs built", "evidence-block", evidenceList(project.pcb, (item) => `
3135:             <li>
3136:               ${resourceLink({ url: item.artifact, status: item.status }, item.name)}
3137:               <span>${item.revision || "Revision"} &middot; ${item.status || "tracked"}</span>
3138:             </li>
3139:           `, "No PCB build has been added yet.))}
3140:
3141:           ${detailBlock("Images", "evidence-block evidence-wide", mediaGrid(project.media))}
3142:         ${customSectionBlocks(project)}
3143:       </div>
3144:
3145:       ${detailBlock("Tools and implementation files", "project-drawer", `
3146:         <div class="project-tooling">
3147:           <div>
3148:             <h4>Tools Used</h4>
3149:             <div class="project-meta">${pillList(project.tools, "tool-tag")}</div>
3150:           </div>
3151:           <div>
3152:             <h4>Languages</h4>
3153:             <div class="project-meta">${pillList(project.languages, "language-tag")}</div>
3154:           </div>
3155:         </div>
3156:       `)}
3157:
3158:       <div class="resource-list">
3159:         ${project.links || []}.map((item) => resourceLink(item)).join("")
3160:       </div>
3161:     </div>
3162:   </article>
3163: `;
3164: }

```

categorySection (script.js, lines 3166-3181)

categorySection part of the public website runtime. In this file, it appears around lines 3166-3181 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, map, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3167: return `.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3166: function categorySection(category, visibleProjects) {
3167:   return `
3168:     <section class="category-section ${visibleProjects.length ? "" : "empty-category-section"}" aria-
labelledby="${escapeHtml(category.id)}-title">
3169:       <div class="category-heading">
3170:         <div>
3171:           <h3 id="${escapeHtml(category.id)}-title">${escapeHtml(category.label)}</h3>
3172:         </div>

```

```

3173:         ${visibleProjects.length ? `<span>${visibleProjects.length} project${visibleProjects.length === 1
? "" : "s"}</span>` : ""}
3174:     </div>
3175:     ${visibleProjects.length && category.description ? `<p class="category-
description">${escapeHtml(category.description)}</p>` : ""}
3176:     <div class="category-projects">
3177:         ${visibleProjects.map(projectCard).join("")}
3178:     </div>
3179: </section>
3180: `;
3181: }

```

escapeRegExp (script.js, lines 3183-3185)

escapeRegExp part of the public website runtime. In this file, it appears around lines 3183-3185 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3184: return String(value).replace(/[/.*+?^\$(){}|\[\]\\]/g, "\\\$&");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3183: function escapeRegExp(value = "") {
3184:   return String(value).replace(/[/.*+?^$(){}|\[\]\\]/g, "\\$&");
3185: }

```

highlightQueryHtml (script.js, lines 3187-3193)

highlightQueryHtml part of the public website runtime. In this file, it appears around lines 3187-3193 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references escapeHtml, trim, escapeRegExp, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3190: if (!cleanQuery) return escaped;; line 3192: return escaped.replace(pattern, '<mark class="search-result-mark">\$1</mark>');

Important local values include escaped at line 3188; cleanQuery at line 3189; pattern at line 3191. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3187: function highlightQueryHtml(value = "", query = "") {
3188:   const escaped = escapeHtml(value);
3189:   const cleanQuery = String(query || "").trim();
3190:   if (!cleanQuery) return escaped;
3191:   const pattern = new RegExp(`${escapeRegExp(escapeHtml(cleanQuery))}`, "ig");
3192:   return escaped.replace(pattern, '<mark class="search-result-mark">$1</mark>');
3193: }

```

clearSearchHighlights (script.js, lines 3195-3197)

clearSearchHighlights part of the public website runtime. In this file, it appears around lines 3195-3197 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3195: function clearSearchHighlights() {
3196:   window.CSS?.highlights?.delete?.(searchHighlightName);

```

```
3197: }
```

textNodeSearchTargets (script.js, lines 3199-3204)

textNodeSearchTargets part of the public website runtime. In this file, it appears around lines 3199-3204 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3200: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3199: function textNodeSearchTargets() {
3200:   return [
3201:     document.querySelector("main"),
3202:     document.querySelector("#section-view-dialog[open] .section-view-content")
3203:   ].filter(Boolean);
3204: }
```

applySearchHighlights (script.js, lines 3206-3238)

applySearchHighlights part of the public website runtime. In this file, it appears around lines 3206-3238 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clearSearchHighlights, trim, normalize, textNodeSearchTargets, forEach, createTreeWalker, acceptNode, includes, closest, nextNode, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 3209: if (!cleanQuery || !window.CSS?.highlights || typeof window.Highlight !== "function") return 0;; line 3216: if (!node.textContent || !normalize(node.textContent).includes(needle)) return NodeFilter.FILTER_REJECT;; line 3217: if (node.parentElement?.closest("script, style, input, textarea, select, button, .search-engine, .site-header")) return NodeFilter.FILTER_REJECT;; line 3218: return NodeFilter.FILTER_ACCEPT;. There are 1 additional return statements in the function body.

Important local values include cleanQuery at line 3208; ranges at line 3210; needle at line 3211; walker at line 3214; node at line 3221; haystack at line 3223; index at line 3224; range at line 3226. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3206: function applySearchHighlights(query) {
3207:   clearSearchHighlights();
3208:   const cleanQuery = String(query || "").trim();
3209:   if (!cleanQuery || !window.CSS?.highlights || typeof window.Highlight !== "function") return 0;
3210:   const ranges = [];
3211:   const needle = normalize(cleanQuery);
3212:
3213:   textNodeSearchTargets().forEach((root) => {
3214:     const walker = document.createTreeWalker(root, NodeFilter.SHOW_TEXT, {
3215:       acceptNode(node) {
3216:         if (!node.textContent || !normalize(node.textContent).includes(needle)) return
NodeFilter.FILTER_REJECT;
3217:         if (node.parentElement?.closest("script, style, input, textarea, select, button, .search-engine,
.site-header")) return NodeFilter.FILTER_REJECT;
3218:         return NodeFilter.FILTER_ACCEPT;
3219:       }
3220:     });
3221:     let node = walker.nextNode();
3222:     while (node) {
3223:       const haystack = normalize(node.textContent);
3224:       let index = haystack.indexOf(needle);
3225:       while (index >= 0) {
3226:         const range = document.createRange();
3227:         range.setStart(node, index);
3228:         range.setEnd(node, index + cleanQuery.length);
3229:         ranges.push(range);
3230:         index = haystack.indexOf(needle, index + Math.max(1, cleanQuery.length));
3231:       }
3232:       node = walker.nextNode();
3233:     }
3234:   });
```

```

3235:
3236:   if (ranges.length) window.CSS.highlights.set(searchHighlightName, new window.Highlight(...ranges));
3237:   return ranges.length;
3238: }

```

queueSearchHighlights (script.js, lines 3240-3243)

queueSearchHighlights part of the public website runtime. In this file, it appears around lines 3240-3243 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references applySearchHighlights. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3240: function queueSearchHighlights() {
3241:   clearTimeout(searchHighlightTimer);
3242:   searchHighlightTimer = setTimeout(() => applySearchHighlights(searchInput?.value || ""), 30);
3243: }

```

ensureSearchPanel (script.js, lines 3245-3283)

ensureSearchPanel part of the public website runtime. In this file, it appears around lines 3245-3283 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createElement, replaceWith, append, querySelector, addEventListener, updateSearchDropdown, preventDefault, closest, and goToSearchResult. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3246: if (!searchInput || !searchWrap || searchPanel) return;; line 3279: if (!button) return;.

Important local values include shell at line 3247; button at line 3278; result at line 3280. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3245: function ensureSearchPanel() {
3246:   if (!searchInput || !searchWrap || searchPanel) return;
3247:   const shell = document.createElement("div");
3248:   shell.className = "search-engine";
3249:   searchWrap.replaceWith(shell);
3250:   shell.append(searchWrap);
3251:   searchPanel = document.createElement("div");
3252:   searchPanel.className = "search-results-panel";
3253:   searchPanel.hidden = true;
3254:   searchPanel.innerHTML = `
3255:     <div class="search-results-tools">
3256:       <span id="search-status">Type to search projects, files, sections, and phrases.</span>
3257:       <label>
3258:         <span>Show</span>
3259:         <select id="search-result-limit" aria-label="Number of search results">
3260:           <option value="5">5</option>
3261:           <option value="10" selected>10</option>
3262:           <option value="15">15</option>
3263:           <option value="20">20</option>
3264:         </select>
3265:       </label>
3266:     </div>
3267:     <div class="search-results-list" role="listbox" aria-label="Search results"></div>
3268:   `;
3269:   shell.append(searchPanel);
3270:   searchStatus = searchPanel.querySelector("#search-status");
3271:   searchLimitSelect = searchPanel.querySelector("#search-result-limit");
3272:   searchLimitSelect.addEventListener("change", () => {
3273:     searchResultLimit = Number(searchLimitSelect.value) || 10;
3274:     updateSearchDropdown();
3275:   });
3276:   searchPanel.addEventListener("mousedown", (event) => event.preventDefault());
3277:   searchPanel.addEventListener("click", (event) => {
3278:     const button = event.target.closest("[data-search-result-index]");
3279:     if (!button) return;
3280:     const result = currentSearchResults[Number(button.dataset.searchResultIndex)];

```

```

3281:     if (result) goToSearchResult(result);
3282:   });
3283: }

```

setSearchMessage (script.js, lines 3285-3289)

setSearchMessage part of the public website runtime. In this file, it appears around lines 3285-3289 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3286: if (!searchStatus) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3285: function setSearchMessage(message, state = "neutral") {
3286:   if (!searchStatus) return;
3287:   searchStatus.textContent = message;
3288:   searchStatus.dataset.state = state;
3289: }

```

renderSearchResults (script.js, lines 3291-3327)

renderSearchResults renders ui, markup, or display output. In this file, it appears around lines 3291-3327 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references ensureSearchPanel, querySelector, trim, searchResultsFor, slice, setAttribute, setSearchMessage, removeAttribute, map, highlightQueryHtml, and 4 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3293: if (!searchPanel) return;; line 3305: return;; line 3312: return;;.

Important local values include list at line 3294; cleanQuery at line 3295; limitedResults at line 3297; visibleText at line 3316. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3291: function renderSearchResults(query, visibleCount) {
3292:   ensureSearchPanel();
3293:   if (!searchPanel) return;
3294:   const list = searchPanel.querySelector(".search-results-list");
3295:   const cleanQuery = String(query || "").trim();
3296:   currentSearchResults = searchResultsFor(cleanQuery);
3297:   const limitedResults = currentSearchResults.slice(0, searchResultLimit);
3298:   searchPanel.hidden = !cleanQuery;
3299:   searchInput.setAttribute("aria-expanded", String(Boolean(cleanQuery)));
3300:
3301:   if (!cleanQuery) {
3302:     list.innerHTML = "";
3303:     setSearchMessage("Type to search projects, files, sections, and phrases.");
3304:     searchInput.removeAttribute("aria-invalid");
3305:     return;
3306:   }
3307:
3308:   if (!currentSearchResults.length) {
3309:     list.innerHTML = "";
3310:     setSearchMessage(`"${cleanQuery}" was not found on this portfolio.`, "not-found");
3311:     searchInput.setAttribute("aria-invalid", "true");
3312:     return;
3313:   }
3314:
3315:   searchInput.removeAttribute("aria-invalid");
3316:   const visibleText = visibleCount
3317:     ? `${visibleCount} visible match${visibleCount === 1 ? "" : "es"} highlighted.`
3318:     : "Matches found elsewhere. Choose a result below.";
3319:   setSearchMessage(`${visibleText} Showing ${limitedResults.length} of ${currentSearchResults.length}.`,
"found");
3320:   list.innerHTML = limitedResults.map((result, index) => `
3321:     <button class="search-result-item" type="button" role="option" data-search-result-index="${index}">
3322:       <span class="search-result-title">${highlightQueryHtml(result.title || "Untitled result",
cleanQuery)}</span>
3323:       <span class="search-result-meta">${escapeHtml([result.type, result.context].filter(Boolean).join("
/ ")))}</span>
3324:       ${searchSnippet(result.text, cleanQuery)} ? `<span class="search-result-

```

```

snippet">${highlightQueryHtml(searchSnippet(result.text, cleanQuery), cleanQuery)}</span>` : ""}
3325:   </button>
3326:   `).join("");
3327: }

```

updateSearchDropdown (script.js, lines 3329-3333)

updateSearchDropdown updates ui, state, cache, or stored data. In this file, it appears around lines 3329-3333 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references applySearchHighlights, and renderSearchResults. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include query at line 3330; visibleCount at line 3331. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3329: function updateSearchDropdown() {
3330:   const query = searchInput?.value || "";
3331:   const visibleCount = applySearchHighlights(query);
3332:   renderSearchResults(query, visibleCount);
3333: }

```

showSearchPanellIfNeeded (script.js, lines 3335-3338)

showSearchPanellIfNeeded controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3335-3338 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, and updateSearchDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3336: if (!searchInput?.value?.trim()) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3335: function showSearchPanelIfNeeded() {
3336:   if (!searchInput?.value?.trim()) return;
3337:   updateSearchDropdown();
3338: }

```

scrollToDomTarget (script.js, lines 3340-3352)

scrollToDomTarget part of the public website runtime. In this file, it appears around lines 3340-3352 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getElementById, closeSectionDialogForRoute, scrollIntoView, replaceState, and queueSearchHighlights. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3342: if (!target) return false;; line 3351: return true;.

Important local values include target at line 3341. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3340: function scrollToDomTarget(id) {
3341:   const target = id ? document.getElementById(id) : null;
3342:   if (!target) return false;
3343:   closeSectionDialogForRoute();
3344:   target.scrollIntoView({ behavior: "smooth", block: "start" });
3345:   try {
3346:     history.replaceState(history.state || { appWindow: "portfolio-base" }, "", `#${id}`);
3347:   } catch {
3348:     // Scrolling is still the important behavior.

```



```

3349:   }
3350:   queueSearchHighlights();
3351:   return true;
3352: }

```

goToSearchResult (script.js, lines 3354-3377)

goToSearchResult part of the public website runtime. In this file, it appears around lines 3354-3377 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references scrollToDomTarget, openParsedSection, queueSearchHighlights, getElementById, setActiveFilter, renderProjects, scrollIntoView, and open. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3355: if (!result) return;; line 3357: if (result.domTarget && scrollToDomTarget(result.domTarget)) return;; line 3362: return;; line 3373: return;.

Important local values include target at line 3370. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3354: function goToSearchResult(result) {
3355:   if (!result) return;
3356:   searchPanel.hidden = true;
3357:   if (result.domTarget && scrollToDomTarget(result.domTarget)) return;
3358:
3359:   if (result.projectId && result.sectionIndex !== undefined) {
3360:     openParsedSection(result.projectId, result.sectionIndex, result.resourcePath || "");
3361:     queueSearchHighlights();
3362:     return;
3363:   }
3364:
3365:   if (result.projectId) {
3366:     if (!document.getElementById(result.projectId)) {
3367:       setActiveFilter("all");
3368:       renderProjects();
3369:     }
3370:     const target = document.getElementById(result.projectId);
3371:     target?.scrollIntoView({ behavior: "smooth", block: "center" });
3372:     queueSearchHighlights();
3373:     return;
3374:   }
3375:
3376:   if (result.url) window.open(result.url, "_blank", "noopener");
3377: }

```

handleSearchInput (script.js, lines 3379-3382)

handleSearchInput handles an event, request, command, or user action. In this file, it appears around lines 3379-3382 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references renderProjects, and updateSearchDropdown. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3379: function handleSearchInput() {
3380:   renderProjects();
3381:   updateSearchDropdown();
3382: }

```

renderProjects (script.js, lines 3384-3409)

renderProjects renders ui, markup, or display output. In this file, it appears around lines 3384-3409 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references normalize, trim, filter, projectVisible, queueSearchHighlights, map, categorySection, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3398: return;; line 3405: return visibleProjects.length || shouldShowEmptyCategory ? categorySection(category, visibleProjects) : "";

Important local values include query at line 3385; visible at line 3386; visibleProjects at line 3403; shouldShowEmptyCategory at line 3404. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3384: function renderProjects() {
3385:   const query = normalize(searchInput.value).trim();
3386:   const visible = projects.filter((project) => projectVisible(project, query));
3387:
3388:   projectCount.textContent = projects.length;
3389:
3390:   if (!visible.length) {
3391:     grid.innerHTML = `
3392:       <div class="empty-state">
3393:         <h3>No projects match that view.</h3>
3394:         <p>Search covers categories, project names, documents, tests, PCB builds, tools, languages, and
links.</p>
3395:       </div>
3396:     `;
3397:     queueSearchHighlights();
3398:     return;
3399:   }
3400:
3401:   grid.innerHTML = categories
3402:     .map((category) => {
3403:       const visibleProjects = visible.filter((project) => project.category === category.id);
3404:       const shouldShowEmptyCategory = !query && (activeFilter === "all" || activeFilter === category.id);
3405:       return visibleProjects.length || shouldShowEmptyCategory ? categorySection(category,
visibleProjects) : "";
3406:     })
3407:     .join("");
3408:   queueSearchHighlights();
3409: }
```

ensureSectionDialog (script.js, lines 3411-3470)

ensureSectionDialog part of the public website runtime. In this file, it appears around lines 3411-3470 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, createElement, append, addEventListener, closeSectionDialog, clearSectionRouteInHistory, remove, updateSectionDialogMinimize, focus, pathFromString, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 3413: if (dialog) return dialog;; line 3449: return;; line 3460: if (!nextPath) return;; line 3466: if (!nodeButton) return;. There are 1 additional return statements in the function body.

Important local values include dialog at line 3412; returnTarget at line 3440; path at line 3446; stack at line 3451; stack at line 3458; nextPath at line 3459; nodeButton at line 3465. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3411: function ensureSectionDialog() {
3412:   let dialog = document.querySelector("#section-view-dialog");
3413:   if (dialog) return dialog;
3414:
3415:   dialog = document.createElement("dialog");
3416:   dialog.id = "section-view-dialog";
3417:   dialog.className = "section-view-dialog";
3418:   dialog.innerHTML = `
3419:     <div class="section-view-shell">
3420:       <div class="section-view-heading">
3421:         <div class="section-view-titlebar">
3422:           <button class="section-view-back" type="button" title="Previous view" aria-label="Previous
view" hidden>&larr;</button>
3423:           <button class="section-view-forward" type="button" title="Next view" aria-label="Next view"
hidden>&rarr;</button>
3424:           <h2 id="section-view-title">Section</h2>
3425:         </div>
3426:         <div class="section-view-actions">
3427:           <button class="section-view-close" type="button" aria-label="Close section">&times;</button>
3428:         </div>
3429:       </div>
3430:       <div class="section-view-content" id="section-view-content"></div>
3431:     </div>
```

```

3432: `;
3433: document.body.append(dialog);
3434: dialog.querySelector(".section-view-close").addEventListener("click", () =>
closeSectionDialog(dialog));
3435: dialog.addEventListener("cancel", () => clearSectionRouteInHistory());
3436: dialog.addEventListener("close", () => {
3437:     document.body.classList.remove("full-window-open");
3438:     dialog.classList.remove("is-minimized-dialog");
3439:     updateSectionDialogMinimize(dialog);
3440:     const returnTarget = document.querySelector(
3441:         `[data-section-project="${dialog.dataset.projectId}"][data-section-
index="${dialog.dataset.sectionIndex}"]`
3442:     );
3443:     returnTarget?.focus({ preventScroll: true });
3444: });
3445: dialog.querySelector(".section-view-back").addEventListener("click", () => {
3446:     const path = pathFromString(dialog.dataset.resourcePath || "");
3447:     if (!path.length) {
3448:         closeSectionDialog(dialog);
3449:         return;
3450:     }
3451:     const stack = sectionForwardStack(dialog);
3452:     stack.push(pathToString(path));
3453:     dialog.dataset.forwardStack = JSON.stringify(stack);
3454:     path.pop();
3455:     openParsedSection(dialog.dataset.projectId, dialog.dataset.sectionIndex, pathToString(path), {
historyMode: "replace", preserveForward: true });
3456: });
3457: dialog.querySelector(".section-view-forward").addEventListener("click", () => {
3458:     const stack = sectionForwardStack(dialog);
3459:     const nextPath = stack.pop();
3460:     if (!nextPath) return;
3461:     dialog.dataset.forwardStack = JSON.stringify(stack);
3462:     openParsedSection(dialog.dataset.projectId, dialog.dataset.sectionIndex, nextPath, { historyMode:
"replace", preserveForward: true });
3463: });
3464: dialog.querySelector("#section-view-content").addEventListener("click", (event) => {
3465:     const nodeButton = event.target.closest("[data-resource-path]");
3466:     if (!nodeButton) return;
3467:     openParsedSection(nodeButton.dataset.sectionProject, nodeButton.dataset.sectionIndex,
nodeButton.dataset.resourcePath);
3468: });
3469: return dialog;
3470: }

```

sectionForwardStack (script.js, lines 3472-3478)

sectionForwardStack part of the public website runtime. In this file, it appears around lines 3472-3478 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references parse. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3474: return JSON.parse(dialog.dataset.forwardStack || "[]"); line 3476: return [];

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3472: function sectionForwardStack(dialog) {
3473:     try {
3474:         return JSON.parse(dialog.dataset.forwardStack || "[]");
3475:     } catch {
3476:         return [];
3477:     }
3478: }

```

updateSectionNavigation (script.js, lines 3480-3486)

updateSectionNavigation updates ui, state, cache, or stored data. In this file, it appears around lines 3480-3486 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, setAttribute, and sectionForwardStack. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include back at line 3481. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3480: function updateSectionNavigation(dialog, path) {
3481:   const back = dialog.querySelector(".section-view-back");
3482:   back.hidden = false;
3483:   back.title = path.length ? "Previous view" : "Back to projects";
3484:   back.setAttribute("aria-label", path.length ? "Previous view" : "Back to projects");
3485:   dialog.querySelector(".section-view-forward").hidden = !sectionForwardStack(dialog).length;
3486: }
```

closeSectionDialog (script.js, lines 3488-3491)

closeSectionDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3488-3491 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references close, and clearSectionRouteInHistory. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3488: function closeSectionDialog(dialog) {
3489:   dialog.close();
3490:   clearSectionRouteInHistory();
3491: }
```

closeOrStepBackSectionDialog (script.js, lines 3493-3504)

closeOrStepBackSectionDialog controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3493-3504 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pathFromString, sectionForwardStack, push, pathToString, stringify, pop, openParsedSection, and closeSectionDialog. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3501: return;.

Important local values include path at line 3494; stack at line 3496. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3493: function closeOrStepBackSectionDialog(dialog) {
3494:   const path = pathFromString(dialog.dataset.resourcePath || "");
3495:   if (path.length) {
3496:     const stack = sectionForwardStack(dialog);
3497:     stack.push(pathToString(path));
3498:     dialog.dataset.forwardStack = JSON.stringify(stack);
3499:     path.pop();
3500:     openParsedSection(dialog.dataset.projectId, dialog.dataset.sectionIndex, pathToString(path), {
historyMode: "replace", preserveForward: true });
3501:     return;
3502:   }
3503:   closeSectionDialog(dialog);
3504: }
```

openParsedSection (script.js, lines 3506-3532)

This function opens a recruiter-facing section window from parsed project data. It handles route state, section history, content rendering, and navigation controls.

The public website uses clickable sections instead of one long flat project page. This function drives that section-window experience.

It calls or references find, filter, sectionHasRenderableContent, pathFromString, nodeAtPath, nodeHasRenderableContent, ensureSectionDialog, applyProjectTemplateToElement, pathToString, syncSectionRouteToHistory, and 7 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 3510: if (!section) return false;; line 3513: if (!node) return false;; line 3514: if (path.length ? !nodeHasRenderableContent(node) : !sectionHasRenderableContent(section)) return false;; line 3531: return true;.

Important local values include project at line 3507; sections at line 3508; section at line 3509; path at line 3511; node at line 3512; dialog at line 3516; category at line 3517. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3506: function openParsedSection(projectId, sectionIndex, resourcePath = "", options = {}) {
3507:   const project = projects.find((item) => item.id === projectId);
3508:   const sections = (project?.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
3509:   const section = sections[Number(sectionIndex)];
3510:   if (!section) return false;
3511:   const path = pathFromString(resourcePath);
3512:   const node = path.length ? nodeAtPath(section, path) : section;
3513:   if (!node) return false;
3514:   if (path.length ? !nodeHasRenderableContent(node) : !sectionHasRenderableContent(section)) return
false;
3515:
3516:   const dialog = ensureSectionDialog();
3517:   const category = categories.find((item) => item.id === project.category) || {};
3518:   applyProjectTemplateToElement(dialog, project, category.accent || "#117c7a");
3519:   dialog.dataset.projectId = projectId;
3520:   dialog.dataset.sectionIndex = String(sectionIndex);
3521:   dialog.dataset.resourcePath = pathToString(path);
3522:   if (!options.preserveForward) dialog.dataset.forwardStack = "[]";
3523:   syncSectionRouteToHistory(projectId, sectionIndex, pathToString(path), options.historyMode || "push");
3524:   dialog.querySelector("#section-view-title").textContent = displayTitle(node.title || section.title,
"Section");
3525:   updateSectionNavigation(dialog, path);
3526:   dialog.querySelector("#section-view-content").innerHTML = parsedSectionContent(section, projectId,
Number(sectionIndex), path);
3527:   document.body.classList.add("full-window-open");
3528:   if (!dialog.open) dialog.showModal();
3529:   dialog.scrollTop = 0;
3530:   queueSearchHighlights();
3531:   return true;
3532: }
```

sections (script.js, lines 3508-3514)

sections part of the public website runtime. In this file, it appears around lines 3508-3514 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references filter, sectionHasRenderableContent, pathFromString, nodeAtPath, and nodeHasRenderableContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3510: if (!section) return false;; line 3513: if (!node) return false;; line 3514: if (path.length ? !nodeHasRenderableContent(node) : !sectionHasRenderableContent(section)) return false;.

Important local values include sections at line 3508; section at line 3509; path at line 3511; node at line 3512. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3508:   const sections = (project?.portfolioView?.sections || []).filter((section) => section.id !== "brief" &&
sectionHasRenderableContent(section));
3509:   const section = sections[Number(sectionIndex)];
3510:   if (!section) return false;
3511:   const path = pathFromString(resourcePath);
3512:   const node = path.length ? nodeAtPath(section, path) : section;
3513:   if (!node) return false;
3514:   if (path.length ? !nodeHasRenderableContent(node) : !sectionHasRenderableContent(section)) return
false;
```

closeSectionDialogForRoute (script.js, lines 3534-3537)

closeSectionDialogForRoute controls a window, dialog, panel, menu, or visible state. In this file, it appears around lines 3534-3537 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references querySelector, and close. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include dialog at line 3535. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3534: function closeSectionDialogForRoute() {
3535:   const dialog = document.querySelector("#section-view-dialog");
3536:   if (dialog?.open) dialog.close();
3537: }
```

applySectionRoute (script.js, lines 3539-3555)

applySectionRoute part of the public website runtime. In this file, it appears around lines 3539-3555 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references closeSectionDialogForRoute, and openParsedSection. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3544: return;.

Important local values include opened at line 3547. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3539: function applySectionRoute(route) {
3540:   isApplyingSectionRoute = true;
3541:   try {
3542:     if (!route) {
3543:       closeSectionDialogForRoute();
3544:       return;
3545:     }
3546:
3547:     const opened = openParsedSection(route.projectId, route.sectionIndex, route.resourcePath, {
3548:       historyMode: "none",
3549:       preserveForward: true
3550:     });
3551:     if (!opened) closeSectionDialogForRoute();
3552:   } finally {
3553:     isApplyingSectionRoute = false;
3554:   }
3555: }
```

restoreSectionRouteAfterCatalogLoad (script.js, lines 3557-3560)

restoreSectionRouteAfterCatalogLoad part of the public website runtime. In this file, it appears around lines 3557-3560 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references initializeSectionHistoryState, applySectionRoute, sectionRouteFromLocation, and sectionRouteFromState. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3557: function restoreSectionRouteAfterCatalogLoad() {
3558:   initializeSectionHistoryState();
3559:   applySectionRoute(sectionRouteFromLocation() || sectionRouteFromState(history.state));
3560: }
```

renderInlineSectionFiles (script.js, lines 3621-3639)

renderInlineSectionFiles renders ui, markup, or display output. In this file, it appears around lines 3621-3639 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references uniqueDownloads, map, normalizeDownloadAsset, filter, escapeHtml, normalizeLinkTarget, isWebsiteLinkItem, linkAttributes, downloadAttribute, fileNameFromUrl, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3623: if (!downloads.length) return ""; line 3625: return `.

Important local values include downloads at line 3622. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3621: function renderInlineSectionFiles(items = []) {
3622:   const downloads = uniqueDownloads(items.map((item) => normalizeDownloadAsset(item,
"File")).filter(Boolean));
3623:   if (!downloads.length) return "";
3624:
3625:   return `
3626:   <div class="section-file-list">
3627:     ${downloads.map((item) => `
3628:       <a
3629:         class="section-file-link"
3630:         href="${escapeHtml(normalizeLinkTarget(item.url, { assumeWeb: isWebsiteLinkItem(item, item.url)
3631:           ${linkAttributes(item.url, item)}
3632:           ${downloadAttribute(item.url, item)}
3633:         >
3634:           ${escapeHtml(item.title || fileNameFromUrl(item.url))}
3635:         </a>
3636:       `).join("")}
3637:   </div>
3638: `;
3639: }
```

normalizeTextColor (script.js, lines 3640-3647)

normalizeTextColor validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 3640-3647 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 3642: return /^#([0-9a-f]{3}|[0-9a-f]{6})\$/i.test(color) ||.

Important local values include color at line 3641. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
3640: function normalizeTextColor(value = "") {
3641:   const color = String(value || "").trim();
3642:   return /^#([0-9a-f]{3}|[0-9a-f]{6})$/i.test(color) ||
3643:     /^rgba?\(/i.test(color) ||
3644:     /^hsla?\(/i.test(color)
3645:     ? color
3646:     : "";
3647: }
```

loadProjectCatalog (script.js, lines 3648-3708)

loadProjectCatalog loads data from disk, network, browser storage, or a service. In this file, it appears around lines 3648-3708 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so script.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references hydrateProjectCategories, normalizeFieldStyles, normalizeSiteContent, normalizeProfile, normalizeFunFacts, renderProfile, renderSiteContent, renderFunFacts, renderSiteSections, renderCategoryFilters, and 11 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 3669: return;; line 3677: return response.json();.

Important local values include catalogUrl at line 3672. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

3648: function loadProjectCatalog() {
3649:   if (window.__PORTFOLIO_CATALOG__) {
3650:     projects = window.__PORTFOLIO_CATALOG__.projects || [];
3651:     categories = hydrateProjectCategories(window.__PORTFOLIO_CATALOG__.categories || [], projects);
3652:     siteSections = window.__PORTFOLIO_CATALOG__.siteSections || [];
3653:     fieldStyles = normalizeFieldStyles(window.__PORTFOLIO_CATALOG__.fieldStyles || {});
3654:     siteContent = normalizeSiteContent(window.__PORTFOLIO_CATALOG__.siteContent || {});
3655:     siteContentRich = window.__PORTFOLIO_CATALOG__.siteContentRich || {};
3656:     profile = normalizeProfile(window.__PORTFOLIO_CATALOG__.profile || {});
3657:     profileRich = window.__PORTFOLIO_CATALOG__.profileRich || {};
3658:     funFacts = normalizeFunFacts(window.__PORTFOLIO_CATALOG__.funFacts || []);
3659:     funFactsRich = window.__PORTFOLIO_CATALOG__.funFactsRich || null;
3660:     renderProfile();
3661:     renderSiteContent();
3662:     renderFunFacts();
3663:     renderSiteSections();
3664:     renderCategoryFilters();
3665:     renderProjects();
3666:     rebuildSearchIndex();
3667:     updateSearchDropdown();
3668:     restoreSectionRouteAfterCatalogLoad();
3669:     return;
3670:   }
3671:
3672:   const catalogUrl = new URL("projects.json", window.location.href);
3673:   catalogUrl.searchParams.set("v", String(Date.now()));
3674:   fetch(catalogUrl.href, { cache: "no-store" })
3675:     .then((response) => {
3676:       if (!response.ok) throw new Error("Could not load projects.json");
3677:       return response.json();
3678:     })
3679:     .then((data) => {
3680:       projects = data.projects || [];
3681:       categories = hydrateProjectCategories(data.categories || [], projects);
3682:       siteSections = data.siteSections || [];
3683:       fieldStyles = normalizeFieldStyles(data.fieldStyles || {});
3684:       siteContent = normalizeSiteContent(data.siteContent || {});
3685:       siteContentRich = data.siteContentRich || {};
3686:       profile = normalizeProfile(data.profile || {});
3687:       profileRich = data.profileRich || {};
3688:       funFacts = normalizeFunFacts(data.funFacts || []);
3689:       funFactsRich = data.funFactsRich || null;
3690:       renderProfile();
3691:       renderSiteContent();
3692:       renderFunFacts();
3693:       renderSiteSections();
3694:       renderCategoryFilters();
3695:       renderProjects();
3696:       rebuildSearchIndex();
3697:       updateSearchDropdown();
3698:       restoreSectionRouteAfterCatalogLoad();
3699:     })
3700:     .catch(() => {
3701:       grid.innerHTML = `
3702:         <div class="empty-state">
3703:           <h3>Project data did not load.</h3>
3704:           <p>The project catalog is temporarily unavailable.</p>
3705:         </div>
3706:       `;
3707:     });
3708: }

```

Representative opening snippet

```

1: const grid = document.querySelector("#project-grid");
2: const searchInput = document.querySelector("#project-search");
3: const projectFilters = document.querySelector("#project-filters");
4: let filterButtons = [...document.querySelectorAll(".filter-button")];
5: const projectCount = document.querySelector("#project-count");
6: const projectTrackCount = document.querySelector("#project-track-count");
7: const projectTrackLabels = document.querySelector("#project-track-labels");
8: const year = document.querySelector("#year");
9: const funFactsCallout = document.querySelector("#fun-facts-callout");
10: const heroEyebrow = document.querySelector(".hero-content .eyebrow");
11: const heroTitle = document.querySelector("#hero-title");
12: const heroCopy = document.querySelector(".hero-copy");
13: const brandName = document.querySelector(".brand strong");
14: const brandSubtitle = document.querySelector(".brand small");
15: const brandIcon = document.querySelector(".brand-icon");
16: const brandText = document.querySelector(".omb-engraving");
17: const headerAvatar = document.querySelector(".header-avatar");
18: const headerAvatarImage = document.querySelector(".header-avatar img");
19: const heroImage = document.querySelector(".hero-image");
20: const resumeSection = document.querySelector("#resume");
21: const contactBand = document.querySelector("#contact");
22: const profilePhoto = document.querySelector(".profile-photo");
23: const contactTitle = document.querySelector("#contact-title");
24: const contactIntro = document.querySelector(".contact-intro");
25: const contactDetails = document.querySelector(".contact-details");
26: const contactLinks = document.querySelector(".contact-links");

```



```

27: const footerOwner = document.querySelector(".site-footer span");
28: const searchWrap = searchInput?.closest(".search-wrap");
29: const aiAssistantForm = document.querySelector("#ai-assistant-form");
30: const aiAssistantPanel = document.querySelector("#ask-ai");
31: const aiAssistantInput = document.querySelector("#ai-assistant-input");
32: const aiAssistantLog = document.querySelector("#ai-assistant-log");
33: const aiAssistantStatus = document.querySelector("#ai-assistant-status");
34: const aiClearChatButton = document.querySelector("#ai-clear-chat");
35:
36: let categories = [];
37: let projects = [];
38: let siteSections = [];
39: let funFacts = [];
40: let funFactsRich = null;
41: let fieldStyles = {};
42: let siteContent = null;
43: let siteContentRich = {};
44: let profile = null;
45: let profileRich = {};
46: let activeFilter = "all";
47: let activeSectionDialogDrag = null;
48: let activeSectionDialogResize = null;
49: let sectionDialogDragEnabled = false;
50: let isApplyingSectionRoute = false;
51: let searchPanel = null;
52: let searchStatus = null;
53: let searchLimitSelect = null;
54: let currentSearchResults = [];
55: let assistantSourceCounter = 0;
56: let assistantSourceMap = new Map();
57: let assistantChatHistory = [];
58: let searchableEntries = [];
59: let searchResultLimit = 10;
60: let searchHighlightTimer = 0;
61: const searchHighlightName = "portfolio-search-highlight";
62:
63: const sectionRouteKey = "portfolio-section";
64: const sectionRouteParams = {
65:   path: "portfolioPath",
66:   project: "portfolioProject",
67:   section: "portfolioSection"
68: };
69:
70: const supportedCodeLanguages = [
71:   { id: "c", label: "C", aliases: ["c"] },
72:   { id: "cpp", label: "C++", aliases: ["cpp", "c++", "cplusplus"] },

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: index.html

Item	Explanation
File	index.html
Category	Website/builder UI source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	10,799 bytes
Extension	.html
MIME type	text/html
Text lines	268

Why this file exists

- Public website HTML shell that recruiters and visitors load in the browser.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Security or auth at line 11	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: <meta name="author" content="Portfolio Owner" />
Cloudflare or AI at line 12	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <meta name="portfolio-ai-endpoint" content="https://omb-portfolio-ai.maurice-baraza-otieno.workers.dev/api/portfolio-ai" />
Installer at line 17	Windows setup, update, shortcut, or installation behavior. Evidence: <link rel="shortcut icon" href="assets/favicon.ico" />
Network request at line 38	Frontend-backend communication or public web/API calls. Evidence: "@context": "https://schema.org",
Compiler or simulator at line 55	Part of the Compile Code workspace or language/tool detection. Evidence: "Python",
Network request at line 126	Frontend-backend communication or public web/API calls. Evidence: href="https://github.com/otienomaurice/omb-portfolio-builder/releases/latest/download/OMB-Portfolio-Builder-Setup-latest-x64.exe"
Compiler or simulator at line 160	Part of the Compile Code workspace or language/tool detection. Evidence: <input id="project-search" type="search" placeholder="LTspice, PCB, frequency response, Verilog..." />
Cloudflare or AI at line 242	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <div class="ai-assistant-log" id="ai-assistant-log" aria-live="polite" aria-label="AI chat messages"></div>
Cloudflare or AI at line 253	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: <button class="ai-clear-chat" id="ai-clear-chat" type="button">Clear chat</button>

JSON/YAML-style keys detected

Item	Explanation
@context	Line 38: "@context": "https://schema.org",
@type	Line 39: "@type": "ProfilePage",
dateModified	Line 40: "dateModified": "2026-06-04",
mainEntity	Line 41: "mainEntity": {

Item	Explanation
@type	Line 42: "@type": "Person",
name	Line 43: "name": "Portfolio Owner",
url	Line 44: "url": "",
image	Line 45: "image": "",
jobTitle	Line 46: "jobTitle": "",
description	Line 47: "description": "Portfolio owner profile.",
sameAs	Line 48: "sameAs": [],
knowsAbout	Line 49: "knowsAbout": [

Representative opening snippet

```

1: <!DOCTYPE html>
2: <html lang="en">
3:   <head>
4:     <meta charset="UTF-8" />
5:     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6:     <meta
7:       name="description"
8:       content="A portfolio website built with OMB Portfolio Builder for projects, documents, diagrams,
source code, tests, links, and profile information."
9:     />
10:    <meta name="robots" content="index, follow" />
11:    <meta name="author" content="Portfolio Owner" />
12:    <meta name="portfolio-ai-endpoint" content="https://omb-portfolio-ai.maurice-baraza-
otieno.workers.dev/api/portfolio-ai" />
13:    <link rel="canonical" href="" />
14:    <link rel="icon" type="image/png" sizes="16x16" href="assets/favicon-16.png" />
15:    <link rel="icon" type="image/png" sizes="32x32" href="assets/favicon-32.png" />
16:    <link rel="icon" type="image/png" sizes="192x192" href="assets/favicon-192.png" />
17:    <link rel="shortcut icon" href="assets/favicon.ico" />
18:    <link rel="apple-touch-icon" sizes="180x180" href="assets/apple-touch-icon.png" />
19:    <meta property="og:type" content="profile" />
20:    <meta property="og:title" content="Portfolio" />
21:    <meta
22:      property="og:description"
23:      content="Explore projects, documents, tests, PCBs, tools, links, and source code."
24:    />
25:    <meta property="og:url" content="" />
26:    <meta property="og:image" content="" />
27:    <meta name="twitter:card" content="summary_large_image" />
28:    <meta name="twitter:title" content="Portfolio" />
29:    <meta
30:      name="twitter:description"
31:      content="A project portfolio built with OMB Portfolio Builder."
32:    />
33:    <meta name="twitter:image" content="" />
34:    <title>Portfolio</title>
35:    <link rel="stylesheet" href="styles.css?v=2026062203" />
36:    <script type="application/ld+json">
37:      {
38:        "@context": "https://schema.org",
39:        "@type": "ProfilePage",
40:        "dateModified": "2026-06-04",
41:        "mainEntity": {
42:          "@type": "Person",
43:          "name": "Portfolio Owner",
44:          "url": "",
45:          "image": "",
46:          "jobTitle": "",
47:          "description": "Portfolio owner profile.",
48:          "sameAs": [],
49:          "knowsAbout": [
50:            "Embedded systems",
51:            "ASIC design",
52:            "Digital hardware",
53:            "Software engineering",
54:            "Microcontrollers",
55:            "Python",
56:            "C++"
57:          ]
58:        }
59:      }
60:    </script>
61:  </head>
62:  <body>
63:    <header class="site-header" id="top">
64:      <a class="brand" href="#top" aria-label="Go to top">
65:        <span class="brand-lockup" aria-hidden="true">

```

```
66:         
67:         <span class="omb-engraving">OMB</span>
68:     </span>
69: <span>
70:     <strong>Portfolio</strong>
71:     <small>Portfolio</small>
72: </span>
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: styles.css

Item	Explanation
File	styles.css
Category	Website/builder UI source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	53,685 bytes
Extension	.css
MIME type	text/css
Text lines	2,735

Why this file exists

- Public website CSS for layout, contact area, projects, responsive behavior, AI panel, and mobile/desktop rendering.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Rich text at line 2	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color-scheme: light;
Rich text at line 27	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 29	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
Rich text at line 38	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: inherit;
Rich text at line 99	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #fff;
Rich text at line 104	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.64rem;
Rich text at line 105	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;
Rich text at line 116	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);
Rich text at line 117	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 132	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);
Rich text at line 133	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.95rem;
Rich text at line 134	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 700;
Rich text at line 138	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--teal);
Rich text at line 196	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--teal);
Rich text at line 197	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 198	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 800;
Rich text at line 213	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: clamp(2.45rem, 6vw, 5.75rem);
Rich text at line 220	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: clamp(1.85rem, 4vw, 3.2rem);

Item	Explanation
Rich text at line 227	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 1.1rem;
Rich text at line 233	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #2e4d62;
Rich text at line 234	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: clamp(1rem, 2vw, 1.18rem);
Rich text at line 252	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 800;
Rich text at line 260	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #fff;
Rich text at line 277	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #083c4c;
Rich text at line 279	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 850;
Rich text at line 282	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: transition: background 160ms ease, border-color 160ms ease, color 160ms ease, transform 160ms ease;
Rich text at line 289	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: stroke: currentColor;
Rich text at line 297	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: rgba(11, 114, 136, 0.72);
Rich text at line 299	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #062f3b;
Rich text at line 334	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);
Rich text at line 356	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 357	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: clamp(0.98rem, 1.4vw, 1.12rem);
Rich text at line 373	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border: 1px solid color-mix(in srgb, var(--line), #ffff 18%);
Rich text at line 385	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 386	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 1rem;
Rich text at line 390	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--muted);
Rich text at line 391	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.92rem;
Rich text at line 434	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #086a7c;
Rich text at line 436	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 437	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 950;
Rich text at line 448	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: var(--ink);
Rich text at line 449	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: clamp(0.95rem, 1.3vw, 1.08rem);
Rich text at line 450	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 720;
Rich text at line 468	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #06384f;
Rich text at line 470	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.94rem;
Rich text at line 471	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;
Rich text at line 478	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: transition: background 140ms ease, border-color 140ms ease, color 140ms ease, transform 140ms ease;
Rich text at line 483	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #041c2a;
Rich text at line 484	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #47b8df;
Rich text at line 498	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: stroke: currentColor;
Rich text at line 547	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #162636;
Rich text at line 548	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: clamp(1.45rem, 2vw, 1.85rem);
Rich text at line 553	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #0b778d;
Rich text at line 554	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 555	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;

Item	Explanation
Cloudflare or AI at line 559	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: .ai-clear-chat {
Rich text at line 568	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #9f2222;
Rich text at line 570	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font: inherit;
Rich text at line 571	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.82rem;
Rich text at line 572	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 850;
Rich text at line 574	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: transition: background 150ms ease, border-color 150ms ease, color 150ms ease, transform 150ms ease;
Cloudflare or AI at line 577	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: .ai-clear-chat:hover,
Cloudflare or AI at line 578	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: .ai-clear-chat:focus-visible {
Rich text at line 579	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: #b42222;
Rich text at line 580	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #ffffff;
Cloudflare or AI at line 585	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: .ai-clear-chat:active {
Rich text at line 586	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #9f2222;
Rich text at line 592	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #8a5a00;
Rich text at line 596	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #a62929;
Rich text at line 625	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #23384b;
Rich text at line 639	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #23384b;
Rich text at line 644	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #102b3b;
Rich text at line 655	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #eaf6ff;
Rich text at line 656	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font: 0.82rem/1.5 Consolas, "Liberation Mono", "Courier New", monospace;
Rich text at line 661	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: inherit;
Rich text at line 666	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: border-color: rgba(16, 121, 144, 0.36);
Rich text at line 706	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #526d82;
Rich text at line 707	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-size: 0.78rem;
Rich text at line 708	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: font-weight: 900;
Rich text at line 716	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: color: #102b3b;

CSS selectors and visual hooks

Item	Explanation
Line 1	:root
Line 21	html
Line 25	body
Line 33	body.full-window-open
Line 37	a
Line 44	input:focus-visible
Line 49	.site-header
Line 64	.brand
Line 71	.brand-lockup
Line 79	.brand-icon
Line 89	.omb-engraving

Item	Explanation
Line 111	.brand small
Line 115	.brand small
Line 120	.header-right
Line 128	.site-nav
Line 137	.site-nav a:hover
Line 141	.header-avatar
Line 152	.header-avatar img
Line 160	.hero
Line 170	.hero-content
Line 174	.hero-image
Line 180	.hero-shade
Line 186	.hero-content
Line 194	.eyebrow
Line 206	p
Line 210	h1
Line 218	h2
Line 225	h3
Line 230	.hero-copy
Line 237	.hero-actions
Line 246	.resource-link
Line 255	.button
Line 259	.button.primary
Line 264	.button.secondary
Line 269	.ai-jump-link
Line 285	.ai-jump-link svg
Line 296	.ai-jump-link:focus-visible
Line 303	.resume-section
Line 308	.resume-actions
Line 314	.resume-viewer
Line 324	.resume-viewer object
Line 331	.resume-viewer p
Line 337	.dynamic-section
Line 347	.dynamic-section-surface
Line 354	.dynamic-section-copy
Line 362	.dynamic-section-grid
Line 368	.dynamic-section-card
Line 380	.dynamic-section-card p
Line 384	.dynamic-section-card h3
Line 389	.dynamic-section-card p
Line 396	.dynamic-link-list
Line 400	.fun-facts-section

Item	Explanation
Line 409	.fun-facts-section[hidden]
Line 413	.fun-facts-window
Line 427	.fun-facts-label
Line 441	.fun-facts-lines
Line 446	.fun-fact-line
Line 454	.builder-download-section
Line 462	.builder-download-link
Line 482	.builder-download-link:focus-visible
Line 489	.builder-download-link:active
Line 494	.builder-download-link svg
Line 505	.sr-only
Line 517	.ai-assistant-panel
Line 528	.ai-assistant-console
Line 545	.ai-assistant-console h2
Line 552	.ai-assistant-status
Line 559	.ai-clear-chat
Line 578	.ai-clear-chat:focus-visible
Line 585	.ai-clear-chat:active
Line 591	.ai-assistant-status[data-state="working"]
Line 595	.ai-assistant-status[data-state="error"]
Line 599	.ai-assistant-log
Line 613	.ai-message
Line 623	.ai-message p
Line 629	.ai-answer-content
Line 634	.ai-answer-content ul
Line 643	.ai-answer-content strong
Line 647	.ai-code-block
Line 660	.ai-code-block code
Line 664	.ai-message-user
Line 670	.ai-message-assistant
Line 674	.ai-message-pending p::after
Line 691	.ai-source-list
Line 698	.ai-source-panel
Line 704	.ai-source-heading
Line 713	.ai-assistant-form button
Line 724	.ai-source-list button
Line 732	.ai-source-list button:focus-visible
Line 738	.ai-assistant-form
Line 745	.ai-assistant-form input
Line 757	.ai-assistant-form input:focus-visible
Line 762	.ai-assistant-form button

Item	Explanation
Line 772	.ai-assistant-form button:focus-visible
Line 776	.summary-band
Line 784	.metric
Line 790	.metric strong
Line 796	.metric span
Line 803	.section
Line 807	.section-heading
Line 816	.search-wrap
Line 825	.search-engine
Line 830	.search-engine .search-wrap
Line 834	.search-wrap input
Line 845	::highlight(portfolio-search-highlight)
Line 850	.search-results-panel
Line 867	.search-results-panel[hidden]
Line 871	.search-results-tools
Line 883	.search-results-tools [data-state="not-found"]
Line 887	.search-results-tools [data-state="found"]
Line 891	.search-results-tools label
Line 898	.search-results-tools select
Line 909	.search-results-list
Line 914	.search-result-item
Line 928	.search-result-item:focus-visible
Line 934	.search-result-title
Line 940	.search-result-snippet
Line 947	.search-result-snippet
Line 951	.search-result-mark
Line 958	.filters

Representative opening snippet

```

1: :root {
2:   color-scheme: light;
3:   --ink: #17202a;
4:   --muted: #526d82;
5:   --line: #c9deea;
6:   --paper: #eef8fc;
7:   --panel: #ffffff;
8:   --teal: #0d7f91;
9:   --amber: #d18b21;
10:  --coral: #bc5648;
11:  --graphite: #223244;
12:  --omb-copper: #c7772d;
13:  --focus: #0b6fce;
14:  --shadow: 0 18px 48px rgba(23, 32, 42, 0.14);
15: }
16:
17: * {
18:   box-sizing: border-box;
19: }
20:
21: html {
22:   scroll-behavior: smooth;
23: }
24:
25: body {
26:   margin: 0;

```

```

27:   color: var(--ink);
28:   background: var(--paper);
29:   font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-
serif;
30:   line-height: 1.5;
31: }
32:
33: body.full-window-open {
34:   overflow: hidden;
35: }
36:
37: a {
38:   color: inherit;
39:   text-decoration: none;
40: }
41:
42: a:focus-visible,
43: button:focus-visible,
44: input:focus-visible {
45:   outline: 3px solid var(--focus);
46:   outline-offset: 3px;
47: }
48:
49: .site-header {
50:   position: sticky;
51:   top: 0;
52:   z-index: 20;
53:   display: flex;
54:   align-items: center;
55:   justify-content: space-between;
56:   gap: 24px;
57:   min-height: 72px;
58:   padding: 12px clamp(18px, 4vw, 56px);
59:   border-bottom: 1px solid rgba(201, 222, 234, 0.84);
60:   background: rgba(238, 248, 252, 0.92);
61:   backdrop-filter: blur(16px);
62: }
63:
64: .brand {
65:   display: inline-flex;
66:   align-items: center;
67:   gap: 12px;
68:   min-width: 0;
69: }
70:
71: .brand-lockup {
72:   position: relative;

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: cloudflare/portfolio-ai-worker.js

Item	Explanation
File	cloudflare/portfolio-ai-worker.js
Category	JavaScript source
Folder role	Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment notes.
Size	10,435 bytes
Extension	.js
MIME type	application/javascript
Text lines	276

Why this file exists

- Cloudflare Worker backend for the public portfolio AI assistant and fallback intelligence path.

How it is used

Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment notes.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

API endpoints mentioned or called

Item	Explanation
/api/portfolio-ai	Line 228. Handles AI assistant questions.

Important implementation signals

Item	Explanation
Network request at line 2	Frontend-backend communication or public web/API calls. Evidence: "https://mauriceotieno.com",
Network request at line 3	Frontend-backend communication or public web/API calls. Evidence: "https://www.mauriceotieno.com",
Network request at line 4	Frontend-backend communication or public web/API calls. Evidence: "https://otienomaurice.github.io",
Network request at line 5	Frontend-backend communication or public web/API calls. Evidence: "http://localhost:8080",
Network request at line 6	Frontend-backend communication or public web/API calls. Evidence: "http://localhost:8081",
Network request at line 7	Frontend-backend communication or public web/API calls. Evidence: "http://127.0.0.1:8080",
Network request at line 8	Frontend-backend communication or public web/API calls. Evidence: "http://127.0.0.1:8081"
Security or auth at line 11	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function json(data, status = 200, headers = {}) {
Security or auth at line 14	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: headers: {
Local storage at line 16	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: "Cache-Control": "no-store",
Security or auth at line 17	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ...headers
Security or auth at line 30	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function corsHeaders(request, env) {
Security or auth at line 31	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const origin = request.headers.get("Origin") "";

Item	Explanation
Security or auth at line 36	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Access-Control-Allow-Headers": "Content-Type",
Cloudflare or AI at line 54	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: function extractOpenAiText(data = {}) {
Security or auth at line 89	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Never invent portfolio projects, credentials, files, test results, or private repository access.",
Cloudflare or AI at line 121	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: async function callOpenAi(body, env) {
Cloudflare or AI at line 122	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const model = env.OPENAI_MODEL "gpt-4.1-mini";
Cloudflare or AI at line 124	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model,
Cloudflare or AI at line 135	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: max_output_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS 1000)
Cloudflare or AI at line 138	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const response = await fetch("https://api.openai.com/v1/responses", {
Security or auth at line 140	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: headers: {
Cloudflare or AI at line 141	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Authorization": `Bearer \${env.OPENAI_API_KEY}`,
Cloudflare or AI at line 151	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: error: data?.error?.message "OpenAI request failed."
Cloudflare or AI at line 156	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: answer: extractOpenAiText(data),
Cloudflare or AI at line 157	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model
Cloudflare or AI at line 166	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: error: "Cloudflare Workers AI binding is not available for this Worker."
Cloudflare or AI at line 170	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const model = env.WORKERS_AI_MODEL "@cf/meta/llama-3.2-3b-instruct";
Cloudflare or AI at line 171	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const data = await env.AI.run(model, {
Cloudflare or AI at line 173	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: max_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS 1000)
Cloudflare or AI at line 178	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model
Cloudflare or AI at line 210	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "The full AI model is not reachable from this Worker right now, so this is a fallback answer from the available context."
Cloudflare or AI at line 214	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "I can answer that in a general way, but the full AI model is not reachable from this Worker right now.",
Network request at line 223	Frontend-backend communication or public web/API calls. Evidence: async fetch(request, env) {
Security or auth at line 224	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const cors = corsHeaders(request, env);
Security or auth at line 225	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: if (request.method === "OPTIONS") return new Response(null, { status: 204, headers: cors });
Cloudflare or AI at line 228	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: if (request.method !== "POST" !url.pathname.endsWith("/api/portfolio-ai")) {
Cloudflare or AI at line 243	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: let provider = env.OPENAI_API_KEY ? "openai" : "cloudflare-workers-ai";
Cloudflare or AI at line 244	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: let result = env.OPENAI_API_KEY
Cloudflare or AI at line 245	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ? await callOpenAi({ ...body, question }, env)
Cloudflare or AI at line 247	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: if (!result.ok && env.OPENAI_API_KEY && env.AI) {
Cloudflare or AI at line 248	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: provider = "cloudflare-workers-ai";

Item	Explanation
Cloudflare or AI at line 254	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model: "worker-fallback",
Cloudflare or AI at line 262	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model: result.model,
Cloudflare or AI at line 269	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: model: "worker-fallback",

Important constants and variables

Item	Explanation
defaultAllowedOrigins	Line 1: const defaultAllowedOrigins = [
origin	Line 31: const origin = request.headers.get("Origin") "";
allowOrigin	Line 32: const allowOrigin = allowedOrigins(env).includes(origin) ? origin : defaultAllowedOrigins[0];
parts	Line 56: const parts = [];
messages	Line 96: const messages = [{ role: "system", content: buildInstructions() }];
model	Line 122: const model = env.OPENAI_MODEL "gpt-4.1-mini";
payload	Line 123: const payload = {
response	Line 138: const response = await fetch("https://api.openai.com/v1/responses", {
data	Line 146: const data = await response.json().catch(() => ({}));
model	Line 170: const model = env.WORKERS_AI_MODEL "@cf/meta/llama-3.2-3b-instruct";
data	Line 171: const data = await env.AI.run(model, {
question	Line 183: const question = String(body.question "").trim();
clean	Line 184: const clean = question.toLowerCase();
context	Line 185: const context = body.context {};
projects	Line 186: const projects = Array.isArray(context.projects) ? context.projects : [];
owner	Line 187: const owner = context.owner context.profile?.displayName "Maurice Otieno";
cors	Line 224: const cors = corsHeaders(request, env);
url	Line 227: const url = new URL(request.url);
body	Line 232: let body = {};
question	Line 239: const question = clamp(body.question, 1200).trim();
provider	Line 243: let provider = env.OPENAI_API_KEY ? "openai" : "cloudflare-workers-ai";
result	Line 244: let result = env.OPENAI_API_KEY

JSON/YAML-style keys detected

Item	Explanation
Content-Type	Line 15: "Content-Type": "application/json; charset=utf-8",
Cache-Control	Line 16: "Cache-Control": "no-store",
Access-Control-Allow-Origin	Line 34: "Access-Control-Allow-Origin": allowOrigin,
Access-Control-Allow-Methods	Line 35: "Access-Control-Allow-Methods": "POST, OPTIONS",
Access-Control-Allow-Headers	Line 36: "Access-Control-Allow-Headers": "Content-Type",
Access-Control-Max-Age	Line 37: "Access-Control-Max-Age": "86400",
Vary	Line 38: "Vary": "Origin"
Authorization	Line 141: "Authorization": `Bearer \${env.OPENAI_API_KEY}`,
Content-Type	Line 142: "Content-Type": "application/json"

Function-by-function detail

json (cloudflare/portfolio-ai-worker.js, lines 11-20)

json named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 11-20 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Response, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 12: return new Response(JSON.stringify(data, null, 2), {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
11: function json(data, status = 200, headers = {}) {
12:   return new Response(JSON.stringify(data, null, 2), {
13:     status,
14:     headers: {
15:       "Content-Type": "application/json; charset=utf-8",
16:       "Cache-Control": "no-store",
17:       ...headers
18:     }
19:   });
20: }
```

allowedOrigins (cloudflare/portfolio-ai-worker.js, lines 22-28)

allowedOrigins named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 22-28 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references split, map, trim, filter, and concat. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 23: return String(env.ALLOWED_ORIGINS || "").

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
22: function allowedOrigins(env) {
23:   return String(env.ALLOWED_ORIGINS || "")
24:     .split(",")
25:     .map((origin) => origin.trim())
26:     .filter(Boolean)
27:     .concat(defaultAllowedOrigins);
28: }
```

corsHeaders (cloudflare/portfolio-ai-worker.js, lines 30-40)

corsHeaders named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 30-40 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, allowedOrigins, and includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 33: return {.

Important local values include origin at line 31; allowOrigin at line 32. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
30: function corsHeaders(request, env) {
31:   const origin = request.headers.get("Origin") || "";
32:   const allowOrigin = allowedOrigins(env).includes(origin) ? origin : defaultAllowedOrigins[0];
33:   return {
34:     "Access-Control-Allow-Origin": allowOrigin,
```

```

35:     "Access-Control-Allow-Methods": "POST, OPTIONS",
36:     "Access-Control-Allow-Headers": "Content-Type",
37:     "Access-Control-Max-Age": "86400",
38:     "Vary": "Origin"
39:   };
40: }

```

clamp (cloudflare/portfolio-ai-worker.js, lines 42-44)

clamp named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 42-44 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slice. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 43: return String(value || "").slice(0, maxLength);.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

42: function clamp(value, maxLength) {
43:   return String(value || "").slice(0, maxLength);
44: }

```

cleanConversation (cloudflare/portfolio-ai-worker.js, lines 46-52)

cleanConversation named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 46-52 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isArray, slice, map, clamp, and filter. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 47: if (!Array.isArray(conversation)) return [];; line 48: return conversation.slice(-8).map((item) => ({

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

46: function cleanConversation(conversation = []) {
47:   if (!Array.isArray(conversation)) return [];
48:   return conversation.slice(-8).map((item) => ({
49:     role: item?.role === "assistant" ? "assistant" : "user",
50:     content: clamp(item?.content || item?.text || "", 1400)
51:   })).filter((item) => item.content);
52: }

```

extractOpenAiText (cloudflare/portfolio-ai-worker.js, lines 54-64)

extractOpenAiText named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 54-64 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, push, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 55: if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();; line 63: return parts.join("\n").trim();.

Important local values include parts at line 56. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

54: function extractOpenAiText(data = {}) {
55:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
56:   const parts = [];

```



```

57:   for (const item of data.output || []) {
58:     for (const content of item.content || []) {
59:       if (typeof content.text === "string") parts.push(content.text);
60:       if (typeof content.output_text === "string") parts.push(content.output_text);
61:     }
62:   }
63:   return parts.join("\n").trim();
64: }

```

extractWorkersAiText (cloudflare/portfolio-ai-worker.js, lines 66-79)

extractWorkersAiText named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 66-79 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references trim, isArray, map, filter, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 9 return statement(s). The most important visible returns are: line 67: if (typeof data.response === "string" && data.response.trim()) return data.response.trim(); line 68: if (typeof data.result?.response === "string" && data.result.response.trim()) return data.result.response.trim(); line 69: if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim(); line 71: return data.output.map((item) => {. There are 5 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

66: function extractWorkersAiText(data = {}) {
67:   if (typeof data.response === "string" && data.response.trim()) return data.response.trim();
68:   if (typeof data.result?.response === "string" && data.result.response.trim()) return
data.result.response.trim();
69:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
70:   if (Array.isArray(data.output)) {
71:     return data.output.map((item) => {
72:       if (typeof item === "string") return item;
73:       if (typeof item?.text === "string") return item.text;
74:       if (typeof item?.content === "string") return item.content;
75:       return "";
76:     }).filter(Boolean).join("\n").trim();
77:   }
78:   return "";
79: }

```

buildInstructions (cloudflare/portfolio-ai-worker.js, lines 81-93)

buildInstructions creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 81-93 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 82: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

81: function buildInstructions() {
82:   return [
83:     "You are the AI assistant for a public engineering portfolio website.",
84:     "Answer naturally like a helpful technical mentor and recruiter-facing portfolio guide.",
85:     "If the question is general, answer the concept directly before mentioning portfolio context.",
86:     "If the question is about the portfolio owner, projects, files, resume, tools, links, or sections,
answer from the supplied portfolio context.",
87:     "Use recent conversation to keep a conversational flow. Greetings should receive short greetings.",
88:     "Only include portfolio links when they are directly relevant to the visitor's question.",
89:     "Never invent portfolio projects, credentials, files, test results, or private repository access.",
90:     "If a requested detail is not in the context, say what is missing and give a useful next step.",
91:     "Keep answers concise, readable, and specific. Use bullets when they make the answer easier to scan."
92:   ].join("\n");
93: }

```

buildMessages (cloudflare/portfolio-ai-worker.js, lines 95-105)

This function converts an incoming chat request into model messages that include system instructions, prior conversation, user question, and portfolio context.

The quality of an AI answer depends heavily on the message structure. This function shapes the prompt before it reaches OpenAI or Workers AI.

It calls or references buildInstructions, cleanConversation, push, and buildUserPrompt. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 104: return messages;.

Important local values include messages at line 96. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
95: function buildMessages(body = {}) {
96:   const messages = [{ role: "system", content: buildInstructions() }];
97:   for (const item of cleanConversation(body.conversation)) {
98:     messages.push({
99:       role: item.role === "assistant" ? "assistant" : "user",
100:      content: item.content
101:    });
102:   }
103:   messages.push({ role: "user", content: buildUserPrompt(body) });
104:   return messages;
105: }
```

buildUserPrompt (cloudflare/portfolio-ai-worker.js, lines 107-119)

buildUserPrompt creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 107-119 and is part of the cloudflare worker/deployment area. used for public ai backend and cloudflare deployment notes.

This function is needed so cloudflare/portfolio-ai-worker.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references clamp, stringify, cleanConversation, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 108: return [.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
107: function buildUserPrompt(body = {}) {
108:   return [
109:     `Visitor question: ${clamp(body.question, 1200)}`,
110:     `Question intent: ${clamp(body.intent || "portfolio_specific", 80)}`,
111:     `Web/public lookup requested by browser: ${body.allowWebSearch === true ? "yes" : "no"}`,
112:     "",
113:     "Recent conversation JSON:",
114:     clamp(JSON.stringify(cleanConversation(body.conversation), null, 2), 6000),
115:     "",
116:     "Portfolio context JSON:",
117:     clamp(JSON.stringify(body.context || {}, null, 2), 20000)
118:   ].join("\n");
119: }
```

callOpenAi (cloudflare/portfolio-ai-worker.js, lines 121-145)

This function calls the OpenAI API from the Cloudflare Worker when an OpenAI key is configured, then extracts the answer from the provider response.

It keeps the API key server-side and gives the public website a stronger AI backend without exposing secrets in browser JavaScript.

It calls or references buildInstructions, buildUserPrompt, fetch, and stringify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include model at line 122; payload at line 123; response at line 138. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
121: async function callOpenAi(body, env) {
122:   const model = env.OPENAI_MODEL || "gpt-4.1-mini";
123:   const payload = {
124:     model,
125:     input: [
```

```

126:     {
127:       role: "developer",
128:       content: [{ type: "input_text", text: buildInstructions() }]
129:     },
130:     {
131:       role: "user",
132:       content: [{ type: "input_text", text: buildUserPrompt(body) }]
133:     }
134:   ],
135:   max_output_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS || 1000)
136: };
137:
138: const response = await fetch("https://api.openai.com/v1/responses", {
139:   method: "POST",
140:   headers: {
141:     "Authorization": `Bearer ${env.OPENAI_API_KEY}`,
142:     "Content-Type": "application/json"
143:   },
144:   body: JSON.stringify(payload)
145: });

```

callWorkersAi (cloudflare/portfolio-ai-worker.js, lines 161-180)

This function calls Cloudflare Workers AI as a provider fallback when OpenAI is not configured.

It gives the website AI a deployable backend path even before or without an OpenAI Worker secret.

It calls or references run, buildMessages, and extractWorkersAiText. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 163: return {; line 175: return {.

Important local values include model at line 170; data at line 171. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

161: async function callWorkersAi(body, env) {
162:   if (!env.AI || typeof env.AI.run !== "function") {
163:     return {
164:       ok: false,
165:       status: 503,
166:       error: "Cloudflare Workers AI binding is not available for this Worker."
167:     };
168:   }
169:
170:   const model = env.WORKERS_AI_MODEL || "@cf/meta/llama-3.2-3b-instruct";
171:   const data = await env.AI.run(model, {
172:     messages: buildMessages(body),
173:     max_tokens: Number(env.OPENAI_MAX_OUTPUT_TOKENS || 1000)
174:   });
175:   return {
176:     ok: true,
177:     answer: extractWorkersAiText(data),
178:     model
179:   };
180: }

```

fallbackWorkerAnswer (cloudflare/portfolio-ai-worker.js, lines 182-220)

This function returns a simple local Worker answer when model calls are unavailable.

It prevents the public AI endpoint from returning a useless failure for basic questions or temporary provider outages.

It calls or references trim, toLowerCase, isArray, slice, map, filter, join, and b. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 190: return `Hi, what can I do for you? I can help with \${owner}'s projects, files, resume links, or related electronics concepts.`; line 193: return []; line 204: return []; line 213: return [.

Important local values include question at line 183; clean at line 184; context at line 185; projects at line 186; owner at line 187. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```

182: function fallbackWorkerAnswer(body = {}) {
183:   const question = String(body.question || "").trim();
184:   const clean = question.toLowerCase();
185:   const context = body.context || {};
186:   const projects = Array.isArray(context.projects) ? context.projects : [];
187:   const owner = context.owner || context.profile?.displayName || "Maurice Otieno";
188:
189:   if (/^(hi|hello|hey|yo)\b/.test(clean)) {
190:     return `Hi, what can I do for you? I can help with ${owner}'s projects, files, resume links, or

```

```

related electronics concepts.`;
191:   }
192:   if (/\\bembedded systems?\\b/.test(clean)) {
193:     return [
194:       "Embedded systems is a field of engineering where software is built into dedicated hardware to
control a device, measure signals, communicate with peripherals, or respond to real-time events.",
195:       "",
196:       "Typical embedded work combines microcontrollers or processors, firmware, timing, interrupts,
communication buses, sensors, actuators, and hardware validation.",
197:       "",
198:       projects.length
199:     ] ? `In this portfolio, I would connect that explanation back to saved projects such as
${projects.slice(0, 3).map((project) => project.title || project.name).filter(Boolean).join(", ")}` when those
projects are relevant.`
200:     : "When project context is available, I connect the explanation to the saved project evidence."
201:     ].join("\\n");
202:   }
203:   if (projects.length &&
\\b(project|portfolio|maurice|file|github|resume|tool|design|test)\\b/.test(clean)) {
204:     return [
205:       "I can use the portfolio context for ${owner}.`,
206:       "",
207:       "Relevant saved project areas include:",
208:       ...projects.slice(0, 6).map((project) => `- ${project.title || project.name || "Untitled
project"}`),
209:       "",
210:       "The full AI model is not reachable from this Worker right now, so this is a fallback answer from
the available context."
211:     ].join("\\n");
212:   }
213:   return [
214:     "I can answer that in a general way, but the full AI model is not reachable from this Worker right
now.",
215:     "",
216:     "A useful answer should define the concept, explain the main parts, give a practical example, and
connect it to a project artifact when portfolio context is relevant.",
217:     "",
218:     `Question received: ${question}`
219:   ].join("\\n");
220: }

```

Representative opening snippet

```

1: const defaultAllowedOrigins = [
2:   "https://mauriceotieno.com",
3:   "https://www.mauriceotieno.com",
4:   "https://otienomaurice.github.io",
5:   "http://localhost:8080",
6:   "http://localhost:8081",
7:   "http://127.0.0.1:8080",
8:   "http://127.0.0.1:8081"
9: ];
10:
11: function json(data, status = 200, headers = {}) {
12:   return new Response(JSON.stringify(data, null, 2), {
13:     status,
14:     headers: {
15:       "Content-Type": "application/json; charset=utf-8",
16:       "Cache-Control": "no-store",
17:       ...headers
18:     }
19:   });
20: }
21:
22: function allowedOrigins(env) {
23:   return String(env.ALLOWED_ORIGINS || "")
24:     .split(",")
25:     .map((origin) => origin.trim())
26:     .filter(Boolean)
27:     .concat(defaultAllowedOrigins);
28: }
29:
30: function corsHeaders(request, env) {
31:   const origin = request.headers.get("Origin") || "";
32:   const allowOrigin = allowedOrigins(env).includes(origin) ? origin : defaultAllowedOrigins[0];
33:   return {
34:     "Access-Control-Allow-Origin": allowOrigin,
35:     "Access-Control-Allow-Methods": "POST, OPTIONS",
36:     "Access-Control-Allow-Headers": "Content-Type",
37:     "Access-Control-Max-Age": "86400",
38:     "Vary": "Origin"
39:   };
40: }
41:
42: function clamp(value, maxLength) {
43:   return String(value || "").slice(0, maxLength);
44: }
45:
46: function cleanConversation(conversation = []) {
47:   if (!Array.isArray(conversation)) return [];

```

```

48:   return conversation.slice(-8).map((item) => ({
49:     role: item?.role === "assistant" ? "assistant" : "user",
50:     content: clamp(item?.content || item?.text || "", 1400)
51:   })).filter((item) => item.content);
52: }
53:
54: function extractOpenAiText(data = {}) {
55:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
56:   const parts = [];
57:   for (const item of data.output || []) {
58:     for (const content of item.content || []) {
59:       if (typeof content.text === "string") parts.push(content.text);
60:       if (typeof content.output_text === "string") parts.push(content.output_text);
61:     }
62:   }
63:   return parts.join("\n").trim();
64: }
65:
66: function extractWorkersAiText(data = {}) {
67:   if (typeof data.response === "string" && data.response.trim()) return data.response.trim();
68:   if (typeof data.result?.response === "string" && data.result.response.trim()) return
data.result.response.trim();
69:   if (typeof data.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
70:   if (Array.isArray(data.output)) {
71:     return data.output.map((item) => {
72:       if (typeof item === "string") return item;

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: cloudflare/wrangler.toml

Item	Explanation
File	cloudflare/wrangler.toml
Category	Configuration or structured data
Folder role	Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment notes.
Size	573 bytes
Extension	.toml
MIME type	unknown
Text lines	18

Why this file exists

- Wrangler configuration used to deploy the Cloudflare Worker.

How it is used

Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment notes.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Cloudflare or AI at line 1	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: name = "omb-portfolio-ai"
Cloudflare or AI at line 2	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: main = "portfolio-ai-worker.js"
Cloudflare or AI at line 7	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: { pattern = "mauriceotieno.com/api/portfolio-ai", zone_name = "mauriceotieno.com" },
Cloudflare or AI at line 8	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: { pattern = "www.mauriceotieno.com/api/portfolio-ai", zone_name = "mauriceotieno.com" }
Cloudflare or AI at line 12	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: OPENAI_MODEL = "gpt-4.1-mini"
Cloudflare or AI at line 13	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: OPENAI_MAX_OUTPUT_TOKENS = "1000"
Cloudflare or AI at line 14	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: WORKERS_AI_MODEL = "@cf/meta/llama-3.2-3b-instruct"
Network request at line 15	Frontend-backend communication or public web/API calls. Evidence: ALLOWED_ORIGINS = "https://mauriceotieno.com,https://www.mauriceotieno.com,https://otienomaurice.github.io"

Representative opening snippet

```
1: name = "omb-portfolio-ai"
2: main = "portfolio-ai-worker.js"
3: compatibility_date = "2026-07-01"
4: workers_dev = true
5:
6: routes = [
7:   { pattern = "mauriceotieno.com/api/portfolio-ai", zone_name = "mauriceotieno.com" },
8:   { pattern = "www.mauriceotieno.com/api/portfolio-ai", zone_name = "mauriceotieno.com" }
9: ]
10:
11: [vars]
12: OPENAI_MODEL = "gpt-4.1-mini"
13: OPENAI_MAX_OUTPUT_TOKENS = "1000"
14: WORKERS_AI_MODEL = "@cf/meta/llama-3.2-3b-instruct"
15: ALLOWED_ORIGINS =
```

```
"https://mauriceotieno.com,https://www.mauriceotieno.com,https://otienomaurice.github.io"  
16:  
17: [ai]  
18: binding = "AI"
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: build/installer.nsh

Item	Explanation
File	build/installer.nsh
Category	NSIS installer script
Folder role	Packaging and installer customization. Used while creating the Windows app installer.
Size	29,416 bytes
Extension	.nsh
MIME type	unknown
Text lines	570

Why this file exists

- NSIS installer customization script for update behavior, shortcut placement, old install detection, and setup details.

How it is used

Packaging and installer customization. Used while creating the Windows app installer.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Installer at line 6	Windows setup, update, shortcut, or installation behavior. Evidence: <code>!ifndef BUILD_UNINSTALLER</code>
Installer at line 8	Windows setup, update, shortcut, or installation behavior. Evidence: <code>Var OMB_ShortcutCheckbox</code>
Installer at line 10	Windows setup, update, shortcut, or installation behavior. Evidence: <code>Var OMB_CreateDesktopShortcutState</code>
Installer at line 24	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${NSD_CreateLabel} 0 0 100% 12u "Tools and shortcuts"</code>
Git command at line 27	Repository state, publishing, branch checks, or release automation. Evidence: <code>\${NSD_CreateLabel} 0 18u 100% 36u "The desktop app includes its own runtime. Users do not need to install Node.js or npm.\$\r\$\nGit for Windows with Git Credential Manager is only needed when publishing to GitHub."</code>
Installer at line 30	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${NSD_CreateCheckbox} 0 62u 100% 12u "Create a desktop shortcut"</code>
Installer at line 31	Windows setup, update, shortcut, or installation behavior. Evidence: <code>Pop \$OMB_ShortcutCheckbox</code>
Installer at line 32	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${If} \$OMB_CreateDesktopShortcutState == ""</code>
Installer at line 33	Windows setup, update, shortcut, or installation behavior. Evidence: <code>StrCpy \$OMB_CreateDesktopShortcutState \${BST_CHECKED}</code>
Installer at line 35	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${If} \$OMB_CreateDesktopShortcutState == \${BST_CHECKED}</code>
Installer at line 36	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${NSD_Check} \$OMB_ShortcutCheckbox</code>
Git command at line 39	Repository state, publishing, branch checks, or release automation. Evidence: <code>\${NSD_CreateCheckbox} 0 82u 100% 24u "Check publishing tools and install Git for Windows if Git/Git Credential Manager is missing"</code>
Git command at line 48	Repository state, publishing, branch checks, or release automation. Evidence: <code>\${NSD_CreateLabel} 0 114u 100% 38u "Existing shared tools are not uninstalled silently. If a compatible tool is already available, setup skips it. If Git is missing or unusable, setup will try to install Git for Windows"</code>
Installer at line 55	Windows setup, update, shortcut, or installation behavior. Evidence: <code>\${NSD_GetState} \$OMB_ShortcutCheckbox \$OMB_CreateDesktopShortcutState</code>

Item	Explanation
Git command at line 66	Repository state, publishing, branch checks, or release automation. Evidence: IfFileExists "\$PROGRAMFILES64\Git\cmd\git.exe" 0 +2
Git command at line 68	Repository state, publishing, branch checks, or release automation. Evidence: IfFileExists "\$PROGRAMFILES\Git\cmd\git.exe" 0 +2
Git command at line 70	Repository state, publishing, branch checks, or release automation. Evidence: IfFileExists "\$PROGRAMFILES32\Git\cmd\git.exe" 0 +2
Git command at line 72	Repository state, publishing, branch checks, or release automation. Evidence: IfFileExists "\$LOCALAPPDATA\Programs\Git\cmd\git.exe" 0 +2
Git command at line 76	Repository state, publishing, branch checks, or release automation. Evidence: nsExec::ExecToStack 'cmd /c git --version'
Security or auth at line 86	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Function OMBCheckGitCredentialManagerAvailable
Git command at line 87	Repository state, publishing, branch checks, or release automation. Evidence: nsExec::ExecToStack 'cmd /c git credential-manager --version'
Security or auth at line 100	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: Call OMBCheckGitCredentialManagerAvailable
Git command at line 105	Repository state, publishing, branch checks, or release automation. Evidence: DetailPrint "Git for Windows and Git Credential Manager were found. Skipping publishing tool installation."
Git command at line 114	Repository state, publishing, branch checks, or release automation. Evidence: MessageBox MB_OK MB_ICONINFORMATION "OMB Portfolio Builder was installed, but Git for Windows was not found and Windows Package Manager is unavailable. Open Publishing target > Install Git inside the app, or install Git
Git command at line 118	Repository state, publishing, branch checks, or release automation. Evidence: DetailPrint "Installing or repairing Git for Windows. This can take several minutes."
Git command at line 119	Repository state, publishing, branch checks, or release automation. Evidence: ExecWait 'winget install --id Git.Git -e --source winget --accept-source-agreements --accept-package-agreements' \$0
Git command at line 121	Repository state, publishing, branch checks, or release automation. Evidence: MessageBox MB_OK MB_ICONINFORMATION "OMB Portfolio Builder was installed, but Git for Windows setup did not complete. Open Publishing target > Install Git inside the app when you are ready to publish."
Git command at line 123	Repository state, publishing, branch checks, or release automation. Evidence: DetailPrint "Git for Windows setup completed."
Installer at line 149	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONINFORMATION "OMB Portfolio Builder \$OMB_ExistingInstallVersion is already installed.\$r\$n\$r\$nThis installer is also version \${VERSION}, so setup will stop without changing the app."
Installer at line 157	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONINFORMATION "OMB Portfolio Builder \$OMB_ExistingInstallVersion is already installed.\$r\$n\$r\$nThis installer is older (\${VERSION}), so setup will stop without changing the app."
Installer at line 161	Windows setup, update, shortcut, or installation behavior. Evidence: DetailPrint "OMB Portfolio Builder \$OMB_ExistingInstallVersion is newer than installer \${VERSION}. Setup stopped."
Installer at line 168	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder workspace:\$r\$n\$r\$n\$PROFILE\OMB\builder\desktop-builder\$r\$n\$r\$nRemove, uninstall, or rename that copy before installing this releas
Installer at line 172	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder workspace:\$r\$n\$r\$n\$DOCUMENTS\OMB\desktop-builder\$r\$n\$r\$nRemove, uninstall, or rename that copy before installing this release. Thi
Installer at line 176	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder workspace:\$r\$n\$r\$n\$PROFILE\OneDrive\Documents\OMB\desktop-builder\$r\$n\$r\$nRemove, uninstall, or rename that copy before installing
Installer at line 180	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder workspace:\$r\$n\$r\$n\$PROFILE\Documents\OMB\desktop-builder\$r\$n\$r\$nRemove, uninstall, or rename that copy before installing this rele
Installer at line 184	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing OMB Portfolio Builder development copy:\$r\$n\$r\$n\$DOCUMENTS\omb-portfolio-builder\$r\$n\$r\$nRemove, uninstall, or rename that copy before installing this release."

Item	Explanation
Installer at line 188	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found an existing OMB Portfolio Builder development copy: %s\n%s\n%PROFILE%\OneDrive\Documents\omb-portfolio-builder%s\n%s\nRemove, uninstall, or rename that copy before installi
File read at line 208	Reads local builder state, project files, website assets, or configuration. Evidence: FileWrite \$0 " try { \$\$packageText = Get-Content -LiteralPath (Join-Path \$\$Directory 'package.json') -Raw -ErrorAction Stop } catch { return \$\$false } %s\n"
File read at line 215	Reads local builder state, project files, website assets, or configuration. Evidence: FileWrite \$0 " try { \$\$packageText = Get-Content -LiteralPath (Join-Path \$\$Directory 'package.json') -Raw -ErrorAction Stop } catch { return \$\$false } %s\n"
Installer at line 220	Windows setup, update, shortcut, or installation behavior. Evidence: FileWrite \$0 " \$\$managedRoot = Join-Path \$Env:LOCALAPPDATA 'OMB Portfolio Builder%s\n"
Git command at line 227	Repository state, publishing, branch checks, or release automation. Evidence: FileWrite \$0 " if (\$\$name -in @('node_modules','git','dist','.vs',\$\$Recycle.Bin','System Volume Information','WinSxS')) { return \$\$true } %s\n"
Installer at line 251	Windows setup, update, shortcut, or installation behavior. Evidence: FileWrite \$0 "\$\$registryKeys = @(HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall*, HKCU:\Software\Microsoft\Windows\CurrentVersion\Uninstall*) %s\n"
Installer at line 261	Windows setup, update, shortcut, or installation behavior. Evidence: FileWrite \$0 "foreach (\$\$common in @(\$\$env:ProgramFiles, \$\$programFilesX86, (Join-Path \$Env:LOCALAPPDATA 'Programs'), \$Env:USERPROFILE, (Join-Path \$Env:PUBLC 'Desktop')) { if (\$\$common -and -not \$\$roots.Contains(\$\$c
Installer at line 282	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup found another OMB Portfolio Builder copy on this machine. %s\n%s\nR3%s\n%s\nTo prevent duplicate installations and stale builder features, remove, uninstall, or rename the exis
Installer at line 285	Windows setup, update, shortcut, or installation behavior. Evidence: MessageBox MB_OK MB_ICONSTOP "Setup could not complete the duplicate-installation scan. To avoid duplicate builder copies, setup will stop. Run the installer again as Administrator, or remove old OMB Portfolio Builder co
Installer at line 298	Windows setup, update, shortcut, or installation behavior. Evidence: Function OMBDisableBuiltInOldUninstallerForInPlaceUpdate
Installer at line 299	Windows setup, update, shortcut, or installation behavior. Evidence: DetailPrint "Preparing in-place update. The old uninstaller handoff will be skipped."
Installer at line 302	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" "UninstallString"
Installer at line 303	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" "QuietUninstallString"
Installer at line 304	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" "UninstallString"
Installer at line 305	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" "QuietUninstallString"
Installer at line 308	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" "UninstallString"
Installer at line 309	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" "QuietUninstallString"
Installer at line 310	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" "UninstallString"
Installer at line 311	Windows setup, update, shortcut, or installation behavior. Evidence: DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" "QuietUninstallString"
Installer at line 315	Windows setup, update, shortcut, or installation behavior. Evidence: Function OMBUninstallExistingInstallIfPresent
Installer at line 326	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R6 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" DisplayVersion
Installer at line 327	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" QuietUninstallString
Installer at line 329	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" UninstallString
Installer at line 332	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R7 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" InstallLocation
Installer at line 337	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R6 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\{UNINSTALL_APP_KEY}" DisplayVersion

Item	Explanation
Installer at line 338	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" QuietUninstallString
Installer at line 340	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" UninstallString
Installer at line 343	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R7 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" InstallLocation
Installer at line 350	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R6 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" DisplayVersion
Installer at line 351	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" QuietUninstallString
Installer at line 353	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" UninstallString
Installer at line 356	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R7 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" InstallLocation
Installer at line 360	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R6 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" DisplayVersion
Installer at line 361	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" QuietUninstallString
Installer at line 363	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" UninstallString
Installer at line 366	Windows setup, update, shortcut, or installation behavior. Evidence: ReadRegStr \$R7 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\\${UNINSTALL_APP_KEY}" InstallLocation
Installer at line 373	Windows setup, update, shortcut, or installation behavior. Evidence: IfFileExists "\$LOCALAPPDATA\Programs\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
Installer at line 374	Windows setup, update, shortcut, or installation behavior. Evidence: StrCpy \$R7 "\$LOCALAPPDATA\Programs\OMB Portfolio Builder"
Installer at line 375	Windows setup, update, shortcut, or installation behavior. Evidence: StrCpy \$R5 "\$LOCALAPPDATA\Programs\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
Installer at line 378	Windows setup, update, shortcut, or installation behavior. Evidence: IfFileExists "\$PROGRAMFILES64\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
Installer at line 380	Windows setup, update, shortcut, or installation behavior. Evidence: StrCpy \$R5 "\$PROGRAMFILES64\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
Installer at line 383	Windows setup, update, shortcut, or installation behavior. Evidence: IfFileExists "\$PROGRAMFILES\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
Installer at line 385	Windows setup, update, shortcut, or installation behavior. Evidence: StrCpy \$R5 "\$PROGRAMFILES\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
Installer at line 440	Windows setup, update, shortcut, or installation behavior. Evidence: DetailPrint "Migrating legacy OMB application install to the AppData per-user install location."

Function-by-function detail

OMBToolsPageCreate (build/installer.nsh, lines 17-52)

OMBToolsPageCreate named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 17-52 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
17: Function OMBToolsPageCreate
```

```

18: nsDialogs::Create 1018
19: Pop $OMB_ToolsDialog
20: ${If} $OMB_ToolsDialog == error
21:     Abort
22: ${EndIf}
23:
24: ${NSD_CreateLabel} 0 0 100% 12u "Tools and shortcuts"
25: Pop $0
26:
27: ${NSD_CreateLabel} 0 18u 100% 36u "The desktop app includes its own runtime. Users do not need to
install Node.js or npm.$\r$\nGit for Windows with Git Credential Manager is only needed when publishing to
GitHub."
28: Pop $0
29:
30: ${NSD_CreateCheckbox} 0 62u 100% 12u "Create a desktop shortcut"
31: Pop $OMB_ShortcutCheckbox
32: ${If} $OMB_CreateDesktopShortcutState == ""
33:     StrCpy $OMB_CreateDesktopShortcutState ${BST_CHECKED}
34: ${EndIf}
35: ${If} $OMB_CreateDesktopShortcutState == ${BST_CHECKED}
36:     ${NSD_Check} $OMB_ShortcutCheckbox
37: ${EndIf}
38:
39: ${NSD_CreateCheckbox} 0 82u 100% 24u "Check publishing tools and install Git for Windows if Git/Git
Credential Manager is missing"
40: Pop $OMB_PublishingToolsCheckbox
41: ${If} $OMB_InstallPublishingToolsState == ""
42:     StrCpy $OMB_InstallPublishingToolsState ${BST_CHECKED}
43: ${EndIf}
44: ${If} $OMB_InstallPublishingToolsState == ${BST_CHECKED}
45:     ${NSD_Check} $OMB_PublishingToolsCheckbox
46: ${EndIf}
47:
48: ${NSD_CreateLabel} 0 114u 100% 38u "Existing shared tools are not uninstalled silently. If a compatible
tool is already available, setup skips it. If Git is missing or unusable, setup will try to install Git for
Windows side-by-side using Windows Package Manager."
49: Pop $0
50:
51: nsDialogs::Show
52: FunctionEnd

```

OMBToolsPageLeave (build/installer.nsh, lines 54-62)

OMBToolsPageLeave named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 54-62 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

54: Function OMBToolsPageLeave
55:     ${NSD_GetState} $OMB_ShortcutCheckbox $OMB_CreateDesktopShortcutState
56:     ${NSD_GetState} $OMB_PublishingToolsCheckbox $OMB_InstallPublishingToolsState
57:     ${If} $OMB_IsUpdateInstall == "1"
58:     ${AndIf} $INSTDIR != $OMB_ExistingInstallLocation
59:         MessageBox MB_OK|MB_ICONSTOP "OMB Portfolio Builder is already installed on this
machine.$\r$\n$\r$\nUpdates must use the current installed
location:$\r$\n$OMB_ExistingInstallLocation$\r$\n$\r$\nSetup will not create another installed copy in a
different directory."
60:         Abort
61:     ${EndIf}
62: FunctionEnd

```

OMBCheckGitAvailable (build/installer.nsh, lines 64-84)

OMBCheckGitAvailable part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 64-84 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
64: Function OMBCheckGitAvailable
65:   StrCpy $R9 "0"
66:   IfFileExists "$PROGRAMFILES64\Git\cmd\git.exe" 0 +2
67:     StrCpy $R9 "1"
68:   IfFileExists "$PROGRAMFILES\Git\cmd\git.exe" 0 +2
69:     StrCpy $R9 "1"
70:   IfFileExists "$PROGRAMFILES32\Git\cmd\git.exe" 0 +2
71:     StrCpy $R9 "1"
72:   IfFileExists "$LOCALAPPDATA\Programs\Git\cmd\git.exe" 0 +2
73:     StrCpy $R9 "1"
74:
75:   ${If} $R9 == "0"
76:     nsExec::ExecToStack 'cmd /c git --version'
77:     Pop $0
78:     Pop $1
79:     ${If} $0 == 0
80:       StrCpy $R9 "1"
81:     ${EndIf}
82:   ${EndIf}
83:   Push $R9
84: FunctionEnd
```

OMBCheckGitCredentialManagerAvailable (build/installer.nsh, lines 86-95)

OMBCheckGitCredentialManagerAvailable part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 86-95 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
86: Function OMBCheckGitCredentialManagerAvailable
87:   nsExec::ExecToStack 'cmd /c git credential-manager --version'
88:   Pop $0
89:   Pop $1
90:   ${If} $0 == 0
91:     Push "1"
92:   ${Else}
93:     Push "0"
94:   ${EndIf}
95: FunctionEnd
```

OMBInstallGitIfNeeded (build/installer.nsh, lines 97-125)

OMBInstallGitIfNeeded part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 97-125 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
97: Function OMBInstallGitIfNeeded
98:   Call OMBCheckGitAvailable
99:   Pop $R9
100:   Call OMBCheckGitCredentialManagerAvailable
101:   Pop $R8
102:
103:   ${If} $R9 == "1"
104:     ${AndIf} $R8 == "1"
105:       DetailPrint "Git for Windows and Git Credential Manager were found. Skipping publishing tool
installation."
106:       Return
107:     ${EndIf}
108:
```

```

109:   DetailPrint "Publishing tools are missing or incomplete. Checking Windows Package Manager."
110:   nsExec::ExecToStack 'where.exe winget'
111:   Pop $0
112:   Pop $1
113:   ${If} $0 != 0
114:       MessageBox MB_OK|MB_ICONINFORMATION "OMB Portfolio Builder was installed, but Git for Windows was not
found and Windows Package Manager is unavailable. Open Publishing target > Install Git inside the app, or
install Git for Windows with Git Credential Manager from https://git-scm.com/download/win."
115:       Return
116:   ${EndIf}
117:
118:   DetailPrint "Installing or repairing Git for Windows. This can take several minutes."
119:   ExecWait 'winget install --id Git.Git -e --source winget --accept-source-agreements --accept-package-
agreements' $0
120:   ${If} $0 != 0
121:       MessageBox MB_OK|MB_ICONINFORMATION "OMB Portfolio Builder was installed, but Git for Windows setup
did not complete. Open Publishing target > Install Git inside the app when you are ready to publish."
122:   ${Else}
123:       DetailPrint "Git for Windows setup completed."
124:   ${EndIf}
125: FunctionEnd

```

OMBReadExistingAppVersion (build/installer.nsh, lines 127-139)

OMBReadExistingAppVersion named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 127-139 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Write. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

127: Function OMBReadExistingAppVersion
128:   StrCpy $OMB_ExistingInstallVersion ""
129:   IfFileExists "$R7\OMB Portfolio Builder.exe" omb_read_existing_app_version 0
130:   Return
131:
132:   omb_read_existing_app_version:
133:   nsExec::ExecToStack 'powershell.exe -NoProfile -ExecutionPolicy Bypass -Command "$$p = '$R7\OMB
Portfolio Builder.exe'; try { $$v = (Get-Item -LiteralPath $$p).VersionInfo.FileVersion; if (-not $$v) { $$v =
(Get-Item -LiteralPath $$p).VersionInfo.ProductVersion }; [Console]::Write($$v) } catch { exit 1 }'"
134:   Pop $R4
135:   Pop $R3
136:   ${If} $R4 == 0
137:       StrCpy $OMB_ExistingInstallVersion $R3
138:   ${EndIf}
139: FunctionEnd

```

OMBStopIfExistingInstallsCurrent (build/installer.nsh, lines 141-164)

OMBStopIfExistingInstallsCurrent named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 141-164 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references older. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

141: Function OMBStopIfExistingInstallIsCurrent
142:   ${If} $OMB_ExistingInstallVersion == ""
143:       Return
144:   ${EndIf}
145:
146:   ${VersionCompare} "$OMB_ExistingInstallVersion" "${VERSION}" $R4
147:   ${If} $R4 == 0
148:       IfSilent omb_existing_install_current_silent
149:       MessageBox MB_OK|MB_ICONINFORMATION "OMB Portfolio Builder $OMB_ExistingInstallVersion is already
installed.$\r\n$\r\nThis installer is also version ${VERSION}, so setup will stop without changing the app."
150:       Quit

```

```

151:
152:     omb_existing_install_current_silent:
153:     DetailPrint "OMB Portfolio Builder $OMB_ExistingInstallVersion is already installed. Setup stopped
because the installed version is current."
154:     Quit
155:     ${ElseIf} $R4 == 1
156:     IfSilent omb_existing_install_newer_silent
157:     MessageBox MB_OK|MB_ICONINFORMATION "OMB Portfolio Builder $OMB_ExistingInstallVersion is already
installed.$\r$\n$\r$\nThis installer is older ({VERSION}), so setup will stop without changing the app."
158:     Quit
159:
160:     omb_existing_install_newer_silent:
161:     DetailPrint "OMB Portfolio Builder $OMB_ExistingInstallVersion is newer than installer ${VERSION}.
Setup stopped."
162:     Quit
163:     ${EndIf}
164: FunctionEnd

```

OMBCheckKnownBuilderWorkspace (build/installer.nsh, lines 166-190)

OMBCheckKnownBuilderWorkspace named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 166-190 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

166: Function OMBCheckKnownBuilderWorkspace
167:   IfFileExists "$PROFILE\OMB\builder\desktop-builder\package.json" 0 +3
168:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder
workspace:$\r$\n$\r$\n$PROFILE\OMB\builder\desktop-builder$\r$\n$\r$\nRemove, uninstall, or rename that copy
before installing this release. This prevents an older working builder from staying on the machine beside the
installed app."
169:   Quit
170:
171:   IfFileExists "$DOCUMENTS\OMB\desktop-builder\package.json" 0 +3
172:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder
workspace:$\r$\n$\r$\n$DOCUMENTS\OMB\desktop-builder$\r$\n$\r$\nRemove, uninstall, or rename that copy before
installing this release. This prevents an older working builder from staying on the machine beside the installed
app."
173:   Quit
174:
175:   IfFileExists "$PROFILE\OneDrive\Documents\OMB\desktop-builder\package.json" 0 +3
176:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder
workspace:$\r$\n$\r$\n$PROFILE\OneDrive\Documents\OMB\desktop-builder$\r$\n$\r$\nRemove, uninstall, or rename
that copy before installing this release. This prevents an older working builder from staying on the machine
beside the installed app."
177:   Quit
178:
179:   IfFileExists "$PROFILE\Documents\OMB\desktop-builder\package.json" 0 +3
180:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing local OMB Portfolio Builder
workspace:$\r$\n$\r$\n$PROFILE\Documents\OMB\desktop-builder$\r$\n$\r$\nRemove, uninstall, or rename that copy
before installing this release. This prevents an older working builder from staying on the machine beside the
installed app."
181:   Quit
182:
183:   IfFileExists "$DOCUMENTS\omb-portfolio-builder\package.json" 0 +3
184:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing OMB Portfolio Builder development
copy:$\r$\n$\r$\n$DOCUMENTS\omb-portfolio-builder$\r$\n$\r$\nRemove, uninstall, or rename that copy before
installing this release."
185:   Quit
186:
187:   IfFileExists "$PROFILE\OneDrive\Documents\omb-portfolio-builder\package.json" 0 +3
188:   MessageBox MB_OK|MB_ICONSTOP "Setup found an existing OMB Portfolio Builder development
copy:$\r$\n$\r$\n$PROFILE\OneDrive\Documents\omb-portfolio-builder$\r$\n$\r$\nRemove, uninstall, or rename that
copy before installing this release."
189:   Quit
190: FunctionEnd

```

OMBWriteDuplicateScanScript (build/installer.nsh, lines 192-272)

OMBWriteDuplicateScanScript named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 192-272 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Hit, IsNullOrWhiteSpace, Contains, Add, DesktopBuilder, not, LiteralPath, BuilderWorkspace, SkipDirectory, Windows, and 9 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 198: `FileWrite $0 " if ([string]::IsNullOrWhiteSpace($$Candidate)) { return }$r$`n";` line 200: `FileWrite $0 " if ($$resolved -match '\\\\OMB-Portfolio-Builder-Setup-[^\\\\]+\\.exe$') { return }$r$`n";` line 205: `FileWrite $0 " if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'package.json'))) { return $$false }$r$`n";` line 206: `FileWrite $0 " if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'main.cjs'))) { return $$false }r`n".` There are 8 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
192: Function OMBWriteDuplicateScanScript
193:   InitPluginsDir
194:   FileOpen $0 "$PLUGINS_DIR\omb-duplicate-scan.ps1" w
195:   FileWrite $0 "$$ErrorActionPreference = 'SilentlyContinue'$r$`n"
196:   FileWrite $0 "$$hits = New-Object System.Collections.Generic.List[string]$r$`n"
197:   FileWrite $0 "function Add-Hit([string]$$Kind, [string]$$Candidate) {$r$`n"
198:   FileWrite $0 "   if ([string]::IsNullOrWhiteSpace($$Candidate)) { return }$r$`n"
199:   FileWrite $0 "   try { $$resolved = (Resolve-Path -LiteralPath $$Candidate -ErrorAction Stop).Path }
catch { $$resolved = $$Candidate }$r$`n"
200:   FileWrite $0 "   if ($$resolved -match '\\\\OMB-Portfolio-Builder-Setup-[^\\\\]+\\.exe$') { return
}$r$`n"
201:   FileWrite $0 "   $$line = $$Kind + ' - ' + $$resolved$r$`n"
202:   FileWrite $0 "   if (-not $$hits.Contains($$line)) { [void]$$hits.Add($$line) }$r$`n"
203:   FileWrite $0 "}$r$`n"
204:   FileWrite $0 "function Test-DesktopBuilder([string]$$Directory) {$r$`n"
205:   FileWrite $0 "   if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'package.json'))) { return
$$false }$r$`n"
206:   FileWrite $0 "   if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'main.cjs'))) { return $$false
}$r$`n"
207:   FileWrite $0 "   if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'builder-site'))) { return
$$false }$r$`n"
208:   FileWrite $0 "   try { $$packageText = Get-Content -LiteralPath (Join-Path $$Directory 'package.json')
-Raw -ErrorAction Stop } catch { return $$false }$r$`n"
209:   FileWrite $0 "   return $$packageText -match 'OMB Portfolio Builder|omb-portfolio-
builder|portfoliobuilder'$r$`n"
210:   FileWrite $0 "}$r$`n"
211:   FileWrite $0 "function Test-BuilderWorkspace([string]$$Directory) {$r$`n"
212:   FileWrite $0 "   if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'package.json'))) { return
$$false }$r$`n"
213:   FileWrite $0 "   if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'main.cjs'))) { return $$false
}$r$`n"
214:   FileWrite $0 "   if (-not (Test-Path -LiteralPath (Join-Path $$Directory 'server.mjs'))) { return
$$false }$r$`n"
215:   FileWrite $0 "   try { $$packageText = Get-Content -LiteralPath (Join-Path $$Directory 'package.json')
-Raw -ErrorAction Stop } catch { return $$false }$r$`n"
216:   FileWrite $0 "   return $$packageText -match 'OMB Portfolio Builder|omb-portfolio-
builder|portfoliobuilder'$r$`n"
217:   FileWrite $0 "}$r$`n"
218:   FileWrite $0 "function Should-SkipDirectory([string]$$Directory) {$r$`n"
219:   FileWrite $0 "   $$name = Split-Path -Leaf $$Directory$r$`n"
220:   FileWrite $0 "   $$managedRoot = Join-Path $env:LOCALAPPDATA 'OMB Portfolio Builder'$r$`n"
221:   FileWrite $0 "   $$legacyRoot = Join-Path $env:USERPROFILE 'OMB'$r$`n"
222:   FileWrite $0 "   $$managedBuilder = Join-Path $$managedRoot 'builder'$r$`n"
223:   FileWrite $0 "   $$managedPortfolio = Join-Path $$managedRoot 'portfolio'$r$`n"
224:   FileWrite $0 "   $$legacyBuilder = Join-Path $$legacyRoot 'builder'$r$`n"
225:   FileWrite $0 "   $$legacyPortfolio = Join-Path $$legacyRoot 'portfolio'$r$`n"
226:   FileWrite $0 "   if ($$Directory -ieq $$managedBuilder -or $$Directory -ieq $$managedPortfolio -or
$$Directory -ieq $$legacyBuilder -or $$Directory -ieq $$legacyPortfolio) { return $$true }$r$`n"
227:   FileWrite $0 "   if ($$name -in @('node_modules', '.git', 'dist', '.vs', '$$Recycle.Bin', 'System Volume
Information', 'WinSxS')) { return $$true }$r$`n"
228:   FileWrite $0 "   if ($$Directory -match '\\\\Windows(\\\\\\$)$') { return $$true }$r$`n"
229:   FileWrite $0 "   return $$false$r$`n"
230:   FileWrite $0 "}$r$`n"
231:   FileWrite $0 "function Scan-Tree([string]$$Root) {$r$`n"
232:   FileWrite $0 "   if (-not (Test-Path -LiteralPath $$Root)) { return }$r$`n"
233:   FileWrite $0 "   $$stack = New-Object System.Collections.Generic.Stack[string>$r$`n"
234:   FileWrite $0 "   $$stack.Push($$Root)$r$`n"
235:   FileWrite $0 "   while ($$stack.Count -gt 0) {$r$`n"
236:   FileWrite $0 "       $$dir = $$stack.Pop()$r$`n"
237:   FileWrite $0 "       if (Should-SkipDirectory $$dir) { continue }$r$`n"
238:   FileWrite $0 "       if ((Split-Path -Leaf $$dir) -ieq 'desktop-builder' -and (Test-DesktopBuilder $$dir))
{ Add-Hit 'Local desktop-builder workspace' $$dir }$r$`n"
239:   FileWrite $0 "       if ((Split-Path -Leaf $$dir) -ieq 'builder' -and (Test-BuilderWorkspace $$dir)) {
Add-Hit 'Local builder workspace' $$dir }$r$`n"
240:   FileWrite $0 "       try { $$children = Get-ChildItem -LiteralPath $$dir -Force -ErrorAction Stop } catch
{ continue }$r$`n"
241:   FileWrite $0 "       foreach ($$child in $$children) {$r$`n"
242:   FileWrite $0 "           if ($$child.PSIsContainer) {$r$`n"
243:   FileWrite $0 "               if (-not (Should-SkipDirectory $$child.FullName)) {
$$stack.Push($$child.FullName) }$r$`n"
244:   FileWrite $0 "           } else { }$r$`n"
245:   FileWrite $0 "           if ($$child.Name -ieq 'OMB Portfolio Builder.exe') { Add-Hit 'Installed app
executable' $$child.FullName }$r$`n"
246:   FileWrite $0 "           if ($$child.Name -like 'OMB-Portfolio-Builder-Portable-*.exe') { Add-Hit
```



```
'Portable app executable' $$child.FullName }$r$n"
247: FileWrite $0 " }$r$n"
248: FileWrite $0 " }$r$n"
249: FileWrite $0 " }$r$n"
250: FileWrite $0 "$r$n"
251: FileWrite $0 "$$registryKeys = @('HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall\*', 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Uninstall\*')$r$n"
252: FileWrite $0 "foreach ($$key in $$registryKeys) {$r$n"
253: FileWrite $0 " Get-ItemProperty $$key -ErrorAction SilentlyContinue | Where-Object { $_.DisplayName
-like 'OMB Portfolio Builder*' } | ForEach-Object {$r$n"
254: FileWrite $0 " if ($$.InstallLocation) { Add-Hit 'Registered Windows installation'
$$.InstallLocation }$r$n"
255: FileWrite $0 " elseif ($$.DisplayIcon) { Add-Hit 'Registered Windows installation' $$.DisplayIcon
}$r$n"
256: FileWrite $0 " }$r$n"
257: FileWrite $0 "$r$n"
258: FileWrite $0 "$$roots = New-Object System.Collections.Generic.List[string]$r$n"
259: FileWrite $0 "Get-CimInstance Win32_LogicalDisk -Filter 'DriveType=3' -ErrorAction SilentlyContinue |
ForEach-Object { [void]$$roots.Add(($$.DeviceID + '\')) }$r$n"
260: FileWrite $0 "$$programFilesX86 = [Environment]::GetEnvironmentVariable('ProgramFiles(x86)')$r$n"
261: FileWrite $0 "foreach ($$common in @($$env:ProgramFiles, $$programFilesX86, (Join-Path
$env:LOCALAPPDATA 'Programs'), $env:USERPROFILE, (Join-Path $env:PUBLIC 'Desktop')) { if ($$common -and -not
$$roots.Contains($$common)) { [void]$$roots.Add($$common) } }$r$n"
262: FileWrite $0 "foreach ($$root in $$roots) { Scan-Tree $$root }$r$n"
263: FileWrite $0 "$$unique = $$hits | Sort-Object -Unique$`r`n"
264: FileWrite $0 "if ($$unique.Count -gt 0) {$r$n"
265: FileWrite $0 " $$preview = $$unique | Select-Object -First 10$`r`n"
266: FileWrite $0 " if ($$unique.Count -gt 10) { $$preview += ('... and ' + ($$unique.Count - 10) + '
more') }$r$n"
267: FileWrite $0 " Write-Output ($$preview -join [Environment]::NewLine)$r$n"
268: FileWrite $0 " exit 42$`r`n"
269: FileWrite $0 "$r$n"
270: FileWrite $0 "exit 0$`r`n"
271: FileClose $0
272: FunctionEnd
```

OMBSaveForDuplicateBuilderCopies (build/installer.nsh, lines 274-288)

This function runs the duplicate-copy scanner generated by the installer script and blocks setup when another builder copy is found.

Duplicate installations were causing shortcuts and updates to point to the wrong app. This function is part of the system-wide guard against that.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
274: Function OMBSaveForDuplicateBuilderCopies
275: DetailPrint "Scanning this machine for existing OMB Portfolio Builder copies."
276: Call OMBWriteDuplicateScanScript
277: nsExec:ExecToStack 'powershell.exe -NoProfile -ExecutionPolicy Bypass -File "$PLUGINSIDIR\omb-
duplicate-scan.ps1"
278: Pop $R4
279: Pop $R3
280:
281: ${If} $R4 == 42
282:   MessageBox MB_OK|MB_ICONSTOP "Setup found another OMB Portfolio Builder copy on this
machine.$r$n$`r`n$R3$r$n$`r`nTo prevent duplicate installations and stale builder features, remove,
uninstall, or rename the existing copy first, then run this installer again."
283:   Quit
284: ${ElseIf} $R4 != 0
285:   MessageBox MB_OK|MB_ICONSTOP "Setup could not complete the duplicate-installation scan. To avoid
duplicate builder copies, setup will stop. Run the installer again as Administrator, or remove old OMB Portfolio
Builder copies manually before installing."
286:   Quit
287: ${EndIf}
288: FunctionEnd
```

OMBStopBuilderProcessesForUpdate (build/installer.nsh, lines 290-296)

This function stops running builder processes before setup tries to replace files.

Windows cannot update files that are still locked by a running application. This makes the update path much smoother.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
290: Function OMBStopBuilderProcessesForUpdate
291:   DetailPrint "Stopping running OMB Portfolio Builder processes before update."
292:   nsExec::ExecToStack 'cmd /c taskkill /IM "OMB Portfolio Builder.exe" /F /T'
293:   Pop $R4
294:   Pop $R3
295:   Sleep 1500
296: FunctionEnd
```

OMBDisableBuiltInOldUninstallerForInPlaceUpdate (build/installer.nsh, lines 298-313)

OMBDisableBuiltInOldUninstallerForInPlaceUpdate named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 298-313 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
298: Function OMBDisableBuiltInOldUninstallerForInPlaceUpdate
299:   DetailPrint "Preparing in-place update. The old uninstaller handoff will be skipped."
300:
301:   SetRegView 64
302:   DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"UninstallString"
303:   DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"QuietUninstallString"
304:   DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"UninstallString"
305:   DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"QuietUninstallString"
306:
307:   SetRegView 32
308:   DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"UninstallString"
309:   DeleteRegValue HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"QuietUninstallString"
310:   DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"UninstallString"
311:   DeleteRegValue HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
"QuietUninstallString"
312:   SetRegView lastused
313: FunctionEnd
```

OMBUninstallExistingInstallIfPresent (build/installer.nsh, lines 315-497)

This installer function checks for existing installations and decides whether setup is a fresh install, blocked duplicate install, or in-place update.

It protects the machine from multiple conflicting builder installs and allows update installs to reuse the correct location.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
315: Function OMBUninstallExistingInstallIfPresent
316:   StrCpy $OMB_ExistingInstallLocation ""
317:   StrCpy $OMB_ExistingInstallVersion ""
318:   StrCpy $OMB_IsUpdateInstall "0"
319:   StrCpy $OMB_ManualApplicationInstall "0"
320:   StrCpy $R7 ""
321:   StrCpy $R6 ""
322:   StrCpy $R5 ""
323:
324:   SetRegView 64
325:   ReadRegStr $R7 HKCU "Software\${APP_GUID}" InstallLocation
326:   ReadRegStr $R6 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
DisplayVersion
327:   ReadRegStr $R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
QuietUninstallString
```

```

328:     ${If} $R5 == ""
329:     ReadRegStr $R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
UninstallString
330:     ${EndIf}
331:     ${If} $R7 == ""
332:     ReadRegStr $R7 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
InstallLocation
333:     ${EndIf}
334:
335:     ${If} $R7 == ""
336:     ReadRegStr $R7 HKLM "Software\${APP_GUID}" InstallLocation
337:     ReadRegStr $R6 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
DisplayVersion
338:     ReadRegStr $R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
QuietUninstallString
339:     ${If} $R5 == ""
340:     ReadRegStr $R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
UninstallString
341:     ${EndIf}
342:     ${If} $R7 == ""
343:     ReadRegStr $R7 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
InstallLocation
344:     ${EndIf}
345:     ${EndIf}
346:
347:     ${If} $R7 == ""
348:     SetRegView 32
349:     ReadRegStr $R7 HKCU "Software\${APP_GUID}" InstallLocation
350:     ReadRegStr $R6 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
DisplayVersion
351:     ReadRegStr $R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
QuietUninstallString
352:     ${If} $R5 == ""
353:     ReadRegStr $R5 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
UninstallString
354:     ${EndIf}
355:     ${If} $R7 == ""
356:     ReadRegStr $R7 HKCU "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
InstallLocation
357:     ${EndIf}
358:     ${If} $R7 == ""
359:     ReadRegStr $R7 HKLM "Software\${APP_GUID}" InstallLocation
360:     ReadRegStr $R6 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
DisplayVersion
361:     ReadRegStr $R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
QuietUninstallString
362:     ${If} $R5 == ""
363:     ReadRegStr $R5 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
UninstallString
364:     ${EndIf}
365:     ${If} $R7 == ""
366:     ReadRegStr $R7 HKLM "Software\Microsoft\Windows\CurrentVersion\Uninstall\${UNINSTALL_APP_KEY}"
InstallLocation
367:     ${EndIf}
368:     ${EndIf}
369:     SetRegView lastused
370:     ${EndIf}
371:
372:     ${If} $R7 == ""
373:     IfFileExists "$LOCALAPPDATA\Programs\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
374:     StrCpy $R7 "$LOCALAPPDATA\Programs\OMB Portfolio Builder"
375:     StrCpy $R5 "$LOCALAPPDATA\Programs\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
376:     ${EndIf}
377:     ${If} $R7 == ""
378:     IfFileExists "$PROGRAMFILES64\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
379:     StrCpy $R7 "$PROGRAMFILES64\OMB Portfolio Builder"
380:     StrCpy $R5 "$PROGRAMFILES64\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
381:     ${EndIf}
382:     ${If} $R7 == ""
383:     IfFileExists "$PROGRAMFILES\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" 0 +3
384:     StrCpy $R7 "$PROGRAMFILES\OMB Portfolio Builder"
385:     StrCpy $R5 "$PROGRAMFILES\OMB Portfolio Builder\Uninstall OMB Portfolio Builder.exe" /S'
386:     ${EndIf}
387:     ${If} $R7 == ""
388:     IfFileExists "$PROFILE\OMB\application\OMB Portfolio Builder\OMB Portfolio Builder.exe" 0 +4
389:     StrCpy $R7 "$PROFILE\OMB\application\OMB Portfolio Builder"
390:     StrCpy $R6 "a legacy app-folder copy"
391:     StrCpy $OMB_ManualApplicationInstall "1"
392:     ${EndIf}
393:     ${If} $R7 == ""
394:     IfFileExists "$PROFILE\OMB\application\OMB Portfolio Builder.exe" 0 +4
395:     StrCpy $R7 "$PROFILE\OMB\application"
396:     StrCpy $R6 "a standalone app-folder copy"
397:     StrCpy $OMB_ManualApplicationInstall "1"
398:     ${EndIf}
399:     ${If} $R7 == ""
400:     IfFileExists "$PROFILE\OneDrive\Documents\OMB\application\OMB Portfolio Builder\OMB Portfolio
Builder.exe" 0 +4
401:     StrCpy $R7 "$PROFILE\OneDrive\Documents\OMB\application\OMB Portfolio Builder"
402:     StrCpy $R6 "a legacy app-folder copy"
403:     StrCpy $OMB_ManualApplicationInstall "1"
404:     ${EndIf}

```

```

405:  ${If} $R7 == ""
406:    IfFileExists "$PROFILE\OneDrive\Documents\OMB\application\OMB Portfolio Builder.exe" 0 +4
407:    StrCpy $R7 "$PROFILE\OneDrive\Documents\OMB\application"
408:    StrCpy $R6 "a standalone app-folder copy"
409:    StrCpy $OMB_ManualApplicationInstall "1"
410:  ${EndIf}
411:  ${If} $R7 == ""
412:    IfFileExists "$PROFILE\OneDrive\Desktop\OMB\application\OMB Portfolio Builder.exe" 0 +4
413:    StrCpy $R7 "$PROFILE\OneDrive\Desktop\OMB\application"
414:    StrCpy $R6 "a standalone app-folder copy"
415:    StrCpy $OMB_ManualApplicationInstall "1"
416:  ${EndIf}
417:  ${If} $R7 == ""
418:    IfFileExists "$DOCUMENTS\OMB\application\OMB Portfolio Builder.exe" 0 +4
419:    StrCpy $R7 "$DOCUMENTS\OMB\application"
420:    StrCpy $R6 "a standalone app-folder copy"
421:    StrCpy $OMB_ManualApplicationInstall "1"
422:  ${EndIf}
423:
424:  ${If} $OMB_ManualApplicationInstall == "1"
425:    StrCpy $R5 ""
426:  ${EndIf}
427:
428:  ${If} $R7 != ""
429:    ${If} $OMB_ManualApplicationInstall == "1"
430:      Call OMBReadExistingAppVersion
431:    ${Else}
432:      StrCpy $OMB_ExistingInstallVersion $R6
433:    ${EndIf}
434:    ${If} $R7 == "$PROFILE\OMB\application\OMB Portfolio Builder"
... excerpt truncated after 120 lines. Open the source file for the rest of this function.

```

OMBUninstallPageCreate (build/installer.nsh, lines 546-560)

OMBUninstallPageCreate named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 546-560 and is part of the packaging and installer customization. used while creating the windows app installer.

This function is needed so build/installer.nsh can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

546: Function OMBUninstallPageCreate
547:   nsDialogs::Create 1018
548:   Pop $OMB_UninstallDialog
549:   ${If} $OMB_UninstallDialog == error
550:     Abort
551:   ${EndIf}
552:
553:   ${NSD_CreateLabel} 0 0 100% 12u "Uninstall OMB Portfolio Builder"
554:   Pop $0
555:
556:   ${NSD_CreateLabel} 0 20u 100% 90u "Uninstall will remove:$\r$\n$\r$\n- OMB Portfolio Builder
application files$\r$\n- OMB Portfolio Builder shortcuts$\r$\n- Local app data created by this application when
Windows requests app-data cleanup$\r$\n$\r$\nUninstall will not silently remove shared tools such as Git for
Windows, Node.js, pnpm, browsers, or your Git repositories."
557:   Pop $0
558:
559:   nsDialogs::Show
560: FunctionEnd

```

Representative opening snippet

```

1: !include LogicLib.nsh
2: !include nsDialogs.nsh
3: !include WinMessages.nsh
4: !include WordFunc.nsh
5:
6: !ifndef BUILD_UNINSTALLER
7: Var OMB_ToolsDialog
8: Var OMB_ShortcutCheckbox
9: Var OMB_PublishingToolsCheckbox
10: Var OMB_CreateDesktopShortcutState
11: Var OMB_InstallPublishingToolsState
12: Var OMB_ExistingInstallLocation
13: Var OMB_ExistingInstallVersion
14: Var OMB_IsUpdateInstall
15: Var OMB_ManualApplicationInstall
16:

```

```

17: Function OMBToolsPageCreate
18:   nsDialogs::Create 1018
19:   Pop $OMB_ToolsDialog
20:   ${If} $OMB_ToolsDialog == error
21:     Abort
22:   ${EndIf}
23:
24:   ${NSD_CreateLabel} 0 0 100% 12u "Tools and shortcuts"
25:   Pop $0
26:
27:   ${NSD_CreateLabel} 0 18u 100% 36u "The desktop app includes its own runtime. Users do not need to
install Node.js or pnpm.$\r$\nGit for Windows with Git Credential Manager is only needed when publishing to
GitHub."
28:   Pop $0
29:
30:   ${NSD_CreateCheckbox} 0 62u 100% 12u "Create a desktop shortcut"
31:   Pop $OMB_ShortcutCheckbox
32:   ${If} $OMB_CreateDesktopShortcutState == ""
33:     StrCpy $OMB_CreateDesktopShortcutState ${BST_CHECKED}
34:   ${EndIf}
35:   ${If} $OMB_CreateDesktopShortcutState == ${BST_CHECKED}
36:     ${NSD_Check} $OMB_ShortcutCheckbox
37:   ${EndIf}
38:
39:   ${NSD_CreateCheckbox} 0 82u 100% 24u "Check publishing tools and install Git for Windows if Git/Git
Credential Manager is missing"
40:   Pop $OMB_PublishingToolsCheckbox
41:   ${If} $OMB_InstallPublishingToolsState == ""
42:     StrCpy $OMB_InstallPublishingToolsState ${BST_CHECKED}
43:   ${EndIf}
44:   ${If} $OMB_InstallPublishingToolsState == ${BST_CHECKED}
45:     ${NSD_Check} $OMB_PublishingToolsCheckbox
46:   ${EndIf}
47:
48:   ${NSD_CreateLabel} 0 114u 100% 38u "Existing shared tools are not uninstalled silently. If a compatible
tool is already available, setup skips it. If Git is missing or unusable, setup will try to install Git for
Windows side-by-side using Windows Package Manager."
49:   Pop $0
50:
51:   nsDialogs::Show
52: FunctionEnd
53:
54: Function OMBToolsPageLeave
55:   ${NSD_GetState} $OMB_ShortcutCheckbox $OMB_CreateDesktopShortcutState
56:   ${NSD_GetState} $OMB_PublishingToolsCheckbox $OMB_InstallPublishingToolsState
57:   ${If} $OMB_IsUpdateInstall == "1"
58:     ${AndIf} $INSTDIR != $OMB_ExistingInstallLocation
59:     MessageBox MB_OK|MB_ICONSTOP "OMB Portfolio Builder is already installed on this
machine.$\r$\n$\r$\nUpdates must use the current installed
location:$\r$\n$OMB_ExistingInstallLocation$\r$\n$\r$\nSetup will not create another installed copy in a
different directory."
60:     Abort
61:   ${EndIf}
62: FunctionEnd
63:
64: Function OMBCheckGitAvailable
65:   StrCpy $R9 "0"
66:   IfFileExists "$PROGRAMFILES64\Git\cmd\git.exe" 0 +2
67:   StrCpy $R9 "1"
68:   IfFileExists "$PROGRAMFILES\Git\cmd\git.exe" 0 +2
69:   StrCpy $R9 "1"
70:   IfFileExists "$PROGRAMFILES32\Git\cmd\git.exe" 0 +2
71:   StrCpy $R9 "1"
72:   IfFileExists "$LOCALAPPDATA\Programs\Git\cmd\git.exe" 0 +2

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: docs/build_complete_guide.py

Item	Explanation
File	docs/build_complete_guide.py
Category	Python tooling
Folder role	Documentation and generated reference area. Used to explain the app and source structure.
Size	222,291 bytes
Extension	.py
MIME type	text/x-python
Text lines	3,273

Why this file exists

- Tracked repository file used by the builder, website, installer, documentation, or release process.

How it is used

Documentation and generated reference area. Used to explain the app and source structure.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Imports, requires, and external dependencies

Item	Explanation
Line 3	import html
Line 4	import json
Line 5	import math
Line 6	import mimetypes
Line 7	import re
Line 8	import shutil
Line 9	import subprocess
Line 10	import textwrap

API endpoints mentioned or called

Item	Explanation
/api/...	Line 103. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/code/save	Line 459. Saves source code into the local compile workspace.
/api/code/compile	Line 652. Compiles or runs a source file and returns terminal output.
/api/code/compile	Line 664. Compiles or runs a source file and returns terminal output.
/api/portfolio-ai	Line 676. Handles AI assistant questions.
/api/portfolio-ai	Line 707. Handles AI assistant questions.

Important implementation signals

Item	Explanation
Rich text at line 14	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: from PIL import Image, ImageDraw, ImageFont
Rich text at line 20	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: from docx.shared import Inches, Pt, RGBColor
Rich text at line 23	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: from reportlab.lib import colors
Installer at line 54	Windows setup, update, shortcut, or installation behavior. Evidence: ".github/workflows/build-windows-builder.yml": "GitHub Actions workflow that builds the Windows installer and portable application, checks the stable download link, enforces release version bumps, uploads artifacts, and
Git command at line 56	Repository state, publishing, branch checks, or release automation. Evidence: ".gitignore": "Git ignore rules for build outputs, temporary files, and local-only data that should not be committed.",
Cloudflare or AI at line 59	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "BRANCHING.md": "Human-readable branch model explaining development, main, feature branches, and release flow.",
Git command at line 60	Repository state, publishing, branch checks, or release automation. Evidence: "Backgrounds/.gitkeep": "Keeps the Backgrounds folder in Git even when no background image files are present.",
Cloudflare or AI at line 61	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "README.md": "Main README for the standalone builder application: install, updates, workspaces, text editing, publishing, build commands, branch model, and uninstall.",
Cloudflare or AI at line 62	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "_headers": "Cloudflare Pages style HTTP headers for security and caching behavior on the public website.",
Installer at line 71	Windows setup, update, shortcut, or installation behavior. Evidence: "assets/omb-app-icon.ico": "Windows ICO application icon used by Electron Builder and shortcuts.",
Installer at line 73	Windows setup, update, shortcut, or installation behavior. Evidence: "build/installer.nsh": "NSIS installer customization script for update behavior, shortcut placement, old install detection, and setup details.",
Cloudflare or AI at line 75	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/README.md": "Cloudflare setup notes for deploying the AI Worker endpoint and wiring the public site to the backend.",
Cloudflare or AI at line 76	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/portfolio-ai-worker.js": "Cloudflare Worker backend for the public portfolio AI assistant and fallback intelligence path.",
Cloudflare or AI at line 77	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/wrangler.toml": "Wrangler configuration used to deploy the Cloudflare Worker.",
Git command at line 78	Repository state, publishing, branch checks, or release automation. Evidence: "docs/.gitkeep": "Keeps the docs folder in Git even before generated documents are added.",
Installer at line 81	Windows setup, update, shortcut, or installation behavior. Evidence: "installer-notes.txt": "Installer license/notes text shown during Windows setup.",
Compiler or simulator at line 83	Part of the Compile Code workspace or language/tool detection. Evidence: "package.json": "Node/Electron package manifest with scripts, dependencies, version, installer settings, and extra resources.",
Security or auth at line 91	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "server.mjs": "Local backend server used by the builder for drafts, parsing, publishing, compiler execution, authentication checks, and file operations.",
Git command at line 100	Repository state, publishing, branch checks, or release automation. Evidence: ("High-Level System Map", "The system has two personalities: a private builder app and a public website. The builder is where content is created and verified. The website is what recruiters see after publishing.", "Owner
Security or auth at line 102	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("Private Builder Versus Public Website", "The builder includes editing controls, local draft storage, authentication checks, compiler tools, and publishing actions. The public website removes those controls and only dis
Git command at line 103	Repository state, publishing, branch checks, or release automation. Evidence: ("Local Backend Role", "The frontend cannot safely do everything by itself. The local backend handles file writes, repository operations, compiler execution, publishing checks, and generated site output.", "template-prev
Git command at line 105	Repository state, publishing, branch checks, or release automation. Evidence: ("GitHub Pages Publishing", "The target repository receives static files. GitHub Pages then serves those files over HTTPS. The builder does not run on the public site.", "Local publish mirror -> git commit -> git push ->
Cloudflare or AI at line 106	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("Cloudflare Role", "Cloudflare can sit in front of the domain and can also host Worker endpoints such as the portfolio AI backend. DNS points the custom domain to the website host.", "Visitor DNS lookup -> Cloudflare DN
Cloudflare or AI at line 107	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("AI Backend Role", "The website AI should answer from portfolio context when the question is about Maurice's work and use general knowledge when the question is general. The Worker endpoint keeps API keys off the public

Item	Explanation
Installer at line 109	Windows setup, update, shortcut, or installation behavior. Evidence: ("Installer And Update Role", "The installer places the app in the per-user AppData Programs folder. Updates should replace that install instead of creating duplicate copies.", "GitHub Release -> installer download -> up
Cloudflare or AI at line 110	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("Branch Model", "Development collects work. Main is the release branch. Feature branches should merge into development first. Main should receive only development.", "feature branch -> development -> main -> release wor
Security or auth at line 111	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("Security Boundary", "Editing is local and authenticated. Public visitors should not get editing tools. Publishing requires GitHub write access to the target repository.", "Visitor can read public site\nOwner can edit b
Cloudflare or AI at line 119	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Cloudflare Role": "cloudflare-ai-flow",
Cloudflare or AI at line 120	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "AI Backend Role": "cloudflare-ai-flow",
Installer at line 122	Windows setup, update, shortcut, or installation behavior. Evidence: "Installer And Update Role": "release-pipeline",
Cloudflare or AI at line 123	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Branch Model": "release-pipeline",
Git command at line 128	Repository state, publishing, branch checks, or release automation. Evidence: ("git status", "Shows what branch you are on and whether files are changed, staged, or clean.", "Run it before edits, before commits, and before merges."),
Git command at line 129	Repository state, publishing, branch checks, or release automation. Evidence: ("git switch development", "Moves your working copy to the development branch.", "Use development as the integration branch before anything goes to main."),
Git command at line 130	Repository state, publishing, branch checks, or release automation. Evidence: ("git switch -c codex/example-change", "Creates a new feature branch from the branch you are currently on.", "Use this before changing the builder so development stays stable."),
Git command at line 131	Repository state, publishing, branch checks, or release automation. Evidence: ("git add <file>", "Stages a changed file for the next commit.", "Only stage files that belong to the current change."),
Git command at line 132	Repository state, publishing, branch checks, or release automation. Evidence: ("git commit -m 'message'", "Records staged changes as a named checkpoint.", "Write messages that explain why the change exists."),
Git command at line 133	Repository state, publishing, branch checks, or release automation. Evidence: ("git push origin <branch>", "Uploads your branch to GitHub.", "This lets GitHub run workflows and enables pull requests."),
Git command at line 134	Repository state, publishing, branch checks, or release automation. Evidence: ("git pull --ff-only", "Updates the current branch only if Git can fast-forward safely.", "Avoids surprise merge commits."),
Git command at line 135	Repository state, publishing, branch checks, or release automation. Evidence: ("git fetch --tags --force", "Downloads branch and tag metadata from GitHub.", "Used before checking whether a release tag already exists."),
Compiler or simulator at line 136	Part of the Compile Code workspace or language/tool detection. Evidence: ("pnpm install --frozen-lockfile", "Installs Node dependencies exactly as pinned in pnpm-lock.yaml.", "Makes release builds repeatable."),
Installer at line 139	Windows setup, update, shortcut, or installation behavior. Evidence: ("pnpm run installer", "Builds the NSIS Windows installer.", "Use this when testing installation and update behavior."),
Cloudflare or AI at line 142	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("wrangler deploy", "Deploys the Cloudflare Worker backend.", "Use it when the AI endpoint code changes."),
Compiler or simulator at line 143	Part of the Compile Code workspace or language/tool detection. Evidence: ("gcc file.c -o app.exe", "Compiles C source into a Windows executable.", "The builder uses similar logic when C tools are available."),
Compiler or simulator at line 145	Part of the Compile Code workspace or language/tool detection. Evidence: ("javac Main.java", "Compiles Java source into .class bytecode.", "Java has a compile step before running."),
Compiler or simulator at line 146	Part of the Compile Code workspace or language/tool detection. Evidence: ("java Main", "Runs compiled Java bytecode.", "The builder shows this output in the terminal panel."),
Compiler or simulator at line 147	Part of the Compile Code workspace or language/tool detection. Evidence: ("python script.py", "Runs Python source directly.", "Python is interpreted by default."),
Compiler or simulator at line 148	Part of the Compile Code workspace or language/tool detection. Evidence: ("node script.js", "Runs JavaScript through Node.js.", "Useful for builder scripts and JavaScript examples."),
Compiler or simulator at line 149	Part of the Compile Code workspace or language/tool detection. Evidence: ("iverilog -g2012 design.sv -o sim.vvp", "Compiles Verilog/SystemVerilog into an Icarus simulation file.", "Prepares HDL for simulation."),
Compiler or simulator at line 150	Part of the Compile Code workspace or language/tool detection. Evidence: ("vvp sim.vvp", "Runs Icarus Verilog simulation output.", "This is how HDL terminal output appears."),
Git command at line 151	Repository state, publishing, branch checks, or release automation. Evidence: ("git push --dry-run origin main", "Tests whether GitHub would accept a push without changing the remote.", "Used during authorization checks."),

Item	Explanation
Git command at line 152	Repository state, publishing, branch checks, or release automation. Evidence: ("git remote -v", "Shows which GitHub repository the local folder talks to.", "Important for publishing targets."),
Git command at line 153	Repository state, publishing, branch checks, or release automation. Evidence: ("git branch --show-current", "Prints the active branch name.", "The builder must know whether it is pushing the right branch."),
Git command at line 154	Repository state, publishing, branch checks, or release automation. Evidence: ("git tag --list builder-v*", "Lists builder release tags.", "Release workflows use this to prevent duplicate releases."),
Installer at line 155	Windows setup, update, shortcut, or installation behavior. Evidence: ("Get-ChildItem", "PowerShell command that lists files and folders.", "Used to inspect workspaces and installer output."),
Process execution at line 157	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: ("Start-Process", "PowerShell command that starts another program.", "The updater uses detached process launching."),
Cloudflare or AI at line 162	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Profile identity and contact details", "Front-page hero text", "Fun facts block", "Portfolio areas", "Project categories", "Project creation flow", "Project overview", "Design sections", "Simulation sections", "Files an
Installer at line 167	Windows setup, update, shortcut, or installation behavior. Evidence: ("The app does not open", "Check whether the installed executable exists under AppData Local Programs. If it does, start it directly. If it fails, reinstall from the latest GitHub Release."),
Installer at line 168	Windows setup, update, shortcut, or installation behavior. Evidence: ("Update does not restart", "The update handoff must close Electron, run the installer, and then launch the new executable. Check the updater log and whether another builder process is still running."),
Security or auth at line 169	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("GitHub authentication succeeds but builder cannot push", "Authentication and repository freshness are different. Pull or load from target first if the remote branch is ahead."),
Local storage at line 171	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: ("Website changes show on desktop but not phone", "Check cache, confirm the published files changed, and verify the mobile renderer is using the same projects.json and CSS."),
Cloudflare or AI at line 172	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("AI endpoint unavailable", "Verify the Worker route, environment secrets, Cloudflare deployment, and browser network errors."),
Local storage at line 173	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: ("Compiler takes too long", "Check whether the first run is installing or detecting tools. Cached runs should be faster after the first successful compile."),
Compiler or simulator at line 174	Part of the Compile Code workspace or language/tool detection. Evidence: ("SystemVerilog output missing", "Check that Icarus Verilog and vvp are available and that the file is a simulation testbench or has a runnable top module."),
Rich text at line 175	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: ("Right-click formatting does not apply", "Confirm the active editor saved its selection before the menu changed font, size, or color."),
Rich text at line 176	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: ("Links are treated as formulas", "URL detection should win before formula detection. Pasted http, https, and www links must become normal links."),
Installer at line 178	Windows setup, update, shortcut, or installation behavior. Evidence: ("Installer reports an old installation", "The installer searches per-user and legacy locations. Remove stale install records only after confirming the active app path."),
Git command at line 186	Repository state, publishing, branch checks, or release automation. Evidence: "What HTML does", "What CSS does", "What JavaScript does", "What JSON does", "What Markdown does", "What YAML does", "What TOML does", "What an executable is", "What an installer is", "What a portable app is", "What a lo
Git command at line 192	Repository state, publishing, branch checks, or release automation. Evidence: ("Electron vs pure web builder", "Electron gives desktop file access and packaging. A pure web builder would be easier to access anywhere, but browser security would block many local file, compiler, and Git operations.")
Installer at line 194	Windows setup, update, shortcut, or installation behavior. Evidence: ("Per-user install vs Program Files", "Per-user installs update without admin prompts. Program Files feels traditional but causes UAC friction and update failures."),
Git command at line 197	Repository state, publishing, branch checks, or release automation. Evidence: ("Git authentication cache vs always prompt", "Caching reduces annoyance. Always prompting is stricter but makes normal publishing painful."),
Cloudflare or AI at line 198	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("Cloudflare Worker AI vs browser-only AI", "Worker AI hides secrets and can access server-side providers. Browser-only AI would expose keys or be limited to local browser models."),
Git command at line 213	Repository state, publishing, branch checks, or release automation. Evidence: ("git", "Git target repo\ncommit + push", 1040, 140, 250, 135, "#FEF3C7"),
Cloudflare or AI at line 216	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("ai", "Cloudflare Worker\nportfolio AI endpoint", 1040, 565, 250, 130, "#CCFBF1"),
Git command at line 222	Repository state, publishing, branch checks, or release automation. Evidence: ("backend", "git", "publishes"),

Item	Explanation
Git command at line 223	Repository state, publishing, branch checks, or release automation. Evidence: ("git", "site", "hosts files"),
Cloudflare or AI at line 225	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: ("site", "ai", "chat API"),
Git command at line 237	Repository state, publishing, branch checks, or release automation. Evidence: ("system", "Filesystem, Git,\ncompilers, updater", 1045, 250, 300, 140, "#DCFCE7"),
Installer at line 279	Windows setup, update, shortcut, or installation behavior. Evidence: ("release", "GitHub Release\ninstaller + portable", 1015, 370, 300, 130, "#EDE9FE"),
Compiler or simulator at line 297	Part of the Compile Code workspace or language/tool detection. Evidence: ("lang", "Language profile\nC, C++, Java,\nPython, JS, HDL", 705, 150, 260, 145, "#FEF3C7"),
Security or auth at line 299	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ("scope", "HDL testbench\n+ signal scope", 1030, 315, 250, 125, "#DCFCE7"),

JSON/YAML-style keys detected

Item	Explanation
.github/workflows/build-windows-builder.yml	Line 54: ".github/workflows/build-windows-builder.yml": "GitHub Actions workflow that builds the Windows installer and portable application, checks the stable download link, enforces releases",
.github/workflows/main-branch-gate.yml	Line 55: ".github/workflows/main-branch-gate.yml": "GitHub Actions guard that rejects direct pushes to main and rejects pull requests into main unless the source branch is development.",
.gitignore	Line 56: ".gitignore": "Git ignore rules for build outputs, temporary files, and local-only data that should not be committed.",
.nojekyll	Line 57: ".nojekyll": "Tells GitHub Pages not to process the site with Jekyll, so folders and static files publish exactly as generated.",
.well-known/security.txt	Line 58: ".well-known/security.txt": "Public security contact file used by browsers, researchers, and scanners to find responsible disclosure information.",
BRANCHING.md	Line 59: "BRANCHING.md": "Human-readable branch model explaining development, main, feature branches, and release flow.",
Backgrounds/.gitkeep	Line 60: "Backgrounds/.gitkeep": "Keeps the Backgrounds folder in Git even when no background image files are present.",
README.md	Line 61: "README.md": "Main README for the standalone builder application: install, updates, workspaces, text editing, publishing, build commands, branch model, and uninstall.",
_headers	Line 62: "_headers": "Cloudflare Pages style HTTP headers for security and caching behavior on the public website.",
assets/apple-touch-icon.png	Line 63: "assets/apple-touch-icon.png": "Apple touch icon shown when the public site is saved on iOS or compatible launch surfaces.",
assets/favicon-16.png	Line 64: "assets/favicon-16.png": "Small browser favicon used by tabs and browser UI.",
assets/favicon-192.png	Line 65: "assets/favicon-192.png": "Large web app icon for Android and installable browser surfaces.",
assets/favicon-32.png	Line 66: "assets/favicon-32.png": "Standard browser favicon size.",
assets/favicon-48.png	Line 67: "assets/favicon-48.png": "Windows/browser icon size used by some desktop surfaces.",
assets/favicon.ico	Line 68: "assets/favicon.ico": "ICO favicon bundle used by browsers that request favicon.ico.",
assets/omb-app-icon-256.png	Line 69: "assets/omb-app-icon-256.png": "Builder application icon source at 256 pixels.",
assets/omb-app-icon-512.png	Line 70: "assets/omb-app-icon-512.png": "Builder application icon source at 512 pixels.",
assets/omb-app-icon.ico	Line 71: "assets/omb-app-icon.ico": "Windows ICO application icon used by Electron Builder and shortcuts.",
assets/omb-mark.png	Line 72: "assets/omb-mark.png": "OMB snake/mark image used for branding in the site and builder.",
build/installer.nsh	Line 73: "build/installer.nsh": "NSIS installer customization script for update behavior, shortcut placement, old install detection, and setup details.",
builder-rich-future-sections.js	Line 74: "builder-rich-future-sections.js": "Builder-side helper code for richer future portfolio sections and section defaults.",
cloudflare/README.md	Line 75: "cloudflare/README.md": "Cloudflare setup notes for deploying the AI Worker endpoint and wiring the public site to the backend.",
cloudflare/portfolio-ai-worker.js	Line 76: "cloudflare/portfolio-ai-worker.js": "Cloudflare Worker backend for the public portfolio AI assistant and fallback intelligence path.",

Item	Explanation
cloudflare/wrangler.toml	Line 77: "cloudflare/wrangler.toml": "Wrangler configuration used to deploy the Cloudflare Worker.",
docs/.gitkeep	Line 78: "docs/.gitkeep": "Keeps the docs folder in Git even before generated documents are added.",
electronics-search.js	Line 79: "electronics-search.js": "Electronics vocabulary and search support used by the website search behavior.",
index.html	Line 80: "index.html": "Public website HTML shell that recruiters and visitors load in the browser.",
installer-notes.txt	Line 81: "installer-notes.txt": "Installer license/notes text shown during Windows setup.",
main.cjs	Line 82: "main.cjs": "Electron main process: creates the desktop window, starts the local backend, handles menus, update handoff, and application lifecycle.",
package.json	Line 83: "package.json": "Node/Electron package manifest with scripts, dependencies, version, installer settings, and extra resources.",
pnpm-lock.yaml	Line 84: "pnpm-lock.yaml": "Pinned dependency lockfile so installs and GitHub Actions use the same dependency versions.",
pnpm-workspace.yaml	Line 85: "pnpm-workspace.yaml": "pnpm workspace declaration for the repository.",
portfolio-README.md	Line 86: "portfolio-README.md": "README for the generated public portfolio website rather than the builder application.",
project-templates.json	Line 87: "project-templates.json": "Appearance template definitions used by projects to control visual feel rather than content.",
projects.json	Line 88: "projects.json": "Public project catalog consumed by the website after parsing builder data.",
robots.txt	Line 89: "robots.txt": "Crawler instruction file for search engines.",
script.js	Line 90: "script.js": "Public website JavaScript for navigation, portfolio rendering, search, AI UI, and interactive project windows.",
server.mjs	Line 91: "server.mjs": "Local backend server used by the builder for drafts, parsing, publishing, compiler execution, authentication checks, and file operations.",
styles.css	Line 92: "styles.css": "Public website CSS for layout, contact area, projects, responsive behavior, AI panel, and mobile/desktop rendering.",
template-preview.css	Line 93: "template-preview.css": "Builder application CSS for dialogs, windows, rich editors, compile code, dark mode, guide windows, and local previews.",
template-preview.html	Line 94: "template-preview.html": "Builder HTML shell loaded inside the Electron app and local preview runtime.",
template-preview.js	Line 95: "template-preview.js": "Builder frontend brain: state handling, rich editors, project builder, previews, parser triggers, publishing UI, compile workspace, and guide windows.",
High-Level System Map	Line 115: "High-Level System Map": "system-overview",
Why Electron Was Chosen	Line 116: "Why Electron Was Chosen": "electron-runtime",
Portfolio Parser Role	Line 117: "Portfolio Parser Role": "builder-to-site-files",
GitHub Pages Publishing	Line 118: "GitHub Pages Publishing": "builder-to-site-files",
Cloudflare Role	Line 119: "Cloudflare Role": "cloudflare-ai-flow",
AI Backend Role	Line 120: "AI Backend Role": "cloudflare-ai-flow",
Compiler Workspace Role	Line 121: "Compiler Workspace Role": "compile-code-flow",
Installer And Update Role	Line 122: "Installer And Update Role": "release-pipeline",
Branch Model	Line 123: "Branch Model": "release-pipeline",
name	Line 206: "name": "system-overview",
title	Line 207: "title": "Private Builder To Public Portfolio",
nodes	Line 208: "nodes": [
edges	Line 218: "edges": [
name	Line 229: "name": "electron-runtime",
title	Line 230: "title": "Electron Runtime Inside The Desktop App",
nodes	Line 231: "nodes": [
edges	Line 239: "edges": [

Item	Explanation
name	Line 249: "name": "builder-to-site-files",
title	Line 250: "title": "How Builder Edits Become Website Files",
nodes	Line 251: "nodes": [
edges	Line 260: "edges": [
name	Line 271: "name": "release-pipeline",
title	Line 272: "title": "Builder Release Pipeline",
nodes	Line 273: "nodes": [
edges	Line 282: "edges": [
name	Line 292: "name": "compile-code-flow",
title	Line 293: "title": "Compile Code Workspace",
nodes	Line 294: "nodes": [
edges	Line 304: "edges": [
name	Line 315: "name": "tool-build-map",
title	Line 316: "title": "What Tool Builds What",
nodes	Line 317: "nodes": [
edges	Line 326: "edges": [
name	Line 337: "name": "file-communication-map",
title	Line 338: "title": "How Key Files Communicate",
nodes	Line 339: "nodes": [
edges	Line 350: "edges": [
name	Line 363: "name": "cloudflare-ai-flow",

Event handlers and user interactions

Item	Explanation
Line 464	"sectionContent.addEventListener('click', async (event) => {\n const button = event.target.closest('[data-compile-run]');\n if (button) await compileActiveFile(project, file);\n})
Line 1951	if ".addEventListener(" in line:

Function-by-function detail

tracked_files (docs/build_complete_guide.py, lines 901-905)

tracked_files named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 901-905 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references run, strip, and splitlines. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 903: return [line.strip() for line in result.stdout.splitlines() if line.strip()].

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

901: def tracked_files() -> list[str]:
902:     result = subprocess.run(["git", "ls-files"], cwd=REPO, text=True, capture_output=True, check=True)
903:     return [line.strip() for line in result.stdout.splitlines() if line.strip()]
904:

```

is_generated_reference_file (docs/build_complete_guide.py, lines 929-932)

`is_generated_reference_file` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 929-932 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `startswith`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 930: return `repo_file` in `GENERATED_REFERENCE_FILES` or `repo_file.startswith(GENERATED_REFERENCE_PREFIXES)`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
929: def is_generated_reference_file(repo_file: str) -> bool:
930:     return repo_file in GENERATED_REFERENCE_FILES or repo_file.startswith(GENERATED_REFERENCE_PREFIXES)
931:
932:
```

primary_tracked_files (docs/build_complete_guide.py, lines 933-936)

`primary_tracked_files` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 933-936 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `is_generated_reference_file`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 934: return `[file for file in files if not is_generated_reference_file(file)]`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
933: def primary_tracked_files(files: list[str]) -> list[str]:
934:     return [file for file in files if not is_generated_reference_file(file)]
935:
936:
```

important_code_files (docs/build_complete_guide.py, lines 960-964)

`important_code_files` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 960-964 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `set`, and `is_generated_reference_file`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 962: return `[file for file in IMPORTANT_CODE_FILES if file in available and not is_generated_reference_file(file)]`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
960: def important_code_files(files: list[str]) -> list[str]:
961:     available = set(files)
962:     return [file for file in IMPORTANT_CODE_FILES if file in available and not
is_generated_reference_file(file)]
963:
964:
```

load_package (docs/build_complete_guide.py, lines 1208-1214)

load_package loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1208-1214 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references loads, and read_text. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1210: return json.loads((REPO / "package.json").read_text(encoding="utf-8")); line 1212: return {}.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1208: def load_package() -> dict:
1209:     try:
1210:         return json.loads((REPO / "package.json").read_text(encoding="utf-8"))
1211:     except Exception:
1212:         return {}
1213:
1214:
```

load_font (docs/build_complete_guide.py, lines 1215-1222)

load_font loads data from disk, network, browser storage, or a service. In this file, it appears around lines 1215-1222 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Path, exists, truetype, str, and load_default. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1219: return ImageFont.truetype(str(font_path), size=size); line 1220: return ImageFont.load_default().

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1215: def load_font(size: int, bold: bool = False) -> ImageFont.ImageFont:
1216:     font_name = "arialbd.ttf" if bold else "arial.ttf"
1217:     font_path = Path(r"C:\Windows\Fonts") / font_name
1218:     if font_path.exists():
1219:         return ImageFont.truetype(str(font_path), size=size)
1220:     return ImageFont.load_default()
1221:
1222:
```

wrap_for_box (docs/build_complete_guide.py, lines 1223-1242)

wrap_for_box named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1223-1242 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references str, splitlines, split, append, and getbbox. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1240: return lines.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1223: def wrap_for_box(label: str, font: ImageFont.ImageFont, max_width: int) -> list[str]:
1224:     lines: list[str] = []
1225:     for part in str(label).splitlines():
1226:         words = part.split()
1227:         if not words:
1228:             lines.append("")
1229:             continue
1230:         current = words[0]
1231:         for word in words[1:]:
1232:             candidate = f"{current} {word}"
```

```

1233:         bbox = font.getbbox(candidate)
1234:         if bbox[2] - bbox[0] <= max_width:
1235:             current = candidate
1236:         else:
1237:             lines.append(current)
1238:             current = word
1239:         lines.append(current)
1240:     return lines
1241:
1242:

```

draw_arrow (docs/build_complete_guide.py, lines 1243-1255)

draw_arrow named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1243-1255 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references line, atan2, int, cos, sin, and polygon. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1243: def draw_arrow(draw: ImageDraw.ImageDraw, start: tuple[int, int], end: tuple[int, int], color: str,
width: int = 4) -> None:
1244:     draw.line([start, end], fill=color, width=width)
1245:     angle = math.atan2(end[1] - start[1], end[0] - start[0])
1246:     length = 18
1247:     spread = 0.55
1248:     points = [
1249:         end,
1250:         (int(end[0] - length * math.cos(angle - spread)), int(end[1] - length * math.sin(angle -
spread))),
1251:         (int(end[0] - length * math.cos(angle + spread)), int(end[1] - length * math.sin(angle +
spread))),
1252:     ]
1253:     draw.polygon(points, fill=color)
1254:
1255:

```

edge_points (docs/build_complete_guide.py, lines 1256-1271)

edge_points named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1256-1271 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references abs. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1269: return start, end.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1256: def edge_points(source: tuple[int, int, int, int], target: tuple[int, int, int, int]) -> tuple[tuple[int,
int], tuple[int, int]]:
1257:     sx1, sy1, sx2, sy2 = source
1258:     tx1, ty1, tx2, ty2 = target
1259:     sc = ((sx1 + sx2) // 2, (sy1 + sy2) // 2)
1260:     tc = ((tx1 + tx2) // 2, (ty1 + ty2) // 2)
1261:     dx = tc[0] - sc[0]
1262:     dy = tc[1] - sc[1]
1263:     if abs(dx) >= abs(dy):
1264:         start = (sx2 if dx >= 0 else sx1, sc[1])
1265:         end = (tx1 if dx >= 0 else tx2, tc[1])
1266:     else:
1267:         start = (sc[0], sy2 if dy >= 0 else sy1)
1268:         end = (tc[0], ty1 if dy >= 0 else ty2)
1269:     return start, end
1270:
1271:

```

make_diagrams (docs/build_complete_guide.py, lines 1272-1321)

make_diagrams named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1272-1321 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references mkdir, load_font, Draw, rounded_rectangle, text, line, edge_points, draw_arrow, getbbox, hypot, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1319: return diagram_paths.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1272: def make_diagrams() -> dict[str, Path]:
1273:     DIAGRAM_DIR.mkdir(parents=True, exist_ok=True)
1274:     title_font = load_font(38, bold=True)
1275:     box_font = load_font(24, bold=True)
1276:     label_font = load_font(18, bold=False)
1277:     diagram_paths: dict[str, Path] = {}
1278:
1279:     for spec in DIAGRAM_SPECS:
1280:         image = Image.new("RGB", (1600, 900), "#F8FBFD")
1281:         draw = ImageDraw.Draw(image)
1282:         draw.rounded_rectangle((24, 24, 1576, 876), radius=26, outline="#B7D7EA", width=3,
fill="#FFFFFF")
1283:         draw.text((60, 48), spec["title"], fill="#0B2545", font=title_font)
1284:         draw.line((60, 105, 1540, 105), fill="#B7D7EA", width=3)
1285:
1286:         boxes: dict[str, tuple[int, int, int, int]] = {}
1287:         for node_id, label, x, y, w, h, fill in spec["nodes"]:
1288:             boxes[node_id] = (x, y, x + w, y + h)
1289:
1290:         for source_id, target_id, *maybe_label in spec["edges"]:
1291:             source = boxes[source_id]
1292:             target = boxes[target_id]
1293:             start, end = edge_points(source, target)
1294:             draw_arrow(draw, start, end, "#2563EB", width=4)
1295:             if maybe_label:
1296:                 label = maybe_label[0]
1297:                 mid = ((start[0] + end[0]) // 2, (start[1] + end[1]) // 2)
1298:                 text_box = label_font.getbbox(label)
1299:                 tw = text_box[2] - text_box[0] + 16
1300:                 th = text_box[3] - text_box[1] + 10
1301:                 if math.hypot(end[0] - start[0], end[1] - start[1]) > max(tw + 28, 95):
1302:                     draw.rounded_rectangle((mid[0] - tw // 2, mid[1] - th // 2, mid[0] + tw // 2, mid[1]
+ th // 2), radius=9, fill="#EFF6FF", outline="#BFD8FE", width=1)
1303:                     draw.text((mid[0] - tw // 2 + 8, mid[1] - th // 2 + 4), label, fill="#1E3A8A",
font=label_font)
1304:
1305:         for node_id, label, x, y, w, h, fill in spec["nodes"]:
1306:             draw.rounded_rectangle((x, y, x + w, y + h), radius=20, fill=fill, outline="#668EA8",
width=3)
1307:             lines = wrap_for_box(label, box_font, w - 28)
1308:             line_height = 30
1309:             total_h = len(lines) * line_height
1310:             start_y = y + (h - total_h) // 2
1311:             for index, line in enumerate(lines):
1312:                 bbox = box_font.getbbox(line)
1313:                 tw = bbox[2] - bbox[0]
1314:                 draw.text((x + (w - tw) // 2, start_y + index * line_height), line, fill="#102033",
font=box_font)
1315:
1316:         path = DIAGRAM_DIR / f"{spec['name']}.png"
1317:         image.save(path)
1318:         diagram_paths[spec["name"]] = path
1319:     return diagram_paths
1320:
1321:
```

add_diagram (docs/build_complete_guide.py, lines 1322-1335)

add_diagram named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1322-1335 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, add_paragraph, add_run, add_picture, str, Inches, Pt, and RGBColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1324: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1322: def add_diagram(doc: Document, path: Path, caption: str) -> None:
1323:     if not path.exists():
1324:         return
1325:     paragraph = doc.add_paragraph()
1326:     paragraph.alignment = WD_ALIGN_PARAGRAPH.CENTER
1327:     run = paragraph.add_run()
1328:     run.add_picture(str(path), width=Inches(6.35))
1329:     cap = doc.add_paragraph(caption)
1330:     cap.alignment = WD_ALIGN_PARAGRAPH.CENTER
1331:     for run in cap.runs:
1332:         run.font.size = Pt(9)
1333:         run.font.color.rgb = RGBColor(85, 85, 85)
1334:
1335:
```

shade (docs/build_complete_guide.py, lines 1336-1342)

shade named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1336-1342 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get_or_add_tcPr, OxmlElement, set, qn, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1336: def shade(cell, fill: str) -> None:
1337:     tc_pr = cell._tc.get_or_add_tcPr()
1338:     shd = OxmlElement("w:shd")
1339:     shd.set(qn("w:fill"), fill)
1340:     tc_pr.append(shd)
1341:
1342:
```

cell_text (docs/build_complete_guide.py, lines 1343-1351)

cell_text named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1343-1351 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_run, str, and Pt. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1343: def cell_text(cell, text: str, bold: bool = False) -> None:
1344:     cell.text = ""
1345:     run = cell.paragraphs[0].add_run(str(text))
1346:     run.bold = bold
1347:     run.font.name = "Calibri"
1348:     run.font.size = Pt(9)
1349:     cell.vertical_alignment = WD_CELL_VERTICAL_ALIGNMENT.CENTER
1350:
1351:
```

add_table (docs/build_complete_guide.py, lines 1352-1368)

add_table named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1352-1368 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Inches, cell_text, shade, and add_row. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1352: def add_table(doc: Document, rows: list[tuple[str, str]]) -> None:
1353:     table = doc.add_table(rows=1, cols=2)
1354:     table.alignment = WD_TABLE_ALIGNMENT.CENTER
1355:     table.autofit = False
1356:     table.columns[0].width = Inches(1.55)
1357:     table.columns[1].width = Inches(4.95)
1358:     head = table.rows[0].cells
1359:     cell_text(head[0], "Item", True)
1360:     cell_text(head[1], "Explanation", True)
1361:     shade(head[0], "E8EEF5")
1362:     shade(head[1], "E8EEF5")
1363:     for label, value in rows:
1364:         cells = table.add_row().cells
1365:         cell_text(cells[0], label, True)
1366:         cell_text(cells[1], value, False)
1367:
1368:
```

add_code (docs/build_complete_guide.py, lines 1369-1376)

add_code named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1369-1376 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_paragraph, add_run, rstrip, set, and qn. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1369: def add_code(doc: Document, text: str) -> None:
1370:     p = doc.add_paragraph()
1371:     p.style = doc.styles["CodeBlock"]
1372:     run = p.add_run((text or "").rstrip())
1373:     run.font.name = "Consolas"
1374:     run._element.rPr.rFonts.set(qn("w: eastAsia"), "Consolas")
1375:
1376:
```

add_note (docs/build_complete_guide.py, lines 1377-1391)

add_note named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1377-1391 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_table, Inches, cell, shade, add_run, RGBColor, Pt, and add_paragraph. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1377: def add_note(doc: Document, title: str, body: str) -> None:
1378:     table = doc.add_table(rows=1, cols=1)
1379:     table.alignment = WD_TABLE_ALIGNMENT.CENTER
1380:     table.autofit = False
1381:     table.columns[0].width = Inches(6.35)
1382:     cell = table.cell(0, 0)
1383:     shade(cell, "F4F8FC")
1384:     title_run = cell.paragraphs[0].add_run(title)
1385:     title_run.bold = True
1386:     title_run.font.color.rgb = RGBColor(31, 77, 120)
1387:     title_run.font.size = Pt(10)
1388:     paragraph = cell.add_paragraph(body)
1389:     paragraph.style = "Body Text"
```

```
1390:
1391:
```

bullets (docs/build_complete_guide.py, lines 1392-1396)

bullets named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1392-1396 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_paragraph. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1392: def bullets(doc: Document, items: list[str]) -> None:
1393:     for item in items:
1394:         doc.add_paragraph(item, style="List Bullet")
1395:
1396:
```

numbers (docs/build_complete_guide.py, lines 1397-1401)

numbers named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1397-1401 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_paragraph. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1397: def numbers(doc: Document, items: list[str]) -> None:
1398:     for item in items:
1399:         doc.add_paragraph(item, style="List Number")
1400:
1401:
```

setup_document (docs/build_complete_guide.py, lines 1402-1451)

setup_document named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1402-1451 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Document, Inches, set, qn, Pt, from_string, add_style, and RGBColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1449: return doc.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1402: def setup_document() -> Document:
1403:     doc = Document()
1404:     section = doc.sections[0]
1405:     section.page_height = Inches(11)
1406:     section.page_width = Inches(8.5)
1407:     for section in doc.sections:
1408:         section.top_margin = Inches(0.65)
1409:         section.bottom_margin = Inches(0.6)
1410:         section.left_margin = Inches(0.65)
1411:         section.right_margin = Inches(0.65)
1412:         section.header_distance = Inches(0.28)
1413:         section.footer_distance = Inches(0.28)
1414:
1415:     normal = doc.styles["Normal"]
```

```

1416:     normal.font.name = "Calibri"
1417:     normal._element.rPr.rFonts.set(qn("w: eastAsia"), "Calibri")
1418:     normal.font.size = Pt(10)
1419:     normal.paragraph_format.space_after = Pt(3)
1420:     normal.paragraph_format.line_spacing = 1.08
1421:
1422:     for name, size, color, before, after in [
1423:         ("Heading 1", 14, "2E74B5", 9, 4),
1424:         ("Heading 2", 12, "2E74B5", 7, 3),
1425:         ("Heading 3", 10.5, "1F4D78", 5, 2),
1426:     ]:
1427:         style = doc.styles[name]
1428:         style.font.name = "Calibri"
1429:         style._element.rPr.rFonts.set(qn("w: eastAsia"), "Calibri")
1430:         style.font.size = Pt(size)
1431:         style.font.color.rgb = RGBColor.from_string(color)
1432:         style.paragraph_format.space_before = Pt(before)
1433:         style.paragraph_format.space_after = Pt(after)
1434:
1435:     code = doc.styles.add_style("CodeBlock", 1)
1436:     code.font.name = "Consolas"
1437:     code._element.rPr.rFonts.set(qn("w: eastAsia"), "Consolas")
1438:     code.font.size = Pt(7.2)
1439:     code.paragraph_format.space_before = Pt(2)
1440:     code.paragraph_format.space_after = Pt(3)
1441:     code.paragraph_format.line_spacing = 1.0
1442:
1443:     header = doc.sections[0].header.paragraphs[0]
1444:     header.text = "OMB Portfolio Builder and Website Complete Guide"
1445:     header.alignment = WD_ALIGN_PARAGRAPH.RIGHT
1446:     for run in header.runs:
1447:         run.font.size = Pt(8)
1448:         run.font.color.rgb = RGBColor(85, 85, 85)
1449:     return doc
1450:
1451:

```

add_cover (docs/build_complete_guide.py, lines 1452-1469)

add_cover named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1452-1469 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_paragraph, add_run, Pt, RGBColor, get, add_note, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1452: def add_cover(doc: Document, package: dict) -> None:
1453:     title = doc.add_paragraph()
1454:     title.alignment = WD_ALIGN_PARAGRAPH.CENTER
1455:     run = title.add_run("OMB Portfolio Builder and Website\nComplete Creation Guide")
1456:     run.bold = True
1457:     run.font.size = Pt(26)
1458:     run.font.color.rgb = RGBColor(11, 37, 69)
1459:     subtitle = doc.add_paragraph()
1460:     subtitle.alignment = WD_ALIGN_PARAGRAPH.CENTER
1461:     r = subtitle.add_run("A beginner-friendly manual explaining the desktop app, public website, GitHub
workflows, installer, parser, compiler tools, AI backend, and publishing system.")
1462:     r.font.size = Pt(12)
1463:     meta = doc.add_paragraph()
1464:     meta.alignment = WD_ALIGN_PARAGRAPH.CENTER
1465:     meta.add_run(f"Repository: otienomaurice/omb-portfolio-builder\nVersion documented:
{package.get('version', 'unknown')}\nGenerated: July 4, 2026")
1466:     add_note(doc, "How this guide is written", "This guide starts from the highest-level idea and then
drills down. It intentionally explains terms that engineers usually skip: what Electron is, what a workflow is,
what a local backend is, why Git branches exist, and why the builder and website are separated.")
1467:     doc.add_page_break()
1468:
1469:

```

add_reading_map (docs/build_complete_guide.py, lines 1470-1483)

add_reading_map named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1470-1483 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `add_heading`, `add_paragraph`, `add_table`, and `add_page_break`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1470: def add_reading_map(doc: Document) -> None:
1471:     doc.add_heading("Reading Map", level=1)
1472:     doc.add_paragraph("Use this manual in layers. First read the high-level design pages. Then read the
app flow pages. After that, use the file-by-file and command pages as a reference when you need to understand a
specific file or command.")
1473:     add_table(doc, [
1474:         ("Part 1", "High-level architecture and big-picture mental model."),
1475:         ("Part 2", "Builder app internals: Electron, local backend, parser, rich editors, previews,
compile code, and publishing."),
1476:         ("Part 3", "Public website internals: HTML, CSS, JavaScript, projects.json, assets, search, AI,
and mobile behavior."),
1477:         ("Part 4", "GitHub, branches, workflows, releases, installers, and updates."),
1478:         ("Part 5", "Every tracked file explained in plain English."),
1479:         ("Part 6", "Commands, tradeoffs, troubleshooting, and safe operating procedures."),
1480:     ])
1481:     doc.add_page_break()
1482:
1483:
```

add_entry_points (docs/build_complete_guide.py, lines 1484-1500)

`add_entry_points` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1484-1500 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `add_heading`, `add_paragraph`, `add_table`, `numbers`, and `add_page_break`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1484: def add_entry_points(doc: Document) -> None:
1485:     doc.add_heading("Where Someone Starts: Entry Points", level=1)
1486:     doc.add_paragraph("An entry point is the first file, executable, or function that starts a flow. This
project has several entry points because normal users, developers, GitHub Actions, Cloudflare, and installers
all start in different places.")
1487:     add_table(doc, [(name, f"{entry}. {meaning}") for name, entry, meaning in ENTRY_POINTS])
1488:     doc.add_heading("Fastest way to orient yourself", level=2)
1489:     numbers(doc, [
1490:         "If you are using the installed app, start with OMB Portfolio Builder.exe.",
1491:         "If you are debugging desktop startup, start with package.json and main.cjs.",
1492:         "If you are debugging local APIs, start with server.mjs.",
1493:         "If you are debugging builder UI, start with template-preview.html, template-preview.js, and
template-preview.css.",
1494:         "If you are debugging the public website, start with index.html, script.js, styles.css, and
projects.json.",
1495:         "If you are debugging installer behavior, start with package.json build.nsh and
build/installer.nsh.",
1496:         "If you are debugging releases, start with .github/workflows/build-windows-builder.yml.",
1497:     ])
1498:     doc.add_page_break()
1499:
1500:
```

add_flow_walkthroughs (docs/build_complete_guide.py, lines 1501-1511)

`add_flow_walkthroughs` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1501-1511 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `add_heading`, `add_diagram`, `add_paragraph`, `add_table`, and `add_page_break`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1501: def add_flow_walkthroughs(doc: Document, diagrams: dict[str, Path]) -> None:
1502:     for title, diagram_name, steps, debug_note in FLOW_WALKTHROUGHS:
1503:         doc.add_heading(f"Detailed Walkthrough: {title}", level=1)
1504:         add_diagram(doc, diagrams[diagram_name], f"Context diagram for {title}.")
1505:         doc.add_paragraph("This walkthrough follows the real direction of work through the system. Read
it slowly: each line is one handoff from one layer to another.")
1506:         add_table(doc, steps)
1507:         doc.add_heading("Debug note", level=2)
1508:         doc.add_paragraph(debug_note)
1509:         doc.add_page_break()
1510:
1511:
```

add_data_contracts (docs/build_complete_guide.py, lines 1512-1524)

add_data_contracts named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1512-1524 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_code, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1512: def add_data_contracts(doc: Document) -> None:
1513:     for title, purpose, example, note in DATA_CONTRACTS:
1514:         doc.add_heading(f"Data Contract: {title}", level=1)
1515:         doc.add_paragraph(purpose)
1516:         doc.add_heading("Example shape", level=2)
1517:         add_code(doc, example)
1518:         doc.add_heading("How to read this", level=2)
1519:         doc.add_paragraph(note)
1520:         doc.add_heading("Why contracts matter", level=2)
1521:         doc.add_paragraph("A data contract is an agreement between two pieces of software. If the
frontend sends one shape and the backend expects another shape, the feature breaks even if both files are
individually valid code.")
1522:         doc.add_page_break()
1523:
1524:
```

endpoint_purpose (docs/build_complete_guide.py, lines 1525-1554)

endpoint_purpose named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1525-1554 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 1527: return "Compiles or runs a source file and returns terminal output."; line 1529: return "Saves source code into the local compile workspace."; line 1531: return "Formats source code for cleaner display and editing."; line 1533: return "Checks compiler/runtime availability.". There are 8 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1525: def endpoint_purpose(endpoint: str) -> str:
1526:     if "code/compile" in endpoint:
1527:         return "Compiles or runs a source file and returns terminal output."
1528:     if "code/save" in endpoint:
1529:         return "Saves source code into the local compile workspace."
1530:     if "code/beautify" in endpoint:
1531:         return "Formats source code for cleaner display and editing."
1532:     if "code/tools" in endpoint:
1533:         return "Checks compiler/runtime availability."
1534:     if "code/install" in endpoint:
1535:         return "Attempts to install missing compiler tools."
1536:     if "publish-target" in endpoint:
```

```

1537:         return "Reads or writes the Git publishing target and authorization state."
1538:     if "publish" in endpoint:
1539:         return "Publishes generated website files to the target repository."
1540:     if "catalog" in endpoint:
1541:         return "Loads project/catalog data for the builder."
1542:     if "templates" in endpoint:
1543:         return "Loads appearance template definitions."
1544:     if "app-update" in endpoint:
1545:         return "Checks or starts builder app update behavior."
1546:     if "security-report" in endpoint:
1547:         return "Returns security/download/auth reporting for the builder."
1548:     if "portfolio-ai" in endpoint:
1549:         return "Handles AI assistant questions."
1550:     if "system-check" in endpoint:
1551:         return "Checks local tool/system readiness."
1552:     return "Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact
behavior."
1553:
1554:

```

extract_api_endpoints (docs/build_complete_guide.py, lines 1555-1567)

extract_api_endpoints named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1555-1567 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, enumerate, read_text, splitlines, findall, rstrip, min, get, str, sorted, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1565: return [(endpoint, file_name, str(line_no)) for (endpoint, file_name), line_no in sorted(endpoints.items())].

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1555: def extract_api_endpoints() -> list[tuple[str, str, str]]:
1556:     endpoints: dict[tuple[str, str], int] = {}
1557:     for file_name in ["template-preview.js", "script.js", "server.mjs", "index.html"]:
1558:         path = REPO / file_name
1559:         if not path.exists():
1560:             continue
1561:         for line_no, line in enumerate(path.read_text(encoding="utf-8", errors="replace").splitlines(),
start=1):
1562:             for match in re.findall(r"['\"](/api/[A-Za-z0-9_./-]+)", line):
1563:                 clean = match.rstrip("/")
1564:                 endpoints[(clean, file_name)] = min(line_no, endpoints.get((clean, file_name), line_no))
1565:     return [(endpoint, file_name, str(line_no)) for (endpoint, file_name), line_no in
sorted(endpoints.items())]
1566:
1567:

```

add_api_endpoint_catalog (docs/build_complete_guide.py, lines 1568-1590)

add_api_endpoint_catalog named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1568-1590 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references extract_api_endpoints, add_heading, add_paragraph, add_page_break, endpoint_purpose, and add_table. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1575: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1568: def add_api_endpoint_catalog(doc: Document) -> None:
1569:     endpoints = extract_api_endpoints()
1570:     doc.add_heading("API Endpoint Catalog", level=1)
1571:     doc.add_paragraph("This catalog is generated from strings found in the source files. It tells you
which /api paths are used and where to start reading when a request fails.")
1572:     if not endpoints:
1573:         doc.add_paragraph("No /api endpoints were found by the scanner.")
1574:         doc.add_page_break()

```

```

1575:         return
1576:         rows = [(endpoint, f"Seen in {file_name} near line {line_no}. Purpose: {endpoint_purpose(endpoint)}")
for endpoint, file_name, line_no in endpoints]
1577:         add_table(doc, rows)
1578:         doc.add_page_break()
1579:         for endpoint, file_name, line_no in endpoints:
1580:             doc.add_heading(f"API Detail: {endpoint}", level=1)
1581:             add_table(doc, [
1582:                 ("Where seen", f"{file_name} near line {line_no}"),
1583:                 ("Purpose", endpoint_purpose(endpoint)),
1584:                 ("Frontend question", "Which button, form, or page action sends this request?"),
1585:                 ("Backend question", "Which handler in server.mjs receives this path and what files/tools
does it touch?"),
1586:                 ("Debugging", "Check browser network status, response JSON, server logs, and whether stale
cache was avoided with no-store or a timestamp."),
1587:             ])
1588:             doc.add_page_break()
1589:
1590:

```

function_purpose (docs/build_complete_guide.py, lines 1591-1629)

function_purpose named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1591-1629 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references lower, and startswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 12 return statement(s). The most important visible returns are: line 1594: return "Renders UI, markup, or display output."; line 1596: return "Handles an event, request, command, or user action."; line 1598: return "Updates UI, state, cache, or stored data."; line 1600: return "Persists data to local state, disk, credentials, or browser storage.". There are 8 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1591: def function_purpose(file_name: str, function_name: str) -> str:
1592:     name = function_name.lower()
1593:     if name.startswith("render"):
1594:         return "Renders UI, markup, or display output."
1595:     if name.startswith("handle"):
1596:         return "Handles an event, request, command, or user action."
1597:     if name.startswith("update"):
1598:         return "Updates UI, state, cache, or stored data."
1599:     if name.startswith("save") or name.startswith("write") or name.startswith("store"):
1600:         return "Persists data to local state, disk, credentials, or browser storage."
1601:     if name.startswith("read") or name.startswith("load") or name.startswith("fetch"):
1602:         return "Loads data from disk, network, browser storage, or a service."
1603:     if "compile" in name or "compiler" in name:
1604:         return "Part of the Compile Code language/tool/build/run flow."
1605:     if "publish" in name or "git" in name or "remote" in name:
1606:         return "Part of publishing, Git, target repository, or authorization behavior."
1607:     if "auth" in name or "credential" in name or "access" in name:
1608:         return "Part of authentication, authorization, or credential checking."
1609:     if "cache" in name:
1610:         return "Reads, writes, or validates cached state."
1611:     if name.startswith("validate") or name.startswith("normalize") or name.startswith("safe") or
name.startswith("sanitize"):
1612:         return "Validates or normalizes input so later code receives safe predictable values."
1613:     if name.startswith("create") or name.startswith("new") or name.startswith("build"):
1614:         return "Creates an object, UI structure, process, payload, or generated artifact."
1615:     if name.startswith("open") or name.startswith("close") or name.startswith("show") or
name.startswith("hide"):
1616:         return "Controls a window, dialog, panel, menu, or visible state."
1617:     if name.startswith("sync"):
1618:         return "Keeps two places aligned, such as local data and target files."
1619:     if file_name == "main.cjs":
1620:         return "Part of Electron desktop startup, window control, workspace setup, menu behavior, or
update handoff."
1621:     if file_name == "server.mjs":
1622:         return "Part of the local backend API, filesystem, compiler, Git, AI, update, or publish
workflow."
1623:     if file_name == "template-preview.js":
1624:         return "Part of the private builder frontend and editing experience."
1625:     if file_name == "script.js":
1626:         return "Part of the public website runtime."
1627:     return "Named function in the repository. Inspect the surrounding lines to learn its exact inputs and
outputs."
1628:
1629:

```

line_excerpt (docs/build_complete_guide.py, lines 1664-1673)

line_excerpt named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1664-1673 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references max, min, len, enumerate, append, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1671: return "\n".join(rendered).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1664: def line_excerpt(lines: list[str], start: int, end: int, max_lines: int = 120) -> str:
1665:     selected = lines[max(start - 1, 0): min(end, len(lines))]
1666:     truncated = len(selected) > max_lines
1667:     selected = selected[:max_lines]
1668:     rendered = [f"{start + index:>5}: {line}" for index, line in enumerate(selected)]
1669:     if truncated:
1670:         rendered.append(f"    ... excerpt truncated after {max_lines} lines. Open the source file for
the rest of this function.")
1671:     return "\n".join(rendered)
1672:
1673:
```

find_block_end (docs/build_complete_guide.py, lines 1674-1689)

find_block_end named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1674-1689 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references range, min, len, sub, count, and startswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1684: return index + 1; line 1686: return index; line 1687: return min(start_index + 80, len(lines)).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1674: def find_block_end(lines: list[str], start_index: int) -> int:
1675:     brace_balance = 0
1676:     saw_brace = False
1677:     for index in range(start_index, min(start_index + 500, len(lines))):
1678:         line = re.sub(r"/\.*$", "", lines[index])
1679:         brace_balance += line.count("{")
1680:         brace_balance -= line.count("}")
1681:         if "{" in line:
1682:             saw_brace = True
1683:         if saw_brace and brace_balance <= 0 and index > start_index:
1684:             return index + 1
1685:         if not saw_brace and index > start_index and not lines[index].startswith((" ", "\t")):
1686:             return index
1687:     return min(start_index + 80, len(lines))
1688:
1689:
```

extract_function_blocks (docs/build_complete_guide.py, lines 1690-1750)

This function scans source files, finds function declarations, estimates function boundaries, and records calls, returns, storage keys, endpoints, and source excerpts.

The documentation generator depends on it to build source-aware function explanations instead of hand-maintained stale lists.

It calls or references enumerate, strip, search, group, lstrip, startswith, next, range, len, min, and 14 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1748: return functions.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1690: def extract_function_blocks(repo_file: str, lines: list[str]) -> list[dict]:
1691:     functions = []
1692:     for index, line in enumerate(lines):
```

```

1693:         stripped = line.strip()
1694:         for pattern in FUNCTION_PATTERNS:
1695:             match = pattern.search(line)
1696:             if not match:
1697:                 continue
1698:             name = match.group(1)
1699:             if line.lstrip().startswith("Function "):
1700:                 end = next((i + 1 for i in range(index + 1, len(lines)) if
lines[i].lstrip().startswith("FunctionEnd")), min(index + 80, len(lines)))
1701:             elif line.lstrip().startswith("def "):
1702:                 end = next((i for i in range(index + 1, len(lines)) if lines[i] and not
lines[i].startswith((" ", "\t", "@"))), min(index + 120, len(lines)))
1703:             else:
1704:                 end = find_block_end(lines, index)
1705:             body = "\n".join(lines[index:end])
1706:             call_candidates = re.findall(r"\b([A-Za-z_$][\w$]*)\s*(\(", body)
1707:             calls = []
1708:             for candidate in call_candidates:
1709:                 if candidate not in COMMON_CALL_WORDS and candidate != name and candidate not in calls:
1710:                     calls.append(candidate)
1711:             endpoint_hits = sorted(set(match.rstrip("/") for match in
re.findall(r"['\"](/api/[A-Za-z0-9_-./-]+)", body)))
1712:             local_storage_keys = sorted(set(match for match in
re.findall(r"(?:localStorage|sessionStorage)\.(?:getItem|setItem|removeItem)\(['\"]([^\"]+)", body)))
1713:             return_lines = []
1714:             local_variables = []
1715:             for offset, body_line in enumerate(lines[index:end], start=index + 1):
1716:                 stripped_body_line = body_line.strip()
1717:                 if re.search(r"\breturn\b", stripped_body_line):
1718:                     return_lines.append({"line": offset, "statement": stripped_body_line[:220]})
1719:                 var_match = re.match(r"^\s*(?:const|let|var)\s+([A-Za-z_$][\w$]*)\s*=", body_line)
1720:                 if var_match and len(local_variables) < 18:
1721:                     local_variables.append({
1722:                         "line": offset,
1723:                         "name": var_match.group(1),
1724:                         "statement": stripped_body_line[:220],
1725:                     })
1726:             functions.append({
1727:                 "line": index + 1,
1728:                 "end_line": end,
1729:                 "name": name,
1730:                 "purpose": function_purpose(Path(repo_file).name, name),
1731:                 "signature": stripped[:240],
1732:                 "excerpt": line_excerpt(lines, index + 1, end, max_lines=120),
1733:                 "line_count": max(end - index, 1),
1734:                 "calls": calls[:30],
1735:                 "endpoints": endpoint_hits[:30],
1736:                 "storage_keys": local_storage_keys[:30],
1737:                 "returns": len(re.findall(r"\breturn\b", body)),
1738:                 "return_lines": return_lines[:12],
1739:                 "local_variables": local_variables,
1740:                 "awaits": len(re.findall(r"\bawait\b", body)),
1741:                 "touches_dom":
bool(re.search(r"document\.querySelector|getElementById|classList|dataset", body)),
1742:                 "touches_files":
bool(re.search(r"readFile|writeFile|appendFile|copyFile|mkdir|rm|unlink|fs\.", body, re.IGNORECASE)),
1743:                 "runs_process": bool(re.search(r"\bspawn\b|\bexecFile\b|\bexec\b", body, re.IGNORECASE)),
1744:                 "uses_network": bool(re.search(r"\bfetch\s*\(|https?://|api/", body, re.IGNORECASE)),
1745:                 "uses_security":
bool(re.search(r"auth|token|secret|credential|permission|authorize|scope", body, re.IGNORECASE)),
1746:             })
1747:             break
1748:         return functions
1750:

```

collect_signal_hits (docs/build_complete_guide.py, lines 1751-1768)

collect_signal_hits named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1751-1768 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references enumerate, strip, search, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1766: return hits.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1751: def collect_signal_hits(lines: list[str]) -> list[dict]:
1752:     hits = []
1753:     for line_no, line in enumerate(lines, start=1):

```

```

1754:         stripped = line.strip()
1755:         if not stripped:
1756:             continue
1757:         for name, pattern, explanation in SOURCE_SIGNAL_RULES:
1758:             if pattern.search(line):
1759:                 hits.append({
1760:                     "line": line_no,
1761:                     "type": name,
1762:                     "evidence": stripped[:220],
1763:                     "meaning": explanation,
1764:                 })
1765:             break
1766:         return hits
1767:
1768:

```

function_detail_rows (docs/build_complete_guide.py, lines 1769-1793)

function_detail_rows named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1769-1793 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, join, expression, and statement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1781: return [; line 1789: ("Async/return clues", f"{item['awaits']} await expression(s), {item['returns']} return statement(s)."),.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1769: def function_detail_rows(item: dict) -> list[tuple[str, str]]:
1770:     touches = []
1771:     if item["touches_dom"]:
1772:         touches.append("browser/Electron DOM")
1773:     if item["touches_files"]:
1774:         touches.append("filesystem")
1775:     if item["runs_process"]:
1776:         touches.append("external processes")
1777:     if item["uses_network"]:
1778:         touches.append("network/API requests")
1779:     if item["uses_security"]:
1780:         touches.append("authentication/security data")
1781:     return [
1782:         ("Source location", f"Lines {item['line']}-{item['end_line']} ({item['line_count']} source
lines)."),
1783:         ("Purpose", item["purpose"]),
1784:         ("Declaration", item["signature"]),
1785:         ("Touches", ", ".join(touches) if touches else "Mostly local calculations or local UI state."),
1786:         ("Calls detected", ", ".join(item["calls"][:12]) if item["calls"] else "No obvious named calls
detected by the generator."),
1787:         ("API endpoints", ", ".join(item["endpoints"]) if item["endpoints"] else "None detected inside
this function body."),
1788:         ("Storage keys", ", ".join(item["storage_keys"]) if item["storage_keys"] else "No local/session
storage keys detected."),
1789:         ("Async/return clues", f"{item['awaits']} await expression(s), {item['returns']} return
statement(s)."),
1790:         ("Debug approach", "Trigger the user action that reaches this function, inspect inputs/state
before the function, then inspect the returned value, DOM update, file write, process output, or API response
after it runs."),
1791:     ]
1792:
1793:

```

function_detail (docs/build_complete_guide.py, lines 1794-1801)

function_detail named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1794-1801 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, lower, and folder_role. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1796: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1794: def function_detail(item: dict, repo_file: str) -> dict:
1795:     custom = IMPORTANT_FUNCTION_DETAILS.get((repo_file, item["name"]), {})
1796:     return {
1797:         "what": custom.get("what") or f"{item['name']} {item['purpose'].lower()} In this file, it appears
around lines {item['line']}-{item['end_line']} and is part of the {folder_role(repo_file).lower()}",
1798:         "why": custom.get("why") or f"This function is needed so {repo_file} can keep that responsibility
in one named place instead of scattering it through unrelated event handlers or route code.",
1799:     }
1800:
1801:
```

sentence_list (docs/build_complete_guide.py, lines 1802-1812)

sentence_list named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1802-1812 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references str, len, append, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1804: return empty; line 1809: return selected[0]; line 1810: return ", ".join(selected[:-1]) + f", and {selected[-1]}".

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1802: def sentence_list(values: list[str], empty: str, limit: int = 10) -> str:
1803:     if not values:
1804:         return empty
1805:     selected = [str(value) for value in values[:limit] if str(value)]
1806:     if len(values) > limit:
1807:         selected.append(f"{len(values) - limit} more")
1808:     if len(selected) == 1:
1809:         return selected[0]
1810:     return ", ".join(selected[:-1]) + f", and {selected[-1]}"
1811:
1812:
```

function_calls_paragraph (docs/build_complete_guide.py, lines 1813-1819)

function_calls_paragraph named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1813-1819 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references sentence_list, and get. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1817: return f"It calls or references {calls}. Endpoint references found inside it are {endpoints}. Browser storage keys found inside it are {storage}."

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1813: def function_calls_paragraph(item: dict) -> str:
1814:     calls = sentence_list(item.get("calls", []), [], "no major named helper calls detected")
1815:     endpoints = sentence_list(item.get("endpoints", []), [], "no direct /api endpoint strings detected")
1816:     storage = sentence_list(item.get("storage_keys", []), [], "no local/session storage keys detected")
1817:     return f"It calls or references {calls}. Endpoint references found inside it are {endpoints}. Browser
storage keys found inside it are {storage}."
1818:
1819:
```

function_returns_paragraph (docs/build_complete_guide.py, lines 1820-1834)

function_returns_paragraph named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1820-1834 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, join, len, and statement. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 6 return statement(s). The most important visible returns are: line 1824: extra = "" if len(return_lines) <= 4 else f" There are {len(return_lines) - 4} additional return statements in the function body."; line 1825: return f"It has {len(return_lines)} return statement(s). The most important visible returns are: {snippets}{extra}"; line 1827: return "It does not expose many explicit return statements, but it produces its result through process output, side effects, or a structured response built after external commands finish."; line 1829: return "It does not mainly return a value; its output is the changed screen state or DOM it updates.". There are 2 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1820: def function_returns_paragraph(item: dict) -> str:
1821:     return_lines = item.get("return_lines", [])
1822:     if return_lines:
1823:         snippets = "; ".join(f"line {entry['line']}: {entry['statement']}" for entry in return_lines[:4])
1824:         extra = "" if len(return_lines) <= 4 else f" There are {len(return_lines) - 4} additional return
statements in the function body."
1825:         return f"It has {len(return_lines)} return statement(s). The most important visible returns are:
{snippets}{extra}"
1826:     if item.get("runs_process"):
1827:         return "It does not expose many explicit return statements, but it produces its result through
process output, side effects, or a structured response built after external commands finish."
1828:     if item.get("touches_dom"):
1829:         return "It does not mainly return a value; its output is the changed screen state or DOM it
updates."
1830:     if item.get("touches_files"):
1831:         return "Its result is mostly side effect based: it reads, writes, copies, or synchronizes files
rather than returning a simple value."
1832:     return "It has no obvious return statement in the extracted body; it likely completes through state
changes, helper calls, or event flow."
1833:
1834:
```

function_variables_paragraph (docs/build_complete_guide.py, lines 1835-1843)

function_variables_paragraph named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1835-1843 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, join, len, and variable. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1838: return "No major local variable declarations were detected in the extracted function body."; line 1841: return f"Important local values include {snippets}{extra}. These variables show what the function is assembling, checking, or handing to another layer.".

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1835: def function_variables_paragraph(item: dict) -> str:
1836:     variables = item.get("local_variables", [])
1837:     if not variables:
1838:         return "No major local variable declarations were detected in the extracted function body."
1839:     snippets = "; ".join(f"{entry['name']} at line {entry['line']}" for entry in variables[:8])
1840:     extra = "" if len(variables) <= 8 else f"; plus {len(variables) - 8} more local variable(s)"
1841:     return f"Important local values include {snippets}{extra}. These variables show what the function is
assembling, checking, or handing to another layer."
1842:
1843:
```

add_function_explanation_docx (docs/build_complete_guide.py, lines 1844-1856)

add_function_explanation_docx named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1844-1856 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references function_detail, add_heading, add_paragraph, function_calls_paragraph, function_returns_paragraph, function_variables_paragraph, and add_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1844: def add_function_explanation_docx(doc: Document, repo_file: str, item: dict, include_excerpt: bool =
True, heading_level: int = 3) -> None:
1845:     details = function_detail(item, repo_file)
1846:     doc.add_heading(f"{item['name']} ({repo_file}, lines {item['line']}-{item['end_line']})",
level=heading_level)
1847:     doc.add_paragraph(details["what"])
1848:     doc.add_paragraph(details["why"])
1849:     doc.add_paragraph(function_calls_paragraph(item))
1850:     doc.add_paragraph(function_returns_paragraph(item))
1851:     doc.add_paragraph(function_variables_paragraph(item))
1852:     if include_excerpt:
1853:         doc.add_paragraph("Relevant source excerpt:")
1854:         add_code(doc, item["excerpt"])
1855:
1856:
```

add_function_explanation_pdf (docs/build_complete_guide.py, lines 1857-1869)

add_function_explanation_pdf named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1857-1869 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references function_detail, append, Paragraph, pdf_paragraph, function_calls_paragraph, function_returns_paragraph, function_variables_paragraph, and pdf_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1857: def add_function_explanation_pdf(story: list, repo_file: str, item: dict, styles, include_excerpt: bool =
True) -> None:
1858:     details = function_detail(item, repo_file)
1859:     story.append(Paragraph(f"{item['name']} ({repo_file}, lines {item['line']}-{item['end_line']})",
styles["Heading3"]))
1860:     story.append(pdf_paragraph(details["what"], styles["SmallBody"]))
1861:     story.append(pdf_paragraph(details["why"], styles["SmallBody"]))
1862:     story.append(pdf_paragraph(function_calls_paragraph(item), styles["SmallBody"]))
1863:     story.append(pdf_paragraph(function_returns_paragraph(item), styles["SmallBody"]))
1864:     story.append(pdf_paragraph(function_variables_paragraph(item), styles["SmallBody"]))
1865:     if include_excerpt:
1866:         story.append(pdf_paragraph("Relevant source excerpt:", styles["SmallBody"]))
1867:         story.append(pdf_code(item["excerpt"], styles))
1868:
1869:
```

important_function_blocks (docs/build_complete_guide.py, lines 1870-1886)

important_function_blocks named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1870-1886 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references extract_source_facts, sorted, and, exists, get, next, and append. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1884: return selected.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1870: def important_function_blocks(files: list[str]) -> list[tuple[str, dict]]:
1871:     facts_by_file = {
1872:         repo_file: extract_source_facts(repo_file)
1873:         for repo_file in sorted({repo_file for repo_file, _name in IMPORTANT_FUNCTIONS})
1874:         if repo_file in files and (REPO / repo_file).exists()
1875:     }
1876:     selected = []
```

```

1877:     for repo_file, function_name in IMPORTANT_FUNCTIONS:
1878:         facts = facts_by_file.get(repo_file)
1879:         if not facts:
1880:             continue
1881:         match = next((item for item in facts["functions"] if item["name"] == function_name), None)
1882:         if match:
1883:             selected.append((repo_file, match))
1884:     return selected
1885:
1886:

```

extract_functions (docs/build_complete_guide.py, lines 1887-1902)

extract_functions named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1887-1902 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, enumerate, read_text, splitlines, search, group, append, and function_purpose. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1900: return function_rows.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1887: def extract_functions() -> list[tuple[str, int, str, str]]:
1888:     function_rows: list[tuple[str, int, str, str]] = []
1889:     for file_name in ["main.cjs", "server.mjs", "template-preview.js", "script.js",
1890: "cloudflare/portfolio-ai-worker.js", "docs/build_complete_guide.py", "build/installer.nsh"]:
1891:         path = REPO / file_name
1892:         if not path.exists():
1893:             continue
1894:         for line_no, line in enumerate(path.read_text(encoding="utf-8", errors="replace").splitlines(),
1895 start=1):
1896:             for pattern in FUNCTION_PATTERNS:
1897:                 match = pattern.search(line)
1898:                 if match:
1899:                     name = match.group(1)
1900:                     function_rows.append((file_name, line_no, name, function_purpose(file_name, name)))
1901:                     break
1902:     return function_rows

```

is_text_code_file (docs/build_complete_guide.py, lines 1908-1916)

is_text_code_file named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1908-1916 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references is_generated_reference_file, endswith, is_file, and lower. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 1911: return False; line 1913: return False; line 1914: return path.is_file() and path.suffix.lower() in TEXT_CODE_EXTENSIONS.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1908: def is_text_code_file(repo_file: str) -> bool:
1909:     path = REPO / repo_file
1910:     if is_generated_reference_file(repo_file):
1911:         return False
1912:     if repo_file.endswith((".docx", ".pdf", ".png", ".ico")):
1913:         return False
1914:     return path.is_file() and path.suffix.lower() in TEXT_CODE_EXTENSIONS
1915:
1916:

```

source_lines (docs/build_complete_guide.py, lines 1917-1923)

source_lines named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1917-1923 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, is_file, read_text, and splitlines. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 1920: return []; line 1921: return path.read_text(encoding="utf-8", errors="replace").splitlines().

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1917: def source_lines(repo_file: str) -> list[str]:
1918:     path = REPO / repo_file
1919:     if not path.exists() or not path.is_file():
1920:         return []
1921:     return path.read_text(encoding="utf-8", errors="replace").splitlines()
1922:
1923:
```

extract_source_facts (docs/build_complete_guide.py, lines 1924-1983)

extract_source_facts named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1924-1983 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references source_lines, join, extract_function_blocks, enumerate, strip, match, len, append, group, findall, and 12 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1960: return {.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
1924: def extract_source_facts(repo_file: str) -> dict:
1925:     lines = source_lines(repo_file)
1926:     text = "\n".join(lines)
1927:     functions = extract_function_blocks(repo_file, lines)
1928:     variables = []
1929:     imports = []
1930:     endpoints = []
1931:     selectors = []
1932:     events = []
1933:     routes = []
1934:     json_keys = []
1935:     for line_no, line in enumerate(lines, start=1):
1936:         stripped = line.strip()
1937:         var_match = re.match(r"^\s*(?:const|let|var)\s+([A-Za-z_$][\w$]*)\s*=", line)
1938:         if var_match and len(variables) < 140:
1939:             variables.append({
1940:                 "line": line_no,
1941:                 "name": var_match.group(1),
1942:                 "statement": stripped[:180],
1943:             })
1944:         if re.match(r"^\s*import\s+", line) or re.match(r"^\s*(?:const|let|var)\s+\S+\s*=\s*require\(", line):
1945:             imports.append({"line": line_no, "statement": stripped[:220]})
1946:         for match in re.findall(r"['\"](/api/[A-Za-z0-9_-./-]+)", line):
1947:             endpoints.append({"line": line_no, "endpoint": match.rstrip("/")})
1948:         route_match = re.search(r"\bapp\.get|post|put|delete|patch\)(['\"]([^\"]+)", line)
1949:         if route_match:
1950:             routes.append({"line": line_no, "method": route_match.group(1).upper(), "path": route_match.group(2)})
1951:         if ".addEventListener(" in line:
1952:             events.append({"line": line_no, "statement": stripped[:180]})
1953:         json_match = re.match(r"^\s*"([^\s]+)"\s*:", line)
1954:         if json_match and len(json_keys) < 120:
1955:             json_keys.append({"line": line_no, "key": json_match.group(1), "statement": stripped[:180]})
1956:         if repo_file.endswith(".css"):
1957:             selector_match = re.match(r"^\s*(\.[A-Za-z0-9_-:\[\]\(\)\s>+.=\\'"]+)\s*\{", line)
1958:             if selector_match and len(selectors) < 140:
1959:                 selectors.append({"line": line_no, "selector": selector_match.group(1).strip()[:160]})
1960:     return {
1961:         "repo_file": repo_file,
1962:         "line_count": len(lines),
1963:         "size_bytes": (REPO / repo_file).stat().st_size if (REPO / repo_file).exists() else 0,
1964:         "description": FILE_DESCRIPTIONS.get(repo_file, "Tracked source/configuration file in the OMB Portfolio Builder repository."),
1965:         "functions": functions,
```



```

1966:         "variables": variables,
1967:         "imports": imports,
1968:         "endpoints": endpoints,
1969:         "routes": routes,
1970:         "selectors": selectors,
1971:         "events": events,
1972:         "json_keys": json_keys,
1973:         "signals": collect_signal_hits(lines),
1974:         "mentions_server": "server.mjs" in text,
1975:         "mentions_template_preview": "template-preview" in text,
1976:         "mentions_script": "script.js" in text,
1977:         "mentions_projects_json": "projects.json" in text,
1978:         "mentions_github": "github" in text.lower(),
1979:         "mentions_cloudflare": "cloudflare" in text.lower() or "worker" in text.lower(),
1980:         "snippet": line_excerpt(lines, 1, min(72, len(lines)), max_lines=72) if lines else "",
1981:     }
1982:
1983:

```

fact_relationship_summary (docs/build_complete_guide.py, lines 1984-2000)

fact_relationship_summary named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 1984-2000 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references get, append, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 1998: return ", ".join(related) if related else "Mostly local to its own feature area."

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

1984: def fact_relationship_summary(facts: dict) -> str:
1985:     related = []
1986:     if facts.get("mentions_template_preview"):
1987:         related.append("builder frontend")
1988:     if facts.get("mentions_server"):
1989:         related.append("local backend")
1990:     if facts.get("mentions_script"):
1991:         related.append("public website runtime")
1992:     if facts.get("mentions_projects_json"):
1993:         related.append("portfolio catalog")
1994:     if facts.get("mentions_cloudflare"):
1995:         related.append("Cloudflare/AI layer")
1996:     if facts.get("mentions_github"):
1997:         related.append("GitHub/release layer")
1998:     return ", ".join(related) if related else "Mostly local to its own feature area."
1999:
2000:

```

folder_role (docs/build_complete_guide.py, lines 2001-2018)

folder_role named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2001-2018 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references startswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 8 return statement(s). The most important visible returns are: line 2003: return "GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs."; line 2005: return "Public web metadata. Browsers, security tools, and crawlers may request this path."; line 2007: return "Public and desktop visual asset. Used for favicons, app icons, branding, or install surfaces."; line 2009: return "Packaging and installer customization. Used while creating the Windows app installer.". There are 4 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2001: def folder_role(repo_file: str) -> str:
2002:     if repo_file.startswith(".github/workflows/"):
2003:         return "GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs."
2004:     if repo_file.startswith(".well-known/"):
2005:         return "Public web metadata. Browsers, security tools, and crawlers may request this path."
2006:     if repo_file.startswith("assets/"):

```

```

2007:         return "Public and desktop visual asset. Used for favicons, app icons, branding, or install
surfaces."
2008:     if repo_file.startswith("build/"):
2009:         return "Packaging and installer customization. Used while creating the Windows app installer."
2010:     if repo_file.startswith("cloudflare/"):
2011:         return "Cloudflare Worker/deployment area. Used for public AI backend and Cloudflare deployment
notes."
2012:     if repo_file.startswith("docs/"):
2013:         return "Documentation and generated reference area. Used to explain the app and source
structure."
2014:     if repo_file.startswith("Backgrounds/"):
2015:         return "Portfolio background asset folder placeholder."
2016:     return "Repository root. Usually an entry file, app config, public website file, package manifest, or
top-level documentation."
2017:
2018:

```

file_category (docs/build_complete_guide.py, lines 2019-2043)

file_category named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2019-2043 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references lower, endswith, and startswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 11 return statement(s). The most important visible returns are: line 2022: return "Folder marker"; line 2024: return "GitHub workflow"; line 2026: return "Image/icon asset"; line 2028: return "Generated document". There are 7 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2019: def file_category(repo_file: str, path: Path) -> str:
2020:     suffix = path.suffix.lower()
2021:     if repo_file.endswith(".gitkeep"):
2022:         return "Folder marker"
2023:     if repo_file.startswith(".github/workflows/"):
2024:         return "GitHub workflow"
2025:     if suffix in {".png", ".ico", ".jpg", ".jpeg", ".webp", ".svg"}:
2026:         return "Image/icon asset"
2027:     if suffix in {".docx", ".pdf"}:
2028:         return "Generated document"
2029:     if suffix in {".js", ".mjs", ".cjs"}:
2030:         return "JavaScript source"
2031:     if suffix in {".html", ".css"}:
2032:         return "Website/builder UI source"
2033:     if suffix in {".json", ".toml", ".yaml", ".yml"}:
2034:         return "Configuration or structured data"
2035:     if suffix in {".md", ".txt", ""}:
2036:         return "Documentation or text control file"
2037:     if suffix == ".nsh":
2038:         return "NSIS installer script"
2039:     if suffix == ".py":
2040:         return "Python tooling"
2041:     return "Repository file"
2042:
2043:

```

image_metadata (docs/build_complete_guide.py, lines 2044-2059)

image_metadata named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2044-2059 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references exists, lower, open, lstrip, upper, getattr, and str. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 2046: return {}; line 2049: return {}; line 2057: return {"error": str(exc)}.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2044: def image_metadata(path: Path) -> dict:
2045:     if not path.exists() or path.suffix.lower() not in {".png", ".jpg", ".jpeg", ".webp", ".ico"}:
2046:         return {}
2047:     try:

```

```

2048:         with Image.open(path) as image:
2049:             return {
2050:                 "format": image.format or path.suffix.lower().lstrip(".").upper(),
2051:                 "width": image.size[0],
2052:                 "height": image.size[1],
2053:                 "mode": image.mode,
2054:                 "frames": getattr(image, "n_frames", 1),
2055:             }
2056:     except Exception as exc:
2057:         return {"error": str(exc)}
2058:
2059:

```

file_specific_notes (docs/build_complete_guide.py, lines 2060-2088)

file_specific_notes named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2060-2088 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, startswith, and endswith. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2086: return notes.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2060: def file_specific_notes(repo_file: str, category: str) -> list[str]:
2061:     notes = []
2062:     if repo_file in FILE_DESCRIPTIONS:
2063:         notes.append(FILE_DESCRIPTIONS[repo_file])
2064:     if repo_file.startswith("assets/favicon"):
2065:         notes.append("This favicon participates in browser tab identity, mobile install surfaces, and
public website branding.")
2066:     if repo_file.startswith("assets/omb-app-icon"):
2067:         notes.append("This app icon is used by the Windows desktop application, installer metadata, and
shortcuts.")
2068:     if repo_file == "assets/omb-mark.png":
2069:         notes.append("This is the OMB visual mark used by the website and builder branding.")
2070:     if repo_file == "_headers":
2071:         notes.append("This file describes security and caching headers for hosts that support static
`_headers` files, such as Cloudflare Pages style deployments.")
2072:     if repo_file == ".nojekyll":
2073:         notes.append("This makes GitHub Pages serve static files directly without Jekyll filtering.")
2074:     if repo_file == ".gitattributes":
2075:         notes.append("This protects binary artifacts such as DOCX, PDF, PNG, ICO, and installers from
line-ending conversion.")
2076:     if repo_file == ".gitignore":
2077:         notes.append("This keeps temporary build output, local-only data, and generated noise out of
version control.")
2078:     if repo_file.endswith(".gitkeep"):
2079:         notes.append("This empty marker exists so Git keeps an otherwise empty folder.")
2080:     if category == "Image/icon asset":
2081:         notes.append("Do not edit binary assets with a text editor. Replace them with properly exported
image/icon files.")
2082:     if category == "GitHub workflow":
2083:         notes.append("Workflow changes should be tested on a feature branch because mistakes can block
merges or releases.")
2084:     if not notes:
2085:         notes.append("Tracked repository file used by the builder, website, installer, documentation, or
release process.")
2086:     return notes
2087:
2088:

```

extract_file_facts (docs/build_complete_guide.py, lines 2089-2118)

extract_file_facts named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2089-2118 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references lower, file_category, is_text_code_file, source_lines, exists, is_file, guess_type, str, stat, folder_role, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2116: return facts.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2089: def extract_file_facts(repo_file: str) -> dict:
2090:     path = REPO / repo_file
2091:     suffix = path.suffix.lower()
2092:     category = file_category(repo_file, path)
2093:     text_file = is_text_code_file(repo_file) or suffix in {".gitignore", ".gitattributes", ".nojekyll",
    "", ".txt", ".md"}
2094:     lines = source_lines(repo_file) if text_file and path.exists() and path.is_file() else []
2095:     mime_type, _encoding = mimetypes.guess_type(str(path))
2096:     facts = {
2097:         "repo_file": repo_file,
2098:         "path": path,
2099:         "exists": path.exists(),
2100:         "size_bytes": path.stat().st_size if path.exists() and path.is_file() else 0,
2101:         "suffix": suffix or "(none)",
2102:         "category": category,
2103:         "mime_type": mime_type or "unknown",
2104:         "folder_role": folder_role(repo_file),
2105:         "description": FILE_DESCRIPTIONS.get(repo_file, "Tracked repository file used by the builder or
public website."),
2106:         "notes": file_specific_notes(repo_file, category),
2107:         "is_text": bool(lines or text_file),
2108:         "line_count": len(lines),
2109:         "snippet": line_excerpt(lines, 1, min(72, len(lines)), max_lines=72) if lines else "",
2110:         "image": image_metadata(path),
2111:     }
2112:     if is_text_code_file(repo_file):
2113:         facts["source"] = extract_source_facts(repo_file)
2114:     else:
2115:         facts["source"] = None
2116:     return facts
2117:
2118:
```

add_file_fact_summary (docs/build_complete_guide.py, lines 2119-2137)

add_file_fact_summary named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2119-2137 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, and add_table. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2119: def add_file_fact_summary(doc: Document, facts: dict) -> None:
2120:     rows = [
2121:         ("File", facts["repo_file"]),
2122:         ("Category", facts["category"]),
2123:         ("Folder role", facts["folder_role"]),
2124:         ("Size", f"{facts['size_bytes']:,} bytes"),
2125:         ("Extension", facts["suffix"]),
2126:         ("MIME type", facts["mime_type"]),
2127:         ("Text lines", f"{facts['line_count']:,}" if facts["is_text"] else "Not a readable text file"),
2128:     ]
2129:     if facts["image"]:
2130:         image = facts["image"]
2131:         if "error" in image:
2132:             rows.append(("Image metadata", f"Could not inspect image: {image['error']}"))
2133:         else:
2134:             rows.append(("Image metadata", f"{image['format']} {image['width']}x{image['height']}, mode
{image['mode']}, frames {image['frames']}"))
2135:     add_table(doc, rows)
2136:
2137:
```

add_file_fact_sections (docs/build_complete_guide.py, lines 2138-2163)

add_file_fact_sections named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2138-2163 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `add_heading`, `add_paragraph`, `add_source_fact_sections`, `add_code`, and `numbers`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2138: def add_file_fact_sections(doc: Document, facts: dict) -> None:
2139:     doc.add_heading("Why this file exists", level=2)
2140:     for note in facts["notes"]:
2141:         doc.add_paragraph(note, style="List Bullet")
2142:     doc.add_heading("How it is used", level=2)
2143:     doc.add_paragraph(facts["folder_role"])
2144:     if facts["source"]:
2145:         add_source_fact_sections(doc, facts["source"], include_excerpts=True)
2146:     elif facts["snippet"]:
2147:         doc.add_heading("Readable content snippet", level=2)
2148:         add_code(doc, facts["snippet"])
2149:     elif facts["image"]:
2150:         doc.add_heading("Asset handling note", level=2)
2151:         doc.add_paragraph("This file is documented by metadata instead of raw bytes. Binary image/icon
bytes are not useful to print in a Word/PDF reference. Use an image editor or icon tool when changing it.")
2152:     else:
2153:         doc.add_heading("Binary or marker note", level=2)
2154:         doc.add_paragraph("This file is either binary, empty, or a marker/config file whose value comes
from its presence and path rather than readable content.")
2155:         doc.add_heading("Safe change checklist", level=2)
2156:         numbers(doc, [
2157:             "Confirm which app, website, installer, workflow, or document consumes this file.",
2158:             "Change it on a feature branch from development.",
2159:             "Regenerate docs if the file purpose, API surface, or behavior changes.",
2160:             "Run the smallest relevant smoke test before merging into development.",
2161:         ])
2162:
2163:
```

code_reference_name (docs/build_complete_guide.py, lines 2164-2167)

`code_reference_name` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2164-2167 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `sub`, and `strip`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2165: `return re.sub(r"^[A-Za-z0-9_-]+", "__", repo_file).strip("__") + suffix`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2164: def code_reference_name(repo_file: str, suffix: str) -> str:
2165:     return re.sub(r"^[A-Za-z0-9_-]+", "__", repo_file).strip("__") + suffix
2166:
2167:
```

remove_directory (docs/build_complete_guide.py, lines 2168-2181)

`remove_directory` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2168-2181 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `exists`, `remove_readonly`, `Path`, `chmod`, and `rmtree`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2170: `return`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2168: def remove_directory(path: Path) -> None:
2169:     if not path.exists():
2170:         return
```

```

2171:
2172:     def remove_readonly(function, item_path, excinfo):
2173:         try:
2174:             Path(item_path).chmod(0o700)
2175:             function(item_path)
2176:         except Exception:
2177:             raise excinfo
2178:
2179:     shutil.rmtree(path, onexc=remove_readonly)
2180:
2181:

```

remove_readonly (docs/build_complete_guide.py, lines 2172-2181)

remove_readonly named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2172-2181 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Path, chmod, and rmtree. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2172:     def remove_readonly(function, item_path, excinfo):
2173:         try:
2174:             Path(item_path).chmod(0o700)
2175:             function(item_path)
2176:         except Exception:
2177:             raise excinfo
2178:
2179:     shutil.rmtree(path, onexc=remove_readonly)
2180:
2181:

```

add_source_fact_summary (docs/build_complete_guide.py, lines 2182-2195)

add_source_fact_summary named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2182-2195 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_table, fact_relationship_summary, str, and len. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2182: def add_source_fact_summary(doc: Document, facts: dict) -> None:
2183:     add_table(doc, [
2184:         ("File", facts["repo_file"]),
2185:         ("Purpose", facts["description"]),
2186:         ("Lines", f"{facts['line_count']:,}"),
2187:         ("Size", f"{facts['size_bytes']:,} bytes"),
2188:         ("Talks to", fact_relationship_summary(facts)),
2189:         ("Functions", str(len(facts["functions"]))),
2190:         ("Variables/constants", str(len(facts["variables"]))),
2191:         ("API mentions", str(len(facts["endpoints"]))),
2192:         ("Signals", str(len(facts["signals"]))),
2193:     ])
2194:
2195:

```

add_source_fact_sections (docs/build_complete_guide.py, lines 2196-2238)

add_source_fact_sections named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2196-2238 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_table, endpoint_purpose, add_function_explanation_docx, add_code, and numbers. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2196: def add_source_fact_sections(doc: Document, facts: dict, include_excerpts: bool = True) -> None:
2197:     doc.add_heading("How this file participates in the system", level=2)
2198:     doc.add_paragraph("This generated section reads the actual file and points out how it communicates
with other pieces of the app. It is intentionally concrete: line numbers, declarations, endpoints, events,
source excerpts, and debugging cues.")
2199:     if facts["imports"]:
2200:         doc.add_heading("Imports, requires, and external dependencies", level=2)
2201:         add_table(doc, [(f"Line {item['line']}", item["statement"]) for item in facts["imports"][:60]])
2202:     if facts["routes"]:
2203:         doc.add_heading("Backend routes implemented here", level=2)
2204:         add_table(doc, [(f"{item['method']} {item['path']}", f"Line {item['line']}").
{endpoint_purpose(item['path'])} for item in facts["routes"][:80]])
2205:     if facts["endpoints"]:
2206:         doc.add_heading("API endpoints mentioned or called", level=2)
2207:         add_table(doc, [(item["endpoint"], f"Line {item['line']}. {endpoint_purpose(item['endpoint'])}"))
for item in facts["endpoints"][:80]])
2208:     if facts["signals"]:
2209:         doc.add_heading("Important implementation signals", level=2)
2210:         add_table(doc, [(f"{item['type']} at line {item['line']}", f"{item['meaning']} Evidence:
{item['evidence']}") for item in facts["signals"][:80]])
2211:     if facts["variables"]:
2212:         doc.add_heading("Important constants and variables", level=2)
2213:         add_table(doc, [(item["name"], f"Line {item['line']}: {item['statement']}") for item in
facts["variables"][:80]])
2214:     if facts["json_keys"]:
2215:         doc.add_heading("JSON/YAML-style keys detected", level=2)
2216:         add_table(doc, [(item["key"], f"Line {item['line']}: {item['statement']}") for item in
facts["json_keys"][:80]])
2217:     if facts["events"]:
2218:         doc.add_heading("Event handlers and user interactions", level=2)
2219:         add_table(doc, [(f"Line {item['line']}", item["statement"]) for item in facts["events"][:80]])
2220:     if facts["selectors"]:
2221:         doc.add_heading("CSS selectors and visual hooks", level=2)
2222:         add_table(doc, [(f"Line {item['line']}", item["selector"]) for item in facts["selectors"][:120]])
2223:     if facts["functions"]:
2224:         doc.add_heading("Function-by-function detail", level=2)
2225:         for item in facts["functions"]:
2226:             add_function_explanation_docx(doc, facts["repo_file"], item,
include_excerpt=include_excerpts, heading_level=3)
2227:         doc.add_heading("Representative opening snippet", level=2)
2228:         add_code(doc, facts["snippet"] or "(empty file)")
2229:         doc.add_heading("Beginner debugging checklist for this file", level=2)
2230:         numbers(doc, [
2231:             "Name the user action, command, or workflow that reaches this file.",
2232:             "Find the exact function or route that owns the behavior.",
2233:             "Check the inputs: DOM state, request body, file path, local storage value, compiler source, or
Git branch.",
2234:             "Check the outputs: returned JSON, updated DOM, written file, terminal output, generated project
catalog, or published website asset.",
2235:             "If the issue crosses a boundary, test the boundary directly: browser console for frontend,
endpoint call for backend, command line for Git/compiler, Cloudflare logs for Worker behavior.",
2236:         ])
2237:
2238:
```

setup_code_reference_document (docs/build_complete_guide.py, lines 2239-2252)

setup_code_reference_document named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2239-2252 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_document, add_paragraph, add_run, Pt, and RGBColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2250: return doc.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2239: def setup_code_reference_document(title: str, subtitle: str) -> Document:
2240:     doc = setup_document()
2241:     title_p = doc.add_paragraph()
2242:     title_p.alignment = WD_ALIGN_PARAGRAPH.LEFT
2243:     run = title_p.add_run(title)
2244:     run.bold = True
2245:     run.font.size = Pt(22)
2246:     run.font.color.rgb = RGBColor(11, 37, 69)
2247:     doc.add_paragraph(subtitle)
2248:     doc.add_paragraph(f"Generated from repository state at {REPO}.")
2249:     doc.add_paragraph("")
2250:     return doc
2251:
2252:

```

compact_generated_docx (docs/build_complete_guide.py, lines 2253-2273)

compact_generated_docx named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2253-2273 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references list, any, qn, get, iter, getparent, and remove. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2271: return removed.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2253: def compact_generated_docx(doc: Document, keep_first_page_break: bool = True) -> int:
2254:     """Remove generator-inserted page breaks so pages fill naturally."""
2255:     kept = 0
2256:     removed = 0
2257:     for paragraph in list(doc.paragraphs):
2258:         has_page_break = any(
2259:             element.tag == qn("w:br") and element.get(qn("w:type")) == "page"
2260:             for element in paragraph._element.iter()
2261:         )
2262:         if not has_page_break:
2263:             continue
2264:         if keep_first_page_break and kept == 0:
2265:             kept += 1
2266:             continue
2267:         parent = paragraph._element.getparent()
2268:         if parent is not None:
2269:             parent.remove(paragraph._element)
2270:             removed += 1
2271:     return removed
2272:
2273:

```

write_single_code_reference_docx (docs/build_complete_guide.py, lines 2274-2284)

write_single_code_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2274-2284 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_code_reference_document, add_source_fact_summary, add_source_fact_sections, compact_generated_docx, and save. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2274: def write_single_code_reference_docx(facts: dict, output_path: Path) -> None:
2275:     doc = setup_code_reference_document(
2276:         f"Code Reference: {facts['repo_file']}",
2277:         "A source-level explanation with function details, file communication, variables, endpoints, and
debugging notes.",
2278:     )
2279:     add_source_fact_summary(doc, facts)
2280:     add_source_fact_sections(doc, facts, include_excerpts=True)
2281:     compact_generated_docx(doc, keep_first_page_break=False)
2282:     doc.save(output_path)

```


2283:
2284:

pdf_styles (docs/build_complete_guide.py, lines 2285-2314)

pdf_styles named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2285-2314 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references getSampleStyleSheet, add, ParagraphStyle, and HexColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2312: return styles.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2285: def pdf_styles():
2286:     styles = getSampleStyleSheet()
2287:     styles.add(ParagraphStyle(
2288:         name="CodeTiny",
2289:         parent=styles["Code"],
2290:         fontName="Courier",
2291:         fontSize=6.4,
2292:         leading=7.6,
2293:         backColor=colors.HexColor("#F5F8FB"),
2294:         borderColor=colors.HexColor("#D8E6F0"),
2295:         borderWidth=0.25,
2296:         borderPadding=4,
2297:         spaceAfter=7,
2298:     ))
2299:     styles.add(ParagraphStyle(
2300:         name="SmallBody",
2301:         parent=styles["BodyText"],
2302:         fontSize=8.5,
2303:         leading=11.0,
2304:         spaceAfter=5,
2305:     ))
2306:     styles.add(ParagraphStyle(
2307:         name="TinyTable",
2308:         parent=styles["BodyText"],
2309:         fontSize=7.0,
2310:         leading=8.5,
2311:     ))
2312:     return styles
2313:
2314:
```

pdf_paragraph (docs/build_complete_guide.py, lines 2315-2318)

pdf_paragraph named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2315-2318 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references Paragraph, escape, str, and replace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2316: return Paragraph(html.escape(str(text)).replace("\n", "
"), style).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2315: def pdf_paragraph(text: str, style) -> Paragraph:
2316:     return Paragraph(html.escape(str(text)).replace("\n", "<br/>"), style)
2317:
2318:
```

pdf_code (docs/build_complete_guide.py, lines 2319-2327)

pdf_code named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2319-2327 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references in, splitlines, replace, wrap, extend, Preformatted, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2325: return Preformatted("\n".join(wrapped_lines), styles["CodeTiny"])).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2319: def pdf_code(text: str, styles) -> Preformatted:
2320:     wrapped_lines = []
2321:     for raw_line in (text or "").splitlines():
2322:         expanded = raw_line.replace("\t", " ")
2323:         chunks = textwrap.wrap(expanded, width=112, replace_whitespace=False, drop_whitespace=False) or
[""]
2324:         wrapped_lines.extend(chunks)
2325:     return Preformatted("\n".join(wrapped_lines), styles["CodeTiny"])
2326:
2327:
```

pdf_table (docs/build_complete_guide.py, lines 2328-2343)

pdf_table named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2328-2343 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_paragraph, extend, Table, setStyle, TableStyle, and HexColor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2341: return table.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2328: def pdf_table(rows: list[tuple[str, str]], styles):
2329:     data = [[pdf_paragraph("Item", styles["TinyTable"]), pdf_paragraph("Explanation",
styles["TinyTable"])]
2330:     data.extend([[pdf_paragraph(label, styles["TinyTable"]), pdf_paragraph(value, styles["TinyTable"])]
for label, value in rows])
2331:     table = Table(data, colWidths=[1.45 * inch, 5.25 * inch], repeatRows=1)
2332:     table.setStyle(TableStyle([
2333:         ("BACKGROUND", (0, 0), (-1, 0), colors.HexColor("#E8EEF5")),
2334:         ("GRID", (0, 0), (-1, -1), 0.25, colors.HexColor("#B7CFE0")),
2335:         ("VALIGN", (0, 0), (-1, -1), "TOP"),
2336:         ("LEFTPADDING", (0, 0), (-1, -1), 5),
2337:         ("RIGHTPADDING", (0, 0), (-1, -1), 5),
2338:         ("TOPPADDING", (0, 0), (-1, -1), 4),
2339:         ("BOTTPADDING", (0, 0), (-1, -1), 4),
2340:     ]))
2341:     return table
2342:
2343:
```

add_pdf_source_fact_sections (docs/build_complete_guide.py, lines 2344-2378)

add_pdf_source_fact_sections named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2344-2378 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, Paragraph, pdf_paragraph, pdf_table, endpoint_purpose, add_function_explanation_pdf, and pdf_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2344: def add_pdf_source_fact_sections(story: list, facts: dict, styles, include_excerpts: bool = True) ->
None:
2345:     story.append(Paragraph("How this file participates in the system", styles["Heading2"]))
2346:     story.append(pdf_paragraph("This generated section reads the actual file and points out line numbers,
declarations, endpoints, events, source excerpts, and debugging cues.", styles["SmallBody"]))
```

```

2347:     if facts["imports"]:
2348:         story.append(Paragraph("Imports, requires, and external dependencies", styles["Heading2"]))
2349:         story.append(pdf_table([(f"Line {item['line']}", item["statement"]) for item in
facts["imports"][:60]], styles))
2350:     if facts["routes"]:
2351:         story.append(Paragraph("Backend routes implemented here", styles["Heading2"]))
2352:         story.append(pdf_table([(f"{item['method']} {item['path']}", f"Line {item['line']}".
{endpoint_purpose(item['path'])}) for item in facts["routes"][:80]], styles))
2353:     if facts["endpoints"]:
2354:         story.append(Paragraph("API endpoints mentioned or called", styles["Heading2"]))
2355:         story.append(pdf_table([(item["endpoint"], f"Line {item['line']}".
{endpoint_purpose(item['endpoint'])}) for item in facts["endpoints"][:80]], styles))
2356:     if facts["signals"]:
2357:         story.append(Paragraph("Important implementation signals", styles["Heading2"]))
2358:         story.append(pdf_table([(f"{item['type']} at line {item['line']}", f"{item['meaning']} Evidence:
{item['evidence']}") for item in facts["signals"][:80]], styles))
2359:     if facts["variables"]:
2360:         story.append(Paragraph("Important constants and variables", styles["Heading2"]))
2361:         story.append(pdf_table([(item["name"], f"Line {item['line']}: {item['statement']}") for item in
facts["variables"][:80]], styles))
2362:     if facts["json_keys"]:
2363:         story.append(Paragraph("JSON/YAML-style keys detected", styles["Heading2"]))
2364:         story.append(pdf_table([(item["key"], f"Line {item['line']}: {item['statement']}") for item in
facts["json_keys"][:80]], styles))
2365:     if facts["events"]:
2366:         story.append(Paragraph("Event handlers and user interactions", styles["Heading2"]))
2367:         story.append(pdf_table([(f"Line {item['line']}", item["statement"]) for item in
facts["events"][:80]], styles))
2368:     if facts["selectors"]:
2369:         story.append(Paragraph("CSS selectors and visual hooks", styles["Heading2"]))
2370:         story.append(pdf_table([(f"Line {item['line']}", item["selector"]) for item in
facts["selectors"][:120]], styles))
2371:     if facts["functions"]:
2372:         story.append(Paragraph("Function-by-function detail", styles["Heading2"]))
2373:         for item in facts["functions"]:
2374:             add_function_explanation_pdf(story, facts["repo_file"], item, styles,
include_excerpt=include_excerpts)
2375:         story.append(Paragraph("Representative opening snippet", styles["Heading2"]))
2376:         story.append(pdf_code(facts["snippet"] or "(empty file)", styles))
2377:
2378:

```

write_single_code_reference_pdf (docs/build_complete_guide.py, lines 2379-2402)

write_single_code_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2379-2402 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_styles, Paragraph, pdf_paragraph, pdf_table, fact_relationship_summary, str, len, Spacer, add_pdf_source_fact_sections, SimpleDocTemplate, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2381: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2379: def write_single_code_reference_pdf(facts: dict, output_path: Path) -> None:
2380:     if not REPORTLAB_AVAILABLE:
2381:         return
2382:     styles = pdf_styles()
2383:     story = [
2384:         Paragraph(f"Code Reference: {facts['repo_file']}", styles["Title"]),
2385:         pdf_paragraph("A source-level explanation with function details, file communication, variables,
endpoints, and debugging notes.", styles["SmallBody"]),
2386:         pdf_table([
2387:             ("File", facts["repo_file"]),
2388:             ("Purpose", facts["description"]),
2389:             ("Lines", f"{facts['line_count']:,}"),
2390:             ("Size", f"{facts['size_bytes']:,} bytes"),
2391:             ("Talks to", fact_relationship_summary(facts)),
2392:             ("Functions", str(len(facts["functions"]))),
2393:             ("Variables/constants", str(len(facts["variables"]))),
2394:             ("API mentions", str(len(facts["endpoints"]))),
2395:             ("Signals", str(len(facts["signals"]))),
2396:         ], styles),
2397:         Spacer(1, 0.12 * inch),
2398:     ]
2399:     add_pdf_source_fact_sections(story, facts, styles, include_excerpts=True)
2400:     SimpleDocTemplate(str(output_path), pagesize=LETTER, rightMargin=0.55 * inch, leftMargin=0.55 * inch,
topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)
2401:
2402:

```

write_master_code_reference_docx (docs/build_complete_guide.py, lines 2403-2431)

write_master_code_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2403-2431 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_code_reference_document, add_diagram, add_heading, numbers, add_table, len, signal, add_page_break, add_source_fact_summary, add_source_fact_sections, and 2 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2403: def write_master_code_reference_docx(code_files: list[str], facts_by_file: dict[str, dict], diagrams:
dict[str, Path]) -> None:
2404:     doc = setup_code_reference_document(
2405:         "OMB Portfolio Builder Code Reference",
2406:         "A generated Word reference that explains the important source files, all discovered functions,
communication paths, variables, endpoints, installers, AI paths, and compiler paths.",
2407:     )
2408:     add_diagram(doc, diagrams["file-communication-map"], "Main source file communication map.")
2409:     add_diagram(doc, diagrams["frontend-backend-cloudflare"], "Frontend, backend, Cloudflare, and AI
boundary.")
2410:     add_diagram(doc, diagrams["compile-code-flow"], "Compile Code workspace and simulator flow.")
2411:     doc.add_heading("How to use this document", level=1)
2412:     numbers(doc, [
2413:         "Start with the file you want to understand.",
2414:         "Read the summary table to understand its role.",
2415:         "Read implementation signals to see whether the file touches Git, files, compilers, Cloudflare,
authentication, DOM, storage, or network calls.",
2416:         "Read function details to understand line ranges, side effects, calls, endpoints, and source
excerpts.",
2417:         "Use the debugging checklist to trace a behavior from click to function to file/API/output.",
2418:     ])
2419:     doc.add_heading("Generated file index", level=1)
2420:     add_table(doc, [(repo_file, f"{facts_by_file[repo_file]['line_count']},} lines,
{len(facts_by_file[repo_file]['functions'])} function(s), {len(facts_by_file[repo_file]['signals'])}
signal(s).") for repo_file in code_files])
2421:     doc.add_page_break()
2422:     for repo_file in code_files:
2423:         facts = facts_by_file[repo_file]
2424:         doc.add_heading(f"Source File: {repo_file}", level=1)
2425:         add_source_fact_summary(doc, facts)
2426:         add_source_fact_sections(doc, facts, include_excerpts=True)
2427:         doc.add_page_break()
2428:     compact_generated_docx(doc, keep_first_page_break=True)
2429:     doc.save(CODE_REFERENCE_MASTER_DOCX)
2430:
2431:
```

write_master_code_reference_pdf (docs/build_complete_guide.py, lines 2432-2470)

write_master_code_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2432-2470 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_styles, Paragraph, pdf_paragraph, get, exists, append, PdfImage, str, pdf_table, len, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2434: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2432: def write_master_code_reference_pdf(code_files: list[str], facts_by_file: dict[str, dict], diagrams:
dict[str, Path]) -> None:
2433:     if not REPORTLAB_AVAILABLE:
2434:         return
2435:     styles = pdf_styles()
2436:     story = [
2437:         Paragraph("OMB Portfolio Builder Code Reference", styles["Title"]),
2438:         pdf_paragraph("A generated PDF reference that explains the important source files, discovered
functions, communication paths, variables, endpoints, installers, AI paths, and compiler paths.",
styles["SmallBody"]),
2439:     ]
```

```

2440:     for diagram_name, caption in [
2441:         ("file-communication-map", "Main source file communication map."),
2442:         ("frontend-backend-cloudflare", "Frontend, backend, Cloudflare, and AI boundary."),
2443:         ("compile-code-flow", "Compile Code workspace and simulator flow."),
2444:     ]:
2445:         path = diagrams.get(diagram_name)
2446:         if path and path.exists():
2447:             story.append(PdfImage(str(path), width=6.6 * inch, height=3.72 * inch))
2448:             story.append(pdf_paragraph(caption, styles["SmallBody"]))
2449:             story.append(Paragraph("Generated file index", styles["Heading1"]))
2450:             story.append(pdf_table([(repo_file, f"{facts_by_file[repo_file]['line_count']},",) lines,
{len(facts_by_file[repo_file]['functions'])} function(s), {len(facts_by_file[repo_file]['signals'])}
signal(s).") for repo_file in code_files], styles))
2451:             story.append(PageBreak())
2452:             for repo_file in code_files:
2453:                 facts = facts_by_file[repo_file]
2454:                 story.append(Paragraph(f"Source File: {repo_file}", styles["Heading1"]))
2455:                 story.append(pdf_table([
2456:                     ("File", facts["repo_file"]),
2457:                     ("Purpose", facts["description"]),
2458:                     ("Lines", f"{facts['line_count']},"),
2459:                     ("Size", f"{facts['size_bytes']}, bytes"),
2460:                     ("Talks to", fact_relationship_summary(facts)),
2461:                     ("Functions", str(len(facts["functions"]))),
2462:                     ("Variables/constants", str(len(facts["variables"]))),
2463:                     ("API mentions", str(len(facts["endpoints"]))),
2464:                     ("Signals", str(len(facts["signals"]))),
2465:                 ], styles))
2466:                 add_pdf_source_fact_sections(story, facts, styles, include_excerpts=True)
2467:                 story.append(PageBreak())
2468:             SimpleDocTemplate(str(CODE_REFERENCE_MASTER_PDF), pagesize=LETTER, rightMargin=0.55 * inch,
leftMargin=0.55 * inch, topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)
2469:
2470:

```

write_single_file_reference_docx (docs/build_complete_guide.py, lines 2471-2475)

write_single_file_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2471-2475 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2471: def write_single_file_reference_docx(
2472:     facts: dict,
2473:     output_path: Path,
2474:     title_prefix: str = "File Reference",
2475:     subtitle: str = "A file-specific explanation covering purpose, owner layer, metadata, source details,
implementation signals, source excerpts, and safe-change rules.",

```

add_pdf_file_fact_sections (docs/build_complete_guide.py, lines 2487-2513)

add_pdf_file_fact_sections named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2487-2513 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references append, Paragraph, pdf_paragraph, add_pdf_source_fact_sections, and pdf_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2487: def add_pdf_file_fact_sections(story: list, facts: dict, styles) -> None:
2488:     story.append(Paragraph("Why this file exists", styles["Heading2"]))
2489:     for note in facts["notes"]:
2490:         story.append(pdf_paragraph(f"- {note}", styles["SmallBody"]))
2491:     story.append(Paragraph("How it is used", styles["Heading2"]))
2492:     story.append(pdf_paragraph(facts["folder_role"], styles["SmallBody"]))
2493:     if facts["source"]:

```

```

2494:         add_pdf_source_fact_sections(story, facts["source"], styles, include_excerpts=True)
2495:     elif facts["snippet"]:
2496:         story.append(Paragraph("Readable content snippet", styles["Heading2"]))
2497:         story.append(pdf_code(facts["snippet"], styles))
2498:     elif facts["image"]:
2499:         story.append(Paragraph("Asset handling note", styles["Heading2"]))
2500:         story.append(pdf_paragraph("This file is documented by metadata instead of raw bytes. Binary
image/icon bytes are not useful to print in a reference document.", styles["SmallBody"]))
2501:     else:
2502:         story.append(Paragraph("Binary or marker note", styles["Heading2"]))
2503:         story.append(pdf_paragraph("This file is either binary, empty, or a marker/config file whose
value comes from its presence and path rather than readable content.", styles["SmallBody"]))
2504:         story.append(Paragraph("Safe change checklist", styles["Heading2"]))
2505:         for item in [
2506:             "Confirm which app, website, installer, workflow, or document consumes this file.",
2507:             "Change it on a feature branch from development.",
2508:             "Regenerate docs if the file purpose, API surface, or behavior changes.",
2509:             "Run the smallest relevant smoke test before merging into development.",
2510:         ]:
2511:             story.append(pdf_paragraph(f"- {item}", styles["SmallBody"]))
2512:
2513:

```

write_single_file_reference_pdf (docs/build_complete_guide.py, lines 2514-2518)

write_single_file_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2514-2518 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references no major named helper calls detected. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2514: def write_single_file_reference_pdf(
2515:     facts: dict,
2516:     output_path: Path,
2517:     title_prefix: str = "File Reference",
2518:     subtitle: str = "A file-specific explanation covering purpose, owner layer, metadata, source details,
implementation signals, source excerpts, and safe-change rules.",

```

write_master_file_reference_docx (docs/build_complete_guide.py, lines 2548-2572)

write_master_file_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2548-2572 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_code_reference_document, add_diagram, Counter, add_heading, add_table, str, len, join, sorted, items, and 5 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2548: def write_master_file_reference_docx(file_facts: list[dict], diagrams: dict[str, Path]) -> None:
2549:     doc = setup_code_reference_document(
2550:         "OMB Portfolio Builder Important Code Reference",
2551:         "A generated Word reference for the code and control files that drive the builder, website,
installer, Cloudflare AI worker, workflows, parser, compile workspace, and generated documentation.",
2552:     )
2553:     add_diagram(doc, diagrams["file-communication-map"], "Main file communication map for the builder and
public website.")
2554:     category_counts = Counter(facts["category"] for facts in file_facts)
2555:     doc.add_heading("Coverage summary", level=1)
2556:     add_table(doc, [
2557:         ("Important files documented", str(len(file_facts))),
2558:         ("Selection rule", "Only behavior-driving code/control files are included. Favicons, binary
assets, and passive marker files are intentionally skipped here."),
2559:         ("Categories", " ".join(f"{name}: {count}" for name, count in sorted(category_counts.items()))),
2560:     ])
2561:     doc.add_heading("File index", level=1)
2562:     add_table(doc, [(facts["repo_file"], f"{facts['category']}. {facts['folder_role']}") for facts in
file_facts])

```

```

2563:     doc.add_page_break()
2564:     for facts in file_facts:
2565:         doc.add_heading(f"Important Code Reference: {facts['repo_file']}", level=1)
2566:         add_file_fact_summary(doc, facts)
2567:         add_file_fact_sections(doc, facts)
2568:         doc.add_page_break()
2569:     compact_generated_docx(doc, keep_first_page_break=True)
2570:     doc.save(FILE_REFERENCE_MASTER_DOCX)
2571:
2572:

```

write_master_file_reference_pdf (docs/build_complete_guide.py, lines 2573-2613)

write_master_file_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2573-2613 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_styles, Counter, Paragraph, pdf_paragraph, get, exists, append, PdfImage, str, pdf_table, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2575: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2573: def write_master_file_reference_pdf(file_facts: list[dict], diagrams: dict[str, Path]) -> None:
2574:     if not REPORTLAB_AVAILABLE:
2575:         return
2576:     styles = pdf_styles()
2577:     category_counts = Counter(facts["category"] for facts in file_facts)
2578:     story = [
2579:         Paragraph("OMB Portfolio Builder Important Code Reference", styles["Title"]),
2580:         pdf_paragraph("A generated PDF reference for the code and control files that drive the builder,
website, installer, Cloudflare AI worker, workflows, parser, compile workspace, and generated documentation.",
styles["SmallBody"]),
2581:     ]
2582:     file_map = diagrams.get("file-communication-map")
2583:     if file_map and file_map.exists():
2584:         story.append(PdfImage(str(file_map), width=6.6 * inch, height=3.72 * inch))
2585:         story.append(Paragraph("Coverage summary", styles["Heading1"]))
2586:         story.append(pdf_table([
2587:             ("Important files documented", str(len(file_facts))),
2588:             ("Selection rule", "Only behavior-driving code/control files are included. Favicons, binary
assets, and passive marker files are intentionally skipped here."),
2589:             ("Categories", ", ".join(f"{name}: {count}" for name, count in sorted(category_counts.items()))),
2590:         ], styles))
2591:         story.append(Paragraph("File index", styles["Heading1"]))
2592:         story.append(pdf_table([(facts["repo_file"], f"{facts['category']}. {facts['folder_role']}") for
facts in file_facts], styles))
2593:         story.append(PageBreak())
2594:         for facts in file_facts:
2595:             story.append(Paragraph(f"Important Code Reference: {facts['repo_file']}", styles["Heading1"]))
2596:             rows = [
2597:                 ("File", facts["repo_file"]),
2598:                 ("Category", facts["category"]),
2599:                 ("Folder role", facts["folder_role"]),
2600:                 ("Size", f"{facts['size_bytes']:,} bytes"),
2601:                 ("Extension", facts["suffix"]),
2602:                 ("MIME type", facts["mime_type"]),
2603:                 ("Text lines", f"{facts['line_count']:,}" if facts["is_text"] else "Not a readable text
file"),
2604:             ]
2605:             if facts["image"]:
2606:                 image = facts["image"]
2607:                 rows.append(("Image metadata", f"Could not inspect image: {image['error']}" if "error" in
image else f"{image['format']} {image['width']}x{image['height']}, mode {image['mode']}, frames
{image['frames']}"))
2608:             story.append(pdf_table(rows, styles))
2609:             add_pdf_file_fact_sections(story, facts, styles)
2610:             story.append(PageBreak())
2611:         SimpleDocTemplate(str(FILE_REFERENCE_MASTER_PDF), pagesize=LETTER, rightMargin=0.55 * inch,
leftMargin=0.55 * inch, topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)
2612:
2613:

```

write_master_repository_file_reference_docx (docs/build_complete_guide.py, lines 2614-2645)

write_master_repository_file_reference_docx persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2614-2645 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references setup_code_reference_document, add_diagram, Counter, add_heading, add_table, str, len, join, sorted, items, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2614: def write_master_repository_file_reference_docx(file_facts: list[dict], diagrams: dict[str, Path]) ->
None:
2615:     doc = setup_code_reference_document(
2616:         "OMB Portfolio Builder Repository File Reference",
2617:         "A generated Word reference with one explanation section for each primary tracked repository
file. Source files receive function-level detail; assets and marker files receive metadata, purpose, usage, and
safe-change notes.",
2618:     )
2619:     add_diagram(doc, diagrams["file-communication-map"], "Main file communication map for the builder and
public website.")
2620:     category_counts = Counter(facts["category"] for facts in file_facts)
2621:     doc.add_heading("Coverage summary", level=1)
2622:     add_table(doc, [
2623:         ("Primary files documented", str(len(file_facts))),
2624:         ("Selection rule", "All primary tracked repository files are included. Generated reference
documents, generated guide diagrams, and generated documentation folders are excluded so this reference does not
recursively document its own output."),
2625:         ("Categories", ", ".join(f"{name}: {count}" for name, count in sorted(category_counts.items()))),
2626:     ])
2627:     doc.add_heading("How to use this reference", level=1)
2628:     numbers(doc, [
2629:         "Open the specific file document when you want to understand one file in isolation.",
2630:         "Use this master reference when you want to scan all files from one document.",
2631:         "For source files, read the function-by-function detail to understand what the function does,
what it calls, what it returns, important local variables, and why it exists.",
2632:         "For assets, marker files, and static configuration, read the metadata and safe-change checklist
instead of looking for function explanations that do not exist.",
2633:     ])
2634:     doc.add_heading("File index", level=1)
2635:     add_table(doc, [(facts["repo_file"], f"{facts['category']}. {facts['folder_role']}") for facts in
file_facts])
2636:     doc.add_page_break()
2637:     for facts in file_facts:
2638:         doc.add_heading(f"File Reference: {facts['repo_file']}", level=1)
2639:         add_file_fact_summary(doc, facts)
2640:         add_file_fact_sections(doc, facts)
2641:         doc.add_page_break()
2642:     compact_generated_docx(doc, keep_first_page_break=True)
2643:     doc.save(ALL_FILE_REFERENCE_MASTER_DOCX)
2644:
2645:
```

write_master_repository_file_reference_pdf (docs/build_complete_guide.py, lines 2646-2686)

write_master_repository_file_reference_pdf persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2646-2686 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pdf_styles, Counter, Paragraph, pdf_paragraph, get, exists, append, PdfImage, str, pdf_table, and 8 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2648: return.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2646: def write_master_repository_file_reference_pdf(file_facts: list[dict], diagrams: dict[str, Path]) ->
None:
2647:     if not REPORTLAB_AVAILABLE:
2648:         return
2649:     styles = pdf_styles()
2650:     category_counts = Counter(facts["category"] for facts in file_facts)
2651:     story = [
2652:         Paragraph("OMB Portfolio Builder Repository File Reference", styles["Title"]),
2653:         pdf_paragraph("A generated PDF reference with one explanation section for each primary tracked
repository file. Source files receive function-level detail; assets and marker files receive metadata, purpose,
```



```

usage, and safe-change notes.", styles["SmallBody"])),
2654:     ]
2655:     file_map = diagrams.get("file-communication-map")
2656:     if file_map and file_map.exists():
2657:         story.append(PdfImage(str(file_map), width=6.6 * inch, height=3.72 * inch))
2658:     story.append(Paragraph("Coverage summary", styles["Heading1"]))
2659:     story.append(pdf_table([
2660:         ("Primary files documented", str(len(file_facts))),
2661:         ("Selection rule", "All primary tracked repository files are included. Generated reference
outputs are excluded to avoid recursive documentation."),
2662:         ("Categories", ", ".join(f"{name}: {count}" for name, count in sorted(category_counts.items()))),
2663:     ], styles))
2664:     story.append(Paragraph("File index", styles["Heading1"]))
2665:     story.append(pdf_table([(facts["repo_file"], f"{facts['category']}. {facts['folder_role']}") for
facts in file_facts], styles))
2666:     story.append(PageBreak())
2667:     for facts in file_facts:
2668:         story.append(Paragraph(f"File Reference: {facts['repo_file']}", styles["Heading1"]))
2669:         rows = [
2670:             ("File", facts["repo_file"]),
2671:             ("Category", facts["category"]),
2672:             ("Folder role", facts["folder_role"]),
2673:             ("Size", f"{facts['size_bytes']:,} bytes"),
2674:             ("Extension", facts["suffix"]),
2675:             ("MIME type", facts["mime_type"]),
2676:             ("Text lines", f"{facts['line_count']:,}" if facts["is_text"] else "Not a readable text
file"),
2677:         ]
2678:         if facts["image"]:
2679:             image = facts["image"]
2680:             rows.append(("Image metadata", f"Could not inspect image: {image['error']}" if "error" in
image else f"{image['format']} {image['width']}x{image['height']}, mode {image['mode']}, frames
{image['frames']}"))
2681:         story.append(pdf_table(rows, styles))
2682:         add_pdf_file_fact_sections(story, facts, styles)
2683:         story.append(PageBreak())
2684:     SimpleDocTemplate(str(ALL_FILE_REFERENCE_MASTER_PDF), pagesize=LETTER, rightMargin=0.55 * inch,
leftMargin=0.55 * inch, topMargin=0.55 * inch, bottomMargin=0.55 * inch).build(story)
2685:
2686:

```

write_repository_file_reference_docs (docs/build_complete_guide.py, lines 2687-2709)

write_repository_file_reference_docs persists data to local state, disk, credentials, or browser storage. In this file, it appears around lines 2687-2709 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references remove_directory, mkdir, extract_file_facts, primary_tracked_files, code_reference_name, write_single_file_reference_docx, append, write_single_file_reference_pdf, exists, write_master_repository_file_reference_docx, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2707: return written.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2687: def write_repository_file_reference_docs(files: list[str], diagrams: dict[str, Path]) -> list[Path]:
2688:     remove_directory(ALL_FILE_REFERENCE_DOCX_DIR)
2689:     remove_directory(ALL_FILE_REFERENCE_PDF_DIR)
2690:     ALL_FILE_REFERENCE_DOCX_DIR.mkdir(parents=True, exist_ok=True)
2691:     ALL_FILE_REFERENCE_PDF_DIR.mkdir(parents=True, exist_ok=True)
2692:     file_facts = [extract_file_facts(file) for file in primary_tracked_files(files)]
2693:     written: list[Path] = []
2694:     for facts in file_facts:
2695:         docx_path = ALL_FILE_REFERENCE_DOCX_DIR / code_reference_name(facts["repo_file"], ".docx")
2696:         write_single_file_reference_docx(facts, docx_path)
2697:         written.append(docx_path)
2698:         pdf_path = ALL_FILE_REFERENCE_PDF_DIR / code_reference_name(facts["repo_file"], ".pdf")
2699:         write_single_file_reference_pdf(facts, pdf_path)
2700:         if pdf_path.exists():
2701:             written.append(pdf_path)
2702:     write_master_repository_file_reference_docx(file_facts, diagrams)
2703:     written.append(ALL_FILE_REFERENCE_MASTER_DOCX)
2704:     write_master_repository_file_reference_pdf(file_facts, diagrams)
2705:     if ALL_FILE_REFERENCE_MASTER_PDF.exists():
2706:         written.append(ALL_FILE_REFERENCE_MASTER_PDF)
2707:     return written
2708:
2709:

```

write_file_reference_docs (docs/build_complete_guide.py, lines 2710-2742)

This function regenerates the focused important-code reference DOCX/PDF artifacts for the selected behavior-driving files.

It gives the user a smaller curated reference that is easier to read than the exhaustive per-file set.

It calls or references `remove_directory`, `makedirs`, `extract_file_facts`, `important_code_files`, `code_reference_name`, `write_single_file_reference_docx`, `append`, `write_single_file_reference_pdf`, `exists`, `write_master_file_reference_docx`, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2740: `return written`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2710: def write_file_reference_docs(files: list[str], diagrams: dict[str, Path]) -> list[Path]:
2711:     remove_directory(FILE_REFERENCE_DOCX_DIR)
2712:     remove_directory(FILE_REFERENCE_PDF_DIR)
2713:     FILE_REFERENCE_DOCX_DIR.mkdir(parents=True, exist_ok=True)
2714:     FILE_REFERENCE_PDF_DIR.mkdir(parents=True, exist_ok=True)
2715:     file_facts = [extract_file_facts(file) for file in important_code_files(files)]
2716:     written: list[Path] = []
2717:     for facts in file_facts:
2718:         docx_path = FILE_REFERENCE_DOCX_DIR / code_reference_name(facts["repo_file"], ".docx")
2719:         write_single_file_reference_docx(
2720:             facts,
2721:             docx_path,
2722:             title_prefix="Important Code Reference",
2723:             subtitle="A focused code/control-file explanation covering purpose, owner layer, APIs,
variables, implementation signals, source excerpts, and debugging rules.",
2724:         )
2725:         written.append(docx_path)
2726:         pdf_path = FILE_REFERENCE_PDF_DIR / code_reference_name(facts["repo_file"], ".pdf")
2727:         write_single_file_reference_pdf(
2728:             facts,
2729:             pdf_path,
2730:             title_prefix="Important Code Reference",
2731:             subtitle="A focused code/control-file explanation covering purpose, owner layer, APIs,
variables, implementation signals, source excerpts, and debugging rules.",
2732:         )
2733:         if pdf_path.exists():
2734:             written.append(pdf_path)
2735:     write_master_file_reference_docx(file_facts, diagrams)
2736:     written.append(FILE_REFERENCE_MASTER_DOCX)
2737:     write_master_file_reference_pdf(file_facts, diagrams)
2738:     if FILE_REFERENCE_MASTER_PDF.exists():
2739:         written.append(FILE_REFERENCE_MASTER_PDF)
2740:     return written
2741:
2742:
```

write_code_reference_docs (docs/build_complete_guide.py, lines 2743-2768)

This function regenerates the exhaustive per-file code-reference DOCX/PDF artifacts from the current repository state.

It keeps the file-specific docs synchronized with the actual code whenever the repository changes.

It calls or references `remove_directory`, `makedirs`, `is_text_code_file`, `extract_source_facts`, `code_reference_name`, `write_single_code_reference_docx`, `append`, `write_single_code_reference_pdf`, `exists`, `write_master_code_reference_docx`, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 2766: `return written`.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2743: def write_code_reference_docs(files: list[str], diagrams: dict[str, Path]) -> list[Path]:
2744:     remove_directory(CODE_REFERENCE_DIR)
2745:     remove_directory(CODE_REFERENCE_DOCX_DIR)
2746:     remove_directory(CODE_REFERENCE_PDF_DIR)
2747:     CODE_REFERENCE_DOCX_DIR.mkdir(parents=True, exist_ok=True)
2748:     CODE_REFERENCE_PDF_DIR.mkdir(parents=True, exist_ok=True)
2749:     code_files = [file for file in files if is_text_code_file(file)]
2750:     facts_by_file = {repo_file: extract_source_facts(repo_file) for repo_file in code_files}
2751:     written: list[Path] = []
2752:     for repo_file in code_files:
2753:         facts = facts_by_file[repo_file]
2754:         docx_path = CODE_REFERENCE_DOCX_DIR / code_reference_name(repo_file, ".docx")
2755:         write_single_code_reference_docx(facts, docx_path)
2756:         written.append(docx_path)
2757:         pdf_path = CODE_REFERENCE_PDF_DIR / code_reference_name(repo_file, ".pdf")
2758:         write_single_code_reference_pdf(facts, pdf_path)
2759:         if pdf_path.exists():
```

```

2760:         written.append(pdf_path)
2761:     write_master_code_reference_docx(code_files, facts_by_file, diagrams)
2762:     written.append(CODE_REFERENCE_MASTER_DOCX)
2763:     write_master_code_reference_pdf(code_files, facts_by_file, diagrams)
2764:     if CODE_REFERENCE_MASTER_PDF.exists():
2765:         written.append(CODE_REFERENCE_MASTER_PDF)
2766:     return written
2767:
2768:

```

add_generated_code_reference (docs/build_complete_guide.py, lines 2769-2789)

add_generated_code_reference named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2769-2789 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, write_code_reference_docs, add_paragraph, add_table, str, relative_to, exists, len, bullets, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2769: def add_generated_code_reference(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
2770:     doc.add_heading("Generated Word And PDF Code References", level=1)
2771:     written = write_code_reference_docs(files, diagrams)
2772:     doc.add_paragraph("The repository includes generated Word and PDF code-reference documents for
detailed per-file study. The complete guide intentionally does not duplicate every function from every file. Use
the file-specific references when you want exhaustive function coverage for a particular source file.")
2773:     add_table(doc, [
2774:         ("Word folder", str(CODE_REFERENCE_DOCX_DIR.relative_to(REPO))),
2775:         ("PDF folder", str(CODE_REFERENCE_PDF_DIR.relative_to(REPO))),
2776:         ("Master Word reference", str(CODE_REFERENCE_MASTER_DOCX.relative_to(REPO))),
2777:         ("Master PDF reference", str(CODE_REFERENCE_MASTER_PDF.relative_to(REPO)) if
CODE_REFERENCE_MASTER_PDF.exists() else "PDF generation skipped because ReportLab is unavailable."),
2778:         ("Artifacts generated", str(len(written))),
2779:         ("Regeneration command", "python docs/build_complete_guide.py"),
2780:     ])
2781:     doc.add_heading("What belongs here versus file-specific docs", level=2)
2782:     bullets(doc, [
2783:         "The complete guide explains architecture, workflows, boundaries, tools, and how the whole system
fits together.",
2784:         "The code-reference DOCX/PDF files explain every discovered function inside each specific file.",
2785:         "When editing one file, open that file's generated reference rather than scrolling through the
complete guide.",
2786:     ])
2787:     doc.add_page_break()
2788:
2789:

```

add_generated_repository_file_reference (docs/build_complete_guide.py, lines 2790-2813)

add_generated_repository_file_reference named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2790-2813 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, write_repository_file_reference_docs, primary_tracked_files, add_paragraph, add_table, str, relative_to, exists, len, bullets, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2790: def add_generated_repository_file_reference(doc: Document, files: list[str], diagrams: dict[str, Path])
-> None:
2791:     doc.add_heading("Generated Word And PDF Repository File References", level=1)
2792:     written = write_repository_file_reference_docs(files, diagrams)
2793:     documented_files = primary_tracked_files(files)
2794:     doc.add_paragraph("The repository now has a file-specific Word document and PDF for each primary
tracked file. Source files receive detailed function explanations. Static assets, favicons, marker files,

```

```

security files, workflows, JSON files, and text documents receive purpose, metadata, usage, and safe-change
notes.")
2795:     add_table(doc, [
2796:         ("Word folder", str(ALL_FILE_REFERENCE_DOCX_DIR.relative_to(REPO))),
2797:         ("PDF folder", str(ALL_FILE_REFERENCE_PDF_DIR.relative_to(REPO))),
2798:         ("Master Word reference", str(ALL_FILE_REFERENCE_MASTER_DOCX.relative_to(REPO))),
2799:         ("Master PDF reference", str(ALL_FILE_REFERENCE_MASTER_PDF.relative_to(REPO)) if
ALL_FILE_REFERENCE_MASTER_PDF.exists() else "PDF generation skipped because ReportLab is unavailable."),
2800:         ("Primary files documented", str(len(documented_files))),
2801:         ("Artifacts generated", str(len(written))),
2802:         ("Excluded", "Generated documentation outputs and generated guide diagrams are skipped to avoid
recursively documenting generated files."),
2803:         ("Regeneration command", "python docs/build_complete_guide.py"),
2804:     ])
2805:     doc.add_heading("How this supports slow curation", level=2)
2806:     bullets(doc, [
2807:         "Each file can be improved independently because it now has its own document.",
2808:         "The complete guide can stay focused on system architecture instead of becoming an unreadable
function dump.",
2809:         "When a file changes, regenerate the references and review only that file's document first.",
2810:     ])
2811:     doc.add_page_break()
2812:
2813:

```

add_generated_file_reference (docs/build_complete_guide.py, lines 2814-2837)

`add_generated_file_reference` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2814-2837 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `add_heading`, `write_file_reference_docs`, `important_code_files`, `add_paragraph`, `add_table`, `str`, `relative_to`, `exists`, `len`, `bullets`, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2814: def add_generated_file_reference(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
2815:     doc.add_heading("Generated Word And PDF Important Code References", level=1)
2816:     written = write_file_reference_docs(files, diagrams)
2817:     selected_files = important_code_files(files)
2818:     doc.add_paragraph("The repository now includes generated file-specific Word and PDF documents for the
important code and control files that explain how the system actually works. This focused set avoids passive
binary assets and marker files so the documentation stays useful for learning the builder internals.")
2819:     add_table(doc, [
2820:         ("Word folder", str(FILE_REFERENCE_DOCX_DIR.relative_to(REPO))),
2821:         ("PDF folder", str(FILE_REFERENCE_PDF_DIR.relative_to(REPO))),
2822:         ("Master Word reference", str(FILE_REFERENCE_MASTER_DOCX.relative_to(REPO))),
2823:         ("Master PDF reference", str(FILE_REFERENCE_MASTER_PDF.relative_to(REPO)) if
FILE_REFERENCE_MASTER_PDF.exists() else "PDF generation skipped because ReportLab is unavailable."),
2824:         ("Important files documented", str(len(selected_files))),
2825:         ("Artifacts generated", str(len(written))),
2826:         ("Skipped here", "Favicons, app icons, binary assets, passive marker files, and generated
reference documents."),
2827:         ("Regeneration command", "python docs/build_complete_guide.py"),
2828:     ])
2829:     doc.add_heading("What this adds beyond the code reference", level=2)
2830:     bullets(doc, [
2831:         "The selected files are the places a developer actually starts when debugging app behavior.",
2832:         "Each selected file gets source excerpts, implementation signals, API/DOM/storage clues, and
safe-change notes.",
2833:         "The master reference acts like a guided map of the app internals rather than a dump of every
asset.",
2834:     ])
2835:     doc.add_page_break()
2836:
2837:

```

add_complete_function_inventory (docs/build_complete_guide.py, lines 2838-2856)

`add_complete_function_inventory` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2838-2856 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `important_code_files`, `add_heading`, `add_paragraph`, `add_table`, `relative_to`, `get`, `folder_role`, and `add_page_break`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2838: def add_complete_function_inventory(doc: Document, files: list[str]) -> None:
2839:     selected_files = important_code_files(files)
2840:     doc.add_heading("Where The File-Level Details Live", level=1)
2841:     doc.add_paragraph("The complete guide explains the whole system: architecture, workflows, layers,
tools, publishing, installer behavior, AI backend, security boundaries, and how files communicate. It
intentionally does not explain every function body. Each important file has its own generated Word/PDF reference
where that file's functions, calls, returns, variables, source excerpts, and purpose are explained in detail.")
2842:     add_table(doc, [
2843:         ("Complete guide job", "Explain the whole builder and website system without becoming a raw
source-code dump."),
2844:         ("All-file docs job", "Give every primary tracked file its own Word/PDF explanation, including
metadata for assets and detailed content for readable files."),
2845:         ("Source-code docs job", "Explain each source file's discovered functions, variables, calls,
returns, endpoints, and excerpts."),
2846:         ("Important-file docs job", "Focus on the behavior-driving files a developer should read
first."),
2847:         ("All-file docs", f"{ALL_FILE_REFERENCE_DOCX_DIR.relative_to(REPO)} and
{ALL_FILE_REFERENCE_PDF_DIR.relative_to(REPO)}"),
2848:         ("Master all-file reference", f"{ALL_FILE_REFERENCE_MASTER_DOCX_DIR.relative_to(REPO)} and
{ALL_FILE_REFERENCE_MASTER_PDF_DIR.relative_to(REPO)}"),
2849:         ("Important file docs", f"{FILE_REFERENCE_DOCX_DIR.relative_to(REPO)} and
{FILE_REFERENCE_PDF_DIR.relative_to(REPO)}"),
2850:         ("Exhaustive code docs", f"{CODE_REFERENCE_DOCX_DIR.relative_to(REPO)} and
{CODE_REFERENCE_PDF_DIR.relative_to(REPO)}"),
2851:     ])
2852:     doc.add_heading("Recommended source-reading order", level=2)
2853:     add_table(doc, [(repo_file, FILE_DESCRIPTIONS.get(repo_file, folder_role(repo_file))) for repo_file
in selected_files])
2854:     doc.add_page_break()
2855:
2856:
```

add_deep_detail_topics (docs/build_complete_guide.py, lines 2857-2870)

`add_deep_detail_topics` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2857-2870 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `add_heading`, `add_paragraph`, `bullets`, and `add_page_break`. Endpoint references found inside it are no direct `/api` endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2857: def add_deep_detail_topics(doc: Document) -> None:
2858:     for title, body in DEEP_DETAIL_TOPICS:
2859:         doc.add_heading(f"Deep Detail: {title}", level=1)
2860:         doc.add_paragraph(body)
2861:         doc.add_heading("How to use this detail", level=2)
2862:         bullets(doc, [
2863:             "Start with the user-visible behavior.",
2864:             "Find the file that owns that behavior.",
2865:             "Trace the data handoff to the next file, endpoint, cache, or generated artifact.",
2866:             "Change the smallest responsible piece and test the flow end to end.",
2867:         ])
2868:         doc.add_page_break()
2869:
2870:
```

add_programming_syntax (docs/build_complete_guide.py, lines 2871-2880)

`add_programming_syntax` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2871-2880 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_code, add_paragraph, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2871: def add_programming_syntax(doc: Document) -> None:
2872:     for title, snippet, explanation in PROGRAMMING_SYNTAX_LESSONS:
2873:         doc.add_heading(f"Programming Syntax: {title}", level=1)
2874:         add_code(doc, snippet)
2875:         doc.add_paragraph(explanation)
2876:         doc.add_heading("How this appears in the project", level=2)
2877:         doc.add_paragraph("You will see this pattern in main.cjs, server.mjs, template-preview.js,
script.js, Cloudflare Worker code, HTML files, CSS files, JSON catalogs, and GitHub workflow YAML.")
2878:         doc.add_page_break()
2879:
2880:
```

add_shell_syntax (docs/build_complete_guide.py, lines 2881-2890)

add_shell_syntax named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2881-2890 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_code, add_paragraph, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2881: def add_shell_syntax(doc: Document) -> None:
2882:     for title, snippet, explanation in SHELL_SYNTAX_LESSONS:
2883:         doc.add_heading(f"Shell Or Script Syntax: {title}", level=1)
2884:         add_code(doc, snippet)
2885:         doc.add_paragraph(explanation)
2886:         doc.add_heading("Why this matters", level=2)
2887:         doc.add_paragraph("Shell scripts glue tools together. They are not the app UI, but they install
tools, build releases, deploy workers, scan directories, and enforce rules.")
2888:         doc.add_page_break()
2889:
2890:
```

add_caching_methods (docs/build_complete_guide.py, lines 2891-2915)

add_caching_methods named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2891-2915 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2891: def add_caching_methods(doc: Document, diagrams: dict[str, Path]) -> None:
2892:     doc.add_heading("Caching Methods Used In The Builder And Website", level=1)
2893:     add_diagram(doc, diagrams["compile-code-flow"], "Compile artifact caching happens inside the Compile
Code workspace.")
2894:     doc.add_paragraph("Caching means remembering a result so the app does not repeat slow or annoying
work. This project uses several different caches, each with a different safety boundary.")
2895:     add_table(doc, [(name, f"{where} Purpose: {purpose}") for name, where, purpose in CACHING_METHODS])
2896:     doc.add_heading("Important safety rule", level=2)
2897:     doc.add_paragraph("A cache must be scoped. Publishing authorization is scoped to the same repository,
branch, user/machine scope, and expiration time. Compiler caches are scoped to source code, language, compiler
path, runtime path, and file name. UI caches like update snooze are local convenience data and do not grant
```

```

publishing access.")
2898:     doc.add_page_break()
2899:     for name, where, purpose in CACHING_METHODS:
2900:         doc.add_heading(f"Cache Detail: {name}", level=1)
2901:         if "Compiler" in name or "Compile" in name or "Terminal" in name or "source" in name:
2902:             add_diagram(doc, diagrams["compile-code-flow"], f"Context diagram for {name}.")
2903:         elif "Publishing" in name or "Extended trust" in name:
2904:             add_diagram(doc, diagrams["release-pipeline"], f"Publishing authorization cache supports this
release/publish workflow.")
2905:         elif "HTTP" in name:
2906:             add_diagram(doc, diagrams["frontend-backend-cloudflare"], f"HTTP no-store protects request
freshness across frontend/backend paths.")
2907:         add_table(doc, [
2908:             ("Where implemented", where),
2909:             ("Why it exists", purpose),
2910:             ("What can go stale", "Cached data can become outdated when files, tools, versions,
repositories, or user choices change."),
2911:             ("How to refresh", "Use the explicit refresh/rebuild/authenticate action for that feature,
clear the relevant local data, or restart the builder when appropriate."),
2912:         ])
2913:         doc.add_page_break()
2914:
2915:

```

add_installer_details (docs/build_complete_guide.py, lines 2916-2951)

add_installer_details named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2916-2951 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, numbers, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

2916: def add_installer_details(doc: Document, diagrams: dict[str, Path]) -> None:
2917:     doc.add_heading("How The Installer Works", level=1)
2918:     add_diagram(doc, diagrams["release-pipeline"], "Installer and update behavior sits at the end of the
release pipeline.")
2919:     doc.add_paragraph("The installer is produced by electron-builder and customized by NSIS script code
in build/installer.nsh. electron-builder gathers app files and creates the installer shell. NSIS controls
Windows setup pages, duplicate install checks, shortcut behavior, Git installation checks, update behavior, and
uninstall pages.")
2920:     add_table(doc, [
2921:         ("electron-builder", "Reads package.json build settings and packages the Electron app."),
2922:         ("NSIS", "Runs installer wizard logic and custom script functions."),
2923:         ("installer.nsh", "Adds OMB-specific setup pages, duplicate detection, update handling,
shortcuts, and Git checks."),
2924:         ("AppData install", "Default per-user install location avoids Program Files admin prompts."),
2925:         ("Update install", "Existing installation is updated in place instead of creating a duplicate
copy."),
2926:     ])
2927:     doc.add_heading("Installer flow", level=2)
2928:     numbers(doc, [
2929:         "customInit runs first.",
2930:         "The installer checks for an existing registered install.",
2931:         "If a current or newer install exists, setup stops.",
2932:         "If an older install exists, setup treats the run as an update and uses the existing location.",
2933:         "If no install exists, setup scans for duplicate builder copies.",
2934:         "The custom tools page asks about desktop shortcut and publishing tools.",
2935:         "Files are installed into the chosen per-user location.",
2936:         "customInstall creates/removes shortcuts and optionally installs Git for Windows.",
2937:         "The uninstaller has its own page explaining what will and will not be removed.",
2938:     ])
2939:     doc.add_page_break()
2940:     for name, description in INSTALLER_FUNCTIONS:
2941:         doc.add_heading(f"Installer Function: {name}", level=1)
2942:         doc.add_paragraph(description)
2943:         add_table(doc, [
2944:             ("File", "build/installer.nsh"),
2945:             ("Syntax family", "NSIS installer scripting"),
2946:             ("Function job", description),
2947:             ("Beginner note", "Installer functions run during setup/uninstall, not while the normal
builder UI is being used."),
2948:         ])
2949:         doc.add_page_break()
2950:
2951:

```

add_function_maps (docs/build_complete_guide.py, lines 2952-2979)

add_function_maps named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2952-2979 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2952: def add_function_maps(doc: Document, diagrams: dict[str, Path]) -> None:
2953:     for file_name, group_purpose, functions in FUNCTION_GROUPS:
2954:         doc.add_heading(f"Function Map: {file_name}", level=1)
2955:         if file_name == "main.cjs":
2956:             add_diagram(doc, diagrams["electron-runtime"], "Electron startup functions sit inside this
runtime flow.")
2957:         elif file_name == "server.mjs":
2958:             add_diagram(doc, diagrams["frontend-backend-cloudflare"], "server.mjs is the local backend
layer in this map.")
2959:         elif file_name == "template-preview.js":
2960:             add_diagram(doc, diagrams["builder-to-site-files"], "template-preview.js is the builder
frontend that sends edits toward parser output.")
2961:         elif file_name == "script.js":
2962:             add_diagram(doc, diagrams["cloudflare-ai-flow"], "script.js owns the public website behavior,
including the AI chat request path.")
2963:             doc.add_paragraph(group_purpose)
2964:             add_table(doc, [(name, description) for name, description in functions])
2965:             doc.add_heading("How to use this map", level=2)
2966:             doc.add_paragraph("When debugging, find the user action first, then find the function group that
owns it. For example, a compile terminal issue starts in template-preview.js, then calls server.mjs, then
returns output back to template-preview.js.")
2967:             doc.add_page_break()
2968:             for name, description in functions:
2969:                 doc.add_heading(f"Function Detail: {file_name} -> {name}", level=1)
2970:                 doc.add_paragraph(description)
2971:                 add_table(doc, [
2972:                     ("File", file_name),
2973:                     ("Function", name),
2974:                     ("Responsibility", description),
2975:                     ("Debug question", "What input does this function receive, what output or state change
does it produce, and what function calls it next?"),
2976:                 ])
2977:                 doc.add_page_break()
2978:
2979:
```

add_architecture (docs/build_complete_guide.py, lines 2980-2999)

add_architecture named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 2980-2999 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, get, add_diagram, add_code, bullets, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
2980: def add_architecture(doc: Document, diagrams: dict[str, Path]) -> None:
2981:     for title, body, diagram in ARCHITECTURE_PAGES:
2982:         doc.add_heading(title, level=1)
2983:         doc.add_paragraph(body)
2984:         diagram_name = ARCHITECTURE_DIAGRAM_BY_TITLE.get(title)
2985:         if diagram_name:
2986:             add_diagram(doc, diagrams[diagram_name], f"Generated block diagram: {title}.")
2987:             doc.add_heading("Text version of the block diagram", level=2)
2988:             add_code(doc, diagram)
2989:             doc.add_heading("Beginner explanation", level=2)
2990:             doc.add_paragraph("Read each arrow as 'this thing talks to the next thing.' The important rule is
that editing happens locally in the builder, while viewing happens publicly on the website after generated
static files are published.")
```



```

2991:         doc.add_heading("What can break", level=2)
2992:         bullets(doc, [
2993:             "If local builder state is not saved, the parser may not see the latest edits.",
2994:             "If Git authentication or branch state is wrong, Apply to site cannot safely push.",
2995:             "If public hosting cache is stale, a phone may show an older copy until refresh or cache
expiry.",
2996:         ])
2997:         doc.add_page_break()
2998:
2999:

```

add_file_communication (docs/build_complete_guide.py, lines 3000-3022)

add_file_communication named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3000-3022 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, bullets, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3000: def add_file_communication(doc: Document, diagrams: dict[str, Path]) -> None:
3001:     doc.add_heading("How The Important Files Communicate", level=1)
3002:     add_diagram(doc, diagrams["file-communication-map"], "Generated map of the main builder files and
public website files.")
3003:     doc.add_paragraph("This is the file-level mental model. The desktop app starts in main.cjs, the
backend lives in server.mjs, the builder screen is template-preview.html with template-preview.css and template-
preview.js, and the public website is index.html with styles.css, script.js, projects.json, and assets.")
3004:     add_table(doc, [
3005:         ("main.cjs -> server.mjs", "Electron starts the backend so the builder UI can call local APIs."),
3006:         ("main.cjs -> template-preview.html", "Electron opens the builder window and loads the builder
HTML file."),
3007:         ("template-preview.html -> template-preview.js", "The builder HTML loads the JavaScript that
handles editing, saving, previewing, compile code, and publishing UI."),
3008:         ("template-preview.js -> server.mjs", "The builder uses fetch requests to ask the backend to save
files, parse projects, run Git, run compilers, or publish."),
3009:         ("server.mjs -> projects.json", "The parser writes public project data used by the website."),
3010:         ("index.html -> script.js/styles.css", "The public page loads its behavior and visual design."),
3011:         ("script.js -> projects.json", "The public website reads the generated catalog to render project
windows."),
3012:     ])
3013:     doc.add_heading("Why this communication shape is useful", level=2)
3014:     bullets(doc, [
3015:         "The public website can stay static and fast.",
3016:         "The builder can safely use local-only features without exposing them online.",
3017:         "The parser creates a clean boundary between editable state and visitor-facing output.",
3018:         "The same project data can be previewed locally before it is published.",
3019:     ])
3020:     doc.add_page_break()
3021:
3022:

```

add_tool_build_map (docs/build_complete_guide.py, lines 3023-3032)

add_tool_build_map named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3023-3032 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3023: def add_tool_build_map(doc: Document, diagrams: dict[str, Path]) -> None:
3024:     doc.add_heading("What Tool Builds What", level=1)
3025:     add_diagram(doc, diagrams["tool-build-map"], "Generated map showing which tool owns each part of the
build and release process.")
3026:     doc.add_paragraph("A common source of confusion is thinking one tool builds everything. In this

```

```

project, each tool has a narrow job. pnpm installs dependencies, Electron runs the app, electron-builder
packages the app, NSIS makes the installer, Git moves files, GitHub Actions automates release builds, Wrangler
deploys Cloudflare Worker code, and compilers run project source code.")
3027:     add_table(doc, [(tool, f"{builds} Used for: {where}") for tool, builds, where in TOOL_BUILD_ROWS])
3028:     doc.add_heading("Beginner shortcut", level=2)
3029:     doc.add_paragraph("When something fails, ask which tool owns that step. Installer failure points
toward NSIS or electron-builder. AI backend failure points toward Cloudflare Worker or Wrangler. Website publish
failure points toward Git, GitHub, branch state, or authentication.")
3030:     doc.add_page_break()
3031:
3032:

```

add_software_decisions (docs/build_complete_guide.py, lines 3033-3051)

add_software_decisions named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3033-3051 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, add_table, bullets, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3033: def add_software_decisions(doc: Document) -> None:
3034:     for title, body in SOFTWARE_DECISIONS:
3035:         doc.add_heading(f"Software Engineering Decision: {title}", level=1)
3036:         doc.add_paragraph(body)
3037:         add_table(doc, [
3038:             ("Decision", title),
3039:             ("Reason", body),
3040:             ("What it protects", "Predictable publishing, safer public/private separation, easier
updates, clearer debugging, or better user experience."),
3041:             ("Tradeoff", "The design may add more files or moving parts, but each moving part has a
defined job."),
3042:         ])
3043:         doc.add_heading("How to evaluate this later", level=2)
3044:         bullets(doc, [
3045:             "If the feature becomes hard to maintain, check whether responsibilities are mixed across too
many files.",
3046:             "If users are confused, improve the builder UI or guide rather than exposing implementation
details.",
3047:             "If publishing becomes risky, tighten the parser and authentication boundary first.",
3048:         ])
3049:         doc.add_page_break()
3050:
3051:

```

add_system_layers (docs/build_complete_guide.py, lines 3052-3067)

add_system_layers named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3052-3067 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, add_page_break, add_table, and add_code. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3052: def add_system_layers(doc: Document, diagrams: dict[str, Path]) -> None:
3053:     doc.add_heading("AI, Frontend, Backend, Cloudflare, And Secrets", level=1)
3054:     add_diagram(doc, diagrams["frontend-backend-cloudflare"], "Generated map of the frontend, backend,
Cloudflare Worker, AI provider, and secret boundary.")
3055:     doc.add_paragraph("This section separates the major layers so you know where to look when something
breaks. The frontend draws and captures input. The backend performs protected work. Cloudflare hosts the public
AI endpoint. Secrets stay server-side.")
3056:     doc.add_page_break()
3057:     for title, body, rows, snippet in SYSTEM_LAYER_SECTIONS:
3058:         doc.add_heading(title, level=1)
3059:         doc.add_paragraph(body)

```

```

3060:         add_table(doc, rows)
3061:         doc.add_heading("Representative code or command", level=2)
3062:         add_code(doc, snippet)
3063:         doc.add_heading("Debug question", level=2)
3064:         doc.add_paragraph("Ask which layer owns the problem: did the UI send the request, did the backend
receive it, did the service have the needed secret or tool, and did the response come back in the format the UI
expects?")
3065:         doc.add_page_break()
3066:
3067:

```

add_workflows (docs/build_complete_guide.py, lines 3068-3101)

add_workflows named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3068-3101 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, numbers, add_paragraph, add_page_break, startswith, read_text, get, add_code, join, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3068: def add_workflows(doc: Document, files: list[str], diagrams: dict[str, Path]) -> None:
3069:     workflows = [
3070:         ("Content Creation Workflow", ["Open the builder.", "Edit profile, front-page text, portfolio
areas, or projects.", "Save project or Save all sections.", "Open preview and inspect the exact public layout.",
"Save draft.", "Apply to site after the target is authenticated."]),
3071:         ("Project Creation Workflow", ["Click Add project.", "Choose category.", "Choose appearance.",
"Enter title.", "Edit overview and sections.", "Add files, images, links, code blocks, and compile-code
evidence.", "Save project so the parser can build the project preview."]),
3072:         ("Publishing Workflow", ["Enter repository target.", "Authenticate GitHub write access.", "Load
from target if the target already has content.", "Save draft.", "Apply to site.", "Git commits and pushes
generated files.", "GitHub Pages or Cloudflare serves the updated website."]),
3073:         ("Update Workflow", ["The installed app checks GitHub Releases.", "If a newer version exists, the
app shows an update dialog.", "The updater downloads the installer.", "The app closes itself.", "The installer
updates the existing AppData install.", "The app relaunches after the installer finishes."]),
3074:         ("Compile Code Workflow", ["Create or import a source file.", "Pick the language.", "For
Verilog/SystemVerilog, mark design files as Design and create or import a Testbench file.", "Keep $dumpfile and
$dumpvars in the HDL testbench so the signal scope can draw waveforms.", "Save source.", "Run beautifier if
desired.", "Compile or run.", "Read terminal output, messages log, and HDL scope when available.", "Append the
source code to a project section when ready."]),
3075:     ]
3076:     for title, steps in workflows:
3077:         doc.add_heading(title, level=1)
3078:         if title in {"Content Creation Workflow", "Project Creation Workflow", "Publishing Workflow"}:
3079:             add_diagram(doc, diagrams["builder-to-site-files"], f"Generated flow diagram supporting
{title}.")
3080:         if title == "Update Workflow":
3081:             add_diagram(doc, diagrams["release-pipeline"], "Generated release/update pipeline diagram.")
3082:         if title == "Compile Code Workflow":
3083:             add_diagram(doc, diagrams["compile-code-flow"], "Generated Compile Code workflow diagram.")
3084:         numbers(doc, steps)
3085:         doc.add_heading("Plain-English mental model", level=2)
3086:         doc.add_paragraph("A workflow is just a repeatable recipe. The builder turns complicated actions
into visible buttons so you do not need to remember shell commands every time.")
3087:         doc.add_page_break()
3088:
3089:         for workflow in [f for f in files if f.startswith(".github/workflows/")]:
3090:             text = (REPO / workflow).read_text(encoding="utf-8", errors="replace")
3091:             doc.add_heading(f"GitHub Workflow File: {workflow}", level=1)
3092:             doc.add_paragraph(FILE_DESCRIPTIONS.get(workflow, "GitHub Actions workflow file."))
3093:             doc.add_heading("What a GitHub workflow is", level=2)
3094:             doc.add_paragraph("A workflow is an automated script GitHub runs in the cloud. Instead of you
manually building installers or enforcing branch rules, GitHub reads this YAML file and executes each job on a
temporary machine.")
3095:             doc.add_heading("Important YAML excerpt", level=2)
3096:             add_code(doc, "\n".join(text.splitlines()[1:80]))
3097:             doc.add_heading("Tradeoff", level=2)
3098:             doc.add_paragraph("Automated workflows reduce mistakes, but a bad workflow can block good merges.
That is why the main branch gate should be strict but understandable.")
3099:             doc.add_page_break()
3100:
3101:

```

file_type (docs/build_complete_guide.py, lines 3102-3109)

file_type named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3102-3109 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references endswith, lower, and lstrip. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 3104: return "folder marker"; line 3106: return path.suffix.lower().lstrip("."); line 3107: return "no extension".

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3102: def file_type(path: Path, repo_path: str) -> str:
3103:     if repo_path.endswith(".gitkeep"):
3104:         return "folder marker"
3105:     if path.suffix:
3106:         return path.suffix.lower().lstrip(".")
3107:     return "no extension"
3108:
3109:
```

add_files (docs/build_complete_guide.py, lines 3110-3139)

add_files named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3110-3139 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_paragraph, get, add_table, file_type, stat, exists, is_file, lower, read_text, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3110: def add_files(doc: Document, files: list[str]) -> None:
3111:     text_exts = {".js", ".mjs", ".cjs", ".html", ".css", ".json", ".md", ".txt", ".yaml", ".yml",
".toml", ".nsh", ""}
3112:     for repo_file in files:
3113:         path = REPO / repo_file
3114:         doc.add_heading(f"File Explained: {repo_file}", level=1)
3115:         doc.add_paragraph(FILE_DESCRIPTIONS.get(repo_file, "Tracked repository file used by the builder
or public website."))
3116:         add_table(doc, [
3117:             ("Type", file_type(path, repo_file)),
3118:             ("Size", f"{path.stat().st_size:,} bytes" if path.exists() else "missing locally"),
3119:             ("Used by", "Builder app, public site, installer, Cloudflare, or GitHub workflow depending on
the path."),
3120:             ("Beginner idea", "A repository is a folder Git tracks. This file is one object Git can
version, review, and publish."),
3121:         ])
3122:         if path.exists() and path.is_file() and path.suffix.lower() in text_exts:
3123:             lines = path.read_text(encoding="utf-8", errors="replace").splitlines()
3124:             doc.add_heading("Representative snippet", level=2)
3125:             add_code(doc, "\n".join(lines[:24]) if lines else "(empty file)")
3126:             if len(lines) > 24:
3127:                 doc.add_paragraph(f"Only the first 24 lines are shown here. The full file has
{len(lines):,} lines and should be opened in the repository when editing.")
3128:             else:
3129:                 doc.add_heading("Binary or asset note", level=2)
3130:                 doc.add_paragraph("This is treated as an asset. The guide explains its purpose, but the raw
binary content is not printed because raw image/icon bytes would not be useful to read in Word.")
3131:                 doc.add_heading("Safe editing rule", level=2)
3132:                 bullets(doc, [
3133:                     "Make changes on a feature branch from development.",
3134:                     "Preview or test the behavior before merging.",
3135:                     "Do not edit generated/public files by hand if the builder parser is supposed to generate
them.",
3136:                 ])
3137:                 doc.add_page_break()
3138:
3139:
```

add_commands (docs/build_complete_guide.py, lines 3140-3154)

add_commands named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3140-3154 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_code, add_table, add_paragraph, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3140: def add_commands(doc: Document) -> None:
3141:     for command, explanation, why in COMMAND_LESSONS:
3142:         doc.add_heading(f"Command Explained: {command}", level=1)
3143:         add_code(doc, command)
3144:         add_table(doc, [
3145:             ("What it does", explanation),
3146:             ("Why it matters", why),
3147:             ("Where used", "Local development, GitHub Actions, builder publishing, installer testing, or
compiler execution."),
3148:             ("Beginner warning", "Run commands from the correct folder. In this project, most development
commands belong in the builder repository root."),
3149:         ])
3150:         doc.add_heading("How to read command output", level=2)
3151:         doc.add_paragraph("Command output is the program talking back to you. Success output usually
names what happened. Error output usually names a missing file, missing tool, denied permission, wrong branch,
or failed compile step.")
3152:         doc.add_page_break()
3153:
3154:
```

add_features (docs/build_complete_guide.py, lines 3155-3177)

add_features named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3155-3177 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, add_diagram, add_paragraph, lower, add_table, and add_page_break. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3155: def add_features(doc: Document, diagrams: dict[str, Path]) -> None:
3156:     for topic in FEATURE_TOPICS:
3157:         doc.add_heading(f"Builder Feature: {topic}", level=1)
3158:         if topic in {"Compile Code workspace", "HDL testbench requirement", "HDL signal scope", "Terminal
output", "Messages log", "Append code to project"}:
3159:             add_diagram(doc, diagrams["compile-code-flow"], f"Context diagram for {topic}.")
3160:             elif topic in {"AI chat", "Cloudflare AI worker"}:
3161:                 add_diagram(doc, diagrams["cloudflare-ai-flow"], f"Context diagram for {topic}.")
3162:             elif topic in {"Installer workflow", "Updater workflow", "Release tags", "Branch protection"}:
3163:                 add_diagram(doc, diagrams["release-pipeline"], f"Context diagram for {topic}.")
3164:             elif topic in {"Project preview", "Full portfolio preview", "Project parser", "Asset
synchronization"}:
3165:                 add_diagram(doc, diagrams["builder-to-site-files"], f"Context diagram for {topic}.")
3166:                 doc.add_paragraph(f"This page explains the {topic.lower()} area of the OMB builder and website
system.")
3167:                 add_table(doc, [
3168:                     ("Owner view", "The builder exposes controls for creating, editing, saving, previewing, or
publishing this area."),
3169:                     ("Visitor view", "The public website shows only parsed, approved output. Editing controls
stay private."),
3170:                     ("Parser role", "The parser converts local builder state into website-safe data and
markup."),
3171:                     ("Risk", "If this area is not saved before preview or publishing, the public site may show an
older version."),
3172:                 ])
3173:                 doc.add_heading("Plain-English example", level=2)
3174:                 doc.add_paragraph("Think of the builder as the kitchen and the public website as the plate served
to visitors. The parser is the person plating the content neatly. Visitors should never see the prep
controls.")
3175:                 doc.add_page_break()
3176:
3177:
```

add_tradeoffs (docs/build_complete_guide.py, lines 3178-3189)

`add_tradeoffs` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3178-3189 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `add_heading`, `add_paragraph`, `add_table`, and `add_page_break`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3178: def add_tradeoffs(doc: Document) -> None:
3179:     for title, body in TRADEOFFS:
3180:         doc.add_heading(f"Tradeoff: {title}", level=1)
3181:         doc.add_paragraph(body)
3182:         add_table(doc, [
3183:             ("Benefit", "The chosen direction fits the current builder goal: owner-controlled, local-
first, preview-before-publish portfolio creation."),
3184:             ("Cost", "Every architecture choice gives up something. Usually the cost is complexity, setup
work, or less flexibility in another direction."),
3185:             ("Decision rule", "Prefer the option that keeps publishing safe, previews accurate, and
editing understandable."),
3186:         ])
3187:         doc.add_page_break()
3188:
3189:
```

add_troubleshooting (docs/build_complete_guide.py, lines 3190-3202)

`add_troubleshooting` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3190-3202 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `add_heading`, `add_paragraph`, `numbers`, and `add_page_break`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
3190: def add_troubleshooting(doc: Document) -> None:
3191:     for title, body in TROUBLESHOOTING_TOPICS:
3192:         doc.add_heading(f"Troubleshooting: {title}", level=1)
3193:         doc.add_paragraph(body)
3194:         numbers(doc, [
3195:             "Read the exact error message before changing anything.",
3196:             "Confirm whether the issue is local builder state, Git state, installer state, public website
cache, or backend service availability.",
3197:             "Fix the smallest proven cause first.",
3198:             "Save a clean checkpoint after the fix works.",
3199:         ])
3200:         doc.add_page_break()
3201:
3202:
```

add_concepts (docs/build_complete_guide.py, lines 3203-3216)

`add_concepts` named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3203-3216 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so `docs/build_complete_guide.py` can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references `add_heading`, `add_paragraph`, `add_table`, and `add_page_break`. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3203: def add_concepts(doc: Document) -> None:
3204:     for concept in CONCEPTS:
3205:         doc.add_heading(f"Beginner Concept: {concept}", level=1)
3206:         doc.add_paragraph(f"{concept} is one of the building blocks behind the OMB builder and portfolio
website. You do not need to memorize every term at once. Learn the role it plays, then come back when you see it
in an error message or file name.")
3207:         add_table(doc, [
3208:             ("Simple meaning", "This concept helps explain how the app, website, GitHub workflow,
installer, or backend behaves."),
3209:             ("Where it appears", "Look for it in README instructions, source files, GitHub Actions logs,
installer behavior, browser behavior, or terminal output."),
3210:             ("How to debug it", "Ask: what is supposed to happen, what actually happened, what file or
service owns that behavior, and what changed most recently?"),
3211:         ])
3212:         doc.add_heading("Tiny mental model", level=2)
3213:         doc.add_paragraph("When confused, draw boxes. Put the user on the left, the thing they click in
the middle, and the file or service that responds on the right. Most software problems become easier once the
boxes are visible.")
3214:         doc.add_page_break()
3215:
3216:

```

add_closing (docs/build_complete_guide.py, lines 3217-3231)

add_closing named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3217-3231 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references add_heading, numbers, and add_note. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

Its result is mostly side effect based: it reads, writes, copies, or synchronizes files rather than returning a simple value.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3217: def add_closing(doc: Document) -> None:
3218:     doc.add_heading("Final Operating Checklist", level=1)
3219:     numbers(doc, [
3220:         "Make feature changes on a feature or codex branch from development.",
3221:         "Update README and the in-app guide when behavior changes.",
3222:         "Run local syntax checks and useful smoke tests.",
3223:         "Merge feature work into development.",
3224:         "Only merge development into main for release-ready builds.",
3225:         "Bump package.json before a new main release.",
3226:         "Confirm GitHub Actions publish a fresh installer and stable latest asset.",
3227:         "Open the builder, check update behavior, and verify the public website link downloads the latest
installer.",
3228:     ])
3229:     add_note(doc, "The core rule", "The builder is the source of truth for creating content. The preview
is the truth test before publishing. GitHub is the delivery path. The public website is the final read-only
result.")
3230:
3231:

```

main (docs/build_complete_guide.py, lines 3232-3271)

main named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 3232-3271 and is part of the documentation and generated reference area. used to explain the app and source structure.

This function is needed so docs/build_complete_guide.py can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references tracked_files, load_package, make_diagrams, setup_document, add_cover, add_reading_map, add_entry_points, add_architecture, add_flow_walkthroughs, add_file_communication, and 20 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

3232: def main() -> None:
3233:     files = tracked_files()
3234:     package = load_package()
3235:     diagrams = make_diagrams()
3236:     doc = setup_document()
3237:     add_cover(doc, package)
3238:     add_reading_map(doc)
3239:     add_entry_points(doc)

```

```

3240:     add_architecture(doc, diagrams)
3241:     add_flow_walkthroughs(doc, diagrams)
3242:     add_file_communication(doc, diagrams)
3243:     add_tool_build_map(doc, diagrams)
3244:     add_software_decisions(doc)
3245:     add_system_layers(doc, diagrams)
3246:     add_data_contracts(doc)
3247:     add_api_endpoint_catalog(doc)
3248:     add_programming_syntax(doc)
3249:     add_shell_syntax(doc)
3250:     add_caching_methods(doc, diagrams)
3251:     add_installer_details(doc, diagrams)
3252:     add_function_maps(doc, diagrams)
3253:     add_complete_function_inventory(doc, files)
3254:     add_generated_repository_file_reference(doc, files, diagrams)
3255:     add_generated_code_reference(doc, files, diagrams)
3256:     add_generated_file_reference(doc, files, diagrams)
3257:     add_deep_detail_topics(doc)
3258:     add_workflows(doc, files, diagrams)
3259:     add_files(doc, files)
3260:     add_commands(doc)
3261:     add_features(doc, diagrams)
3262:     add_tradeoffs(doc)
3263:     add_troubleshooting(doc)
3264:     add_concepts(doc)
3265:     add_closing(doc)
3266:     removed_breaks = compact_generated_docx(doc, keep_first_page_break=True)
3267:     print(f"Compacted generated DOCX layout by removing {removed_breaks} page breaks.")
3268:     doc.save(OUTPUT)
3269:     print(OUTPUT)
3270:
3271:

```

Representative opening snippet

```

1: from __future__ import annotations
2:
3: import html
4: import json
5: import math
6: import mimetypes
7: import re
8: import shutil
9: import subprocess
10: import textwrap
11: from collections import Counter
12: from pathlib import Path
13:
14: from PIL import Image, ImageDraw, ImageFont
15: from docx import Document
16: from docx.enum.table import WD_CELL_VERTICAL_ALIGNMENT, WD_TABLE_ALIGNMENT
17: from docx.enum.text import WD_ALIGN_PARAGRAPH
18: from docx.oxml import OxmlElement
19: from docx.oxml.ns import qn
20: from docx.shared import Inches, Pt, RGBColor
21:
22: try:
23:     from reportlab.lib import colors
24:     from reportlab.lib.pagesizes import LETTER
25:     from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
26:     from reportlab.lib.units import inch
27:     from reportlab.platypus import Image as PdfImage
28:     from reportlab.platypus import PageBreak, Paragraph, Preformatted, SimpleDocTemplate, Spacer, Table,
TableStyle
29:
30:     REPORTLAB_AVAILABLE = True
31: except ImportError:
32:     REPORTLAB_AVAILABLE = False
33:
34:
35: REPO = Path(__file__).resolve().parents[1]
36: OUTPUT = REPO / "docs" / "OMB_Portfolio_Builder_Complete_Guide.docx"
37: DIAGRAM_DIR = REPO / "docs" / "guide-diagrams"
38: CODE_REFERENCE_DIR = REPO / "docs" / "code-reference"
39: CODE_REFERENCE_DOCX_DIR = REPO / "docs" / "code-reference-docx"
40: CODE_REFERENCE_PDF_DIR = REPO / "docs" / "code-reference-pdf"
41: CODE_REFERENCE_MASTER_DOCX = REPO / "docs" / "OMB_Portfolio_Builder_Code_Reference.docx"
42: CODE_REFERENCE_MASTER_PDF = REPO / "docs" / "OMB_Portfolio_Builder_Code_Reference.pdf"
43: ALL_FILE_REFERENCE_DOCX_DIR = REPO / "docs" / "file-reference-docx"
44: ALL_FILE_REFERENCE_PDF_DIR = REPO / "docs" / "file-reference-pdf"
45: ALL_FILE_REFERENCE_MASTER_DOCX = REPO / "docs" / "OMB_Portfolio_Builder_File_Reference.docx"
46: ALL_FILE_REFERENCE_MASTER_PDF = REPO / "docs" / "OMB_Portfolio_Builder_File_Reference.pdf"
47: FILE_REFERENCE_DOCX_DIR = REPO / "docs" / "important-code-reference-docx"
48: FILE_REFERENCE_PDF_DIR = REPO / "docs" / "important-code-reference-pdf"
49: FILE_REFERENCE_MASTER_DOCX = REPO / "docs" / "OMB_Portfolio_Builder_Important_Code_Reference.docx"
50: FILE_REFERENCE_MASTER_PDF = REPO / "docs" / "OMB_Portfolio_Builder_Important_Code_Reference.pdf"
51:
52:
53: FILE_DESCRIPTIONS = {
54:     "github/workflows/build-windows-builder.yml": "GitHub Actions workflow that builds the Windows

```



```

installer and portable application, checks the stable download link, enforces release version bumps, uploads
artifacts, and publishes GitHub Releases.",
55:   ".github/workflows/main-branch-gate.yml": "GitHub Actions guard that rejects direct pushes to main
and rejects pull requests into main unless the source branch is development.",
56:   ".gitignore": "Git ignore rules for build outputs, temporary files, and local-only data that should
not be committed.",
57:   ".nojekyll": "Tells GitHub Pages not to process the site with Jekyll, so folders and static files
publish exactly as generated.",
58:   ".well-known/security.txt": "Public security contact file used by browsers, researchers, and scanners
to find responsible disclosure information.",
59:   "BRANCHING.md": "Human-readable branch model explaining development, main, feature branches, and
release flow.",
60:   "Backgrounds/.gitkeep": "Keeps the Backgrounds folder in Git even when no background image files are
present.",
61:   "README.md": "Main README for the standalone builder application: install, updates, workspaces, text
editing, publishing, build commands, branch model, and uninstall.",
62:   "_headers": "Cloudflare Pages style HTTP headers for security and caching behavior on the public
website.",
63:   "assets/apple-touch-icon.png": "Apple touch icon shown when the public site is saved on iOS or
compatible launch surfaces.",
64:   "assets/favicon-16.png": "Small browser favicon used by tabs and browser UI.",
65:   "assets/favicon-192.png": "Large web app icon for Android and installable browser surfaces.",
66:   "assets/favicon-32.png": "Standard browser favicon size.",
67:   "assets/favicon-48.png": "Windows/browser icon size used by some desktop surfaces.",
68:   "assets/favicon.ico": "ICO favicon bundle used by browsers that request favicon.ico.",
69:   "assets/omb-app-icon-256.png": "Builder application icon source at 256 pixels.",
70:   "assets/omb-app-icon-512.png": "Builder application icon source at 512 pixels.",
71:   "assets/omb-app-icon.ico": "Windows ICO application icon used by Electron Builder and shortcuts.",
72:   "assets/omb-mark.png": "OMB snake/mark image used for branding in the site and builder.",

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: builder-rich-future-sections.js

Item	Explanation
File	builder-rich-future-sections.js
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	21,340 bytes
Extension	.js
MIME type	application/javascript
Text lines	410

Why this file exists

- Builder-side helper code for richer future portfolio sections and section defaults.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Rich text at line 82	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <button type="button" data-rich-action="font-small">Small</button>
Rich text at line 83	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <button type="button" data-rich-action="font-normal">Normal</button>
Rich text at line 84	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <button type="button" data-rich-action="font-large">Large</button>
Rich text at line 97	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: data-placeholder="Write the overview here. You can paste images directly into this area."
Browser DOM at line 125	Builder or website interface behavior in the browser/Electron renderer. Evidence: if (editor.dataset.richEditor === "pending-description") {
Browser DOM at line 139	Builder or website interface behavior in the browser/Electron renderer. Evidence: if (editor.dataset.richEditor === "pending-description") {
Browser DOM at line 193	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.querySelector("#pending-editor")?.scrollIntoView({ behavior: "smooth", block: "nearest" });
Browser DOM at line 204	Builder or website interface behavior in the browser/Electron renderer. Evidence: const type = button.dataset.openEditor;
Browser DOM at line 205	Builder or website interface behavior in the browser/Electron renderer. Evidence: const mode = button.dataset.mode "add";
Browser DOM at line 206	Builder or website interface behavior in the browser/Electron renderer. Evidence: const index = button.dataset.index;
Browser DOM at line 208	Builder or website interface behavior in the browser/Electron renderer. Evidence: const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);
Browser DOM at line 213	Builder or website interface behavior in the browser/Electron renderer. Evidence: sectionId: button.dataset.sectionId,
Browser DOM at line 222	Builder or website interface behavior in the browser/Electron renderer. Evidence: const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);
Browser DOM at line 226	Builder or website interface behavior in the browser/Electron renderer. Evidence: sectionId: button.dataset.sectionId,

Item	Explanation
Cloudflare or AI at line 238	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: // their richDescription field back to the project model. Fallback to
Browser DOM at line 245	Builder or website interface behavior in the browser/Electron renderer. Evidence: const richEditor = form.querySelector("[data-rich-editor='pending-description']");
Browser DOM at line 393	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.addEventListener("contextmenu", (event) => {

Important constants and variables

Item	Explanation
text	Line 34: const text = plainTextFromRich(rich).trim();
label	Line 73: const label = editor?.type === "custom-section" ? "Section overview" : "Subsection overview";
folder	Line 74: const folder = pendingEditorFolder(editor);
originalPopulateRichEditor	Line 108: const originalPopulateRichEditor = typeof populateRichEditor === "function" ? populateRichEditor : null;
originalSaveRichEditorToProject	Line 109: const originalSaveRichEditorToProject = typeof saveRichEditorToProject === "function" ? saveRichEditorToProject : null;
originalOpenPendingEditor	Line 110: const originalOpenPendingEditor = typeof openPendingEditor === "function" ? openPendingEditor : null;
originalOpenEditorFromButton	Line 111: const originalOpenEditorFromButton = typeof openEditorFromButton === "function" ? openEditorFromButton : null;
originalRenderPendingEditor	Line 112: const originalRenderPendingEditor = typeof renderPendingEditor === "function" ? renderPendingEditor : null;
originalRenderSectionContent	Line 113: const originalRenderSectionContent = typeof renderSectionContent === "function" ? renderSectionContent : null;
originalSavePendingEditor	Line 114: const originalSavePendingEditor = typeof savePendingEditor === "function" ? savePendingEditor : null;
originalRenderCustomSection	Line 115: const originalRenderCustomSection = typeof renderCustomSection === "function" ? renderCustomSection : null;
originalParseProjectForPortfolio	Line 116: const originalParseProjectForPortfolio = typeof parseProjectForPortfolio === "function" ? parseProjectForPortfolio : null;
originalParsedSectionHasContent	Line 117: const originalParsedSectionHasContent = typeof parsedSectionHasContent === "function" ? parsedSectionHasContent : null;
originalPreviewSectionHasRenderableContent	Line 118: const originalPreviewSectionHasRenderableContent = typeof previewSectionHasRenderableContent === "function" ? previewSectionHasRenderableContent : null;
rich	Line 140: const rich = extractRichSummary(editor);
title	Line 154: const title = pendingEditor.title "";
description	Line 155: const description = pendingEditor.description "";
isTool	Line 156: const isTool = pendingEditor.type === "tool";
isSimpleText	Line 157: const isSimpleText = pendingEditor.type === "text-array";
isSectionDetails	Line 158: const isSectionDetails = pendingEditor.type === "custom-section";
usesRich	Line 159: const usesRich = pendingEditorUsesRichDescription();
project	Line 202: const project = selectedProject();
type	Line 204: const type = button.dataset.openEditor;
mode	Line 205: const mode = button.dataset.mode "add";
index	Line 206: const index = button.dataset.index;
section	Line 208: const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);
item	Line 209: const item = mode === "edit" ? section?.items?.[Number(index)] : {};
section	Line 222: const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);
project	Line 242: const project = selectedProject();
title	Line 244: const title = form.elements.title.value.trim();

Item	Explanation
richEditor	Line 245: const richEditor = form.querySelector("[data-rich-editor='pending-description']");
richDescription	Line 246: const richDescription = richEditor ? extractRichSummary(richEditor) : null;
description	Line 247: const description = richEditor ? richPlainTextOrSpacer(richDescription) : form.elements.description?.value.trim() "";
section	Line 253: const section = (project.sections []).find((item) => item.id === pendingEditor.sectionId);
nextItem	Line 256: const nextItem = {
section	Line 278: const section = (project.sections []).find((item) => item.id === pendingEditor.sectionId);
section	Line 297: const section = (project.sections []).find((item) => item.id === sectionId);
parsed	Line 352: const parsed = originalParseProjectForPortfolio(project);
sectionId	Line 355: const sectionId = parsedSection.id.replace("custom:", "");
sourceSection	Line 356: const sourceSection = (project.sections []).find((section) => section.id === sectionId);
editor	Line 394: const editor = event.target.closest("[data-rich-editor]");
block	Line 395: const block = event.target.closest(".rich-block");

Event handlers and user interactions

Item	Explanation
Line 393	document.addEventListener("contextmenu", (event) => {

Function-by-function detail

hasRichContent (builder-rich-future-sections.js, lines 23-31)

hasRichContent named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 23-31 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 24: return Boolean(rich?.blocks?.some((block) =>.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

23:     function hasRichContent(rich) {
24:         return Boolean(rich?.blocks?.some((block) =>
25:             (block.type === "image" && block.url) ||
26:             (block.type === "formula" && block.formula) ||
27:             block.text ||
28:             block.title ||
29:             block.caption
30:         ));
31:     }

```

richPlainTextOrSpacer (builder-rich-future-sections.js, lines 33-36)

richPlainTextOrSpacer named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 33-36 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references plainTextFromRich, trim, and hasRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 35: return text || (hasRichContent(rich) ? "\u200B" : "");.

Important local values include text at line 34. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
33:     function richPlainTextOrSpacer(rich) {
34:         const text = plainTextFromRich(rich).trim();
35:         return text || (hasRichContent(rich) ? "\u200B" : "");
36:     }
```

pendingEditorUsesRichDescription (builder-rich-future-sections.js, lines 41-43)

pendingEditorUsesRichDescription named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 41-43 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 42: return Boolean(editor && ["custom", "custom-section"].includes(editor.type));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
41:     function pendingEditorUsesRichDescription(editor = pendingEditor) {
42:         return Boolean(editor && ["custom", "custom-section"].includes(editor.type));
43:     }
```

pendingEditorFolder (builder-rich-future-sections.js, lines 48-57)

pendingEditorFolder named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 48-57 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 49: if (!editor) return "custom-overview";; line 51: return `custom-\${slugify(editor.sectionId || "section")}-overview`;; line 54: return `custom-\${slugify(editor.sectionId || "section")}-subsection`;; line 56: return "custom-overview";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
48:     function pendingEditorFolder(editor = pendingEditor) {
49:         if (!editor) return "custom-overview";
50:         if (editor.type === "custom-section") {
51:             return `custom-${slugify(editor.sectionId || "section")}-overview`;
52:         }
53:         if (editor.type === "custom") {
54:             return `custom-${slugify(editor.sectionId || "section")}-subsection`;
55:         }
56:         return "custom-overview";
57:     }
```

pendingEditorRichBlocks (builder-rich-future-sections.js, lines 62-65)

pendingEditorRichBlocks named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 62-65 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, and richBlocksFromValue. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 63: if (!editor) return textBlocksFromPlainText("");; line 64: return richBlocksFromValue(editor.richDescription, editor.description || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
62:     function pendingEditorRichBlocks(editor = pendingEditor) {
63:         if (!editor) return textBlocksFromPlainText("");
64:         return richBlocksFromValue(editor.richDescription, editor.description || "");
65:     }
```

renderPendingRichDescriptionEditor (builder-rich-future-sections.js, lines 72-103)

renderPendingRichDescriptionEditor renders ui, markup, or display output. In this file, it appears around lines 72-103 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pendingEditorFolder, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 75: return `.

Important local values include label at line 73; folder at line 74. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
72:     function renderPendingRichDescriptionEditor(editor = pendingEditor) {
73:         const label = editor?.type === "custom-section" ? "Section overview" : "Subsection overview";
74:         const folder = pendingEditorFolder(editor);
75:         return `
76:         <div class="pending-rich-description rich-design-summary-field">
77:             <div class="summary-builder-heading">
78:                 <span>${label}</span>
79:             </div>
80:             <div class="rich-summary-panel">
81:                 <div class="rich-summary-toolbar" aria-label="${escapeHtml(label)} tools">
82:                     <button type="button" data-rich-action="font-small">Small</button>
83:                     <button type="button" data-rich-action="font-normal">Normal</button>
84:                     <button type="button" data-rich-action="font-large">Large</button>
85:                     <button type="button" data-rich-action="align-left">Left</button>
86:                     <button type="button" data-rich-action="align-center">Center</button>
87:                     <button type="button" data-rich-action="align-right">Right</button>
88:                     <button type="button" data-rich-action="add-image">Add image</button>
89:                     <button type="button" data-rich-action="add-formula">Add formula</button>
90:                     <button type="button" data-rich-action="edit-block">Edit selected</button>
91:                     <button class="danger-icon" type="button" data-rich-action="delete-block">Delete
selected</button>
92:                 </div>
93:                 <div
94:                     class="rich-summary-editor"
95:                     data-rich-editor="pending-description"
96:                     data-rich-folder="${escapeHtml(folder)}"
97:                     data-placeholder="Write the overview here. You can paste images directly into this area."
98:                     aria-label="${escapeHtml(label)} rich editor"
99:                 ></div>
100:             </div>
101:         </div>
102:         `;
103:     }
```

section (builder-rich-future-sections.js, lines 208-209)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 208-209 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include section at line 208; item at line 209. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
208:                const section = (project.sections || []).find((item) => item.id ===
button.dataset.sectionId);
209:                const item = mode === "edit" ? section?.items?.[Number(index)] : {};
```

section (builder-rich-future-sections.js, lines 222-230)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 222-230 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and openPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include section at line 222. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
222:                const section = (project.sections || []).find((item) => item.id ===
button.dataset.sectionId);
223:                openPendingEditor({
224:                    type,
225:                    mode: "edit",
226:                    sectionId: button.dataset.sectionId,
227:                    title: section?.title || "",
228:                    description: section?.description || "",
229:                    richDescription: section?.richDescription || null
230:                });
```

section (builder-rich-future-sections.js, lines 253-262)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 253-262 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 254: if (!section) return;.

Important local values include section at line 253; nextItem at line 256. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
253:                const section = (project.sections || []).find((item) => item.id ===
pendingEditor.sectionId);
254:                if (!section) return;
255:                section.items = section.items || [];
256:                const nextItem = {
257:                    title,
258:                    description,
259:                    richDescription,
260:                    type: "Text",
261:                    status: "draft"
262:                };
```

section (builder-rich-future-sections.js, lines 278-291)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 278-291 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, setStatus, scheduleAutosave, renderAll, and originalSavePendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 279: if (!section) return;; line 287: return;; line 289: return originalSavePendingEditor(form);.

Important local values include section at line 278. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
278:         const section = (project.sections || []).find((item) => item.id ===
pendingEditor.sectionId);
279:         if (!section) return;
280:         section.title = title;
281:         section.description = description;
282:         section.richDescription = richDescription;
283:         pendingEditor = null;
284:         setStatus("Saved in the project editor. Click Save project to include this version in the
portfolio preview.");
285:         scheduleAutosave();
286:         renderAll();
287:         return;
288:     }
289:     return originalSavePendingEditor(form);
290: };
291: }
```

section (builder-rich-future-sections.js, lines 297-301)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 297-301 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 298: if (!section) return `<p class="evidence-empty">Section not found.</p>`; line 299: return `

Important local values include section at line 297. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
297:         const section = (project.sections || []).find((item) => item.id === sectionId);
298:         if (!section) return `<p class="evidence-empty">Section not found.</p>`;
299:         return `
300:         <div class="section-window-heading">
301:         <h3>${escapeHtml(section.title || "Custom section")}</h3>
```

sourceSection (builder-rich-future-sections.js, lines 356-360)

sourceSection named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 356-360 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, hasRichContent, clone, and richPlainTextOrSpacer. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include sourceSection at line 356. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
356:         const sourceSection = (project.sections || []).find((section) => section.id ===
sectionId);
357:         if (hasRichContent(sourceSection?.richDescription)) {
358:             parsedSection.rich = clone(sourceSection.richDescription);
359:             parsedSection.description = sourceSection.description ||
richPlainTextOrSpacer(sourceSection.richDescription);
360:         }
```

improveRichContextMenu (builder-rich-future-sections.js, lines 392-405)

improveRichContextMenu named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 392-405 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references addEventListener, closest, preventDefault, selectRichBlock, currentRichBlock, and syncRichContextMenuControls. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 396: if (!editor && !block) return;

Important local values include editor at line 394; block at line 395. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
392:     function improveRichContextMenu() {
393:         document.addEventListener("contextmenu", (event) => {
394:             const editor = event.target.closest("[data-rich-editor]");
395:             const block = event.target.closest(".rich-block");
396:             if (!editor && !block) return;
397:             event.preventDefault();
398:             activeSummaryEditor = editor || block.closest("[data-rich-editor]");
399:             selectRichBlock(block || currentRichBlock(activeSummaryEditor));
400:             syncRichContextMenuControls(activeSummaryBlock);
401:             summaryContextMenu.style.left = `${event.clientX}px`;
402:             summaryContextMenu.style.top = `${event.clientY}px`;
403:             summaryContextMenu.hidden = false;
404:         }, true);
405:     }
```

Representative opening snippet

```
1: // JavaScript source code
2: (() => {
3:     "use strict";
4:
5:     /**
6:      * Extend the portfolio builder with rich description editors for custom
7:      * sections and subsections. This patch adds a new rich editor to the
8:      * pending editor for custom items so that images, formulas and other
9:      * formatted content can be used in overviews of future sections. It
10:     * hooks into existing helper functions such as `populateRichEditor`
11:     * and `saveRichEditorToProject` by overriding them at runtime. To
12:     * avoid interfering with built-in behaviour, everything is wrapped
13:     * inside an IIFE and existing functions are stored before being
14:     * redefined. If a function is not present in the current context
15:     * (because the builder script changed), the patch simply does
16:     * nothing.
17:     */
18:
19:     // Guard against applying the patch multiple times.
20:     if (window.builderRichFutureSectionsPatchLoaded) return;
21:
22:     // Helpers to detect whether an object has meaningful rich content.
23:     function hasRichContent(rich) {
24:         return Boolean(rich?.blocks?.some((block) =>
25:             (block.type === "image" && block.url) ||
26:             (block.type === "formula" && block.formula) ||
27:             block.text ||
28:             block.title ||
29:             block.caption
30:         ));
31:     }
32:
33:     function richPlainTextOrSpacer(rich) {
34:         const text = plainTextFromRich(rich).trim();
35:         return text || (hasRichContent(rich) ? "\u200B" : "");
36:     }
37:
38:     // Determine whether the pending editor should use a rich editor. All
39:     // custom sections and subsections use rich editors. Tools, design
40:     // objects and other built-in forms continue to use plain text areas.
41:     function pendingEditorUsesRichDescription(editor = pendingEditor) {
42:         return Boolean(editor && ["custom", "custom-section"].includes(editor.type));
43:     }
44:
45:     // Build a folder name for images uploaded via the pending editor. For
46:     // custom sections and subsections, use a prefix based on the section
47:     // id so that uploaded images are organised under docs/<project>/<folder>/.
48:     function pendingEditorFolder(editor = pendingEditor) {
49:         if (!editor) return "custom-overview";
50:         if (editor.type === "custom-section") {
51:             return `custom-${slugify(editor.sectionId || "section")}-overview`;
```

```

52:     }
53:     if (editor.type === "custom") {
54:         return `custom-`${slugify(editor.sectionId || "section")}-subsection`;
55:     }
56:     return "custom-overview";
57: }
58:
59: // Derive blocks from a rich value or fallback plain text. When the
60: // editor is empty, provide a single text block so that the content
61: // area is editable.
62: function pendingEditorRichBlocks(editor = pendingEditor) {
63:     if (!editor) return textBlocksFromPlainText("");
64:     return richBlocksFromValue(editor.richDescription, editor.description || "");
65: }
66:
67: // Render the rich description field for the pending editor. This
68: // function builds a UI similar to other rich editors in the builder,
69: // including a toolbar and editable area with appropriate data
70: // attributes. The label varies depending on whether we're editing
71: // a section or a subsection.
72: function renderPendingRichDescriptionEditor(editor = pendingEditor) {

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: electronics-search.js

Item	Explanation
File	electronics-search.js
Category	JavaScript source
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	9,856 bytes
Extension	.js
MIME type	application/javascript
Text lines	162

Why this file exists

- Electronics vocabulary and search support used by the website search behavior.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Rich text at line 8	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: "phase margin", "slew rate", "common mode", "common mode range", "cmrr", "psrr", "loop gain",
Security or auth at line 45	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "oscilloscope", "scope capture", "logic analyzer", "spectrum analyzer", "network analyzer", "function generator",
Compiler or simulator at line 59	Part of the Compile Code workspace or language/tool detection. Evidence: "static timing analysis", "sta", "fpga", "asic", "rtl", "verilog", "systemverilog", "vhdl",
Compiler or simulator at line 61	Part of the Compile Code workspace or language/tool detection. Evidence: "modelsim", "questa", "iverilog", "yosys", "formal verification", "assertion", "coverage", "axi",
Installer at line 80	Windows setup, update, shortcut, or installation behavior. Evidence: "potentiometer", "trimmer", "thermistor", "varistor", "photodiode", "phototransistor", "hall effect sensor",
Cloudflare or AI at line 87	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "layout", "route", "solder", "assemble", "test", "verify", "model", "analyze", "optimize",
Compiler or simulator at line 100	Part of the Compile Code workspace or language/tool detection. Evidence: "ltspice", "spice", "pspice", "ngspice", "matlab", "simulink", "python", "numpy", "scipy",
Git command at line 101	Repository state, publishing, branch checks, or release automation. Evidence: "excel", "kiCad", "kicad", "vivado", "stm32cubeide", "git", "github", "markdown", "c",

Important constants and variables

Item	Explanation
analogTerms	Line 2: const analogTerms = [
powerTerms	Line 20: const powerTerms = [
pcbTerms	Line 33: const pcbTerms = [
testTerms	Line 44: const testTerms = [
digitalTerms	Line 55: const digitalTerms = [

Item	Explanation
embeddedTerms	Line 66: const embeddedTerms = [
components	Line 76: const components = [
actions	Line 85: const actions = [
metrics	Line 92: const metrics = [
toolTerms	Line 99: const toolTerms = [
phraseTerms	Line 105: const phraseTerms = [];
categoryKeywords	Line 114: const categoryKeywords = {
allKeywords	Line 120: const allKeywords = [...new Set([
haystack	Line 139: const haystack = [
buckets	Line 149: const buckets = new Set(categoryKeywords[project?.category] []);
compactTerm	Line 154: const compactTerm = term.replace(/[/^a-z0-9+#]+/g, " ").trim();

JSON/YAML-style keys detected

Item	Explanation
analog-mixed-signal	Line 115: "analog-mixed-signal": [...analogTerms, ...powerTerms, ...pcbTerms, ...testTerms, ...components, ...metrics],

Function-by-function detail

normalize (electronics-search.js, lines 134-136)

normalize validates or normalizes input so later code receives safe predictable values. In this file, it appears around lines 134-136 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so electronics-search.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references toLowerCase. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 135: return String(value || "").toLowerCase();.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
134: function normalize(value) {
135:   return String(value || "").toLowerCase();
136: }
```

projectSpecificKeywords (electronics-search.js, lines 138-158)

projectSpecificKeywords named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 138-158 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so electronics-search.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references map, join, Set, forEach, includes, toLowerCase, add, replace, and trim. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 157: return [...buckets];.

Important local values include haystack at line 139; buckets at line 149; compactTerm at line 154. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
138: function projectSpecificKeywords(project, categoryLabel = "") {
139:   const haystack = [
140:     project?.title,
```

```

141:     project?.summary,
142:     project?.category,
143:     categoryLabel,
144:     ...(project?.focus || []),
145:     ...(project?.tools || []).map((item) => typeof item === "string" ? item : [item.name, item.title,
item.label, item.description].join(" ")),
146:     ...(project?.languages || []),
147:   ].map(normalize).join(" ");
148:
149:   const buckets = new Set(categoryKeywords[project?.category] || []);
150:   toolTerms.forEach((term) => {
151:     if (haystack.includes(term.toLowerCase())) buckets.add(term);
152:   });
153:   allKeywords.forEach((term) => {
154:     const compactTerm = term.replace(/^[a-z0-9+]/g, " ").trim();
155:     if (compactTerm && haystack.includes(compactTerm)) buckets.add(term);
156:   });
157:   return [...buckets];
158: }

```

Representative opening snippet

```

1: (function () {
2:   const analogTerms = [
3:     "op amp", "operational amplifier", "fully differential op amp", "differential amplifier",
"instrumentation amplifier",
4:     "transimpedance amplifier", "transconductance amplifier", "current mirror", "cascode", "folded
cascode",
5:     "common source", "common gate", "common drain", "emitter follower", "source follower", "differential
pair",
6:     "active load", "gain stage", "output stage", "input stage", "bias circuit", "bandgap reference",
7:     "voltage reference", "current reference", "low noise", "offset voltage", "input bias current", "gain
bandwidth",
8:     "phase margin", "slew rate", "common mode", "common mode range", "cmrr", "psrr", "loop gain",
9:     "stability", "compensation", "miller compensation", "frequency response", "bode plot", "pole zero",
10:    "small signal", "large signal", "noise analysis", "thermal noise", "flicker noise", "shot noise",
11:    "analog filter", "low pass filter", "high pass filter", "band pass filter", "notch filter", "active
filter",
12:    "rc filter", "rlc circuit", "resonance", "q factor", "cutoff frequency", "impedance", "admittance",
13:    "matching network", "attenuator", "buffer", "comparator", "window comparator", "schmitt trigger",
"hysteresis",
14:    "oscillator", "vco", "voltage controlled oscillator", "relaxation oscillator", "ring oscillator",
"crystal oscillator",
15:    "pll", "phase locked loop", "charge pump", "sample and hold", "track and hold", "analog switch",
16:    "multiplexer", "adc", "dac", "data converter", "sigma delta", "successive approximation", "sar adc",
17:    "pipeline adc", "integrator", "differentiator", "rectifier", "precision rectifier", "peak detector"
18:  ];
19:
20:   const powerTerms = [
21:     "ac to dc", "charger", "battery charger", "power supply", "bench supply", "linear regulator", "ldo",
22:     "switching regulator", "buck converter", "boost converter", "buck boost", "flyback", "forward
converter",
23:     "sepic", "inverter", "rectifier bridge", "diode bridge", "full wave rectifier", "half wave
rectifier",
24:     "ripple", "load regulation", "line regulation", "transient response", "power stage", "mosfet driver",
25:     "gate driver", "current limit", "overvoltage", "undervoltage", "reverse polarity", "thermal
shutdown",
26:     "power path", "battery management", "bms", "cc cv", "constant current", "constant voltage", "trickle
charge",
27:     "precharge", "protection circuit", "fuse", "polyfuse", "tvS diode", "esd diode", "snubber", "clamp",
28:     "soft start", "inrush current", "efficiency", "loss budget", "heat sink", "thermal resistance",
"power dissipation",
29:     "load switch", "ideal diode", "current sense", "shunt resistor", "hall sensor", "isolation",
"transformer",
30:     "optoisolator", "dc dc", "ac dc", "emi filter", "common mode choke", "ferrite bead", "ground loop"
31:   ];
32:
33:   const pcbTerms = [
34:     "pcb", "printed circuit board", "schematic", "layout", "footprint", "symbol", "netlist", "bom",
35:     "gerber", "drill file", "pick and place", "assembly", "bring up", "board bring up", "prototype",
36:     "revision", "rev a", "dfm", "dft", "design rule", "clearance", "creepage", "trace width", "via",
37:     "microvia", "blind via", "buried via", "ground plane", "power plane", "stackup", "controlled
impedance",
38:     "differential pair routing", "return path", "star ground", "analog ground", "digital ground", "guard
ring",
39:     "decoupling capacitor", "bypass capacitor", "bulk capacitor", "test point", "connector", "header",
40:     "silkscreen", "solder mask", "copper pour", "thermal relief", "stitching via", "keepout", "fiducial",
41:     "kicad", "altium", "eagle", "orcad", "cadence allegro", "pcbway", "jlcpcb", "osh park"
42:   ];
43:
44:   const testTerms = [
45:     "oscilloscope", "scope capture", "logic analyzer", "spectrum analyzer", "network analyzer", "function
generator",
46:     "signal generator", "multimeter", "dmm", "electronic load", "power analyzer", "curve tracer",
"probe",
47:     "differential probe", "current probe", "bench test", "validation", "verification",
"characterization",
48:     "measurement", "calibration", "test plan", "test report", "waveform", "transient", "dc sweep", "ac
sweep",
49:     "monte carlo", "corner analysis", "temperature sweep", "load test", "line test", "frequency sweep",

```

```

50:     "gain measurement", "phase measurement", "jitter", "rise time", "fall time", "duty cycle", "pwm",
51:     "latency", "throughput", "noise floor", "snr", "thd", "thd+n", "eye diagram", "ber", "debug",
52:     "troubleshooting", "root cause", "failure analysis", "regression test", "unit test", "integration
test"
53: ];
54:
55: const digitalTerms = [
56:     "digital design", "logic design", "combinational logic", "sequential logic", "finite state machine",
"fsm",
57:     "flip flop", "latch", "counter", "register", "shift register", "clock divider", "clock domain
crossing",
58:     "cdc", "metastability", "synchronizer", "debounce", "timing closure", "setup time", "hold time",
59:     "static timing analysis", "sta", "fpga", "asic", "rtl", "verilog", "systemverilog", "vhdl",
60:     "testbench", "uvm", "simulation", "synthesis", "place and route", "bitstream", "vivado", "quartus",
61:     "modelsim", "questa", "iverilog", "yosys", "formal verification", "assertion", "coverage", "axi",
62:     "apb", "ahb", "spi controller", "i2c controller", "uart", "fifo", "ram", "rom", "bram",
63:     "pll clock", "mmcm", "serdes", "lvds", "gpio", "interrupt", "dma", "bus protocol"
64: ];
65:
66: const embeddedTerms = [
67:     "embedded", "firmware", "microcontroller", "mcu", "stm32", "stm32cubeide", "arm cortex", "cortex m",
68:     "bare metal", "rtos", "freertos", "scheduler", "interrupt service routine", "isr", "timer",
69:     "pwm output", "adc input", "dac output", "gpio", "spi", "i2c", "uart", "usb", "can bus",
70:     "ethernet", "bootloader", "flash memory", "eeprom", "dma", "low power", "sleep mode", "watchdog",
71:     "sensor interface", "motor control", "control loop", "pid", "encoder", "hall sensor", "imu",
72:     "temperature sensor", "pressure sensor", "display", "lcd", "oled", "ble", "wifi", "esp32",

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: package.json

Item	Explanation
File	package.json
Category	Configuration or structured data
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	2,412 bytes
Extension	.json
MIME type	application/json
Text lines	85

Why this file exists

- Node/Electron package manifest with scripts, dependencies, version, installer settings, and extra resources.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Security or auth at line 5	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "author": "Maurice Otieno",
Installer at line 10	Windows setup, update, shortcut, or installation behavior. Evidence: "pack": "electron-builder --win portable --publish never",
Installer at line 11	Windows setup, update, shortcut, or installation behavior. Evidence: "installer": "electron-builder --win nsis --publish never",
Installer at line 12	Windows setup, update, shortcut, or installation behavior. Evidence: "dist": "electron-builder --win --publish never"
Installer at line 16	Windows setup, update, shortcut, or installation behavior. Evidence: "electron-builder": "^26.0.12"
Cloudflare or AI at line 37	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "cloudflare/**",
Installer at line 50	Windows setup, update, shortcut, or installation behavior. Evidence: "installer-notes.txt",
Security or auth at line 54	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers",
Installer at line 64	Windows setup, update, shortcut, or installation behavior. Evidence: "nsis"
Installer at line 70	Windows setup, update, shortcut, or installation behavior. Evidence: "nsis": {
Installer at line 76	Windows setup, update, shortcut, or installation behavior. Evidence: "include": "build/installer.nsh",
Installer at line 77	Windows setup, update, shortcut, or installation behavior. Evidence: "createDesktopShortcut": "always",
Installer at line 78	Windows setup, update, shortcut, or installation behavior. Evidence: "createStartMenuShortcut": true,
Installer at line 79	Windows setup, update, shortcut, or installation behavior. Evidence: "deleteAppDataOnUninstall": true,
Installer at line 81	Windows setup, update, shortcut, or installation behavior. Evidence: "license": "installer-notes.txt",
Installer at line 82	Windows setup, update, shortcut, or installation behavior. Evidence: "shortcutName": "OMB Portfolio Builder"

JSON/YAML-style keys detected

Item	Explanation
name	Line 2: "name": "omb-portfolio-builder",
version	Line 3: "version": "0.2.34",
private	Line 4: "private": false,
author	Line 5: "author": "Maurice Otieno",
description	Line 6: "description": "Standalone Windows desktop application for building and publishing portfolio websites.",
main	Line 7: "main": "main.cjs",
scripts	Line 8: "scripts": {
dev	Line 9: "dev": "electron .",
pack	Line 10: "pack": "electron-builder --win portable --publish never",
installer	Line 11: "installer": "electron-builder --win nsis --publish never",
dist	Line 12: "dist": "electron-builder --win --publish never"
devDependencies	Line 14: "devDependencies": {
electron	Line 15: "electron": "^39.2.7",
electron-builder	Line 16: "electron-builder": "^26.0.12"
build	Line 18: "build": {
appId	Line 19: "appId": "com.mauriceotieno.portfoliobuilder",
productName	Line 20: "productName": "OMB Portfolio Builder",
copyright	Line 21: "copyright": "Copyright (c) Maurice Otieno",
asar	Line 22: "asar": true,
directories	Line 23: "directories": {
output	Line 24: "output": "dist"
files	Line 26: "files": [
extraResources	Line 30: "extraResources": [
from	Line 32: "from": ". ",
to	Line 33: "to": "site",
filter	Line 34: "filter": [
win	Line 60: "win": {
icon	Line 61: "icon": "assets/omb-app-icon.ico",
target	Line 62: "target": [
portable	Line 67: "portable": {
artifactName	Line 68: "artifactName": "OMB-Portfolio-Builder-Portable-\${version}-\${arch}.\${ext}"
nsis	Line 70: "nsis": {
artifactName	Line 71: "artifactName": "OMB-Portfolio-Builder-Setup-\${version}-\${arch}.\${ext}",
oneClick	Line 72: "oneClick": false,
perMachine	Line 73: "perMachine": false,
allowElevation	Line 74: "allowElevation": false,
allowToChangeInstallationDirectory	Line 75: "allowToChangeInstallationDirectory": true,
include	Line 76: "include": "build/installer.nsh",
createDesktopShortcut	Line 77: "createDesktopShortcut": "always",
createStartMenuShortcut	Line 78: "createStartMenuShortcut": true,

Item	Explanation
deleteAppDataOnUninstall	Line 79: "deleteAppDataOnUninstall": true,
runAfterFinish	Line 80: "runAfterFinish": true,
license	Line 81: "license": "installer-notes.txt",
shortcutName	Line 82: "shortcutName": "OMB Portfolio Builder"

Representative opening snippet

```

1: {
2:   "name": "omb-portfolio-builder",
3:   "version": "0.2.34",
4:   "private": false,
5:   "author": "Maurice Otieno",
6:   "description": "Standalone Windows desktop application for building and publishing portfolio
websites.",
7:   "main": "main.cjs",
8:   "scripts": {
9:     "dev": "electron .",
10:    "pack": "electron-builder --win portable --publish never",
11:    "installer": "electron-builder --win nsis --publish never",
12:    "dist": "electron-builder --win --publish never"
13:  },
14:  "devDependencies": {
15:    "electron": "^39.2.7",
16:    "electron-builder": "^26.0.12"
17:  },
18:  "build": {
19:    "appId": "com.mauriceotieno.portfoliobuilder",
20:    "productName": "OMB Portfolio Builder",
21:    "copyright": "Copyright (c) Maurice Otieno",
22:    "asar": true,
23:    "directories": {
24:      "output": "dist"
25:    },
26:    "files": [
27:      "main.cjs",
28:      "package.json"
29:    ],
30:    "extraResources": [
31:      {
32:        "from": ".",
33:        "to": "site",
34:        "filter": [
35:          "assets/**",
36:          "Backgrounds/**",
37:          "cloudflare/**",
38:          "docs/**",
39:          "server.mjs",
40:          "index.html",
41:          "styles.css",
42:          "script.js",
43:          "electronics-search.js",
44:          "builder-rich-future-sections.js",
45:          "projects.json",
46:          "project-templates.json",
47:          "template-preview.html",
48:          "template-preview.css",
49:          "template-preview.js",
50:          "installer-notes.txt",
51:          "README.md",
52:          "portfolio-README.md",
53:          ".nojekyll",
54:          "_headers",
55:          ".well-known/**",
56:          "robots.txt"
57:        ]
58:      }
59:    ],
60:    "win": {
61:      "icon": "assets/omb-app-icon.ico",
62:      "target": [
63:        "portable",
64:        "nsis"
65:      ]
66:    },
67:    "portable": {
68:      "artifactName": "OMB-Portfolio-Builder-Portable-${version}-${arch}.${ext}"
69:    },
70:    "nsis": {
71:      "artifactName": "OMB-Portfolio-Builder-Setup-${version}-${arch}.${ext}",
72:      "oneClick": false,

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: project-templates.json

Item	Explanation
File	project-templates.json
Category	Configuration or structured data
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	6,428 bytes
Extension	.json
MIME type	application/json
Text lines	184

Why this file exists

- Appearance template definitions used by projects to control visual feel rather than content.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

JSON/YAML-style keys detected

Item	Explanation
templates	Line 2: "templates": [
id	Line 4: "id": "appearance-light-blue-red-click",
label	Line 5: "label": "Light blue, navy hover, red click",
description	Line 6: "description": "Light blue engineering background with calm navy hover states and a sharp red click flash.",
visual	Line 7: "visual": {
style	Line 8: "style": "schematic",
palette	Line 9: "palette": ["#e9f8ff", "#1f6ed4", "#ef4444", "#ffffff"],
background	Line 10: "background": "#e9f8ff",
panel	Line 11: "panel": "#ffffff",
accent	Line 12: "accent": "#1f6ed4",
hover	Line 13: "hover": "#1f6ed4",
click	Line 14: "click": "#ef4444",
clickText	Line 15: "clickText": "#ffffff",
line	Line 16: "line": "#b8def3",
text	Line 17: "text": "#172636",
interaction	Line 18: "interaction": "Navy-blue hover with a red active click flash."
id	Line 22: "id": "appearance-white-blue-click",
label	Line 23: "label": "Clean white, blue hover, white click",
description	Line 24: "description": "Minimal white project panels with professional blue hover behavior and quiet white click feedback.",

Item	Explanation
visual	Line 25: "visual": {
style	Line 26: "style": "device",
palette	Line 27: "palette": ["#ffffff", "#2563eb", "#f8bfff"],
background	Line 28: "background": "#ffffff",
panel	Line 29: "panel": "#ffffff",
accent	Line 30: "accent": "#2563eb",
hover	Line 31: "hover": "#2563eb",
click	Line 32: "click": "#ffffff",
clickText	Line 33: "clickText": "#172636",
line	Line 34: "line": "#d8e2ef",
text	Line 35: "text": "#172636",
interaction	Line 36: "interaction": "Blue hover with a clean white click state."
id	Line 40: "id": "appearance-deep-navy-cyan-click",
label	Line 41: "label": "Deep navy, cyan hover, white click",
description	Line 42: "description": "Dark navy technical background with cyan hover states and high-contrast white click feedback.",
visual	Line 43: "visual": {
style	Line 44: "style": "chip",
palette	Line 45: "palette": ["#10263c", "#0f6f96", "#ffffff"],
background	Line 46: "background": "#10263c",
panel	Line 47: "panel": "#f8bfff",
accent	Line 48: "accent": "#0f6f96",
hover	Line 49: "hover": "#0f6f96",
click	Line 50: "click": "#ffffff",
clickText	Line 51: "clickText": "#142536",
line	Line 52: "line": "#8ccfe8",
text	Line 53: "text": "#f8bfff",
interaction	Line 54: "interaction": "Cyan hover with a white active click state."
id	Line 58: "id": "appearance-warm-red-pale-click",
label	Line 59: "label": "Warm red, wine hover, pale click",
description	Line 60: "description": "Warm signal-lab styling with wine-red hover behavior and a soft pale-red click response.",
visual	Line 61: "visual": {
style	Line 62: "style": "waveform",
palette	Line 63: "palette": ["#fff7f5", "#c2413b", "#ff4f2"],
background	Line 64: "background": "#fff7f5",
panel	Line 65: "panel": "#ffffff",
accent	Line 66: "accent": "#b9332e",
hover	Line 67: "hover": "#c2413b",
click	Line 68: "click": "#ff4f2",
clickText	Line 69: "clickText": "#2f2020",
line	Line 70: "line": "#efb0aa",
text	Line 71: "text": "#2f2020",

Item	Explanation
interaction	Line 72: "interaction": "Wine-red hover with a pale red click state."
id	Line 76: "id": "appearance-emerald-mint-click",
label	Line 77: "label": "Emerald bench, green hover, mint click",
description	Line 78: "description": "Instrumentation-style panels with emerald hover states and a light mint click response.",
visual	Line 79: "visual": {
style	Line 80: "style": "instrument",
palette	Line 81: "palette": ["#f0fdf6", "#16865a", "#efff6"],
background	Line 82: "background": "#f0fdf6",
panel	Line 83: "panel": "#ffffff",
accent	Line 84: "accent": "#16865a",
hover	Line 85: "hover": "#16865a",
click	Line 86: "click": "#efff6",
clickText	Line 87: "clickText": "#182f26",
line	Line 88: "line": "#9adfbf",
text	Line 89: "text": "#182f26",
interaction	Line 90: "interaction": "Emerald hover with a soft mint click state."
id	Line 94: "id": "appearance-amber-cream-click",
label	Line 95: "label": "Amber rail, gold hover, cream click",
description	Line 96: "description": "Power-rail inspired appearance with amber hover behavior and warm cream click feedback.",
visual	Line 97: "visual": {

Representative opening snippet

```

1: {
2:   "templates": [
3:     {
4:       "id": "appearance-light-blue-red-click",
5:       "label": "Light blue, navy hover, red click",
6:       "description": "Light blue engineering background with calm navy hover states and a sharp red click
flash.",
7:       "visual": {
8:         "style": "schematic",
9:         "palette": ["#e9f8ff", "#1f6ed4", "#ef4444", "#ffffff"],
10:        "background": "#e9f8ff",
11:        "panel": "#ffffff",
12:        "accent": "#1f6ed4",
13:        "hover": "#1f6ed4",
14:        "click": "#ef4444",
15:        "clickText": "#ffffff",
16:        "line": "#b8def3",
17:        "text": "#172636",
18:        "interaction": "Navy-blue hover with a red active click flash."
19:      }
20:    },
21:    {
22:      "id": "appearance-white-blue-click",
23:      "label": "Clean white, blue hover, white click",
24:      "description": "Minimal white project panels with professional blue hover behavior and quiet white
click feedback.",
25:      "visual": {
26:        "style": "device",
27:        "palette": ["#ffffff", "#2563eb", "#f8fbff"],
28:        "background": "#ffffff",
29:        "panel": "#ffffff",
30:        "accent": "#2563eb",
31:        "hover": "#2563eb",
32:        "click": "#ffffff",
33:        "clickText": "#172636",
34:        "line": "#d8e2ef",
35:        "text": "#172636",
36:        "interaction": "Blue hover with a clean white click state."
37:      }
38:    },
39:  ]

```

```

40:     "id": "appearance-deep-navy-cyan-click",
41:     "label": "Deep navy, cyan hover, white click",
42:     "description": "Dark navy technical background with cyan hover states and high-contrast white click
feedback.",
43:     "visual": {
44:         "style": "chip",
45:         "palette": ["#10263c", "#0f6f96", "#ffffff"],
46:         "background": "#10263c",
47:         "panel": "#f8fbff",
48:         "accent": "#0f6f96",
49:         "hover": "#0f6f96",
50:         "click": "#ffffff",
51:         "clickText": "#142536",
52:         "line": "#8ccfe8",
53:         "text": "#f8fbff",
54:         "interaction": "Cyan hover with a white active click state."
55:     }
56: },
57: {
58:     "id": "appearance-warm-red-pale-click",
59:     "label": "Warm red, wine hover, pale click",
60:     "description": "Warm signal-lab styling with wine-red hover behavior and a soft pale-red click
response.",
61:     "visual": {
62:         "style": "waveform",
63:         "palette": ["#fff7f5", "#c2413b", "#fff4f2"],
64:         "background": "#fff7f5",
65:         "panel": "#ffffff",
66:         "accent": "#b9332e",
67:         "hover": "#c2413b",
68:         "click": "#fff4f2",
69:         "clickText": "#2f2020",
70:         "line": "#efb0aa",
71:         "text": "#2f2020",
72:         "interaction": "Wine-red hover with a pale red click state."

```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference: projects.json

Item	Explanation
File	projects.json
Category	Configuration or structured data
Folder role	Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.
Size	1,490 bytes
Extension	.json
MIME type	application/json
Text lines	46

Why this file exists

- Public project catalog consumed by the website after parsing builder data.

How it is used

Repository root. Usually an entry file, app config, public website file, package manifest, or top-level documentation.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

JSON/YAML-style keys detected

Item	Explanation
categories	Line 2: "categories": [
id	Line 4: "id": "analog-mixed-signal",
label	Line 5: "label": "Analog and Mixed Signal",
description	Line 6: "description": "Circuit-level work including amplifiers, filters, converters, measurement front ends, power stages, and mixed-signal interfaces.",
accent	Line 7: "accent": "#117c7a"
id	Line 10: "id": "digital",
label	Line 11: "label": "Digital",
description	Line 12: "description": "Digital design projects involving logic design, FPGA workflows, verification, timing, and ASIC-oriented architecture.",
accent	Line 13: "accent": "#52606d"
id	Line 16: "id": "embedded",
label	Line 17: "label": "Embedded",
description	Line 18: "description": "MCU-based systems with firmware, board bring-up, interfaces, sensing, control, and hardware/software integration.",
accent	Line 19: "accent": "#d18b21"
funFacts	Line 22: "funFacts": [],
funFactsRich	Line 23: "funFactsRich": null,
profile	Line 24: "profile": {
brandImage	Line 25: "brandImage": "",
brandText	Line 26: "brandText": "Portfolio",

Item	Explanation
contactIntro	Line 27: "contactIntro": "",
displayName	Line 28: "displayName": "",
email	Line 29: "email": "",
githubUrl	Line 30: "githubUrl": "",
heroImage	Line 31: "heroImage": "",
linkedinUrl	Line 32: "linkedinUrl": "",
phone	Line 33: "phone": "",
portfolioLabel	Line 34: "portfolioLabel": "Portfolio",
profileImage	Line 35: "profileImage": "",
resumeUrl	Line 36: "resumeUrl": "",
websiteUrl	Line 37: "websiteUrl": ""
siteContent	Line 39: "siteContent": {
heroCopy	Line 40: "heroCopy": "Add your projects, documents, diagrams, source code, images, profile details, and links.\nSave drafts locally, preview the site, then publish when your target repository is ready.",
heroEyebrow	Line 41: "heroEyebrow": "Engineering portfolio",
heroTitle	Line 42: "heroTitle": "Build a portfolio that presents your work clearly."
projects	Line 44: "projects": [],
siteSections	Line 45: "siteSections": []

Representative opening snippet

```

1: {
2:   "categories": [
3:     {
4:       "id": "analog-mixed-signal",
5:       "label": "Analog and Mixed Signal",
6:       "description": "Circuit-level work including amplifiers, filters, converters, measurement front
ends, power stages, and mixed-signal interfaces.",
7:       "accent": "#117c7a"
8:     },
9:     {
10:      "id": "digital",
11:      "label": "Digital",
12:      "description": "Digital design projects involving logic design, FPGA workflows, verification,
timing, and ASIC-oriented architecture.",
13:      "accent": "#52606d"
14:    },
15:    {
16:      "id": "embedded",
17:      "label": "Embedded",
18:      "description": "MCU-based systems with firmware, board bring-up, interfaces, sensing, control, and
hardware/software integration.",
19:      "accent": "#d18b21"
20:    }
21:  ],
22:  "funFacts": [],
23:  "funFactsRich": null,
24:  "profile": {
25:    "brandImage": "",
26:    "brandText": "Portfolio",
27:    "contactIntro": "",
28:    "displayName": "",
29:    "email": "",
30:    "githubUrl": "",
31:    "heroImage": "",
32:    "linkedinUrl": "",
33:    "phone": "",
34:    "portfolioLabel": "Portfolio",
35:    "profileImage": "",
36:    "resumeUrl": "",
37:    "websiteUrl": ""
38:  },
39:  "siteContent": {
40:    "heroCopy": "Add your projects, documents, diagrams, source code, images, profile details, and
links.\nSave drafts locally, preview the site, then publish when your target repository is ready.",
41:    "heroEyebrow": "Engineering portfolio",
42:    "heroTitle": "Build a portfolio that presents your work clearly."
43:  },

```



```
44:   "projects": [],  
45:   "siteSections": []  
46: }
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference:

[.github/workflows/build-windows-builder.yml](#)

Item	Explanation
File	.github/workflows/build-windows-builder.yml
Category	GitHub workflow
Folder role	GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs.
Size	3,867 bytes
Extension	.yml
MIME type	unknown
Text lines	107

Why this file exists

- GitHub Actions workflow that builds the Windows installer and portable application, checks the stable download link, enforces release version bumps, uploads artifacts, and publishes GitHub Releases.
- Workflow changes should be tested on a feature branch because mistakes can block merges or releases.

How it is used

GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Security or auth at line 11	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: permissions:
Installer at line 16	Windows setup, update, shortcut, or installation behavior. Evidence: name: Build Windows installer and portable app
Compiler or simulator at line 23	Part of the Compile Code workspace or language/tool detection. Evidence: - name: Set up Node.js
Compiler or simulator at line 24	Part of the Compile Code workspace or language/tool detection. Evidence: uses: actions/setup-node@v4
Compiler or simulator at line 26	Part of the Compile Code workspace or language/tool detection. Evidence: node-version: "24"
Installer at line 44	Windows setup, update, shortcut, or installation behavior. Evidence: throw "Windows setup installer was not found."
File read at line 57	Reads local builder state, project files, website assets, or configuration. Evidence: \$package = Get-Content package.json -Raw ConvertFrom-Json
Network request at line 73	Frontend-backend communication or public web/API calls. Evidence: \$latestInstallerUrl = "https://github.com/otienoma/urice/omb-portfolio-builder/releases/latest/download/OMB-Portfolio-Builder-Setup-latest-x64.exe"
File read at line 74	Reads local builder state, project files, website assets, or configuration. Evidence: \$index = Get-Content index.html -Raw
Installer at line 75	Windows setup, update, shortcut, or installation behavior. Evidence: if (\$index -notmatch [regex]::Escape(\$latestInstallerUrl)) {
Installer at line 76	Windows setup, update, shortcut, or installation behavior. Evidence: throw "The portfolio download link must use the stable latest installer URL: \$latestInstallerUrl"
Git command at line 83	Repository state, publishing, branch checks, or release automation. Evidence: git fetch --tags --force
Git command at line 85	Repository state, publishing, branch checks, or release automation. Evidence: \$tagExists = [bool](git tag --list \$tag)

Representative opening snippet

```
1: name: Build Windows Portfolio Builder
2:
3: on:
4:   workflow_dispatch:
5:     push:
6:       branches:
7:         - main
8:       tags:
9:         - "builder-v*"
10:
11: permissions:
12:   contents: write
13:
14: jobs:
15:   build-windows:
16:     name: Build Windows installer and portable app
17:     runs-on: windows-latest
18:
19:     steps:
20:       - name: Check out repository
21:         uses: actions/checkout@v4
22:
23:       - name: Set up Node.js
24:         uses: actions/setup-node@v4
25:         with:
26:           node-version: "24"
27:
28:       - name: Set up pnpm
29:         uses: pnpm/action-setup@v4
30:         with:
31:           version: "11.5.3"
32:
33:       - name: Install desktop builder dependencies
34:         run: pnpm install --frozen-lockfile
35:
36:       - name: Build Windows artifacts
37:         run: pnpm run dist
38:
39:       - name: Create stable latest download names
40:         shell: pwsh
41:         run: |
42:           $setup = Get-ChildItem dist -Filter "OMB-Portfolio-Builder-Setup-*-x64.exe" | Sort-Object
LastWriteTime -Descending | Select-Object -First 1
43:           if (-not $setup) {
44:             throw "Windows setup installer was not found."
45:           }
46:           Copy-Item $setup.FullName "dist/OMB-Portfolio-Builder-Setup-latest-x64.exe" -Force
47:
48:           $portable = Get-ChildItem dist -Filter "OMB-Portfolio-Builder-Portable-*-x64.exe" | Sort-Object
LastWriteTime -Descending | Select-Object -First 1
49:           if ($portable) {
50:             Copy-Item $portable.FullName "dist/OMB-Portfolio-Builder-Portable-latest-x64.exe" -Force
51:           }
52:
53:       - name: Resolve release version
54:         id: release_meta
55:         shell: pwsh
56:         run: |
57:           $package = Get-Content package.json -Raw | ConvertFrom-Json
58:           $version = [string]$package.version
59:           if (-not $version) {
60:             throw "package.json version is required before publishing a release."
61:           }
62:           if ($env:GITHUB_REF -like "refs/tags/*") {
63:             $tag = $env:GITHUB_REF_NAME
64:           } else {
65:             $tag = "builder-v$version"
66:           }
67:           "version=$version" >> $env:GITHUB_OUTPUT
68:           "release_tag=$tag" >> $env:GITHUB_OUTPUT
69:
70:       - name: Verify portfolio download link follows latest release
71:         shell: pwsh
72:         run: |
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.

Important Code Reference:

.github/workflows/main-branch-gate.yml

Item	Explanation
File	.github/workflows/main-branch-gate.yml
Category	GitHub workflow
Folder role	GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs.
Size	1,259 bytes
Extension	.yml
MIME type	unknown
Text lines	40

Why this file exists

- GitHub Actions guard that rejects direct pushes to main and rejects pull requests into main unless the source branch is development.
- Workflow changes should be tested on a feature branch because mistakes can block merges or releases.

How it is used

GitHub automation. GitHub Actions reads this during CI/release/branch-gate runs.

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Security or auth at line 11	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: permissions:

Representative opening snippet

```
1: name: Main Branch Gate
2:
3: on:
4:   pull_request:
5:     branches:
6:       - main
7:   push:
8:     branches:
9:       - main
10:
11: permissions:
12:   contents: read
13:
14: jobs:
15:   require-development-source:
16:     name: Require development as the main source branch
17:     runs-on: ubuntu-latest
18:
19:     steps:
20:       - name: Verify main only receives development
21:         shell: bash
22:         env:
23:           HEAD_COMMIT_MESSAGE: ${ github.event.head_commit.message }
24:         run: |
25:           if [ "${ github.event_name }" = "push" ]; then
26:             if printf '%s' "$HEAD_COMMIT_MESSAGE" | grep -Eq '^Merge pull request #[0-9]+ from
[^/]+/development$'; then
27:               echo "main was updated by a development release PR."
28:               exit 0
29:             fi
30:
31:             echo "::error::Direct pushes to main are not allowed. Open a pull request from development"
```

```
into main."
32:         exit 1
33:     fi
34:
35:     if [ "${{ github.head_ref }}" != "development" ]; then
36:         echo "::error::Pull requests into main must come from development. Merge this branch into
development first, test development, then open the development -> main release pull request."
37:         exit 1
38:     fi
39:
40:     echo "main release source is development."
```

Safe change checklist

- Confirm which app, website, installer, workflow, or document consumes this file.
- Change it on a feature branch from development.
- Regenerate docs if the file purpose, API surface, or behavior changes.
- Run the smallest relevant smoke test before merging into development.